BigBlueBam

The Complete Manual

Sixteen apps that behave like one. A field guide.

Open source. Self-hosted. Built for humans and AI agents.

Table of Contents

BigBlueBam....17
- Overview....18
  - Key concepts....18
  - Where to find it....19
- The sixteen apps....20
  - Plan and build the work....20
  - Communicate and coordinate....20
  - Grow the business....20
  - Run operations....20
- How the apps work together....22
- Pervasive presence: the apps are live, shared spaces....23
- Working with AI agents....25
- Open source and self-hosting....26
- User Stories....27
  - Story: A lead becomes a project becomes an invoice....27
  - Story: A support ticket becomes a fix....27
  - Story: An idea becomes shipped work....27
  - Story: An agent works alongside the team....27
  - Story: A hallway moment, on demand....27
  - Story: One place to stand....27

Bam....29
- Overview....30
  - Key concepts....31
  - Where to find it....32
- Feature reference....34
  - Project dashboard and project list....34
  - Kanban board....34
  - Task detail drawer....34
  - Creating and editing tasks....35
  - Subtasks and parent tasks....36
  - Sprints....36
  - Carry-forward....37
  - Epics....37
  - Phases (columns)....37
  - Task states....38
  - Labels....38
  - Custom fields....38
  - Saved views....38
  - Alternate views (List, Timeline, Calendar, Workload)....38
  - Task templates....39
  - Import....39
  - Export....39
  - iCal feed....40
  - Comments, reactions, and revisions....40
  - Time tracking....40
  - Attachments....40
  - Task links....40
  - Reports and project dashboard....40
  - Audit log....40
  - People and members....41

My Work....41
Settings....42
Email delivery (SMTP)....43
Command palette and keyboard shortcuts....45
Notifications....45
Cross-app integrations....45
Working with AI agents....46
Working together (live presence)....48
User Stories....49
Story: Spin up a project from a template....49
Story: Create and work a task end to end....49
Story: Track your personal load....49
Story: Plan and run a sprint....50
Story: Carry forward unfinished work....50
Story: Group work under an epic....50
Story: Customize the board....51
Story: Slice the board with views....51
Story: Import an existing backlog....51
Story: Report on delivery....52
Story: Subscribe to a project calendar....52
Story: Deep-link a task across apps....52
Story: Sync tasks from an external system with an agent....53
Related....54
Banter....57
Overview....58
Key concepts....58
Where to find it....60
Feature reference....61
Send a message....61
Reply in a thread....62
React to a message....63
Pin a message....63
Bookmark a message....63
Edit or delete your message....64
Create a channel....64
Browse and join channels....65
Start a direct message or group DM....65
Configure a channel....66
Rename, leave, or delete from the sidebar....67
Mark a channel read....67
Search messages....67
Past calls and audio....68
Keyboard shortcuts....68
Preferences....69
Admin: organization policy....69
Admin: organize the sidebar with channel groups....70
Admin: define @mention groups....70
Admin: import from Slack....70
Working with AI agents....70
Working together (live presence)....73
User Stories....74

Story: Get into Banter and read #general....74
Story: Send your first message....74
Story: Attach a file to a message....74
Story: Create a channel for a new topic....75
Story: Find and join an existing channel....75
Story: Start a direct message....76
Story: Hold a side conversation in a thread....76
Story: React, pin, and bookmark a message....76
Story: Edit or delete something you posted....77
Story: Find an old message with search....77
Story: Configure and tidy a channel....77
Story: Schedule a message for later (automation and agents)....78
Story: Have an agent listen for a pattern and respond....78
Story: Share a task, sprint, or ticket into a channel....79
Story: Set up Banter for the whole org....79
Story: Migrate a Slack workspace....79
Story: Review a past call....80
Related....81
Beacon....85
Overview....86
Key concepts....86
Where to find it....87
Feature reference....89
Knowledge Home....89
Browse the article list....89
Read an article (detail page)....90
Create and edit an article....91
Lifecycle actions....92
Search....93
Saved queries....94
Knowledge Graph....94
Tags....95
Links between articles....96
Comments....96
Attachments....96
Fridge Cleanout (governance dashboard)....97
Expiry Policy Settings....98
Working with AI agents....99
Working together (live presence)....101
User Stories....102
Story: Write and publish your first article....102
Story: Find an answer by meaning....102
Story: Save and reuse a search....103
Story: Link related articles....103
Story: Explore how knowledge connects....104
Story: Challenge an article that looks wrong....104
Story: Keep knowledge fresh (governance sweep)....104
Story: Set the freshness policy for a team....105
Story: Discuss and attach evidence to an article....105
Story: Ingest knowledge as an agent (idempotent import)....106
Related....107

Bearing....111
Overview....112
Key concepts....112
Where to find it....113
Feature reference....115
Goals Dashboard....115
My Goals....116
At Risk....116
Periods....117
Goal Detail....119
Create a Goal....120
Add and edit Key Results....120
Record progress on a Key Result....121
Post a status update....121
Watch a goal....121
Reports and CSV export....122
Working with AI agents....122
Working together (live presence)....125
User Stories....126
Story: Set up your first quarter....126
Story: Create your first objective....126
Story: Break a goal into key results....127
Story: Record weekly progress....127
Story: Post a status update for the team....127
Story: Triage the goals that are behind....128
Story: Review your own goals across periods....128
Story: Watch a goal you care about....128
Story: Close out the quarter....129
Story: Provision a quarter with an agent....129
Story: Wire key results to real delivery (agent)....130
Story: Run a weekly status sweep (agent)....130
Related....131
Bench....135
Overview....136
Key concepts....137
Where to find it....138
Feature reference....139
Dashboards list....139
Dashboard view....139
Dashboard edit....141
Widget Templates gallery....141
Widget Wizard....142
Widget Edit (live preview)....143
Ad-Hoc Explorer....143
Scheduled Reports....144
Saved Queries....146
Bench Settings....146
Registered data sources....148
Sources that do not return data....148
Working with AI agents....149
Working together (live presence)....151

User Stories....152
Story: Open and read a dashboard....152
Story: Build a dashboard from scratch....152
Story: Add a custom widget with the wizard....153
Story: Drop in a prebuilt widget from Templates....153
Story: Reorder a dashboard's widgets....153
Story: Tune a widget with the live preview....154
Story: Explore a source ad hoc....154
Story: Save a query and run it later....154
Story: Schedule a recurring report....155
Story: Duplicate a dashboard as a starting template....155
Story: Have an agent summarize a dashboard and flag anomalies....155
Related....157
Bill....161
Overview....162
Key concepts....162
Where to find it....164
Feature reference....166
Dashboard....166
Invoice list....166
Create an invoice....168
Invoice from time entries....168
Invoice detail....169
Edit an invoice....171
Clients....171
Expenses....171
Rates....173
Reports....173
Settings....174
Recurring billing....175
PDF generation....176
Public invoice view....177
Working with AI agents....177
Working together (live presence)....180
User Stories....181
Story: Set up your billing identity....181
Story: Add a client and send your first invoice....181
Story: Record a payment to close out....182
Story: Capture, approve, and reimburse an expense....182
Story: Send an invoice for approval before billing....183
Story: Turn a won Bond deal into an invoice....183
Story: Bill tracked time from a project....184
Story: Put a standing fee on a recurring schedule....184
Story: Close out a billing period....185
Story: Chase overdue invoices automatically....185
Related....186
Blank....189
Overview....190
Key concepts....190
Where to find it....191
Feature reference....193

Forms list....193
Form Builder....194
Field Settings....195
Form Settings dialog....196
Publish dialog....198
Preview....198
Responses....199
Analytics....201
Per-form Settings page....201
Multi-page forms....202
Blank Settings (app-level)....202
Public form....203
Working with AI agents....203
Working together (live presence)....206
User Stories....207
Story: Build and publish your first form....207
Story: Collect and review responses....207
Story: Analyze results....208
Story: Duplicate a form as a template....208
Story: Restrict a form to your organization or a project....208
Story: Gate a public form by sign-in and email domain, and get notified....209
Story: Collect file uploads on a form....209
Story: Build a multi-page survey....210
Story: Close a form to stop accepting responses....210
Story: Have an agent draft, create, and publish a form....211
Related....212
Blast....215
Overview....216
Key concepts....216
Where to find it....218
Feature reference....219
Campaigns list....219
Create a campaign....220
Campaign detail and Send Now....221
Visual builder....222
Templates....222
Segments....224
Analytics....226
Sender Domains....227
SMTP (platform relay)....227
Tracking and unsubscribe (automatic, no UI)....228
Provider webhooks (automatic, no UI)....228
Why a send was rejected....228
Working with AI agents....229
Working together (live presence)....231
User Stories....232
Story: Set up outbound email before your first send....232
Story: Send a one-off campaign from scratch....232
Story: Reuse a saved template for a campaign....233
Story: Author and version a reusable template....233
Story: Build a targeted segment from CRM data....234

Story: Review campaign performance....234
Story: Verify a sending domain for deliverability....235
Story: A recipient unsubscribes or a message bounces (automatic)....235
Story: Let an agent draft a campaign for your approval....235
Story: Nurture a Bond contact group, then react in Bolt....236
Related....237
Blueprint....241
Overview....242
Key concepts....243
Where to find it....244
Feature reference....245
Browse and filter diagrams....245
Create a diagram....246
The editor canvas....247
Add nodes....247
Edit a node....248
Move and pin nodes....248
Duplicate a node....248
Delete a node....249
Connect nodes with edges....249
Edit an edge....249
Delete an edge....249
Snap to grid....249
Auto-layout....249
Export a diagram....250
Mermaid import....250
Snapshots (versions)....250
Star and archive....251
Link a node to another app's entity....252
Promote a single node to a Bam task....252
Promote a whole graph to Bam tasks....253
Generate a diagram from a Bam project....253
Two-way sync with Bam....253
Live collaboration refresh....254
Comments, People, and version History panels....254
Working with AI agents....255
Working together (live presence)....257
User Stories....258
Story: Create your first flowchart....258
Story: Visualize an existing Bam project....258
Story: Turn a mind map into a project plan....259
Story: Keep a diagram and its tasks in sync....259
Story: Reorganize a messy graph....260
Story: Have an agent draft a diagram from a document....260
Story: Review and snapshot before an agent rewrite....260
Story: Curate your diagram library....261
Related....262
Board....265
Overview....266
Key concepts....267
Where to find it....268

Feature reference....269
Board list (All Boards)....269
Creating a board from a template....270
Creating a blank board....271
The infinite canvas....271
The board toolbar....273
Locking a board....273
Chat....274
Version history (snapshots)....274
Archive, restore, and delete....275
Duplicating a board....275
Starring a board....276
Templates browser....276
Fixing a board with an integrity issue....276
Export....276
Collaborators (no in-app management today)....277
Real-time presence and audio....277
Element limits....277
Working with AI agents....277
Working together (live presence)....280
User Stories....281
Story: Create your first board from a template....281
Story: Create a blank board....281
Story: Brainstorm on the canvas....281
Story: Collaborate live with teammates....282
Story: Share a board link....282
Story: Chat while you whiteboard....283
Story: Star and find your boards....283
Story: Snapshot and restore a board....283
Story: Promote brainstorm stickies into Bam tasks....284
Story: Lock a finished board....284
Story: Fix a board flagged with an integrity issue....284
Story: Archive, restore, or delete a board....285
Story: Export a board image....285
Story: Have an agent set up and seed a board....286
Related....287
Bolt....291
Overview....292
Key concepts....292
Where to find it....294
Feature reference....295
Automations home....295
Enabling and disabling an automation....296
Creating and editing an automation....296
Test Run....299
Duplicating an automation....300
Deleting an automation....300
Per-automation executions....300
Execution Log (organization-wide)....300
Execution detail....301
Retrying a failed run....302

Templates....302
Working with AI agents....303
Working together (live presence)....306
User Stories....307
Story: Create your first automation from a template....307
Story: Create an automation by hand....307
Story: Auto-create a Bam task when a Bond deal goes stale....308
Story: Turn an automation off and back on....309
Story: Watch a run and read its detail....309
Story: Retry a failed run....309
Story: Schedule a recurring automation....310
Story: Operate Bolt from an AI agent....310
Related....312
Bond....315
Overview....316
Key concepts....316
Where to find it....318
Feature reference....319
Choosing the active pipeline....319
Pipeline Board....319
Creating a deal....320
Deal detail and outcomes....321
Stage History and Related items....322
Contacts list....322
Creating a contact....323
Bulk-importing contacts....323
Contact detail....324
Companies list....325
Creating a company....325
Company detail....326
Logging activity....326
Analytics....327
Bond Settings: Pipelines....327
Bond Settings: Custom Fields....328
Bond Settings: Lead Scoring....329
Restoring deleted records....329
Working with AI agents....329
Working together (live presence)....332
User Stories....333
Story: Set up your first sales pipeline....333
Story: Add and qualify a contact....333
Story: Create a deal and move it to close....334
Story: Work the board with swimlanes....334
Story: Triage stale deals....335
Story: Forecast revenue....335
Story: Manage companies....335
Story: Configure lead scoring....336
Story: Extend the schema with custom fields....336
Story: Restore a deleted record....337
Story: Deduplicate contacts (agent-assisted)....337
Story: Hand a deal off across the suite....338

Related....339
Book....343
Overview....344
Key concepts....345
Where to find it....346
Feature reference....347
Week view....347
Day view....347
Month view....348
Timeline view....348
New Event and Edit Event....349
Event detail and RSVP....350
Booking Pages list....351
Booking page editor....352
Public Meet page....353
Calendars....353
Working Hours....354
Connections....355
iCal feed....357
In-app Help....357
Working with AI agents....357
Working together (live presence)....360
User Stories....361
Story: Set your working hours....361
Story: Schedule a one-off meeting....361
Story: View and adjust an event....361
Story: Cancel an event....362
Story: RSVP to an invitation....362
Story: Manage multiple calendars....362
Story: Create and share a public booking page....363
Story: Book time as an outside visitor....363
Story: See everything happening this week across apps....364
Story: Find a common meeting time across several people (agent-driven)....364
Story: Subscribe to a Book calendar from an outside app (iCal, agent or API)....365
Related....366
Brief....369
Overview....370
Key concepts....370
Where to find it....372
Feature reference....374
Home....374
Browse the document list....374
Create a document....375
Write and format....376
The "Brief summary (optional)" field....377
Set visibility....377
Set an icon....377
Choose a project....377
Start from a template....378
Save a draft and publish....378
Read a document....379

Edit an existing document and co-edit in real time....380
Comments....380
Star a document....381
Duplicate a document....381
Export a document....381
Archive and restore....381
Promote to Beacon....381
Linked Items....382
Folders and project scope....382
Search....382
Working with AI agents....383
Working together (live presence)....386
User Stories....387
Story: Write your first document and publish it....387
Story: Start from a template....387
Story: Organize documents with folders and project scope....387
Story: Co-edit a document in real time....388
Story: Find a document....388
Story: Review a document with comments....388
Story: Star a document for quick access....389
Story: Duplicate a document to reuse its structure....389
Story: Export a document to a file....389
Story: Archive a document and restore it later....390
Story: Promote a document into Beacon knowledge....390
Story: Link a spec document to a task (agent driven)....390
Story: Snapshot and restore a version (agent driven)....391
Story: Agent authors a document end to end....391
Related....392
Bureau....395
Overview....396
Key concepts....396
Where to find it....398
Feature reference....399
Browse floors....399
The live floor view (presence canvas)....400
Recent chats....401
Move yourself between rooms....401
See who is in a room....402
Knock and respond to a knock....402
Summon (Bring everyone here)....403
Ring (Invite)....403
Hunt....404
Book and cancel a room....404
Door states (set a room's privacy)....405
Set your status and DND....406
The cross-app docked box (audio huddles, mic, cam, screen)....406
Admin - Floors list....407
Admin - Floor editor....408
Admin - Offices....409
Admin - Settings....410
Working with AI agents....410

Working together (live presence)....413
User Stories....414
Story: Drop into the office and join a room....414
Story: Knock on a busy colleague's office....414
Story: Triage knocks at your own office....415
Story: Bring everyone here (teleport the room)....415
Story: Invite one person to your screen....415
Story: Hunt a teammate....416
Story: Book the conference room....416
Story: Recover what was said in a room....417
Story: Build a floor (admin)....417
Story: Assign an office owner (admin)....418
Story: Agent staffs a room and pulls humans in....418
Related....419
Helpdesk....423
Overview....424
Key concepts....424
Where to find it....426
Feature reference....427
Choose your support portal (org picker)....427
Register (create an account)....427
Login....427
Verify email....428
My Tickets list....428
New Ticket....429
Ticket detail (the conversation)....430
Notifications prompt....432
In-app Help....432
The agent queue (REST and MCP only)....433
Settings and admin....433
Working with AI agents....434
Working together (live presence)....437
User Stories....438
Story: File your first support ticket....438
Story: Track a ticket and reply to an agent....438
Story: Filter your tickets by status....439
Story: Attach a file or inline image....439
Story: Change a ticket's priority....439
Story: Mark a ticket as a duplicate....439
Story: Close a ticket, then reopen it....440
Story: Share a ticket to a Banter channel....440
Story: Turn on browser notifications....440
Story: Work the agent queue....441
Story: Merge duplicate tickets....441
Story: Set up a new support portal....441
Story: Reconcile a customer from an external system....442
Story: Report on ticket trends....442
Flagged behavior (known mismatches)....443
Related....444
Answer Key....447
Glossary....457

Index.....471

CHAPTER 1

BigBlueBam

In this chapter

Overview . Key concepts . Where to find it . The sixteen apps . Plan and build the work . Communicate and coordinate . Grow the business . Run operations . How the apps work together . Pervasive presence: the apps are live, shared spaces

Overview

Most teams do not run one tool. They run a project tracker, a chat app, a CRM, a help desk, a docs editor, a whiteboard, a scheduler, an invoicing tool, and a handful of others, and then they spend their day moving information between all of them by hand. A deal closes in one app and nobody tells the project app. A decision gets made in chat and never reaches the doc. The "AI feature" each vendor added last year is a chat box in the corner that can summarize a page but cannot actually do anything.

BigBlueBam is the answer to that sprawl: sixteen applications that behave like one product. They share a single login, a single organization and user model, and a single set of permissions. A deal in the CRM can become a project. A sticky note on the whiteboard can become a task. A won deal can become an invoice. You do not re-key anything, and you do not lose the thread when work moves from one app to the next.

Three ideas run through the whole suite:

- **It is one system, built recently, on one stack.** Not a decade of acquired products stitched together with webhooks. Because the apps share a data model, the connections between them are real, not bolted on.
- **Agents are first-class, not a sidebar.** Nearly every action a person can take in BigBlueBam is also available as an MCP tool, so an AI agent can do real work on the same boards, with the same permissions and the same audit trail as a person. The suite ships more than 730 of these tools. This is the architecture, not an add-on.
- **Presence is pervasive.** Every place you work is a live, shared surface. You can see which teammates are on the same task, deal, document, diagram, or board, pull them into a voice or video huddle with a tap, and co-edit in real time. Voice and video are ambient here, the digital equivalent of bumping into someone in the hallway, not a scheduled meeting.

BigBlueBam is open source and self-hostable. You can read it, run it on your own hardware, and export your data with plain SQL. It is built for small-to-medium teams (2 to 50 people), and it scales out by running more copies of any stateless service.

Key concepts

A few ideas explain how the whole suite hangs together.

- **One organization, shared everywhere.** You sign in once. Your organization, your teammates, and your roles are the same in every app. Switching from the project board to the CRM to the help desk does not mean switching accounts.
- **One permission model.** Roles (owner, admin, member, viewer, guest) and per-action permissions apply consistently across the apps, so who-can-see-what is decided in one place, not re-invented per tool.

- **The apps cross-reference each other.** A help desk ticket can spawn a project task. A document can link to a knowledge-base article. A booking can create a CRM contact. These links are first-class, so each side can show the other.
- **Events connect the apps.** When something happens in one app (a task changes state, a deal goes stale, a form is submitted), it can emit an event that an automation in Bolt picks up to do the next thing, with no glue code.
- **Agents work the same surfaces you do.** An agent authenticates like a user, is bound by the same permissions, and every action it takes is recorded. It is not a separate, weaker API. It is the same product.
- **The apps are live, not solo.** Presence, real-time co-editing, and one-tap voice and video are woven through the suite, so working together does not mean scheduling a meeting. A virtual office (Bureau) carries your presence across every app, and an agent can be pulled into a call too.
- **Yours to run.** The software is open source. Run the whole stack with one command, point it at managed databases if you like, and your data stays in a Postgres database you can query directly.

Where to find it

Everything lives under one domain. Each app has its own path, and a Launchpad and a command palette move you between them.

- The **Launchpad** lists every app available to your organization and is the fastest way to jump between them.
- The **command palette** (press Cmd+K or Ctrl+K in any app) searches and navigates without the mouse.
- Each app is served at its own path: the project app at `/b3/`, chat at `/banter/`, the CRM at `/bond/`, the help desk at `/helpdesk/`, and so on. A full path map is in the project README.
- The **"(?)" Help Center** in each app's top bar opens the same documentation you are reading now, per app, with search and cross-references.

The sixteen apps

Here is the whole suite at a glance, grouped by what you are trying to do. Each app is deep enough to stand on its own and is genuinely better because it lives next to the others. The tool counts are the number of MCP actions an agent can take in that app.

Plan and build the work

- **Bam (123 tools), project management.** Sprint-powered Kanban boards with configurable phases, swimlanes, carry-forward sprints, and Board, List, Timeline, Calendar, and Workload views. The hub the other apps feed work into.
- **Board (40 tools), visual collaboration.** An infinite canvas for real-time brainstorming, with sticky notes that can be promoted straight into Bam tasks.
- **Brief (48 tools), collaborative documents.** Real-time documents that can graduate into the knowledge base when they are ready to be canonical.
- **Beacon (38 tools), knowledge base.** Hybrid search, a knowledge graph, and built-in freshness governance so the answers stay honest instead of rotting.
- **Blueprint (36 tools), structured diagrams.** Diagrams that are data, not drawings: every node and edge is a real object you and your agents can read, edit, version, and turn into tasks.
- **Bearing (30 tools), goals and OKRs.** Set objectives and key results, track progress, and catch the goals slipping behind before the period closes.

Communicate and coordinate

- **Banter (69 tools), team chat.** Real-time channels and DMs where your conversations and your AI agents work in the same room.
- **Bureau (37 tools), virtual office.** Live floors and rooms with a presence widget that follows you across the suite, so a remote team has a place to be.
- **Book (25 tools), scheduling.** Team calendars, availability, and public booking links, with bookings that can flow into the CRM.

Grow the business

- **Bond (69 tools), CRM.** A lightweight CRM with visual pipelines, stale-deal detection, and a full agent surface. Deals can become projects and invoices.
- **Blast (28 tools), email campaigns.** Turn Bond contacts into campaigns with a visual builder, saved segments, and open and click analytics.
- **Helpdesk (11 tools), support portal.** A branded support portal per organization, where customer tickets resolve on the project board.

Run operations

- **Bill (47 tools), invoicing.** Turn client work into billed, tracked, and reconciled revenue, with PDFs and recurring billing.

- **Bench (32 tools), analytics.** Shared dashboards, ad-hoc queries, and scheduled reports built from the data the rest of the suite already creates.
- **Blank (20 tools), forms.** Build forms and surveys in a visual editor, publish them to a public link, and collect responses.
- **Bolt (24 tools), automation.** Event-driven workflows that connect every app: when this happens, if these conditions hold, then run these steps.

How the apps work together

The suite earns its name when work crosses app boundaries without anyone copying and pasting. A few of the connections that ship today:

- **From lead to delivery to paid.** A Bond deal can be promoted into a Bam project, the delivered work can be turned into a Bill invoice from tracked time, and a public Book booking can create the Bond contact in the first place.
- **From conversation to work.** A Banter message can be shared as a task or a ticket, and a task can be deep-linked back into a channel by its human-readable id. A Board sticky or a Blueprint node can be promoted into Bam tasks.
- **From support to fix.** A Helpdesk ticket can spawn a Bam task, and the task then shows a Helpdesk tab linking the two, so the engineer and the support agent see the same thread.
- **From knowledge to reuse.** A Brief document can be promoted into a Beacon knowledge article, and documents and articles can link to the tasks and entities they describe.
- **One notification surface and one activity stream.** Mentions, assignments, and cross-app events land in a shared feed and a unified activity view, so you do not watch sixteen inboxes.
- **Automation as the connective tissue.** Bolt listens for events from any app (a deal goes stale, a form is submitted, a task is assigned) and runs the next step, so the integrations above can be extended without writing glue code.

Pervasive presence: the apps are live, shared spaces

BigBlueBam is built for teams that work together all day, whether they share a room or are spread across the world. The apps are not solo tools you each use in your own tab. They are live, shared surfaces. Open a task, a deal, a document, a diagram, a ticket, or a board and you can see who else is there with you, right on the page.

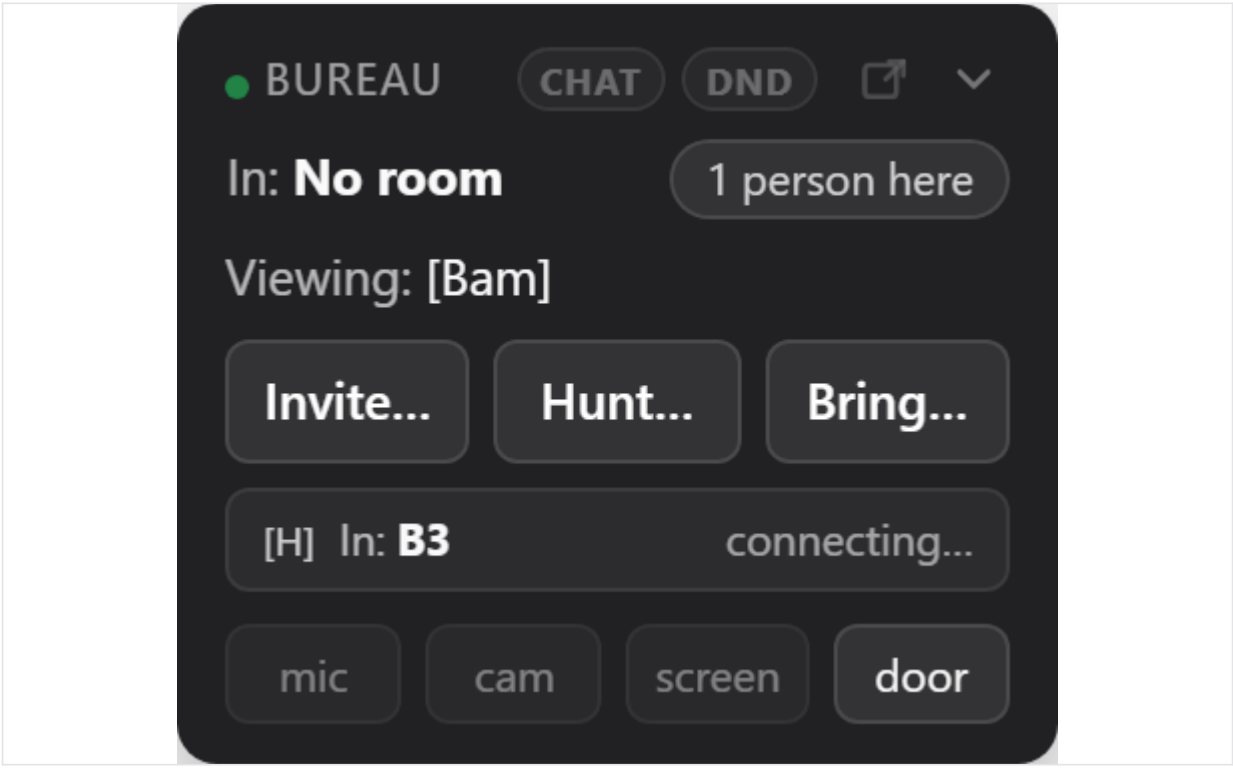


Figure 1.1 *The Bureau dock, available in every app: presence and chat at the top, the Invite, Hunt, and Bring controls for pulling people together, and mic, camera, screen-share, and door controls for the huddle itself.*

That presence is the doorway to working together in the moment:

- **See who is here.** A presence strip on the work surfaces (tasks in Bam, deals in Bond, documents in Brief, diagrams in Blueprint, tickets in Helpdesk, knowledge in Beacon, and the Board canvas) shows the teammates looking at the same thing you are.
- **Pull someone in with a tap.** From that strip you can ring a teammate into a voice or video huddle on the spot. It is a tap, not a calendar invite.
- **Work in the same place.** Documents in Brief, the canvas in Board, and diagrams in Blueprint are co-edited live, with each other's cursors and changes appearing as they happen. Banter carries real-time channels, DMs, and calls or huddles in the same room as the conversation.
- **A place to be.** Bureau is the always-on virtual office: live floors and rooms, with a presence widget that follows you across every app, so a teammate deep in the CRM is still findable and reachable.

- **Bring an agent into the room.** Presence and calls are not limited to people. An AI agent can be invited into a Banter call to listen and help.

The mental model is a hallway, not a meeting. Voice and video here are not a scheduled call or a formal meeting; they are what happens when you bump into a colleague in the hall or wander over to their desk. A quick question, a shared screen, two people looking at the same thing for a minute, then back to work. Geography stops mattering: the team is together even when it is apart.

Working with AI agents

BigBlueBam treats an AI agent as a teammate that uses the product, not as a chatbot pasted onto it.

- **Parity, not a sidebar.** Nearly every action a person can take is also an MCP tool. The suite exposes more than 730 of them across the sixteen apps and a shared platform layer (identity, search, activity, approvals, and more). An agent can plan a sprint, move a deal, triage a ticket, draft a campaign, or run a report.
- **Same permissions, same audit trail.** An agent authenticates like any user and is bound by the same roles and per-action permissions. Every action it takes is recorded, and destructive actions require an explicit two-step confirmation.
- **Bring your own agent.** The platform gives an agent the same access a person has. Running the agent is up to you. Agent operation is a capability the suite enables, not a turnkey service that the install deploys for you.

Open source and self-hosting

The software is open source. You can read it, run it, host it, and export from it.

- **One command to run the whole stack.** The entire suite comes up with Docker Compose. Data services (Postgres, Redis, object storage, vector search) can be swapped for managed cloud equivalents by changing environment variables only.
- **Your data is yours.** Everything lives in a Postgres database you can query. There is no proprietary export wall and no "contact sales to get your data out."
- **Scales by copies.** The application services are stateless, so you scale by running more of them. A team of one can run it from a single board; a team of fifty can run dozens of projects side by side.

User Stories

These walk through scenarios that cross several apps, which is where the suite is different from sixteen separate tools.

Story: A lead becomes a project becomes an invoice

A prospect books a call through a public Book link. The booking creates a contact in Bond, where it moves through the pipeline until the deal is won. The won deal is promoted into a Bam project, the team delivers the work and tracks time against its tasks, and at the end of the month Bill turns that tracked time into an invoice. No one re-typed the client's details at any step.

Story: A support ticket becomes a fix

A customer files a ticket in your organization's Helpdesk portal. Support triages it and spawns a Bam task for engineering. The task carries a Helpdesk tab that links back to the ticket, so when the engineer closes the task, support can see it and reply to the customer. The whole thread stays connected.

Story: An idea becomes shipped work

The team sketches a feature on a Board canvas, turns the sticky notes into Bam tasks, and writes the spec in a Brief document that links to those tasks. When the spec stabilizes, it is promoted into Beacon so the next person who asks "how does this work" finds the canonical answer instead of re-deriving it.

Story: An agent works alongside the team

An AI agent assigned to the project watches for tickets, drafts replies, and files tasks, all through the same MCP tools and under the same permissions a human teammate has. Its actions show up in the activity stream next to everyone else's, and anything destructive waits for a person to confirm.

Story: A hallway moment, on demand

You open a task in Bam and notice from the presence strip that a teammate is already looking at it. Instead of typing a paragraph, you ring them into a huddle straight from the card, talk it through for two minutes while you both look at the same task, and get back to work. No meeting was scheduled, no link was pasted, and nobody left the app they were in.

Story: One place to stand

A distributed team keeps Bureau open as its virtual office. Presence follows each person across every app, so a teammate deep in the CRM is still reachable, and a quick question does not require scheduling a meeting. When a decision lands, it becomes a task, a doc, or a ticket without leaving the suite.

CHAPTER 2

Bam

Plan it, drag it, ship it.

Bam is where the work lives. Every project is a board you shape yourself, with your own phases, task states, and fields, run in time-boxed sprints that carry unfinished work forward instead of dropping it. This chapter takes you from your first project to closing a sprint and reading the reports, and it shows where an AI agent can pick up the same board your team uses. By the end you will be able to set up a project, run a sprint start to finish, and find any task fast.

At a glance

- Configurable boards: phases, task states, labels, and fields, set per project
- Time-boxed sprints with carry-forward that never quietly loses work
- Rich task cards: subtasks, time tracking, start and due dates, custom fields
- Board, List, Timeline, and Calendar views of the very same work
- Full MCP access, so an agent can run the board with nearly a person's authority

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Project dashboard and project list . Kanban board . Task detail drawer . Creating and editing tasks . Subtasks and parent tasks . Sprints

> *Bam is the BigBlueBam flagship: a multi-user Kanban planner where teams run projects on configurable boards, plan and close sprints, track tasks with rich detail, and report on delivery. Reach for it when a team needs to organize work into phases and time-boxed iterations.*

Overview

Bam is where work gets planned and tracked. Each project[1] is a board with its own phases (columns), task[2] states, labels, epics, sprints, custom fields, and templates. You move tasks across the board by dragging cards, group them into time-boxed sprints, and carry unfinished work forward when a sprint[3] closes. Cards carry a full task record: assignee, priority[4], story points, start and due dates[5], subtasks, comments, attachments, time entries, and custom fields.

> **TIP**
> Card positions are stored as floating-point values, so reordering a long column never renumbers its neighbors. Drag freely.

> ***Carry-forward is a first-class concept, so unfinished work follows you instead of falling through the cracks.***

Bam is built for small-to-medium teams. It supports many concurrent projects, and every project can be shaped differently because phases, states, and custom fields are all configurable per project (and priorities per org). The same data is reachable through MCP tools, so an AI agent can create tasks, run reports, or plan a sprint with nearly the same authority a human has in the UI.

> ***Every board action is also an MCP tool, so an agent can plan a sprint with nearly the authority a person has.***

Bam connects to the rest of the suite. Helpdesk tickets can spawn Bam tasks (a Helpdesk tab then appears on the task), GitHub commits and pull requests can link back to a task, Slack receives sprint and task notifications, and Banter messages can reference a task by its human id so a deep link opens the right board with the task drawer open.

The core objects you work with are projects, phases, task states, tasks, sprints, epics, labels, custom fields, and saved views.

---

[1] **Project.** A board with its own phases, states, labels, epics, sprints, custom fields, and templates. Every project has a task id prefix (2 to 6 uppercase letters, for example MAGE or FRND) that prefixes every task id. Project roles are admin, member, and viewer.

[2] **Task.** The unit of work. Each task has a human id in the form PREFIX-N (for example MAGE-38). A task holds a title, rich-text description, phase, state, sprint, epic, assignee, reporter, priority, story points, time estimate and logged time, start and due dates, labels, custom fields, links, parents and subtasks, comments, and attachments.

[3] **Sprint.** A time-boxed iteration with a status of planned, active, completed, or cancelled. Only one sprint can be active per project at a time. The default sprint length is 14 days. A sprint carries a goal, start and end dates, velocity, and retrospective notes.

[4] **Priority.** A per-org configurable value. The default set is Critical, High, Medium, Low, and None. New tasks default to Medium.

[5] **Start and due dates.** Each task can carry a start date and a due date. These drive the Timeline (Gantt) and Calendar views and feed the overdue and My Work groupings.

Key concepts

- **Organization (org)** - The top-level tenant. Org roles are owner, admin, and member, plus a guest role. Priorities are configured at the org level.
- **Project** - A board with its own phases, states, labels, epics, sprints, custom fields, and templates. Every project has a task id prefix (2 to 6 uppercase letters, for example MAGE or FRND) that prefixes every task id. Project roles are admin, member, and viewer.
- **Phase**[6] - A board column. A phase can carry a WIP limit, a color, start and terminal flags, and an auto-state that is applied when a task enters the phase.
- **Task state**[7] - The status of a task, orthogonal to its phase. Each state belongs to a category (todo, active, blocked, review, done, or cancelled) and may be a closed state. Tasks in a closed state count toward velocity.
- **Task** - The unit of work. Each task has a human id in the form PREFIX-N (for example MAGE-38). A task holds a title, rich-text description, phase, state, sprint, epic[8], assignee, reporter, priority, story points, time estimate and logged time, start and due dates, labels, custom fields, links, parents and subtasks, comments, and attachments.
- **Priority** - A per-org configurable value. The default set is Critical, High, Medium, Low, and None. New tasks default to Medium.
- **Sprint** - A time-boxed iteration with a status of planned, active, completed, or cancelled. Only one sprint can be active per project at a time. The default sprint length is 14 days. A sprint carries a goal, start and end dates, velocity, and retrospective notes.
- **Carry-forward**[9] - When you complete an active sprint, each incomplete task can be carried forward to a target sprint, moved to the backlog, or cancelled. Carried tasks show a carry-forward badge and remember their original sprint for reporting.
- **Epic** - A grouping of tasks across sprints, with a status of open, in progress, or closed. Epics roll up task counts and story points.
- **Label**[10] - A project-scoped tag with a name and color.
- **Custom field**[11] - A project-scoped field stored on each task. The seven field types are text, number, date, select, multi-select, checkbox, and url. A field can be required and can be shown on the card.

---

[6] **Phase.** A board column. A phase can carry a WIP limit, a color, start and terminal flags, and an auto-state that is applied when a task enters the phase.

[7] **Task state.** The status of a task, orthogonal to its phase. Each state belongs to a category (todo, active, blocked, review, done, or cancelled) and may be a closed state. Tasks in a closed state count toward velocity.

[8] **Epic.** A grouping of tasks across sprints, with a status of open, in progress, or closed. Epics roll up task counts and story points.

[9] **Carry-forward.** When you complete an active sprint, each incomplete task can be carried forward to a target sprint, moved to the backlog, or cancelled. Carried tasks show a carry-forward badge and remember their original sprint for reporting.

[10] **Label.** A project-scoped tag with a name and color.

[11] **Custom field.** A project-scoped field stored on each task. The seven field types are text, number, date, select, multi-select, checkbox, and url. A field can be required and can be shown on the card.

- **Saved view**[12] - A saved filter, sort, swimlane[13], and view-type preset. It can be private to you or shared with the project. Saved views persist the Board, List, Timeline, and Calendar view types.
- **Swimlane** - A horizontal grouping of the board. The options are No Swimlanes, By Assignee, By Priority, and By Epic.
- **Start and due dates** - Each task can carry a start date and a due date. These drive the Timeline (Gantt) and Calendar views and feed the overdue and My Work groupings.
- **Watcher**[14] - A user subscribed to a task's notifications. Watchers are populated by the system as people get involved with a task. There is no manual add-watcher or remove-watcher control in the Bam UI.
- **Done-gate**[15] - A task cannot move into a closed state while it still has open subtasks. The board surfaces this as an alert and the API returns an INCOMPLETE_SUBTASKS error.

> **NOTE**
> Only one sprint can be active per project at a time. Plan the next one while the current sprint runs, then start it the moment this one closes.

> **WARNING**
> The done-gate is strict. Close or cancel the open subtasks first, or the API returns INCOMPLETE_SUBTASKS and the card stays exactly where it was.

Where to find it

Bam is served at `/b3/`. From the left sidebar you reach **Dashboard** (the project list), **My Work** (your personal task queue), and the **Projects** list with a plus button to create a project. Opening a project lands you on its board at `/projects/:id/board`.

> **TRY IT**
> Open /b3/, click Projects, and create a project with a two-to-six-letter prefix like DOCS. Your very first task will be DOCS-1.

To use Bam you need an authenticated session (a login cookie or a `bbam_` API key) and membership in at least one project. Some actions require a project admin role or an org owner or admin role. On a brand-new install, Bam routes you to `/b3/bootstrap` to create the first account until a SuperUser exists.

---

[12] **Saved view.** A saved filter, sort, swimlane, and view-type preset. It can be private to you or shared with the project. Saved views persist the Board, List, Timeline, and Calendar view types.
[13] **Swimlane.** A horizontal grouping of the board. The options are No Swimlanes, By Assignee, By Priority, and By Epic.
[14] **Watcher.** A user subscribed to a task's notifications. Watchers are populated by the system as people get involved with a task. There is no manual add-watcher or remove-watcher control in the Bam UI.
[15] **Done-gate.** A task cannot move into a closed state while it still has open subtasks. The board surfaces this as an alert and the API returns an INCOMPLETE_SUBTASKS error.

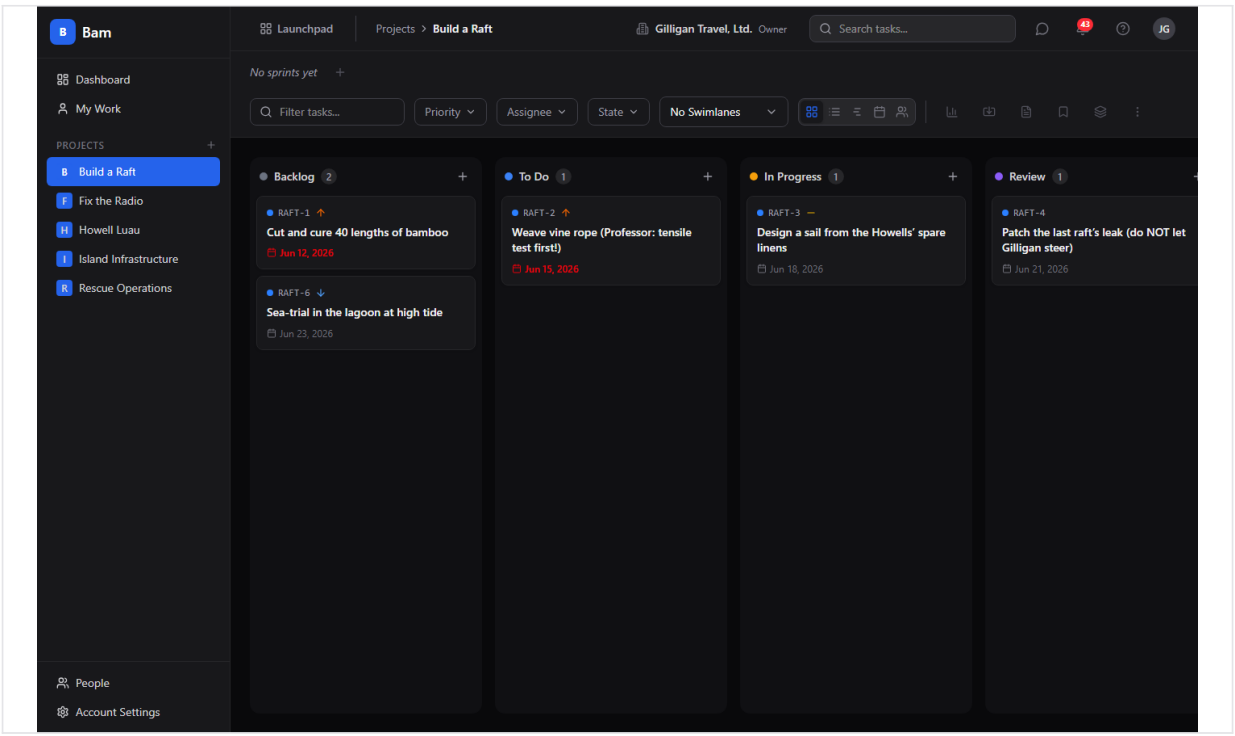


Figure 2.1 *Kanban board*

Feature reference

Project dashboard and project list

The dashboard is your home page in Bam. It lists every project you can see.

To create a project:

1. From the sidebar, click **Dashboard**, or click the plus button next to **Projects**.
2. Click **New Project** (the empty state reads **No projects yet** with a **Create Project** button).
3. Enter a project **name** and a **task id prefix** of 2 to 6 uppercase letters.
4. Optionally set a description, icon, color, project template, and default sprint duration.
5. Submit. You land on the new project's board with its phases and states seeded from the template you chose.

To open a project, click its card on the dashboard or its entry in the sidebar. Project changes require an admin role; deleting (archiving) a project requires admin plus the relevant org permission.

Kanban board

The board is the primary view. Phases are columns and tasks are cards you drag between them.

The board header carries the sprint selector, a filter bar, the swimlane selector (**No Swimlanes**, **By Assignee**, **By Priority**, **By Epic**), a lane-sort control (used with the epic swimlane), the view switcher (Board, List, Timeline, Calendar, Workload), and action icons for the dashboard, **Import tasks**, **Task templates**, **Saved views**, **Manage epics**, and a **Project options** three-dot menu.

To add a task to a column, use the column's add control to open the create dialog, or type into the inline add input under a column.

To move a task, drag its card to another column or to a new position. The board persists the new position immediately. If you drag a task with open subtasks into a closed state, the move is blocked and you see an alert.

To act on a single card, right-click it. The context menu offers **Open detail**, **Add subtask…**, **Duplicate**, **Priority** (Critical, High, Medium, Low, None), **Move to phase**, **Set state**, **Assign to…** (Unassigned plus org members), **Change parent task** (searchable), and **Delete task** (which warns that deletion is irreversible).

The **Project options** menu holds **Manage Phases**, **Custom Fields**, **Export**, and **Delete Project**.

Task detail drawer

Opening a task slides in a detail drawer with everything about that task.

To open a task, click its card or choose **Open detail** from the card's context menu. The drawer header shows the task's human id (click it to copy), a state selector, and a priority selector, plus controls to **Share to Banter**, **Duplicate task**, and close.

The drawer body has tabs: **Details**, **Comments**, **Activity**, and a **Helpdesk** tab that appears only when the task was created from a Helpdesk ticket. On the **Details** tab you can edit the title and rich-text description, manage parent tasks and subtasks, add labels, upload attachments, and add task links. The right sidebar holds **Assignee**, **Reporter**, **Sprint**, **Phase**, **Epic**, **Story Points**, **Time Logged**, **Start Date**, **Due Date**, **Labels**, and **Custom Fields**, and a **Delete Task** action at the bottom.

The **Assignee** picker lists all members of the org, not just members of the current project, so you can assign work to anyone in the org. To edit any field, change its control in the sidebar; the drawer saves the change as you go. The description auto-saves shortly after you stop typing.

A task's watchers are maintained automatically as people get involved (for example, being assigned or commenting). There is no watcher list or add-watcher control in the drawer.

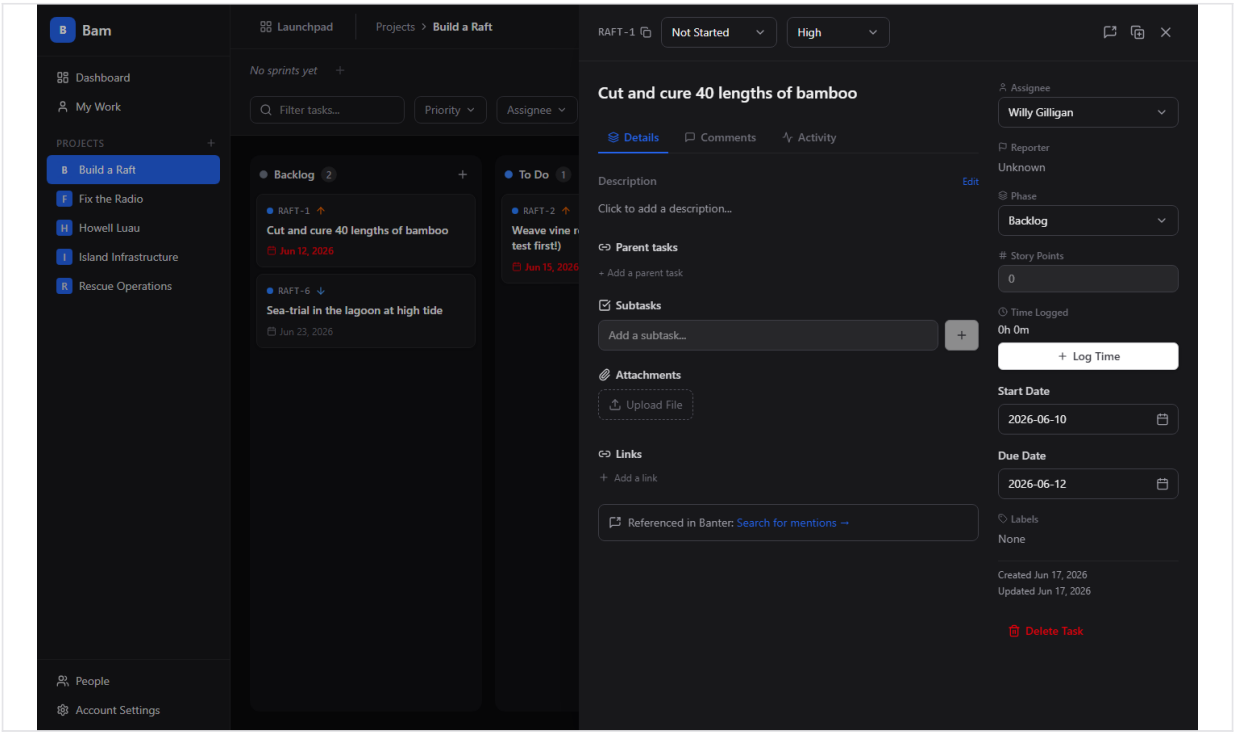


Figure 2.2 Task detail

Creating and editing tasks

To create a task from the create dialog:

1. Trigger the dialog from a column's add control or press the **N** shortcut.

2. Fill in **Title** (placeholder "What needs to be done?"), and optionally **Phase**, **Priority**, **Story Points**, **Assignee**, **Due Date**, **Labels**, and a description. The **Assignee** picker lists every org member.
3. Click **Create Task**.

To edit a task, open its drawer and change fields inline, or use the card context menu for quick priority, phase, state, and assignee changes.

To duplicate a task, use **Duplicate task** in the drawer header or **Duplicate** in the card context menu. You can include subtasks when duplicating.

Bam blocks moving or updating a task into a closed state while it has open subtasks. Parent-child links are guarded against self-parenting and cycles.

Subtasks and parent tasks

A task can have multiple parents and multiple subtasks.

To add a subtask, open the task, and on the **Details** tab type a title into the **Add a subtask…** input and confirm. The subtask progress bar tracks how many subtasks are done.

To mark a subtask done, click its checkbox. Click the row body to open the subtask in the drawer.

To add a parent, in the **Parent tasks** section click **+ Add a parent task** (or **+ Add another parent**), search by title or id, and pick a task. Remove a parent with the X on its chip.

Sprints

Sprints are time-boxed iterations you run from the board's sprint selector.

To create a sprint, open the sprint selector and choose **Create sprint**, then set a name (placeholder "Sprint 1"), start and end dates, and an optional goal.

To start a sprint, select it and choose **Start this sprint**. Only one sprint can be active per project, so starting a sprint while another is active is rejected. Starting a sprint sends a Slack notification and fires a sprint-started event.

To complete a sprint, choose **Complete Sprint**. The complete dialog is titled **Complete Sprint: NAME** and lists every incomplete task with a per-task dropdown: **Carry forward**, **Move to backlog**, or **Cancel task**. Add optional **Retrospective Notes (optional)** and click **Complete Sprint**. Velocity is computed from closed tasks.

To delete a planned sprint, choose **Delete this sprint** from the selector. To cancel an active or planned sprint, the Cancel action moves its tasks back to the backlog and sets the sprint status to cancelled.

To review a finished sprint, choose **View sprint report**.

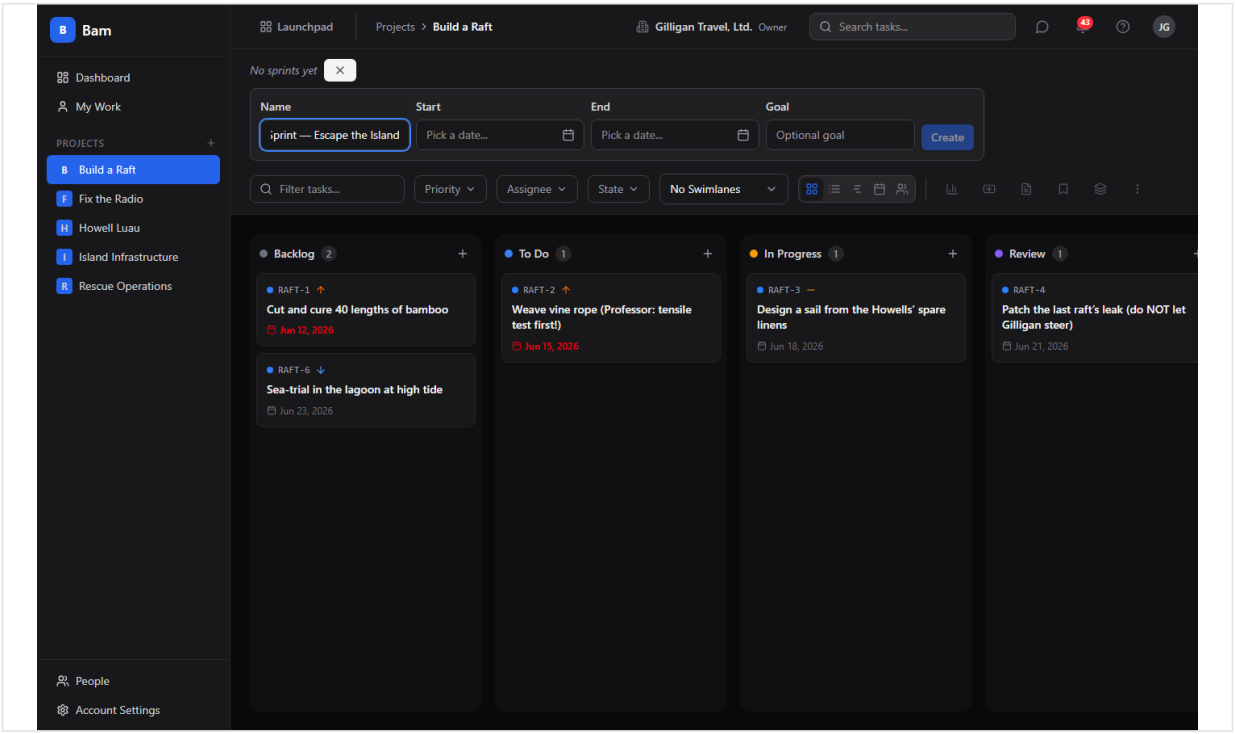


Figure 2.3 Sprint planning

Carry-forward

When you complete a sprint, carry-forward decides what happens to unfinished work.

In the **Complete Sprint** dialog, set each incomplete task's dropdown:

- **Carry forward** moves the task to the target sprint, increments its carry-forward count, and stamps its original sprint.
- **Move to backlog** removes the task from the sprint (descoped).
- **Cancel task** leaves the task in the completed sprint.

Carried-forward cards show a carry-forward badge on the board, and the original sprint is preserved so reports stay accurate.

Epics

Epics group tasks across sprints.

To manage epics, click **Manage epics** in the board header to open the epic manager, then create an epic with a name, description, color, and optional start and target dates.

To assign a task to an epic, open the task and set its **Epic** in the drawer sidebar.

To view epic progress, open an epic's detail page (click its chip) for a group-by-sprint task list and a burnup chart, with rollups of task and story-point counts.

Phases (columns)

Phases are the board's columns and define your workflow.

To manage phases, open the **Project options** menu and choose **Manage Phases**. From the phase manager you can add, edit, reorder, and delete phases, and set a WIP limit, color, start and terminal flags, and an auto-state applied on entry. When you delete a phase you can re-home its tasks to another phase.

Task states

Task states are the per-project status values, seeded from the project template. Each state has a category (todo, active, blocked, review, done, or cancelled) and a closed flag. You set a task's state from the drawer header, the card context menu's **Set state**, or by dragging into a phase with an auto-state.

Labels

Labels are project-scoped tags.

To add a label to a task, open the create dialog or the task drawer and pick from the project's labels. Labels are managed per project and are also visible org-wide to the cross-app resolver.

Custom fields

Custom fields add project-specific data to every task.

To manage custom fields, open the **Project options** menu and choose **Custom Fields**. Create a field of one of the seven types: text, number, date, select, multi-select, checkbox, or url. A field can be marked required and can be shown on the card. Custom-field values appear in the **Custom Fields** section of the task drawer sidebar.

Saved views

Saved views remember a filter, sort, swimlane, and view-type combination.

To save a view, set your filters, swimlane, and view, then click **Saved views** in the board header and save the current configuration. A saved view can be private to you or shared with the project; only the owner can edit or delete it. Saved views persist the Board, List, Timeline, and Calendar view types. Workload is a view mode you can switch to, but it is not a persistable saved-view type.

Alternate views (List, Timeline, Calendar, Workload)

The view switcher in the board header toggles between five views: **Board**, **List**, **Timeline**, **Calendar**, and **Workload**.

To change view, click the matching icon in the view switcher.

- **List** is a flat, sortable table of the project's tasks.
- **Timeline** lays tasks out by date as a Gantt chart, using each task's start and due dates.
- **Calendar** places tasks on their due dates.
- **Workload** shows load per user; clicking a user there filters the board to that user.

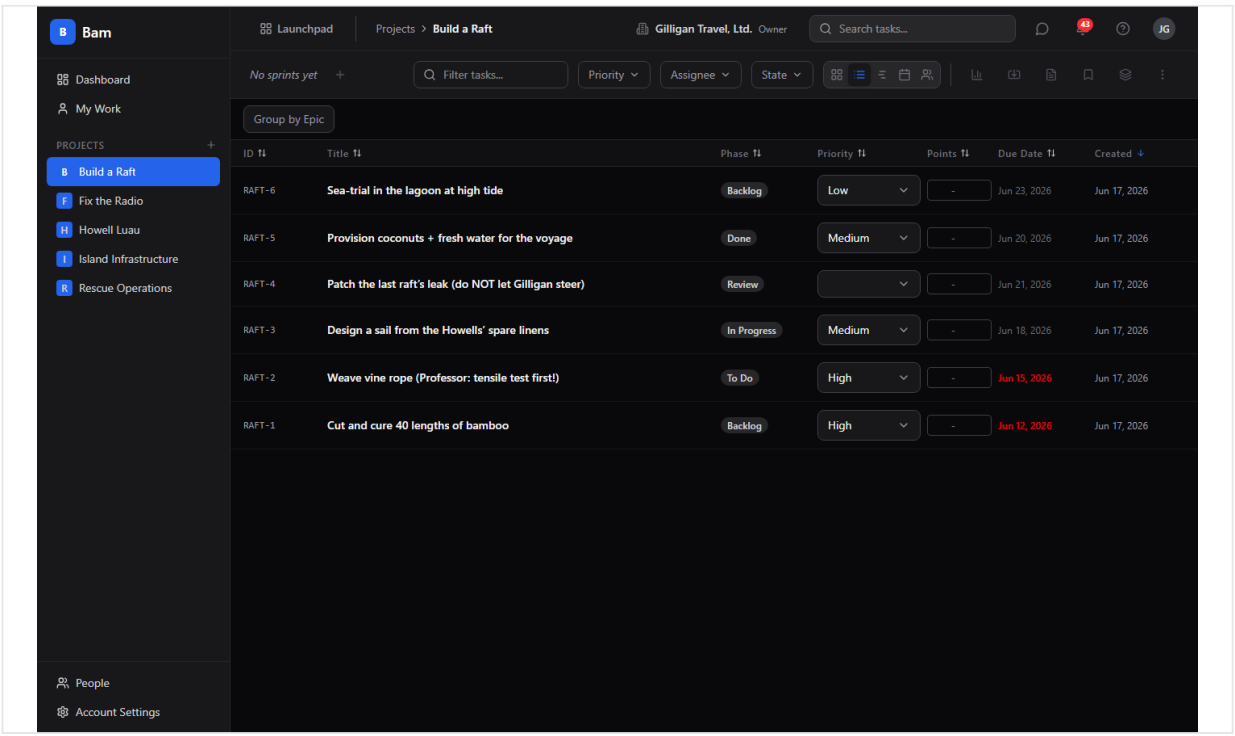


Figure 2.4 List view

Task templates

Templates are reusable task blueprints.

To manage templates, click **Task templates** in the board header. A template can carry a name, title pattern, description, priority, labels, phase, story points, and a list of subtask titles.

To create a task from a template, apply the template; Bam creates the task and its subtasks, and you can override fields at apply time.

Import

Import brings an existing backlog into a project.

To import, click **Import tasks** in the board header to open the import dialog. You can import from CSV, Trello, Jira, or GitHub Issues. For CSV you map columns, preview a dry run before committing, and configure value maps, link mappings, custom-field mapping, a duplicate strategy (create, skip, or update), and a date locale (us or iso). Unmatched phases and labels are created automatically.

Export

Export downloads a project's tasks.

To export, open the **Project options** menu and choose **Export**. Pick a **Format** of JSON or CSV, optionally scope to a specific sprint (or All sprints), and click **Export**. Export is rate-limited.

iCal feed

Bam can publish tasks with due dates as a calendar feed.

To subscribe, generate a calendar token for the project (or use your personal feed), then add the feed URL to an external calendar app. Project feeds are served at `/projects/:id/calendar.ics` and your personal feed at `/me/calendar.ics`, each authenticated by a token in the query string. You can create and revoke calendar tokens per project.

Comments, reactions, and revisions

Discuss work directly on the task.

To comment, open a task, go to the **Comments** tab, write in the editor, and click **Comment**. To react, click a reaction on a comment; the five reactions are thumbs-up, heart, rocket, eyes, and party. You can edit and delete your own comments. Editing snapshots the previous version, and an "(edited)" badge opens the full revision history.

Time tracking

Log time against a task as separate entries.

To log time, open a task and in the **Time Logged** section of the sidebar click **Log Time**, enter minutes, a date, and an optional description, then save. Each entry is its own row and the total rolls up into the task's logged time. Your personal time entries are also available across a date range.

Attachments

Attach files to a task.

To add an attachment, open a task and in the **Attachments** section click **Upload File** and choose a file. Each attachment shows its size and date with download and delete controls.

Task links

Attach external URLs to a task.

To add a link, open a task and use the **Links** section on the **Details** tab to add an http(s) URL with a title. A task can hold up to 50 links.

Reports and project dashboard

Bam reports on delivery at the project level.

To view reports, open a project's dashboard at `/projects/:id/dashboard` or its reports at `/projects/:id/reports`. Available reports include velocity, burndown, cumulative flow, cycle time (average and median lead time), overdue tasks, workload, time tracking over a date range, and status distribution. Burndown can target a specific sprint or the active sprint.

Audit log

Every project keeps an append-only activity record.

To view it, open the project audit log at `/projects/:id/audit-log`. Entries are written to the append-only, monthly-partitioned activity log.

People and members

Manage the people in your org and projects.

To manage people, open **/people** (org admins and owners) for the people list and each person's detail page at **/people/:userId**. The detail page has tabs **Overview**, **Projects**, **Access**, and **Activity**. From **Overview** you edit identity (display name, timezone) and membership (role, default org). From **Projects** you add the user to projects with a role. From **Access** you manage API keys, sessions, and password resets. Platform SuperUsers use **/superuser/people**.

To add a member to a project, use the project's member controls (project admin only) or add the user to projects from their **Projects** tab. Note that assignment is not gated on project membership: the task **Assignee** picker lists every org member, so you can assign a task to someone who is not yet a project member.

My Work

My Work is your personal queue across every project. It lists only the tasks assigned to you (it queries each project's task list filtered to your user id), so it never shows another person's work.

To open it, click **My Work** in the sidebar. The page groups your open tasks into **Overdue**, **Due This Week**, **In Progress**, and **All My Tasks**. Tasks land in **Overdue** or **Due This Week** based on their due date, in **In Progress** if they have a start date, and in **All My Tasks** otherwise. Completed tasks are hidden, and when nothing is assigned you see "All caught up!".

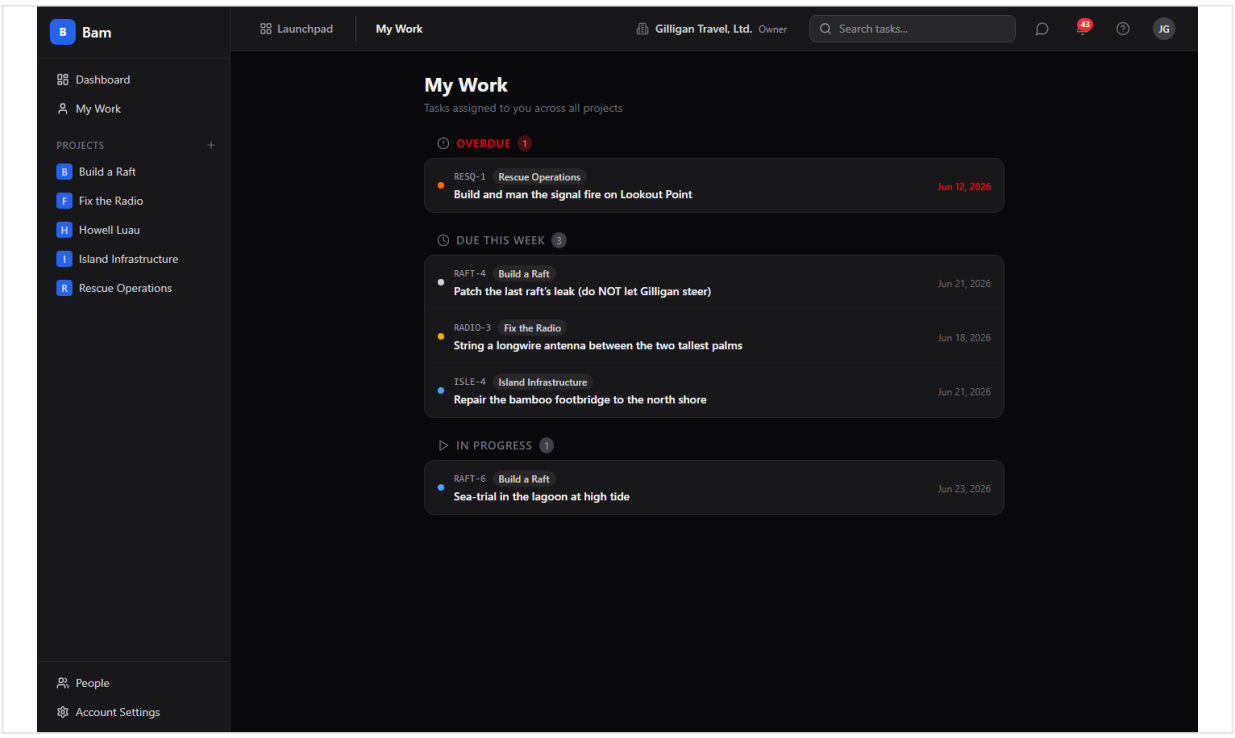


Figure 2.5 My Work

Settings

Settings control your profile and the org's configuration.

To open settings, go to **/settings**. The ten tabs are **Profile**, **Appearance** (System, Light, Dark), **Notifications**, **Members**, **Tasks** (which includes **Manage Priorities**), **Permissions**, **Launchpad**, **Integrations** (API keys, agents and service accounts, webhooks, Slack, and the per-org SMTP override), **AI Providers**, and **Helpdesk**. Use the **Tasks** tab's **Manage Priorities** to configure the org's priority set. The per-org email relay lives on the **Integrations** tab; the platform-wide relay it falls back to is set in the SuperUser Console (see **Email delivery (SMTP)** below).

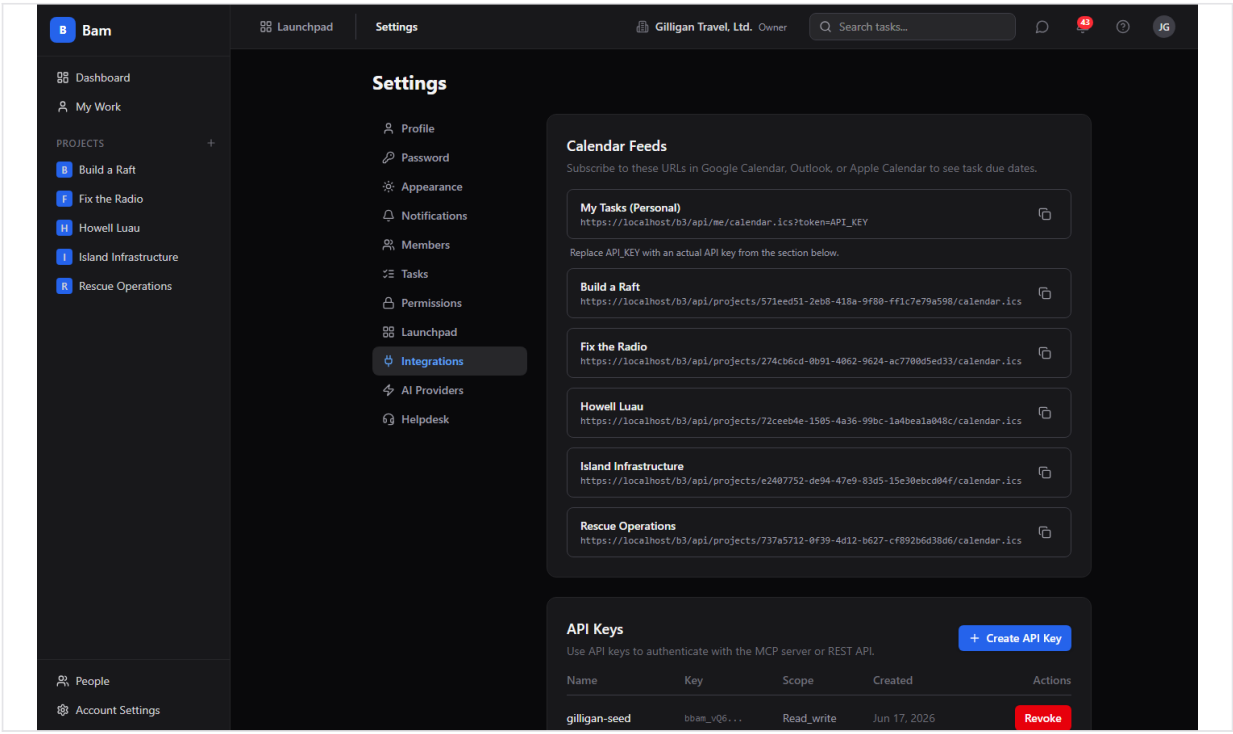


Figure 2.6 Integrations and API keys

Email delivery (SMTP)

BigBlueBam sends outbound mail (org invitations, password resets, system and reporting alerts, and Blast campaigns) through an SMTP relay. There are two levels of configuration, and the system always picks the most specific one that is set.

The resolution order is: the **org override** first, then the **platform relay**, then the server's **environment default**. In plain terms: if your organization has set its own relay, your org's mail goes through it; if it has not, your org's mail falls back to the platform relay a SuperUser configured; and if neither is set, the system uses the SMTP values baked into the server's environment variables. This used to live under **Account Settings > Integrations**; the platform relay has since moved to the SuperUser Console, and the per-org override now sits in its own card on the **Integrations** tab.

Platform email (SMTP). The platform relay is the system-wide, fallback sender. It is what the platform uses for new-org-setup invitations, the password-reset fallback, and system and reporting alerts, and it is the relay any organization falls back to when it has not configured its own. Only a platform SuperUser can set it.

To configure the platform relay, open the **SuperUser Console** at **/superuser**, click the **Platform** tab, and scroll to the **Platform SMTP relay** card. Fill in **SMTP Host**, **SMTP Port** (587 for STARTTLS, 465 for TLS-only), **SMTP Username**, **SMTP Password**, and **From Address**, set **Use TLS (secure)** for port 465, and save. Any field you leave blank falls back to the matching server environment variable (`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `EMAIL_FROM`). Use **Test connection** to verify login and TLS, or enter an address

and click **Send test email** to confirm end-to-end delivery. Changes take effect within 30 seconds.

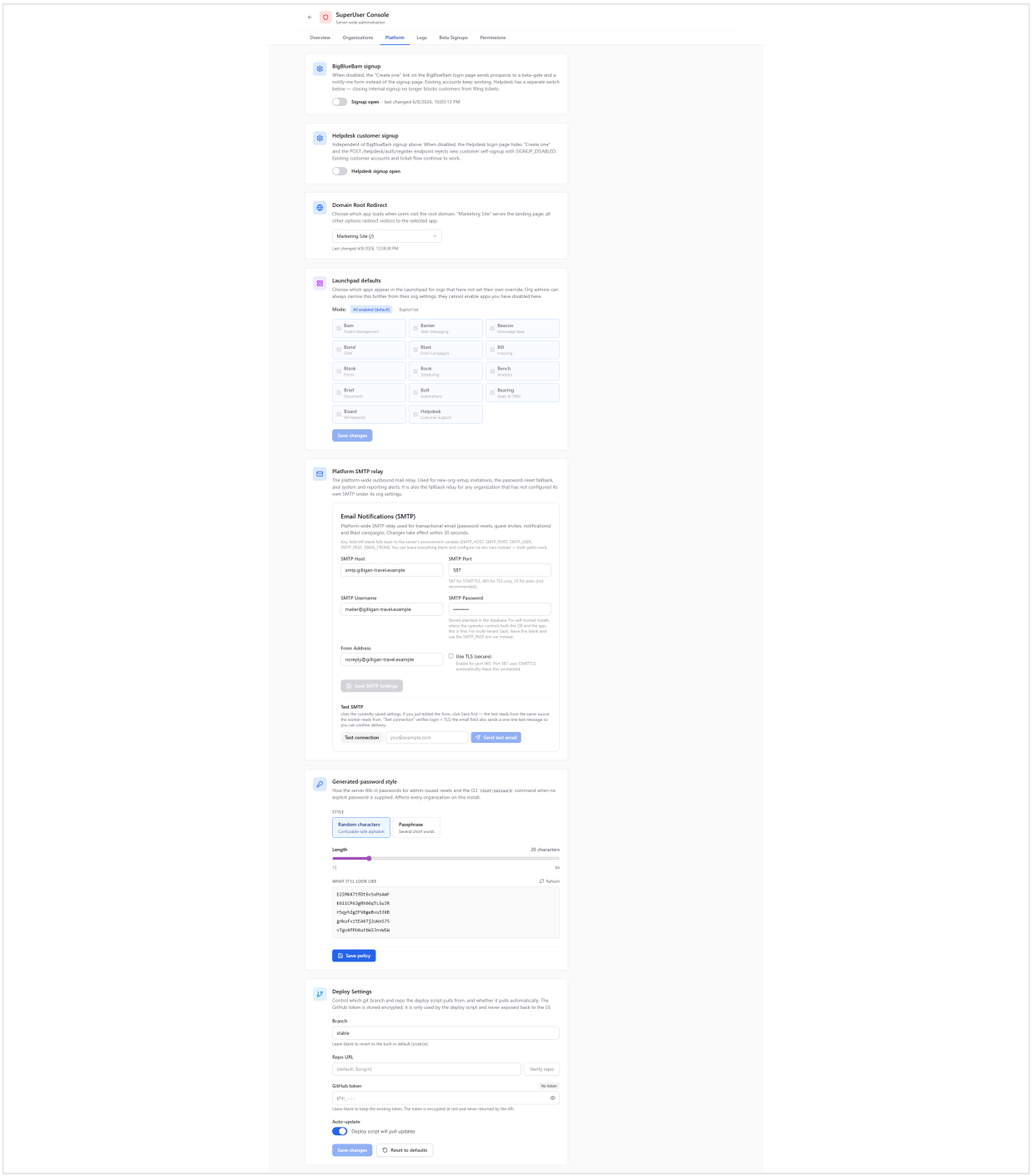


Figure 2.7 *Platform email (SMTP)*

Organization email (SMTP). An org admin or owner can give their organization its own relay, which overrides the platform relay for that org's outbound mail (Blast campaigns and member or guest invitations). Leaving it blank keeps the org on the platform relay.

To configure the org override, open **/settings**, select the **Integrations** tab, and scroll to the **Organization Email (SMTP)** card. Fill in the same fields and save; the card confirms whether the org is using its own relay or falling back to the platform relay. The relay password is masked on read, so leaving the dots in place keeps the stored password unchanged. **Test relay** checks whichever relay will actually send (your org override if set, otherwise the platform fallback). To stop using the org relay, click **Use platform relay**, which clears the override. Changes take effect within 30 seconds.

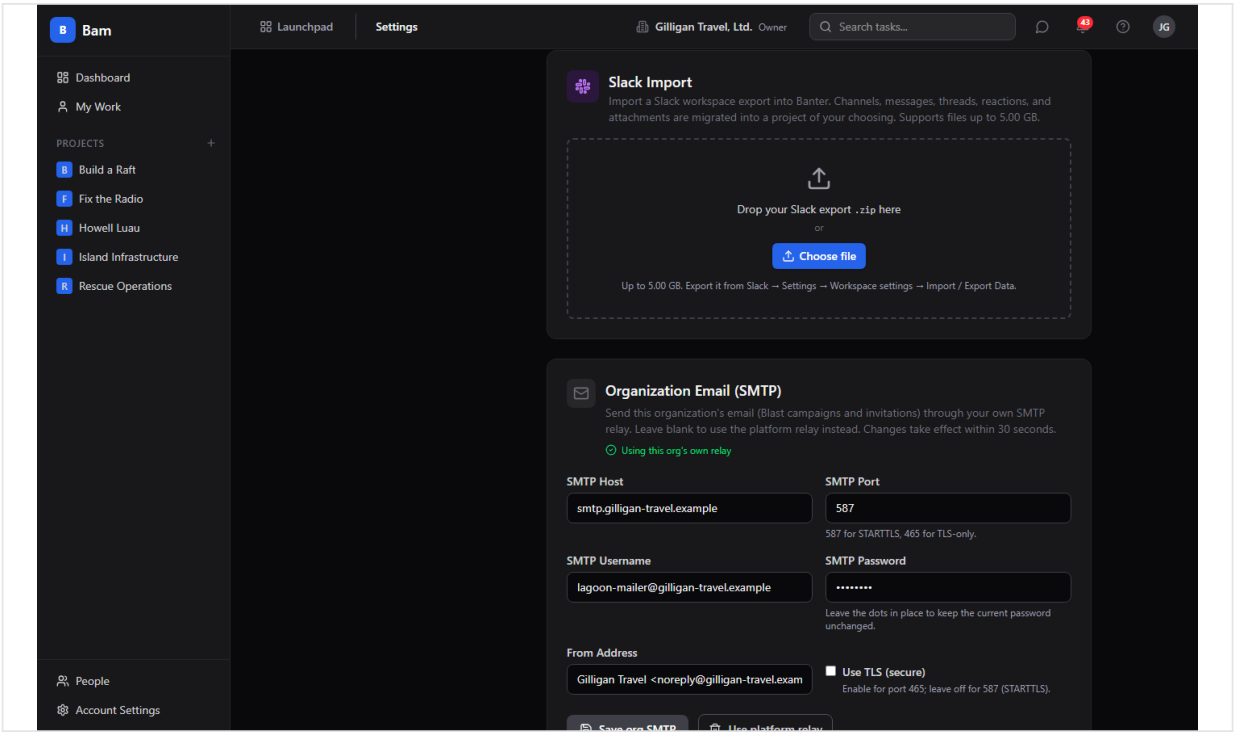


Figure 2.8 *Organization email (SMTP)*

Command palette and keyboard shortcuts

Bam has power-user navigation built in.

Press **Ctrl/Cmd+K** to open the command palette. Other shortcuts: **N** creates a task, **S** or **/** focuses search, **F** toggles the filter, **Esc** closes the current overlay, and **?** opens the **Keyboard Shortcuts** overlay and Help.

Notifications

Bam delivers in-app notifications.

To review notifications, open your notifications list; you can mark a single notification read, or mark all read.

Cross-app integrations

Bam connects to other apps in the suite:

- **Helpdesk** - Tasks created from a Helpdesk ticket show a **Helpdesk** tab in the drawer linked to the source ticket.
- **GitHub** - Commits and pull requests can link to a task and appear in the drawer's linked section.
- **Slack** - Starting and completing a sprint, and task events, can post to Slack.
- **Banter** - Use **Share to Banter** from the task drawer to post a task into a channel. Banter messages that reference a task's human id deep-link back into Bam.
- **Bolt** - Task, sprint, epic, and comment changes emit events to Bolt.
- **Launchpad** - Cross-app navigation links Bam to the rest of the suite.

Working with AI agents

Agents drive Bam through its MCP tools, which accept natural identifiers (project, phase, sprint, and label names, a user's email, or a human id like FRND-42) as well as UUIDs. The catalog spans nine Bam tool files and lets an agent do nearly everything a human can on the board:

- **Tasks** - `search_tasks`, `get_task`, `bam_get_task_by_human_id`, `create_task`, `update_task`, `move_task`, `delete_task`, `bulk_update_tasks`, `log_time`, `duplicate_task`, `import_csv`, `task_upsert_by_external_id`, `bam_add_task_parent`, `bam_remove_task_parent`, `bam_list_task_parents`, `bam_list_task_subtasks`.
- **Sprints** - `list_sprints`, `create_sprint`, `start_sprint`, `complete_sprint`, `get_sprint_report`.
- **Projects** - `list_projects`, `get_project`, `create_project`, `test_slack_webhook`, `disconnect_github_integration`.
- **Epics** - `bam_create_epic`, `bam_update_epic`, `bam_get_epic`, `bam_list_epics`.
- **Members and users** - `list_members`, `get_my_tasks`, `bam_find_user_by_email`, `bam_find_user`, `bam_invite_member`, `bam_admin_reset_password`, `bam_send_password_reset_link`.
- **Reports** - `get_velocity_report`, `get_burndown`, `get_cumulative_flow`, `get_overdue_tasks`, `get_workload`, `get_status_distribution`, `get_cycle_time_report`, `get_time_tracking_report`.
- **Templates** - `list_templates`, `create_from_template`.
- **Comments** - `list_comments`, `add_comment`.
- **Import** - `import_github_issues`, `suggest_branch_name`, `bam_import_csv`.

Resolver helpers (`bam_list_labels`, `bam_list_phases`, `bam_list_states`) and identity tools (`get_me`, `update_me`, `list_my_orgs`, `switch_active_org`) round out the Bam-specific catalog.

On top of the Bam tools, agents reach Bam through the cross-cutting agentic platform that every app shares:

- **Identity, audit, heartbeat** - An agent identifies itself and reports liveness with `agent_heartbeat`, reviews its own recent actions with `agent_audit`, and describes its

capabilities with `agent_self_report`. Every write an agent makes is recorded with an `agent` actor type in the activity log.

- **Approval queues** - For work that needs a human sign-off, an agent files a proposal with `proposal_create`, a human or another agent lists pending items with `proposal_list`, and a decision is recorded with `proposal_decide`.
- **Cross-app read plane** - `search_everything` fans a query out across apps (Bam tasks included) with normalized scoring, `resolve_references` turns mentions like FRND-42 into resolved entities, and `activity_query` reads the unified cross-app activity view. Composite views `project_view` and `account_view` assemble a rounded picture of a project or account spanning multiple apps.
- **Idempotent upserts** - `task_upsert_by_external_id` keys on the project plus an external id and emits a `task.upserted` event with a `created` flag, so an agent can sync tasks from an external system without creating duplicates.
- **Visibility preflight** - Before an agent posts a Bam task into a shared surface, it calls `can_access(asker_user_id, entity_type, entity_id)` and drops anything the asker is not allowed to see.
- **Agent policies and webhooks** - A per-agent kill switch and tool allowlist (`bam.*`, `task.*`, and so on) gate every service-account call, and subscribed Bolt events are pushed to agent runners via signed outbound webhooks.

Destructive tools require human review. `delete_task` and `disconnect_github_integration` use the two-step `confirm_action` token flow: the tool stages the action and returns a time-limited token, and a second call confirms it. When reviewing agent work, watch the project audit log and the task **Activity** tab; both record the agent actor. See `docs/apps/bam/mcp-tools.md` for the full tool catalog.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. When you open a task, a presence strip shows exactly who else is on it, and you can ring a teammate into a voice or video huddle straight from the card. Your location in Bam shows in the Bureau office, so the team can find you. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Spin up a project from a template

Who: A team lead or org admin starting a new effort. **Goal:** Create a project that arrives with phases and states already set up. **Before you start:** A session with permission to create projects.

Steps

1. In the sidebar, click **Dashboard**, or click the plus next to **Projects**.
2. Click **New Project** (or **Create Project** in the empty state).
3. Enter a project **name** and a **task id prefix** of 2 to 6 uppercase letters.
4. Choose a project template (for example kanban_standard, scrum, bug_tracking, or minimal) and submit.

Result: You land on the new project's board. Its phases and task states are seeded from the template (for example kanban_standard gives Backlog, To Do, In Progress, Review, Done).

Related: Manage Phases and Custom Fields to shape the board further. Agents use `create_project`.

Story: Create and work a task end to end

Who: A team member doing the day-to-day work. **Goal:** Take a task from creation to done with full detail along the way. **Before you start:** Membership in a project.

Steps

1. Press **N** or use a column's add control to open the create dialog.
2. Fill in **Title**, then optionally **Phase**, **Priority**, **Story Points**, **Assignee** (any org member), **Due Date**, and **Labels**, and click **Create Task**.
3. Drag the card across phases as work progresses.
4. Open the task, log time in **Time Logged** with **Log Time**, write a note on the **Comments** tab, react to a teammate's comment, and upload a file under **Attachments**.
5. When finished, set a closed state. If the task still has open subtasks, the move is blocked until you finish them.

Result: The task moves to a closed state and counts toward sprint velocity, with its time, discussion, and files attached.

Related: Subtasks, the done-gate, and `create_task` / `update_task` / `log_time` for agents.

Story: Track your personal load

Who: Anyone with assigned tasks. **Goal:** See what you owe and what is overdue across all projects. **Before you start:** Tasks assigned to you.

Steps

1. Click **My Work** in the sidebar.
2. Scan the **Overdue**, **Due This Week**, **In Progress**, and **All My Tasks** groups. Only tasks assigned to you appear here.
3. Click any task to jump to its project board and act on it.

Result: You have a single, cross-project view of your own work, with completed tasks hidden.

Related: Agents use `get_my_tasks`.

Story: Plan and run a sprint

Who: A team lead running iterations. **Goal:** Plan a sprint, work it, and close it cleanly. **Before you start:** A project with tasks; no other active sprint.

Steps

1. In the board's sprint selector, choose **Create sprint** and set a name, start and end dates, and an optional goal.
2. Add tasks to the sprint (set each task's **Sprint** in its drawer, or filter to the sprint).
3. Choose **Start this sprint** to make it active.
4. Work the board through the sprint.
5. Choose **Complete Sprint**, decide each leftover task's fate, add **Retrospective Notes (optional)**, and click **Complete Sprint**.
6. Review velocity with **View sprint report**.

Result: The sprint is completed, leftover tasks are routed, and velocity is recorded.

Related: Carry-forward; `create_sprint`, `start_sprint`, `complete_sprint`, `get_sprint_report`.

Story: Carry forward unfinished work

Who: A team lead closing a sprint with leftover tasks. **Goal:** Move unfinished tasks to the next sprint without losing history. **Before you start:** An active sprint with incomplete tasks.

Steps

1. Choose **Complete Sprint** on the active sprint.
2. For each incomplete task, set the dropdown to **Carry forward**.
3. Pick the target sprint and click **Complete Sprint**.

Result: The carried tasks move to the target sprint with a carry-forward badge, and each task's original sprint is preserved for reporting.

Related: The Plan and run a sprint story; `complete_sprint`.

Story: Group work under an epic

Who: A planner organizing a larger initiative. **Goal:** Track related tasks across sprints under one epic. **Before you start:** A project with tasks.

Steps

1. Click **Manage epics** in the board header and create an epic with a name and color.
2. Open each task and set its **Epic** in the drawer sidebar.
3. Open the epic's detail page to see its tasks grouped by sprint and a burnup chart.

Result: Tasks are grouped under the epic, and the epic page shows progress rollups.

Related: Agents use `bam_create_epic`, `bam_update_epic`, `bam_get_epic`.

Story: Customize the board

Who: A project admin tailoring the workflow. **Goal:** Shape phases, fields, and labels to fit how the team works. **Before you start:** Project admin role.

Steps

1. Open **Project options** and choose **Manage Phases** to add, reorder, and configure phases (WIP limits, colors, auto-state on entry).
2. Open **Project options** and choose **Custom Fields** to add a field (for example a select or url field) and mark it shown on the card.
3. Add and color the labels the team needs.

Result: The board reflects the team's workflow, with the right columns, fields, and labels.

Related: Phases, Custom fields, Labels.

Story: Slice the board with views

Who: Anyone who wants a focused cut of the board. **Goal:** Filter and group the board, then save the cut for reuse. **Before you start:** A project with tasks.

Steps

1. In the view switcher, pick **Board**, **List**, **Timeline**, **Calendar**, or **Workload**.
2. Apply swimlanes (**By Assignee**, **By Priority**, or **By Epic**) and set filters.
3. Click **Saved views** and save the configuration, optionally sharing it with the project.

Result: Your filtered, grouped view is saved and reusable. (Workload is a view mode you can switch to but cannot save as a view type.)

Related: Saved views, Alternate views. Tasks need start and due dates for the Timeline and Calendar views to place them.

Story: Import an existing backlog

Who: Someone migrating work into Bam. **Goal:** Bring tasks from a CSV or another tool into a project. **Before you start:** Project membership and a source file or connected tool.

Steps

1. Click **Import tasks** in the board header.
2. Choose CSV, Trello, Jira, or GitHub Issues.

3. For CSV, map columns, set value, link, and custom-field maps, choose a duplicate strategy (create, skip, or update), and run the preview.
4. Confirm the import.

Result: Tasks land in the project, with unmatched phases and labels created automatically.

Related: Agents use `import_csv` / `bam_import_csv` / `import_github_issues`.

Story: Report on delivery

Who: A lead or stakeholder checking progress. **Goal:** Read the delivery metrics for a project. **Before you start:** A project with sprint and task history.

Steps

1. Open the project dashboard at `/projects/:id/dashboard`, or its reports at `/projects/:id/reports`.
2. Review velocity, burndown, cumulative flow, cycle time, overdue tasks, workload, time tracking, and status distribution.
3. Scope burndown to a specific sprint or the active sprint as needed.

Result: You have the metrics to judge pace and risk.

Related: Agents use `get_velocity_report`, `get_burndown`, `get_cumulative_flow`, `get_overdue_tasks`, `get_workload`, `get_status_distribution`, `get_cycle_time_report`, `get_time_tracking_report`.

Story: Subscribe to a project calendar

Who: Anyone who wants due dates in their calendar app. **Goal:** See task due dates in an external calendar. **Before you start:** Access to the project and a calendar token.

Steps

1. Generate a calendar token for the project (or use your personal feed).
2. Copy the feed URL (`/projects/:id/calendar.ics` or `/me/calendar.ics`) with the token in the query string.
3. Add the feed to your external calendar app as a subscribed calendar.

Result: Tasks with due dates appear in your calendar and refresh on the app's schedule. Revoke the token any time to cut access.

Related: iCal feed.

Story: Deep-link a task across apps

Who: Anyone who shares a task reference in another app. **Goal:** Open a task directly from a reference like MAGE-38. **Before you start:** Access to the project that owns the task.

Steps

1. From a Banter message or email that references a task (for example MAGE-38), open `/b3/tasks/ref/MAGE-38`.

2. Bam resolves the reference to the task.

Result: The project board opens with that task's drawer open.

Related: Share to Banter; the `bam_get_task_by_human_id` and `resolve_references` tools.

Story: Sync tasks from an external system with an agent

Who: An AI agent keeping a Bam project in step with an external tracker. **Goal:** Create or update tasks idempotently without making duplicates. **Before you start:** A service-account API key whose agent policy allows the Bam task tools, and an external id per source record.

Steps

1. The agent resolves the project by name or id and confirms write access.
2. For each external record, the agent calls `task_upsert_by_external_id` with the project, the external id, and the task fields.
3. Bam creates the task on first sight and updates it on subsequent calls, emitting a `task.upserted` event with a `created` flag each time.
4. For any task the agent intends to surface in a shared channel, it first calls `can_access` and drops entities the asker may not see.

Result: The project stays mirrored to the external system with no duplicate tasks, and every change is recorded under the agent actor in the audit log.

Related: The Working with AI agents section; `task_upsert_by_external_id`, `can_access`, `agent_audit`.

Related

- **Helpdesk** (`/helpdesk/`) - Tickets can create Bam tasks; a created task shows a Helpdesk tab linking to its source ticket.
- **Banter** (`/banter/`) - Share tasks to channels and deep-link back via human ids.
- **Bolt** (`/bolt/`) - Receives task, sprint, epic, and comment events from Bam.
- **Bench** (`/bench/`) and the project reports - Delivery metrics and dashboards.
- Bam MCP-tools reference and guide in `docs/apps/bam/` (`mcp-tools.md`, `guide.md`). Treat the code as the source of truth where the reference lags.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What is the maximum length of a project's task id prefix?

a) 2 letters
b) 6 letters
c) 8 letters
d) There is no limit

2. How many sprints can be active in a single project at the same time?

a) Unlimited
b) One
c) Two
d) One per member

3. When you close an active sprint with unfinished tasks, what three things can happen to each incomplete task?

4. What prevents a task from moving into a closed state?

a) A WIP limit on the phase
b) Open subtasks
c) A missing assignee
d) An active sprint

5. What priority does a new task get by default, and how many custom field types does Bam support?

6. Which of these is NOT one of Bam's saved view types?

a) Board
b) List
c) Timeline
d) Mind map

7. Which roles exist at the organization level?

a) owner, admin, and member, plus a guest role
b) admin and member only
c) owner and viewer only
d) There are no roles at the org level

8. A task state is orthogonal to its phase and belongs to one of six categories. Name them.

9. What are the three project roles?

a) admin, member, and viewer
b) owner, admin, and member
c) lead, contributor, and guest
d) admin and member

10. What are the four swimlane options for grouping the board?

11. How do you manually add a watcher to a task in the Bam UI?

a) From the task detail drawer
b) Through the command palette

c) You cannot; watchers are populated by the system as people get involved with the task
d) By assigning the task to them

12. An epic can hold one of three statuses, and it rolls up two metrics across its tasks. What are they?

CHAPTER 3

Banter

Where the conversation lives.

Banter is the chat layer of BigBlueBam: channels, direct messages, threads, reactions, pins, and bookmarks, all sitting right next to the rest of the suite. It shares your Bam identity and your org's people, so there is no second login and no second roster to manage. This chapter walks you from your first visit to #general through posting, threading, searching, and the admin policies that govern who can do what, and it shows where an AI agent can post, schedule, and listen on the same channels your team uses. By the end you will be able to set up channels, run a conversation, and find any message fast.

At a glance

- Channels, DMs, and threads that keep side conversations out of the main timeline
- Rich messages up to 40,000 characters with mentions, reactions, pins, and bookmarks
- Full-text search across every channel you can see, with author, date, and attachment filters
- Org-level policy for who can create channels, retention, file size, and @mention groups
- MCP access, so an agent can post, schedule, DM, and listen for patterns on a channel

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Send a message . Reply in a thread . React to a message . Pin a message . Bookmark a message . Edit or delete your message

> *Banter is the chat layer of BigBlueBam. It gives your team channels, direct messages, threads, reactions, pins, and bookmarks so conversations live alongside the rest of the suite. Reach for it when work needs a back-and-forth instead of a ticket or a task.*

Overview

Banter is a real-time messaging app shared by everyone in your organization. You talk in **channels** (open rooms anyone can join, or private rooms by invitation), in **direct messages** (one-to-one or small groups), and in **threads** that hang off a single message[16] so a side conversation does not flood the main timeline.

> ***Threads hang off a single message, so a side conversation stays focused instead of drowning the channel.***

Banter does not have its own login or its own user list. You sign in to BigBlueBam (Bam) first, and Banter uses that same identity and the same people. Everyone you can message is already a member of your org. If you open Banter while signed out, you see "Please log in to BigBlueBam first to access Banter" with a link back to the main app.

> **NOTE**
> Banter rides on your BigBlueBam identity. Everyone you can message is already a member of your org, and signing out of Bam signs you out of Banter too.

Posting, threading, reacting, pinning, bookmarking, mentions, DMs, search, attachments, and scheduled posts driven by automation or AI agents are all working.

Live audio is **not** started from inside Banter. The voice and video write actions were retired; every call write endpoint returns HTTP 410 Gone, and live audio for a channel[17] now happens in the suite-wide Bureau docked box (which joins a shared room derived from the channel). Banter keeps a read-only view of past calls and their transcripts, but you do not start or join a call from inside Banter. See "Past calls and audio" below.

Key concepts

- **Channel** - a named conversation space. Channels are `public` (anyone in the org can find and join), `private` (invite only), or a DM type (`dm` / `group_dm`). Each channel has a name, an optional topic and description, members with roles, and a slug that is unique within your org.
- **#general** - the default channel. The first time anyone opens Banter in an org that has no channels, Banter creates `#general`, marks it the default, and auto-joins every active member (the creator becomes owner). You cannot delete or leave the default channel.

[16] **Message.** a single post. Messages support rich text (bold, italic, code, links), @mentions, and emoji, up to 40,000 characters. You can edit or delete your own messages.

[17] **Channel.** a named conversation space. Channels are `public` (anyone in the org can find and join), `private` (invite only), or a DM type (`dm` / `group_dm`). Each channel has a name, an optional topic and description, members with roles, and a slug that is unique within your org.

- **Channel role**[18] - your standing inside one channel: `owner`, `admin`, `member`, or `viewer`. A **viewer** is read-only and cannot post. Channel admins manage pins, members, and settings.
- **Direct message (DM)** - a private conversation with one other person. A **group DM** is a private conversation with 3 to 8 people total.
- **Message** - a single post. Messages support rich text (bold, italic, code, links), @mentions, and emoji, up to 40,000 characters. You can edit or delete your own messages.
- **Thread**[19] - replies attached to a parent message. A reply stays in the thread unless you tick **Also send to channel**, which also mirrors it into the main timeline.
- **Reaction**[20] - an emoji you toggle on a message. Click once to add, click again to remove.
- **Pin**[21] - a channel-wide marker that any member can see in the channel's pinned list. Only channel admins can pin or unpin.
- **Bookmark**[22] - your own private save of a message, with an optional note. Bookmarks are visible only to you and live on the Bookmarks page.
- **Mention**[23] - typing `@` plus a name notifies that person. Org admins can also define @mention user groups (for example `@engineering`) that notify everyone in the group.
- **Presence**[24] - your live status: online, idle, in a call, do not disturb, or offline.
- **Scheduled message**[25] - a message queued to post later at a set time, or deferred automatically when a channel is inside its quiet hours[26].
- **Quiet hours** - a per-channel policy that holds non-urgent posts until the channel's allowed hours. Used mostly by automated and agent posts.
- **Agent pattern subscription**[27] - a rule that lets an agent or service account listen to a channel and react when messages match a pattern.

---

[18] **Channel role.** your standing inside one channel: `owner`, `admin`, `member`, or `viewer`. A **viewer** is read-only and cannot post. Channel admins manage pins, members, and settings.

[19] **Thread.** replies attached to a parent message. A reply stays in the thread unless you tick **Also send to channel**, which also mirrors it into the main timeline.

[20] **Reaction.** an emoji you toggle on a message. Click once to add, click again to remove.

[21] **Pin.** a channel-wide marker that any member can see in the channel's pinned list. Only channel admins can pin or unpin.

[22] **Bookmark.** your own private save of a message, with an optional note. Bookmarks are visible only to you and live on the Bookmarks page.

[23] **Mention.** typing `@` plus a name notifies that person. Org admins can also define @mention user groups (for example `@engineering`) that notify everyone in the group.

[24] **Presence.** your live status: online, idle, in a call, do not disturb, or offline.

[25] **Scheduled message.** a message queued to post later at a set time, or deferred automatically when a channel is inside its quiet hours.

[26] **Quiet hours.** a per-channel policy that holds non-urgent posts until the channel's allowed hours. Used mostly by automated and agent posts.

[27] **Agent pattern subscription.** a rule that lets an agent or service account listen to a channel and react when messages match a pattern.

Where to find it

Banter is served at `/banter/`. Reach it from the Launchpad app switcher in the top-left of any BigBlueBam app. The default landing channel is `#general` (`/banter/channels/general`); a bare `/banter/` visit redirects there.

> **TRY IT**
> Open the Launchpad app switcher in the top-left, choose Banter, and land in #general; type a message and press Enter to post your first line.

Prerequisites:

- You must be signed in to BigBlueBam. Banter has no separate login.
- You must belong to an organization. Everyone you can message is a member of that org.
- To create channels, your org's policy must allow it for your role (set under **Banter Administration**).
- To manage org settings, channel groups, @mention groups, or a Slack import, you need org admin or owner access.

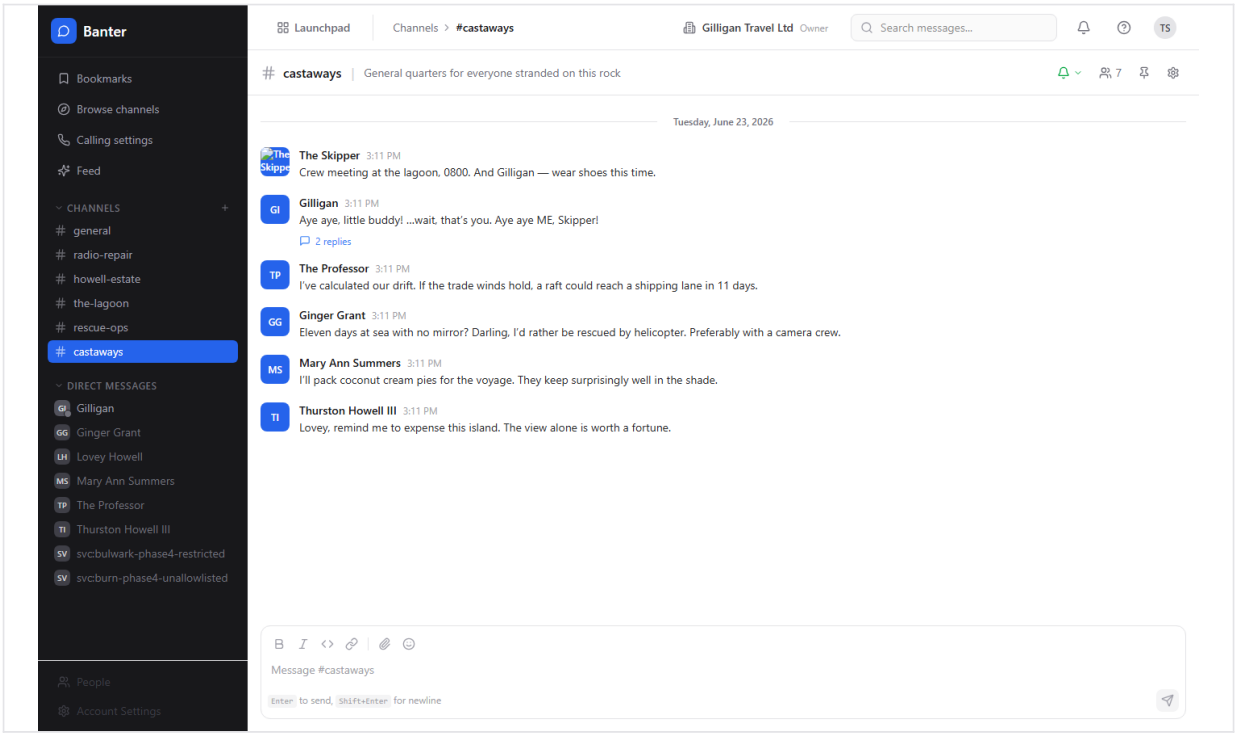


Figure 3.1 Channels & sidebar

Feature reference

Send a message

The compose box sits at the bottom of every channel and DM.

To post a message:

1. Open a channel from the sidebar or a DM from the **Direct Messages** section.
2. Click the text box that reads `Message #<channel>` and type.
3. Use the toolbar above the box for formatting: **Bold**, **Italic**, **Code**, **Link**, **Attach file**, and **Emoji**. The emoji picker offers a palette of 24 common emoji.
4. Type `@` to mention someone; pick a name from the autocomplete list.
5. Press **Enter** to send. Use **Shift+Enter** for a new line. The hint under the box reads "Enter to send, Shift+Enter for newline".

Your draft is saved per channel, so switching channels does not lose what you were typing.

> **TIP**
> Because drafts are kept per channel, you can start a reply in one room, jump to another to grab a detail, and come back to find your half-written message still waiting.

To attach a file: click **Attach file**, pick one or more files, and they upload to storage. While the upload runs you see "Uploading…". Once a file is selected, the box shows "N files attached" with each filename and a **Remove all** link. The file is linked to the message when you send it, so the recipients see the attachment in the channel. You can send a message that is only an attachment with no text.

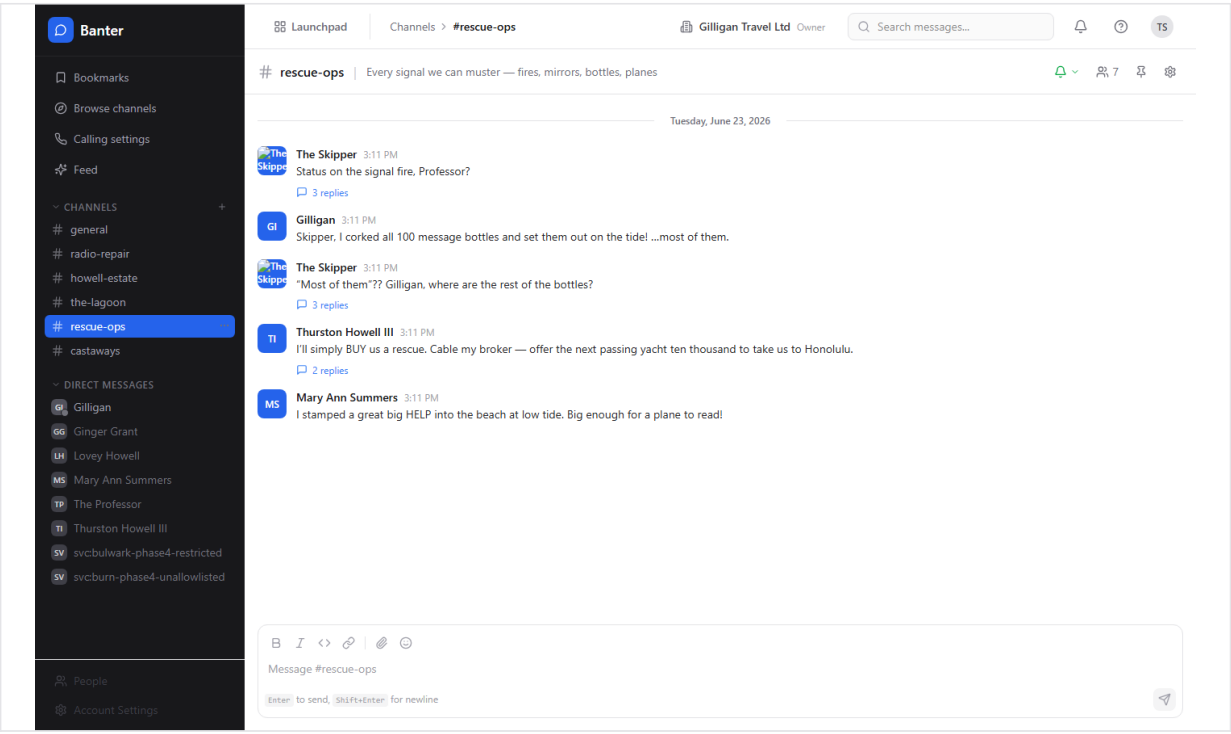


Figure 3.2 Channel conversation

Reply in a thread

Threads keep a focused discussion attached to one message.

To reply in a thread:

1. Hover over the message you want to reply to.
2. In the hover bar, click **Reply in thread**. The right-hand **Thread** panel opens.
3. Type your reply in the box labeled "Reply...".
4. To also post your reply into the main channel timeline, tick **Also send to channel** before sending.
5. Press **Enter** or click the send button.

The parent message shows a "N reply" / "N replies" link that reopens the thread. Replying notifies the parent author and prior repliers.

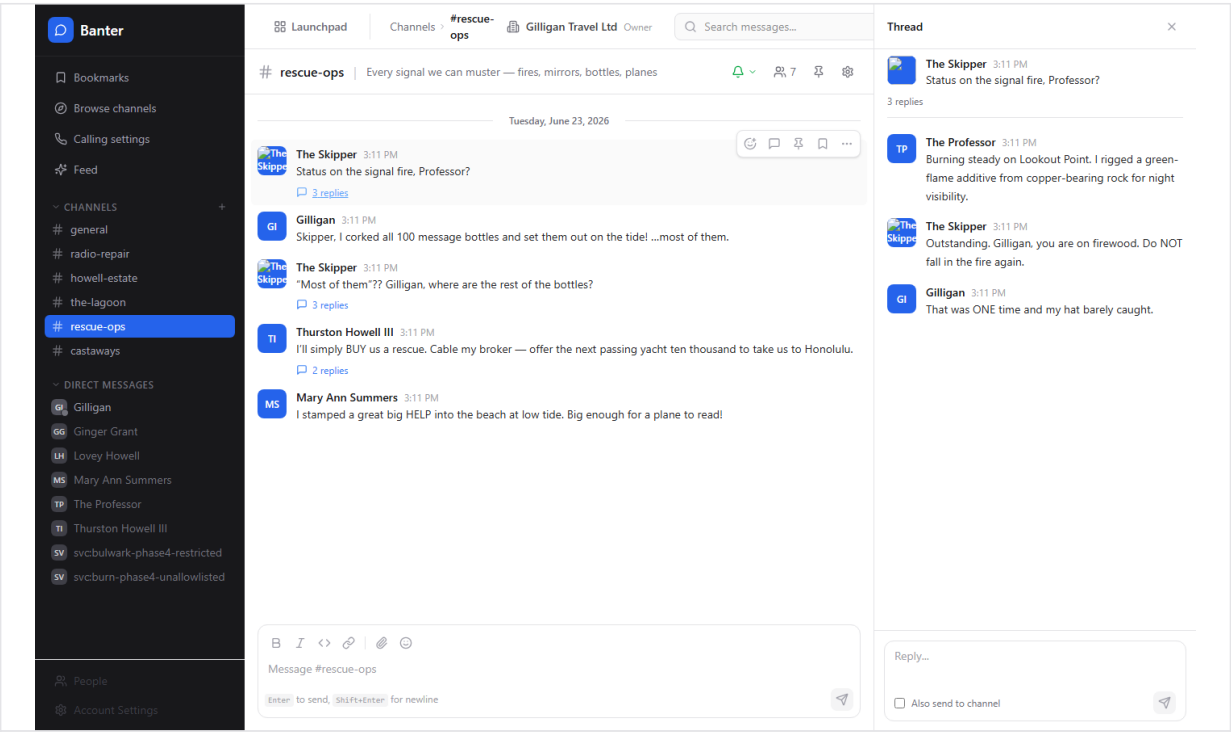


Figure 3.3 *Threaded replies*

React to a message

To add or remove a reaction:

1. Hover over a message and click **Add reaction**.
2. Choose an emoji. The quick reactions are thumbs up, heart, laughing, party, eyes, and rocket.
3. Click the same emoji badge again to remove your reaction.

Reaction badges under a message show the emoji and a running count.

Pin a message

Pins are channel-wide and visible to every member. Only channel admins can pin.

To pin or unpin:

1. Hover over the message and click **Pin message** (or open the more menu and choose **Pin to channel**).
2. To remove it, hover and click **Unpin message** (or **Unpin from channel** in the more menu).
3. View the channel's pins with the **Pinned messages** button in the channel header.

If you are not a channel admin, pinning fails with "Couldn't pin - channel admins only".

Bookmark a message

Bookmarks are private to you, with an optional note.

To bookmark:

1. Hover over a message and click **Bookmark** (or choose **Bookmark** in the more menu).
2. To remove it, click **Remove bookmark**.
3. Open all your saved messages from the **Bookmarks** quick action in the sidebar.

On the **Bookmarks** page each row links to the channel and shows the author and a preview. Use **Go to message** to jump there or **Remove bookmark** to clear it. When empty, the page reads "No bookmarks yet".

Edit or delete your message

To edit:

1. Hover over your own message and open the more menu.
2. Click **Edit message**, change the text, and click **Save** (or press Enter). Click **Cancel** or press Esc to discard.

Edited messages show "(edited)". A message may be locked by its author or thread starter; a lock icon and the note "This message is locked - nobody may edit it." indicate you cannot edit it. Org admins and owners can always edit.

To delete:

1. Open the more menu on your own message.
2. Click **Delete message**.

Deletes are soft (the message is removed from view). Channel and org admins can delete others' messages.

Create a channel

To create a single channel:

1. In the sidebar, find the **Channels** section and click the + button.
2. In the inline **New Channel** box, type a name in the `channel-name` field (lowercase letters, numbers, and hyphens, up to 80 characters).
3. Click **Create**. You are taken to the new channel.

Whether you can create channels depends on your org policy ("Who can create channels"). The server enforces that policy on every create: with **Everyone** any member can create, with **Admins only** only org admins and owners can, and with **Organization owners only** only owners can. Channel creation is rate-limited to 5 per hour.

To add many channels at once (org admins):

1. Right-click the + button. The tooltip reads "Create channel - right-click to add many".
2. In the **Add many channels** dialog, paste one channel name per line. The dialog notes "Paste one channel name per line. Up to 50 at a time." and summarizes how many are valid, duplicate, or invalid.
3. Choose **Public** or **Private**.

4. Click **Create N channels**. Each row reports back as created, duplicate, invalid, or error. If any rows are left, **Create remaining** continues.

Note: the **Add many channels** dialog uses a coarser permission gate than the single + create. It admits any org admin or owner (and any member when your org's policy allows members to create channels) but does not apply the **Organization owners only** distinction. If your org is set to owners-only, create those channels one at a time with the + button to be sure the policy is honored, or have an owner run the bulk dialog.

Browse and join channels

To find and join open channels:

1. Click **Browse channels** in the sidebar. The **Browse Channels** page opens.
2. Type in the `Search channels...` box to filter.
3. Each result shows the channel name, a **Private** pill if applicable, the topic, the member count ("N members"), and when it was last active ("Last active ...").
4. Click **Join** on a channel you want. Channels you already belong to show **Joined**.

When nothing matches you see "No channels found" and "Try a different search term". Joining is only allowed on public channels; private channels require an invitation.

Start a direct message or group DM

To start a one-to-one DM:

1. In the sidebar, open the **Direct Messages** section.
2. Click a team member listed there. A DM opens (or reuses an existing one).

A group DM holds 3 to 8 people total and must be allowed by your org's group-DM setting. If the people list fails to load, the sidebar shows "Couldn't load people - tap to retry"; with no teammates it shows "No team members found".

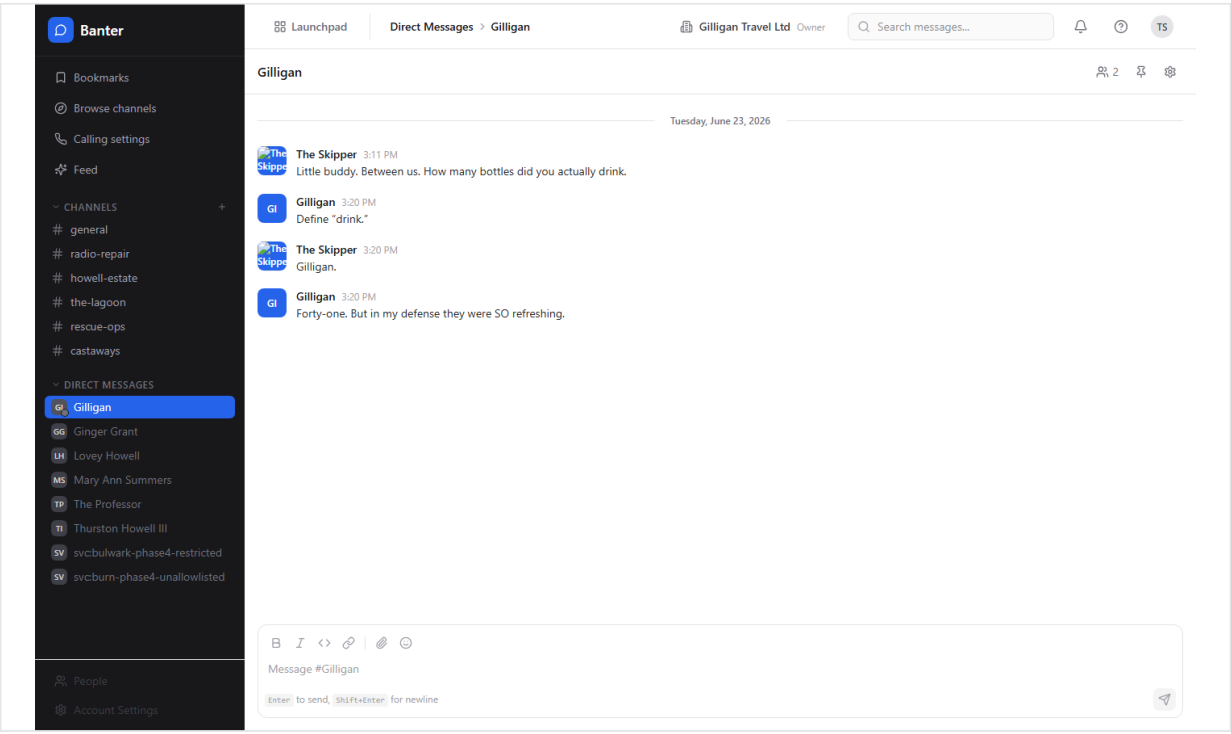


Figure 3.4 Direct messages

Configure a channel

Channel admins manage a channel from its settings.

To change channel settings:

1. In the channel header, click the **Channel settings** button (gear icon). The **Channel Settings** modal opens.
2. Edit **Channel name**, **Topic**, or **Description**.
3. Toggle **Allow bots** and **Allow huddles** as needed.
4. Click **Save Changes**. A "Saved!" confirmation appears.
5. Under **Members (N)**, review the member list.

To add a member from this modal, type an email or username into the **Email or username** field and click **Add**. The server resolves the identifier to a person in your org (exact email first, then a handle built from the display name) and adds them. If no active member matches, you get a "No active user in this organization matches ..." error.

To archive (delete) a channel:

1. Open the channel's **Channel Settings** modal, or use the per-channel hover menu in the sidebar.
2. In the **Danger Zone** (shown only for non-default channels), click **Delete Channel**.
3. Confirm at the prompt "Are you sure you want to delete #<name>? This will archive the channel and all its messages. This action cannot be undone." by clicking **Yes, delete channel**.

> **WARNING**
> Deleting a channel archives it along with every message in it, and there is no undo. The default channel is the exception: it cannot be deleted or left.

Deleting a channel archives it; it disappears from the active list. The default channel cannot be deleted or left.

Rename, leave, or delete from the sidebar

Hover a channel in the sidebar to reveal its menu:

- **Rename** - rename in place.
- **Channel settings** - open the settings modal.
- **Leave channel** - leave (hidden for the default channel). The last remaining owner cannot leave while other members stay; you see "LAST_OWNER_CANNOT_LEAVE" if you try.
- **Delete channel** - archive it; click again at "Click again to confirm".

Mark a channel read

Banter tracks your read position automatically. Opening a channel clears its unread badge, and your read cursor syncs across devices, so a channel you read on your laptop is not flagged unread on your phone. Unread channels show a dot, and channels with unread mentions show a count badge. The read cursor is also broadcast (`channel.read_cursor_synced`) so open tabs update live.

Search messages

The **Search** page (reached from the header search box or the sidebar) has a query box (`Search messages...`) and **Filters** for **Channel** (`All channels`), **Author** (`Anyone`), **From date**, **To date**, and **Has attachments**. Use **Clear all filters** to reset.

To search:

1. Open Search from the header search box or the sidebar.
2. Type at least two characters in the `Search messages...` box and press Enter.
3. Optionally click **Filters** and narrow by **Channel**, **Author**, **From date**, **To date**, or **Has attachments**.
4. Read the result rows; each shows the channel, author, time, and a snippet with the matched terms highlighted. Click a row to jump to that channel.

When nothing matches you see "No results found" and "Try different keywords". Search runs Postgres full-text search over message text and is scoped to channels you can see.

The same search is available to AI agents through the MCP tool `banter_search_messages`, and there are companion endpoints for channel search (`GET /v1/search/channels`) and call-transcript search (`GET /v1/search/transcripts`).

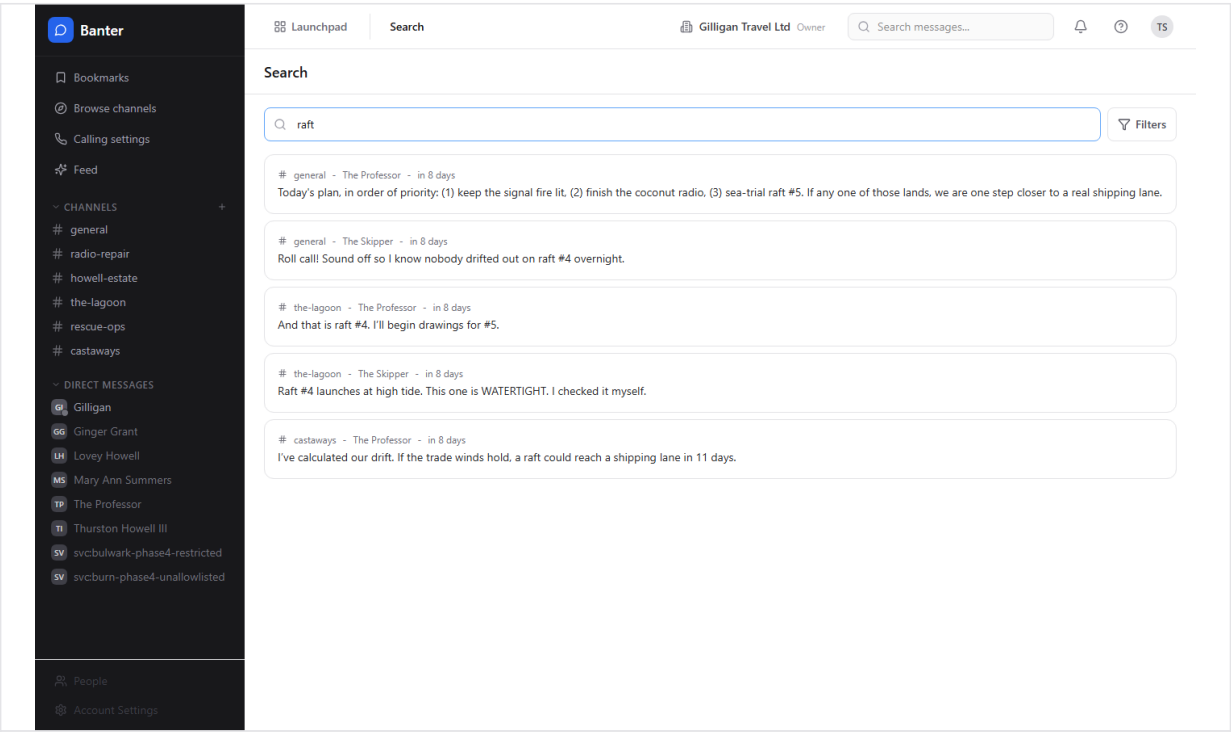


Figure 3.5 Message search

Past calls and audio

Banter no longer hosts the live call experience. Live audio for a channel happens in the suite-wide **Bureau** docked box, which joins a shared room derived from the channel (`huddle-banter-<channel_id>`), not inside Banter. There is no "start call" button in a Banter channel, and every call write endpoint in the API returns HTTP 410 Gone with `X-Deprecated-Replacement: bureau-docked-box`. The matching MCP write call tools target those retired endpoints and will fail if invoked.

What remains in Banter is read-only history:

1. Open a past call's playback page at `/banter/calls/:id`.
2. Review its type (voice, video, or huddle), duration, participants, and transcript.

Do not expect to start, join, or end a call from Banter. For live audio, use the Bureau docked box.

Keyboard shortcuts

- **Ctrl/Cmd+K** - quick channel switcher.
- **Ctrl/Cmd+Shift+M** - toggle mute on the current channel.
- **Esc** - close the open thread or the channel switcher.
- **?** - open Help.
- **Up arrow** in an empty compose box - edit your last message.

Preferences

Open the **Preferences** page from the user menu. Sections include **Profile**, **Theme** (`Light`, `Dark`, `System`), **Notifications** (`Desktop notifications`, `Notification sound`), and **Messaging** (`Enter to send`, `Show typing indicators`, `Compact mode`). Click **Save Preferences** to apply.

Your theme choice (`Light`, `Dark`, `System`) is saved locally in your browser (`bbam-theme`). The five other toggles - **Notification sound**, **Desktop notifications**, **Enter to send**, **Show typing indicators**, and **Compact mode** - are stored on the server, so they round-trip and follow you across devices.

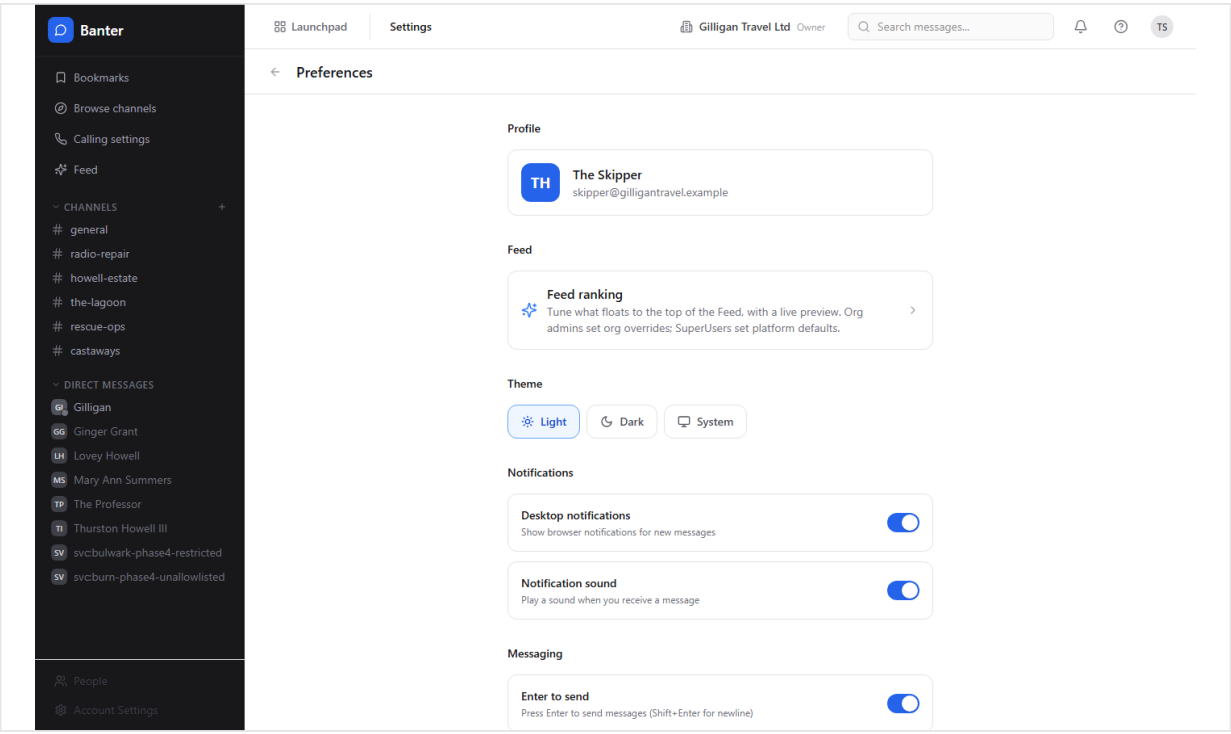


Figure 3.6 Preferences

Admin: organization policy

Org admins open **Banter Administration** from the Admin link.

To set org-wide policy:

1. Go to **Banter Administration**.
2. Under **Channel Settings**, set **Who can create channels** (`Everyone`, `Admins only`, or `Organization owners only`) and the **Default Channel**.
3. Under **Message Settings**, set message retention days and the maximum file size.
4. The **Voice & Video** section and the Voice Agent / STT / TTS / LLM block configure the external audio integration used by the Bureau audio layer (LiveKit host/key/secret, plus STT, TTS, and LLM providers). Set these only if you run that integration. **Test Connection** validates LiveKit credentials.
5. Save your changes.

All three **Who can create channels** options take effect: **Everyone** lets any member create channels, **Admins only** limits it to org admins and owners, and **Organization owners only** limits it to owners. The single + create in the sidebar enforces whichever you choose. (The bulk **Add many channels** dialog applies a coarser admin-or-owner gate; see "Create a channel".)

Admin: organize the sidebar with channel groups

Channel groups are sidebar buckets that group related channels.

To manage groups:

1. As an org admin, open the channel-groups admin area.
2. Create a group with a name, reorder groups, edit, or delete them.

Channel groups carry a name, a position, and a default collapsed state, and can optionally be tied to a project.

Admin: define @mention groups

User groups let you mention a whole team at once, for example `@engineering`.

To create one:

1. As an org admin, open the user-groups admin area.
2. Create a group with a **name**, a **handle** (the @mention text), and an optional description.
3. Add members to the group.

Mentioning the group's handle in a message notifies every member.

Admin: import from Slack

Org admins can migrate a Slack workspace export.

To run an import:

1. Open the Slack import area under admin.
2. Upload your Slack export `.zip`. Banter parses it and shows a preview of users and channels.
3. Map each user (`auto_match`, `send_invite`, create a `stub`, `map_existing` to an existing user, or `skip`) and each channel (`import_new`, `merge_existing`, or `skip`).
4. Choose options such as preserving timestamps and whether to bring over attachments, reactions, pins, and DMs. A dry-run option is available.
5. Start the import and poll its status until it completes.
6. You can abort an in-progress import and optionally clean up any stub users it created.

Working with AI agents

Agents and service accounts drive a large share of Banter activity through the MCP tool set: over 50 core Banter tools (54 as of this writing) plus 3 subscription tools. The full catalog is in

the Banter MCP-tools reference; the most common flows are below. Channel and user arguments accept a UUID, a bare name or `#name`, or an email or `@handle`, resolved server-side.

- **Post, schedule, DM, react, pin.** Agents post with `banter_post_message` (a channel can be given by `#name`), reply with `banter_reply_to_thread`, react with `banter_react`, pin with `banter_pin_message` and unpin with `banter_unpin_message`, and DM with `banter_send_dm` or `banter_send_group_dm` (each creates-or-reuses the conversation and posts in one call). Edits use `banter_edit_message`. Destructive tools (`banter_delete_channel`, `banter_delete_message`) require an explicit confirm step via the `confirm_action` flow.
- **Scheduled posts and quiet hours.** Use `banter_schedule_post` with a required `scheduled_at` to queue a message for later. If a channel has a quiet-hours policy, a normal post inside the quiet window is held and delivered later (when `defer_if_quiet` is set) or rejected with `QUIET_HOURS`. A worker delivers scheduled and deferred messages at the right time. List pending ones with `banter_list_scheduled_messages` and cancel one with `banter_cancel_scheduled_message`. Banter emits `message.scheduled` and `message.quiet_hours_deferred` events so automations can react.
- **Passive listening (pattern subscriptions).** An agent can subscribe a channel to a pattern with `banter_subscribe_pattern` (kinds: `interrogative`, `keyword`, `regex` which is admin only, and `mention`). Matches fire a `message.matched` event (on source `banter`) that the agent can act on. Manage subscriptions with `banter_unsubscribe_pattern` and `banter_list_subscriptions`. Subscriptions are gated by the channel's `agent_subscription_policy` and by org-level agent policies; a blocked subscription is recorded but marked not effective (`effective:false`) with a reason.
- **Share suite entities.** Agents can drop a Bam task or sprint, or a Helpdesk ticket, into a channel with `banter_share_task`, `banter_share_sprint`, and `banter_share_ticket`.
- **Read and resolve.** `banter_list_messages`, `banter_search_messages`, `banter_search_channels`, `banter_browse_channels`, `banter_list_channel_members`, `banter_get_unread`, `banter_mark_read`, `banter_find_user_by_email`, and `banter_find_user_by_handle` let an agent read context before acting.

> ***Scheduled posts and quiet hours mean an agent can queue a message and trust it lands at the right moment.***

These per-app tools sit on top of the cross-cutting agentic platform. Agents identify themselves and prove liveness with `agent_heartbeat`; high-impact actions can be routed through an approval queue with `proposal_create` / `proposal_list` / `proposal_decide`. Cross-app discovery uses `search_everything` and `resolve_references` (canonical mention syntax) rather than per-app search alone, and the unified activity view stitches Banter posts in with the rest of the suite. Every service-account tool call is checked against the org's `agent_policies` (per-agent kill switch plus glob allowlist, for example `banter.*`), and subscribed Bolt events can be pushed to agent runners via signed outbound webhooks.

What a human should know when reviewing agent work: agents posting into shared Banter surfaces honor per-entity visibility (they call `can_access` for each cited entity and drop anything the asker is not allowed to see), and replies route human-in-the-loop follow-ups to the thread author. Scheduled and deferred posts will appear later than the moment the agent ran, which is expected. Banter emits Bolt events on the `banter` source (`channel.created`, `message.posted`, `message.mentioned`, `message.edited`, `reaction.added`, `message.scheduled`, `message.quiet_hours_deferred`, and `message.matched`) that you can wire into automations or audit.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. Banter is the suite's real-time room: channels, DMs, and calls or huddles in the same place as the conversation. You can invite an AI agent into a call to listen and help. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Get into Banter and read #general

Who: A new team member opening Banter for the first time. **Goal:** Reach the team's main channel and start reading. **Before you start:** You are signed in to BigBlueBam and belong to an org.

Steps

1. Open the Launchpad app switcher in the top-left and choose Banter.
2. Banter loads at `#general`. If this is the very first visit for your whole org, `#general` is created automatically and everyone is added.
3. Read the timeline. Unread channels in the sidebar show a dot; mentions show a count badge.
4. Open the **Direct Messages** section in the sidebar to see who you can message.

Result: You are in `#general`, can see your channels and teammates, and your read position is tracked from here on.

Related: Create a channel; Start a direct message.

Story: Send your first message

Who: Any member. **Goal:** Post a formatted message with a mention to a channel. **Before you start:** You are a member (not a viewer) of the channel.

Steps

1. Open the channel from the sidebar.
2. Click the box that reads `Message #<channel>` and type your text.
3. Select a word and click **Bold** or **Italic** in the toolbar to format it; click **Code** for inline code or **Link** to add a URL.
4. Type `@` and pick a teammate from the autocomplete to mention them.
5. Click **Emoji** to add an emoji from the palette.
6. Press **Enter** to send (use **Shift+Enter** for a new line).

Result: Your message appears in the timeline, the mentioned person is notified, and your draft is cleared.

Related: React to a message; Reply in a thread; Attach a file. An agent can post the same way with `banter_post_message`.

Story: Attach a file to a message

Who: Any member of the channel. **Goal:** Share a file alongside a message. **Before you start:** The file is within your org's maximum file size.

Steps

1. Open the channel and click **Attach file** in the compose toolbar.
2. Pick one or more files. While they upload, the box shows "Uploading...".
3. Confirm the "N files attached" preview lists your files. Use **Remove all** to clear them and start over.
4. Type a message if you want one, then press **Enter** to send. An attachment-only message with no text is allowed.

Result: The message posts with its attachments, and everyone in the channel can see and open them.

Related: Search supports a **Has attachments** filter to find messages with files later.

Story: Create a channel for a new topic

Who: A member whose org allows channel creation, or an org admin. **Goal:** Stand up a new channel and start posting in it. **Before you start:** Your org's "Who can create channels" policy permits your role.

Steps

1. In the sidebar, click the + next to **Channels**.
2. In the **New Channel** box, type a name in the `channel-name` field using lowercase letters, numbers, and hyphens.
3. Click **Create**.
4. You land in the new channel; post your first message to seed the conversation.

Result: The channel exists, you own it, and it shows in your sidebar.

Related: To create several at once, an org admin can right-click the + for **Add many channels** (which uses a coarser admin-or-owner gate). Agents create channels with `banter_create_channel`.

Story: Find and join an existing channel

Who: Any member looking for the right room. **Goal:** Discover a public channel and join it. **Before you start:** None.

Steps

1. Click **Browse channels** in the sidebar.
2. On the **Browse Channels** page, type in the `Search channels...` box.
3. Scan the results for the right topic and member count.
4. Click **Join** on the channel you want.

Result: You are a member; the channel appears in your sidebar and shows **Joined** in the browser.

Related: Agents can list public channels with `banter_browse_channels` and join with `banter_join_channel`.

Story: Start a direct message

Who: Any member. **Goal:** Have a private one-to-one conversation. **Before you start:** The other person is an active member of your org.

Steps

1. Open the **Direct Messages** section in the sidebar.
2. Click the person you want to message.
3. Type in the compose box and press **Enter**.

Result: A private DM opens (or an existing one is reused) and your message is delivered.

Related: For a small group, an org that allows group DMs can start one with 3 to 8 people. Agents use `banter_send_dm` and `banter_send_group_dm`.

Story: Hold a side conversation in a thread

Who: Any member. **Goal:** Reply to a specific message without cluttering the channel. **Before you start:** There is a message you want to respond to.

Steps

1. Hover the message and click **Reply in thread**.
2. In the **Thread** panel, type your reply in the "Reply..." box.
3. If the whole channel should see it too, tick **Also send to channel**.
4. Press **Enter** to send.

Result: Your reply is attached to the thread, the parent shows an updated reply count, and the parent author and prior repliers are notified.

Related: Agents reply with `banter_reply_to_thread`.

Story: React, pin, and bookmark a message

Who: Any member (pinning requires channel admin). **Goal:** Acknowledge a message, surface it for the channel, and save it for yourself. **Before you start:** None for reacting or bookmarking; channel admin for pinning.

Steps

1. Hover the message and click **Add reaction**, then pick an emoji.
2. If you are a channel admin, click **Pin message** to add it to the channel's pinned list. Open **Pinned messages** in the header to review pins.
3. Click **Bookmark** to save the message privately. Find it later under **Bookmarks** in the sidebar.

Result: Your reaction shows under the message, the pin (if you added one) is visible to all members, and the bookmark is saved to your Bookmarks page.

Related: Edit or delete your message. Agents use `banter_react`, `banter_pin_message`, `banter_unpin_message`, and `banter_create_bookmark`.

Story: Edit or delete something you posted

Who: The message author (or a channel/org admin for deletion). **Goal:** Fix a typo or remove a message. **Before you start:** The message is yours and is not locked.

Steps

1. Hover your message and open the more menu.
2. Click **Edit message**, change the text, and click **Save** (or press Enter; press Esc to cancel).
3. To remove it instead, open the more menu and click **Delete message**.

Result: Edited messages show "(edited)"; deleted messages disappear from the timeline.

Related: Agents use `banter_edit_message` and `banter_delete_message` (delete requires confirmation).

Story: Find an old message with search

Who: Any member. **Goal:** Locate a past message across the channels you can see. **Before you start:** You are in Banter.

Steps

1. Click the header search box or open **Search** from the sidebar.
2. Type at least two characters of what you remember and press Enter.
3. Click **Filters** and narrow by **Channel**, **Author**, **From date**, **To date**, or **Has attachments** if the results are broad.
4. Click a result row to jump to that message's channel.

Result: You see matching messages with the search terms highlighted, scoped to channels you belong to.

Related: An agent can run the same search with `banter_search_messages`.

Story: Configure and tidy a channel

Who: A channel admin. **Goal:** Set a channel's name, topic, and behavior, add a member, and manage its lifecycle. **Before you start:** You are an admin or owner of the channel.

Steps

1. In the channel header, click **Channel settings**.
2. Edit **Channel name**, **Topic**, and **Description**.
3. Toggle **Allow bots** and **Allow huddles** as needed.
4. Click **Save Changes** and confirm the "Saved!" message.
5. To add a member, type an email or username into the **Email or username** field and click **Add**.
6. To archive the channel, open the **Danger Zone**, click **Delete Channel**, and confirm with **Yes, delete channel**.

Result: The channel reflects your changes, the new member is added, or the channel is archived and removed from the active list.

Related: Agents update channels with `banter_update_channel`, add members with `banter_add_channel_members`, and archive with `banter_archive_channel`.

Story: Schedule a message for later (automation and agents)

Who: A workflow or an AI agent. **Goal:** Post a message at a specific future time, or respect a channel's quiet hours. **Before you start:** An agent or automation with permission to post to the channel. There is no in-app scheduler in the SPA today.

Steps

1. The agent calls `banter_schedule_post` with the channel, the content, and a future `scheduled_at`.
2. If the channel has a quiet-hours policy and the post lands inside the quiet window, it is deferred automatically when `defer_if_quiet` is set, or rejected with `QUIET_HOURS` otherwise.
3. A worker delivers the message at the scheduled time.
4. Pending scheduled messages for a channel can be listed with `banter_list_scheduled_messages` and cancelled with `banter_cancel_scheduled_message` before they send.

Result: The message posts at the right time. Banter emits `message.scheduled` (and `message.quiet_hours_deferred` when deferred) for downstream automations.

Related: Pattern subscriptions let an agent listen for matching messages and react.

Story: Have an agent listen for a pattern and respond

Who: An AI agent or service account, plus a channel admin to allow it. **Goal:** Trigger agent action whenever messages in a channel match a pattern. **Before you start:** The channel's `agent_subscription_policy` and the org's agent policies allow the agent.

Steps

1. The agent subscribes with `banter_subscribe_pattern`, choosing a kind: `interrogative`, `keyword`, `mention`, or `regex` (regex is admin only).
2. When a message matches, Banter fires a `message.matched` event (on source `banter`).
3. The agent acts on the match (for example, replies in the thread, routes to a human, or shares a suite entity), honoring `can_access` on anything it cites.
4. Remove the subscription with `banter_unsubscribe_pattern`; review active ones with `banter_list_subscriptions`.

Result: The agent reacts automatically to matching traffic. Blocked subscriptions are recorded as not effective (`effective:false`) with a reason, so you can see why an agent is not listening.

Related: Share a suite entity into the channel after a match.

Story: Share a task, sprint, or ticket into a channel

Who: A member or an agent. **Goal:** Bring a Bam task or sprint, or a Helpdesk ticket, into a conversation. **Before you start:** You can see the entity you want to share.

Steps

1. From an automation or agent, call `banter_share_task`, `banter_share_sprint`, or `banter_share_ticket` with the channel and the entity.
2. The shared entity is posted into the channel for everyone to discuss.

Result: The channel shows a reference to the entity, keeping the discussion next to the work.

Related: Reply in a thread to discuss the shared item without flooding the channel.

Story: Set up Banter for the whole org

Who: An org admin or owner. **Goal:** Decide who can create channels, set retention and file limits, and organize the sidebar. **Before you start:** You have org admin or owner access.

Steps

1. Open **Banter Administration**.
2. Under **Channel Settings**, set **Who can create channels** and the **Default Channel**.
3. Under **Message Settings**, set retention days and maximum file size.
4. Create channel groups to bucket channels in the sidebar.
5. Define @mention user groups (for example `@engineering`) and add members.

Result: New channels follow your policy, the sidebar is organized, and teams can be mentioned as a group.

Related: Migrate an existing Slack workspace with the Slack import. All three "Who can create channels" options are enforced on the single + create; the bulk **Add many channels** dialog uses a coarser admin-or-owner gate.

Story: Migrate a Slack workspace

Who: An org admin. **Goal:** Bring channels, history, and people over from Slack. **Before you start:** You have a Slack export `.zip` and admin access.

Steps

1. Open the Slack import area under admin and upload the `.zip`.
2. Review the preview of users and channels.
3. Map each user (`auto_match`, `send_invite`, `stub`, `map_existing`, or `skip`) and each channel (`import_new`, `merge_existing`, or `skip`).
4. Choose options such as preserving timestamps and importing attachments, reactions, pins, and DMs. Run a dry run first if you want to validate.
5. Start the import and poll its status to completion.

Result: Your Slack content is imported into Banter according to your mapping; you can abort and clean up stubs if needed.

Related: Channel groups and @mention groups help organize the imported channels and teams.

Story: Review a past call

Who: Any member with access to the channel where the call happened. **Goal:** Look back at a finished call and read its transcript. **Before you start:** A call took place; live audio is handled by the Bureau docked box, not Banter.

Steps

1. Open the call's playback page at `/banter/calls/:id`.
2. Review the call type (voice, video, or huddle), duration, and participants.
3. Read the transcript.

Result: You have a read-only record of the call. You cannot start, join, or end a call from Banter; use the Bureau docked box for live audio.

Related: None.

Related

- **BigBlueBam (Bam)** at `/b3/` - the host app that owns your identity, organizations, and people. You sign in there before using Banter, and tasks and sprints shared into Banter come from Bam.
- **Helpdesk** at `/helpdesk/` - tickets shared into a Banter channel come from here.
- **Bureau docked box** - the suite-wide live-audio layer that replaced Banter calls. Use it for voice and video; Banter keeps only the read-only call history.
- **Bolt** at `/bolt/` - the automation engine that consumes Banter events (`channel.created`, `message.posted`, `message.mentioned`, `message.edited`, `reaction.added`, `message.scheduled`, `message.quiet_hours_deferred`, and `message.matched`, all on source `banter`) and can drive Banter actions.
- **Banter MCP-tools reference** in `docs/apps/banter/` - the full catalog of the Banter tools (over 50 core tools plus the 3 subscription tools) used by AI agents.
- **Banter guide** in `docs/apps/banter/` - product-level overview. Where it advertises live voice calls, follow this help doc instead: those calls are retired in Banter, and live audio is handled by the Bureau docked box.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What is the maximum length of a single Banter message?

a) 4,000 characters
b) 10,000 characters
c) 40,000 characters
d) There is no limit

2. What are the three channel types besides DMs?

a) public, private, and a DM type
b) open, closed, and hidden
c) team, project, and personal
d) default, custom, and archived

3. What are the four channel roles, and which one is read-only?

4. How many people total can a group DM hold?

a) 2 people
b) 3 to 8 people
c) Up to 10 people
d) Up to 50 people

5. Who can pin a message in a channel, and who can everyone see the pin?

6. Who can see your bookmarks?

a) Everyone in the channel
b) Only channel admins
c) Only you
d) Anyone in your org

7. Which of these is one of the live presence statuses in Banter?

a) away on vacation
b) do not disturb
c) in a meeting
d) snoozed

8. What happens the first time anyone opens Banter in an org that has no channels?

9. What are the three options for the 'Who can create channels' org policy?

a) Everyone, Admins only, Organization owners only
b) Members, Managers, Owners
c) Open, Restricted, Locked
d) Everyone, Verified, Owners

10. How is channel creation from the single + button rate-limited?

11. What happens to a reply in a thread unless you tick 'Also send to channel'?

a) It is discarded
b) It stays in the thread only
c) It posts only to the parent author
d) It is queued for later

12. What does Banter now do about live audio calls, and where does live audio happen

instead?

13. What is a quiet hours policy, and which posts does it mostly affect?

14. What keyboard shortcut opens the quick channel switcher?

a) Ctrl/Cmd+K
b) Ctrl/Cmd+Shift+M
c) Esc
d) Up arrow

CHAPTER 4

Beacon

Knowledge that stays true, not just written once.

Beacon is your team's knowledge base, built so articles stay accurate over time instead of rotting quietly after the day they were written. You write in Markdown, find answers by meaning with hybrid search, walk a knowledge graph of typed links, and lean on a freshness dashboard that surfaces stale knowledge before it misleads anyone. This chapter takes you from your first published article through search, linking, the governance sweep, and expiry policy, and it shows where an agent can ingest and verify knowledge through the same tools. By the end you will be able to publish an article, find it by meaning, and keep the whole library verified.

At a glance

- Knowledge articles with a full lifecycle: Draft, Active, Pending Review, Archived, Retired
- A freshness signal that flags stale, expiring, or unverified knowledge before it bites
- Hybrid search: semantic plus tags plus link traversal plus keyword fallback
- A Knowledge Graph of typed links and implicit Tag Affinity edges
- Per-scope expiry policy and a Fridge Cleanout dashboard for keeping the library verified

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Knowledge Home . Browse the article list . Read an article (detail page) . Create and edit an article . Lifecycle actions . Search

> *Beacon is your team's knowledge base: a place to write, search, link, and keep articles current. It is the app you reach for when knowledge needs to stay accurate over time, not just get written once and forgotten.*

Overview

Beacon[28] stores your team's knowledge as **Beacons** (knowledge articles). Each Beacon has a title, a short summary, a Markdown body, an owner, a visibility[29] level, and an expiry date. Beacons move through a lifecycle (Draft, Active, Pending Review, Archived, Retired) and carry a freshness[30] signal so stale knowledge surfaces before it misleads anyone.

> ***Freshness is the whole point: Beacon shows you when knowledge is going stale before anyone acts on it.***

Beyond plain storage, Beacon connects articles into a **Knowledge Graph**[31] of typed links and shared tags[32], runs **hybrid search** (semantic plus tags plus link[33] traversal plus keyword fallback), and gives governance owners a freshness-focused dashboard for keeping the library verified. Expiry policies let an org set how long knowledge stays trusted before it needs re-verification[34].

Beacon is part of the BigBlueBam suite. It shares your platform login with Bam and the other apps, pulls its project list from Bam, and publishes lifecycle events to Bolt so automations can react when knowledge is created, updated, verified, or retired.

Key concepts

- **Beacon** - A single knowledge article. Has a globally unique slug, a title, a summary (up to 500 characters), a Markdown body, a version number, a status[35], a visibility, an owner, an optional project, and an expiry date.
- **Status** - The article's place in its lifecycle. The five statuses you will see in the UI are **Draft**, **Active**, **Pending Review**, **Archived**, and **Retired**. New articles start as Draft; publishing makes them Active.

---

[28] **Beacon.** A single knowledge article. Has a globally unique slug, a title, a summary (up to 500 characters), a Markdown body, a version number, a status, a visibility, an owner, an optional project, and an expiry date.

[29] **Visibility.** Who can see the article: **Public**, **Organization**, **Project**, or **Private**. Public and Organization are visible to all org members; Private is visible only to the owner or creator; Project is visible to the owner, creator, or members of that project.

[30] **Freshness.** A computed signal based on when the article was last verified and when it expires. The detail page and cards show one of four states: **Verified recently**, **Content is stale**, **Expiring soon**, or **Needs verification**.

[31] **Knowledge Graph.** The network of Beacons. Nodes are articles; edges are either explicit typed links or implicit "Tag Affinity" edges between articles that share enough tags.

[32] **Tags.** Free-form labels on an article. Tags drive filtering, search tag expansion, and implicit graph edges. They are a search and filter facet: you add and remove tags as chips on the Search screen to widen or narrow results. Note: the editor shows a "Tags (comma-separated)" field, but tags typed there are not saved today (a known limitation; see Create and edit an article). Tags that appear on articles come from agent or API writes, not from the editor field.

[33] **Link.** A typed connection between two articles: **Related To**, **Supersedes**, **Depends On**, **Conflicts With**, or **See Also**. Links are read-only in the UI today (created through MCP tools or the API; see Links between articles).

[34] **Verification.** A recorded review confirming an article is still accurate. Verifying resets the expiry date and raises the verification count.

[35] **Status.** The article's place in its lifecycle. The five statuses you will see in the UI are **Draft**, **Active**, **Pending Review**, **Archived**, and **Retired**. New articles start as Draft; publishing makes them Active.

- **Visibility** - Who can see the article: **Public**, **Organization**, **Project**, or **Private**. Public and Organization are visible to all org members; Private is visible only to the owner or creator; Project is visible to the owner, creator, or members of that project.
- **Freshness** - A computed signal based on when the article was last verified and when it expires. The detail page and cards show one of four states: **Verified recently**, **Content is stale**, **Expiring soon**, or **Needs verification**.
- **Verification** - A recorded review confirming an article is still accurate. Verifying resets the expiry date and raises the verification count.
- **Challenge**[36] - A flag that an Active article may be wrong. Challenging moves it to Pending Review.
- **Tags** - Free-form labels on an article. Tags drive filtering, search tag expansion, and implicit graph edges. They are a search and filter facet: you add and remove tags as chips on the Search screen to widen or narrow results. Note: the editor shows a "Tags (comma-separated)" field, but tags typed there are not saved today (a known limitation; see Create and edit an article). Tags that appear on articles come from agent or API writes, not from the editor field.
- **Link** - A typed connection between two articles: **Related To**, **Supersedes**, **Depends On**, **Conflicts With**, or **See Also**. Links are read-only in the UI today (created through MCP tools or the API; see Links between articles).
- **Knowledge Graph** - The network of Beacons. Nodes are articles; edges are either explicit typed links or implicit "Tag Affinity" edges between articles that share enough tags.
- **Saved query**[37] - A named, reusable search configuration you can reopen later. Scope can be Private, Project, or Organization.
- **Expiry policy**[38] - Per-scope rules (System, Organization, Project) that set the minimum, maximum, and default number of days before an article expires, plus a grace period.

Where to find it

Beacon lives at `/beacon/`. You must be logged in to BigBlueBam first; Beacon has no login of its own and uses the shared platform session. If you are not signed in, Beacon shows "Please log in to BigBlueBam first to access Beacon." with a "Go to BigBlueBam Login" link to `/b3/`.

The left sidebar has six navigation items: **Home** (`/`), **Browse** (`/list`), **Search** (`/search`), **Graph** (`/graph`), **Dashboard** (`/dashboard`), and **Beacon Settings** (`/settings`). At the top of the sidebar is a project scope selector; it shows the active project name or **"All Projects"** by default, and it filters most views to that scope.

---

[36] **Challenge.** A flag that an Active article may be wrong. Challenging moves it to Pending Review.

[37] **Saved query.** A named, reusable search configuration you can reopen later. Scope can be Private, Project, or Organization.

[38] **Expiry policy.** Per-scope rules (System, Organization, Project) that set the minimum, maximum, and default number of days before an article expires, plus a grace period.

What you can do depends on your role. Editing access works like this: a SuperUser can edit any Beacon; an org Admin or Owner can edit any Beacon in the org; a Member can edit only Beacons they own or created. Editing System-level expiry policy requires SuperUser.

Press the ? key (when your cursor is not in a text field) to open the in-app help from any Beacon screen.

> **TRY IT**
> Open any Beacon screen and press the ? key while your cursor is outside a text field to pop the in-app help.

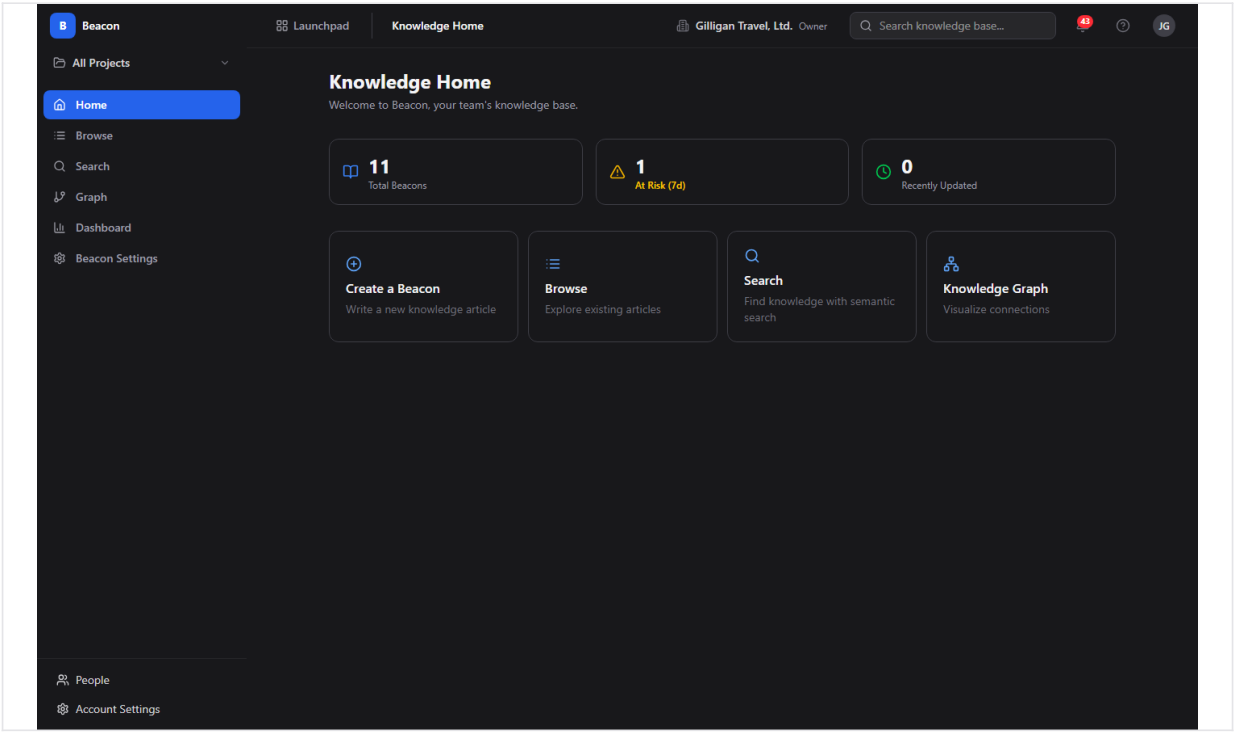


Figure 4.1 Knowledge home

Feature reference

Knowledge Home

The home screen is your starting point. Its heading is **"Knowledge Home"** with the subtitle "Welcome to Beacon, your team's knowledge base."

To use Knowledge Home:

1. Open Beacon at `/beacon/` or click **Home** in the sidebar.
2. Read the three stat cards at the top: **"Total Beacons"**, **"At Risk (7d)"**, and **"Recently Updated"**. Clicking "Total Beacons" or "At Risk (7d)" takes you to Browse; clicking "Recently Updated" takes you to the Graph.
3. Use a quick-action card to jump into a task: **"Create a Beacon"**, **"Browse"**, **"Search"**, or **"Knowledge Graph"**.
4. Scroll to **"Recent Activity"** to see up to 8 recently touched articles; click one to open its detail page.

Browse the article list

Browse lists the Beacons you can see, with filtering. The screen is titled "Article list" in docs and rendered at `/list`.

To browse and filter:

1. Click **Browse** in the sidebar.
2. Type into the search input (placeholder **"Search beacons..."**) to filter by text.
3. Click a status chip to filter by lifecycle state: **All**, **Active**, **Pending Review**, **Draft**, or **Archived**.
4. Read the project indicator near the toolbar: it shows "Showing beacons for: <name>" when scoped to a project, or "Showing all org beacons" otherwise.
5. Click a card to open that article's detail page.
6. Click **"Load more"** at the bottom to fetch the next page of results.

To start a new article from here, click **"New Beacon"** (or, if the list is empty, **"Create Beacon"** in the "No beacons yet. Create your first one." empty state).

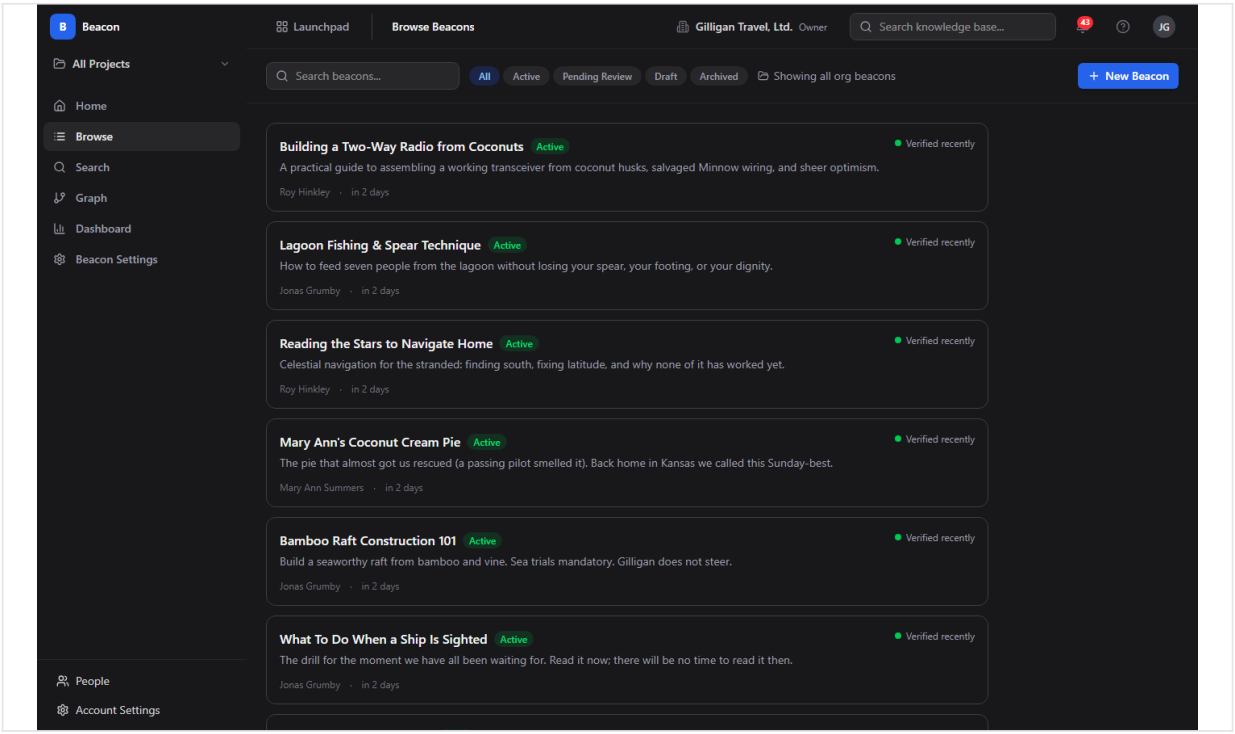


Figure 4.2 Browse articles

Read an article (detail page)

The detail page shows one article in full, with its lifecycle actions and metadata.

What you see:

- The title, a **StatusBadge**, a **LifecycleActions** row, presence chips, and an **"Edit"** button.
- The summary, the rendered Markdown body, and the Tags row.
- An **Attachments** panel and a **Comments** section.
- A right sidebar with **Status**, **Owner**, **Project** ("Organization-wide" if the article has no project), **Freshness**, **Expires** (a date), **Last Verified** ("Never" if it has never been verified), **Verifications** (a count), **Tags**, **Linked Beacons** (each links to its target), and **Version** (vN, with an expandable history). A **"View in Graph"** link opens the article in the Graph.

To open an article, click any card in Browse, Search, the Graph, or the Recent Activity list, then read or act on it from the detail page.

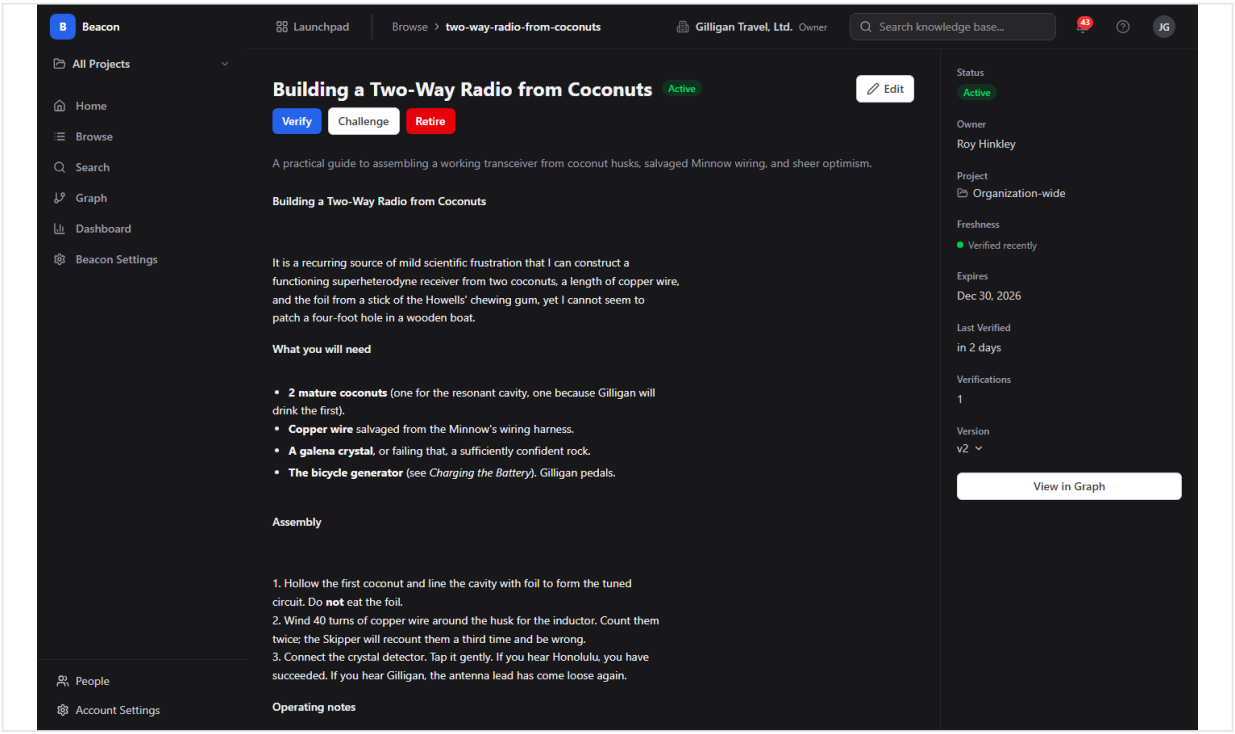


Figure 4.3 Article detail

Create and edit an article

The editor is reached at `/create` (from the "Create a Beacon" card on Home or "New Beacon" in Browse) and at `/<idOrSlug>/edit` (the "Edit" button on a detail page). The header reads **"Create Beacon"** or **"Edit Beacon"**.

The body field is a **Markdown** editor, labeled **"Body (Markdown)"**. You write Markdown directly; there is no rich-text formatting toolbar. When you edit an **existing** article the body is a **live collaborative editor**: anyone else who opens the same article edits the body with you in real time, and you see their cursors and changes as they type (the label reads "live co-editing"). While creating a brand-new article (before the first save) the body is a plain text area, since there is no shared article to join yet.

To write a new article:

1. Open the editor with **"New Beacon"** or the **"Create a Beacon"** card.
2. Type a title in the title field (placeholder "Beacon title…").
3. Optionally add a **Summary** (up to 500 characters; a live "N characters remaining" counter shows your budget).
4. Write the article in **Body (Markdown)** (placeholder "Write your knowledge article in Markdown…").
5. Choose **Project (optional)** from the select. The default option is "Organization-wide (no project)". This selector is shown only when creating; when editing, the project is read-only.
6. Choose a **Visibility**: Public, Organization, Project, or Private. The editor defaults to Organization.

7. Click **"Save as Draft"** to keep it as a Draft, or **"Publish"** to create it and immediately make it Active. Both buttons are disabled until the title is non-empty.

Known limitation: the editor includes a **"Tags (comma-separated)"** field (placeholder "e.g. onboarding, deployment, api"), but anything you type there is currently not saved on either create or edit. Do not rely on the editor to set tags. Tags on an article today come from agent or API writes (`beacon_upsert_by_slug` and the tag write endpoints), and you filter by them on the Search screen.

> **NOTE**
> The editor's tag field does not save today. Tags you see on articles were written by an agent or the API, and you filter by them on the Search screen rather than setting them on the editor.

Editing an existing article works the same way, except the Project field is read-only, the body is the live collaborative editor described above (your edits sync to anyone editing alongside you), and saving bumps the article's version and writes a version snapshot. If someone changed the article's title, summary, or visibility since you opened it, the save is rejected with a "this article changed since you opened it" notice so you do not silently overwrite their change; the body itself merges live and is not subject to that conflict check.

There is no screenshot of the editor in this set.

Lifecycle actions

Lifecycle actions live in the row under the title on the detail page. Which buttons appear depends on the article's current status. Each button opens a confirmation dialog titled "<Action> Beacon".

- **Draft** shows **Publish**. Publishing moves the article to Active and recomputes its expiry. Dialog text: "This will make the beacon visible to others based on its visibility setting."
- **Active** shows **Verify**, **Challenge**, and **Retire**.
- **Pending Review** shows **Publish** and **Retire**.
- **Archived** shows **Restore** and **Retire**. **Restore** returns an Archived article to Active and resets its expiry. Dialog text: "Restore this beacon to Active status."

> **WARNING**
> Retired is a terminal state. The Restore button still appears on a Retired article, but the server only restores Archived ones, so Restore returns an error and the status will not change. To bring back retired knowledge, recreate or re-author it.

To run a lifecycle action:

1. Open the article's detail page.
2. Click the action button in the lifecycle row.
3. Read the confirmation dialog and click the matching button to confirm, or **Cancel** to back out.

Known limitation: a **Retired** article also shows a **Restore** button, but the server only restores articles that are currently Archived. Retired is a terminal state, so clicking **Restore** on a Retired article returns an error and the status does not change. To bring a Retired article back, recreate or re-author it rather than relying on Restore.

Known limitation: a **Pending Review** article shows a **Publish** button, but the server only publishes articles that are currently Draft. Clicking **Publish** on a Pending Review article returns an error ("Cannot publish a beacon with status 'PendingReview'; must be Draft") and the status does not change. To return a Pending Review article to Active, verify it instead, which records a verification and resets its expiry.

Search

Search runs a hybrid query across every Beacon you can access. The heading is **"Search"** with the subtitle "Find knowledge across all accessible Beacons."

To search:

1. Click **Search** in the sidebar.
2. Type a natural-language query in the primary input (placeholder **"Search Beacons..."**).
3. Optionally narrow with the QueryBuilder controls:
4. Watch the footer's live "~N Beacons match" count update as you adjust filters.
5. Click a result card to open the article.

Each **ResultCard** shows the title, a StatusBadge, the summary, a highlighted matching passage, clickable tag chips ("Add '<tag>' to filters"), match-source badges (**"Semantic match"**, **"Tag expansion"**, **"Link traversal"**, **"Keyword match"**), the freshness state, a verification count, the owner, and up to two linked beacons with their link type.

Your search configuration is reflected in the URL, so you can copy the link and share an exact search with a teammate.

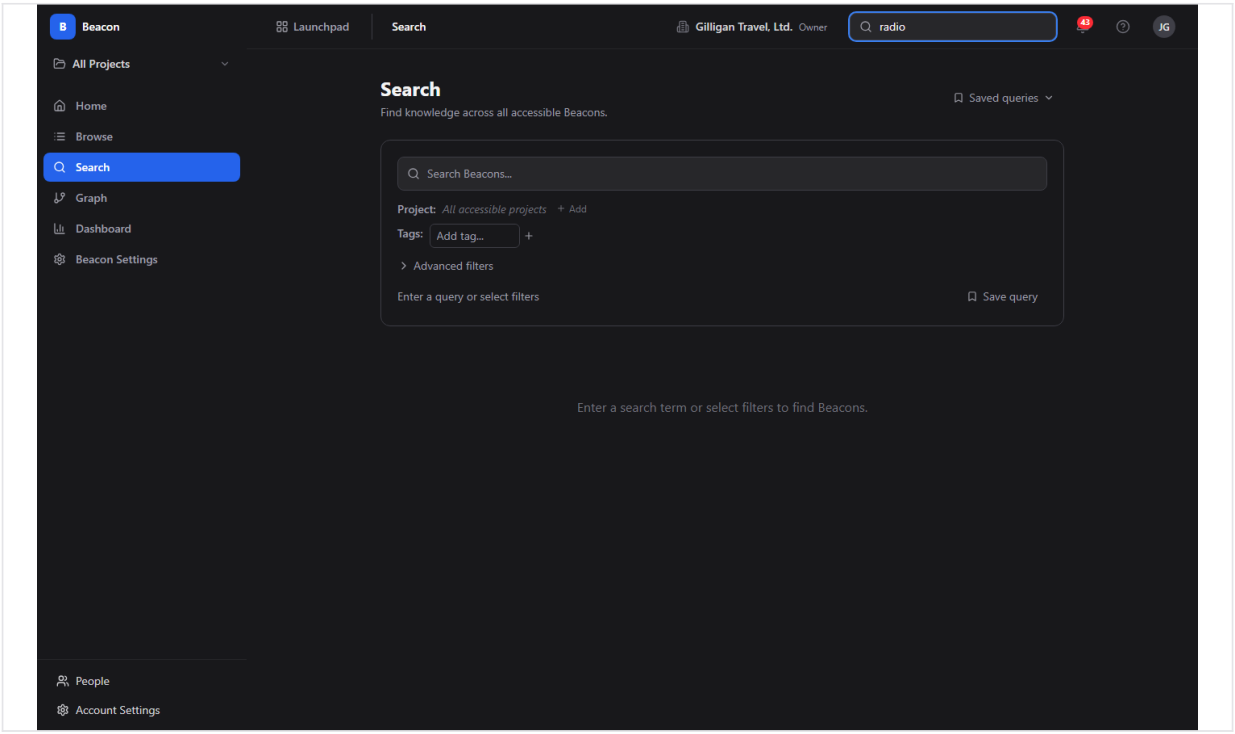


Figure 4.4 Hybrid search

Saved queries

You can name and reuse a search configuration.

To save a query:

1. Build the search you want in Search.
2. Click **"Save query"** in the footer to open the "Save Search Query" dialog.
3. Enter a **Name** and pick a **Scope**: "Private (only me)", Project, or Organization.
4. Confirm to save.

To reuse a saved query, open the **"Saved queries"** dropdown at the top right of the Search screen, then click a saved entry to load it. In the **SavedQueriesPanel** you can click an entry to load it, or click its trash icon to delete it.

Knowledge Graph

The Graph visualizes how articles connect. The heading is **"Knowledge Graph"**.

When no article is focused, the Graph shows a Knowledge Home layout: a "Hub Beacons" canvas of the most-connected articles on the left, and on the right an **"At Risk"** list (expiring within 7 days) and a **"Recently Updated"** list.

When you focus an article, the Graph draws a force-directed canvas of that article's neighbors, with a breadcrumb trail, node and edge counts, and an **EdgeLegend** explaining the colors.

To explore the graph:

1. Click **Graph** in the sidebar, or click **"View in Graph"** on a detail page to start from a specific article.
2. With an article focused, set **Hops:** to 1, 2, or 3 to widen or narrow how many steps out the graph reaches.
3. Use the implicit-edges eye toggle ("Show/Hide implicit edges") to show or hide Tag Affinity edges.
4. Use the **"Filter by Status"** dropdown (Active, Pending Review, Draft, Archived) to dim non-matching nodes; "Filtered nodes are dimmed, not hidden." Click "Clear" to reset.
5. Click a node to open its **NodePopover**, which shows the title, status, summary, tags, freshness, verification count, and owner, plus actions **"View Beacon"** (open detail) and **"Explore from here"** (re-center the graph on that node).

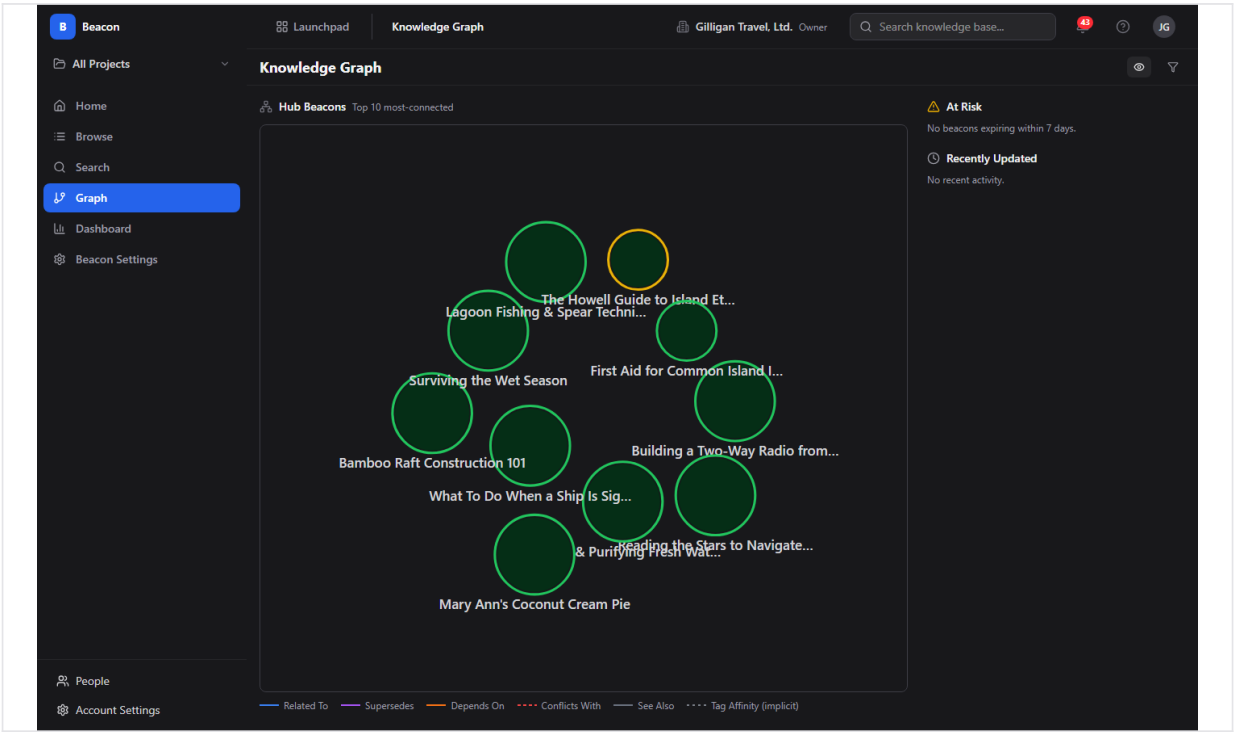


Figure 4.5 Knowledge graph

Tags

Tags label articles for filtering, search expansion, and implicit graph edges. In the UI today, tags are a search and filter facet rather than something you add or remove on an individual article.

To work with tags:

1. Open the **Search** screen and find the **Tags:** control in the QueryBuilder.
2. Type into the "Add tag..." typeahead and pick a tag to add it as a filter chip; click a chip's remove control to drop it from the filter.
3. On a **ResultCard**, click a tag chip ("Add '<tag>' to filters") to add that tag to your search.

These tag chips add and remove tags from your search filter only; they do not change the tags stored on an article. The article detail page and graph node popovers display an article's tags read-only. The editor's "Tags (comma-separated)" field does not persist (see Create and edit an article), so the tags you see on articles today are written by agents or the API, not from the UI. Writing tags goes through the tag endpoints (or `beacon_tag_add` / `beacon_tag_remove`): you may add 1 to 20 tags at a time, each 1 to 128 characters, and removing a tag that is not present returns an error. Article tags also drive the implicit "Tag Affinity" edges in the Graph.

Links between articles

Links create typed connections between two articles. The available types are Related To, Supersedes, Depends On, Conflicts With, and See Also.

In the UI today, links are read-only: the detail page lists an article's existing links under **Linked Beacons** (each row opens its target), and the Graph draws them as typed edges. There is no human "create a link" or "remove a link" control in the Beacon SPA. Links are created and removed through the MCP tools `beacon_link_create` and `beacon_link_remove` (or the equivalent API), so an agent or integration sets up the connections that you then read and navigate.

To follow an existing link:

1. Open the source article's detail page.
2. Under **Linked Beacons**, click a target to jump to it.
3. Open the **Graph** (or click **"View in Graph"**) to see the link as a typed edge in context.

Creating a duplicate link (same source, target, and type) is rejected at the tool/API layer.

Comments

Each article has a threaded discussion. The heading is **"Comments (n)"**.

To comment:

1. On the detail page, scroll to the Comments section.
2. Type into the box labeled "Add a comment. Markdown supported." Markdown is supported in comments.
3. Click **"Post Comment"**.
4. To reply, click **Reply** on a comment (threads nest up to 4 levels deep).
5. To remove your own comment, click **Delete**. The confirm reads "Delete this comment? Replies will also be removed." Admins, owners, and SuperUsers can delete others' comments.

Attachments

You can attach files to an article. The heading is **"Attachments (n)"**.

To attach a file:

1. On the detail page, find the Attachments panel.
2. Drop a file on the drop zone ("Drop a file here or") or click **"Choose File"**. The limit is "Max 10 MB. Images, PDF, text, office docs."
3. Each attached file row shows a thumbnail or icon, the filename, the size, the uploader, and the time, with **Download** and **Delete** actions.
4. To remove an attachment, click **Delete** and confirm "Delete attachment '<name>'?". The uploader, or an admin, can delete an attachment.

Uploading requires edit access to the article.

Fridge Cleanout (governance dashboard)

The Dashboard is a freshness and governance console. Its header is **"Fridge Cleanout"** with the subtitle "Knowledge governance dashboard". It has four tabs: **Overview**, **At-Risk**, **Archived**, and **Agent Activity**.

This dashboard is freshness-only. It does not show article-view counts or search-pattern analytics; those metrics do not exist in Beacon. Everything here is about keeping knowledge verified and cleaning out stale articles.

To use each tab:

- **Overview**: read four cards - **"Freshness Score"** (a percentage), **"At-Risk (7 days)"**, **"Archived Backlog"** (articles archived more than 30 days), and **"Total Active"** - plus a "Freshness Breakdown" bar that reads "X of Y active beacons verified within 30 days".
- **At-Risk**: a table of articles expiring within 7 days (Title, Expires, Owner, Status, Actions). Click a row's **Verify** or **Challenge** button to act on one article. To act in bulk, tick the checkboxes and click **"Verify Selected (n)"**.
- **Archived**: articles archived for 30 or more days (Title, Archived Since, Owner, Actions). Each row offers **Restore** and **Retire**. Tick checkboxes and click **"Retire Selected (n)"** to retire several at once.
- **Agent Activity**: recent verification events, each reading "Verified by <owner> <time>" with the article's verification count (shown as v<count>) and current status.

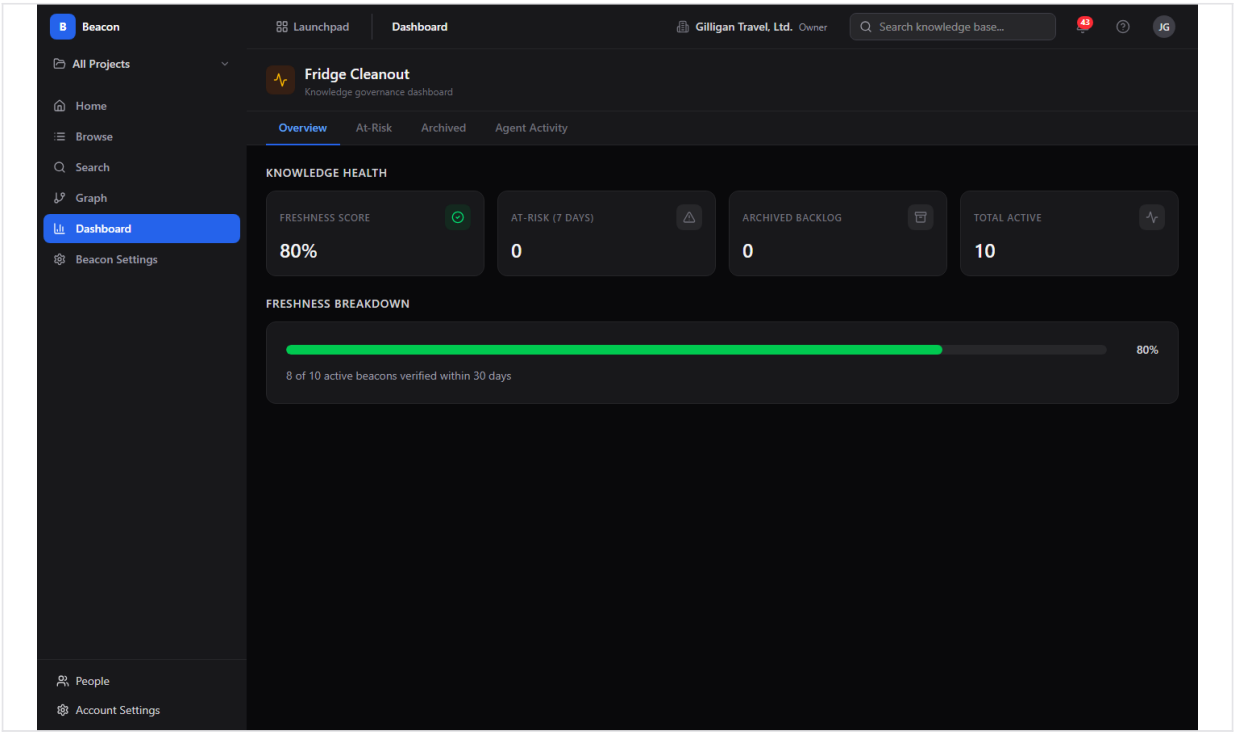


Figure 4.6 Freshness dashboard

Expiry Policy Settings

Beacon Settings is where you set how long knowledge stays trusted. The header is **"Expiry Policy Settings"** with the subtitle "Manage knowledge freshness policies across the hierarchy".

To view and set policy:

1. Click **Beacon Settings** in the sidebar.
2. Read the **"Effective Policy (Your Context)"** card, which shows the resolved Min, Max, Default Expiry, and Grace Period that apply to you right now. Policies resolve in order: Project, then Organization, then System.
3. Edit a policy in the appropriate editor:
4. In an editor, set Min, Max, and Default expiry (in days) plus the Grace period. Client validation enforces min <= default <= max and keeps values within the parent scope's bounds.
5. Click **"Save Policy"**. Success or warning text appears; editors you lack permission to change show "Read-only - requires higher permissions to edit".

There is no screenshot of Beacon Settings in this set.

A few notes on freshness mechanics that policy controls:

- A daily expiry sweep moves Active articles to Pending Review when they expire, moves Pending Review articles to Archived after the grace period, and deletes very old Drafts.

- The lifecycle includes an internal "Expired" value, but the live sweep never sets it and the UI has no badge for it, so you will not see an "Expired" status on any article in normal use. The states you encounter are Draft, Active, Pending Review, Archived, and Retired.

Working with AI agents

Agents drive Beacon through its MCP tools (30 of them). They can author and update knowledge, run retrieval to ground answers, verify articles, and build or query the graph and policy. Comments and attachments have no MCP tools, so agents cannot post comments or upload files; those stay human-only.

> ***Beacon ships 30 MCP tools, so an agent can author, verify, and graph knowledge with the same reach a person has.***

Common agent flows and the Beacon-specific tools behind them:

- **Idempotent ingestion.** Agents write or refresh knowledge with `beacon_upsert_by_slug` (the `POST /entries/upsert` write plane). The tool is keyed on the article's slug and returns a `created` flag so the agent can tell an insert from an update. Re-running it with the same slug updates in place rather than creating duplicates. This is the recommended path for bulk or repeated imports; there is no human button for it. Because the editor's tag field does not persist, agent and API writes are how tags get set on articles.
- **Authoring and lifecycle.** Beyond upsert, agents have the full set: `beacon_create`, `beacon_update`, `beacon_get`, `beacon_list`, `beacon_publish`, `beacon_verify`, `beacon_challenge`, `beacon_retire`, `beacon_restore`, `beacon_versions`, and `beacon_version_get`. Each tool resolves an article by UUID, slug, or title.
- **Grounding and retrieval (RAG).** Agents pull knowledge with `beacon_search` or `beacon_search_context`. The context variant always turns on graph and tag expansion and pre-fetches linked articles, so an agent gets a richer slice of the graph in one call. `beacon_suggest` powers typeahead.
- **Automated verification.** `beacon_verify` accepts verification types `AgentAutomatic`, `AgentAssisted`, and `ScheduledReview`, plus an optional confidence score. Agent verifications show up under the dashboard's **Agent Activity** tab, so a human can review what the agent confirmed. The daily expiry sweep also enqueues Pending Review articles for an agent verification pass.
- **Graph and governance.** Agents can build and read the graph with `beacon_link_create`, `beacon_link_remove`, `beacon_graph_neighbors`, `beacon_graph_hubs`, and `beacon_graph_recent`, and read or set expiry policy with `beacon_policy_get`, `beacon_policy_set`, and `beacon_policy_resolve`. Treat policy as the four expiry and grace fields (min, max, default expiry days, grace period days). Link creation and removal are agent/API only; the Beacon SPA has no human link-management control.

- **Tags and saved queries.** `beacon_tags_list` reads tags in scope; `beacon_tag_add` and `beacon_tag_remove` write them. `beacon_query_save`, `beacon_query_list`, `beacon_query_get`, and `beacon_query_delete` manage saved searches.

> **TIP**
> When an agent needs context, beacon_search_context beats plain beacon_search: it always turns on graph and tag expansion and pre-fetches linked articles, so it pulls a richer slice of the graph in a single call.

Platform agent rails. Beacon's tools run inside the suite-wide agentic platform, so the same guardrails apply here as everywhere:

- **Visibility preflight.** Before an agent cites a Beacon in a shared surface, it must confirm the asker can see it by calling `can_access` (entity type `beacon.entry`). Beacons are visibility-gated; refusal reasons include `beacon_private_not_owner` and `beacon_project_not_member`. Anything the asker cannot see is dropped from the agent's answer.
- **Cross-app retrieval.** `search_everything` fans out across the suite and includes Beacon results with normalized scoring, and `resolve_references` turns canonical mentions into Beacon links. Both honor the same visibility rules.
- **Identity, heartbeat, and audit.** Agent and service accounts are tagged with a `kind` and emit heartbeats (`agent_heartbeat`); their Beacon writes are attributed in the unified activity view alongside Bam, Bond, and Helpdesk events, so you can audit what an agent created, verified, or retired.
- **Approvals and policy.** High-impact agent work can be routed through the proposal queue (`proposal_create` / `proposal_list` / `proposal_decide`) for a human to approve. Per-agent kill switches and tool allowlists (the `beacon.*` prefix) gate which agents may touch Beacon at all, and subscribed Bolt events can be pushed to agent runners over signed outbound webhooks.

For the full tool catalog, see `docs/apps/beacon/mcp-tools.md`.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. When you open a knowledge page, a presence strip shows who else is reading or editing it, and you can ring a teammate into a huddle from the page. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

Beacon goes a step further than presence on the article body itself: the editor for an existing article is **real-time co-editing**. Two or more people can write the same Markdown body at once and see each other's cursors and edits live, the way you would in a shared document, so a team can draft or correct an article together instead of trading "who has it open" messages. The shared text merges automatically (it is a CRDT), so simultaneous edits do not clobber each other; only the article's metadata (title, summary, visibility) uses the last-writer-wins conflict notice.

User Stories

Story: Write and publish your first article

Who: A new team member who has knowledge to capture. **Goal:** Get an article published and visible to the team. **Before you start:** Be logged in to BigBlueBam and have create access (any Member can create).

Steps

1. Open Beacon at `/beacon/`. On **Knowledge Home**, click the **"Create a Beacon"** card (or click **Browse** then **"New Beacon"**).
2. In the editor, type a clear title in the field (placeholder "Beacon title...").
3. Add a short **Summary** (up to 500 characters).
4. Write the article in **Body (Markdown)** using Markdown syntax.
5. Leave **Project (optional)** as "Organization-wide (no project)" or pick a project.
6. Choose a **Visibility**. The default is Organization. (The editor also shows a "Tags (comma-separated)" field, but tags typed there are not saved today, so skip it; have an agent or the API set tags instead.)
7. Click **"Publish"** to create and activate the article in one step. (To keep working on it privately first, click **"Save as Draft"**, then later open the article and click **Publish** in the lifecycle row.)

Result: The article is Active and visible according to its visibility setting. It appears in Browse, Search, and the Graph, and it has an expiry date computed from the effective policy.

Related: Lifecycle actions; Create and edit an article. An agent does the same with `beacon_create` then `beacon_publish`, or in one idempotent call with `beacon_upsert_by_slug`.

Story: Find an answer by meaning

Who: Anyone looking for knowledge without knowing exact keywords. **Goal:** Locate the right article using a natural-language question. **Before you start:** Be logged in. Articles must already exist and be visible to you.

Steps

1. Click **Search** in the sidebar.
2. Type your question in the primary input (placeholder "Search Beacons...").
3. If results are too broad, open **"Advanced filters"** and narrow by **Status**, **Tags**, **Project:**, or **Freshness**.
4. Watch the "~N Beacons match" count to confirm you have narrowed sensibly.
5. Read the match-source badges on each result ("Semantic match", "Tag expansion", "Link traversal", "Keyword match") to judge why it matched.
6. Click the best result to open the article.

Result: You land on the article's detail page with the relevant passage highlighted.

Related: Saved queries; the agent equivalents are `beacon_search` and `beacon_search_context`.

Story: Save and reuse a search

Who: Anyone who runs the same lookup repeatedly. **Goal:** Keep a search configuration to reopen later. **Before you start:** Have a useful search built in Search.

Steps

1. In Search, configure your query and filters as you want them.
2. Click **"Save query"** in the footer.
3. In the "Save Search Query" dialog, enter a **Name** and pick a **Scope**: "Private (only me)", Project, or Organization.
4. Confirm to save.
5. Later, open the **"Saved queries"** dropdown at the top right and click your saved entry to load it.

Result: The saved query reloads its full configuration with one click. Organization-scoped queries are reusable by teammates.

Related: Search. Agents use `beacon_query_save`, `beacon_query_list`, `beacon_query_get`, and `beacon_query_delete`.

Story: Link related articles

Who: A knowledge owner organizing related content. **Goal:** Connect two articles with a typed relationship. **Before you start:** Links are created through MCP tools or the API; there is no link-creation control in the Beacon SPA, so this story runs through an agent or integration with edit access to the source article.

Steps

1. An agent (or API caller) invokes `beacon_link_create` with the source article, the target article, and a link type: Related To, Supersedes, Depends On, Conflicts With, or See Also.
2. Open the source article's detail page and confirm the new link appears under **Linked Beacons**.
3. Open the **Graph** or click **"View in Graph"** to see the new edge in context.

Result: The two articles are connected. The edge shows in the Graph with its type label, and readers can navigate between them from **Linked Beacons**.

Related: Knowledge Graph; Links between articles. Agents use `beacon_link_create` and `beacon_link_remove`.

Story: Explore how knowledge connects

Who: Someone trying to understand a topic and its dependencies. **Goal:** Walk the graph outward from a starting article. **Before you start:** Have at least a few linked or tagged articles.

Steps

1. Click **Graph** in the sidebar, or click **"View in Graph"** from an article's detail page.
2. If you started without a focal node, click a hub in **"Hub Beacons"** to focus it.
3. Set **Hops:** to 2 or 3 to widen the view.
4. Toggle the implicit-edges eye control to show or hide Tag Affinity edges.
5. Click a node to open its **NodePopover**, then click **"Explore from here"** to re-center on a neighbor, or **"View Beacon"** to open it.

Result: You see the network around your topic and can move through related articles without losing your place.

Related: Link related articles. Agents use `beacon_graph_neighbors`, `beacon_graph_hubs`, and `beacon_graph_recent`.

Story: Challenge an article that looks wrong

Who: Any reader who spots an inaccuracy. **Goal:** Flag an Active article so an owner reviews it. **Before you start:** Be able to read the article (challenging needs read access plus challenge permission).

Steps

1. Open the article's detail page (or find it in the dashboard's **At-Risk** tab).
2. Click **Challenge** in the lifecycle row, or **Challenge** on the At-Risk row.
3. Read the confirmation ("Flag this beacon for review. It will move to Pending Review status.") and confirm.

Result: The article moves to **Pending Review** and surfaces for an owner to verify, update, or retire.

Related: Keep knowledge fresh; lifecycle actions. The agent tool is `beacon_challenge`.

Story: Keep knowledge fresh (governance sweep)

Who: A knowledge owner, admin, or governance lead. **Goal:** Verify what is about to expire and clear out old archived articles. **Before you start:** Have edit access to the articles you will act on.

Steps

1. Click **Dashboard** in the sidebar to open **Fridge Cleanout**.
2. On **Overview**, check the **"Freshness Score"** and **"At-Risk (7 days)"** cards.

3. Open the **At-Risk** tab. For an article that is still correct, click **Verify**; for one that is wrong, click **Challenge**. To verify several, tick their checkboxes and click **"Verify Selected (n)"**.
4. Open the **Archived** tab. For each old article, click **Restore** to bring it back to Active or **Retire** to remove it. To retire several, tick their checkboxes and click **"Retire Selected (n)"**.

Result: At-risk articles are verified (their expiry reset) and stale archived articles are retired or restored. The Freshness Score improves.

Related: Expiry Policy Settings; lifecycle actions. Agents use `beacon_verify` and `beacon_retire`; agent verifications appear under the **Agent Activity** tab as "Verified by <owner>" rows with the verification count (v<count>) and status.

Story: Set the freshness policy for a team

Who: An org Admin, Owner, or SuperUser. **Goal:** Control how long knowledge stays trusted before it needs re-verification. **Before you start:** Have policy-edit permission for the scope you want to change (System policy requires SuperUser).

Steps

1. Click **Beacon Settings** in the sidebar.
2. Review the **"Effective Policy (Your Context)"** card to see what currently applies.
3. In the **Organization Policy** editor (or **Project Policy** after picking a project), set Min, Max, and Default expiry (in days) and the Grace period.
4. Confirm the values satisfy min <= default <= max and stay within the parent scope's bounds.
5. Click **"Save Policy"** and read the success or warning text.

Result: New and republished articles in that scope compute their expiry from the updated policy, and the daily sweep enforces the grace period.

Related: Fridge Cleanout. Agents use `beacon_policy_get`, `beacon_policy_set`, and `beacon_policy_resolve`.

Story: Discuss and attach evidence to an article

Who: A reviewer or contributor adding context. **Goal:** Leave a comment and attach supporting evidence. **Before you start:** Be able to read the article; uploading needs edit access.

Steps

1. Open the article's detail page.
2. In the Comments section, type in the box labeled "Add a comment. Markdown supported." and click **"Post Comment"**. Use **Reply** to respond in a thread.
3. In the Attachments panel, drop a file or click **"Choose File"** (max 10 MB: images, PDF, text, office docs).

4. Confirm the file appears in the list, with **Download** and **Delete** actions.

Result: The discussion and evidence are recorded on the article for the next reader or reviewer.

Related: Read an article. Comments and attachments have no MCP tools, so this story is human-only.

Story: Ingest knowledge as an agent (idempotent import)

Who: An AI agent or an integration importing knowledge. **Goal:** Create or update articles repeatably without making duplicates. **Before you start:** The agent runs through the MCP server with a service account and the right permissions (the `beacon.*` tool prefix must be allowed by its agent policy).

Steps

1. The agent calls `beacon_upsert_by_slug` with a stable `slug`, plus `title`, `body_markdown`, and any of `summary`, `visibility`, `project_id`, `metadata`, or `change_note`.
2. The tool returns `data`, a `created` flag, and an `idempotency_key`.
3. The agent reads the `created` flag: `true` means a new article was inserted, `false` means an existing one was updated in place.
4. On the next run with the same slug, the agent calls the tool again; the existing article is updated rather than duplicated.

Result: Knowledge stays in sync with the source system across repeated runs. Each upsert emits an `entry.upserted` Bolt event carrying the `created` flag, so automations can react to inserts and updates differently. The write is attributed to the agent in the unified activity view for later audit.

Related: Working with AI agents. A human can review the result on the article's detail page.

Related

- **Bam** (`/b3/`) - Beacon shares Bam's login and pulls its project list from Bam. Sign in to Bam before using Beacon.
- **Bolt** (`/bolt/`) - Beacon publishes lifecycle events (source `beacon`): `entry.upserted`, `beacon.created`, `beacon.updated`, `beacon.published`, `beacon.verified`, `beacon.challenged`, `comment.created`, `attachment.uploaded`, and `beacon.expired` (emitted when an article is retired). Use Bolt to automate reactions to knowledge changes.
- **Cross-app search** - Beacon articles surface in the platform-wide `search_everything` and `resolve_references` retrieval, so other apps and agents can find and cite knowledge subject to the same visibility rules.
- `docs/apps/beacon/mcp-tools.md` - the full Beacon MCP tool reference.
- `docs/apps/beacon/guide.md` - the Beacon product guide.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What are the five statuses a Beacon moves through in its lifecycle?

a) Draft, Active, Pending Review, Archived, Retired
b) New, Open, Reviewed, Closed, Deleted
c) Draft, Published, Stale, Expired, Removed
d) Pending, Active, Verified, Archived, Locked

2. A Beacon's summary is limited to how many characters?

a) 140 characters
b) 280 characters
c) 500 characters
d) There is no limit

3. What are the four freshness states a Beacon can show?

4. Which visibility level is visible only to the owner or creator?

a) Public
b) Organization
c) Project
d) Private

5. What does challenging an Active article do to its status?

6. Which of these is NOT one of Beacon's typed link types?

a) Related To
b) Supersedes
c) Blocks
d) Conflicts With

7. What does verifying an article do besides recording a review?

8. What is the editor's default visibility when you create a new Beacon?

a) Public
b) Organization
c) Project
d) Private

9. What is the attachment size limit on an article?

a) 2 MB
b) 5 MB
c) 10 MB
d) 25 MB

10. In what order do expiry policies resolve when computing your effective policy?

11. How many tags can you add at a time through the tag write endpoints?

a) 1 to 5
b) 1 to 10
c) 1 to 20
d) Up to 50

12. What does the daily expiry sweep do to Active articles when they expire, and to

Pending Review articles after the grace period?

13. What does the beacon_upsert_by_slug tool return so an agent can tell an insert from an update?

a) A status code
b) A created flag
c) A version number
d) A diff

14. Comments and attachments have no MCP tools. What does that mean for agents?

CHAPTER 5

Bearing

Set the objective, watch the drift, act in time.

Bearing is where your team's objectives get measurable. You set a Period, add Goals inside it, and break each goal into Key Results with a start value, a target, and a current value. As you check in, Bearing rolls those numbers into one progress percentage and compares it against where you should be by now, so a goal that is slipping shows up while there is still time to fix it. By the end of this chapter you will be able to stand up a quarter, run weekly check-ins, triage what is behind, and close the period out.

At a glance

- Periods that scope the whole UI to one quarter, half, year, or custom span
- Goals scoped to Organization, Team, Project, or Individual
- Key Results in Number, Percentage, Currency, or Yes/No, clamped 0 to 100 percent
- Automatic status from actual progress against expected, with an At Risk triage view
- Agent and REST reach for linking key results to Bam work, reports, and status overrides the UI does not surface

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Goals Dashboard . My Goals . At Risk . Periods . Goal Detail . Create a Goal

> *Bearing is the goals and OKR tracker for BigBlueBam. Teams use it to set objectives for a time period, attach measurable key results, check in on progress, and surface the goals that are falling behind before the period ends.*

Overview

Bearing organizes work around objectives. You create a **Period**[39] (a named time box such as a quarter), add **Goals** inside it, and break each goal[40] into **Key Results** that carry a start value, a target value, and a current value. As you record progress on the key results, Bearing rolls those numbers up into a single goal progress percentage and compares it against where you should be by now, so a goal that is drifting becomes visible while there is still time to act.

> ***Bearing compares actual progress against where you should be by now, so a drifting goal becomes visible while there is still time to act.***

The product is built for small to medium teams running a regular OKR cadence: set goals at the start of a period, check in weekly with status updates, triage the goals that are at risk[41], and close the period out at the end. Goals can be scoped to the whole organization, a team, a project, or an individual, so the same period can hold company objectives next to team objectives.

Bearing can read progress from your real work when an agent or API caller links a key result to a Bam epic, project, or sprint. As linked Bam tasks reach a done state, a background job recomputes the key result and its goal automatically. That linking is available through the MCP tools and REST, not through a human screen in this build.

Bearing signs you in with your existing BigBlueBam (Bam) session, so you reach it the same way you reach the other apps in the suite.

Key concepts

- **Period** - A named time box that goals live inside, for example "Q2 2026". A period has a type, a start date, an end date, and a status. The whole Bearing UI is scoped to one selected period at a time.
- **Period status**[42] - The lifecycle of a period. The Bearing UI shows these as **draft**, **active**, **completed**, and **archived**. You activate a period to make it the working scope[43] and complete it when the period is over.

[39] **Period.** A named time box that goals live inside, for example "Q2 2026". A period has a type, a start date, an end date, and a status. The whole Bearing UI is scoped to one selected period at a time.
[40] **Goal.** An objective owned by one user inside a period. A goal has a title, an optional description, a scope, a status, and a cached progress percentage that is the average of its key results.
[41] **At Risk.** The view that lists goals whose status is at_risk or behind, sorted by how far they have fallen behind expected progress.
[42] **Period status.** The lifecycle of a period. The Bearing UI shows these as **draft**, **active**, **completed**, and **archived**. You activate a period to make it the working scope and complete it when the period is over.
[43] **Scope.** Who a goal is for: **Organization**, **Team**, **Project**, or **Individual**. The dashboard can filter and group goals by scope.

- **Goal** - An objective owned by one user inside a period. A goal has a title, an optional description, a scope, a status, and a cached progress percentage that is the average of its key results.
- **Scope** - Who a goal is for: **Organization**, **Team**, **Project**, or **Individual**. The dashboard can filter and group goals by scope.
- **Goal status**[44] - Where a goal stands: **draft**, **on_track**, **at_risk**, **behind**, **achieved**, **missed**, or **cancelled**. Bearing derives status automatically from how actual progress compares to expected progress[45], unless someone overrides it.
- **Key Result (KR)** - A measurable outcome under a goal. Each KR has a metric type (**Number**, **Percentage**, **Currency**, or **Yes/No**), a start value, a target value, a current value, and an optional unit. KR progress is clamped between 0 and 100 percent.
- **Check-in**[46] - Recording a new current value for a key result. Each check-in updates the KR's progress, rolls up to the goal's progress, and writes a point-in-time snapshot for the sparkline and history chart.
- **Status update**[47] - A status post against a goal. It records a status tag, an optional body, and a snapshot of the goal's status and progress at the time it was posted.
- **Watcher**[48] - A user subscribed to a goal who is emailed when the goal's status changes.
- **Expected progress** - Where a goal should be based on how much of the period has elapsed. Bearing compares actual against expected to decide whether a goal is on track, at risk, or behind.
- **At Risk** - The view that lists goals whose status is at_risk or behind, sorted by how far they have fallen behind expected progress.

> **NOTE**
> A goal's overall progress is the average of its key results, and each key result's progress is clamped between 0 and 100 percent. A current value past the target does not push a key result above 100.

Where to find it

Bearing is served at `/bearing/`. Reach it from the BigBlueBam app switcher the same way you reach the other apps.

Before Bearing is usable:

---

[44] **Goal status.** Where a goal stands: **draft**, **on_track**, **at_risk**, **behind**, **achieved**, **missed**, or **cancelled**. Bearing derives status automatically from how actual progress compares to expected progress, unless someone overrides it.

[45] **Expected progress.** Where a goal should be based on how much of the period has elapsed. Bearing compares actual against expected to decide whether a goal is on track, at risk, or behind.

[46] **Check-in.** Recording a new current value for a key result. Each check-in updates the KR's progress, rolls up to the goal's progress, and writes a point-in-time snapshot for the sparkline and history chart.

[47] **Status update.** A status post against a goal. It records a status tag, an optional body, and a snapshot of the goal's status and progress at the time it was posted.

[48] **Watcher.** A user subscribed to a goal who is emailed when the goal's status changes.

- You must be logged in to BigBlueBam. If you are not, Bearing shows a "Please log in to BigBlueBam first" screen with a link to `/b3/`.
- You work inside your active organization. Bearing scopes everything to that org.
- **At least one Period must exist and be selected.** The dashboard and the at-risk view are gated on a selected period. A fresh org has no periods, so the first thing to do is create one on the **Periods** page and select it in the sidebar.

> **TIP**
> The Dashboard and the At Risk view are both gated on a selected period. In a fresh org with no periods, create one on the Periods page and select it in the sidebar first, or those views will only show a prompt.

Press **?** anywhere outside a text field to open the in-app help viewer.

Feature reference

The Bearing sidebar carries a brand mark, a **period scope selector** dropdown, and four navigation items with these exact labels: **Dashboard**, **My Goals**, **At Risk**, and **Periods**. The period scope selector sets the one period that the whole UI is scoped to; your choice is remembered between visits, and Bearing auto-selects the first active period if you have not chosen one.

Goals Dashboard

The Dashboard is the home view at `/`. It shows the goals in the selected period, with filters and a search box.

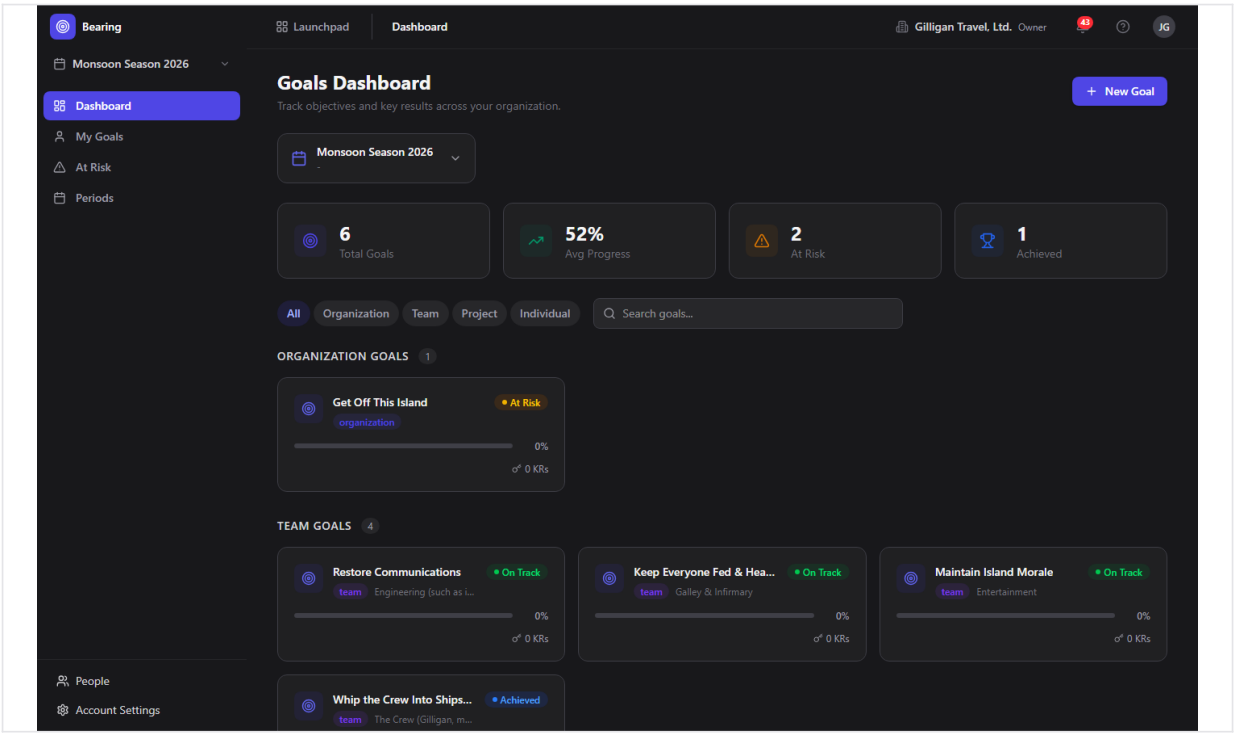


Figure 5.1 Goal dashboard

What you see, top to bottom:

- The header **"Goals Dashboard"** with the subtitle "Track objectives and key results across your organization." and a **New Goal** button.
- A period selector card showing the selected period's name, date range, and time remaining.
- A stats row with four cards: **Total Goals**, **Avg Progress**, **At Risk**, and **Achieved**. These read from the selected period's report and populate with the period's real totals.
- Scope tabs: **All**, **Organization**, **Team**, **Project**, **Individual**.
- A search box with placeholder "Search goals...".

- A grid of goal cards. With the **All** tab selected, cards are grouped by scope. Each card shows the goal's title, scope badge, project or team, a progress bar (actual against expected), the owner avatar, and the key result count.

To open a goal, click its card. To browse a different period, change the selection in the sidebar period selector.

If no period is selected you see a "Select a period" prompt. If the period has no goals you see "No goals found" with a **Create First Goal** button.

My Goals

My Goals at `/my-goals` lists every goal you own across all periods, not just the selected one.

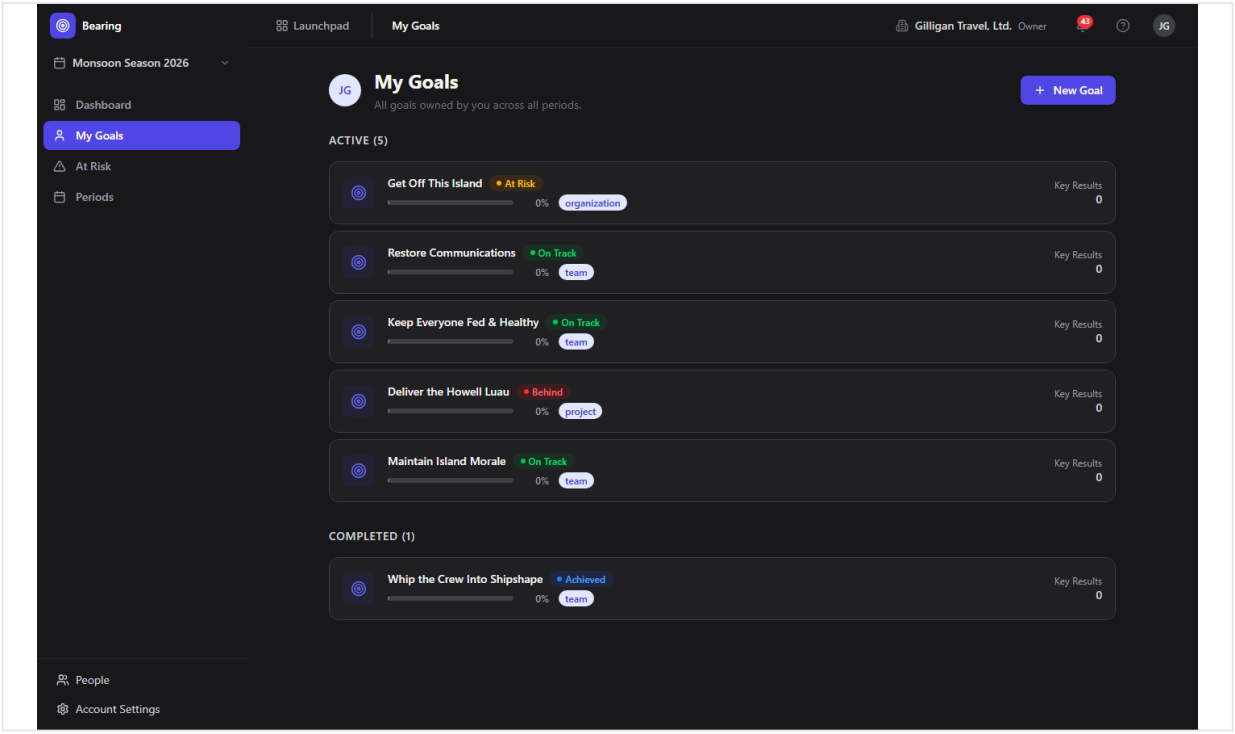


Figure 5.2 My goals

- The header reads **"My Goals"** with the subtitle "All goals owned by you across all periods." and a **New Goal** button that takes you to the Dashboard to create one.
- Goals are split into **Active** and **Completed** sections, where completed means a goal that is achieved or missed.
- Each row shows the goal icon, title, status badge, progress bar, period and scope badges, and the key result count. Click a row to open the goal.

If you own no goals you see "No goals assigned to you" with a **Go to Dashboard** button.

At Risk

At Risk at `/at-risk` is the triage view. It lists the goals in the selected period whose status is at_risk or behind, sorted so the goals that have fallen furthest behind appear first.

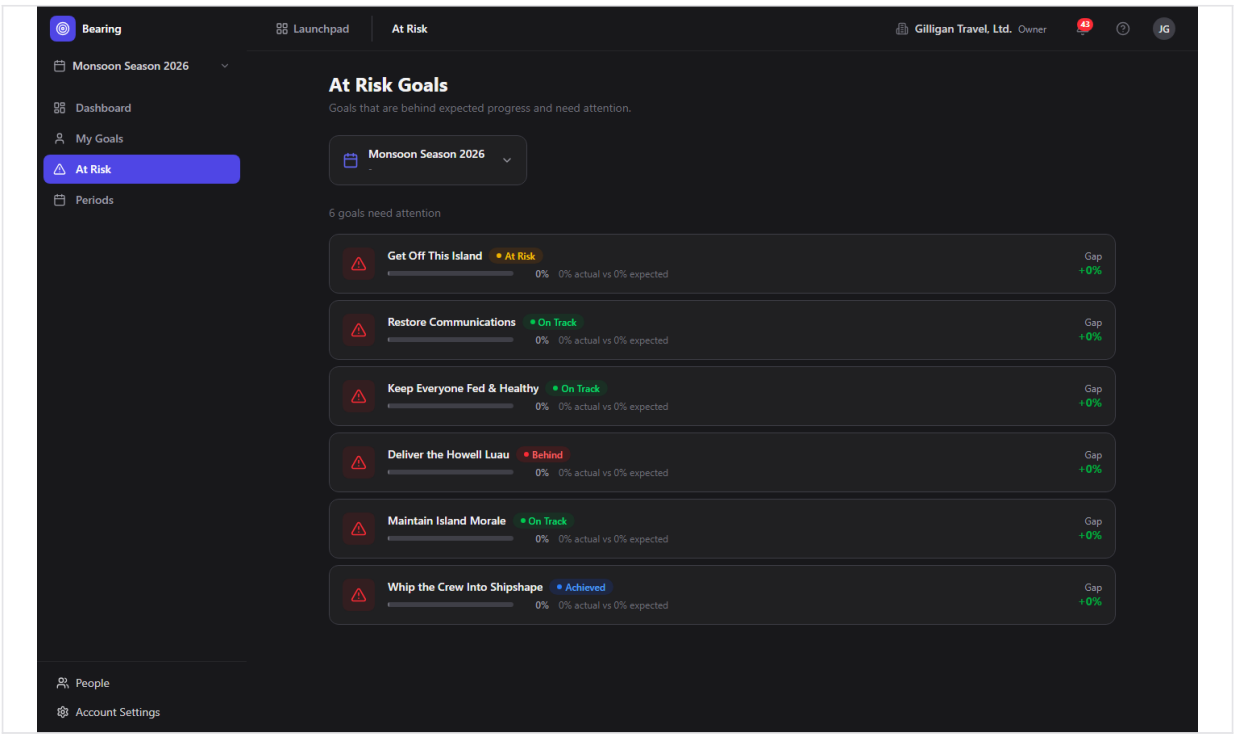


Figure 5.3 *At-risk goals*

- The header reads **"At Risk Goals"** with the subtitle "Goals that are behind expected progress and need attention.".
- Each row shows a red alert badge, the goal title, its status badge, a progress bar, an "actual vs expected" readout, a color-coded **Gap**, and the owner avatar. Click a row to open the goal.

If nothing is behind you see "All goals are on track".

Periods

The Periods page at `/periods` is where you create and manage the time boxes that hold goals.

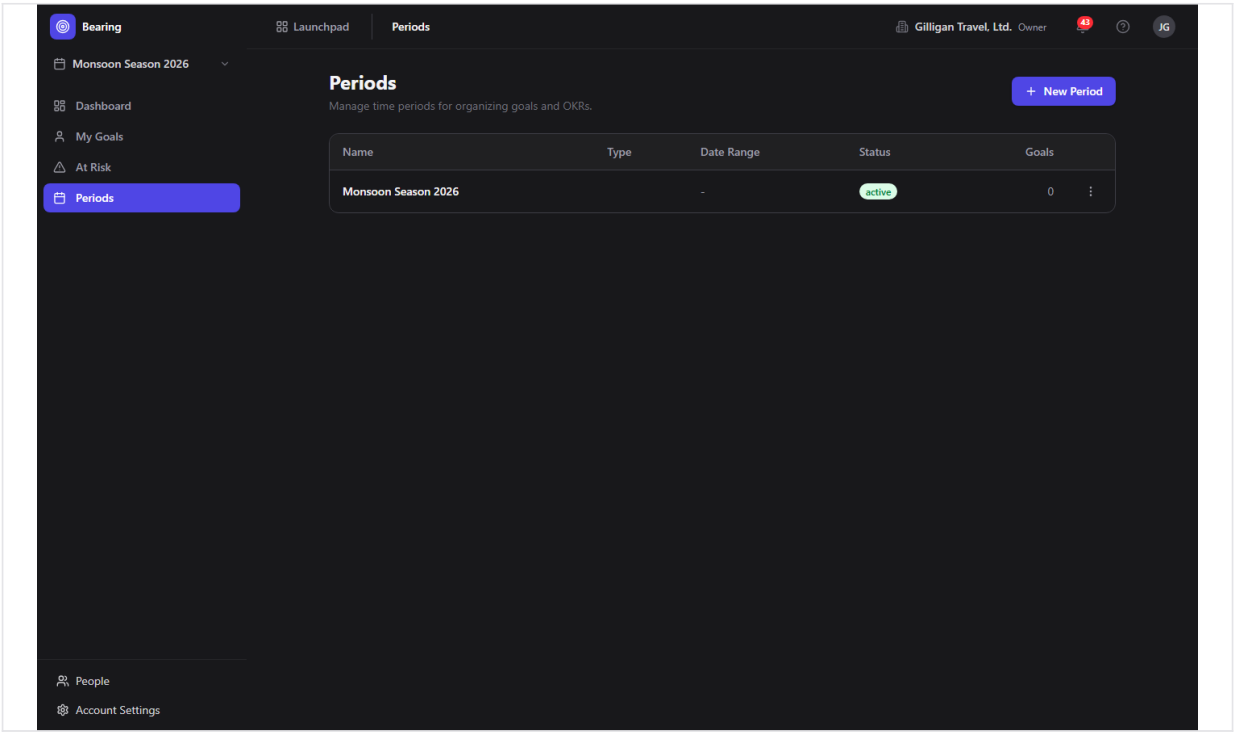


Figure 5.4 Periods

- The header reads **"Periods"** with the subtitle "Manage time periods for organizing goals and OKRs." and a **New Period** button.
- A table with columns **Name**, **Type**, **Date Range**, **Status**, and **Goals**, plus a row menu.
- Each row has a "..." menu with **Edit**, an **Activate** item (shown only while the period's status is planning), a **Complete** item (shown only when the period is active), and **Delete**.

To create a period:

1. Click **New Period**. The Period Form dialog opens.
2. Type a **Name**, for example "Q2 2026".
3. Choose a **Type**: **Quarter**, **Half Year**, **Year**, or **Custom**.
4. Set the **Start Date** and **End Date**. The start date must be before the end date.
5. Click **Create Period**.

To edit a period, choose **Edit** from the row menu. The same dialog opens with a **Save Changes** button.

To activate a newly created period so it becomes the working scope, choose **Activate** from the row menu (shown while the period is in planning). To complete a period when it is over, choose **Complete** from the row menu (available only while the period is active). To delete a period, choose **Delete** and confirm. Deleting is blocked with a conflict error if the period still has goals.

> **WARNING**
> Deleting a period that still holds goals fails with a conflict error. Move or delete the goals inside it first, then delete the period.

Goal Detail

Open a goal from any list to reach its detail page at `/goals/:id`. This is where most day-to-day work happens.

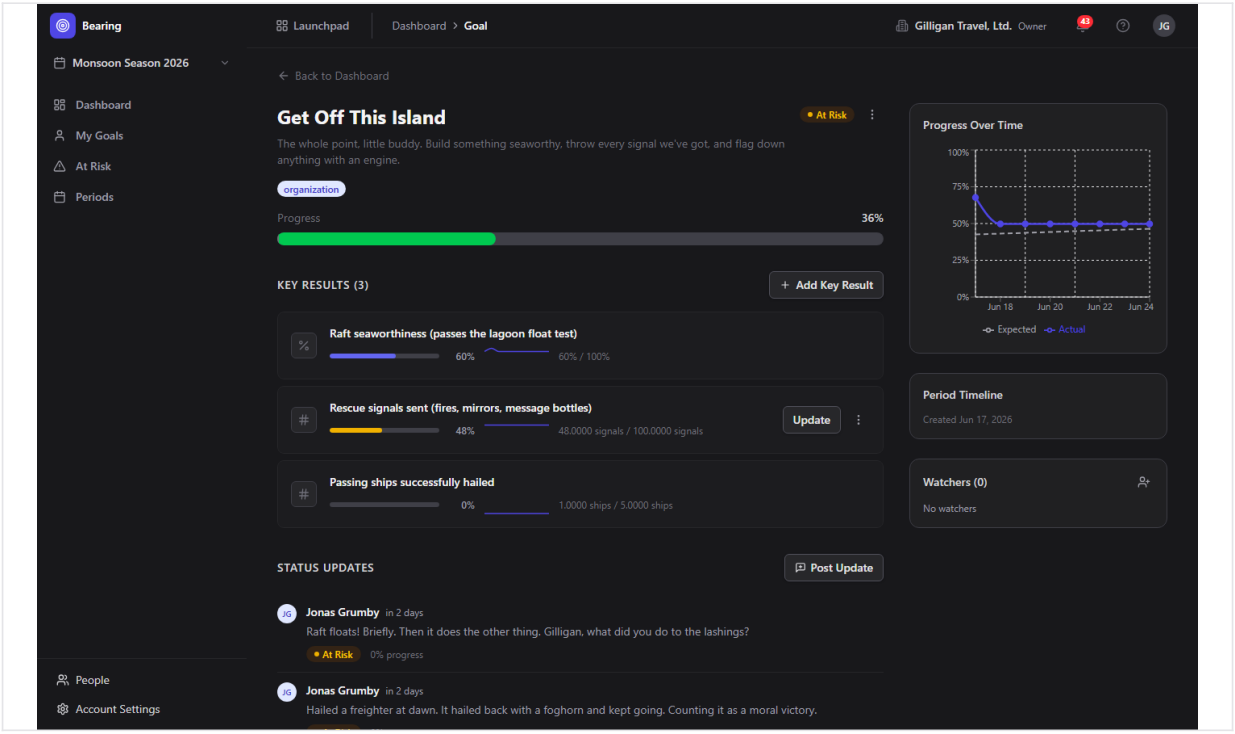


Figure 5.5 Goal detail

The page has a **Back to Dashboard** link, the goal title and description, a status badge, and a "..." menu with **Edit** and **Delete Goal**.

To edit a goal's title or description:

1. Open the "..." menu and choose **Edit**.
2. Change the **Title** input and the description box.
3. Click **Save**, or **Cancel** to discard.

Below the header is a meta row with the owner avatar and name, the scope badge, the period-name badge, and the project-name badge, followed by a progress bar reading "{n}% (expected: {m}%)".

The right sidebar holds a **Progress Over Time** chart that plots the goal's actual progress against expected over the period, a **Period Timeline** card showing the goal's creation date, and the **Watchers** card.

Note: the **Period Timeline** card's time-remaining badge is inert in this build (it is fed an empty end date), so it does not count down. The progress chart, the progress-bar values, and the **Watchers** card all work as described.

Create a Goal

Goals are created from the Dashboard.

1. On the **Dashboard**, click **New Goal**. The dialog opens with the title "Create Goal" and the description "Set a new objective for this period.".
2. Type a **Title**, for example "Increase customer retention by 15%".
3. Optionally fill in **Description (optional)** with why the goal matters.
4. Choose a **Scope**: **Organization**, **Team**, or **Project** (the dialog defaults to **Team**).
5. Click **Create Goal**.

Bearing creates the goal in the selected period, assigns you as the owner, and opens the new goal's detail page.

Note that the Create Goal dialog does not offer an **Individual** scope or an owner field; new goals are owned by you. Reassigning a goal to another owner is done through the AI agent tool `bearing_goal_update` or REST, not the human UI.

Add and edit Key Results

Key Results live in the **Key Results (n)** section of the goal detail page.

To add a key result:

1. On the goal detail page, click **Add Key Result** (or **Add First Key Result** if there are none yet). The dialog opens with the title "Add Key Result" and the description "Define a measurable outcome for this goal.".
2. Type a **Title**, for example "Increase monthly active users".
3. Choose a **Metric Type**: **Number**, **Percentage**, **Currency**, or **Yes/No**.
4. Set the **Start Value** and the **Target Value**.
5. For **Number** or **Currency** metrics, optionally set a **Unit (optional)**, for example "users", "$", or "EUR".
6. Click **Add Key Result**.

To edit a key result, hover its row, open the "..." menu, and choose **Edit**. The same dialog opens with a **Save Changes** button. To delete one, open the "..." menu and choose **Delete**, then confirm.

Each key result row shows a metric icon, the title, a progress bar, a sparkline of recent values, and a "current / target" readout. The goal's overall progress is the average of its key results' progress, and that progress renders correctly throughout Bearing (see the note under Record progress below).

Record progress on a Key Result

You record progress (a check-in) by setting a key result's current value.

1. On the goal detail page, hover the key result's row.
2. Click **Update**.
3. Type the new current value in the inline number field.
4. Click **Save**, or **Cancel** to back out. Pressing Enter saves and Escape cancels.

A check-in records the new value, updates the key result's progress, recomputes the goal's overall progress as the average of its key results, re-derives the goal's status, and writes a snapshot row that feeds the KR sparkline and the goal history chart.

Goal and key-result progress display the real percentage. Earlier builds capped the stored progress value at a precision that overflowed for any figure of 10 percent or more, which left progress bars stuck at 0 percent; that storage limit has been widened, so bars and percentages reflect the actual numbers.

Post a status update

Status updates are the weekly check-in narrative on a goal.

1. On the goal detail page, in the **Status Updates** section, click **Post Update**. The dialog opens with the title "Post Status Update" and the description "Share progress with your team.".
2. Choose a status chip: **On Track**, **At Risk**, **Behind**, or **Achieved**.
3. Type your note in the **Update** box, for example what changed and any blockers.
4. Click **Post Update**.

Your update appears in the feed with your name, a relative timestamp, the body, a status badge for the status you chose, and the goal's progress at the time you posted. Posting an update snapshots the goal's status and progress.

Watch a goal

The **Watchers** card on the goal detail page subscribes people to status-change emails.

1. On the goal detail page, find the **Watchers (n)** card in the right sidebar.
2. Click the add (person-plus) icon to reveal the input.
3. Type a user ID or email into the **User ID or email** field.
4. Click the + button, or press Enter, to subscribe that person.
5. To stop watching, hover a watcher chip and click the **X**.

The form subscribes whoever you enter, resolving the value to an active member of your org by user ID first, then by email. To remove a watcher who is not you, you must be the goal's owner or an org admin or owner.

> **TRY IT**
> On any goal's detail page, open the Watchers card, click the person-plus icon, type a

> teammate's email, and press Enter to subscribe them to that goal's status-change emails.

When a watched goal's status changes, a background job emails the watchers (using real email when SMTP is configured for the deployment), with an unsubscribe link per recipient.

Reports and CSV export

Bearing can produce period, at-risk, and owner reports, plus CSV exports of goals and key results. These are available through the API and the MCP tools, not through a screen in the Bearing SPA.

- A **period report** is a markdown summary table with per-goal and per-key-result detail for one period.
- An **at-risk report** is a markdown list of every at_risk and behind goal across the org.
- An **owner report** is a markdown rollup of one user's goals grouped by period.
- **CSV exports** dump all goals (`goals-export.csv`) or all key results (`key-results-export.csv`).

To produce any of these, use the AI agent tools `bearing_report` and `bearing_at_risk` described next, or call the REST routes directly.

Working with AI agents

Bearing exposes a set of MCP tools so AI agents can drive the same goals and key results you manage in the UI, and reach a few capabilities the human UI does not surface (linking, reports, status override, and CSV-style detail). Agents forward your bearer token and org context, and many tools accept human labels (a period name, a goal or key result title, an owner's email) in place of UUIDs. When a key result title is shared across multiple goals, the resolver fails closed and asks the agent to disambiguate with a UUID.

Period tools:

- `bearing_periods` - list periods, optionally filtered by status or year.
- `bearing_period_get` - one period plus its stats (goal count, average progress, at-risk count).
- `bearing_period_create` - create a period (cadence quarter, half, year, month, or custom) with start and end dates.
- `bearing_period_update` - change a period's name, type, dates, or status (set status to archived to archive it).
- `bearing_period_delete` - delete a period.
- `bearing_period_activate` - make a period the live planning window.
- `bearing_period_complete` - close a period out at the end of the cadence.

Goal tools:

- `bearing_goals` - list goals filtered by period, scope, owner, or status.
- `bearing_goal_get` - one goal with its key results and progress detail.

- `bearing_goal_create` - create a goal, naming the period by label and the owner by email.
- `bearing_goal_update` - update a goal, including reassigning the owner. This is the only way to change a goal's owner, since the UI has no owner picker.
- `bearing_goal_delete` - delete a goal and its key results.
- `bearing_goal_status_override` - force a goal's status (for example to at_risk or achieved), bypassing the automatic progress-derived status. There is no human screen for this; it is agent and REST only.
- `bearing_goal_updates` - list the status updates posted on a goal.
- `bearing_goal_history` - get a goal's point-in-time progress snapshots.
- `bearing_goal_watchers` - list a goal's watchers.
- `bearing_goal_watch` - add a watcher to a goal.
- `bearing_goal_unwatch` - remove a watcher (yourself, or anyone if you are the owner or an org admin or owner).

Key result tools:

- `bearing_kr_list` - list the key results under a goal.
- `bearing_kr_get` - get one key result.
- `bearing_kr_create` - add a key result to a goal.
- `bearing_kr_update` - update a key result's definition and, when a current value is supplied, record a value check-in.
- `bearing_kr_delete` - delete a key result.
- `bearing_kr_link` - link a key result to a Bam epic, project, sprint, or task so its progress tracks real delivery. There is no human screen for this in the current build; it is agent and REST only.
- `bearing_kr_links` - list the Bam-entity links on a key result.
- `bearing_kr_unlink` - remove a link from a key result.
- `bearing_kr_history` - get a key result's value and progress snapshot history.

Update and report tools:

- `bearing_update_post` - post a goal status update, the same as the **Post Update** dialog.
- `bearing_report` - generate a period, at-risk, or owner report as markdown or JSON. There is no human reports screen; this is how reports are produced.
- `bearing_at_risk` - return the org-wide list of at-risk and behind goals.

When an agent links a key result to Bam work with `bearing_kr_link`, a background recompute job keeps that key result and its goal current as the linked Bam tasks reach a done state. Bearing also emits events (goal created, updated, status changed, achieved, deleted, watcher added or removed; key result created, updated, value updated, linked, deleted; period activated, completed, archived) that Bolt automations can react to.

> ***Link a key result to Bam delivery and it moves on its own as the work lands, no manual check-in required.***

Beyond the Bearing-specific tools, agents working here also use the cross-cutting platform surface that every BigBlueBam app shares:

- **Identity and heartbeat.** Agent and service callers are first-class identities (`users.kind` is human, agent, or service), and runners report liveness and capabilities through `agent_heartbeat`, `agent_self_report`, and `agent_audit`. Goals, key results, and updates created by an agent are attributed to that agent in the same feeds humans read.
- **Approval queues.** A risky or far-reaching change (for example bulk-creating a quarter of goals, or overriding statuses) can be staged for a human through `proposal_create` / `proposal_list` / `proposal_decide` instead of applied directly.
- **Unified activity and cross-app search.** `search_everything` fans out across apps so a Bearing goal turns up alongside related Bam tasks, Bond deals, and Beacon docs, and the unified activity view threads Bearing changes into one timeline.
- **Visibility preflight.** Before an agent cites a Bearing goal in a shared surface such as a Banter post or a Brief doc, it calls `can_access` for each entity and drops anything the asker is not allowed to see.
- **Policies and webhooks.** Every service-account tool call passes the platform `agent_policies` allowlist and kill switch before it runs, so an agent only reaches the Bearing tools its policy permits, and outbound webhooks can push subscribed Bearing events to an agent runner.

For reviewers: agent-created goals and updates show up in the same lists and feeds as human ones, attributed to the agent. See the Bearing MCP-tools reference under `docs/apps/bearing/` for the full catalog and parameters.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. Your presence in Bearing shows in the Bureau virtual office, so teammates can see where you are and gather without booking a call. The suite's see-who-is-here strips, one-tap huddles, and real-time co-editing live on its collaborative surfaces. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Set up your first quarter

Who: A team lead standing up OKRs in a fresh org. **Goal:** Have an active period selected so goals can be created. **Before you start:** You are logged in to BigBlueBam and Bearing is open. No period exists yet.

Steps

1. In the sidebar, click **Periods**.
2. Click **New Period**.
3. In the **Name** field type your period name, for example "Q2 2026".
4. Choose a **Type** of **Quarter**.
5. Set the **Start Date** and **End Date** to cover the quarter.
6. Click **Create Period**.
7. In the period's "..." menu, choose **Activate**.
8. In the sidebar period scope selector at the top, select the period you just created.

Result: The period is active and selected, and scopes the whole UI. The Dashboard now shows this period and is ready for goals.

Related: You can also start working in a period just by selecting it in the sidebar selector. See "Provision a quarter with an agent" below for the agent counterpart.

Story: Create your first objective

Who: A goal owner setting an objective for the quarter. **Goal:** Create a goal in the current period and open it. **Before you start:** A period exists and is selected in the sidebar.

Steps

1. In the sidebar, click **Dashboard**.
2. Click **New Goal**.
3. Type a **Title**, for example "Increase customer retention by 15%".
4. Optionally fill in **Description (optional)**.
5. Choose a **Scope** of **Organization**, **Team**, or **Project**.
6. Click **Create Goal**.

Result: The goal is created in the selected period with you as the owner, and Bearing opens its detail page.

Related: Add Key Results next. To set a different owner, use the agent tool `bearing_goal_create` with an owner email, or `bearing_goal_update` afterward.

Story: Break a goal into key results

Who: A goal owner who wants measurable outcomes. **Goal:** Add the key results that define what success means for the goal. **Before you start:** You are on the goal's detail page.

Steps

1. In the **Key Results** section, click **Add Key Result** (or **Add First Key Result**).
2. Type a **Title**, for example "Increase monthly active users".
3. Choose a **Metric Type**: **Number**, **Percentage**, **Currency**, or **Yes/No**.
4. Set the **Start Value** and **Target Value**.
5. For a **Number** or **Currency** metric, optionally set a **Unit (optional)**.
6. Click **Add Key Result**.
7. Repeat for each measurable outcome.

Result: Each key result appears as a row with a progress bar and a "current / target" readout. The goal's progress is the average of its key results and renders the real percentage.

Related: Record progress on a key result next. To wire a key result to real Bam delivery, an agent can use `bearing_kr_link`.

Story: Record weekly progress

Who: A goal owner doing a weekly check-in. **Goal:** Update a key result's current value so the goal's progress and status recompute. **Before you start:** The goal has at least one key result.

Steps

1. Open the goal's detail page.
2. Hover the key result row and click **Update**.
3. Type the new current value in the inline field.
4. Click **Save**.

Result: The key result's value and progress update, the goal's overall progress and status recompute from the average, and a snapshot is written for the sparkline. Progress shows the true percentage.

Related: Post a status update to add narrative for the team. An agent can record the same check-in with `bearing_kr_update`.

Story: Post a status update for the team

Who: A goal owner sharing progress. **Goal:** Leave a check-in note and a status on the goal. **Before you start:** You are on the goal's detail page.

Steps

1. In the **Status Updates** section, click **Post Update**.
2. Choose a status chip: **On Track**, **At Risk**, **Behind**, or **Achieved**.
3. Type your note in the **Update** box.

4. Click **Post Update**.

Result: The update appears in the feed with your name, the time, the status badge, and the goal's progress at the time you posted. Watchers are notified when the status changes.

Related: Watch a goal to receive its status-change emails. An agent can post the same update with `bearing_update_post`.

Story: Triage the goals that are behind

Who: A team lead reviewing where the quarter is slipping. **Goal:** Find the goals that are at risk or behind and act on the worst ones. **Before you start:** A period is selected and has goals with recorded progress.

Steps

1. In the sidebar, click **At Risk**.
2. Review the list, which is sorted by how far each goal has fallen behind expected progress, with the worst at the top.
3. Read each row's "actual vs expected" readout and **Gap**.
4. Click the goal that needs the most attention to open it.
5. On the goal, record updated key result values and post a status update explaining the plan.

Result: You have identified the goals that need attention and refreshed their progress and narrative.

Related: An agent can pull the same list anytime with `bearing_at_risk`, or generate a markdown at-risk report with `bearing_report`.

Story: Review your own goals across periods

Who: Any individual contributor or lead who owns goals. **Goal:** See every goal you own, current and finished, in one place. **Before you start:** You own at least one goal in any period.

Steps

1. In the sidebar, click **My Goals**.
2. Read the **Active** section for goals still in flight.
3. Read the **Completed** section for goals that are achieved or missed.
4. Click any row to open that goal.

Result: You see all of your goals across every period, grouped by whether they are still active.

Story: Watch a goal you care about

Who: A stakeholder who wants to be told when a goal's status changes. **Goal:** Subscribe someone to a goal's status-change emails. **Before you start:** You are on the goal's detail page.

Steps

1. In the right sidebar, find the **Watchers** card.
2. Click the add (person-plus) icon.

3. Type a user ID or email into the **User ID or email** field.
4. Click the + button, or press Enter, to subscribe that person.

Result: The watcher's avatar appears on the card. When the goal's status changes, a background job emails them.

Related: The form subscribes whoever you enter, resolving the value by user ID first, then by email. An agent can do the same with `bearing_goal_watch`.

Story: Close out the quarter

Who: A team lead wrapping up a period. **Goal:** Mark the period complete and produce a retrospective report. **Before you start:** The period is active and its goals are up to date.

Steps

1. In the sidebar, click **Periods**.
2. Find the period's row and open its "..." menu.
3. Choose **Complete**.

Result: The period is marked completed and a period.completed event fires for any Bolt automations watching it.

Related: For a retrospective, an agent can run `bearing_report` with the period type to produce a markdown summary table with per-goal and per-key-result detail.

Story: Provision a quarter with an agent

Who: An AI agent provisioning OKRs from a brief. **Goal:** Create a period, goals, and key results in one pass without manual UI work. **Before you start:** The agent has a Bearing-scoped policy and the org context.

Steps

1. The agent creates or activates the period with `bearing_period_create` and `bearing_period_activate` (or fetches an existing one with `bearing_periods` or `bearing_period_get`).
2. For each objective, the agent calls `bearing_goal_create`, naming the period by label and the owner by email.
3. For each measurable outcome, the agent calls `bearing_kr_create` against the goal title.
4. The agent records starting values with `bearing_kr_update`.

Result: A fully populated period with owned goals and key results, ready for the team. The same goals appear in the human Dashboard and My Goals views, with progress rendering correctly.

Related: This is the agent counterpart to the first four human stories above. For a far-reaching batch, the agent can stage the plan with `proposal_create` for a human to approve first.

Story: Wire key results to real delivery (agent)

Who: An AI agent connecting OKRs to Bam work. **Goal:** Make a key result's progress track actual task completion automatically. **Before you start:** The goal and key result exist, and the Bam epic, project, or sprint to track is known.

Steps

1. The agent calls `bearing_kr_link` to bind the key result to a Bam epic, project, sprint, or task.
2. As linked Bam tasks reach a done state, the recompute background job updates the key result's progress and rolls it up to the goal.

Result: The key result and its goal move on their own as delivery progresses, with no manual check-ins. This linking is available only through agents or REST in the current build.

Related: There is no human linking screen, so this story has no UI equivalent. The agent can review the current links with `bearing_kr_links` and remove one with `bearing_kr_unlink`.

Story: Run a weekly status sweep (agent)

Who: An AI agent producing a weekly OKR digest. **Goal:** Summarize at-risk goals and period progress for the team. **Before you start:** Periods and goals exist with recorded progress.

Steps

1. The agent calls `bearing_at_risk` for the org-wide list of at-risk and behind goals.
2. The agent calls `bearing_report` with the period or owner type for a markdown summary.
3. Before quoting any goal in a shared channel, the agent calls `can_access` for each cited goal and drops anything the audience cannot see.
4. The agent relays the summary into the channel the team reads, for example a Banter post or a Brief document.

Result: The team gets a current OKR status summary without anyone opening the Bearing UI.

Related: Humans can read the same at-risk list on the **At Risk** page.

Related

- **Bam** (`/b3/`) - the project planning tool whose epics, projects, sprints, and tasks a key result can be linked to so its progress tracks real delivery. Bearing requires a Bam session to sign in.
- **Bolt** (`/bolt/`) - the automation engine. Bearing emits goal, key result, and period events (goal created, status changed, achieved, key result value updated, period activated and completed, and more) that Bolt rules can react to.
- **Banter** (`/banter/`) and **Brief** (`/brief/`) - common destinations for agent-generated OKR digests produced by `bearing_report` and `bearing_at_risk`.
- **Bench** (`/bench/`) - analytics dashboards that can surface goal progress alongside other org metrics.
- See the Bearing MCP-tools reference and guide under `docs/apps/bearing/` for the full tool catalog and parameter details.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What must exist and be selected before the Dashboard and At Risk views will show goals?

a) At least one Goal
b) At least one Period
c) At least one Key Result
d) An active Bolt automation

2. Which four metric types can a Key Result use?

a) Number, Percentage, Currency, Yes/No
b) Integer, Float, Money, Boolean
c) Count, Ratio, Dollars, Flag
d) Number, Date, Text, Select

3. How is a goal's overall progress calculated from its key results, and what range is each key result's progress clamped to?

4. What are the four scopes a goal can have?

a) Company, Squad, Repo, Person
b) Organization, Team, Project, Individual
c) Global, Department, Sprint, User
d) Org, Group, Board, Owner

5. How does Bearing decide whether a goal is on track, at risk, or behind?

6. What happens if you try to delete a period that still has goals?

a) The goals are deleted with it
b) The goals move to the next period
c) Deletion is blocked with a conflict error
d) The period is archived instead

7. The Periods row menu only shows Activate while the period is in which status?

a) active
b) planning
c) completed
d) archived

8. Which goal scope is not offered in the Create Goal dialog, and how can a goal be reassigned to a different owner?

9. What does a check-in do when you set a key result's new current value?

a) Only updates that one key result's bar
b) Updates the KR, rolls up the goal's progress, re-derives the goal's status, and writes a snapshot
c) Emails all watchers a digest
d) Marks the goal achieved

10. What does linking a key result to Bam work do, and where is that linking available?

11. How does the Watchers form resolve the value you type into the User ID or email field?

a) By display name only
b) By user ID first, then by email
c) By email first, then by handle
d) It always requires a UUID

12. What two sections does the My Goals page split goals into, and what does the Completed section contain?

CHAPTER 6

Bench

Chart the whole suite in one place.

Bench is the read-and-visualize layer over the rest of BigBlueBam. You assemble dashboards out of widgets, where each widget queries an approved data source like Bond deals or Blast campaigns and renders a chart, KPI card, or table. Bench never stores your business data; it reads other products' tables through an allowlisted registry and scopes every query to your active organization. By the end of this chapter you will be able to build a dashboard, explore a source ad hoc, save and schedule queries, and see where an agent can summarize a dashboard and flag anomalies for you.

At a glance

- Shared dashboards built from widgets, each charting one registered data source
- Eleven chart types, from bar and line to KPI card, gauge, and progress bar
- Ad-Hoc Explorer for a quick aggregate without building a whole dashboard
- Saved queries and cron-driven scheduled reports to Email, Banter, or Brief
- Full MCP surface, so an agent can read dashboards, run queries, and compare periods

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Dashboards list . Dashboard view . Dashboard edit . Widget Templates gallery . Widget Wizard . Widget Edit (live preview)

> *Bench builds shared analytics dashboards and ad-hoc queries across the other BigBlueBam apps. Reach for it when you want one place to chart deals, campaigns, goals, knowledge-base activity, and other suite data, then share or schedule those views for your team.*

Overview

Bench is a read-and-visualize layer on top of the rest of the suite. You assemble **dashboards** out of **widgets**, where each widget[49] runs a query against an approved **data source**[50] (a product plus an entity, such as Bond deals or Blast campaigns) and renders the result as a chart, KPI card, or table. You can also explore a source interactively in the **Ad-Hoc Explorer**, save query definitions, and schedule recurring report deliveries.

Bench does not store your business data. It queries other products' tables through a compile-time **data source registry** that allowlists exactly which tables and columns Bench is allowed to read, and always scopes every query to your active organization. Because the registry is an allowlist, you can only chart sources and fields that have been registered.

> ***Bench reads other apps but never writes them, and every query is org-scoped through an allowlisted registry.***

Bench sits between the operational apps (Bam, Bond, Blast, Beacon, Bearing, and others) and the people who need a roll-up view of them. Operators and leads build dashboards in the UI. AI agents can do the same work through MCP tools: they read and summarize dashboards, run ad-hoc queries, build dashboards and widgets, manage saved queries and scheduled reports, and run anomaly and period-comparison checks.

[49] **Widget.** One visualization on a dashboard. A widget has a name, a chart type, a data source and entity, and a query configuration. The renderer draws bar, line, area, pie, donut, KPI card, counter, and table widgets distinctly; any other type falls back to a table.

[50] **Data source.** A registered `(product, entity)` pair that maps to one underlying table with declared measures, dimensions, and filters. Bench can only query registered sources and columns, and every query is automatically scoped to your organization.

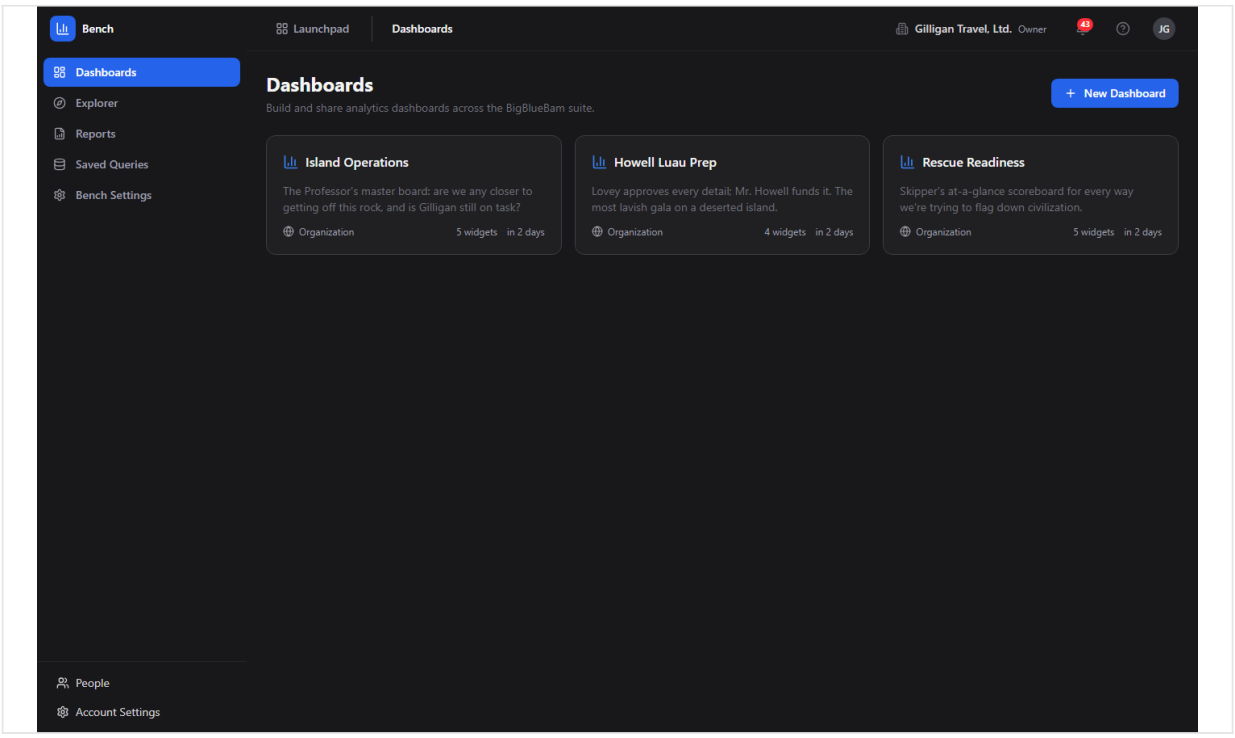


Figure 6.1 Dashboard list

Key concepts

- **Dashboard**[51] - A named container of widgets with a saved layout. Each dashboard has a **visibility** of Private, Project, or Organization, an optional linked project, and an optional auto-refresh interval. One dashboard per org can be the default.
- **Widget** - One visualization on a dashboard. A widget has a name, a chart type, a data source and entity, and a query configuration. The renderer draws bar, line, area, pie, donut, KPI card, counter, and table widgets distinctly; any other type falls back to a table.
- **Data source** - A registered `(product, entity)` pair that maps to one underlying table with declared measures, dimensions, and filters. Bench can only query registered sources and columns, and every query is automatically scoped to your organization.
- **Measure**[52] - A numeric aggregation in a query: count, sum, avg, min, or max of a field. A query needs at least one measure.
- **Dimension**[53] - A field to group by (for example, deal stage or campaign).

[51] **Dashboard.** A named container of widgets with a saved layout. Each dashboard has a **visibility** of Private, Project, or Organization, an optional linked project, and an optional auto-refresh interval. One dashboard per org can be the default.

[52] **Measure.** A numeric aggregation in a query: count, sum, avg, min, or max of a field. A query needs at least one measure.

[53] **Dimension.** A field to group by (for example, deal stage or campaign).

- **Filter**[54] - A condition on a field. Operators include equals, not equals, greater/less than (and -or-equal), in, is null, is not null, between, and like.
- **Saved query**[55] - A reusable query definition (data source, entity, and query config) you can keep and re-run later.
- **Scheduled report**[56] - A cron-driven dashboard snapshot delivered to a target on a schedule. Delivery methods are Email, Banter Channel, and Brief Document; export formats are PDF, PNG, and CSV.
- **Materialized view**[57] - A pre-computed rollup that the worker refreshes automatically. Three are exposed to you as queryable `bench:` data sources: Daily Task Throughput, Pipeline Snapshot, and Campaign Engagement.

> **NOTE**
> A materialized view is a pre-computed rollup the worker refreshes for you. Bench surfaces three as bench: data sources: Daily Task Throughput, Pipeline Snapshot, and Campaign Engagement.

Where to find it

Bench lives at `/bench/`. You must be logged in to BigBlueBam first. If you open Bench while signed out, you will see a "Bench Analytics" screen with the message "Please log in to BigBlueBam first to access Bench." and a "Go to BigBlueBam Login" button that sends you to `/b3/`.

Once you are in, the left sidebar has five destinations: **Dashboards**, **Explorer**, **Reports**, **Saved Queries**, and **Bench Settings**. The header carries the Launchpad app switcher, breadcrumbs, an organization switcher, a notifications bell, and your user menu. Bench reads your active organization from the auth store; use the organization switcher in the header to change which org's data you are charting. Pressing the `?` key anywhere outside a text input opens the in-app Help.

Some write actions require a higher API-key scope. Creating dashboards and widgets needs `read_write`; creating, sending, and deleting scheduled reports needs admin scope plus the matching report permission.

[54] **Filter.** A condition on a field. Operators include equals, not equals, greater/less than (and -or-equal), in, is null, is not null, between, and like.
[55] **Saved query.** A reusable query definition (data source, entity, and query config) you can keep and re-run later.
[56] **Scheduled report.** A cron-driven dashboard snapshot delivered to a target on a schedule. Delivery methods are Email, Banter Channel, and Brief Document; export formats are PDF, PNG, and CSV.
[57] **Materialized view.** A pre-computed rollup that the worker refreshes automatically. Three are exposed to you as queryable `bench:` data sources: Daily Task Throughput, Pipeline Snapshot, and Campaign Engagement.

Feature reference

Dashboards list

The landing view at `/bench/`. It shows every dashboard you can see as a grid of cards.

Each card shows the dashboard name, an optional description, a visibility pill (Private with a lock icon, Project with a people icon, or Organization with a globe icon), the widget count ("N widgets"), and the relative time it was last updated. A kebab menu (the horizontal dots that appear on hover) holds **View**, **Duplicate**, and **Delete** (Delete is destructive).

To create a dashboard:

1. Click **New Dashboard** (the button with the plus icon, top right).
2. Bench creates a dashboard named "Untitled Dashboard" with Private visibility and immediately opens it in the edit view.
3. Set its name, description, and visibility there, then add widgets.

To open, duplicate, or delete a dashboard:

1. Click a card (or its kebab menu **View**) to open the read view.
2. Choose **Duplicate** from the kebab menu to clone the dashboard and its widgets into a new copy. The copy is forced to Private visibility.
3. Choose **Delete** from the kebab menu to remove the dashboard and its widgets. This is permanent.

If you have no dashboards yet, the page shows "No dashboards yet" with the hint "Create your first dashboard to start visualizing data."

Dashboard view

The read-only view of one dashboard at `/dashboards/:id`. Each widget renders its chart along with the query's execution time in milliseconds.

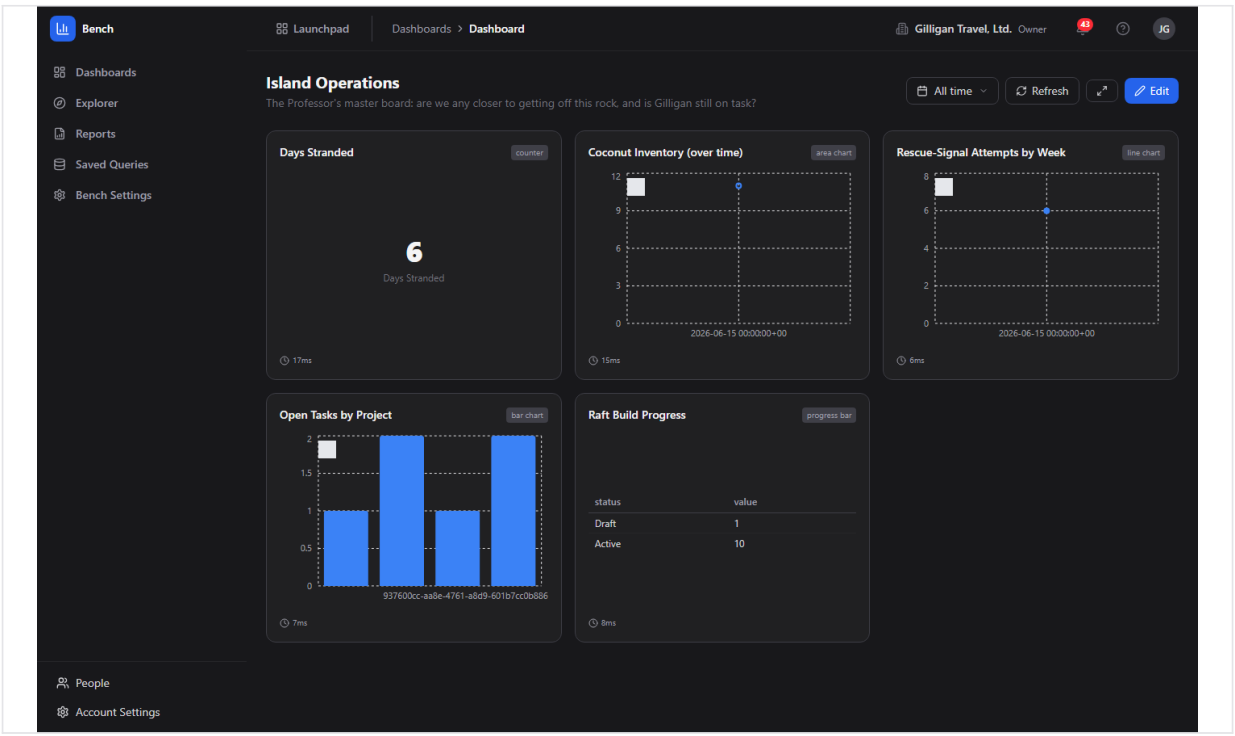


Figure 6.2 Dashboard view

The toolbar (top right) has:

- A date-range picker. You can pick a preset (Today, Last 7 days, Last 30 days, Last 90 days, This month, This quarter, This year) or a custom From/To range and click **Apply**.
- **Refresh** (circular-arrow icon), which re-runs every widget query.
- A fullscreen toggle (the expand icon, no text label), which fills the screen for a wall-board display.
- **Edit**, which opens the edit view.

To view a dashboard:

1. Open it from the Dashboards list.
2. Read each widget; the small clock and number under a widget is how long that widget's query took.
3. Pick a date range from the picker to scope the time-based widgets, or click **Refresh** to re-pull the data, or the fullscreen toggle for a kiosk display.

If the dashboard has a saved auto-refresh interval, the view re-fetches on that schedule on its own.

If a dashboard has no widgets, the view shows "This dashboard has no widgets yet." with an **Add widgets** link to the edit view.

The date-range picker scopes the dashboard's widgets: the selected preset or custom From/To range is passed into each widget's query, so changing the range re-filters the widgets that have a time field. Widgets with no time dimension run their own stored query unchanged.

Dashboard edit

The edit view at `/dashboards/:id/edit`, titled **Edit Dashboard**. Use it to rename a dashboard, set its visibility, and manage its widgets.

The metadata block has three fields: **Name**, **Description**, and a **Visibility** select (Private, Project, Organization). Click **Save** (the disk icon, top right) to persist them.

The **Widgets** section has two buttons:

- **Templates** (sparkles icon) toggles the Widget Templates gallery inline.
- **Custom Widget** (plus icon) opens the Widget Wizard.

Each existing widget appears as a row with a drag handle, its name, a "data_source / entity - type" subline, an **Edit** link to the Widget Edit page, and a trash icon that deletes the widget.

To edit a dashboard:

1. Open the dashboard and click **Edit**.
2. Change **Name**, **Description**, or **Visibility** and click **Save**.
3. Add widgets with **Templates** or **Custom Widget**, edit a widget with its **Edit** link, or remove one with its trash icon.
4. To reorder widgets, drag a widget row by its drag handle and drop it where you want it. The new order is saved to the dashboard layout and survives a reload.

Widget Templates gallery

A gallery of prebuilt widget presets, shown inline in the Dashboard edit view when you click **Templates**. It is titled **Widget Templates** and has a **Close** action.

Category filter pills across the top (All, Project Management, CRM, Email Marketing, Support, Cross-Product) narrow the grid. Each preset card shows a type icon, a type pill, the preset name, a short description, and its category.

To add a preset widget:

1. In the Dashboard edit view, click **Templates**.
2. Optionally click a category pill to filter.
3. Click a preset card. Bench instantiates it as a working widget on the dashboard (carrying a full query config that maps to registered sources and fields) and closes the gallery.

The presets in the Project Management, CRM, Email Marketing, and Cross-Product categories build widgets against registered sources that return data. The **Support** presets (Open Tickets, Tickets by Priority) build cleanly but query the Tickets source, which does not return data in this release (see "Sources that do not return data"), so those widgets show "No data".

Widget Wizard

A four-step builder for a custom widget, reached from a dashboard's edit view via **Custom Widget**, at `/dashboards/:id/widgets/new`. Titled **New Widget**.

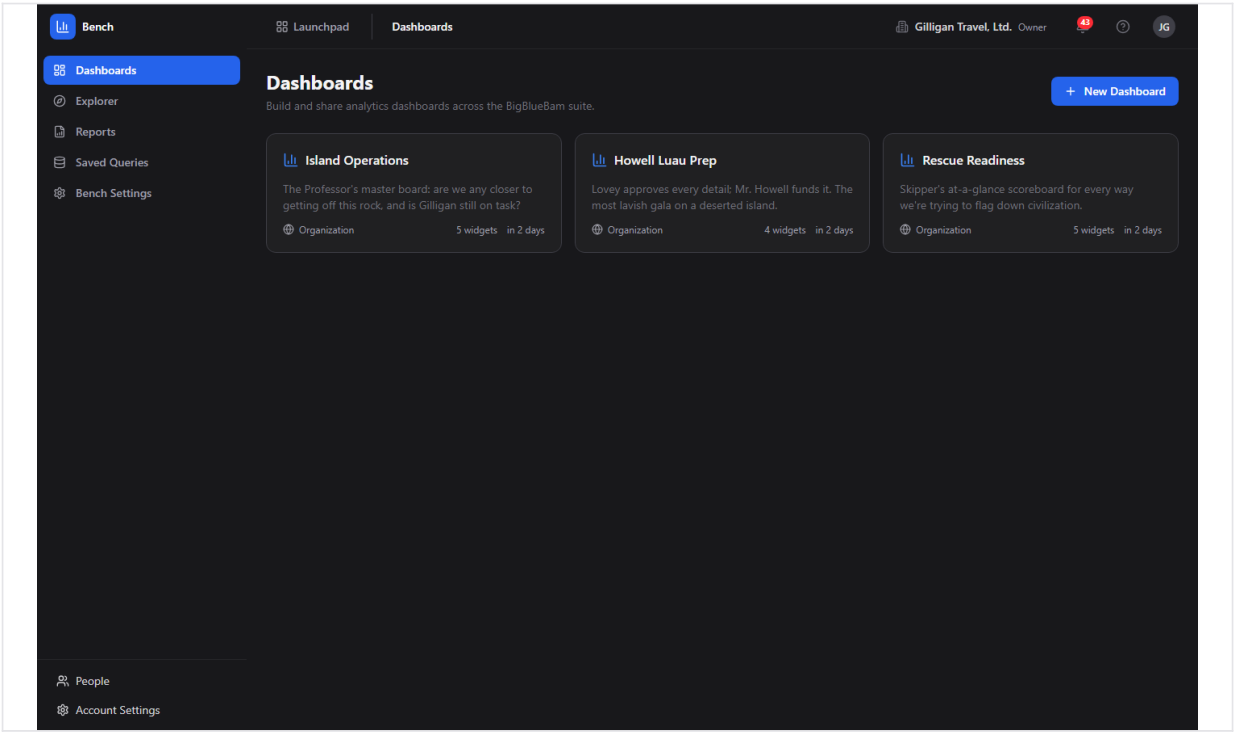


Figure 6.3 *Widget wizard*

The step indicator names the four steps: **Data Source**, **Measures & Dimensions**, **Chart Type**, **Name & Style**.

To build a widget with the wizard:

1. Step 1 (**Data Source**): pick a registered source from the list. Each entry shows the product pill, the source label, and its description.
2. Step 2 (**Measures & Dimensions**): check the measures to aggregate and the dimensions to group by. Each measure lists its allowed aggregations; each dimension lists its type.
3. Step 3 (**Chart Type**): pick one of the eleven types: Bar Chart, Line Chart, Area Chart, Pie Chart, Donut Chart, KPI Card, Counter, Table, Funnel, Gauge, Progress Bar.
4. Step 4 (**Name & Style**): type a **Widget Name**, then confirm the summary panel (Source, Measures, Dimensions, Type).
5. Click **Create Widget**. Bench builds the query config (using the first allowed aggregation for each chosen measure and your dimensions as group-bys) and returns you to the Dashboard edit view.

> **TIP**
> The Widget Wizard and the Widget Edit page offer the same eleven chart types, but Widget

> Edit adds a live preview that re-runs the query as you change the source, measures, and dimensions. Tune there when you want to see the result before you save.

Use **Back** to revisit a step, or **Cancel** on the first step to leave without creating.

Widget Edit (live preview)

The full widget editor at `/widgets/:id/edit`, titled **Edit Widget**. It has two columns: configuration on the left, a live preview on the right.

On the left you can change **Widget Name**, the **Data Source** select, the **Chart Type** grid (the same eleven types as the wizard), the **Measures** and **Dimensions** checkboxes, and the **Display Options** checkboxes (**Show legend** and **Stacked**).

On the right, the **Preview** panel renders live query results. A **Refresh** button re-runs the preview query, and a line below it reads "Query took Nms" with the row count.

To edit a widget:

1. From the Dashboard edit view, click a widget's **Edit** link.
2. Adjust the name, data source, chart type, measures, dimensions, or display options.
3. Watch the Preview update; click its **Refresh** to re-pull.
4. Click **Save**. Bench writes the change and returns you to the Dashboard edit view.

Ad-Hoc Explorer

An interactive query view at `/explorer`, titled **Ad-Hoc Explorer**, with the subtitle "Query any data source interactively."

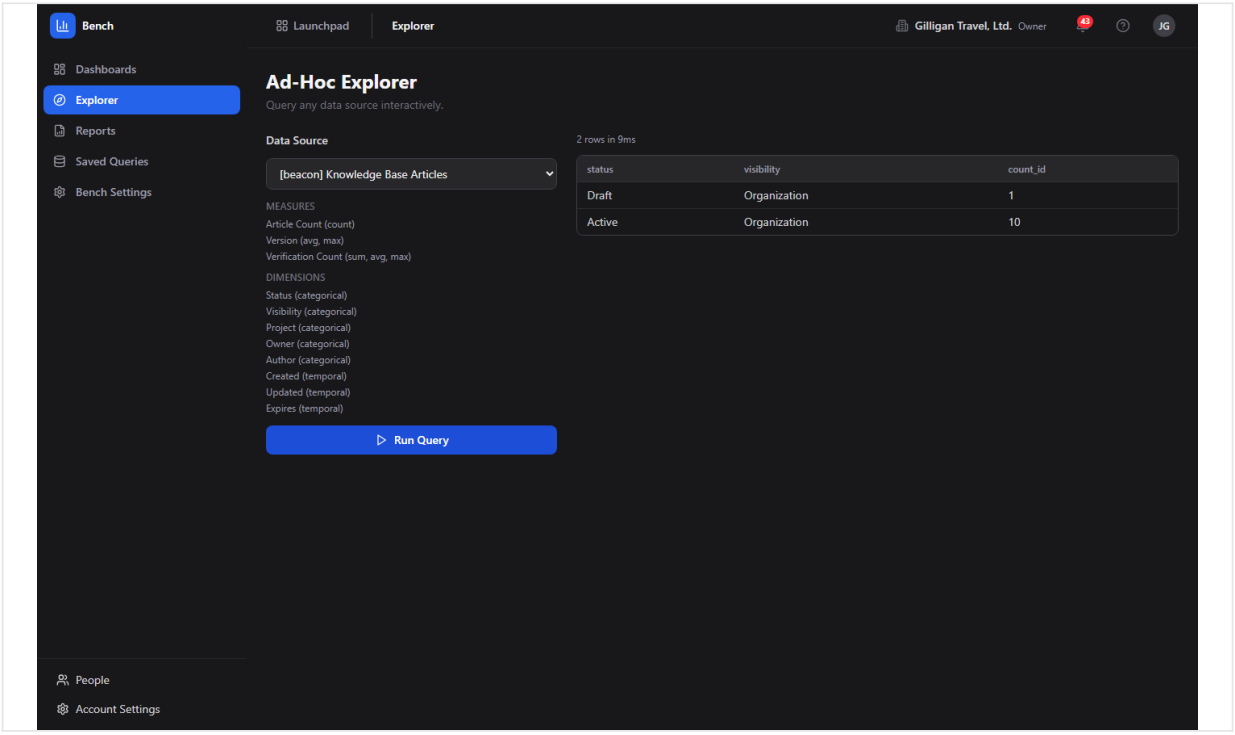


Figure 6.4 Ad-hoc explorer

The left panel has a **Data Source** dropdown (each option is shown as "[product] Label"), read-only **Measures** and **Dimensions** lists for the selected source, and a **Run Query** button. The right panel shows an error banner when a query fails, the generated SQL (in non-production builds only), a "N rows in Xms" line, and the results table.

To explore a source:

1. Choose a source from the **Data Source** dropdown.
2. Review its Measures and Dimensions.
3. Click **Run Query**.
4. Read the results table, plus the row count and timing.

When you open the Explorer from a saved query's **Run** button, it loads that saved query's source and full configuration and runs it for you (a "Loaded saved query: <name>" note appears). When you open the Explorer directly and pick a source yourself, **Run Query** runs one fixed shape per source: the first measure with its first aggregation, plus the first two dimensions, limited to 50 rows.

> **TRY IT**
> Open the Explorer, pick Knowledge Base Articles from the Data Source dropdown, and click Run Query to see the fixed shape: the first measure, the first two dimensions, and up to fifty rows.

> *Known limitation: when you pick a source manually in the Explorer there are no measure, dimension, or filter controls; it runs the fixed shape described above. For arbitrary measures, dimensions, and filters without going through a saved query, an agent can use the `bench_query_ad_hoc` MCP tool.*

Scheduled Reports

The reports list at `/reports`, titled **Scheduled Reports**, with the subtitle "Automated dashboard snapshots delivered on a schedule."

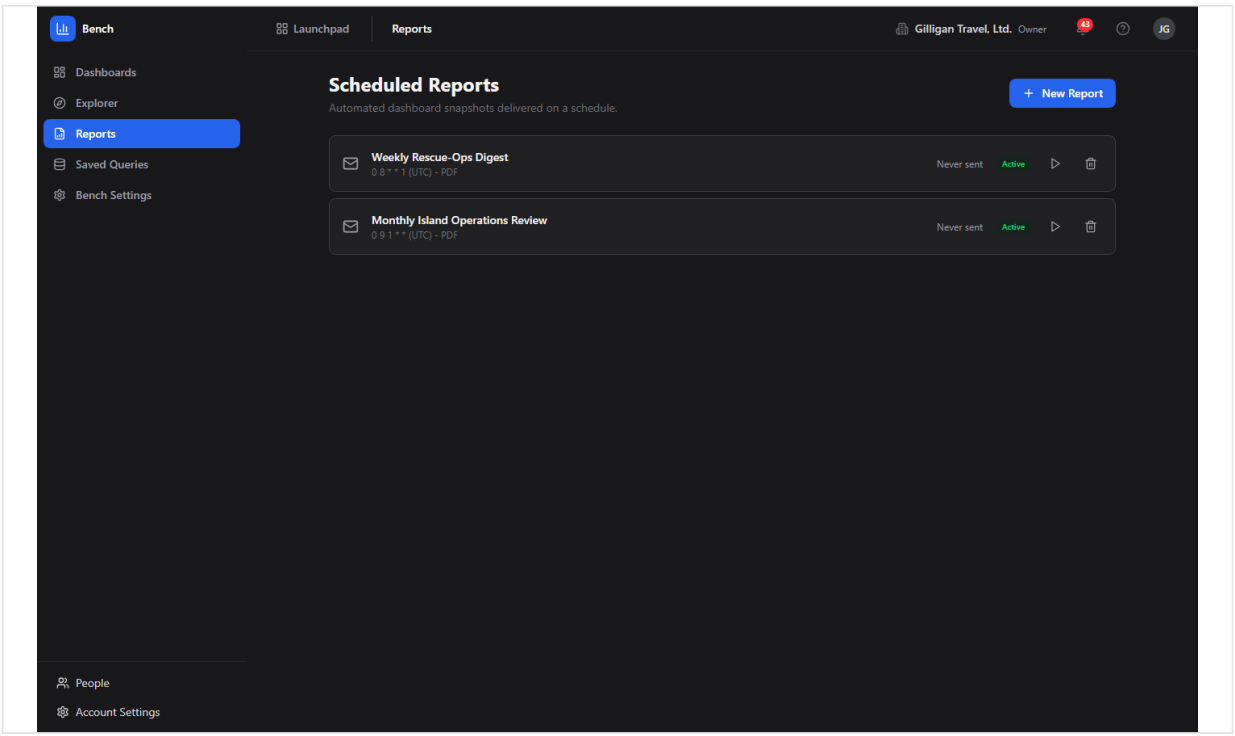


Figure 6.5 Scheduled reports

Each row has a delivery-method icon (envelope for Email, speech bubble for Banter Channel, document for Brief Document), the report name, a "cron (timezone) - FORMAT" subline, a last-sent line ("Last sent X" or "Never sent"), an optional delivery-status pill, an **Active**/**Paused** pill, a **Send now** play button, and a delete trash icon.

To create a scheduled report:

1. Click **New Report**. A dialog titled **New Scheduled Report** opens.
2. Fill in **Report Name**.
3. Choose a **Dashboard** to render.
4. Choose a **Schedule** preset (Every day at 9 AM, Every Monday at 9 AM, First of every month at 9 AM, Every Friday at 5 PM, or Custom). For Custom, type a cron expression. The detected timezone is shown below.
5. Choose a **Delivery Method** (Email, Banter Channel, or Brief Document). The target field relabels to **Email Address**, **Channel**, or **Document** accordingly, and you enter the destination.
6. Choose an **Export Format** (PDF, PNG, or CSV).
7. Leave **Enable immediately** checked to activate the schedule now, or uncheck it to create it paused.
8. Click **Create Report**.

To run a report immediately, click its **Send now** play button. To remove one, click its trash icon. Creating a report needs admin API-key scope plus the report-create permission; Send now and Delete need their matching permissions.

> *Known limitation: report delivery is a stub in this release. The worker logs "simulating delivery (stub)" and stamps the row's status; it does not produce or send an actual PDF, email, Banter post, or Brief document. On success the worker writes a `delivered` status, but the row's status pill only colors `sent`, `queued`, and `failed`, so a delivered report shows the neutral gray pill rather than a green one. Treat Send now and the schedule as wiring that updates status, not as confirmed delivery.*

WARNING
Send now and the schedule only stamp the row's status; they do not produce or send an actual PDF, email, Banter post, or Brief document. A successful run even shows the neutral gray pill rather than a green one, so treat this as wiring, not confirmed delivery.

Saved Queries

The saved-query list at `/saved-queries`, titled **Saved Queries**, with the subtitle "Reusable query definitions you can run from the ad-hoc explorer."

Each row has the query name, an optional description, a "data_source/entity - Created DATE" subline, a **Run** button, an **Edit** pencil, and a **Delete** trash icon.

To save a query:

1. Click **New Query**. A dialog titled **New Saved Query** opens.
2. Fill in **Name**, an optional **Description**, and choose a **Data Source**.
3. Check the **Measures** to aggregate and the **Dimensions** to group by (at least one measure is required).
4. Click **Create**. Bench saves the query with a real configuration built from your measures and dimensions.

To edit or run a saved query:

1. Click the **Edit** pencil to change the name, description, data source, measures, or dimensions (dialog titled **Edit Saved Query**), then click **Update**. Editing reloads the saved configuration so you keep its definition.
2. Click **Run** to open the Explorer with that saved query loaded and executed.

If you have none yet, the page shows "No saved queries" with the hint "Create a query to save and re-run later from the explorer."

Bench Settings

A read-only settings view at `/settings`, titled **Settings**, with the subtitle "Configure Bench data sources and preferences."

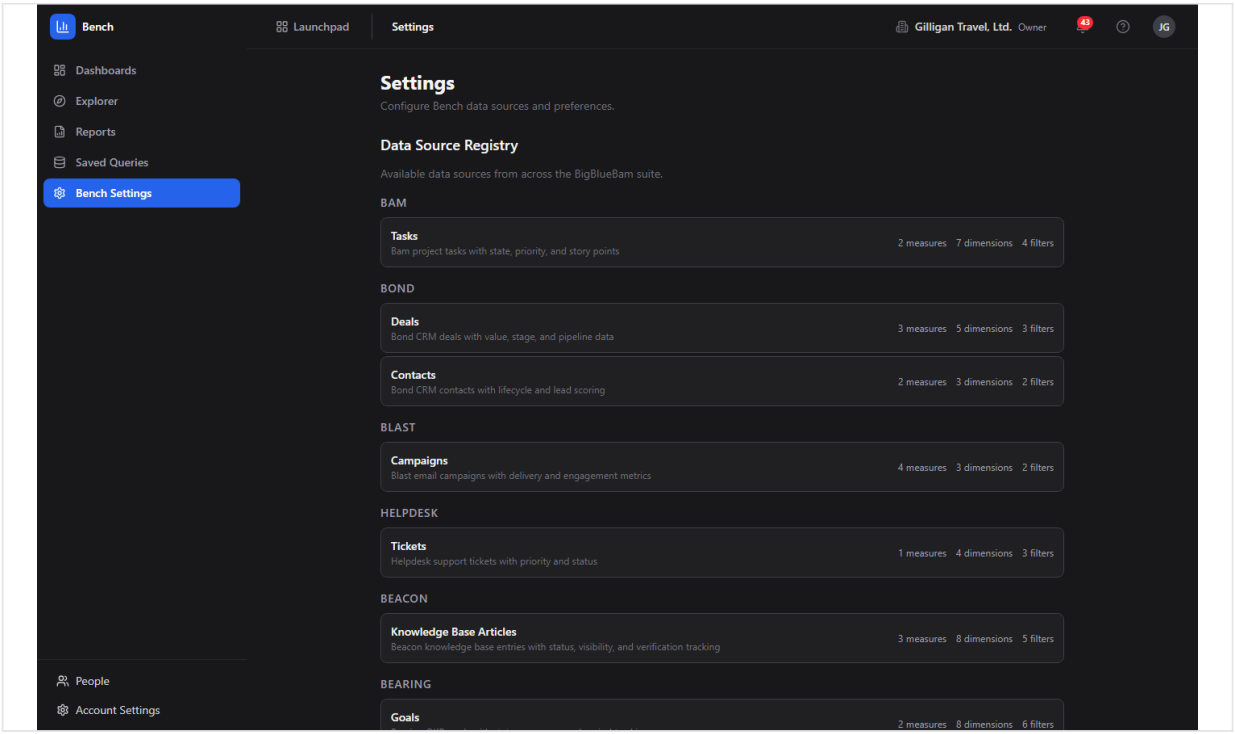


Figure 6.6 Bench settings

It has three sections, none of them editable:

- **Data Source Registry** - every registered source grouped by product, each showing its label, description, and counts of measures, dimensions, and filters.
- **Cache** - the Default Cache TTL (60 seconds).
- **Query Execution** - the Statement Timeout (10,000 ms).

To review what Bench can query:

1. Open **Bench Settings** from the sidebar.
2. Scroll the **Data Source Registry** to see which products and entities are available and how many measures, dimensions, and filters each exposes.

Registered data sources

These are the sources Bench can query, from the data source registry. Each query is automatically scoped to your organization.

Product	Entity	Label	Backing table
bam	tasks	Tasks	tasks
bond	deals	Deals	bond_deals
bond	contacts	Contacts	bond_contacts
blast	campaigns	Campaigns	blast_campaigns
helpdesk	tickets	Tickets	tickets
beacon	articles	Knowledge Base Articles	beacon_entries
bearing	goals	Goals	bearing_goals
bureau	floor_analytics	Bureau Floor Analytics (Daily)	bureau_floor_analytics
bench	daily_task_throughput	Daily Task Throughput	bench_mv_daily_task_throughput
bench	pipeline_snapshot	Pipeline Snapshot	bench_mv_pipeline_snapshot
bench	campaign_engagement	Campaign Engagement	bench_mv_campaign_engagement

Notes on specific sources:

- The Bam tasks source, the Bond, Blast, Beacon, and Bearing sources, and the Pipeline Snapshot and Campaign Engagement materialized views all return data and back the working dashboard widgets and Explorer queries. The Bam tasks source is org-scoped by its `org_id` column.
- The Bureau Floor Analytics source is a nightly rollup, not a live feed. Its rows are written once per floor per day for the previous day, so a one-day window is usually empty during the day. Use a Last 7 days window for Bureau widgets. The agent anomaly and period-comparison tools do not work against this source, because it has no `created_at` column.

Sources that do not return data

Two registered sources do not return data in this release, because their backing tables have no organization column and the query builder scopes every query by `organization_id`:

- **helpdesk:tickets (Tickets)** - the `tickets` table is scoped by `project_id` with no org column, so the org filter fails and the query returns nothing. The Support gallery presets (Open Tickets, Tickets by Priority) are affected. Fixing this needs a schema change to give the table an org column (or an org-scoping join), not a registry tweak.
- **bench:daily_task_throughput (Daily Task Throughput)** - this materialized view is grouped by project and day with no organization column, unlike the Pipeline Snapshot and Campaign Engagement views, so the org filter fails and it returns

nothing. The Daily Task Throughput gallery preset works around this by charting the org-scoped Bam tasks source with a daily time bucket instead.

If you need task analytics today, use the **bam: Tasks** source (now org-scoped) or build a daily-throughput widget from the Bam tasks source with a day time bucket, as the gallery preset does.

Working with AI agents

Agents drive Bench through the Model Context Protocol. The Bench MCP tools call the same bench-api routes the UI uses, with the caller's bearer token, so an agent sees only data its token is allowed to see and every query is org-scoped. The full Bench tool surface lives in `apps/mcp-server/src/tools/bench-tools.ts` and is summarized in this app's MCP-tools reference.

What agents commonly do in Bench:

- **Discover what can be queried** - `bench_list_data_sources` lists every source with its schema. Agents should call this first to learn valid field names.
- **Run ad-hoc queries** - `bench_query_ad_hoc` runs a structured query (measures, dimensions, filters, limit) against any registered source. This is the freeform query path the Explorer UI does not expose for manually-picked sources.
- **Read dashboards and widgets** - `bench_list_dashboards`, `bench_get_dashboard`, `bench_list_widgets`, and `bench_query_widget` read a dashboard and run its widget queries. `bench_summarize_dashboard` is the canonical "read this dashboard for me" entry point: it fetches the dashboard and runs every widget query, returning the bundle for summarization. Widgets backed by a non-returning source come back as a per-widget error rather than failing the whole call.
- **Manage and trigger reports** - `bench_list_scheduled_reports` lists schedules; `bench_generate_report` triggers an immediate delivery (the same Send now path, still a delivery stub).
- **Detect anomalies and compare periods** - `bench_detect_anomalies` compares the most recent period against the previous one and flags a change over 30% with a severity of high, medium, or low. `bench_compare_periods` compares two arbitrary date ranges and returns both values, the percent change, and the direction. There is no UI for either; both are agent-only. Both filter on a `created_at` column, so they work only for sources that have one with a valid org column (Bond, Blast, Beacon, Bearing) and not for the helpdesk:tickets, Bureau, or materialized-view sources.

> ***An agent can flag a metric that moved sharply, marking any change over thirty percent as high, medium, or low severity.***

When you review agent work in Bench, the things to confirm are that the agent queried a source that actually returns data, that visibility on any dashboard it created matches your

intent (the default is Private), and that a report it created or triggered points at the right delivery target. Note again that triggering a report does not yet send anything.

Bench also participates in the cross-cutting agentic platform. Every agent action carries an agent or service identity that is recorded in the unified activity log, and agents send periodic `agent_heartbeat` calls so operators can see which runners are live. High-impact or cross-app steps an agent is unsure about can be routed to a human approval queue with the platform `proposal_create` and `proposal_decide` tools. Per-agent kill switches and tool allowlists in `agent_policies` gate which tools a service account may call, and outbound webhooks can push Bench Bolt events (such as `report.delivered`, source `bench`) to subscribed agent runners. Before an agent reposts Bench results into another app's shared surface, it should run the platform `can_access` visibility check for each cited entity and drop anything the asking user is not allowed to see. Cross-app reads (`search_everything`, `activity_query`) let an agent pull context from other apps to frame a Bench summary.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. Your presence in Bench shows in the Bureau virtual office, so teammates can see where you are and gather without booking a call. The suite's see-who-is-here strips, one-tap huddles, and real-time co-editing live on its collaborative surfaces. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Open and read a dashboard

Who: Anyone with access to Bench. **Goal:** See the current numbers on an existing dashboard. **Before you start:** You are logged in to BigBlueBam and at least one dashboard exists in your organization.

Steps

1. Go to `/bench/`. The **Dashboards** list shows the dashboards you can see.
2. Click a dashboard card.
3. Read each widget's chart or KPI; the clock and number under a widget is its query time.
4. Pick a date range from the picker to scope the time-based widgets, or click **Refresh** (circular-arrow icon) to re-pull the data.
5. Click the fullscreen toggle (the expand icon) if you want a kiosk display.

Result: You are on the dashboard read view with current data for your active organization, scoped to the range you chose.

Related: The `bench_get_dashboard` and `bench_summarize_dashboard` tools let an agent read the same dashboard.

Story: Build a dashboard from scratch

Who: An operator or lead with `read_write` API-key scope. **Goal:** Create a new dashboard and add widgets to it. **Before you start:** You are logged in and know which sources you want to chart.

Steps

1. On the **Dashboards** list, click **New Dashboard**.
2. Bench creates "Untitled Dashboard" and opens it in the **Edit Dashboard** view.
3. Set **Name**, **Description**, and **Visibility**, then click **Save**.
4. Click **Templates** to add a prebuilt widget, or **Custom Widget** to open the Widget Wizard.
5. Add the widgets you want, drag their rows to order them, then click the back arrow to return to the read view.

Result: A new dashboard with your widgets in the order you set, visible according to the visibility you chose.

Related: An agent can do this with `bench_create_dashboard` plus `bench_add_widget`.

Story: Add a custom widget with the wizard

Who: An operator or lead with `read_write` scope. **Goal:** Add a widget that charts a specific measure grouped by a dimension. **Before you start:** You are in a dashboard's **Edit Dashboard** view.

Steps

1. Click **Custom Widget** to open the **New Widget** wizard.
2. Step 1: pick a **Data Source** that returns data (for example Bond Deals or Bam Tasks).
3. Step 2: check one or more **Measures** and the **Dimensions** to group by.
4. Step 3: pick a **Chart Type**.
5. Step 4: type a **Widget Name** and confirm the summary.
6. Click **Create Widget**.

Result: The widget is added to the dashboard and you are back in the **Edit Dashboard** view.

Related: `bench_add_widget` does the same from an agent, with full control over the query config.

Story: Drop in a prebuilt widget from Templates

Who: An operator or lead with `read_write` scope. **Goal:** Add a common widget quickly without building a query. **Before you start:** You are in a dashboard's **Edit Dashboard** view.

Steps

1. Click **Templates** to open the **Widget Templates** gallery.
2. Optionally click a category pill (for example CRM or Email Marketing) to filter.
3. Click a preset card.

Result: Bench instantiates the preset as a working widget and closes the gallery. Presets in the Project Management, CRM, Email Marketing, and Cross-Product categories return data; the Support presets build cleanly but show "No data" because the Tickets source is not org-scoped in this release.

Story: Reorder a dashboard's widgets

Who: An operator or lead with `read_write` scope. **Goal:** Change the order widgets appear in. **Before you start:** The dashboard has at least two widgets.

Steps

1. Open the dashboard and click **Edit**.
2. Grab a widget row by its drag handle (the grip icon at the left of the row).
3. Drag it above or below another widget and drop it.

Result: The widgets reorder, and the new order is saved to the dashboard layout, so it survives a reload and shows the same way in the read view.

Story: Tune a widget with the live preview

Who: An operator or lead with `read_write` scope. **Goal:** Adjust a widget's source, chart type, or fields and confirm the result before saving. **Before you start:** The dashboard already has the widget you want to change.

Steps

1. In the **Edit Dashboard** view, click the widget's **Edit** link.
2. Change the **Data Source**, **Chart Type**, **Measures**, **Dimensions**, or **Display Options** (**Show legend**, **Stacked**).
3. Watch the **Preview** panel; click its **Refresh** to re-pull.
4. Click **Save**.

Result: The widget is updated and you are back in the **Edit Dashboard** view.

Related: `bench_update_widget` makes the same change from an agent.

Story: Explore a source ad hoc

Who: Anyone with access to Bench. **Goal:** Pull a quick aggregate from a source without building a dashboard. **Before you start:** You are logged in.

Steps

1. Open **Explorer** from the sidebar.
2. Choose a source from the **Data Source** dropdown (pick one that returns data, such as Knowledge Base Articles).
3. Review the **Measures** and **Dimensions** for the source.
4. Click **Run Query**.
5. Read the results table and the row count and timing.

Result: A table of the fixed query shape (first measure, first two dimensions, up to 50 rows) for that source.

Related: For arbitrary measures, dimensions, and filters, save a query with the configuration you want and run it here, or use the `bench_query_ad_hoc` tool from an agent.

Story: Save a query and run it later

Who: Anyone with `read_write` scope. **Goal:** Keep a reusable query definition and re-run it. **Before you start:** You are logged in.

Steps

1. Open **Saved Queries** from the sidebar.
2. Click **New Query**.
3. Enter a **Name**, an optional **Description**, and pick a **Data Source**.
4. Check the **Measures** and **Dimensions** you want, then click **Create**.

5. Later, click **Run** on the saved row to open the Explorer with the query loaded and executed, or the **Edit** pencil to change it.

Result: The query appears in the Saved Queries list with a real configuration, and **Run** opens the Explorer showing its results.

Related: An agent can save a query with `bench_create_saved_query`.

Story: Schedule a recurring report

Who: An admin (admin API-key scope plus the report-create permission). **Goal:** Have a dashboard snapshot delivered on a schedule. **Before you start:** The dashboard you want to send already exists.

Steps

1. Open **Reports** from the sidebar and click **New Report**.
2. Enter a **Report Name** and pick a **Dashboard**.
3. Choose a **Schedule** preset, or pick Custom and type a cron expression.
4. Choose a **Delivery Method** (Email, Banter Channel, or Brief Document) and enter the target.
5. Choose an **Export Format** (PDF, PNG, or CSV).
6. Leave **Enable immediately** checked, then click **Create Report**.
7. Optionally click the report's **Send now** play button.

Result: The report appears in the list with an **Active** pill and its schedule. Note that delivery is a stub in this release: status is stamped but no artifact is actually sent, and a successful run shows the neutral status pill rather than a colored one.

Related: `bench_generate_report` triggers the same Send now path from an agent.

Story: Duplicate a dashboard as a starting template

Who: An operator or lead with `read_write` scope. **Goal:** Reuse an existing dashboard's layout and widgets as the basis for a new one. **Before you start:** A dashboard you want to copy exists.

Steps

1. On the **Dashboards** list, open the source dashboard's kebab menu.
2. Click **Duplicate**. Bench clones the dashboard and its widgets; the copy is Private.
3. Open the copy and click **Edit** to rename it and adjust its widgets and visibility.

Result: A new Private dashboard that mirrors the original's widgets.

Related: `bench_duplicate_dashboard` clones a dashboard from an agent.

Story: Have an agent summarize a dashboard and flag anomalies

Who: An AI agent acting for a user, plus the user reviewing the result. **Goal:** Get a written summary of a dashboard and a heads-up on metrics that moved sharply. **Before you start:**

The agent has a token scoped to the org and the dashboard ID (it can find it with `bench_list_dashboards`).

Steps

1. The agent calls `bench_summarize_dashboard` with the dashboard ID to fetch the dashboard and run every widget query.
2. The agent calls `bench_detect_anomalies` for the metrics it cares about, on a source that has a `created_at` column and a valid org column (for example Bond Deals or Blast Campaigns).
3. The agent composes a summary, noting any metric flagged as an anomaly and its severity.
4. Before reposting the summary into another app's shared surface, the agent runs the platform `can_access` check for each cited entity and drops anything the asking user cannot see.
5. You review the summary and confirm the cited dashboard and metrics.

Result: A summary of the dashboard with anomalies flagged, safe to share.

Related: `bench_compare_periods` for an explicit before/after comparison; the platform `proposal_create` tool to route an uncertain action to a human queue.

Related

- **Bam, Bond, Blast, Beacon, Bearing, Bureau, Helpdesk** - the products whose data Bench charts. Bench reads their tables through the registry; it does not write to them.
- **Banter** - a report delivery method (Banter Channel) and a target for cross-app agent posts.
- **Brief** - a report delivery method (Brief Document).
- **Bolt** - Bench emits the `report.delivered` event (source `bench`) that Bolt automations and outbound webhooks can subscribe to.
- See this app's MCP-tools reference in `docs/apps/bench/` for the full Bench tool catalog, and `docs/apps/bench/guide.md` for the narrative guide.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What does Bench store about your business data?

a) A full copy of every source table
b) Only the dashboards and widgets you build
c) Nothing; it queries other products' tables through a registry
d) A nightly snapshot of all org data

2. How many chart types does the Widget Wizard offer at the Chart Type step?

a) Five
b) Eight
c) Eleven
d) Fourteen

3. What are the three visibility options a dashboard can have?

4. What is the minimum number of measures a query needs?

a) Zero
b) At least one
c) At least two
d) Exactly one per dimension

5. Name the five numeric aggregations a measure can use.

6. When you duplicate a dashboard, what visibility does the copy get?

a) It keeps the original's visibility
b) Organization
c) Project
d) It is forced to Private

7. What query shape runs when you pick a source manually in the Explorer?

a) Every measure and dimension for the source
b) The first measure with its first aggregation, plus the first two dimensions, limited to 50 rows
c) Only a row count
d) A custom query you type in

8. What are the three delivery methods for a scheduled report?

9. What are the three export formats for a scheduled report?

a) PDF, PNG, and CSV
b) PDF, DOCX, and XLSX
c) HTML, CSV, and JSON
d) PNG, SVG, and PDF

10. Why do the helpdesk:tickets and Daily Task Throughput sources return no data in this release?

11. At what change does bench_detect_anomalies flag a metric, and with what severities?

a) Over 10%, with pass or fail
b) Over 30%, with high, medium, or low severity

c) Over 50%, with a single alert flag
d) Any change, ranked one through five

12. What are the read-only values shown in Bench Settings for Cache TTL and the statement timeout?

CHAPTER 7

Bill

Bill it, send it, get paid.

Bill turns the work your team already tracked into invoices a client can read and pay. You build a draft, fill it with line items, finalize it to lock a permanent number, send it, and record payments until the balance hits zero. Along the way it captures expenses with an approval step, resolves billing rates so tracked time prices itself, and reports on revenue, aging, and profitability. By the end of this chapter you can stand up your billing identity, send a finalized invoice, and read the numbers at the close of a period.

At a glance

- Draft-to-paid invoice lifecycle: finalize locks the number, payments close the balance
- Money in integer cents everywhere, so the inputs never round or guess
- Recurring schedules that bill a client on a cadence without re-drafting
- Time-based billing that resolves the most specific rate that applies
- Full MCP coverage, so an agent can draft and collect while a human approves and sends

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Dashboard . Invoice list . Create an invoice . Invoice from time entries . Invoice detail . Edit an invoice

> *Bill turns the work your team does into invoices, tracks who has paid and who is overdue, captures project expenses, and reports on revenue and profitability. Reach for it when you need to bill a client, record a payment, or close out a billing period.*

Overview

Bill is the invoicing and billing app in the BigBlueBam suite. You create an **invoice**[58] for a **client**[59], fill it with **line items**, finalize it to assign a permanent number, send it, and record **payments** until it is fully paid. Alongside invoices, Bill tracks **expenses** (with an approval step), **rates** that drive time-based billing, and a set of financial **reports**.

Bill is org-scoped: every client, invoice, expense[60], rate[61], and setting belongs to your active organization. It connects to the rest of the suite in several places. It can pull billable time from Bam projects and tasks, generate an invoice from a won Bond deal, route an invoice for approval through Bam's approval queue, and emit events that Bolt automation rules can react to.

A few conventions to know up front. All money in Bill is stored and entered in **integer cents**, not dollars. The amount, rate, and payment[62] inputs in the UI are labeled "(cents)" and take whole numbers, so typing `150` means one dollar and fifty cents. Invoices are only editable while they are in `draft`; once you finalize an invoice, its number and line items are locked.

> **TIP**
> Every amount, rate, and payment field is in whole cents, not dollars. Typing 150 means one dollar and fifty cents, so multiply your dollar figure by 100 before you enter it and nothing rounds.

Roles matter for the money-sensitive steps. Any member who can write to Bill can build a draft invoice, add line items, manage clients, log expenses, record payments, and configure rates and settings[63]. But the steps that commit money or sign off on it - **Finalize**, **Re-send / Send**, **Approve** or **Reject** an expense, and **Mark reimbursed** - require an admin or owner role. A member can prepare the work; an admin or owner approves and sends it.

Key concepts

- **Invoice** - A billable document addressed to one client. It snapshots your company details (the "from" side) and the client details (the "to" side) when it is created, and

[58] **Invoice.** A billable document addressed to one client. It snapshots your company details (the "from" side) and the client details (the "to" side) when it is created, and carries an invoice date, due date, line items, tax, totals, and payment history. Its number is literally `DRAFT` until you finalize it.

[59] **Client.** The recipient of an invoice. Carries name, email, phone, address, tax ID, default payment terms (in days), default payment instructions, and notes. A Bill client is separate from a Bond contact or company, though it can be linked to a Bond company.

[60] **Expense.** A project cost: description, amount in cents, category, vendor, date, and a billable flag. Expenses start as `pending`, a reviewer moves them to `approved` or `rejected`, and an approved expense can be moved on to `reimbursed`.

[61] **Rate.** A billing rate scoped to your org, a project, a user, or a user-on-a-project, with an amount in cents and a type (hourly, daily, or fixed). When Bill bills tracked time, it resolves the most specific rate that applies.

[62] **Payment.** A recorded receipt against an invoice: an amount in cents, a method (Bank Transfer, Credit Card, Check, Cash, Stripe, PayPal, or Other), an optional reference, and a date.

[63] **Settings.** Your company identity and the defaults new invoices inherit: invoice prefix, default tax rate, payment terms, payment instructions, footer text, and terms text.

carries an invoice date, due date, line items, tax, totals, and payment history. Its number is literally `DRAFT` until you finalize it.

- **Invoice status**[64] - The lifecycle of an invoice. The values that the app actually sets are `draft` (editable, no number yet), `sent` (finalized, number assigned, edits locked), `viewed` (a client opened the public link), `partially_paid` (some payment recorded, balance remains), `paid` (paid in full), and `void` (cancelled). **Overdue** is not a stored status: it is computed from the due date as any unpaid, finalized invoice that is past due. The Invoices list **Overdue** filter pill and the Dashboard **Overdue** tile both draw from that computed set, so they populate correctly.
- **Line item**[65] - One row on an invoice: a description, quantity, unit (default `hours`), and unit price in cents. The amount is quantity times unit price. Line items can only be added, edited, or removed while the invoice is a draft.
- **Client** - The recipient of an invoice. Carries name, email, phone, address, tax ID, default payment terms (in days), default payment instructions, and notes. A Bill client is separate from a Bond contact or company, though it can be linked to a Bond company.
- **Payment** - A recorded receipt against an invoice: an amount in cents, a method (Bank Transfer, Credit Card, Check, Cash, Stripe, PayPal, or Other), an optional reference, and a date.
- **Expense** - A project cost: description, amount in cents, category, vendor, date, and a billable flag. Expenses start as `pending`, a reviewer moves them to `approved` or `rejected`, and an approved expense can be moved on to `reimbursed`.
- **Rate** - A billing rate scoped to your org, a project, a user, or a user-on-a-project, with an amount in cents and a type (hourly, daily, or fixed). When Bill bills tracked time, it resolves the most specific rate that applies.
- **Settings** - Your company identity and the defaults new invoices inherit: invoice prefix, default tax rate, payment terms, payment instructions, footer text, and terms text.
- **Recurring schedule**[66] - A subscription that bills one client on a cadence (weekly, monthly, quarterly, or annually) from a saved line-item template. It carries a status (`active`, `paused`, or `cancelled`), a next-run date, a count of invoices generated so far, and a mode: auto-finalize (each generated invoice gets a number and is marked sent) or draft (each generated invoice is left as a draft to review). A daily worker sweep

[64] **Invoice status.** The lifecycle of an invoice. The values that the app actually sets are `draft` (editable, no number yet), `sent` (finalized, number assigned, edits locked), `viewed` (a client opened the public link), `partially_paid` (some payment recorded, balance remains), `paid` (paid in full), and `void` (cancelled). **Overdue** is not a stored status: it is computed from the due date as any unpaid, finalized invoice that is past due. The Invoices list **Overdue** filter pill and the Dashboard **Overdue** tile both draw from that computed set, so they populate correctly.

[65] **Line item.** One row on an invoice: a description, quantity, unit (default `hours`), and unit price in cents. The amount is quantity times unit price. Line items can only be added, edited, or removed while the invoice is a draft.

[66] **Recurring schedule.** A subscription that bills one client on a cadence (weekly, monthly, quarterly, or annually) from a saved line-item template. It carries a status (`active`, `paused`, or `cancelled`), a next-run date, a count of invoices generated so far, and a mode: auto-finalize (each generated invoice gets a number and is marked sent) or draft (each generated invoice is left as a draft to review). A daily worker sweep materializes invoices from schedules whose next-run date has arrived, and you can also generate one on demand.

materializes invoices from schedules whose next-run date has arrived, and you can also generate one on demand.

- **Public view token**[67] - A long opaque token on each invoice that grants read-only access (and a PDF) without logging in. It is how a sent invoice can be viewed by someone outside your org.

> **NOTE**
> Overdue is not a stored status. It is computed live from the due date as any unpaid, finalized invoice that is past due, which is why the Dashboard Overdue tile, the Invoices list Overdue pill, and the Reports overdue table all agree.

> ***Point Bill at a project and a date range and tracked time prices itself, resolving the most specific rate that applies.***

Where to find it

Bill is served at `/bill/`. You must be logged in to BigBlueBam first; if you are not, Bill shows a "Please log in to BigBlueBam first to access Bill." screen with a "Go to BigBlueBam Login" link to `/b3/`. Everything is scoped to your active organization, and mutating actions require the matching Bill permission on your account.

> **TRY IT**
> Open /bill/, click Bill Settings, fill in your Company Name and an Invoice Prefix, and click Save Settings. Watch for the Saved! confirmation, then your next invoice carries those details on its from side.

The left sidebar carries a fixed **New Invoice** quick-action button, then the nav items **Dashboard**, **Invoices**, **Recurring**, **Clients**, **Expenses**, **Rates**, and **Reports**, and under a **Bill Settings** subheading the **Bill Settings** item. Press `?` anywhere in Bill to open the embedded Help viewer.

[67] **Public view token.** A long opaque token on each invoice that grants read-only access (and a PDF) without logging in. It is how a sent invoice can be viewed by someone outside your org.

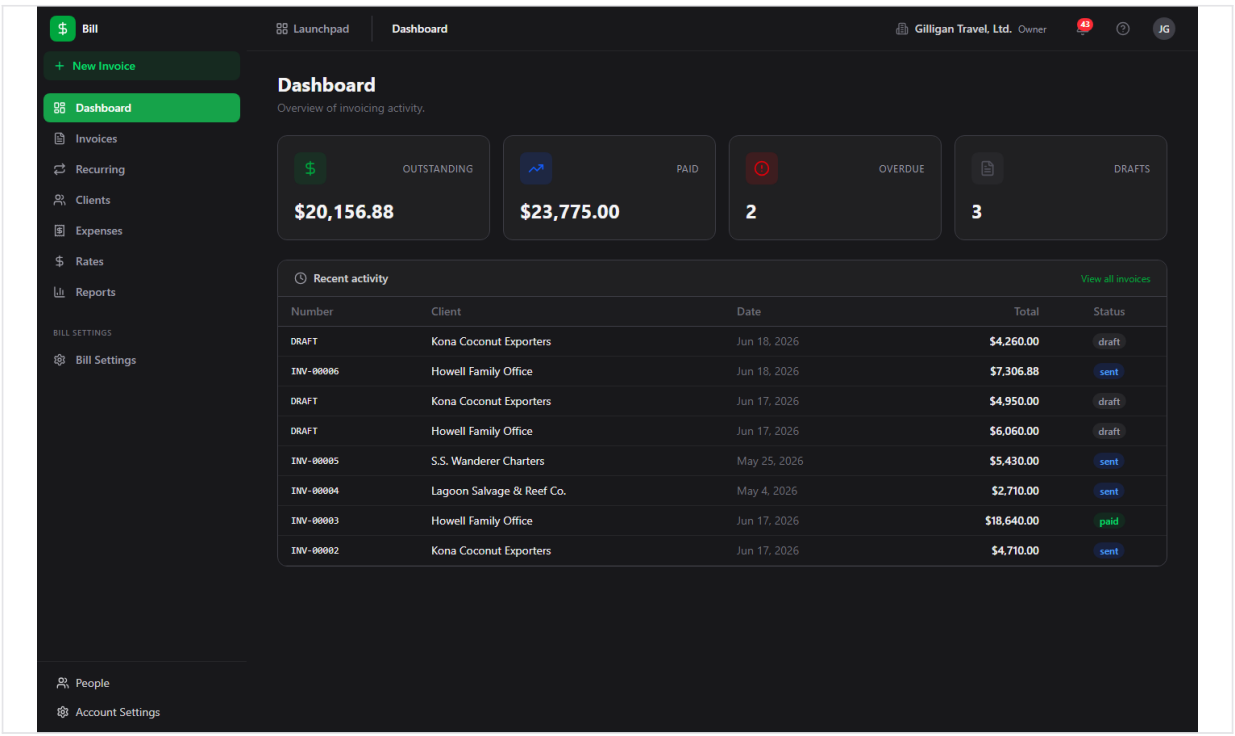


Figure 7.1 *Billing dashboard*

Feature reference

Dashboard

The Dashboard is the default view at `/bill/`, titled **Dashboard** with the subtitle "Overview of invoicing activity." It shows four stat tiles - **Outstanding**, **Paid**, **Overdue**, and **Drafts** - and a **Recent activity** table listing recent invoices (Number, Client, Date, Total, Status) with a **View all invoices** link.

The **Outstanding**, **Paid**, and **Drafts** numbers are computed in the browser from your full invoice list. The **Overdue** tile is different: it counts the invoices returned by the overdue report (unpaid, finalized invoices whose due date has passed), so it reflects what is actually past due. Clicking the **Overdue** tile opens the Invoices list pre-filtered to overdue. Clicking the other tiles or **View all invoices** returns you to the Dashboard.

To open an invoice from the Dashboard:

1. Go to `/bill/` from the sidebar **Dashboard** item.
2. In the **Recent activity** table, click any row.
3. You land on that invoice's detail page.

To jump straight to overdue invoices:

1. Click the **Overdue** tile.
2. The Invoices list opens with the **Overdue** filter applied, showing every past-due unpaid invoice.

Invoice list

The Invoices list at `/bill/invoices` is titled **Invoices** with the subtitle "Manage and track all your invoices." It is the main place to scan, filter, and act on invoices.

The columns are Number, Client, Date, Due (the due date), Total, Due (the remaining balance, which is total minus amount paid), Status, and Actions. The two "Due" columns are a date and a balance respectively. Above the table is a row of status filter pills; the empty pill is labeled **All**. Per-row actions include **PDF**, **Finalize** (drafts only), and **Request approval**.

To filter the list:

1. Open **Invoices** in the sidebar.
2. Click a status pill (**All**, or a specific status such as **Draft**, **Sent**, or **Paid**).
3. The table reloads with only invoices in that status.

The **Overdue** pill shows every invoice that is unpaid, finalized, and past its due date. Unlike the other pills, which match a stored status, the **Overdue** pill draws from the overdue report, so it computes past-due invoices from each invoice's due date. The Dashboard's **Overdue** tile deep-links here.

To finalize a draft from the list:

1. Find a draft row.
2. Click **Finalize** in its Actions cell. Finalize requires an admin or owner role.
3. The invoice is assigned a number and moves to `sent`; it can no longer be edited.

To download a PDF from the list:

1. Click **PDF** on any row.
2. The invoice PDF opens inline.

To request approval from the list:

1. Click **Request approval** on a row.
2. In the **Request approval for {number}** modal, choose an **Approver** from the select (the list is active Bam users), optionally type a **Message (optional)**, and click **Send request**.
3. The helper text reads "Sends a Banter DM to the approver via Bolt." The request is posted to Bam's approval queue with subject type `bill.invoice`; the approver acts on it inside Bam.

The two entry points for creating invoices live at the top of this view: **From Time Entries** (see "Invoice from time entries") and **New Invoice** (see "Create an invoice").

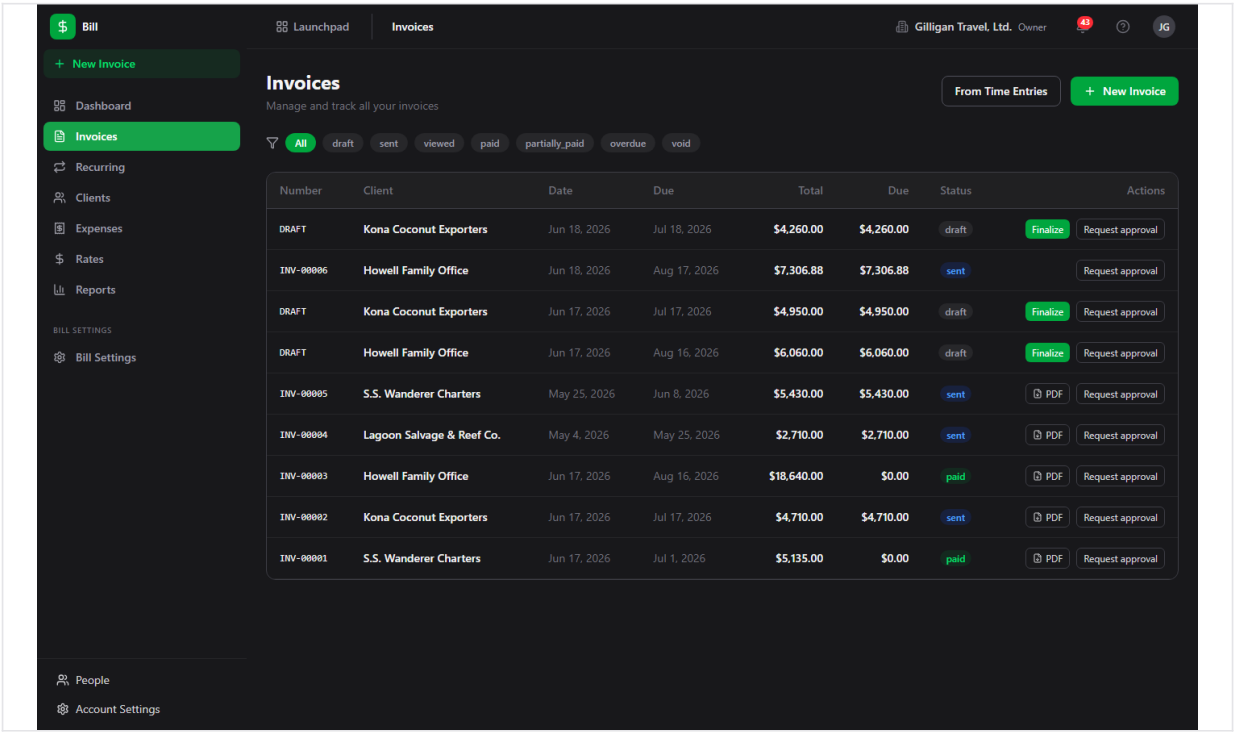


Figure 7.2 *Invoice list*

Create an invoice

The New Invoice form at `/bill/invoices/new` (titled **New Invoice**) builds a draft invoice manually. Reach it from the sidebar **New Invoice** button, the Invoices list **New Invoice** button, or a client's **+ New Invoice** link. Any member with write access can create a draft.

To create an invoice:

1. Click **New Invoice**.
2. Choose a **Client** from the select. The **Create Draft Invoice** button stays disabled until a client is chosen.
3. In the **Line Items** table, fill in Description, Qty, and Unit Price (in cents) for each row. The Amount column is computed. Click **+ Add line item** to add another row, or the **X** at the end of a row to remove it.
4. In the totals box, review the **Subtotal**, edit the **Tax Rate** if needed (the **Tax** and **Total** update automatically).
5. Optionally add **Notes (internal)**.
6. Click **Create Draft Invoice**. Bill creates the draft, adds each line item, and opens the new invoice's detail page. Use **Cancel** to return to the Dashboard instead.

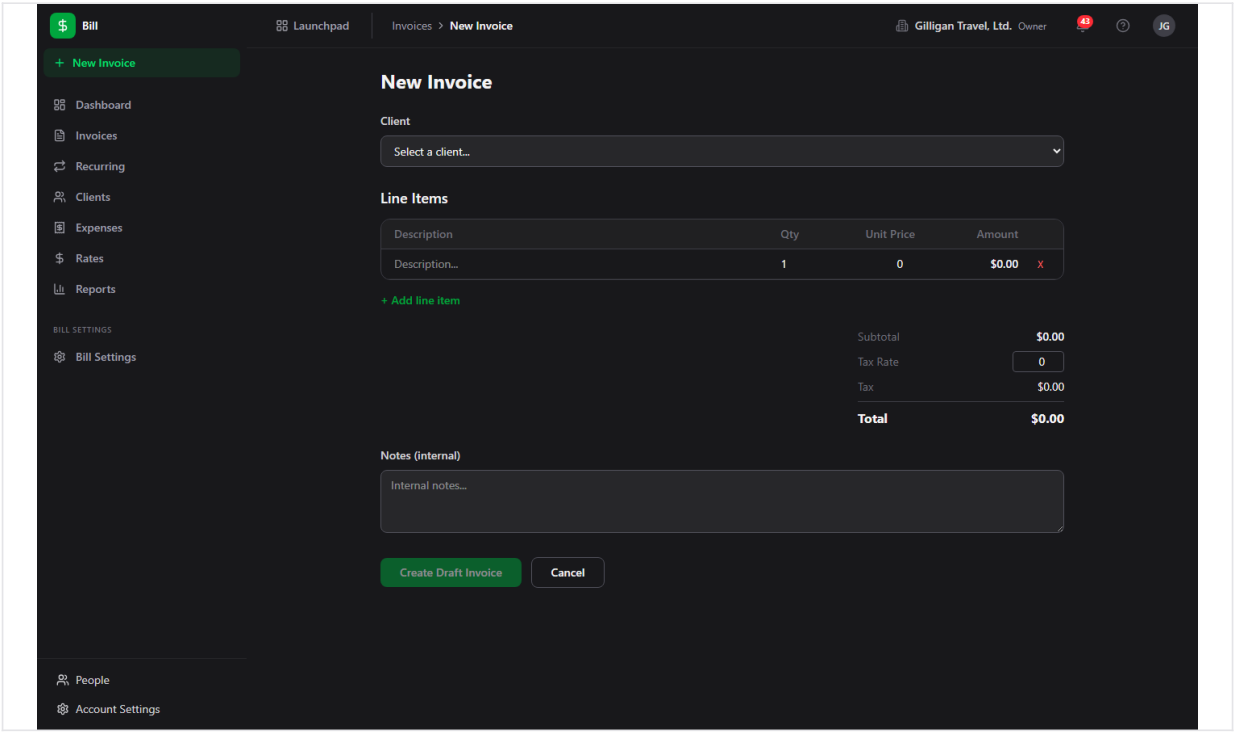


Figure 7.3 New invoice form

Invoice from time entries

The page at `/bill/invoices/from-time` is titled **Invoice from Time Entries** with the subtitle "Generate an invoice from Bam time tracking data." It turns billable hours logged in Bam into a draft invoice in one pass.

To generate an invoice from tracked time:

1. From the Invoices list, click **From Time Entries** (or go to `/bill/invoices/from-time`).
2. Pick a Bam project in the **Project** select.
3. Pick the recipient in the **Bill to client** select.
4. Set the **Date From** and **Date To** range.
5. Click **Generate Invoice**.

Every billable time entry on that project between the two dates is grouped by team member and task, priced with the configured billing rate, and added as a line item on a new draft invoice. Each contributing user needs a billing rate configured for the project (set one under Rates or through the API), or generation fails with an error explaining which rate is missing. When generation succeeds you land on the new draft's detail page, where you can review the line items, then Finalize and send. Use **Create Manually Instead** to switch to the blank New Invoice form.

This flow is backed by `POST /bill/api/v1/invoices/from-time-entries`, which accepts either a `date_from`/`date_to` range (what the wizard sends) or an explicit list of `time_entry_ids`. The `bill_create_invoice_from_time` MCP tool drives the same range form for agents; see "Working with AI agents."

Invoice detail

The detail page at `/bill/invoices/:id` shows everything about one invoice: header with number, client, and a status badge; From and To panels; Invoice Date, Due Date, Total, and Amount Due tiles; the read-only line items with a Subtotal / Tax / Discount / Total footer; the payments section; and an inline **PDF preview** iframe once a PDF exists.

The header actions vary by status:

- **Edit** appears on drafts and opens the edit form.
- **Finalize** appears on drafts; it assigns the number and locks the invoice. Admin or owner only.
- **Re-send** appears on sent or viewed invoices; it marks the invoice sent again and queues an email if email is configured. Admin or owner only.
- **Void** appears on any non-draft, non-void invoice; it cancels the invoice.
- **Duplicate** is always available; it clones the invoice (and its line items) into a new draft and opens that draft.
- **Request approval** opens an inline form that posts to Bam's approval queue, the same flow as the list view.
- **PDF** opens the invoice PDF.

To record a payment from the detail page:

1. Open the invoice. The **Record Payment** control appears when the status is not draft, void, or paid.

2. Enter the **Amount (cents)**.
3. Choose a **Method** (Bank Transfer, Credit Card, Check, Cash, Stripe, PayPal, or Other).
4. Click **Save Payment**.
5. The payment appears in the list (Date, Method, Amount). When recorded payments reach the total, the invoice flips to `paid`; a partial payment shows `partially_paid`. Bill rejects a payment that would exceed the remaining balance.

> **WARNING**
> A payment cannot push an invoice past paid. Bill rejects any receipt larger than the remaining balance, so record the actual amount received rather than rounding up to the total.

To void an invoice:

1. Open a sent, viewed, or paid invoice (void is blocked on drafts; delete a draft instead).
2. Click **Void** in the header.
3. The invoice moves to `void` and drops out of revenue reports.

To duplicate an invoice:

1. Click **Duplicate** in the header.
2. Bill creates a new draft with the same line items and opens it for editing.

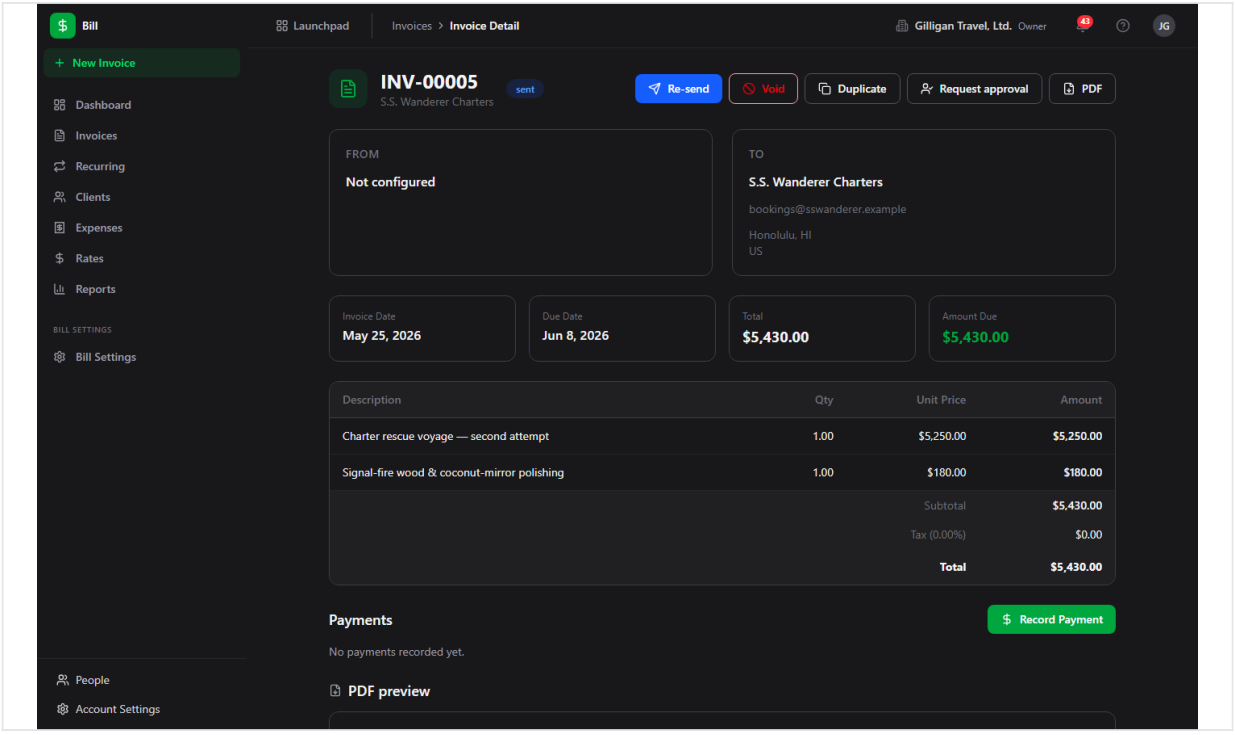


Figure 7.4 Invoice detail

Edit an invoice

The edit form at `/bill/invoices/:id/edit` is titled **Edit Invoice {number}** and is available only for drafts. A non-draft shows "Only draft invoices can be edited." with a "Back to Invoice" link.

The editable fields are **Tax Rate (%)**, **Internal Notes**, **Footer Text (on invoice)**, and **Terms & Conditions (on invoice)**. The edit form cannot add or remove line items and cannot change the client. To change line items, do it on the New Invoice form before creating, or duplicate the invoice and rebuild it.

To edit a draft:

1. On the invoice detail page, click **Edit**.
2. Adjust **Tax Rate (%)**, **Internal Notes**, **Footer Text (on invoice)**, or **Terms & Conditions (on invoice)**.
3. Click **Save Changes** (or **Cancel** to discard).

Clients

The Clients list at `/bill/clients` is titled **Clients** with the subtitle "Manage billing clients." Columns are Name, Email, Phone, and Terms (shown as "{n} days").

To add a client:

1. Open **Clients** in the sidebar.
2. Click **New Client** to reveal the inline form.
3. Enter **Name** and **Email**.
4. Click **Create Client**.

To open and edit a client:

1. Click a client row to open `/bill/clients/:id`.
2. Click **Edit** to enable inline editing of Contact Information (Name, Email, Phone) and Address & Tax (Address Line 1, Address Line 2, City, State, ZIP, Tax ID) plus Notes.
3. Click **Save** (or **Cancel**).

The client detail page also shows **Total Billed**, **Total Paid**, and **Outstanding Balance** summary cards computed from this client's invoices, the read-only address, tax ID, payment terms, and notes, and an **Invoices** table (Invoice #, Date, Status, Total, Paid, Due). The **+ New Invoice** link opens the New Invoice form; note that it does not pre-select this client, so choose the client again on the form.

Expenses

The Expenses list at `/bill/expenses` is titled **Expenses** with the subtitle "Track project expenses and receipts." Columns are Description, Category, Vendor, Date, Amount, Status, and Actions. A row of status pills filters the list: **All** (empty), **pending**, **approved**, **rejected**, and **reimbursed**. The **reimbursed** pill shows expenses you have paid back to the submitter.

To log an expense:

1. Open **Expenses** and click **New Expense** to go to `/bill/expenses/new` (titled **New Expense**).
2. Fill in **Description**, **Amount (cents)**, and **Date**.
3. Choose a **Category** (Software, Travel, Hardware, Contractor, Office, Meals, or Other) and optionally a **Vendor**.
4. Tick **Billable to client** if the cost should be invoiceable.
5. Click **Submit Expense**. The expense is created as `pending`.

To approve or reject an expense:

1. On the Expenses list, find a `pending` row.
2. Click **Approve** or **Reject** in its Actions cell. Approving and rejecting require an admin or owner role.
3. The status changes to `approved` or `rejected`. Only pending expenses can be approved or rejected; once approved, an expense is locked from edits.

To mark an approved expense reimbursed:

1. On the Expenses list, find an `approved` row.
2. Click **Mark reimbursed** in its Actions cell. This is an admin or owner action.
3. The status changes to `reimbursed`, the terminal step of the expense lifecycle (`pending` to `approved` to `reimbursed`). Only approved expenses can be marked reimbursed.

Editing an expense, deleting one, and attaching a receipt are supported by the API but have no button in the current UI. To attach a receipt or edit an expense, use the API (`POST /bill/api/v1/expenses/:id/receipt`, `PATCH /bill/api/v1/expenses/:id`) or the matching MCP tools.

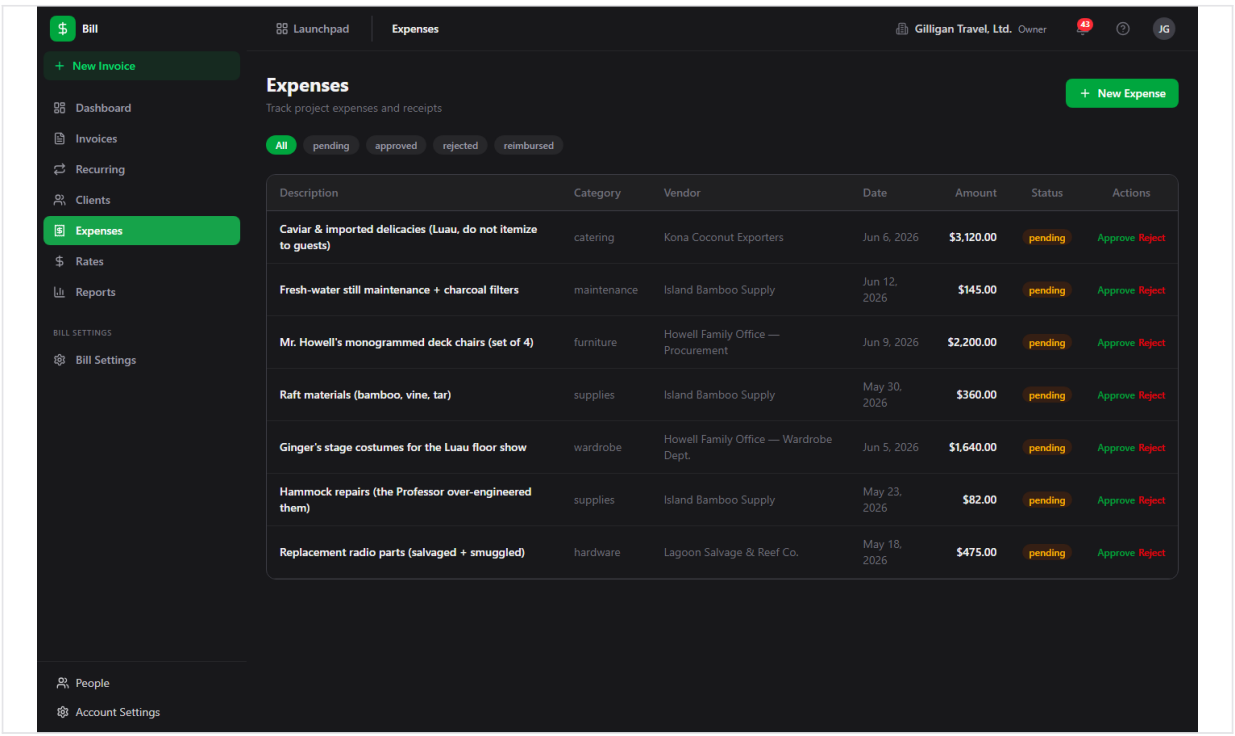


Figure 7.5 Expense list

Rates

The Rates list at `/bill/rates` is titled **Billing Rates** with the subtitle "Configure hourly, daily, or fixed rates per org/project/user." Columns are Rate, Type, Scope (derived as User + Project, User, Project, or Organization), Effective From, and Actions.

To add a rate:

1. Open **Rates** and click **New Rate** to reveal the inline form.
2. Enter **Rate (cents)** (the form defaults to 15000) and choose a **Type** (Hourly, Daily, or Fixed).
3. Click **Create Rate**.

To delete a rate, click **Delete** in its Actions cell.

The inline form only sets amount and type, so rates created in the UI are organization-scoped. Project-specific, user-specific, and date-bounded rates are supported by the data model and the resolver but must be created through the API (`POST /bill/api/v1/rates` with `project_id`, `user_id`, `effective_from`, or `effective_to`) or the `bill_create_rate` MCP tool, which accepts those scopes. When Bill bills time, it resolves the most specific rate that applies, preferring a user-and-project rate, then a user rate, then a project rate, then the organization rate, restricted to rates whose date range covers the work date.

Reports

The Reports view at `/bill/reports` is titled **Financial Reports**. It renders four tables:

- **Revenue by Month** (Month, Invoiced, Collected, Count).
- **Outstanding Aging** (Client, with 0-30, 31-60, 61-90, and 90+ day buckets).
- **Project Profitability** (Project, Revenue, Expenses, Profit, Margin). Only approved expenses count against profit.
- **Overdue Invoices** (Invoice, Client, Amount Due, Days Overdue). Each row links to the invoice. It computes overdue from each invoice's due date; the Dashboard tile and the Invoices list **Overdue** filter draw from the same set.

Revenue and profitability exclude draft, void, and written-off invoices. Outstanding and overdue also exclude paid invoices. Report views require Bill report permissions.

To read your reports:

1. Open **Reports** in the sidebar.
2. Review the four tables in place.
3. Click any **Overdue Invoices** row to jump to that invoice.

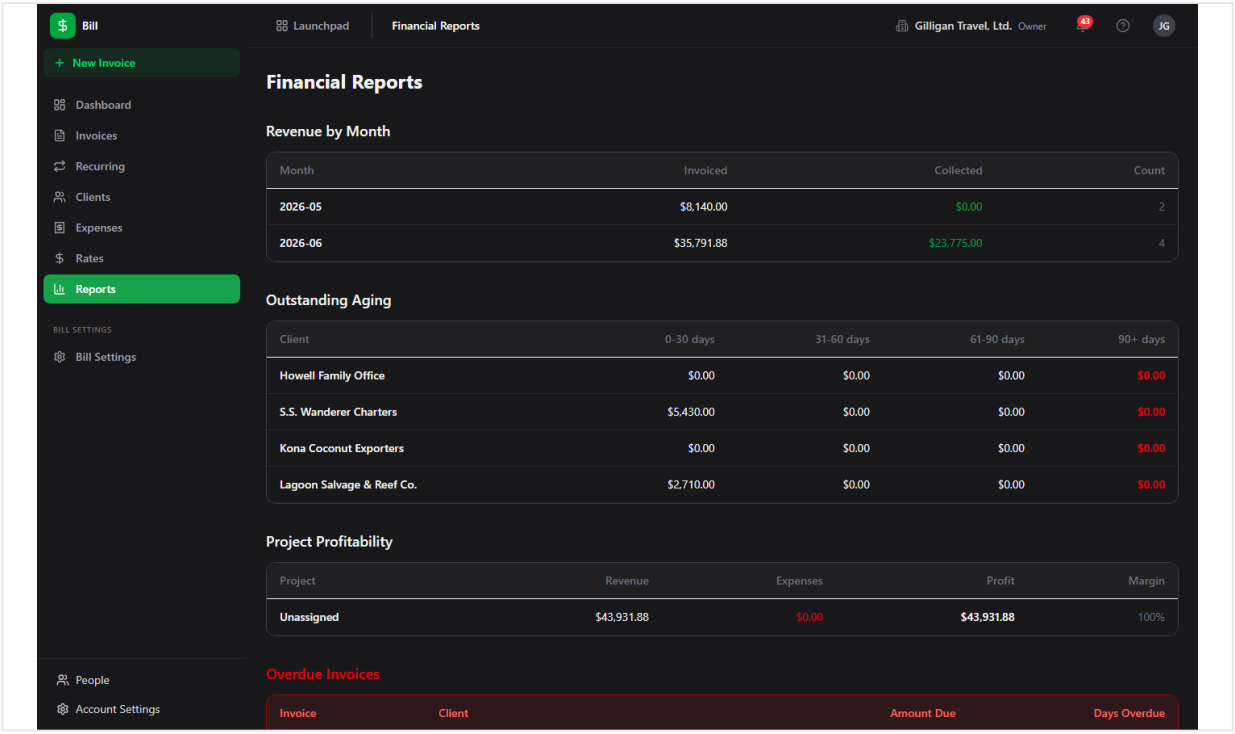


Figure 7.6 Financial reports

Settings

The Settings view at `/bill/settings` is titled **Billing Settings**. It has two sections.

Company Information holds Company Name, Email, Phone, Tax ID, and Address. These are snapshotted onto each invoice's "from" side at creation time.

Invoice Defaults holds Invoice Prefix, Default Tax Rate (%), Payment Terms (days), Default Payment Instructions, Default Footer Text, and Default Terms & Conditions. New invoices inherit these defaults.

To update your billing identity and defaults:

1. Open **Bill Settings** in the sidebar.
2. Edit any field under **Company Information** or **Invoice Defaults**.
3. Click **Save Settings**. The button shows "Saved!" on success.

The settings record also stores a company logo URL and a default currency, both reachable through the API and the `bill_update_settings` MCP tool, but neither has a field in the current Settings form.

Recurring billing

The Recurring page at `/bill/recurring` is titled **Recurring** with the subtitle "Subscription schedules that generate invoices automatically." It is where you set up standing fees that bill a client on a cadence without re-drafting the invoice each cycle.

The table columns are Name, Cadence, Next run, Generated (the count of invoices produced so far), Mode (Auto-finalize or Draft), Status, and Actions. A row of filter pills above it filters by status: **All** (empty), **active**, **paused**, and **cancelled**. Each schedule carries a client, a cadence (weekly, monthly, quarterly, or annually), a saved line-item template, an optional tax rate, a start date, an optional end date, and a mode.

The **Mode** determines what each run produces. With **Auto-finalize** on, every generated invoice is assigned a number and marked sent automatically. With it off (**Draft**), each run produces a draft for you to review, finalize, and send by hand. Schedules generate on their own through a daily worker sweep that materializes invoices from every active schedule whose next-run date has arrived; you can also force one immediately with **Generate now**.

> ***Set a recurring schedule to Auto-finalize and every cycle's invoice numbers, finalizes, and sends itself.***

To create a recurring schedule:

1. Open **Recurring** in the sidebar and click **New Recurring Schedule**.
2. Give the schedule a **Schedule name** and pick a **Client**.
3. Choose a **Cadence** (weekly, monthly, quarterly, or annually) and a **Start date**; optionally set an **End date** and a **Tax rate (%)**.
4. Tick **Auto-finalize generated invoices** to bill automatically, or leave it unchecked to generate drafts for review.
5. In the **Line items (template)** table, add a Description, Qty, and Unit price (cents) for each row. The Subtotal updates as you type.
6. Optionally add **Notes (internal)**, then click **Create schedule**. The schedule starts `active`.

The per-row actions depend on status:

- **Generate now** materializes an invoice immediately from the template (available on active and paused schedules). A banner reports the generated invoice number, or "draft" when the schedule is in draft mode.
- **Pause** stops generation on an active schedule; **Resume** restarts a paused one.
- **Cancel** stops all future generation permanently. It is an admin or owner action; existing invoices the schedule already produced are kept. Cancelling prompts for confirmation and cannot be undone.

Creating, updating, pausing, resuming, and generating-now follow the same write permission as drafting an invoice, so any member who can create invoices can run these. Cancelling a schedule is restricted to an admin or owner, mirroring invoice deletion. When a run produces an invoice, Bill emits a `recurring.invoice_generated` event on the `bill` source for Bolt rules to consume.

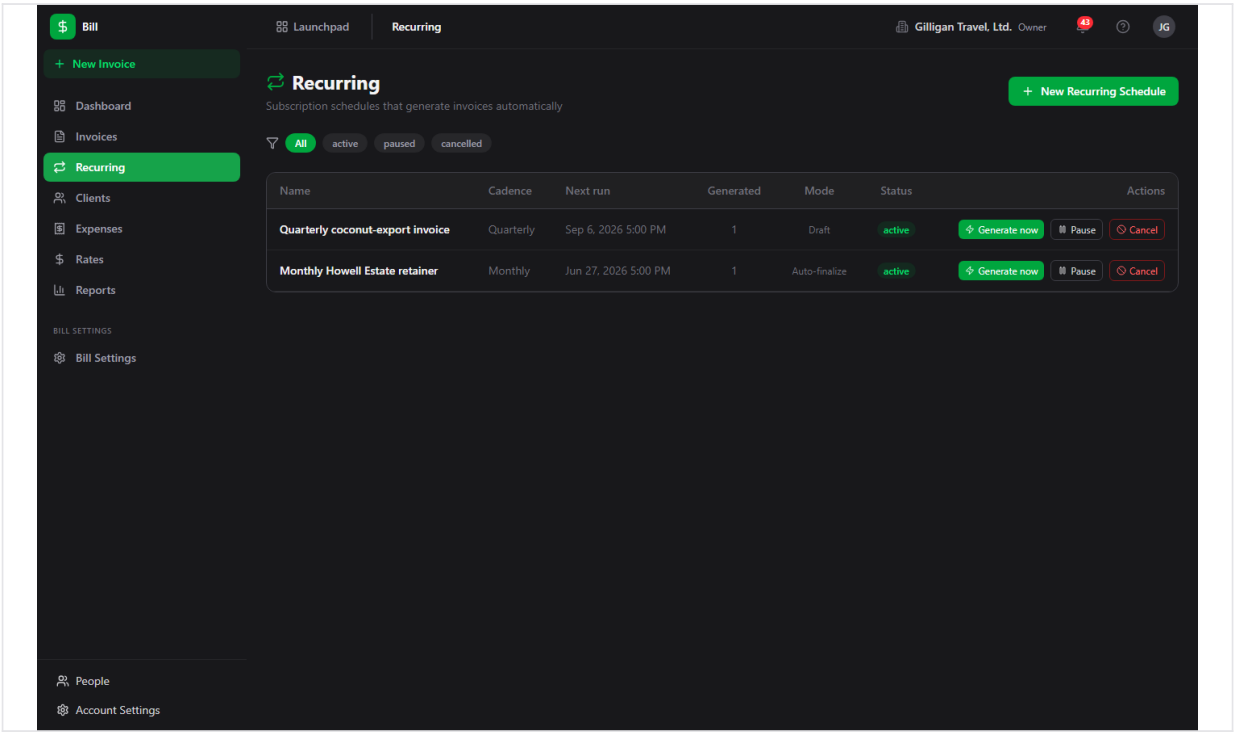


Figure 7.7 Recurring billing

PDF generation

Every invoice can be downloaded as a PDF. The **PDF** action on the list, the **PDF** action in the detail header, and the inline **PDF preview** iframe all render the invoice through Bill's PDF service (`GET /bill/api/v1/invoices/:id/pdf`). A draft's PDF is named `DRAFT.pdf`; a finalized invoice's PDF is named after its number.

When you finalize an invoice, Bill enqueues a background PDF job; when you send one, it enqueues an email job. While those run, the detail page can show a "PDF generating..." state. You do not need to trigger PDF generation manually; the PDF action always renders on demand.

Public invoice view

Each invoice carries a public view token. Anyone with the token URL can read a safe subset of the invoice and fetch its PDF without logging in, through `GET /bill/api/v1/invoice/:token` and `GET /bill/api/v1/invoice/:token/pdf`. The first open of the token URL marks a sent invoice as `viewed`. The public response omits internal fields such as the organization ID, the creator, the linked Bond deal, and tax IDs.

There is no Bill SPA page for the public view and no client-facing payment action on it. The public page is view-and-PDF only: the client cannot pay through it, so you record payments yourself from the invoice detail page. The token URL is intended to be shared with a client, typically from a sent-invoice email.

Working with AI agents

Bill exposes 47 MCP tools (catalogued in `apps/mcp-server/src/tools/bill-tools.ts`), which together cover the full invoicing and recurring-billing lifecycle from chat or a Bolt automation rule. Many tools resolve fuzzy identifiers: a client can be named by name or email, and a Bam project by name, and the tool resolves it to an ID. Mutating tools run under the same per-action permissions as the UI, so an agent acting as a member can build drafts but cannot finalize, send, or approve expenses unless its account has the admin or owner grant.

What agents commonly do:

- **List and inspect:** `bill_list_invoices`, `bill_get_invoice`, `bill_get_invoice_jobs`, `bill_list_clients`, `bill_get_client`, `bill_list_expenses`, `bill_list_rates`, `bill_get_settings`.
- **Build a draft:** `bill_create_invoice` (blank draft for a client), then `bill_add_line_item`, `bill_update_line_item`, and `bill_delete_line_item` per line. `bill_update_invoice` adjusts a draft's dates, tax, discount, terms, notes, and footer. For a Bond-deal handoff, `bill_create_invoice_from_deal` pulls the deal value into one line item; in a Bolt rule the deal ID is passed as `{{ event.deal.id }}` from a `deal.*` event, and it must be a UUID.
- **Bill tracked time:** `bill_create_invoice_from_time` takes a project, a billing client, and a `date_from`/`date_to` range, and the Bill API resolves the matching time entries, groups them by user and task, prices each group with the resolved rate, and builds one line item per group. This is the same range form the in-app wizard uses.
- **Finish and collect:** `bill_finalize_invoice` to assign the number and lock the invoice, `bill_send_invoice` to mark it sent, `bill_record_payment` to log receipts, and `bill_delete_payment` to reverse one. `bill_void_invoice`, `bill_duplicate_invoice`, and `bill_delete_invoice` cover the rest of the invoice lifecycle.
- **Clients:** `bill_create_client`, `bill_update_client`, and `bill_delete_client` manage the recipient records.
- **Expenses and rates:** `bill_create_expense`, `bill_update_expense`, `bill_delete_expense`, `bill_approve_expense`, and `bill_reject_expense` for costs;

`bill_create_rate`, `bill_update_rate`, `bill_delete_rate`, and `bill_resolve_rate` for billing rates (the create tool accepts project, user, and effective-date scoping that the UI form does not). Marking an approved expense reimbursed is a UI and REST action (`POST /bill/api/v1/expenses/:id/reimburse`) with no dedicated MCP tool.

- **Recurring schedules:** `bill_list_recurring_invoices` and `bill_get_recurring_invoice` to read schedules and their line-item template; `bill_create_recurring_invoice` to stand one up (client, cadence, mode, and template line items); `bill_update_recurring_invoice` to adjust it; `bill_pause_recurring_invoice`, `bill_resume_recurring_invoice`, and `bill_cancel_recurring_invoice` to control its lifecycle (cancel is admin or owner); and `bill_generate_recurring_invoice_now` to materialize an invoice on demand.
- **Reporting:** `bill_get_overdue`, `bill_get_revenue_summary`, `bill_get_profitability`, and `bill_get_outstanding` back the Reports view's data.
- **Settings:** `bill_get_settings` and `bill_update_settings` read and write the org billing identity and invoice defaults, including the logo URL and default currency the UI form omits.

A typical agent flow is: an agent drafts an invoice from a deal or from logged time, fills line items, and records a payment when one lands, while leaving the **Finalize** and **Send** steps for an admin or owner to approve. For high-stakes actions, route the work through the platform approval queue (the `proposal_create` / `proposal_list` / `proposal_decide` tools) so a human signs off before money moves.

One caution for agent-driven invoicing: the invoice number format the live finalizer produces is `{prefix}-{number:05d}` (for example `INV-00001`). Year-based forms like `INV-2026-0042` that appear in some seed data and examples are not what finalize generates.

Bill participates in the cross-cutting agentic platform that every BigBlueBam app shares:

- **Identity and heartbeat.** Agent and service accounts carry `users.kind` (`agent` or `service`), which is mirrored onto each activity row's `actor_type`, so Bill writes made by an agent are attributable. Agent runners report liveness with `agent_heartbeat`.
- **Approvals.** Beyond the in-app **Request approval** flow (which posts to Bam's queue), agents can stage any decision in the durable proposal queue with `proposal_create` and let a human resolve it with `proposal_decide`.
- **Unified activity.** Bill invoice and payment activity is queryable across apps through `activity_query` and `activity_by_actor`, and surfaces in the unified activity view alongside the other apps.
- **Policies and webhooks.** Per-agent kill switches and tool allowlists (the `bill.*` glob) gate which Bill tools a given agent may call. Subscribed Bill events can be pushed to agent runners over HMAC-signed outbound webhooks.
- **Visibility.** Before an agent surfaces Bill data (an invoice total, a client balance) into a shared surface in another app, it should run the platform `can_access` check for each

cited `bill.invoice` entity and drop anything the asker is not allowed to see, per `docs/reference/agent-conventions.md`.

For the full tool catalog, see the MCP-tools reference in `docs/apps/bill/`.

Working together (live presence)

Like every BigBlueBam app, Bill carries the persistent Bureau presence dock, so collaboration is ambient rather than scheduled. From anywhere in Bill you can see who is around and ring, knock, or invite a teammate into a live voice or video huddle that follows you in a floating window as you both move through the suite. Voice and video here are the digital equivalent of bumping into someone in the hallway or stopping by their desk, not a booked meeting. The deeper per-record collaboration (a presence strip showing who is on a specific item, and real-time co-editing of the same record) lives on the document, board, diagram, and task surfaces, described in the Introduction; in Bill, the shared layer is the always-on Bureau dock.

User Stories

Story: Set up your billing identity

Who: An org admin or owner setting up Bill for the first time. **Goal:** Make new invoices carry your company details and sensible defaults. **Before you start:** You are logged in to BigBlueBam and have Bill settings permission.

Steps

1. Open Bill at `/bill/` and click **Bill Settings** in the sidebar.
2. Under **Company Information**, fill in Company Name, Email, Phone, Tax ID, and Address.
3. Under **Invoice Defaults**, set the Invoice Prefix, Default Tax Rate (%), Payment Terms (days), Default Payment Instructions, Default Footer Text, and Default Terms & Conditions.
4. Click **Save Settings** and wait for the "Saved!" confirmation.

Result: Your company identity is stored. Every new invoice snapshots these details onto its "from" side and inherits the defaults you set.

Related: Add your first client next (Story: Add a client and send your first invoice). An agent can set the same fields, including the logo URL and default currency, with `bill_update_settings`.

Story: Add a client and send your first invoice

Who: Anyone who bills clients. (A member can build the draft; an admin or owner finalizes and sends.) **Goal:** Create a client, invoice them, and send it. **Before you start:** Billing identity is set in Bill Settings. You have permission to create clients and invoices.

Steps

1. Click **Clients** in the sidebar, then **New Client**.
2. Enter the client **Name** and **Email** and click **Create Client**.
3. Click **New Invoice** (sidebar or Invoices list).
4. Choose your new client in the **Client** select.
5. In **Line Items**, enter a Description, Qty, and Unit Price (in cents) for each row, using **+ Add line item** for more rows.
6. Set the **Tax Rate** if needed and review **Subtotal**, **Tax**, and **Total**.
7. Click **Create Draft Invoice**. You land on the draft's detail page.
8. Click **Finalize** in the header (admin or owner). The invoice gets its number and locks.
9. Click **Re-send** to dispatch it (this queues an email if email is configured).

Result: The client has a finalized, sent invoice with a permanent number. It appears in the Invoices list as `sent`.

Related: Record the payment when it arrives (Story: Record a payment to close out). An agent can do the same with `bill_create_client`, `bill_create_invoice`, `bill_add_line_item`, `bill_finalize_invoice`, and `bill_send_invoice`.

Story: Record a payment to close out

Who: Anyone tracking receivables. **Goal:** Log payments against an invoice and watch it reach paid. **Before you start:** The invoice is finalized (status is sent, viewed, or partially_paid).

Steps

1. Open **Invoices** and click the invoice row.
2. In the payments section, find **Record Payment**.
3. Enter the **Amount (cents)** received.
4. Choose a **Method** (for example, Bank Transfer).
5. Click **Save Payment**.
6. Repeat for additional payments until the recorded total reaches the invoice total.

Result: A partial payment shows the invoice as `partially_paid`; when payments reach the total it becomes `paid`. The Dashboard **Paid** and **Outstanding** tiles reflect the change.

Related: Bill rejects a payment larger than the remaining balance. An agent can record payments with `bill_record_payment` and reverse one with `bill_delete_payment`.

Story: Capture, approve, and reimburse an expense

Who: A team member logging a project cost, and an admin or owner reviewing it. **Goal:** Record an expense and move it through approval and reimbursement. **Before you start:** You have permission to create expenses; the reviewer has the admin or owner role needed to approve and reimburse.

Steps

1. Click **Expenses** in the sidebar, then **New Expense**.
2. Fill in **Description**, **Amount (cents)**, and **Date**.
3. Choose a **Category** and optionally a **Vendor**.
4. Tick **Billable to client** if it should be invoiceable, then click **Submit Expense**. The expense is `pending`.
5. The reviewer opens the Expenses list, finds the pending row, and clicks **Approve** (or **Reject**).
6. After paying the submitter back, the reviewer clicks **Mark reimbursed** on the approved row.

Result: The expense moves to `approved` (or `rejected`), and an approved expense can be moved on to `reimbursed`. Approved expenses count against project profit in the Project Profitability report and can no longer be edited.

Related: Receipt upload and expense edits are available through the API and the `bill_update_expense` tool. An agent can log expenses with `bill_create_expense`, list them with `bill_list_expenses`, and decide them with `bill_approve_expense` / `bill_reject_expense`. Marking reimbursed is a UI and REST action with no dedicated MCP tool.

Story: Send an invoice for approval before billing

Who: Someone who needs sign-off before an invoice goes out. **Goal:** Route an invoice to a colleague for approval. **Before you start:** The invoice exists. You know which Bam user should approve it.

Steps

1. On the Invoices list, click **Request approval** on the invoice row (or use **Request approval** on the invoice detail page).
2. In the **Request approval for {number}** dialog, pick an **Approver** from the select.
3. Optionally type a **Message (optional)**.
4. Click **Send request**.

Result: A request is posted to Bam's approval queue (subject type `bill.invoice`) and a Banter DM goes to the approver via Bolt. The approver reviews and decides inside Bam.

Related: After approval, finalize and send the invoice (Story: Add a client and send your first invoice). An agent can stage a comparable sign-off in the platform proposal queue with `proposal_create`.

Story: Turn a won Bond deal into an invoice

Who: An account owner, or a Bolt rule, converting closed-won revenue into a bill. **Goal:** Generate a draft invoice from a Bond deal with the deal value as a line item. **Before you start:** The deal exists in Bond and you have its UUID. A matching Bill client exists (or you know its name/email).

Steps

1. Have an agent call `bill_create_invoice_from_deal` with the deal's UUID and the billing client. In a Bolt rule, pass `{{ event.deal.id }}` from the triggering `deal.*` event.
2. The tool creates a draft invoice with the deal value as one line item and emits `invoice.created`.
3. In Bill, open the new draft, adjust line items or tax if needed, and click **Finalize** (admin or owner).
4. Click **Re-send** to send it.

Result: A draft invoice tied to the deal becomes a finalized, sent invoice. A human stays in control of finalize and send.

Related: This is the canonical Bond-to-Bill handoff. See "Working with AI agents" for the tool details.

Story: Bill tracked time from a project

Who: Anyone billing logged hours. **Goal:** Generate an invoice from Bam time entries over a date range. **Before you start:** A rate exists that covers the work (set one under Rates, via the API, or with `bill_create_rate` for project/user scope). You know the project, the client, and the date range to bill.

Steps

1. Confirm a rate applies by checking Rates, or have an agent call `bill_resolve_rate` for the project, user, and date.
2. From the Invoices list, click **From Time Entries**.
3. Pick the **Project**, the **Bill to client**, and the **Date From** / **Date To** range.
4. Click **Generate Invoice**. Bill groups every billable entry on that project in the range by user and task, prices each group with the resolved rate, and builds one line item per group on a new draft.
5. Review the generated line items on the draft, click **Finalize**, then **Re-send**.

Result: A draft invoice built from real tracked time, ready to finalize and send.

Related: Configure rates first (the Rates feature). An agent can do the same with `bill_create_invoice_from_time` (project, client, and a `date_from`/`date_to` range), and resolve a rate with `bill_resolve_rate`.

Story: Put a standing fee on a recurring schedule

Who: Anyone who bills the same client the same amount on a regular cadence (a retainer, a subscription, a quarterly standing order). **Goal:** Bill a client automatically on a cadence without re-drafting the invoice each cycle. **Before you start:** The client exists in Bill. You have permission to create invoices (creating a schedule uses the same write permission).

Steps

1. Click **Recurring** in the sidebar, then **New Recurring Schedule**.
2. Enter a **Schedule name** and pick the **Client**.
3. Choose a **Cadence** (weekly, monthly, quarterly, or annually) and a **Start date**; set an **End date** and a **Tax rate (%)** if needed.
4. Decide the mode: tick **Auto-finalize generated invoices** to bill and send automatically, or leave it unchecked to generate drafts you review first.
5. Fill in the **Line items (template)** table with the descriptions, quantities, and unit prices (cents) to bill each cycle.
6. Click **Create schedule**. It starts `active` with a next-run date on the start date.

Result: The schedule appears in the Recurring list as `active`. A daily worker sweep generates an invoice from the template whenever the next-run date arrives, advancing the next run by the cadence. Auto-finalize schedules send each invoice automatically; draft schedules leave each one for you to finalize and send.

Related: Use **Generate now** to produce an invoice immediately, **Pause** / **Resume** to stop and restart generation, and **Cancel** (admin or owner) to end the schedule while keeping the invoices it already produced. An agent can do all of this with `bill_create_recurring_invoice`, `bill_pause_recurring_invoice`, `bill_resume_recurring_invoice`, `bill_cancel_recurring_invoice`, and `bill_generate_recurring_invoice_now`. Build a Bolt rule on the `recurring.invoice_generated` event to react when a schedule bills.

Story: Close out a billing period

Who: A finance owner reviewing the month. **Goal:** See revenue, aging, profitability, and what is overdue. **Before you start:** You have Bill report permissions.

Steps

1. Click **Reports** in the sidebar.
2. Read **Revenue by Month** for invoiced and collected totals.
3. Read **Outstanding Aging** to see balances by client and age bucket.
4. Read **Project Profitability** to compare revenue against approved expenses per project.
5. Read **Overdue Invoices**, and click any row to open and chase that invoice.

Result: A complete financial snapshot for the period, with overdue invoices computed from due dates.

Related: An agent can pull the same data with `bill_get_revenue_summary`, `bill_get_profitability`, `bill_get_outstanding`, and `bill_get_overdue`. The Dashboard's **Overdue** tile and the Invoices list **Overdue** filter draw from the same overdue set.

Story: Chase overdue invoices automatically

Who: A finance owner who wants overdue reminders to send themselves. **Goal:** Have Bill email clients whose invoices are past due, and let automation escalate. **Before you start:** Invoices have due dates; SMTP is configured for real email (otherwise reminders are logged).

Steps

1. Finalize and send invoices as usual so they have due dates.
2. Each day at 09:00 UTC, Bill's reminder sweep finds invoices past their due date that are not paid or void, and that were not reminded in the last 7 days.
3. The sweep emails the client (or logs the reminder if SMTP is not set), records the reminder timestamp and count, and emits an `invoice.overdue` event on the `bill` source.
4. Build a Bolt rule on `invoice.overdue` to escalate or notify the account owner.

Result: Overdue clients get periodic reminders without manual effort, and downstream automation can react to the `invoice.overdue` event.

Related: Bill also emits `invoice.created`, `invoice.finalized`, `invoice.sent`, `invoice.paid`, and `payment.recorded`, all on the `bill` source, for Bolt rules to consume.

Related

- **Bam** (`/b3/`) - Projects, tasks, and time entries feed time-based invoicing and rate resolution; invoice approvals route through Bam's approval queue.
- **Bond** (`/bond/`) - Won deals convert into draft invoices via `bill_create_invoice_from_deal`; a Bill client can link to a Bond company.
- **Bolt** (`/bolt/`) - Automation rules consume Bill events such as `invoice.paid`, `invoice.overdue`, `payment.recorded`, and `recurring.invoice_generated`, and can trigger `bill_create_invoice_from_deal`.
- **Banter** (`/banter/`) - Approval requests reach the approver as a Banter DM.
- Bill's own docs in `docs/apps/bill/`: the MCP-tools reference (the full 47-tool catalog) and `guide.md`.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. How is money stored and entered throughout Bill?

a) In dollars and cents
b) In integer cents
c) In the org's default currency only
d) As a free-text string

2. Which invoice action requires an admin or owner role rather than plain write access?

a) Adding a line item to a draft
b) Creating a draft invoice
c) Finalizing an invoice
d) Managing clients

3. Once you finalize an invoice, what two things become locked?

4. Is 'overdue' a stored invoice status?

a) Yes, the app sets it like draft or paid
b) No, it is computed from the due date
c) Only on recurring invoices
d) Only after a reminder email sends

5. Name the six invoice status values that the app actually sets.

6. What three states make up the expense lifecycle from submission to terminal step?

7. When Bill bills tracked time, which rate does it use?

a) The organization rate always
b) The most specific rate that applies
c) The highest rate available
d) The rate on the first time entry

8. What happens when you record a payment larger than the remaining balance?

a) The invoice flips to paid and the extra is credited
b) Bill rejects the payment
c) The overage becomes a new draft invoice
d) The client is refunded automatically

9. What is the difference between a recurring schedule's Auto-finalize mode and its Draft mode?

10. Can a client pay through the public invoice view that the view token opens?

a) Yes, it has a Pay Now button
b) No, it is view-and-PDF only and you record payments yourself
c) Only with Stripe configured
d) Only for partially_paid invoices

11. What invoice number format does the live finalizer produce, and what form should you not expect?

12. Which expenses count against profit in the Project Profitability report?

a) All logged expenses
b) Only approved expenses

c) Only billable expenses
d) Only reimbursed expenses

13. Marking an approved expense reimbursed has no dedicated MCP tool. How does an agent or user do it?

CHAPTER 8

Blank

Build it, publish it, collect it.

Blank is the intake layer for the whole suite. You build a form by dragging fields from a palette, decide who can reach it, and publish to a public URL that anyone with the link can fill out, no code required. Responses land in a Responses table and an Analytics view, and they can route into Bond or Helpdesk when that wiring is configured. By the end of this chapter you will be able to design a form, gate who can submit it, and read the numbers that come back.

At a glance

- Drag-and-drop builder with 21 field types, from Short Text to NPS to File Upload
- A public, login-free form at /forms/<slug> once you publish
- Layered access: visibility, sign-in requirement, and allowed email domains
- Real file uploads validated by a background worker, with status pills per submission
- Full MCP coverage, so an agent can draft, publish, and report on a form

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Forms list . Form Builder . Field Settings . Form Settings dialog . Access control and notifications . Publish dialog

> *Blank is the forms and surveys app in BigBlueBam. Build a form with drag-and-drop fields, publish it to a public URL, and collect and review responses without writing any code.*

Overview

Blank lets you build forms and surveys, publish them, and gather responses from anyone with the link. You start from the Forms list, design the form[68] in a visual builder by adding fields from a palette, set who can see it, then publish to get a shareable public URL. Responses flow back into a Responses table and an Analytics view.

> ***Design in a visual builder, then publish to a public URL anyone with the link can fill out, no code required.***

The core objects are **forms** and **submissions**. A form holds a set of ordered fields and a collection of behavior settings (visibility[69], confirmation[70], theming, response limits). A submission[71] is one filled-in response, stored against the form. Forms move through a lifecycle from draft to published, and an author can later close a form to stop accepting responses.

Blank fits alongside the rest of the suite as the intake layer. Forms can be scoped to your organization or to a specific Bam project, and submitted responses can be routed into Bond (as a contact) or Helpdesk (as a ticket) when that routing is configured at the API level.

Authoring happens in the Blank SPA, which requires a BigBlueBam login. Filling out and submitting a published form does not require a login when the form is set to public visibility.

Key concepts

- **Form** - the top-level object you build and share. It has a name, a slug (used in its public URL), a description, fields, and behavior settings.
- **Field**[72] - a single input or element on the form. Fields are ordered, can be marked required, and each has a stable **field key** (letters, digits, and underscores; must start with a letter or underscore) used as the storage key for answers.
- **Field type**[73] - what kind of input a field is, for example Short Text, Email, Rating, NPS, or Dropdown. You pick the type from the builder palette when you add a field. A few layout-only types (Section Header, Paragraph, Hidden Field, and Page Break) carry no submitted answer.

---

[68] **Form.** the top-level object you build and share. It has a name, a slug (used in its public URL), a description, fields, and behavior settings.

[69] **Visibility.** who can reach the form through its URL: **Public** (anyone with the link), **Organization** (members of your org), or **Project members** (members of a chosen Bam project).

[70] **Confirmation.** what a respondent sees after submitting: a message, a redirect, or a custom page.

[71] **Submission.** one response to a form. Submissions are listed in the Responses view and aggregated in Analytics.

[72] **Field.** a single input or element on the form. Fields are ordered, can be marked required, and each has a stable **field key** (letters, digits, and underscores; must start with a letter or underscore) used as the storage key for answers.

[73] **Field type.** what kind of input a field is, for example Short Text, Email, Rating, NPS, or Dropdown. You pick the type from the builder palette when you add a field. A few layout-only types (Section Header, Paragraph, Hidden Field, and Page Break) carry no submitted answer.

- **Form status**[74] - the lifecycle state of a form: **draft** (still being built), **published** (live and accepting responses), or **closed** (no longer accepting responses). New forms start as draft.
- **Form type**[75] - a label on the form (Public, Internal, or Embedded) found on the per-form Settings page.
- **Visibility** - who can reach the form through its URL: **Public** (anyone with the link), **Organization** (members of your org), or **Project members** (members of a chosen Bam project).
- **Logic (conditional display)** - a field can be configured to show only when another field meets a condition (for example, equals or contains a value). This is stored on the field and is available to agents through the MCP field tools; the visual builder does not yet expose a conditional editor.
- **Submission** - one response to a form. Submissions are listed in the Responses view and aggregated in Analytics.
- **Confirmation** - what a respondent sees after submitting: a message, a redirect, or a custom page.
- **Public form URL**[76] - the shareable link to your published form, in the form `https://YOUR-DOMAIN/forms/<slug>`. This is the server-rendered page the Publish dialog hands you.

> **NOTE**
> Conditional display is stored on the field and reachable through the MCP field tools, but the visual builder has no conditional editor yet. To show a field only when another answer matches, set it up through an agent or the API for now.

Where to find it

Blank is served at `/blank/`. Reach it from the Launchpad app switcher in the top-left of the layout chrome, or go directly to `/blank/`.

The left sidebar has two items: **Forms** (the Forms list at `/blank/`) and **Blank Settings** (a read-only app information page at `/blank/settings`). Pressing the question-mark key opens this Help.

Prerequisites:

- A BigBlueBam session or a `bbam_` / `bbam_svc_` API key. Unauthenticated visitors to the authoring SPA see a "Please log in to BigBlueBam first" screen with a link to `/b3/`.
- Membership in an organization. All forms and submissions are scoped to your org.

---

[74] **Form status.** the lifecycle state of a form: **draft** (still being built), **published** (live and accepting responses), or **closed** (no longer accepting responses). New forms start as draft.
[75] **Form type.** a label on the form (Public, Internal, or Embedded) found on the per-form Settings page.
[76] **Public form URL.** the shareable link to your published form, in the form `https://YOUR-DOMAIN/forms/<slug>`. This is the server-rendered page the Publish dialog hands you.

- Publishing a form requires the `blank.form.publish` permission. Closing a form requires `blank.form.close`, and deleting a submission requires `blank.submission.delete`.

Feature reference

Forms list

The Forms list at `/blank/` is the home screen. It is headed **Forms** with the subtitle "Build forms and surveys to capture responses from anyone." Each form appears as a card showing its name, description, a status pill (draft, published, or closed), the field count, the submission count, and a relative "updated" time.

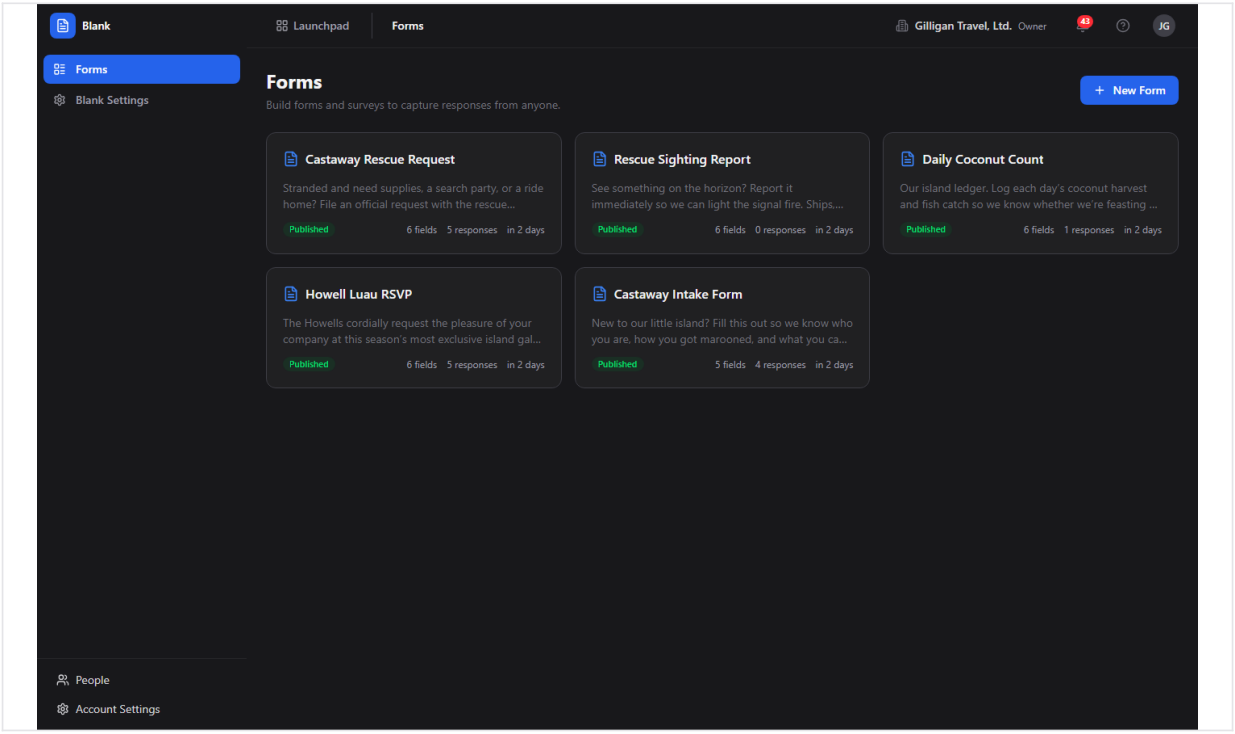


Figure 8.1 Forms list

To create a form:

1. Click **New Form** (the button with the plus icon, top-right).
2. Blank creates a draft named "Untitled Form" with a generated slug and opens it in the builder.
3. Continue in the builder to rename it and add fields.

To open an existing form:

1. Click the form's card. The builder opens at `/blank/forms/<id>/edit`.

Each card has an overflow menu (the three-dot icon that appears on hover). It contains:

- **Preview** - opens the full-page Form Preview.
- **Responses** - opens the Responses table for the form.
- **Duplicate** - clones the form and its fields into a new draft. The copy's name gets "(Copy)" appended and it starts as a draft.

- **Delete** - permanently removes the form. This also removes its fields and submissions. This action is destructive and cannot be undone.

> **WARNING**
> Deleting a form removes its fields and every submission with it, permanently and with no undo. If you only want to stop new responses, close the form or turn off Accept Responses instead, which leaves the data intact.

If you have no forms yet, the list shows an empty state: "No forms yet" with "Create your first form to start collecting responses."

Form Builder

The builder at `/blank/forms/<id>/edit` is where you design a form. It has a left field palette, a center canvas, an optional right field-config panel, and an optional live-preview panel.

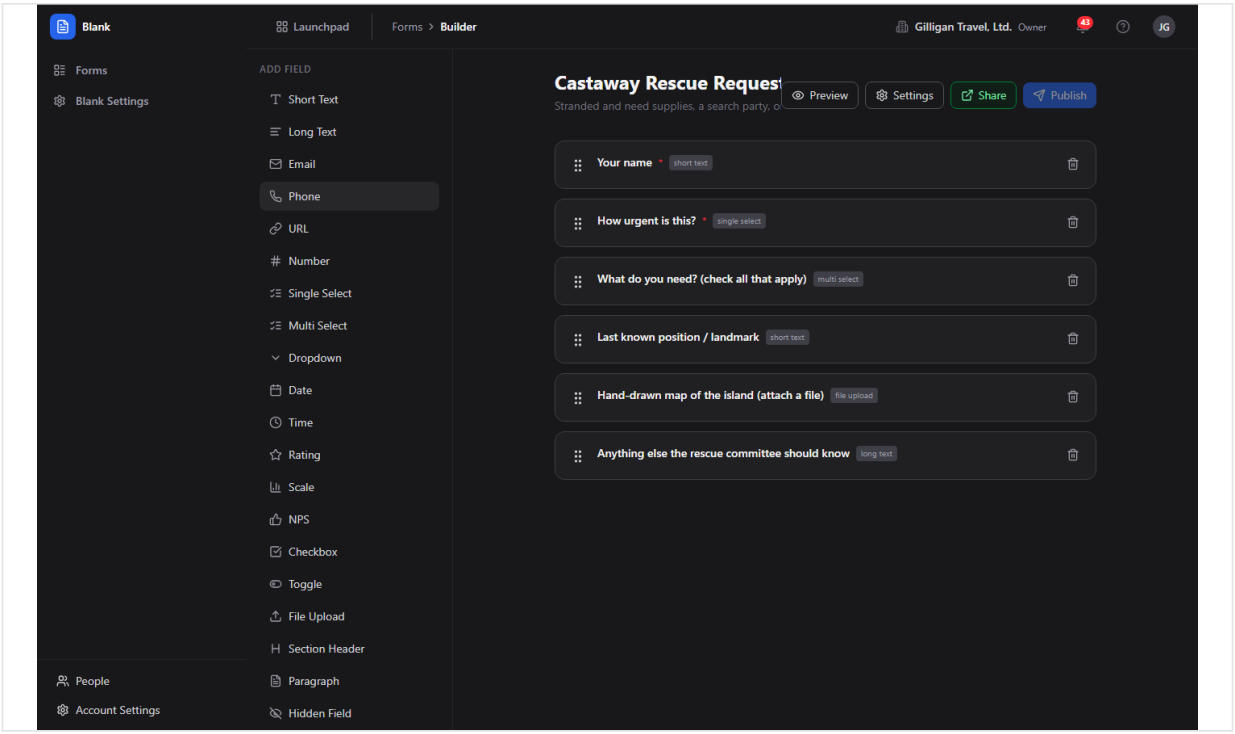


Figure 8.2 Form builder

The left palette is headed **Add Field**. Click any type to append a field of that type to the canvas. The available types are:

Short Text, Long Text, Email, Phone, URL, Number, Single Select, Multi Select, Dropdown, Date, Time, Rating, Scale, NPS, Checkbox, Toggle, File Upload, Section Header, Paragraph, Hidden Field, and Page Break.

To name the form:

1. In the center canvas, edit the form Title input at the top.
2. Edit the Description input below it. Both save automatically as you type.

To add a field:

1. Click a type in the **Add Field** palette on the left.
2. The new field appears at the bottom of the canvas. Click it to configure it in the Field Settings panel on the right.

To reorder fields:

1. Drag a field row by its grip handle and drop it in the new position. The order is saved automatically.

To delete a field:

1. Click the trash icon on the field row in the canvas.

The top-right action bar of the builder has these controls:

- **Preview** (eye icon) - toggles the live-preview panel on the right.
- **Settings** (gear icon) - opens the Form Settings dialog (visibility, expiration, and project).
- **Share** (external-link icon) - appears only after the form is published. It reopens the "Your form is live" dialog with the public URL.
- **Publish** (send icon) - publishes the form. It is disabled once the form is already published.

If the form has no fields, the canvas shows "Click a field type on the left to add it".

The palette's **Page Break** item splits a form into pages. Adding one inserts a layout-only divider in the builder rather than a stray input, and the public form pages at that boundary. See "Multi-page forms" below. You can also page a form by setting the **Page Number** on individual fields in Field Settings.

Field Settings

When you select a field in the canvas, the right panel is headed **Field Settings**. The controls available depend on the field's type. Every control saves to the field when you change it or leave the input.

Common controls:

- **Label** - the visible question text.
- **Key** - the field key (shown in a monospace font). This is the storage key for the answer.
- **Description** - helper text shown under the label.
- **Placeholder** - placeholder text inside the input.
- **Required** - a checkbox that makes the field mandatory.
- **Page Number** - which page the field belongs to in a multi-page form.

Type-specific controls:

- **Options editor** - for Single Select, Multi Select, and Dropdown. Use "+ Add option" to add choices.
- **Min Length** and **Max Length** - for text fields.

- **Min Value** and **Max Value** - for numeric fields.
- **Scale Min** and **Scale Max** - for Scale fields.
- **Regex Pattern** - an optional validation pattern applied to string answers.

Layout-only fields (Section Header, Paragraph, Hidden Field, and Page Break) do not show validation controls like the regex pattern, because they collect no answer.

Form Settings dialog

The **Settings** button in the builder action bar opens the Form Settings dialog, titled **Form Settings**. It covers visibility, expiration, project scoping, who may submit (sign-in and allowed email domains), and submission notifications (email and Banter).

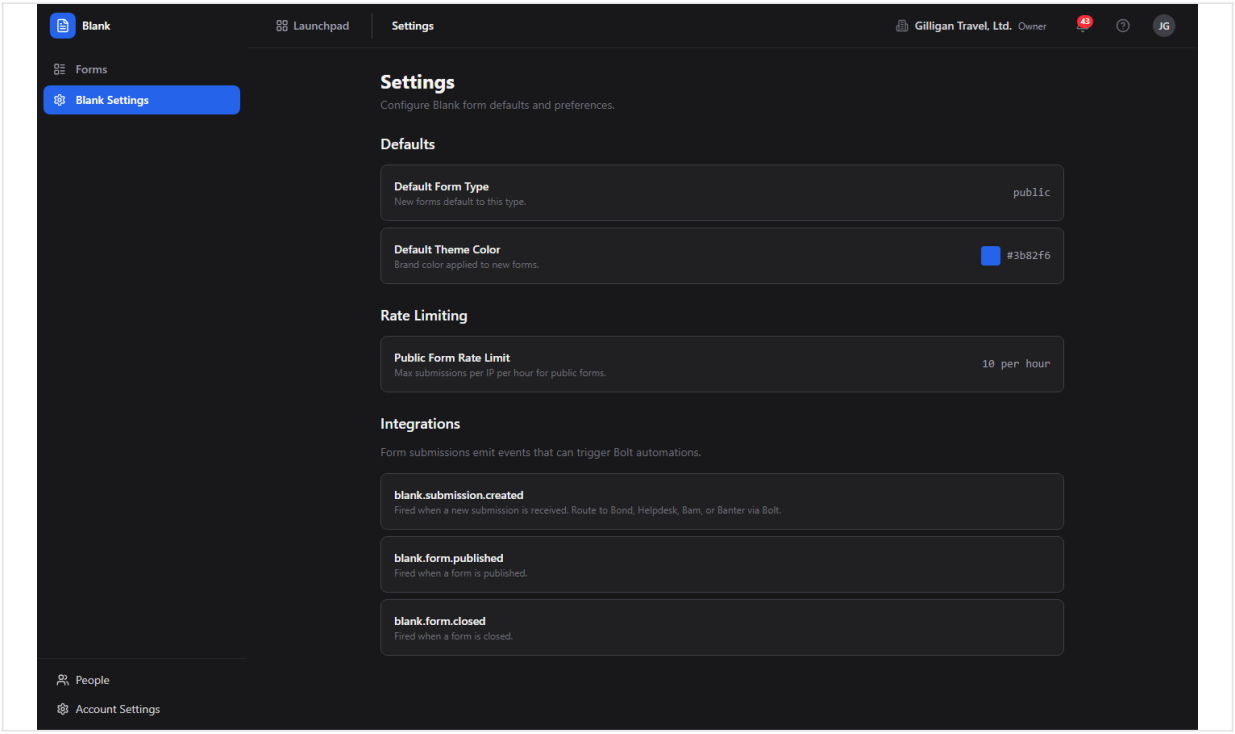


Figure 8.3 Form settings

To set visibility:

1. Click **Settings** in the builder.
2. Under Visibility, choose one of the radio options:
3. If you choose **Project members**, a **Project** picker appears. Select a Bam project.
4. Click **Save**. Save is disabled if you chose Project members without picking a project.

To set an expiration:

1. In the Form Settings dialog, use the **Expires at** datetime field to set when the form stops accepting responses.
2. Use the **Clear** button next to it to remove an expiration.
3. Click **Save**.

Use **Cancel** to close the dialog without saving.

Access control and notifications

Below visibility and expiration, the Form Settings dialog has controls for who may submit and what happens when they do. These are enforced and delivered for real.

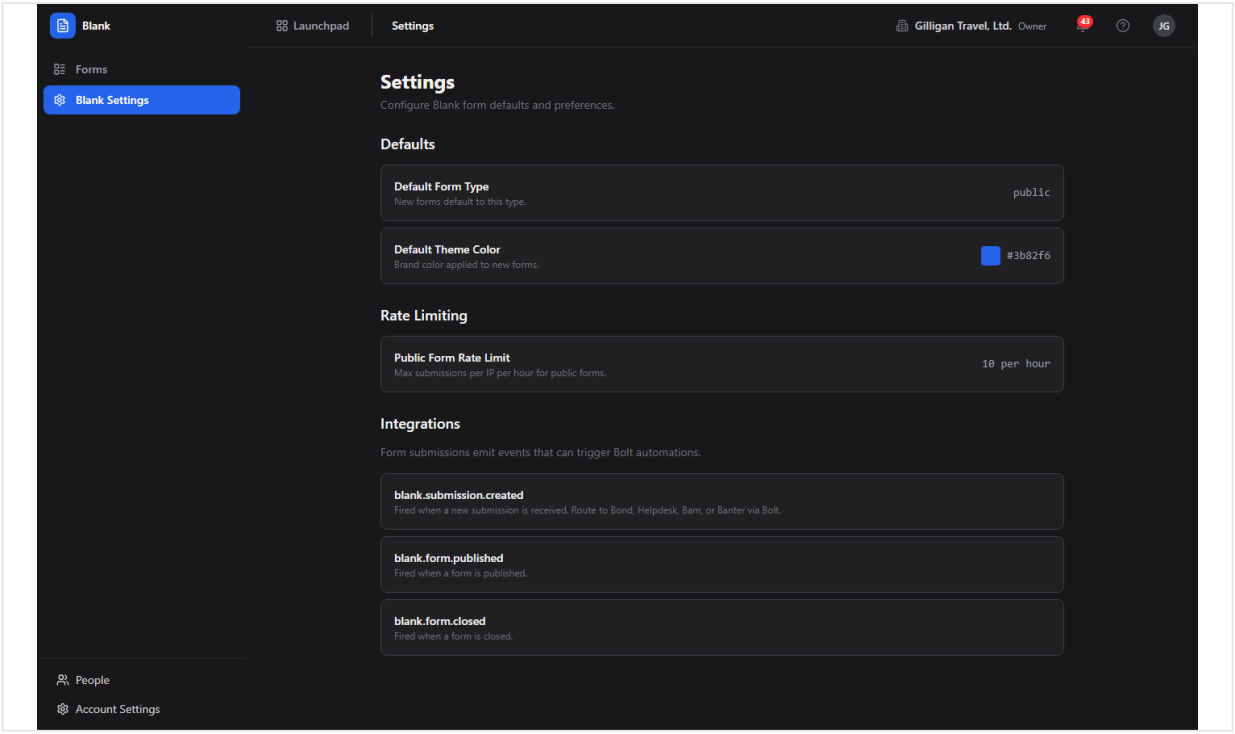


Figure 8.4 Access control and notifications

- **Require sign-in to submit** - a checkbox. When on, a visitor must be signed in to a BigBlueBam account before they can submit (or upload a file). An anonymous attempt is rejected with a "You must be signed in to submit this form" error.
- **Allowed email domains** - a comma-separated list, for example `acme.com, example.org`. When set, only a submitter whose email is on one of these domains may submit; everyone else is rejected with a "Submissions from ... are not allowed" error. The submitter's email is taken from their signed-in account, the form's email field, or an explicit email in the response, in that order. Leave the field blank to allow any domain. A leading `@` is tolerated and stripped.
- **Email me on new submissions** - a checkbox that turns on notification emails for this form.
- **Notification recipients** - a comma-separated list of email addresses to notify on each new submission. Used together with **Email me on new submissions**.
- **Post to Banter channel** - an optional picker. When a channel is chosen, each new submission also posts a short message to that Banter channel.

To gate who can submit:

1. Click **Settings** in the builder.

2. Check **Require sign-in to submit** to force authentication, and/or enter one or more domains under **Allowed email domains**.
3. Click **Save**.

To get notified of submissions:

1. Click **Settings** in the builder.
2. Check **Email me on new submissions** and enter one or more addresses under **Notification recipients**.
3. (Optional) Pick a **Post to Banter channel** target.
4. Click **Save**.

> *Note: the sign-in and allowed-domain gates are enforced on the public submit and file-upload paths. Combine them with **Visibility** (org or project membership) for layered access control. Notification and confirmation emails are dispatched through your deployment's SMTP transport; where SMTP is not configured, the worker logs the message instead of delivering it.*

Publish dialog

When you click **Publish** in the builder, Blank publishes the form (it must have at least one field) and opens the publish-result dialog, titled **Your form is live**.

> **TRY IT**
> Open /blank/, click New Form, drop a single Rating field on the canvas, then click Publish and copy the URL from the Your form is live dialog.

The dialog shows:

- A read-only public URL of the shape `https://YOUR-DOMAIN/forms/<slug>`.
- A **Copy** button that copies the URL (the button text changes to "Copied").
- An **Open in new tab** link.
- The form's current visibility label.
- A **Done** button to close the dialog.

To share a published form:

1. Click **Publish** (or **Share**, if the form is already published) in the builder.
2. In the **Your form is live** dialog, click **Copy** to copy the public URL, or **Open in new tab** to view it.
3. Click **Done**.

> *Note: the URL handed out here is the server-rendered page at `/forms/<slug>`. That is the canonical public form. This is the link to share with respondents.*

Preview

Preview shows the form exactly as a respondent would see it, without accepting input.

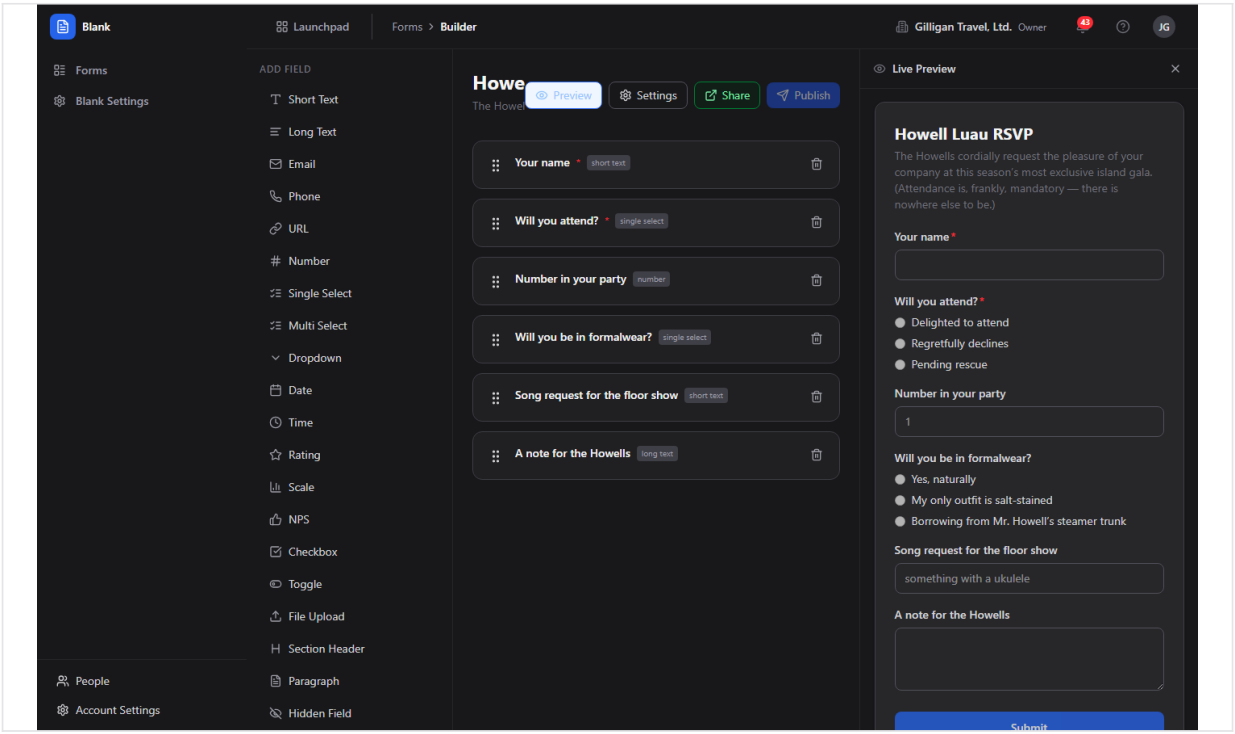


Figure 8.5 *Live preview*

There are two ways to preview:

- In the builder, click **Preview** (eye icon) to toggle the live-preview panel on the right. The panel renders the form as you edit and splits a multi-page form at page boundaries with Previous and Next controls and a progress bar.
- From the Forms list overflow menu, click **Preview**, or go to `/blank/forms/<id>/preview`, for the full-page Form Preview. The header has a "Back to Builder" link and a "PREVIEW MODE" label, and all inputs are disabled.

Responses

The Responses view lists every submission for a form. Open it from a form card's overflow menu (**Responses**) or at `/blank/forms/<id>/responses`. The page is titled "<form name> - Responses" and shows the submission count. The back chevron returns you to the builder.

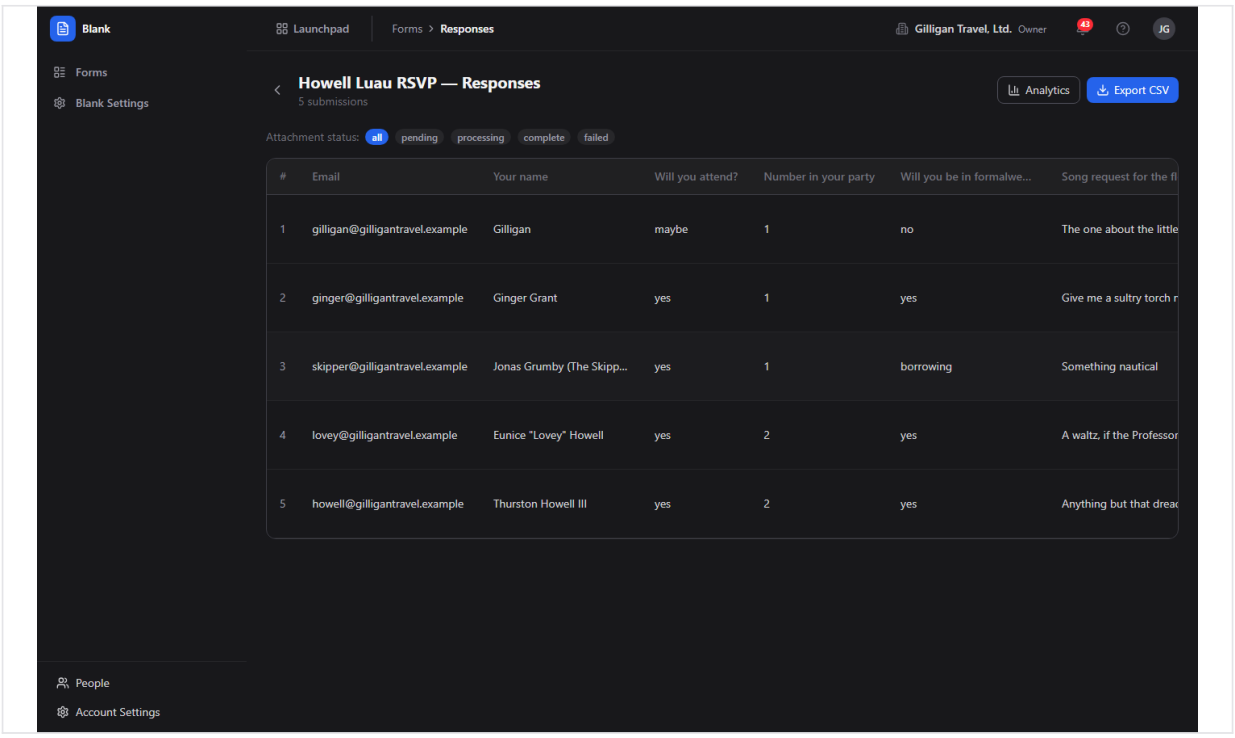


Figure 8.6 Responses table

Actions on the Responses page:

- **Analytics** - opens the Analytics view for this form.
- **Export CSV** - downloads all submissions for the form as a CSV file named `submissions.csv`.

To filter by attachment processing state:

1. Use the **Attachment status:** filter pills above the table: all, pending, processing, complete, or failed. This filters the visible rows.

The table columns are: a row number (#), **Email**, the first five display fields' answers, **Files** (a processing-status pill), and **Date**. If there are no submissions, the table shows "No responses yet."

File uploads are real. When a respondent attaches a file to a File Upload field, the bytes are uploaded to object storage (MinIO/S3) before the form is submitted, and the submission carries a stored-object reference. A background worker then validates each stored file (extension and MIME allowlist, confirming the object actually landed in storage), records the verified file on the submission, and mints a short-lived download link. The **Files** pill reflects that real outcome: **pending** (awaiting the worker), **processing**, **complete** (every file validated and stored), or **failed** (a file was rejected, for example a disallowed type or a missing object). Uploads are constrained to a size cap and an allowlist of image, document, spreadsheet, CSV, plain-text, and zip types; SVG is always blocked.

To export responses:

1. Open the Responses view for the form.
2. Click **Export CSV**. The browser downloads `submissions.csv` with every submission.

Analytics

The Analytics view aggregates the responses to a form. Open it with the **Analytics** button on the Responses page or at `/blank/forms/<id>/analytics`. The page is titled "<form name> - Analytics" and shows the total submission count.

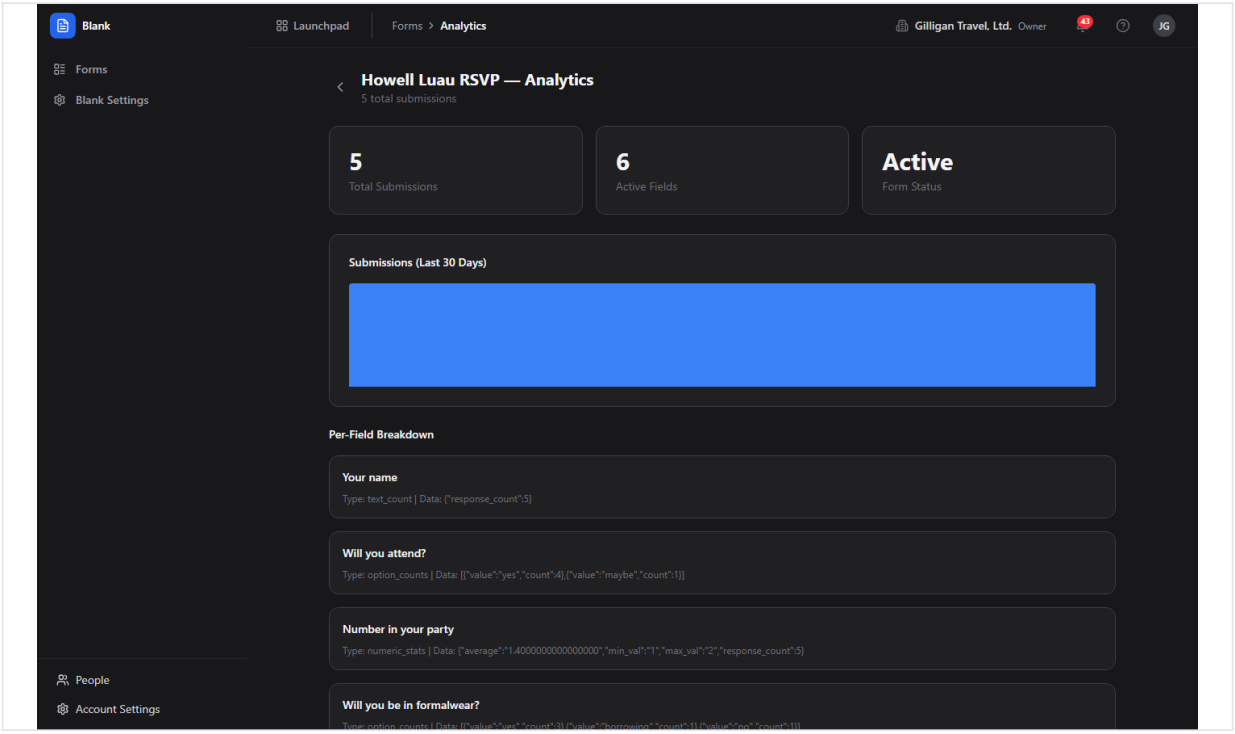


Figure 8.7 Form analytics

It contains:

- Summary cards: **Total Submissions**, **Active Fields**, and **Form Status**.
- A **Submissions (Last 30 Days)** bar chart driven by the daily submission trend.
- A **Per-Field Breakdown** section. For each non-decorative field, this shows aggregated values: option counts for select, multi-select, and dropdown fields; numeric statistics (average, minimum, maximum, and count) for rating, scale, NPS, and number fields; and a text count for other fields.

Per-form Settings page

Each form has a settings page at `/blank/forms/<id>/settings` with additional toggles. Reach it by entering the URL directly. The builder's **Settings** button opens the in-builder Form Settings dialog instead, which covers only visibility, expiration, and project.

The per-form Settings page has these sections:

- **Access** - **Form Type** (Public, Internal, Embedded), **Accept Responses** (whether new submissions are accepted), and **One per Email** (limit to one submission per email address).
- **Confirmation** - **Type** (Show Message, Redirect, or Custom Page) and a **Confirmation Message** textarea.
- **Branding** - a **Theme Color** picker and a **Show Progress Bar** checkbox for multi-page forms.
- **Notifications** - an **Email on Submit** checkbox to be notified when someone submits.

Each control saves immediately when you change it.

Email on submit is real. When a form has **Email on Submit** turned on (with recipients configured) or a respondent supplies their email, a background worker sends two kinds of message: a confirmation email to the respondent (using the form's confirmation message) and a notification email to each configured recipient summarizing the new submission. Delivery goes out through the configured SMTP transport; if SMTP is not configured in your deployment, the worker logs the message it would have sent instead of dropping it silently, so the wiring is the same whether or not mail is delivered. The richer access and notification controls (recipient list, Banter channel, sign-in requirement, allowed domains) live in the in-builder **Form Settings** dialog described above; this page exposes the single **Email on Submit** toggle.

Multi-page forms

A longer form can be split across pages so respondents work through it a section at a time.

There are two ways to page a form:

- **Page Break field.** In the builder, click **Page Break** in the **Add Field** palette wherever you want a page boundary. It appears as a divider in the builder (it collects no answer) and the public form starts a new page there.
- **Page Number.** Select a field in the canvas and set its **Page Number** in Field Settings, assigning each field to the page you want it on.

To show a progress bar across the pages, open the per-form Settings page at `/blank/forms/<id>/settings` and enable **Show Progress Bar** under Branding. Use **Preview** to confirm the pages before you publish.

Blank Settings (app-level)

The **Blank Settings** item in the sidebar opens a read-only information page at `/blank/settings`, headed **Settings**. It displays the app defaults (default form type Public, default theme color #3b82f6), the rate-limiting policy (10 submissions per hour), and the events Blank emits. Nothing on this page is editable.

Public form

A published form is served as a self-contained HTML page at `https://YOUR-DOMAIN/forms/<slug>`. This is the page the Publish dialog gives you and the one respondents fill out. It renders the form's fields, splits a multi-page form across pages with Back and Next controls and a progress bar, optionally collects the respondent's email, and shows the confirmation (message, redirect, or page) after a successful submission.

Public submission is anonymous for forms set to Public visibility. Forms set to Organization or Project members visibility check that the caller belongs to the org or the chosen project before rendering or accepting a submission. The public submit endpoint is rate-limited to 10 submissions per hour per IP, and a form can also enforce a per-email limit, a maximum response count, and an expiration date.

A form can additionally require sign-in and restrict submitters to an allowed set of email domains, both enforced on the public submit and file-upload paths (see "Access control and notifications" under Form Settings dialog). With **Require sign-in to submit** on, an unauthenticated submit or upload is rejected. With **Allowed email domains** set, a submitter whose email is not on an allowed domain is rejected. These gates stack with visibility, the per-email limit, the maximum response count, and the expiration date.

File Upload fields are accepted on the public form: the respondent's file is uploaded to object storage before the form is submitted, the submission references the stored object, and a background worker validates the file (type allowlist, and confirming the object landed) and records it with a download link. The **Files** pill on the Responses table reflects that real outcome.

Working with AI agents

Agents drive Blank through MCP tools that proxy to the same API the SPA uses. The tools share your permissions, so an agent can only do what you could do. There are 20 Blank MCP tools, and together they cover the full authoring lifecycle: generate, create, edit fields, publish, close, duplicate, delete, and read or export responses.

> **TIP**
> Several MCP tools have no matching screen in the SPA, including blank_close_form, blank_get_embed_code, and per-field edits. When you need to close a form or grab its embed snippet, reach for an agent or the API rather than hunting for a button that is not there.

> ***The MCP tools share your permissions, so an agent can drive the full authoring lifecycle but only ever do what you could.***

Generate and create:

- Draft a form from a description with `blank_generate_form`, which returns a suggested form specification (it does not save anything). Its heuristics seed fields when the description mentions name, email, phone, NPS, rating or satisfaction, feedback or comment, and bug or issue.
- Create the draft with `blank_create_form`, which accepts inline field definitions.

Edit structure:

- Add a field with `blank_add_field` (it accepts the conditional-display arguments `conditional_on_field_id`, `conditional_operator`, and `conditional_value`).
- Change a field with `blank_update_field`, remove one with `blank_delete_field`, and reorder them with `blank_reorder_fields`.

Inspect, publish, and manage lifecycle:

- List and read forms with `blank_list_forms` and `blank_get_form`.
- Update form metadata with `blank_update_form`.
- Publish a draft with `blank_publish_form` and stop new responses with `blank_close_form`.
- Clone a form with `blank_duplicate_form`, get its iframe snippet with `blank_get_embed_code`, and remove a form with `blank_delete_form` (destructive).

Read and report on responses:

- List and read submissions with `blank_list_submissions` and `blank_get_submission`.
- Summarize and report on results with `blank_summarize_responses` and `blank_get_form_analytics` (both return the same analytics payload the Analytics view uses).
- Export raw responses with `blank_export_submissions` (returns CSV text), and remove a single response with `blank_delete_submission` (destructive).

Some tools have no matching screen in the SPA: `blank_generate_form` (the builder has no AI-draft entry point), `blank_get_submission` (the Responses table has no per-submission detail view), `blank_add_field` / `blank_update_field` / `blank_delete_field` / `blank_reorder_fields` (an agent can edit fields directly, whereas a person does the same work in the builder canvas), `blank_close_form`, `blank_get_embed_code`, and `blank_delete_submission`. When reviewing agent work, check the form's fields and visibility in the builder before publishing, and confirm the public URL from the **Your form is live** dialog.

Blank also participates in the suite-wide agent platform. Agent activity is attributed to an agent identity and surfaced in the unified activity feed; long-running agents send heartbeats. Destructive or sensitive steps (publishing, closing, or deleting) can be routed through the approval-queue tools so a human decides before the action lands. Per-agent policies (kill switch plus a `blank.*` tool allowlist) gate which Blank tools a service account may call, and subscribed Blank events can be pushed to agent runners through signed outbound webhooks. Before an agent quotes a submission's contents into a shared surface it should call the visibility preflight (`can_access`) for that entity and drop anything the asker is not allowed to see.

For the full tool catalog and argument shapes, see the MCP-tools reference in `docs/apps/blank/mcp-tools.md`.

Working together (live presence)

Like every BigBlueBam app, Blank carries the persistent Bureau presence dock, so collaboration is ambient rather than scheduled. From anywhere in Blank you can see who is around and ring, knock, or invite a teammate into a live voice or video huddle that follows you in a floating window as you both move through the suite. Voice and video here are the digital equivalent of bumping into someone in the hallway or stopping by their desk, not a booked meeting. The deeper per-record collaboration (a presence strip showing who is on a specific item, and real-time co-editing of the same record) lives on the document, board, diagram, and task surfaces, described in the Introduction; in Blank, the shared layer is the always-on Bureau dock.

User Stories

Story: Build and publish your first form

Who: A team member collecting feedback or intake. **Goal:** Create a form, add a few fields, and get a shareable public URL. **Before you start:** You are logged in to BigBlueBam and have the `blank.form.publish` permission.

Steps

1. Go to `/blank/`. On the **Forms** list, click **New Form**.
2. The builder opens with an "Untitled Form" draft. Click the form Title input in the center canvas and type a name. Add a Description below it.
3. In the **Add Field** palette on the left, click the field types you want, for example **Rating**, **NPS**, and **Long Text**. Each appears in the canvas.
4. Click a field in the canvas to open the **Field Settings** panel on the right. Set its **Label**, mark it **Required** if needed, and adjust type-specific options.
5. Reorder fields by dragging their grip handles. Remove a field with its trash icon.
6. Click **Settings** in the top-right action bar. Choose a **Visibility** option, set an **Expires at** date if you want one, and click **Save**.
7. Click **Publish**. In the **Your form is live** dialog, click **Copy** to copy the public URL.

Result: The form's status pill reads "published" on the Forms list, and you have a `https://YOUR-DOMAIN/forms/<slug>` URL to share.

Related: "Collect and review responses." An agent can do the same with `blank_create_form` then `blank_publish_form`.

Story: Collect and review responses

Who: The form author after sharing the link. **Goal:** Let people submit the form and review what came in. **Before you start:** The form is published and set to a visibility that lets your respondents reach it.

Steps

1. Share the public URL (from the **Your form is live** dialog) with respondents. They open `/forms/<slug>`, fill out the fields, and submit. After submitting they see the confirmation you configured.
2. Back in Blank, go to the **Forms** list and open the form's overflow menu (three-dot icon).
3. Click **Responses**.
4. Review the table. Each row shows the row number, **Email**, the first five fields' answers, the **Files** status, and the **Date**.
5. To narrow the list by attachment processing state, click a pill under **Attachment status:** (all, pending, processing, complete, or failed).

6. Click **Export CSV** to download every submission as `submissions.csv`.

Result: You have reviewed the responses in the table and, if you exported, a CSV file of all submissions.

Related: "Analyze results." An agent can pull the same data with `blank_list_submissions` and `blank_export_submissions`.

Story: Analyze results

Who: The form author wanting aggregate numbers. **Goal:** See totals, the recent trend, and per-field breakdowns. **Before you start:** The form has at least one submission.

Steps

1. Open the form's **Responses** view.
2. Click **Analytics**.
3. Read the summary cards: **Total Submissions**, **Active Fields**, and **Form Status**.
4. Review the **Submissions (Last 30 Days)** bar chart for the recent trend.
5. Scroll to **Per-Field Breakdown** to see option counts for choice fields and numeric statistics (average, minimum, maximum, count) for rating, scale, NPS, and number fields.

Result: You understand how many responses arrived, when, and how each field was answered.

Related: An agent can request the same payload with `blank_summarize_responses` or `blank_get_form_analytics` and write a summary.

Story: Duplicate a form as a template

Who: Someone who reuses a form structure for a new round. **Goal:** Start a new form from an existing one without rebuilding it. **Before you start:** You have an existing form to copy.

Steps

1. On the **Forms** list, open the source form's overflow menu (three-dot icon).
2. Click **Duplicate**.
3. A new draft appears in the list with "(Copy)" in its name and the same fields.
4. Open the copy, adjust its title, fields, and settings, then click **Publish** when ready.

Result: A new draft form with the same fields, ready to edit and publish independently of the original.

Related: An agent can clone a form with `blank_duplicate_form`.

Story: Restrict a form to your organization or a project

Who: An author who wants the form limited to internal people. **Goal:** Stop people outside your org or project from viewing or submitting the form. **Before you start:** You are editing the form in the builder.

Steps

1. Click **Settings** in the builder action bar.
2. Under Visibility, choose **Organization** to limit to your org, or **Project members** to limit to a Bam project.
3. If you chose **Project members**, select a project in the **Project** picker that appears.
4. Click **Save**.

Result: People who are not in the org (or not members of the chosen project) get a forbidden response when they open the public URL. Members can view and submit as normal.

Related: "Gate a public form by sign-in and email domain" covers the complementary `requires_login` and `allowed_domains` gates, which are enforced on the public submit and file-upload paths and stack with visibility.

Story: Gate a public form by sign-in and email domain, and get notified

Who: An author who wants only known people to submit and wants to hear about each response. **Goal:** Require sign-in and an allowed email domain to submit, and email the right people when a submission arrives. **Before you start:** You are editing the form in the builder.

Steps

1. Click **Settings** in the builder action bar.
2. Check **Require sign-in to submit** to force authentication (optional but recommended for internal forms).
3. Under **Allowed email domains**, enter the domains you accept, comma separated, for example `acme.com, example.org`. Leave it blank to allow any domain.
4. Check **Email me on new submissions** and, under **Notification recipients**, enter the addresses to notify, comma separated.
5. (Optional) Pick a **Post to Banter channel** target to also post each submission to a channel.
6. Click **Save**, then **Publish**.

Result: An unauthenticated visitor is turned away with a sign-in error, a submitter on a disallowed domain is rejected, and every accepted submission triggers a confirmation email to the submitter plus a notification email to your recipients (and a Banter post if configured). Where SMTP is not configured, the worker logs the messages instead of delivering them.

Related: "Restrict a form to your organization or a project" covers the visibility gate that stacks with these. An agent can set the same fields with `blank_update_form` (`requires_login`, `allowed_domains`, `notify_on_submit`, `notify_emails`).

Story: Collect file uploads on a form

Who: An author who needs respondents to attach a document or image. **Goal:** Add a file-upload field and review the stored files that come in. **Before you start:** You are editing the form in the builder.

Steps

1. In the **Add Field** palette, click **File Upload**. The field appears in the canvas; set its **Label** and mark it **Required** if needed.
2. Publish the form and share the public URL.
3. A respondent attaches a file; it is uploaded to object storage before they submit, and the submission references the stored object.
4. Open the form's **Responses** view. Watch the **Files** pill move from **pending** to **complete** as the background worker validates and stores each file (or to **failed** if a file is rejected).
5. Use the **Attachment status:** filter pills to find submissions whose files are still pending or that failed validation.

Result: Respondents can attach files, the files are stored for real, and the Responses table tells you which submissions have validated attachments.

Related: Uploads are limited to a size cap and an allowlist of image, document, spreadsheet, CSV, plain-text, and zip types (SVG is always blocked).

Story: Build a multi-page survey

Who: An author with a longer survey to split across pages. **Goal:** Group fields onto separate pages so respondents page through them. **Before you start:** The form has several fields.

Steps

1. In the builder, click **Page Break** in the **Add Field** palette wherever you want a page boundary. It appears as a divider in the canvas.
2. Add or arrange the fields for each page around the page breaks. (Alternatively, select a field, open **Field Settings**, and set its **Page Number**.)
3. (Optional) Open the per-form Settings page at `/blank/forms/<id>/settings` and enable **Show Progress Bar** under Branding.
4. Click **Preview** to confirm the pages, then **Publish**.

Result: The public form shows fields grouped by page with Back and Next controls and, if enabled, a progress bar.

Related: Page breaks and per-field Page Number both page a form; use whichever fits how you build.

Story: Close a form to stop accepting responses

Who: An author or agent wrapping up a collection. **Goal:** Stop new submissions on a published form. **Before you start:** You have the `blank.form.close` permission. There is no Close button in the SPA; closing is done through the API or an agent.

Steps

1. Call `POST /blank/api/v1/forms/<id>/close` with your session cookie or API key, or have an agent call `blank_close_form`.

2. Blank sets the form's status to closed, turns off **Accept Responses**, and emits a `form.closed` event (source `blank`).

Result: The form's public page no longer accepts submissions, and its status pill reads "closed" on the Forms list. There is no in-app control to reopen or un-publish a form.

Related: To pause without closing, an author can turn off **Accept Responses** on the per-form Settings page instead.

Story: Have an agent draft, create, and publish a form

Who: A user delegating form creation to an AI agent. **Goal:** Go from a plain-language description to a published, shareable form. **Before you start:** The agent runs with your credentials and your publish permission. Its `blank.*` policy allowlist must permit the tools below.

Steps

1. Ask the agent to draft the form. It calls `blank_generate_form` with your description and returns a suggested specification of fields. Nothing is saved yet.
2. Review the suggested fields with the agent and adjust.
3. The agent calls `blank_create_form` with the field definitions inline to create the draft. It can refine the structure further with `blank_add_field`, `blank_update_field`, `blank_delete_field`, and `blank_reorder_fields`.
4. The agent calls `blank_publish_form` to publish it. If publishing is routed through the approval queue, you approve the proposal first.
5. Open the form in the builder to confirm the fields and visibility, then copy the public URL from the **Your form is live** dialog (open it with the **Share** button).
6. Later, ask the agent to report on results with `blank_summarize_responses`.

Result: A published form created from a description, with a public URL you can share and an agent that can summarize responses on demand.

Related: "Build and publish your first form" covers the same flow by hand.

Related

- **Bond** - submissions can be routed to create a Bond contact when routing is configured for the form at the API level.
- **Helpdesk** - submissions can be routed to create a Helpdesk ticket when routing is configured for the form at the API level.
- **Bam** - forms can be scoped to a Bam project through Project members visibility, and the Form Settings dialog loads your Bam projects to choose from.
- **Bolt** - Blank emits `form.published`, `form.closed`, and `submission.created` events (source `blank`) that Bolt automations can react to.
- MCP tools reference: `docs/apps/blank/mcp-tools.md`.
- App guide: `docs/apps/blank/guide.md`.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What status does a brand-new form start in?

a) published
b) draft
c) closed
d) active

2. How many submissions per hour per IP does the public submit endpoint allow?

a) 5
b) 10
c) 50
d) There is no limit

3. What are the three visibility options that control who can reach a form through its URL?

4. Which file type does Blank always block on uploads, regardless of the size cap?

a) PDF
b) CSV
c) SVG
d) ZIP

5. Before Blank will publish a form, what minimum does the form need?

6. What does deleting a form remove besides the form itself?

a) Nothing else
b) Its fields and submissions
c) Only its submissions
d) Only its fields

7. Which four field types are layout-only and carry no submitted answer?

a) Short Text, Email, Number, Date
b) Section Header, Paragraph, Hidden Field, Page Break
c) Rating, Scale, NPS, Checkbox
d) Single Select, Multi Select, Dropdown, Toggle

8. What is the shape of the public form URL that the Publish dialog hands you?

9. Where in the SPA can an author close a published form?

a) The builder action bar
b) The Forms list overflow menu
c) The per-form Settings page
d) Nowhere; there is no Close button in the SPA, so closing is done through the API or an agent

10. When 'Allowed email domains' is set, in what order is the submitter's email determined?

11. What does the 'complete' Files pill on the Responses table mean?

a) The submission was deleted
b) Every file was validated and stored

c) The form is closed
d) The worker has not run yet

12. What two requirements gate getting to the Blank authoring SPA in the first place?

CHAPTER 9

Blast

Write it once, send it everywhere, watch what lands.

Blast turns your Bond CRM contacts into an audience for bulk email. You compose a message once, in a visual block editor, raw HTML, or from a saved template, pick who should receive it, and send. After every send, Blast records what happened to each recipient and rolls the results into per-campaign and org-wide analytics. This chapter walks you from a first draft through Send Now to reading opens, clicks, bounces, and unsubscribes, and it shows where an agent can assemble a campaign for your approval.

At a glance

- Three ways to build an email body: visual block editor, raw HTML, or a saved template
- Segments that filter your Bond contacts and cache a contact count
- Per-recipient delivery tracking plus per-campaign and org-wide analytics
- Automatic open, click, bounce, complaint, and unsubscribe handling with no UI to wire
- A CAN-SPAM gate that blocks any send missing an unsubscribe link or a physical address

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Campaigns list . Create a campaign . Campaign detail and Send Now . Visual builder . Templates . Template gallery

> *Blast is the email campaign tool in the BigBlueBam suite. Marketers and operators use it to design HTML emails, target groups of Bond CRM contacts, send to those contacts, and track opens, clicks, bounces, and unsubscribes.*

Overview

Blast turns your Bond CRM contacts into an audience for bulk email. You write a message once (in a visual block editor, raw HTML, or from a saved template[77]), pick who should receive it, and send. After a send, Blast records what happened to each recipient and rolls the results up into per-campaign[78] and org-wide analytics.

The core objects you work with are **campaigns** (a single bulk send), **templates** (reusable email designs), **segments** (saved filters over Bond contacts), and **sender domains** (the domains you verify so mailbox providers trust your mail). Each campaign tracks counters for sent, delivered, opened, clicked, bounced, unsubscribed, and complained, and exposes a per-recipient delivery table.

Blast fits into the suite in two directions. It pulls its audience from **Bond** (contacts are the recipient pool, and segments filter on Bond contact fields), and it pushes activity into **Bolt** (every send and engagement event[79] is published so automation rules can react). It shares its login and SMTP relay with the rest of the platform: there is no separate Blast account, and there is no Blast-specific email server.

> ***Blast pulls its audience from Bond and pushes every send and engagement event into Bolt, so your automation can react the moment an email lands.***

Two things are worth knowing before you rely on Blast for production sending. First, outbound email goes through one platform-wide SMTP relay that a SuperUser configures in the Bam app, not in Blast. Second, the in-app campaign detail page only exposes **Send Now**; scheduling, pausing, cancelling, editing, and deleting a campaign are available through the API and MCP tools but are not wired into the campaign detail UI today.

Key concepts

- **Campaign** - A single bulk email send. Has a name, subject, body (HTML), optional plain-text body, an optional template, an optional segment[80], From name/email, reply-to, and rollup counters.

[77] **Template.** A reusable email design: name, subject template, HTML body, an optional visual block layout, and a description. Templates carry a version number that bumps automatically each time you save an edit.

[78] **Campaign.** A single bulk email send. Has a name, subject, body (HTML), optional plain-text body, an optional template, an optional segment, From name/email, reply-to, and rollup counters.

[79] **Engagement event.** An append-only record of an open, click, unsubscribe, bounce, or complaint. These feed the analytics counters and are published to Bolt.

[80] **Segment.** A saved filter over Bond contacts, built from one or more conditions joined by **ALL conditions (AND)** or **ANY condition (OR)**. Each segment caches a contact count so you can see roughly how many people match.

- **Campaign status**[81] - One of six states: `draft`, `scheduled`, `sending`, `paused`, `sent`, `cancelled`. A draft is editable, deletable, and sendable. A scheduled campaign can be sent or cancelled. A sending campaign can be paused or cancelled. `sent` is the terminal success state and the only one analytics counts. There is no "archived" state.
- **Template** - A reusable email design: name, subject template, HTML body, an optional visual block layout, and a description. Templates carry a version number that bumps automatically each time you save an edit.
- **Segment** - A saved filter over Bond contacts, built from one or more conditions joined by **ALL conditions (AND)** or **ANY condition (OR)**. Each segment caches a contact count so you can see roughly how many people match.
- **Recipient / send log**[82] - One row per person per campaign. It tracks the status of that one email (queued, sent, delivered, bounced, complained, or failed) and drives the **Recipients** table on the campaign detail page.
- **Engagement event** - An append-only record of an open, click, unsubscribe[83], bounce, or complaint. These feed the analytics counters and are published to Bolt.
- **Unsubscribe** - An org-wide suppression keyed on email. Once an address unsubscribes (or files a spam complaint), it is excluded from future sends across the whole org.
- **Sender domain**[84] - A domain you add and verify (SPF, DKIM, DMARC) so mailbox providers accept your mail.
- **Merge fields**[85] - Placeholders the sender fills in per recipient: `{{first_name}}`, `{{last_name}}`, `{{email}}`, `{{company}}`, and `{{unsubscribe_url}}`.
- **CAN-SPAM gate**[86] - A compliance check that runs when you send or schedule. The HTML body must contain both an unsubscribe mechanism and a physical mailing address, or the send is rejected.

> **WARNING**
> Unsubscribes are org-wide and keyed on the email address. Once an address unsubscribes or files a spam complaint, it is suppressed across the whole org and excluded from every future send. There is no per-campaign opt back in.

[81] **Campaign status.** One of six states: `draft`, `scheduled`, `sending`, `paused`, `sent`, `cancelled`. A draft is editable, deletable, and sendable. A scheduled campaign can be sent or cancelled. A sending campaign can be paused or cancelled. `sent` is the terminal success state and the only one analytics counts. There is no "archived" state.
[82] **Recipient / send log.** One row per person per campaign. It tracks the status of that one email (queued, sent, delivered, bounced, complained, or failed) and drives the **Recipients** table on the campaign detail page.
[83] **Unsubscribe.** An org-wide suppression keyed on email. Once an address unsubscribes (or files a spam complaint), it is excluded from future sends across the whole org.
[84] **Sender domain.** A domain you add and verify (SPF, DKIM, DMARC) so mailbox providers accept your mail.
[85] **Merge fields.** Placeholders the sender fills in per recipient: `{{first_name}}`, `{{last_name}}`, `{{email}}`, `{{company}}`, and `{{unsubscribe_url}}`.
[86] **CAN-SPAM gate.** A compliance check that runs when you send or schedule. The HTML body must contain both an unsubscribe mechanism and a physical mailing address, or the send is rejected.

Where to find it

Blast is served at `/blast/`. It is a separate SPA but shares its login with the Bam app: there is no separate Blast sign-in. If you are not signed in to BigBlueBam, Blast shows a "Please log in to BigBlueBam first" screen that links to `/b3/`. Sign in there once and return to `/blast/`.

The left sidebar has two groups. The primary group is **Campaigns**, **Templates**, **Segments**, and **Analytics**. Below it, a **Blast Settings** group holds **Domains** and **SMTP**. The header carries the Launchpad trigger, breadcrumbs, the org switcher, the notifications bell, and the user menu. Press `?` anywhere to open in-app help.

Prerequisites:

- A BigBlueBam account and membership in an org (shared with Bam).
- Bond contacts in your org, since segments and sends draw from the contact pool.
- A configured platform SMTP relay for any real delivery (see the SMTP feature below).
- For sender-domain changes you need `admin` scope; most other write actions need `read_write` scope.

Feature reference

Campaigns list

The campaigns list is the home view of Blast, at `/blast/`. It shows every campaign in a table and is where you start a new one.

To browse and filter campaigns:

1. Open Blast at `/blast/`. The header reads **Campaigns** with the subtitle "Create and manage email campaigns".
2. Use the **Search campaigns…** box to filter the table by campaign name. This filter is applied in your browser to the names already loaded.
3. Click a status button to filter by state. The buttons are **All**, **draft**, **scheduled**, **sending**, **sent**, **paused**, and **cancelled**. Click the active status again to clear it.
4. Read each row: **Campaign** (name plus subject), **Status**, **Sent**, **Open Rate**, **Click Rate**, and **Date**. Open Rate and Click Rate show a dash until the campaign has sent.
5. Click any row to open that campaign's detail page.

To start a new campaign, click **New Campaign** at the top right. This opens the create form.

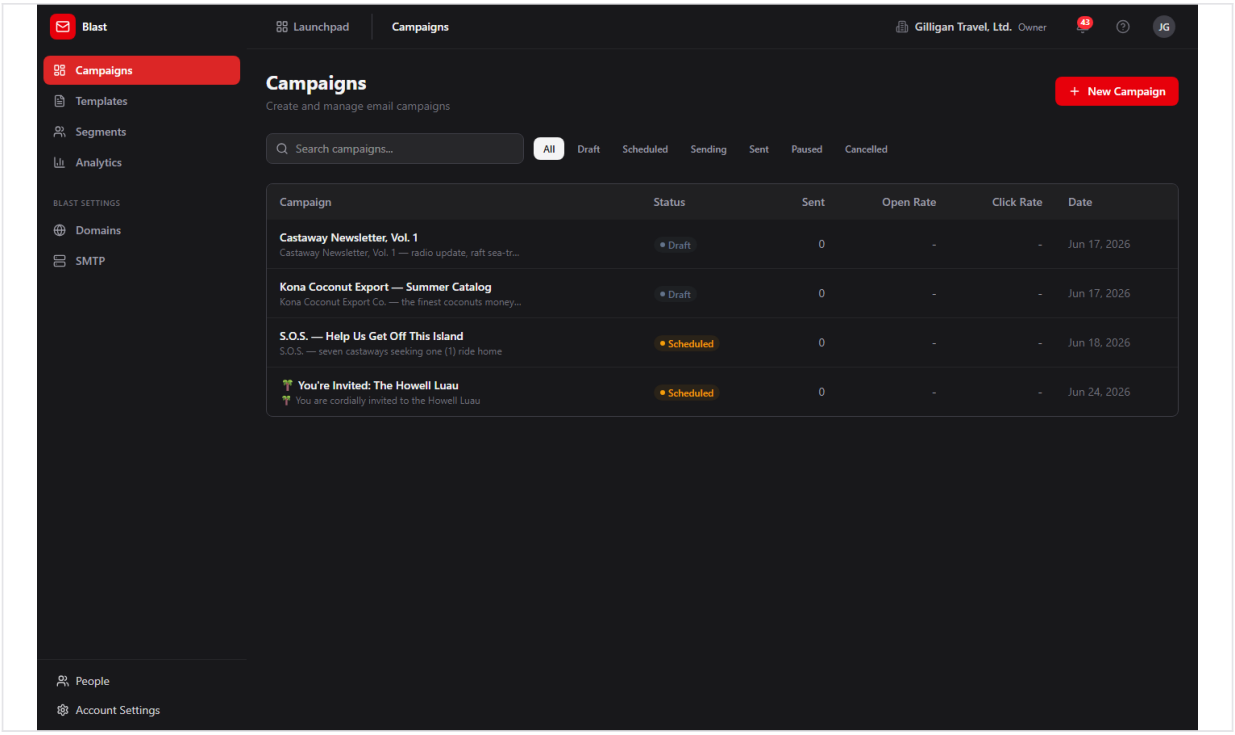


Figure 9.1 Campaign list

Create a campaign

The create form is at `/blast/campaigns/new`. It collects the campaign settings and lets you build the email body in one of three ways. Creating a campaign always produces a `draft`; nothing is sent from this page.

To create a campaign:

1. From the campaigns list, click **New Campaign**. The header reads **New Campaign**.
2. Fill in **Campaign Name *** (for example, "April Product Launch") and **Subject Line ***. Both are required; the **Create Campaign** button stays disabled until they have values.
3. Under **From**, optionally enter a sender **Name** and **Email**.
4. Choose a **Segment**. The default is **All contacts**; the dropdown also lists your saved segments with their cached counts in parentheses.
5. Choose a content mode with the **Content:** toggle: **Visual Builder**, **HTML**, or **From Template**.
6. Make sure the body includes an unsubscribe link and a physical mailing address (the new-template footer already includes an `{{unsubscribe_url}}` link). Without both, the campaign cannot be sent or scheduled later.
7. Click **Create Campaign**. The button reads "Creating..." while it saves, then returns you to the campaigns list with the new draft.

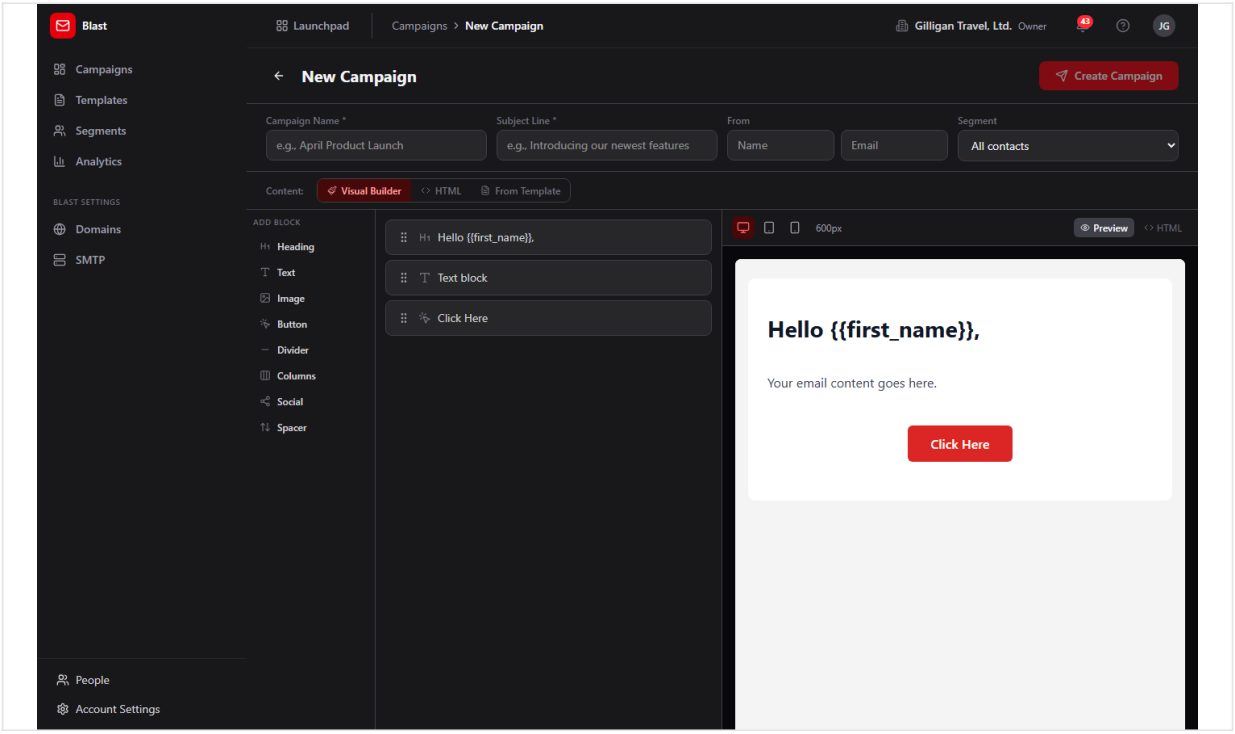


Figure 9.2 New campaign form

> *Segment caveat: choosing a segment here records which segment the campaign is associated with, but the sender does not yet filter recipients by segment. See "Send a campaign (Send Now)" for what actually happens at send time.*

Campaign detail and Send Now

The detail page is at `/blast/campaigns/:id`. It is where you send a draft and where you review results after a send.

To open and read a campaign:

1. From the campaigns list, click the campaign row. The page shows the back arrow, the campaign name and subject, and a status badge.
2. Read the primary metrics: **Total Sent**, **Delivered**, **Opened** (with open rate), and **Clicked** (with click rate).
3. Read the secondary metrics: **Bounced**, **Unsubscribed**, and **Complaints**.
4. If the campaign has click data, read the **Top Clicked Links** table.
5. Under **Campaign Details**, read **From**, **Sent Date**, **Scheduled**, **Recipients**, and **Created**.
6. Scroll to the **Recipients** table for per-recipient delivery. It pages 25 rows at a time with **Prev** and **Next**. Each row shows **Email**, a **Status** icon (Delivered, Sent, Bounced, Failed, or Pending), and the **Sent**, **Delivered**, and **Bounced** times.

To send a campaign immediately:

1. Open a campaign whose status is `draft` or `scheduled`. The **Send Now** button appears at the top right only for these two states.
2. Click **Send Now**. The button reads "Sending..." while it submits.
3. The CAN-SPAM check runs first. If the HTML body is missing an unsubscribe mechanism or a physical address, the send is rejected with a validation error (see "Why a send was rejected" below).
4. If the check passes, the status flips to `sending`, the send job is queued in the background worker, and the campaign begins delivering.

> *No screenshot exists for the campaign detail page or the **Send Now** button yet.*
>
> *What Send Now does not expose: scheduling, pausing, cancelling, editing, and deleting a campaign all exist in the API and MCP tools, but the detail page only renders **Send Now**. To schedule, pause, cancel, edit, or delete a campaign today you must use the API or an MCP tool (`blast_update_campaign`, `blast_pause_campaign`, `blast_cancel_campaign`, and the schedule path inside `blast_send_campaign`).*

NOTE

The campaign detail page only renders Send Now today. Scheduling, pausing, cancelling, editing, and deleting a campaign all exist in the API and MCP tools, but you reach them through the API or a tool, not the detail UI.

Visual builder

The visual builder is the drag-and-drop email designer shared by the campaign create form and the template editor. It produces email-safe inline HTML so your message renders consistently across mail clients.

To build an email body visually:

1. In the campaign create form choose the **Visual Builder** content mode, or open the template editor and select the **Visual** mode.
2. From the left **palette**, click a block to add it. The blocks are **Heading**, **Text**, **Image**, **Button**, **Divider**, **Columns**, **Social**, and **Spacer**.
3. In the center list, reorder blocks by dragging the handle, and use each block's **Duplicate** and **Remove** controls.
4. Select a block to edit its properties in the right-hand editor.
5. Use the preview at the bottom to check the layout. Switch widths with the desktop, tablet, and mobile toggles (600, 480, and 320 pixels), and toggle between the rendered view and the source with the Code/Eye control.
6. Insert merge fields in text where you want them personalized, for example `Hello {{first_name}},`.

The block layout is saved alongside the email so you can reopen and keep editing it later.

Templates

Template gallery

The gallery lists your reusable templates at `/blast/templates`.

To manage templates:

1. Open **Templates** from the sidebar. The header reads **Templates** with the subtitle "Reusable email templates for your campaigns".
2. Use the **Search templates…** box to find a template by name.
3. Each card shows the template name, the subject template, a version badge such as `v2`, the description, and the relative time it was last updated.
4. Hover a card to reveal **Duplicate** (copy) and **Delete** (trash). **Delete** asks "Delete this template?" before removing it.
5. Click a card to open it in the editor.

To start a new template, click **New Template** at the top right.

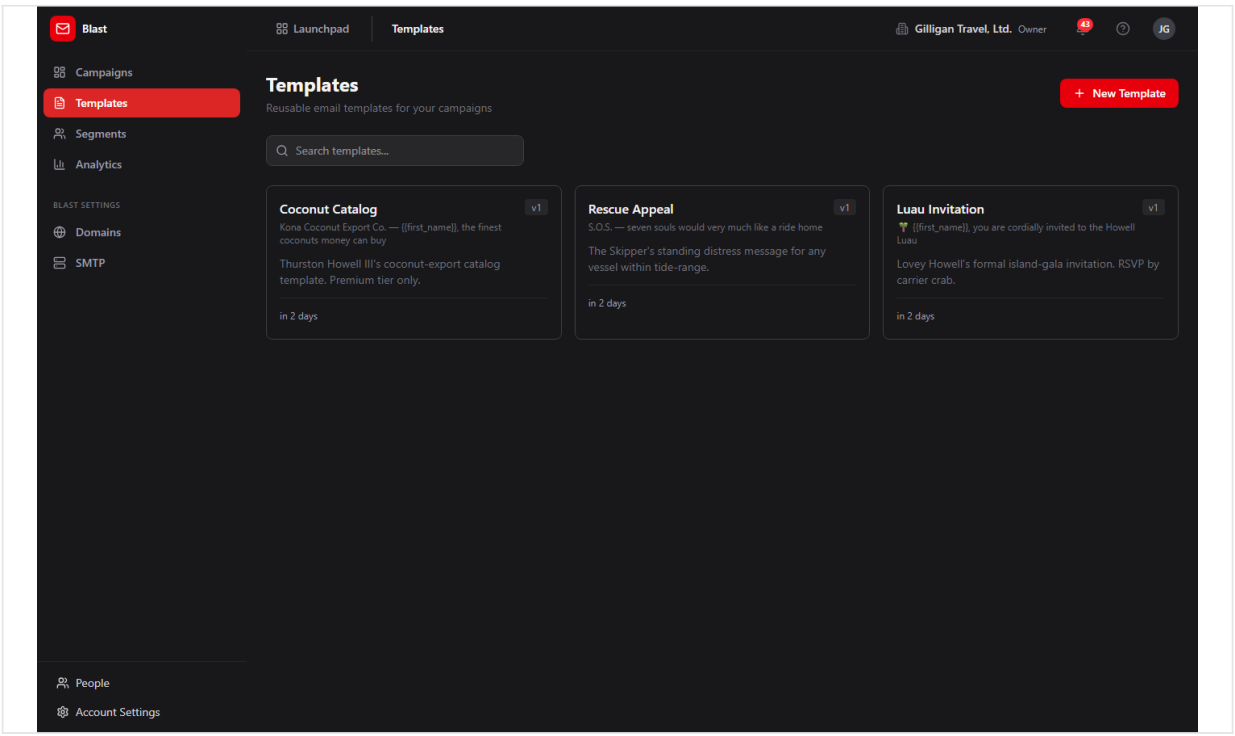


Figure 9.3 Template gallery

Template editor

The editor is at `/blast/templates/new` for a new template and `/blast/templates/:id/edit` for an existing one. Saving an edit bumps the template's version automatically.

To author a template:

1. Click **New Template** from the gallery, or click an existing template to edit it.
2. Fill in **Template Name *** and **Subject Line *** (the subject shows a live merge preview). Both are required; **Save Template** stays disabled until they have values. Optionally add a **Description**.
3. Build the body using the **Visual** / **HTML** toggle. **Visual** opens the block builder; **HTML** gives you a raw editor with a live iframe preview.
4. Click **Save Template**. The button reads "Saving..." while it saves.

New templates start with a default block layout that includes a footer with an `{{unsubscribe_url}}` link, which helps satisfy the CAN-SPAM check when a campaign uses the template.

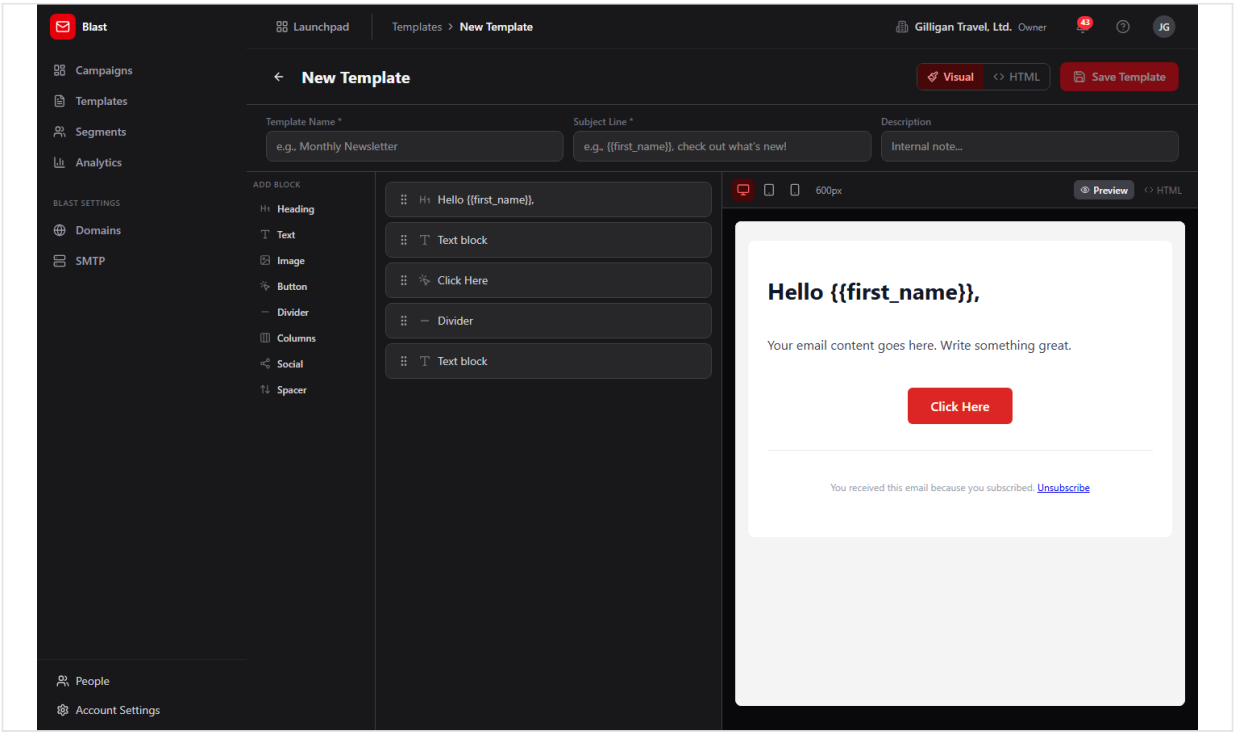


Figure 9.4 Template editor

Segments

Segment list

The segment list is at `/blast/segments`. Segments are saved filters over your Bond contacts.

To manage segments:

1. Open **Segments** from the sidebar. The header reads **Segments** with the subtitle "Target specific groups of contacts for your campaigns".
2. Use the **Search segments…** box to find a segment by name.
3. Read each row: **Segment** (name plus description), **Contacts** (the cached count), **Conditions** (for example, "2 condition(s), match all"), and **Last Updated**.
4. Under **Actions**, click **Recalculate count** (refresh icon) to recompute how many contacts currently match. Click **Delete** (trash icon) to remove the segment; it asks "Delete this segment?" first.

To start a new segment, click **New Segment** at the top right.

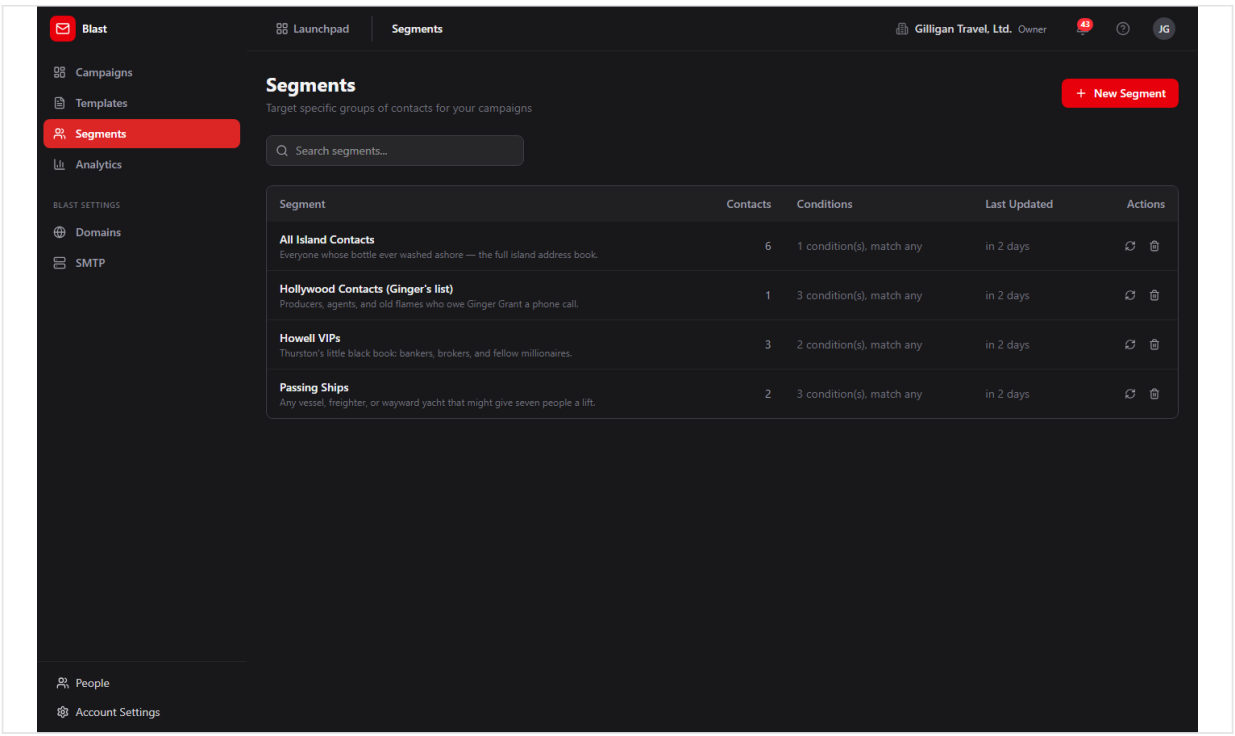


Figure 9.5 Segment list

Segment builder and preview

The builder is at `/blast/segments/new`. It is a create-only form; there is no edit page in the UI.

To build a segment:

1. Click **New Segment** from the segment list. The header reads **New Segment**.
2. Enter a **Segment Name** and an optional **Description**.
3. Choose a **Match** mode: **ALL conditions (AND)** to require every condition, or **ANY condition (OR)** to match any one.
4. Add one or more conditions. Each condition has a field, an operator, and a value:
5. Click **Add Condition** to add another row, or the trash icon to remove a row.
6. Click **Create Segment** (it reads "Saving..." while it saves), or **Cancel** to discard.

After creating, return to the segment list and click **Recalculate count** to see how many Bond contacts match.

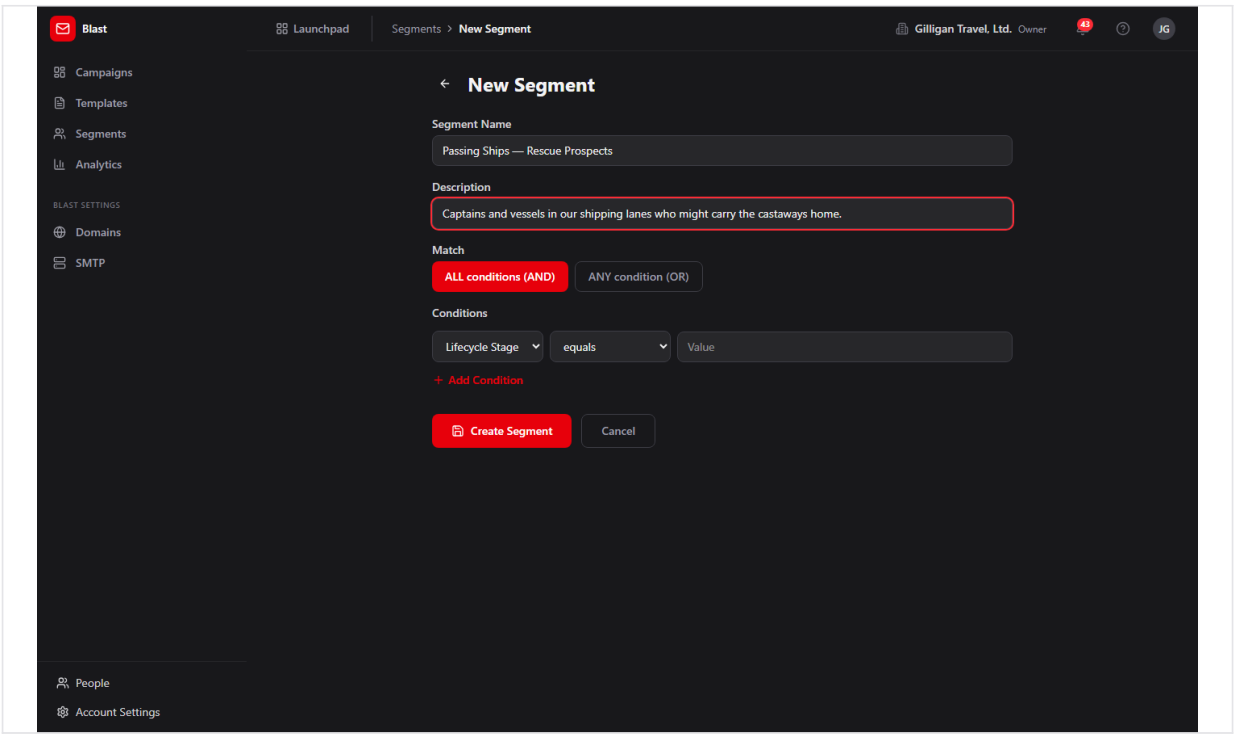


Figure 9.6 Segment builder

> *The builder UI exposes six fields and seven operators. The backend supports more (it can also filter on `email`, `first_name`, `last_name`, and use the `is_set` and `is_not_set` operators), but those are reachable only through the API or the `blast_create_segment` / `blast_update_segment` MCP tools, not through the builder UI.*
>
> *Segments filter only on Bond contact columns. There is no targeting by tags, activity history, or custom fields.*

Analytics

The analytics dashboard is at `/blast/analytics`, titled **Email Analytics** with the subtitle "Overview of your email campaign performance". It rolls up performance across all sent campaigns in your org. Only campaigns in the `sent` state are counted.

To review org-wide email performance:

1. Open **Analytics** from the sidebar.
2. Read the stat cards: **Total Sent**, **Delivered**, **Avg Open Rate** (with total opens), **Avg Click Rate** (with total clicks), and **Bounce Rate** (with total bounced).
3. Read the tiles **Campaigns Sent** and **Unsubscribes**.
4. Read the **Weekly Engagement Trend** table for **Period**, **Campaigns**, **Sent**, **Open Rate**, and **Click Rate** by week.

For a single campaign's analytics, open its detail page instead (see "Campaign detail and Send Now").

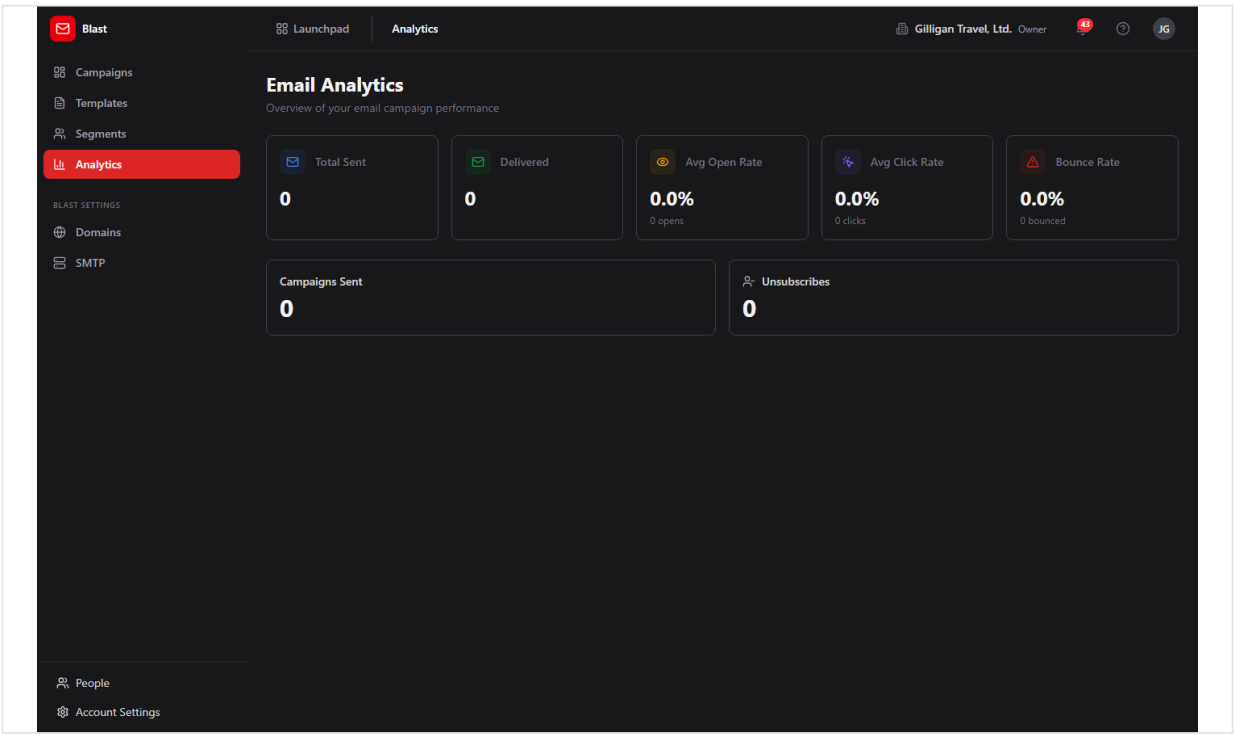


Figure 9.7 Analytics dashboard

Sender Domains

The Sender Domains page is at `/blast/settings/domains`, under **Blast Settings**. Verifying a domain improves deliverability by proving to mailbox providers that you are authorized to send from it. Adding and removing domains needs `admin` scope.

To add and verify a sending domain:

1. Open **Domains** under **Blast Settings**. The header reads **Sender Domains** with the subtitle "Verify your sending domains for better deliverability".
2. In the add row, type your domain (for example, "company.com") and click **Add Domain**.
3. On the new domain's card, read the **Required DNS Records** list (each row shows a record type, name, and value) and add those records at your DNS host.
4. Click **Verify DNS** (refresh icon) to run live lookups. The **SPF**, **DKIM**, and **DMARC** indicators turn to a green check when each record is found.
5. To remove a domain, click **Remove** (trash icon) and confirm "Remove this domain?".

> *No screenshot exists for the Sender Domains page yet.*

SMTP (platform relay)

The SMTP page is at `/blast/settings/smtp`, titled **SMTP Settings**. It is an information card, not a form. Blast keeps no SMTP credentials of its own. Every Blast campaign is sent through a single platform-wide relay stored in the platform settings and configured once in the Bam app under **Account Settings -> Integrations**. The same relay sends transactional email and Blast campaigns.

What you see depends on your role:

- **Platform SuperUser**: a blue info box and an **Open Account Settings** button that opens `/b3/settings`, where you open the **Integrations** tab to edit the relay.
- **Org owner or admin**: an amber box and a **View in Account Settings** button that opens `/b3/settings`. You can view the configuration there, but only a SuperUser can change it.
- **Other roles**: a neutral box telling you to contact your platform SuperUser or org admin.

To configure outbound email, a SuperUser opens **SMTP** under **Blast Settings**, clicks **Open Account Settings**, then opens the **Integrations** tab in the Bam app and enters the relay details. This is a prerequisite for any real send.

> *No screenshot exists for the SMTP Settings page yet.*

Tracking and unsubscribe (automatic, no UI)

Blast handles opens, clicks, and unsubscribes through endpoints that run without any operator action and without a signed-in user. You do not configure these; they are described here so you understand the metrics.

- **Open tracking**: a 1x1 pixel embedded in each email records an open and increments the campaign's opened counter.
- **Click tracking**: links are rewritten so a click is recorded (with the destination URL) and then redirects to the real target. Only `http`/`https` links are allowed.
- **Unsubscribe**: the unsubscribe link in each email opens a confirmation page with a **Confirm Unsubscribe** button. Confirming suppresses that address across the whole org and increments the unsubscribed counter.

Provider webhooks (automatic, no UI)

Your email provider can notify Blast of bounces and complaints. When a bounce arrives, Blast marks the recipient bounced and increments the bounced counter. When a spam complaint arrives, Blast marks the recipient complained, increments the complaints counter, and automatically unsubscribes that address org-wide. There is no UI for this; the results show up in your campaign and analytics counters.

Why a send was rejected

When you click **Send Now** (or schedule a campaign through the API), Blast runs a CAN-SPAM compliance check before sending. The campaign is rejected with a validation error unless its HTML body contains both:

1. An unsubscribe mechanism (for example, an `{{unsubscribe_url}}` merge field, or text containing "unsubscribe" or "opt-out").
2. A physical mailing address (a recognizable street address, a P.O. box, or a `{{physical_address}}` merge field).

There is no warning before you click **Send Now**; the error surfaces from the send request. The simplest fix is to use the default template footer, which already includes an `{{unsubscribe_url}}` link, and to add your mailing address to the body.

> **TIP**
> The default template footer already includes an unsubscribe link, so the fastest way to clear the CAN-SPAM gate is to build on a template and just add your mailing address to the body.

Working with AI agents

Agents drive Blast through 28 MCP tools backed by the same Blast API the UI uses. Most action tools resolve campaigns, templates, and segments by human-readable name as well as by UUID, so an agent can say "send the April Product Launch campaign" without knowing the ID. Name resolution prefers an exact match, accepts a single fuzzy match, and returns a clean error when a name is ambiguous.

> ***Most agent tools resolve campaigns, templates, and segments by name, so an agent can act on the April Product Launch campaign without ever knowing its ID.***

Common agent actions and their tools:

- **Draft and reuse content**: `blast_create_template`, `blast_get_template`, `blast_list_templates`, `blast_update_template`, `blast_duplicate_template`, and `blast_preview_template` (renders a template with sample merge data without sending).
- **Create and edit a campaign draft**: `blast_draft_campaign` (resolves a template or segment by name or UUID and creates a `draft`), `blast_update_campaign`, `blast_list_campaigns`, and `blast_get_campaign`.
- **Build and check audiences**: `blast_create_segment` and `blast_update_segment` (full field and operator set, including the ones the builder UI hides), `blast_list_segments`, `blast_get_segment`, `blast_preview_segment` (first 50 matching contacts), `blast_evaluate_segment` (the full resolved send list, read-only), `blast_recalculate_segment_count`, and `blast_check_unsubscribed` (confirm an address is not suppressed before targeting it).
- **Send and control a send**: `blast_send_campaign`, `blast_pause_campaign`, and `blast_cancel_campaign`.
- **Review results**: `blast_get_campaign_analytics`, `blast_get_campaign_device_analytics`, `blast_list_campaign_recipients`, `blast_get_engagement_summary`, and `blast_get_engagement_trend`.

The key human-in-the-loop control is on sending. `blast_send_campaign` takes `require_human_approval`, which defaults to `true`. With the default, the tool does not send immediately; it schedules the campaign about an hour out and returns a note that a human

must confirm before it goes. Only `require_human_approval: false` triggers an immediate send. When you review agent work, expect agent-initiated sends to land as scheduled campaigns awaiting a person, and treat any immediate send as an explicit override.

> **TRY IT**
> Have an agent call blast_send_campaign with the default require_human_approval set to true, then open the campaigns list and watch it land as a scheduled campaign about an hour out, awaiting your confirmation before any mail leaves.

Two tools named like AI features are not wired to a model. `blast_draft_email_content` and `blast_suggest_subject_lines` return hard-coded templated strings, not model-generated copy. Use them as scaffolding, not as a writing assistant.

Blast agents also run on the platform-wide agentic surface that spans every app:

- **Identity and heartbeat**: every agent action is tagged with the actor's kind (`human`, `agent`, or `service`) on the activity log, and agent runners post `agent_heartbeat` so operators can see which agents are live.
- **Proposals (approval queues)**: an agent can stage a campaign send or a segment change as a durable proposal with `proposal_create`, and a human clears it with `proposal_decide`. This is the recommended pattern for sends beyond the per-tool `require_human_approval` gate.
- **Unified activity and search**: `activity_query`, `activity_by_actor`, and `search_everything` let an agent (or a reviewer) trace what an agent did across Blast, Bond, and the rest of the suite.
- **Visibility preflight**: before an agent posts campaign results into a shared surface, it calls `can_access` for each cited entity and drops anything the asking user is not allowed to see.
- **Agent policies and webhooks**: a per-agent kill switch and tool allowlist (the `blast.*` prefix) gate which Blast tools a given service account may call, and outbound webhooks push subscribed Blast Bolt events to agent runners.

For the full tool catalog and schemas, see mcp-tools.md.

Working together (live presence)

Like every BigBlueBam app, Blast carries the persistent Bureau presence dock, so collaboration is ambient rather than scheduled. From anywhere in Blast you can see who is around and ring, knock, or invite a teammate into a live voice or video huddle that follows you in a floating window as you both move through the suite. Voice and video here are the digital equivalent of bumping into someone in the hallway or stopping by their desk, not a booked meeting. The deeper per-record collaboration (a presence strip showing who is on a specific item, and real-time co-editing of the same record) lives on the document, board, diagram, and task surfaces, described in the Introduction; in Blast, the shared layer is the always-on Bureau dock.

User Stories

Story: Set up outbound email before your first send

Who: A platform SuperUser (or an org admin verifying the setup). **Goal:** Make sure Blast can actually deliver mail before anyone sends a campaign. **Before you start:** You are signed in to BigBlueBam. SMTP relay credentials are configured by a SuperUser; org admins can only view them.

Steps

1. Open Blast at `/blast/` and click **SMTP** under **Blast Settings** in the sidebar.
2. Read the **SMTP Settings** card. It explains that Blast uses one platform-wide relay, not its own credentials.
3. As a SuperUser, click **Open Account Settings**. (As an org admin, click **View in Account Settings** to confirm the relay is set, then ask a SuperUser if it is not.)
4. In the Bam app at `/b3/settings`, open the **Integrations** tab and enter or confirm the SMTP relay details.
5. Return to Blast.

Result: The platform relay is configured, so future **Send Now** actions can actually deliver email.

Related: "Verify a sending domain for deliverability" below, which improves deliverability once SMTP works.

Story: Send a one-off campaign from scratch

Who: A marketer or operator. **Goal:** Compose and send a single email to your contacts. **Before you start:** You are signed in, the platform SMTP relay is configured, and your org has Bond contacts. You need `read_write` scope.

Steps

1. From the campaigns list at `/blast/`, click **New Campaign**.
2. Enter a **Campaign Name *** and a **Subject Line ***.
3. Optionally set the **From** Name and Email, and choose a **Segment** (or leave it on **All contacts**).
4. With the **Content:** toggle on **Visual Builder**, add blocks from the palette and edit them. Make sure the body has an unsubscribe link and your physical mailing address; the default template footer covers the unsubscribe link.
5. Click **Create Campaign**. You return to the campaigns list with a new `draft`.
6. Click the new campaign's row to open it, then click **Send Now**.
7. If the CAN-SPAM check passes, the status moves to `sending` and delivery begins. If it fails, add the missing unsubscribe link or address and try again.

Result: The campaign is sending, and its detail page begins filling in delivery and engagement metrics.

Related: "Review campaign performance" below. Agents do the same with `blast_draft_campaign` then `blast_send_campaign`.

> *Recipient scope caveat: the sender currently delivers to your whole org's eligible contacts (excluding unsubscribed and empty-email contacts) regardless of which segment you chose. Segment-targeted sending is not yet honored by the sender.*

Story: Reuse a saved template for a campaign

Who: A marketer who has a house style. **Goal:** Build a campaign from an existing template instead of starting blank. **Before you start:** At least one template exists in **Templates**.

Steps

1. From the campaigns list, click **New Campaign**.
2. Enter a **Campaign Name *** (the subject prefills from the template, but you can edit it).
3. Set the **Content:** toggle to **From Template** and pick a template from **Choose a template...**. The subject and body load from the template.
4. Adjust the content as needed.
5. Click **Create Campaign**, open the new draft, and click **Send Now**.

Result: A campaign built on your template is sending.

Related: "Author and version a reusable template" below.

Story: Author and version a reusable template

Who: A designer or marketer maintaining brand templates. **Goal:** Create a reusable email design and keep it updated over time. **Before you start:** You are signed in with `read_write` scope.

Steps

1. Open **Templates** and click **New Template**.
2. Enter a **Template Name *** and **Subject Line ***, and optionally a **Description**.
3. Build the body in **Visual** mode (or switch to **HTML**). Keep the footer's `{{unsubscribe_url}}` link so campaigns using this template pass the CAN-SPAM check.
4. Click **Save Template**.
5. To update later, open the template, edit it, and click **Save Template** again; the version badge bumps (for example, `v1` to `v2`).
6. To fork a template, hover its card in the gallery and click **Duplicate**.

Result: A versioned, reusable template is available in the gallery and selectable in **From Template** when creating a campaign.

Related: Agents do this with `blast_create_template`, `blast_update_template`, and `blast_duplicate_template`.

Story: Build a targeted segment from CRM data

Who: A marketer who wants to target a subset of contacts. **Goal:** Save a reusable filter over Bond contacts and see how many match. **Before you start:** Your org has Bond contacts with the relevant fields populated.

Steps

1. Open **Segments** and click **New Segment**.
2. Enter a **Segment Name** and an optional **Description**.
3. Choose **ALL conditions (AND)** or **ANY condition (OR)**.
4. Add conditions, for example field **Lifecycle Stage**, operator **equals**, value `customer`. Use **Add Condition** for more rows.
5. Click **Create Segment**.
6. Back on the segment list, click **Recalculate count** on the new segment to see how many Bond contacts match.

Result: A saved segment with a contact count, selectable in the **Segment** dropdown on the campaign create form.

Related: Agents do this with `blast_create_segment` (which also exposes the `email`, `first_name`, `last_name` fields and the `is_set`/`is_not_set` operators) and can preview matches with `blast_preview_segment` or resolve the full send list with `blast_evaluate_segment`. Remember the sender does not yet honor the chosen segment at send time.

Story: Review campaign performance

Who: A marketer or manager. **Goal:** Understand how a send performed, both per campaign and across the org. **Before you start:** At least one campaign has reached the `sent` state.

Steps

1. For a single campaign, open it from the campaigns list and read **Total Sent**, **Delivered**, **Opened** (with open rate), and **Clicked** (with click rate), plus the secondary **Bounced**, **Unsubscribed**, and **Complaints**.
2. Read the **Top Clicked Links** table to see which links drew clicks.
3. Page through the **Recipients** table with **Prev** and **Next** to inspect per-recipient delivery status.
4. For an org-wide view, open **Analytics** and read **Total Sent**, **Delivered**, **Avg Open Rate**, **Avg Click Rate**, and **Bounce Rate**, plus the **Weekly Engagement Trend** table.

Result: You can see open, click, bounce, and unsubscribe performance for one campaign and for the org as a whole.

Related: Agents read the same data with `blast_get_campaign_analytics`, `blast_get_campaign_device_analytics`, `blast_get_engagement_summary`, and `blast_get_engagement_trend`.

Story: Verify a sending domain for deliverability

Who: An org admin. **Goal:** Prove your sending domain is authorized so more mail reaches inboxes. **Before you start:** You have `admin` scope and access to your domain's DNS settings.

Steps

1. Open **Domains** under **Blast Settings**.
2. Type your domain (for example, "company.com") and click **Add Domain**.
3. On the domain card, copy each row from **Required DNS Records** into your DNS host.
4. Click **Verify DNS** (refresh icon). The **SPF**, **DKIM**, and **DMARC** indicators turn green when verification succeeds.

Result: A verified sender domain that improves deliverability for your campaigns.

Related: "Set up outbound email before your first send" (SMTP must work first).

Story: A recipient unsubscribes or a message bounces (automatic)

Who: No operator; the system handles this. **Goal:** Honor unsubscribes and record bounces and complaints without manual work. **Before you start:** A campaign has sent and your provider can post bounce/complaint notifications.

Steps

1. A recipient clicks the unsubscribe link, lands on the confirmation page, and clicks **Confirm Unsubscribe**. Their address is suppressed org-wide.
2. When your provider reports a bounce, Blast marks the recipient bounced and increments the bounced counter.
3. When your provider reports a spam complaint, Blast marks the recipient complained, increments the complaints counter, and auto-unsubscribes the address.

Result: Suppressions and bounce/complaint counts update automatically and appear in the campaign and analytics metrics. Suppressed addresses are excluded from future sends.

Related: "Review campaign performance" to see the resulting counters. An agent can confirm an address is not suppressed before targeting it with `blast_check_unsubscribed`.

Story: Let an agent draft a campaign for your approval

Who: An operator working with an AI agent. **Goal:** Have an agent assemble a campaign draft and a target audience, then approve the send yourself. **Before you start:** The agent's service account has the `blast.*` tool allowlist enabled, the SMTP relay works, and your org has Bond contacts.

Steps

1. Ask the agent to build the audience. It calls `blast_create_segment` over Bond contact fields and can sanity-check the size with `blast_preview_segment` or `blast_evaluate_segment`.
2. Ask the agent to draft the campaign. It calls `blast_draft_campaign`, resolving your template and segment by name, and the campaign lands as a `draft`.
3. The agent calls `blast_send_campaign` with the default `require_human_approval: true`. The campaign is scheduled about an hour out and the tool returns a note that a human must confirm.
4. You review the draft in Blast (open it from the campaigns list), check the body for the unsubscribe link and physical address, then either click **Send Now** to send it now or let the scheduled time pass after confirming.

Result: The agent did the assembly work, but no mail left the system without your review. The audit log shows the agent as the actor on the draft and you as the actor on the send.

Related: "Send a one-off campaign from scratch". Agents can also stage the send as a `proposal_create` for an explicit approval queue, and you clear it with `proposal_decide`.

Story: Nurture a Bond contact group, then react in Bolt

Who: An operator (or an agent) wiring email into cross-app automation. **Goal:** Email a group of Bond contacts and have Bolt automation react to the engagement. **Before you start:** You have Bond contacts, a Blast segment over them, a working SMTP relay, and at least one Bolt rule listening for Blast events.

Steps

1. In Blast, build a segment over your Bond contacts (see "Build a targeted segment from CRM data").
2. Create a campaign and send it (see "Send a one-off campaign from scratch").
3. As the campaign sends and recipients open, click, unsubscribe, or bounce, Blast publishes events from source `blast`: `campaign.created`, `campaign.sent`, `campaign.completed`, and the `engagement.opened`, `engagement.clicked`, `engagement.unsubscribed`, and `engagement.bounced` events.
4. In Bolt, a rule that listens for one of these events runs its action, for example creating a Bam task to follow up or updating the contact's record. Bond contact activity timelines also reflect the campaign engagement.

Result: Email activity in Blast drives downstream automation in Bolt and shows up against contacts in Bond, closing the loop from CRM data to email to follow-up.

Related: See the Bond and Bolt help docs. An agent can drive the Blast side end to end with `blast_create_segment`, `blast_draft_campaign`, and `blast_send_campaign`, gating the actual send behind a human via `require_human_approval`.

Related

- **Bond** - The CRM that owns the contacts Blast sends to. Segments filter on Bond contact fields, and campaign engagement flows back to Bond contact activity timelines.
- **Bolt** - The automation engine that reacts to Blast events (`campaign.sent`, `engagement.opened`, and the rest) to create tasks or update records.
- **Bam (the core app)** - Hosts your login and the platform SMTP relay. Configure outbound email at `/b3/settings` under **Account Settings -> Integrations**.
- Blast MCP tools reference - The full catalog of the 28 Blast MCP tools agents use.
- Blast guide - Additional product context. Note: the guide mentions an "archived" campaign state and "A/B testing" that the current build does not implement; treat this help doc's feature list as authoritative.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. How many states can a campaign be in?

a) Four
b) Five
c) Six
d) Seven

2. Which campaign status is the terminal success state and the only one analytics counts?

a) delivered
b) sending
c) sent
d) scheduled

3. What two things must a campaign's HTML body contain to pass the CAN-SPAM gate?

4. Which two campaign statuses make the Send Now button appear on the detail page?

a) draft and scheduled
b) draft and sending
c) scheduled and paused
d) sent and draft

5. Where does Blast pull its audience from, and where does it push its send and engagement events?

6. Where is the platform SMTP relay actually configured?

a) In Blast under SMTP settings
b) In the Bam app under Account Settings then Integrations
c) Per campaign on the create form
d) In the Sender Domains page

7. What does blast_send_campaign do by default when require_human_approval is true?

8. How many rows does the Recipients table show per page on the campaign detail page?

a) 10
b) 25
c) 50
d) 100

9. What are the two match modes for joining a segment's conditions?

10. What scope do you need to add or remove a sender domain?

a) read
b) read_write
c) admin
d) No scope is required

11. When a spam complaint arrives, what three things does Blast do automatically?

12. What do the blast_draft_email_content and blast_suggest_subject_lines tools return?

a) Model-generated copy
b) Hard-coded templated strings
c) An error until a model is configured
d) The contents of a saved template

CHAPTER 10

Blueprint

Diagrams your team and your agents can both read.

Blueprint draws flowcharts, graphs, org charts, and mind maps, but stores each one as a typed graph of nodes and edges rather than a flat picture. Because every box and every line is a real database row, every change is an auditable action, and the same actions are open to AI agents as named tools. This chapter walks you from a blank canvas to a laid-out diagram, then into the round-trip with Bam where a node and its task stay in sync. By the end you will be able to build a diagram, arrange it with one click, and turn it into a project plan.

At a glance

- Typed node and edge graph, so every change is an ordinary auditable action
- Five shapes, five edge kinds, and server-side ELK auto-layout in four flavors
- Two-way Bam sync: a linked node and its task share a title and description
- Generate a diagram from a Bam project, or promote a whole graph into tasks
- Full MCP surface, so an agent can emit a fully wired diagram in one call

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Browse and filter diagrams . Create a diagram . The editor canvas . Add nodes . Edit a node . Move and pin nodes

> *Blueprint is a diagram editor whose diagrams are a typed relational graph of nodes and edges, not an opaque drawing. Reach for it when you want a flowchart, graph, org chart, mind map, or system map that every teammate, and every AI agent, can read, edit, version, and turn into Bam tasks with a full audit trail.*

Overview

Blueprint stores every diagram[87] as a set of database rows: a diagram record, one row per node[88], and one row per edge[89]. Because the graph is structured rather than a flat image, every change you make is an ordinary, auditable action, and the same actions are exposed to AI agents as MCP tools. That is the core idea: a node is a first-class object that can be styled, nested, linked to a task in Bam, promoted into a new task, or generated from an existing one.

You work with three kinds of object. A **diagram** is the top-level artifact (a flowchart, a graph, an org chart). A **node** is a box on the canvas with a shape[90], label, description, size, and position. An **edge** is a connection between two nodes with a direction, a kind, and arrow markers. Diagrams live in your organization and can optionally belong to a project, which controls who can see them.

Blueprint is wired into Bam, the Kanban and sprint tool. You can generate a diagram from an existing Bam project (one node per task, edges drawn from parent/child links), and you can promote a diagram into Bam (one task per node, parent links from the edges). A node linked to a Bam task stays in two-way sync: rename the node and the task title updates, rename the task and the node updates.

> ***Link a node to a Bam task once and the two stay in lockstep, both directions, nothing to configure.***

Several capabilities in Blueprint are agent-driven or API-driven rather than point-and-click. Anything an agent can do to a diagram, it does through a named tool, so an agent can read a process description and emit a fully wired diagram in a single call, with the same permissions and the same audit trail as a person. A few backend capabilities still have no rendered screen (Mermaid import, image export, template seeding); where that is the case this document says so plainly rather than promising a button that does not exist.

> ***One call in, a complete laid-out diagram out, audited exactly like a person's edit.***

[87] **Diagram.** the top-level artifact. Has a name, a type, a layout algorithm, a visibility level, an optional project, and an archived flag.
[88] **Node.** a vertex on the canvas. Carries a shape, label, markdown description, position, width and height, a pinned flag, fill color, an optional parent (for nesting), and an optional cross-product link.
[89] **Edge.** a connection between two nodes. Carries a kind (Default, Dependency, Flow, Reference, Inherits) and an end marker (Filled arrow, Open arrow, No arrow / None).
[90] **Shape.** the visual form of a node. The palette offers Rounded, Rectangle, Diamond, Ellipse, and Hexagon.

Key concepts

- **Diagram** - the top-level artifact. Has a name, a type, a layout algorithm[91], a visibility[92] level, an optional project, and an archived flag.
- **Diagram type**[93] - a tag that picks the type icon and the sidebar grouping. The New diagram dialog offers Flowchart, Graph, Sequence, Class, and Mind map. Generate from Bam writes the type org_chart.
- **Node** - a vertex on the canvas. Carries a shape, label, markdown description, position, width and height, a pinned[94] flag, fill color, an optional parent (for nesting), and an optional cross-product link.
- **Shape** - the visual form of a node. The palette offers Rounded, Rectangle, Diamond, Ellipse, and Hexagon.
- **Edge** - a connection between two nodes. Carries a kind (Default, Dependency, Flow, Reference, Inherits) and an end marker (Filled arrow, Open arrow, No arrow / None).
- **Pinned** - a node excluded from auto-layout. A pinned node keeps the position you set by hand when you run a layout pass.
- **Layout algorithm** - the auto-arrange engine, run server-side by ELK. The layout dropdown offers Layered, Force-directed, Tree, and Grid, and all four apply a layout. Tree maps to ELK's mrtree and Grid maps to ELK's rectpacking. The radial algorithm is reachable only through the API or an agent.
- **Linked entity**[95] - a cross-product reference from a node to an object in another app, such as a Bam task, a Beacon entry, a Bond deal, or a Bearing goal. Only Bam tasks have live two-way sync.
- **Visibility** - who can see the diagram. Private (you, plus collaborators), Project (project members, or the org if there is no project), or Organization (everyone in your org).
- **Collaborator**[96] - a user granted explicit access to one diagram at a role of owner, editor, commenter, or viewer, on top of any project- or org-level visibility. Managed in the editor's **People** panel, or by an agent through the API.
- **Snapshot (version)** - a saved, immutable copy of the whole graph that you can restore later. Browsed and restored from the editor's **History** panel.

---

[91] **Layout algorithm.** the auto-arrange engine, run server-side by ELK. The layout dropdown offers Layered, Force-directed, Tree, and Grid, and all four apply a layout. Tree maps to ELK's mrtree and Grid maps to ELK's rectpacking. The radial algorithm is reachable only through the API or an agent.

[92] **Visibility.** who can see the diagram. Private (you, plus collaborators), Project (project members, or the org if there is no project), or Organization (everyone in your org).

[93] **Diagram type.** a tag that picks the type icon and the sidebar grouping. The New diagram dialog offers Flowchart, Graph, Sequence, Class, and Mind map. Generate from Bam writes the type org_chart.

[94] **Pinned.** a node excluded from auto-layout. A pinned node keeps the position you set by hand when you run a layout pass.

[95] **Linked entity.** a cross-product reference from a node to an object in another app, such as a Bam task, a Beacon entry, a Bond deal, or a Bearing goal. Only Bam tasks have live two-way sync.

[96] **Collaborator.** a user granted explicit access to one diagram at a role of owner, editor, commenter, or viewer, on top of any project- or org-level visibility. Managed in the editor's **People** panel, or by an agent through the API.

- **Comment**[97] - a note attached to the whole diagram or anchored to a single node, with a resolved flag for review threads. Read, written, and resolved in the editor's **Comments** panel, or by an agent through the API.
- **Star**[98] - your personal favorite flag on a diagram, used to filter the list.

> **TIP**
> The layout dropdown gives you Layered, Force-directed, Tree, and Grid, but the radial arrangement never shows up there. If you want a radial layout, an agent can apply it through blueprint_apply_layout.

Where to find it

Blueprint is served at `/blueprint/`. Reach it from the platform Launchpad in the top header of any BigBlueBam app.

You need a signed-in BigBlueBam account; the app refuses to load when you are not authenticated and links you to `/b3/` to log in with the message "Please log in to BigBlueBam first to access Blueprint." To use the Bam-integration features (generate from Bam, promote to tasks, two-way sync), you also need a Bam project with tasks, because those flows talk to Bam directly using your shared session.

Press the `?` key from the diagram list or the editor (when no field is focused) to open the in-app help viewer at `/blueprint/help`.

> **TRY IT**
> Open /blueprint/, press N to drop a node, type a Label in the Inspector, then set the layout dropdown to Layered and click Apply layout to watch ELK arrange it server-side.

[97] **Comment.** a note attached to the whole diagram or anchored to a single node, with a resolved flag for review threads. Read, written, and resolved in the editor's **Comments** panel, or by an agent through the API.
[98] **Star.** your personal favorite flag on a diagram, used to filter the list.

Feature reference

Browse and filter diagrams

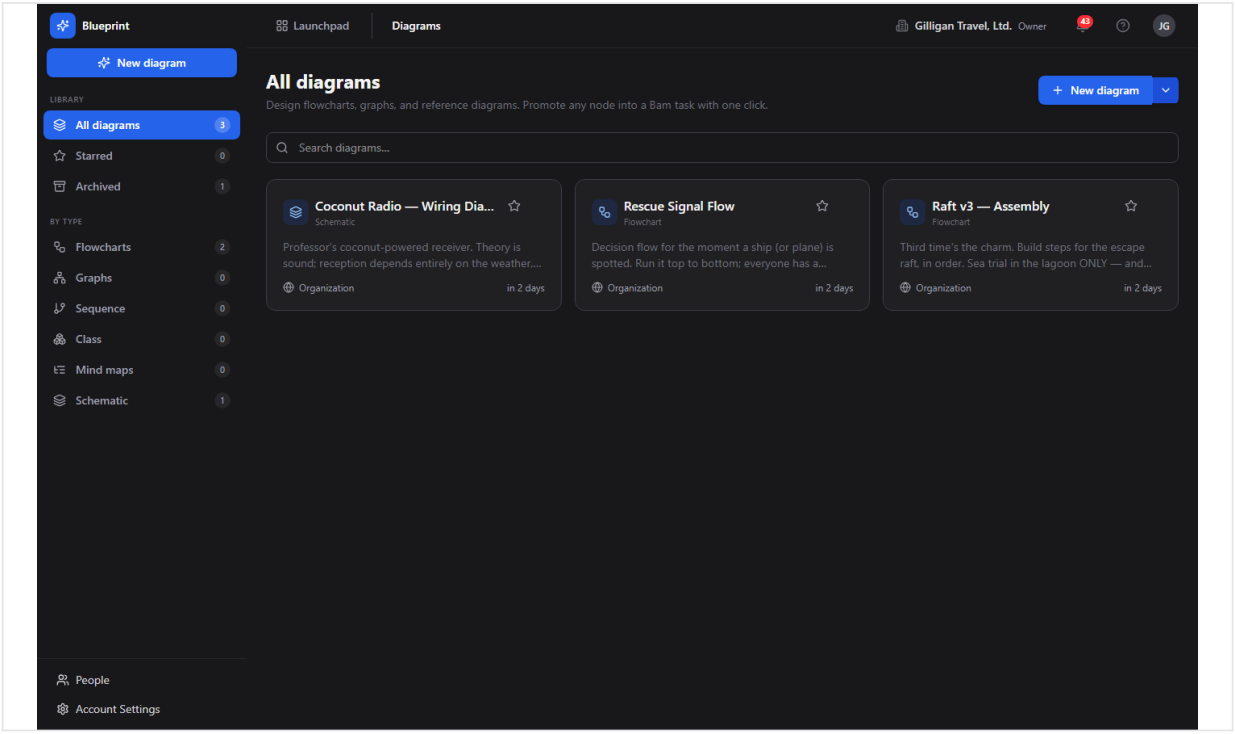


Figure 10.1 Diagram list

The diagram list at `/blueprint/` shows every diagram you can see, as a grid of cards. Each card shows the type icon, the diagram name, the lowercase type label, a Star toggle, and a More menu. The card footer shows the visibility (Private, Project, or Organization, each with its own icon) and the relative time the diagram was last updated.

To find a diagram:

1. Use the left sidebar to filter. Under **Library**, click **All diagrams**, **Starred**, or **Archived**. Under **By type**, click **Flowcharts**, **Graphs**, **Sequence**, **Class**, or **Mind maps**. Each pill shows a count.
2. Type in the **Search diagrams…** box to filter the visible cards by name or description.
3. Click a card to open it in the editor, or open the card's More menu and choose **Open editor**.

The page heading reflects the active filter (for example All diagrams, Starred diagrams, or Flowchart diagrams). When nothing matches, an empty state appears with a New diagram button.

Create a diagram

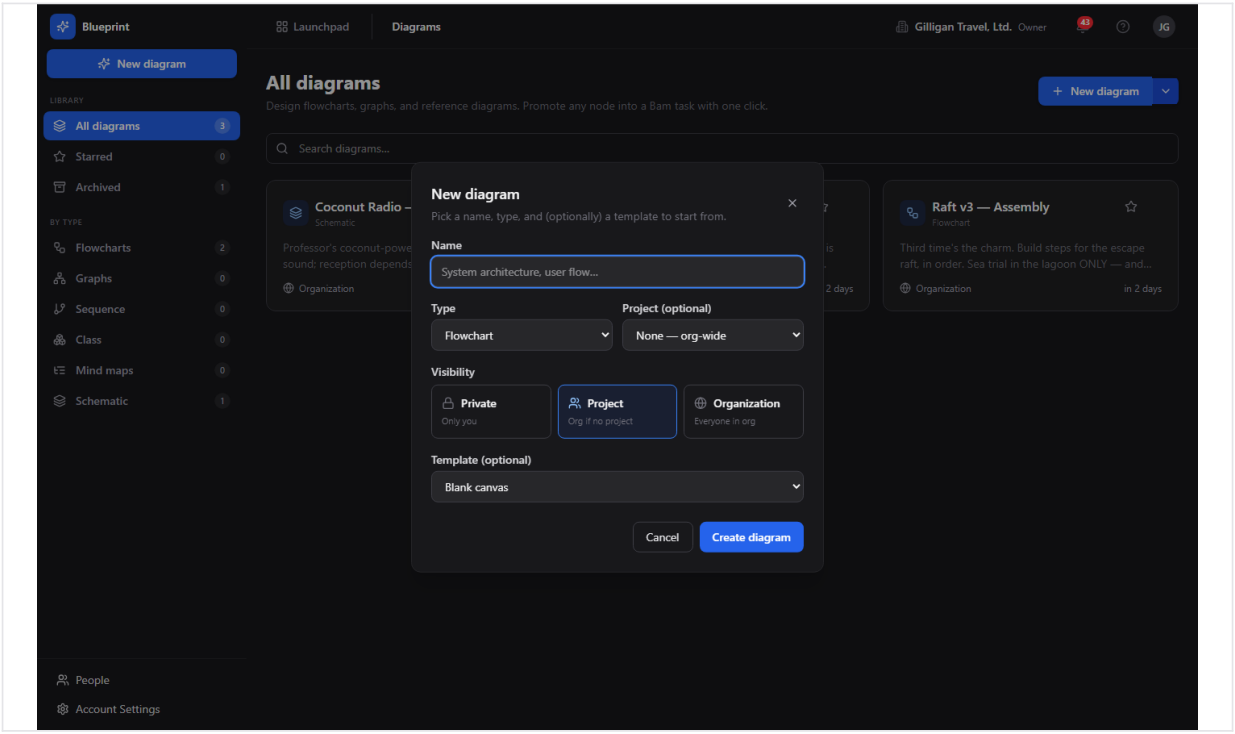


Figure 10.2 New diagram dialog

To create a blank diagram:

1. Click **New diagram** at the top right of the list, or **New diagram** in the sidebar.
2. In the **New diagram** dialog, enter a **Name** (required, up to 200 characters). The placeholder suggests "System architecture, user flow...".
3. Choose a **Type**: Flowchart, Graph, Sequence, Class, or Mind map.
4. Optionally choose a **Project (optional)**. Leave it on **None - org-wide** for a diagram that is not tied to a project.
5. Pick a **Visibility**: **Private** (Only you), **Project** (Project members, or Org if no project), or **Organization** (Everyone in org). The dialog defaults to **Project**; if you choose Project without a project, the diagram is created Private.
6. Optionally pick a **Template (optional)**. The empty option is labeled **Blank canvas**, and system templates show a "system" suffix. See the note below before relying on this.
7. Click **Create diagram**. The new diagram opens in the editor.

The split button next to New diagram has a chevron menu with two entries: **Blank diagram or template** (opens the same dialog) and **From Bam project tasks...** (opens the Generate from Bam dialog, described later).

Note on templates: the Template select is present, but a chosen template does not seed any nodes or edges into the new diagram. The backend accepts a template id and ignores its content, and there is no UI to create templates today, so the list is effectively empty unless

rows were inserted directly into the database. Treat every new diagram as a blank canvas regardless of the template you pick.

> **NOTE**
> The Template select in the New diagram dialog is wired but inert today: the backend accepts a template id and ignores its content. Treat every new diagram as a blank canvas no matter which template you pick.

The editor canvas

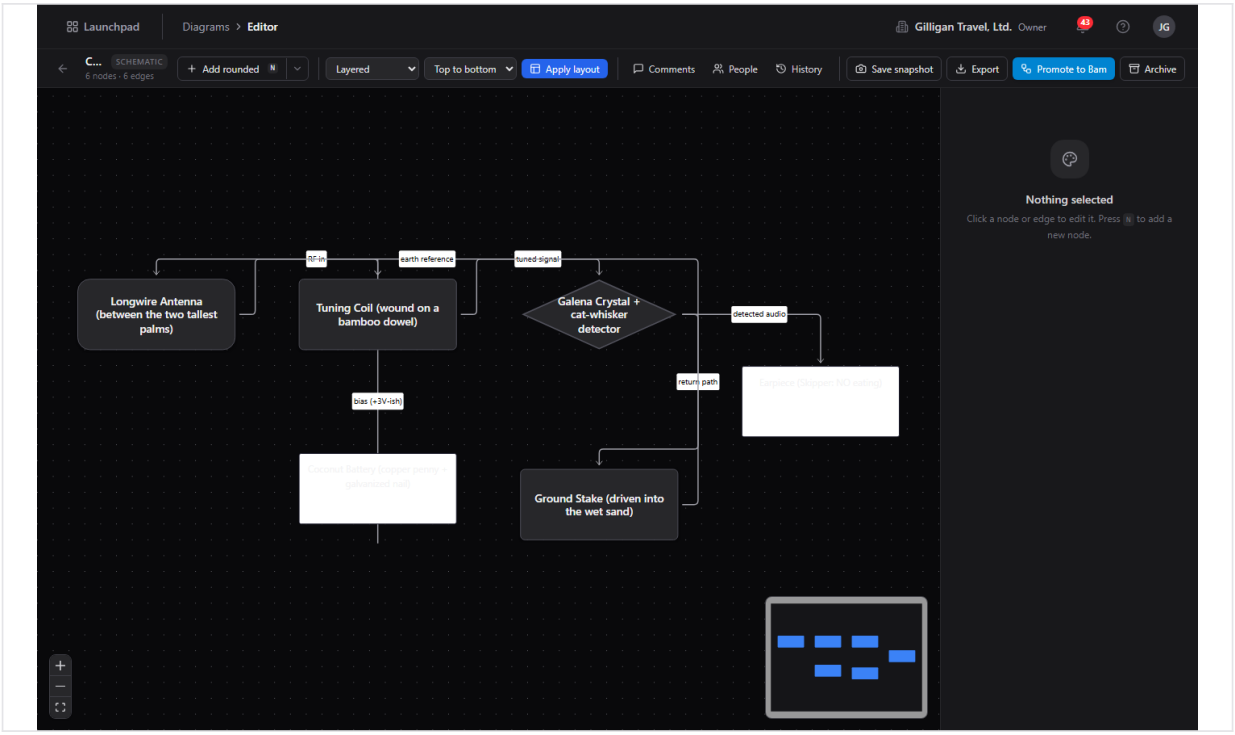


Figure 10.3 Diagram editor

Opening a diagram shows the editor at `/blueprint/d/<id>`. The canvas is an infinite, pannable React Flow surface with a dotted background, zoom and fit controls at the bottom left, and a minimap at the bottom right. The top bar shows a Back arrow (titled **Back to diagrams**), the diagram name, an uppercase type chip, and a count line reading `N nodes · M edges`.

The right side of the screen is the **Inspector**, which shows the properties of whatever node or edge you have selected. When nothing is selected it reads **Nothing selected** with the hint "Click a node or edge to edit it. Press **N** to add a new node."

Add nodes

You can add a node four ways:

- Click the **Add <shape>** split button in the top bar. The body adds the last shape you used; the chevron opens a menu listing **Rounded**, **Rectangle**, **Diamond**, **Ellipse**, and **Hexagon**, with a **Last** badge on the most recent.
- Press the **N** key to add a node with the last-used shape.

- Right-click empty canvas and pick a shape under **Add node** in the pane menu.
- Drag a connection out of a node handle and release it on empty canvas. An **Add connected node** palette opens at the drop point; pick a shape and the new node is created already wired to the origin node, with the arrow following the direction you dragged.

Every new node starts with the label **New node** and is selected immediately so the Inspector is open and ready for you to type. The last-used shape is remembered per browser.

Edit a node

Select a node to edit it in the Inspector on the right. The node panel header shows **Node** and a short id, a Pin toggle, and a Delete button.

To edit a node:

1. Select the node on the canvas.
2. Edit the **Label** field; the change saves when you leave the field or press Enter.
3. Edit the **Description** using the rich-text editor (it supports bold, italic, code, and links). The placeholder reads "Describe this node… **bold**, *italic*, `code`, links all work."
4. Change the **Shape** with the dropdown.
5. Set **Width** (40 to 2000) and **Height** (30 to 2000).
6. Pick a **Fill color** swatch, or click **Clear** to remove the fill.

You can also resize a node directly on the canvas: select it and drag the resize handles that appear around it. The new size is saved when you finish dragging.

The node right-click menu offers the same actions plus a few extras: **Duplicate** (with the keyboard hint), **Pin position** or **Unpin (allow auto-layout)**, a **Change shape** list, the **Fill color** swatches (White, Red, Amber, Green, Blue, Violet, Pink, Zinc, and Clear), **Link to entity…**, **Promote to Bam task**, and **Delete node**.

Move and pin nodes

Drag a node to reposition it. The new position is saved when you release the drag.

To keep a node where you put it during an auto-layout pass, **pin** it: select the node and click the Pin toggle in the Inspector, or right-click the node and choose **Pin position**. A pinned node is excluded from layout and cannot be dragged until you unpin it. To release it, click the Pin toggle again or choose **Unpin (allow auto-layout)**.

Duplicate a node

To copy a node, select it and press **Cmd/Ctrl+D**, or right-click it and choose **Duplicate**. The clone appears offset down and to the right and copies the label, shape, size, style, and any linked entity. Edges connected to the original are not copied; wire the clone in yourself if you need it connected.

Delete a node

To delete a node, select it and press **Delete**, or right-click it and choose **Delete node**, or click the trash button in the Inspector. A confirmation appears asking "Delete this node and its connected edges?"; deleting a node removes every edge attached to it.

If the node is linked to a Bam task, a richer dialog appears titled **Delete "<label>"?**. It explains that the node is linked to a Bam task and offers three buttons: **Cancel**, **Delete node only**, and **Delete node + task**. The last option also deletes the linked Bam task, including its subtasks, comments, time entries, and activity.

Connect nodes with edges

To draw an edge, hover a node to reveal its connection handles (top, left, right, bottom), then drag from a handle to a handle on another node. Releasing on a valid handle creates the edge. Releasing on empty canvas opens the **Add connected node** palette so you can create the target node and the edge in one gesture.

New edges use a smooth line with a closed (filled) arrowhead by default.

Edit an edge

Select an edge to edit it in the Inspector. The edge panel header shows **Edge** and a short id and a Delete button.

To edit an edge:

1. Select the edge.
2. Edit the **Label** field.
3. Choose a **Kind**: Default, Dependency, Flow, Reference, or Inherits.
4. Choose an **End marker**: Filled arrow, Open arrow, or None.

The edge right-click menu offers the same **Edge kind** and **End marker** choices, plus **Delete edge**.

Delete an edge

To delete an edge, select it and press **Delete**, right-click it and choose **Delete edge**, or click the trash button in the Inspector.

Snap to grid

To line nodes up on a grid, right-click empty canvas and choose **Snap to grid** under **Align**; a checkmark shows it is on. While it is on, dragging a node quantizes its position to the grid. To clean up an existing diagram in one pass, choose **Snap nodes to grid now**, which rounds every node onto the grid and saves the moves. The snap-on setting is remembered per browser.

Auto-layout

Auto-layout arranges the graph automatically using the ELK engine, server-side. Pinned nodes keep their hand-set positions.

To run a layout:

1. In the top bar, choose an algorithm from the layout dropdown: **Layered**, **Force-directed**, **Tree**, or **Grid**.
2. Choose a direction: **Top to bottom**, **Left to right**, **Bottom to top**, or **Right to left**. Direction applies to layered layouts.
3. Click **Apply layout**.

You can also right-click empty canvas and pick an algorithm under **Reorganize**, where the current algorithm carries a **Current** badge.

All four dropdown choices apply a layout. **Layered** and **Force-directed** map to ELK's layered and force algorithms. **Tree** maps to ELK's mrtree (a tree layout) and **Grid** maps to ELK's rectpacking (a compact grid-like packing). The radial algorithm is not in the dropdown and is reachable only through the API or an agent using `blueprint_apply_layout`.

Export a diagram

To export, click the **Export** dropdown in the top bar. It offers **Mermaid (.mmd)** and **JSON**, a separator, and **Reload from server**. Pick a format and the file downloads, named after the diagram.

You can also export from the pane right-click menu via **Export as Mermaid** and **Export as JSON**.

Only Mermaid and JSON export are offered in the editor. The underlying tool advertises SVG and PNG, but those are deferred to a worker render job and the in-process export service rejects them with "Format ... is not supported in-process", so image export is not available from the editor today.

Mermaid import

The backend can import a Mermaid source string and turn it into a Blueprint graph, materializing its nodes and edges into the typed tables. There is no rendered screen for pasting Mermaid into the editor yet, so this is currently an API or agent capability rather than a button. Import can replace the existing graph or append to it, and you can run a layout pass afterward to position the new nodes.

Snapshots (versions)

A snapshot saves the entire graph so you can return to it later.

To save a snapshot, click **Save snapshot** (the camera button) in the top bar, choose **Save snapshot** from the pane right-click menu, or use the **Save snapshot** button at the top of the **History** panel. You can enter an optional label when prompted ("Snapshot label (optional)"). Each snapshot is numbered automatically, one higher than the latest.

To browse and restore versions, open the **History** panel from the top bar (the clock icon). Each entry shows its version number (for example `v2`), its label or "Unlabeled", who saved it,

and how long ago. Click **Restore** on a version, then **Confirm** the prompt ("Replace the current graph with v<n>?") to roll the diagram back to that snapshot. See the Comments, People, and version History panels section for the full panel walkthrough.

Star and archive

To star a diagram, click the **Star** toggle on its card in the list (the title reads Star or Unstar). Starred diagrams show under the **Starred** sidebar filter.

To archive a diagram, use the card's More menu and choose **Archive**, or in the editor click **Archive** in the top bar, or choose **Archive diagram** from the pane right-click menu. Archiving is a soft delete: the diagram is hidden from the default list but its nodes, edges, comments, and snapshots are preserved. You confirm before it archives ("Archive this diagram? It will be hidden from the default list."). Archived diagrams show under the **Archived** sidebar filter. Archiving requires admin scope on your API key when done by an agent.

> **WARNING**
> Deleting a node removes every edge attached to it, and if the node is linked to a Bam task, choosing Delete node + task also deletes that task along with its subtasks, comments, time entries, and activity. Archiving only hides a diagram; node and task deletion is the destructive one to watch.

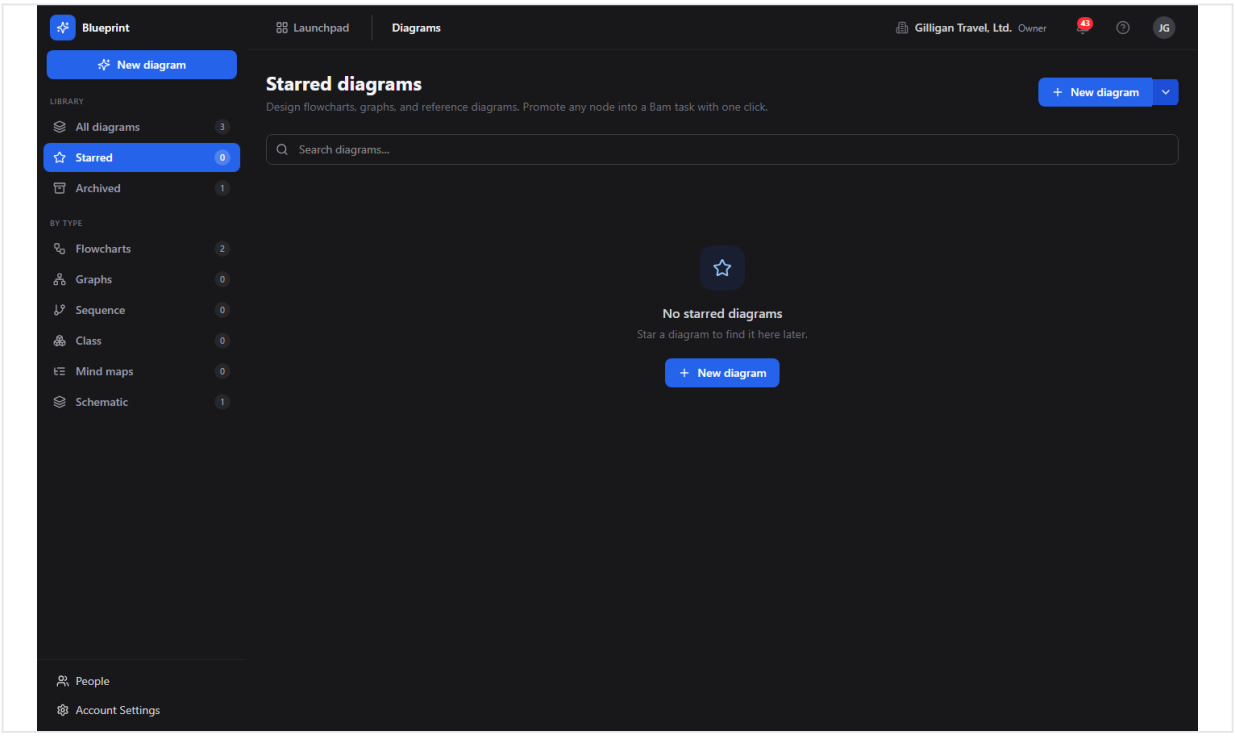


Figure 10.4 Starred diagrams

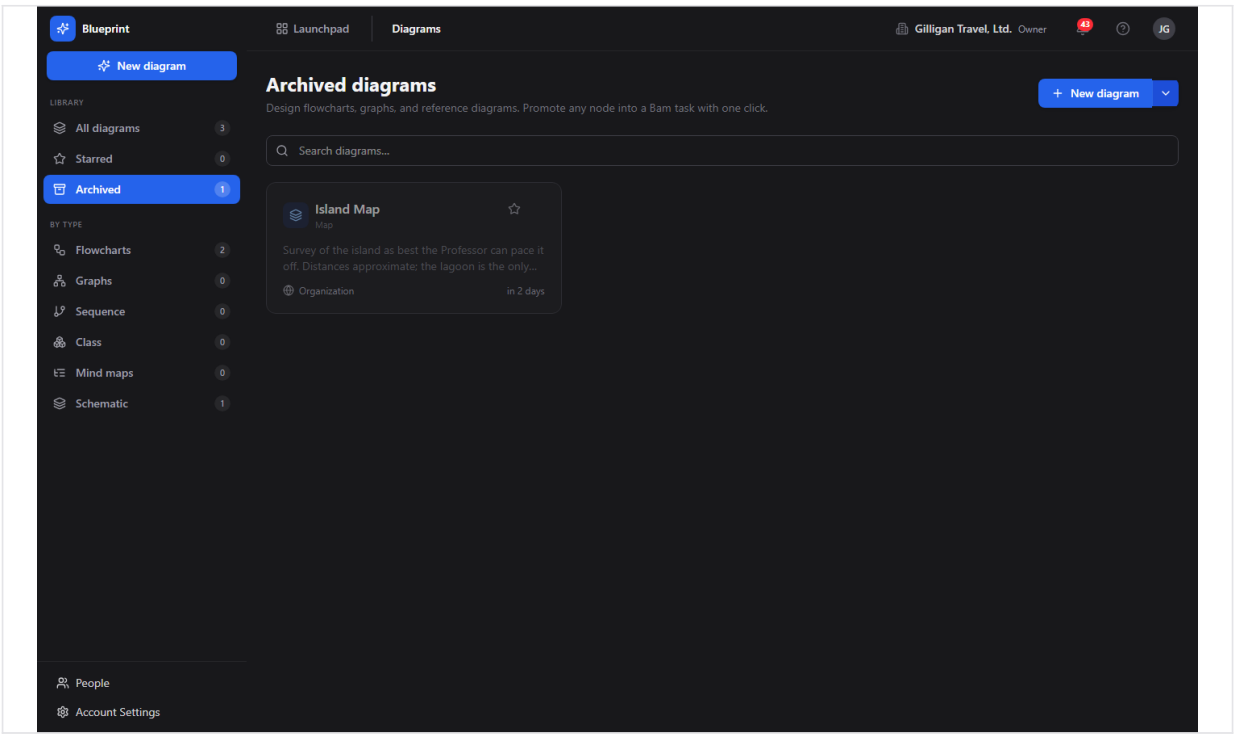


Figure 10.5 Archived diagrams

Link a node to another app's entity

A node can carry a cross-product link to an object in another app, so the diagram references entities across the suite. The canonical link types are bam.task, beacon.entry, bearing.goal, bond.deal, bond.contact, and helpdesk.ticket. Only bam.task links stay in live two-way sync.

To link a node:

1. Select the node.
2. In the Inspector, under **Linked entity**, type the link type into the **type** field (for example bam.task) and the entity id (a UUID) into the **entity id (uuid)** field.
3. Click **Link**.

Alternatively, right-click the node, choose **Link to entity…**, and enter the reference type and UUID when prompted. A linked node shows a small badge on the canvas.

Promote a single node to a Bam task

To turn one node into a Bam task:

1. Select the node.
2. Click **Promote to task** in the Inspector, or right-click the node and choose **Promote to Bam task**.
3. If the diagram is not tied to a project, enter the Bam project id (a UUID) when prompted.
4. The app creates the task in Bam, then links the node back to it so future title and description edits sync both ways.

Under the hood the Blueprint backend returns a task payload; the app posts that payload to Bam itself using your session, then links the node to the new task. The backend does not create the task on its own.

Promote a whole graph to Bam tasks

To turn an entire diagram into a Bam project plan:

1. Click **Promote to Bam** (the sky-blue button) in the top bar.
2. If the diagram has no project, enter a Bam project id (a UUID) when prompted.
3. Choose an edge direction when prompted: `source-parent` (default; an edge A to B means A is the parent of B), `target-parent` (A to B means B is the parent of A), or `none` (skip parent links).
4. Watch the progress overlay. It creates one task per node, reuses any task a node is already linked to, then wires parent and child links from the edges.
5. When it finishes, the summary reads "Created N task(s)", plus "reused M existing" and "wired K parent link(s)" where applicable. Any failures are listed.

As with single-node promotion, the backend returns a plan and the app executes it against Bam; the diagram itself does not create tasks server-side. After promotion, each node is linked to its new task, so the badges appear and two-way sync is active.

Generate a diagram from a Bam project

To build a diagram automatically from an existing project:

1. In the list, click the chevron next to **New diagram** and choose **From Bam project tasks...**.
2. In the **Generate from Bam project** dialog, choose a **Project**.
3. Optionally check **Include completed tasks**.
4. Optionally check **Run auto-layout (layered, top-to-bottom)** (on by default).
5. Click **Generate**.

The result is a new diagram with one node per task and edges drawn from existing parent/child links, opened already laid out. Each node is linked to its task, so renaming a node updates the task and renaming the task updates the node. Generated diagrams are written with the type org_chart.

Two-way sync with Bam

Once a node is linked to a Bam task (through generation, promotion, or a manual link), the two stay in sync. Editing the node's label or description in Blueprint updates the linked task in Bam. Changing the task's title or description in Bam updates every node linked to it. This happens automatically; there is nothing to configure.

Live collaboration refresh

When more than one person has a diagram open, Blueprint keeps everyone's canvas current. When anyone adds, moves, edits, or deletes a node or edge, every other open editor refetches the graph and updates. Your own zoom and pan position are not affected by other people's edits. The presence strip in the top bar shows who else is on the diagram, and the selected node is URL-addressable (`?node=<id>`) so a follower can be brought to the same node.

Comments, People, and version History panels

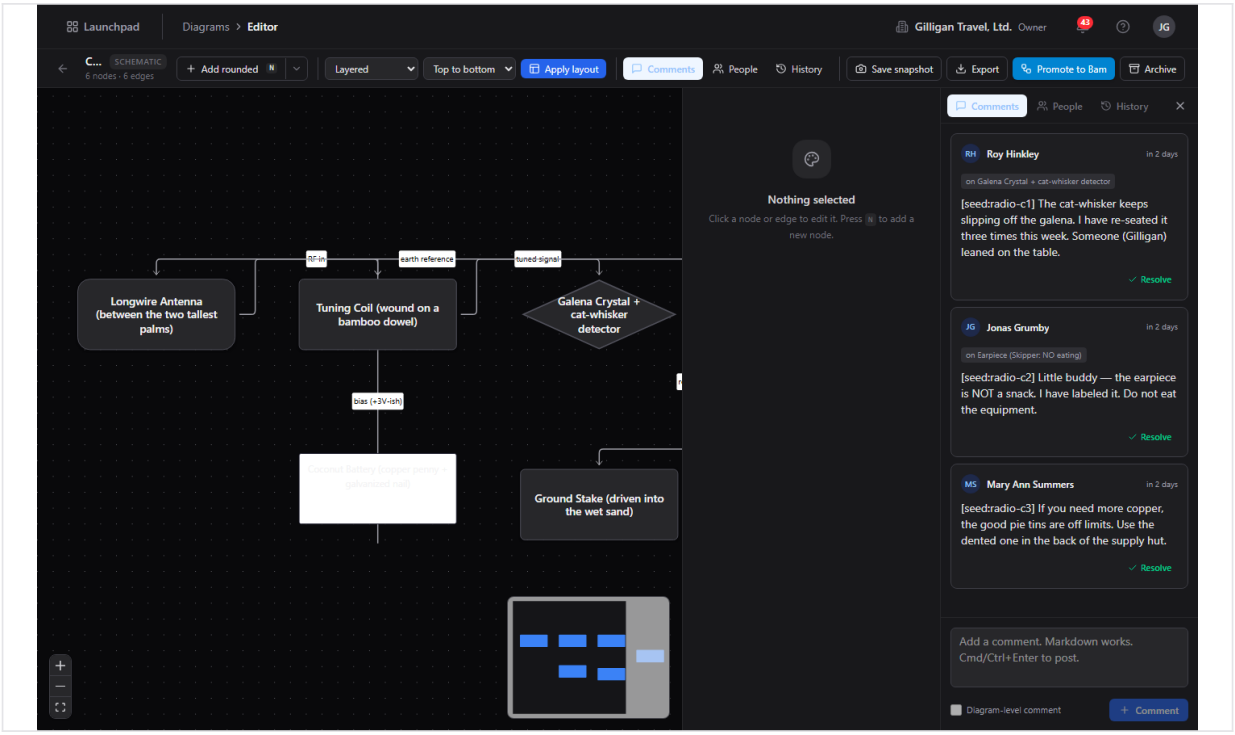


Figure 10.6 Comments & collaboration

The editor's top bar carries three toggle buttons, just left of the snapshot and export controls, that open a panel on the right side of the canvas: **Comments**, **People**, and **History**. Each button opens its panel and highlights while active; click it again, or the panel's close (X) button, to dismiss it. The three tabs share one panel, so you can switch between them without losing your place. Comments, collaborators, and snapshots are all preserved when a diagram is archived.

Comments. Open the **Comments** panel (the speech-bubble icon) to read, write, and resolve notes on the diagram. Each comment shows its author, the relative time it was posted, and the rendered markdown body; a comment anchored to a node shows an "on <node label>" chip. To add one, type in the composer at the bottom (markdown works) and click **Comment** or press **Cmd/Ctrl+Enter**. The composer's checkbox controls anchoring: with a node selected on the canvas it reads **Pin to <node>** and attaches the comment to that node; with nothing selected it posts a diagram-level comment. Click **Resolve** to close a review thread (or **Reopen** to undo); resolved comments are hidden until you click **Show N resolved**.

People (collaborators). Open the **People** panel (the people icon) to grant explicit per-diagram access on top of project- or org-level visibility. Search by name or email in the **Add by name or email** box and pick a person to add them (as an editor by default). Each collaborator row shows their name, a **role** dropdown (**Owner**, **Editor**, **Commenter**, or **Viewer**), and a trash button to remove them. Change a role from the dropdown. When a diagram has no direct collaborators the panel explains that people with project or org access can still see it.

History (versions). Open the **History** panel (the clock icon) to browse and restore snapshots. The **Save snapshot** button at the top captures the current graph as a new version. Each entry shows its version number, its label or "Unlabeled", who saved it, and how long ago. Click **Restore** then **Confirm** to roll the graph back to that version. See Snapshots (versions) for how snapshots are numbered and what they capture.

All three panels are also fully agent- and API-driven, so an agent manages comments, collaborators, and versions through the same MCP tools described under Working with AI agents.

Working with AI agents

Blueprint is built to be driven by AI agents. Every structural change is exposed as an MCP tool, so an agent acts on a diagram with the same permissions and the same audit trail as a person. There are 36 Blueprint MCP tools, covering the full surface: diagram read and write, nodes, edges, layout and generation, the Bam round-trip, versions, comments, collaborators, stars, Mermaid import, and templates.

Common agent flows:

- **Author a whole diagram in one call.** An agent reads a process description, org roster, or task list and calls `blueprint_create` to make the diagram, then `blueprint_generate` with a list of node specs and edge specs. The server materializes the ids, wires up parent containers, and runs auto-layout. This is the headline path; `blueprint_generate` accepts 1 to 500 node specs and up to 2000 edge specs, with `replace: true` to wipe the existing graph first or `auto_layout: false` to skip the post-generate pass.
- **Turn a Mermaid block into an editable graph.** `blueprint_import_mermaid` parses a Mermaid source string (for example from a Brief, a Beacon entry, or an LLM draft) into the typed node/edge graph, then `blueprint_apply_layout` positions it.
- **Edit incrementally.** `blueprint_add_node`, `blueprint_update_node`, `blueprint_move_node`, `blueprint_duplicate_node`, and `blueprint_delete_node` cover nodes; `blueprint_add_edge`, `blueprint_update_edge`, and `blueprint_delete_edge` cover edges; `blueprint_apply_layout` re-arranges. Through `blueprint_apply_layout` an agent can use the radial algorithm that the UI dropdown does not reach.
- **Read the graph.** `blueprint_list`, `blueprint_get`, `blueprint_read_nodes`, `blueprint_read_edges`, `blueprint_search`, and `blueprint_export`. Note

`blueprint_search` matches only on name and description and filters the visible list client-side; it does not search node labels yet.

- **Round-trip with Bam.** `blueprint_generate_from_bam` turns a project's tasks into a diagram; `blueprint_promote_graph_to_tasks` and `blueprint_promote_node_to_task` return a plan the caller executes against Bam; `blueprint_link_entity` attaches a cross-product reference so the two-way sync hook fires.
- **Version, review, and share.** `blueprint_snapshot_version` / `blueprint_list_versions` / `blueprint_restore_version` manage restore points; `blueprint_list_comments` / `blueprint_add_comment` / `blueprint_update_comment` (set `resolved: true` to resolve a thread) drive review; `blueprint_list_collaborators` / `blueprint_add_collaborator` / `blueprint_remove_collaborator` manage explicit access; `blueprint_star` / `blueprint_unstar` set the caller's personal favorite; `blueprint_list_templates` lists templates.

Destructive tools follow a preview-then-commit pattern: `blueprint_archive`, `blueprint_delete_node`, `blueprint_delete_edge`, `blueprint_restore_version`, and `blueprint_remove_collaborator` first return a confirmation prompt and act only when called again with `confirm_action: true`. `blueprint_archive` also requires admin scope. Service-account calls are checked against the platform agent policy kill switch and the `blueprint.*` allowlist before they run, and an agent posting cited entities into shared surfaces should call the platform `can_access` tool first. Agent runs are recorded in the unified activity feed under the agent identity, surface a heartbeat, and can route work through the platform proposal queue for human approval.

A few tool descriptions describe roadmap behavior that the running app does not yet deliver. `blueprint_create` claims a template seeds the diagram, but template content is never applied. `blueprint_export` advertises svg and png, which the in-process export service rejects. `blueprint_promote_node_to_task` and `blueprint_promote_graph_to_tasks` return a payload or plan only; the caller performs the actual task creation in Bam and then back-links the node with `blueprint_link_entity`. Plan for these gaps when reviewing agent work.

For the full tool catalog and schemas, see the Blueprint MCP-tools reference in `docs/apps/blueprint/`.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. The editor shows who else is in the diagram, your changes and cursors sync live, and you can ring a collaborator into a huddle while you work the same graph. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Create your first flowchart

Who: Anyone with a BigBlueBam account who wants to sketch a process. **Goal:** Build a small flowchart, connect and label the steps, tidy the layout, and save a copy. **Before you start:** Be signed in. No project is required.

Steps

1. Go to `/blueprint/` from the Launchpad.
2. Click **New diagram**. Enter a **Name**, leave **Type** on **Flowchart**, choose a **Visibility**, and click **Create diagram**.
3. In the editor, press **N** (or click **Add <shape>**) to add a node. With the node selected, type a **Label** in the Inspector.
4. Add two or three more nodes the same way.
5. Drag from one node's handle to another node's handle to connect them. Repeat until the steps are linked.
6. Select an edge and set its **Label** and **Kind** in the Inspector if you want to annotate the flow.
7. Set the layout dropdown to **Layered**, pick a direction such as **Top to bottom**, and click **Apply layout**.
8. Click **Save snapshot** and enter an optional label.

Result: A laid-out flowchart with labeled steps and a saved snapshot you can return to.

Related: Export the diagram as Mermaid or JSON from the **Export** menu. An agent would build the same diagram with `blueprint_create` plus `blueprint_generate`.

Story: Visualize an existing Bam project

Who: A project lead who wants a picture of a project's task structure. **Goal:** Generate a diagram from a Bam project, with each node tied to its task. **Before you start:** Have a Bam project that contains tasks, in the same org and session.

Steps

1. Go to `/blueprint/`.
2. Click the chevron next to **New diagram** and choose **From Bam project tasks…**.
3. In the **Generate from Bam project** dialog, choose your **Project**.
4. Check **Include completed tasks** if you want finished work in the diagram.
5. Leave **Run auto-layout (layered, top-to-bottom)** checked.
6. Click **Generate**.

Result: A new diagram opens, laid out, with one node per task and edges drawn from the project's parent/child links. Each node is linked to its task, so the two stay in sync.

Related: This is the reverse of promoting a graph to tasks. An agent uses `blueprint_generate_from_bam`.

Story: Turn a mind map into a project plan

Who: A planner who brainstormed work as a mind map and now wants real tasks. **Goal:** Promote the diagram into a Bam project, one task per node, with parent links from the edges. **Before you start:** Have a diagram of the work and a target Bam project id.

Steps

1. Open the diagram in the editor.
2. Make sure the structure reads parent-to-child along the edges (an edge from the parent idea to the child idea).
3. Click **Promote to Bam** in the top bar.
4. If prompted, enter the Bam project id (a UUID).
5. When prompted for edge direction, choose `source-parent` so each edge makes the source node the parent.
6. Watch the progress overlay until it reports the created tasks and wired parent links.

Result: One Bam task per node, with parent and child relationships matching the edges. Each node is now linked to its task, and the badges appear on the canvas.

Related: Promote a single node instead with **Promote to task** in the Inspector. An agent uses `blueprint_promote_graph_to_tasks` and then creates the tasks with Bam tools.

Story: Keep a diagram and its tasks in sync

Who: Anyone maintaining a diagram that mirrors live work in Bam. **Goal:** Confirm that edits flow both directions between a node and its linked task. **Before you start:** Have a diagram whose nodes are linked to Bam tasks (from generation or promotion).

Steps

1. Open the diagram in the editor.
2. Select a linked node (it shows a link badge) and change its **Label** in the Inspector.
3. Open the linked task in Bam and confirm its title now matches.
4. In Bam, rename the task.
5. Back in Blueprint, reload the diagram (or wait for the live refresh) and confirm the node label updated.

Result: The node and the task hold the same title and description, kept in sync automatically.

Related: Link more nodes with the Inspector **Link** button or the node menu's **Link to entity…**. An agent uses `blueprint_link_entity` and `blueprint_update_node`.

Story: Reorganize a messy graph

Who: Anyone whose diagram has drifted into a tangle. **Goal:** Auto-arrange the graph while keeping a few key nodes fixed. **Before you start:** Have a diagram open in the editor.

Steps

1. Pin the nodes you want to keep in place: select each one and click the Pin toggle in the Inspector, or right-click and choose **Pin position**.
2. In the layout dropdown, choose an algorithm: **Layered** or **Force-directed** for edge-aware arrangements, **Tree** for a hierarchy, or **Grid** for a compact packing.
3. Pick a direction and click **Apply layout**.
4. If you want everything aligned to a grid afterward, right-click empty canvas and choose **Snap nodes to grid now**.

Result: The unpinned nodes are arranged by the chosen algorithm; the pinned nodes stay where you put them.

Related: For a radial layout, use an agent with `blueprint_apply_layout`, since the UI dropdown does not offer it.

Story: Have an agent draft a diagram from a document

Who: A user working with an AI agent that has MCP access. **Goal:** Produce a complete, laid-out diagram from a written process description without drawing it by hand. **Before you start:** Have an agent connected to the MCP server with a key allowed to use `blueprint.*` tools.

Steps

1. Give the agent the source text (a process description, a roster, or a task list) and ask it to build a Blueprint diagram.
2. The agent calls `blueprint_create` to make the diagram.
3. The agent calls `blueprint_generate` with node specs and edge specs extracted from your text. The server creates the nodes, wires the edges and any parent containers, and runs auto-layout. (If the source is already a Mermaid block, the agent can use `blueprint_import_mermaid` instead, then `blueprint_apply_layout`.)
4. Open the diagram from `/blueprint/` to review it.
5. Adjust labels, shapes, and layout in the editor as needed.

Result: A fully wired, laid-out diagram you can refine by hand, created in a single agent call.

Related: The agent can then call `blueprint_promote_graph_to_tasks` to turn the diagram into a Bam plan. Every agent mutation is audited like a human edit.

Story: Review and snapshot before an agent rewrite

Who: A diagram owner who wants a safe restore point before an agent reworks the graph. **Goal:** Capture the current graph, let the agent rewrite it, and roll back if needed. **Before you start:** Have a diagram open, and an agent with `blueprint.*` access.

Steps

1. In the editor, click **Save snapshot** and label it (for example "before agent rewrite").
2. Ask the agent to make its changes. It can read the graph with `blueprint_read_nodes` and `blueprint_read_edges`, and rewrite with `blueprint_generate` (`replace: true`) or incremental node/edge tools.
3. Have the agent add a comment summarizing what it changed with `blueprint_add_comment`, and read review feedback with `blueprint_list_comments`.
4. If the result is wrong, ask the agent to restore the labeled snapshot with `blueprint_restore_version` (it lists versions with `blueprint_list_versions` first).

Result: The diagram is either the agent's improved version or rolled back to your snapshot, with the review thread recorded.

Related: Listing and restoring versions, and reading comments, are also in the editor's **History** and **Comments** panels; an agent does the same with `blueprint_list_versions`, `blueprint_restore_version`, and `blueprint_list_comments`.

Story: Curate your diagram library

Who: Anyone keeping a tidy set of diagrams. **Goal:** Star the diagrams you use most and archive the ones you no longer need. **Before you start:** Have several diagrams in your org.

Steps

1. Go to `/blueprint/`.
2. Click the **Star** toggle on the cards you want to keep handy.
3. Click **Starred** in the sidebar to see only those.
4. For a stale diagram, open its More menu and choose **Archive**, then confirm.
5. Click **Archived** in the sidebar to review or find archived diagrams later.

Result: Your most-used diagrams are one filter click away, and stale ones are hidden from the default list without being destroyed.

Related: Archiving is reversible at the data level since nodes, edges, comments, and snapshots are preserved. An agent archives with `blueprint_archive` (which requires admin scope and a confirmation step).

Related

- **Bam** (`/b3/`) - the Kanban and sprint tool Blueprint exchanges work with. Generate a diagram from a Bam project, promote a diagram or node into Bam tasks, and rely on two-way sync between a node and its linked task.
- **Beacon, Bearing, Bond, Helpdesk** - nodes can carry cross-product links to a Beacon entry, a Bearing goal, a Bond deal or contact, or a Helpdesk ticket via the Inspector's **Linked entity** section or the node menu's **Link to entity...**.
- **Brief** - Mermaid export from Blueprint produces a source string you can embed elsewhere, and `blueprint_import_mermaid` turns a Mermaid block back into an editable graph.
- Blueprint MCP-tools reference and guide in `docs/apps/blueprint/` for the full catalog of the 36 agent tools.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. How does Blueprint store a diagram?

a) As a single flat image file
b) As an opaque collaborative drawing blob
c) As a typed relational graph of node and edge rows
d) As a Mermaid source string only

2. Which shapes does the node palette offer?

a) Rounded, Rectangle, Diamond, Ellipse, and Hexagon
b) Circle, Square, Triangle, and Star
c) Box, Pill, and Cloud
d) Only Rectangle and Ellipse

3. Which cross-product link type stays in live two-way sync?

a) beacon.entry
b) bond.deal
c) bam.task
d) bearing.goal

4. What are the five edge kinds you can choose?

a) Default, Dependency, Flow, Reference, and Inherits
b) Solid, Dashed, Dotted, Curved, and Straight
c) Parent, Child, Sibling, Peer, and Root
d) Up, Down, Left, Right, and None

5. What type does Generate from Bam write onto the new diagram?

a) flowchart
b) graph
c) mind_map
d) org_chart

6. Which export formats are actually offered in the editor?

a) SVG and PNG
b) Mermaid (.mmd) and JSON
c) PDF and DOCX
d) CSV and XML

7. What does it mean to pin a node, and what effect does pinning have during auto-layout?

8. Name Blueprint's three visibility levels and who can see a diagram at each.

9. When you promote a whole graph to Bam tasks, what does the source-parent edge direction mean?

10. What are the four collaborator roles available in the People panel?

11. How is each snapshot numbered, and what does a snapshot capture?

12. When an agent calls blueprint_generate, how many node specs and edge specs can it pass?

CHAPTER 11

Board

An infinite canvas your whole team can draw on.

Board is where you think in space instead of in lists. It gives you an infinite Excalidraw canvas for sketches, retros, architecture maps, and brainstorms, with sticky notes, shapes, arrows, and frames that auto-save and sync live as your team draws together. This chapter walks you from your first template through real-time collaboration, version snapshots, and locking, and it shows the marquee handoff where stickies become tracked Bam tasks. By the end you will be able to spin up a board, run a live session on it, and turn the ideas into work.

At a glance

- Infinite Excalidraw canvas: stickies, text, shapes, arrows, frames, and images
- Live collaboration with shared cursors and auto-save as you draw
- Named version snapshots you can restore at any time
- Promote brainstorm stickies into real Bam tasks in a named project and phase
- 40 MCP tools, so an agent can read, build, and tidy a board with your permissions

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Board list (All Boards) . Creating a board from a template . Creating a blank board . The infinite canvas . Adding sticky notes . Adding text

> *Board gives your team an infinite canvas for sketching ideas, running retros, mapping architecture, and brainstorming together. Reach for it when a list or a document is not enough and you need to draw, group, and arrange thoughts in space.*

Overview

Board[99] is BigBlueBam's whiteboarding app. Each board is one infinite canvas powered by Excalidraw, where you place sticky notes, text, shapes, arrows, freehand drawings, frames, and images. Boards auto-save as you work and sync in real time, so several people can edit the same canvas at once and see each other's cursors.

> ***One infinite canvas, many cursors. The whole team draws together and sees each other move in real time.***

Boards live inside your BigBlueBam organization and can optionally be scoped to a Bam project. Because projects, project members, phases, and tasks all live in Bam, Board can hand work back to Bam: sticky notes from a brainstorm can be promoted into Bam tasks in a named project and phase. You organize boards with stars, archive, templates, and named version[100] snapshots.

Board does not have its own login. It reads your shared BigBlueBam session, and your permissions come from Bam. If you are not signed in, Board shows a "Please log in to BigBlueBam first" screen with a link to the main app.

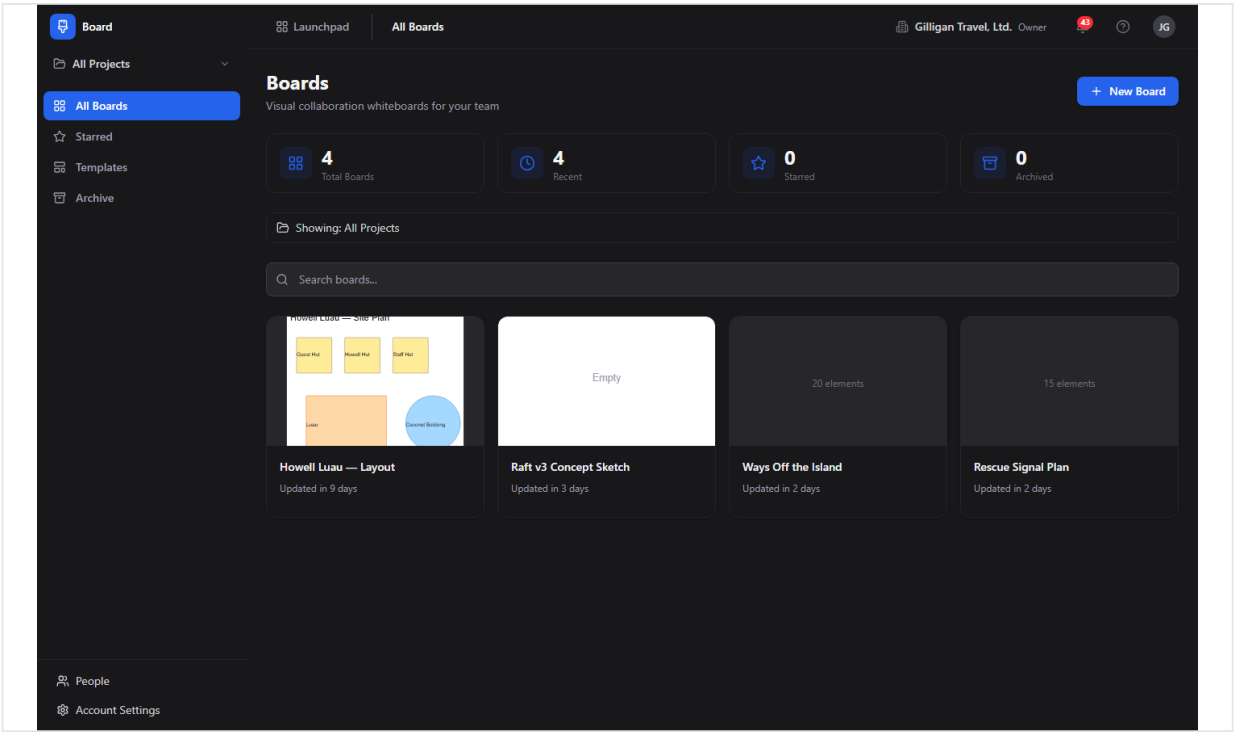


Figure 11.1 All boards

[99] **Board.** one infinite-canvas whiteboard. It has a name, a description, an emoji icon, a background pattern, a visibility setting, an optional project association, and a locked flag.
[100] **Version.** a named snapshot of a board's canvas that you can restore later.

Key concepts

- **Board** - one infinite-canvas whiteboard. It has a name, a description, an emoji icon, a background[101] pattern, a visibility[102] setting, an optional project association, and a locked flag.
- **Element**[103] - anything on the canvas: a sticky note[104], text, a shape[105], a connector (arrow or line), an image, or a frame[106]. The canvas scene is stored whole, and a per-element copy is kept so the board can be searched, read, and promoted.
- **Sticky note** - a colored note. The default color is yellow. (Excalidraw has no native sticky; Board builds one from a rectangle with bound text.)
- **Shape** - a rectangle, diamond, ellipse, or freehand drawing from Excalidraw's toolbar.
- **Frame** - a labeled region on the canvas that groups the elements inside it. Frames are used when reading or summarizing a board by section.
- **Background** - the canvas backdrop pattern. One of `dots` (default), `grid`, `lines`, or `plain`.
- **Visibility** - who can see a board. `private` (creator plus explicitly added collaborators), `project` (project members plus collaborators plus creator), or `organization` (every member of your org). The default is `project`.
- **Collaborator**[107] - a person granted access to a single board with either `view` or `edit` permission. There is no in-app screen to manage collaborators today; manage them through the visibility setting or have an agent or API client do it (see Feature reference).
- **Lock**[108] - a board-level read-only switch. When a board is locked, only the board creator or an org admin or owner can edit it.
- **Star**[109] - your personal favorite marker on a board.
- **Version** - a named snapshot of a board's canvas that you can restore later.

---

[101] **Background.** the canvas backdrop pattern. One of `dots` (default), `grid`, `lines`, or `plain`.
[102] **Visibility.** who can see a board. `private` (creator plus explicitly added collaborators), `project` (project members plus collaborators plus creator), or `organization` (every member of your org). The default is `project`.
[103] **Element.** anything on the canvas: a sticky note, text, a shape, a connector (arrow or line), an image, or a frame. The canvas scene is stored whole, and a per-element copy is kept so the board can be searched, read, and promoted.
[104] **Sticky note.** a colored note. The default color is yellow. (Excalidraw has no native sticky; Board builds one from a rectangle with bound text.)
[105] **Shape.** a rectangle, diamond, ellipse, or freehand drawing from Excalidraw's toolbar.
[106] **Frame.** a labeled region on the canvas that groups the elements inside it. Frames are used when reading or summarizing a board by section.
[107] **Collaborator.** a person granted access to a single board with either `view` or `edit` permission. There is no in-app screen to manage collaborators today; manage them through the visibility setting or have an agent or API client do it (see Feature reference).
[108] **Lock.** a board-level read-only switch. When a board is locked, only the board creator or an org admin or owner can edit it.
[109] **Star.** your personal favorite marker on a board.

- **Template**[110] - a reusable board blueprint you start new boards from. System templates ship with the platform; your org can also create its own.
- **Integrity issue**[111] - a flag on a board whose project link is broken (cross-org, missing, or auto-detached). You resolve it by detaching the board from the project or reassigning it to a valid one.
- **Roles**[112] - org roles resolve as viewer, member, admin, owner. Admins and owners get full access to any board in the org.

Where to find it

Board is served at `/board/`. A single board's canvas is at `/board/<boardId>`, and its version history is at `/board/<boardId>/versions`.

> **TRY IT**
> Open /board/, click New Board, pick a Retro template, and click Use. You will land on a pre-filled canvas with no layout work.

Prerequisites:

- A signed-in BigBlueBam session. Board uses the shared Bam session cookie.
- An organization. To scope a board to a project, that project must exist in Bam and belong to your org.
- Edit access to a board to change it. Locked boards are editable only by the creator or an org admin or owner.

The left sidebar has four nav items: **All Boards**, **Starred**, **Templates**, and **Archive**. A project scope selector at the top of the sidebar lets you switch between "All Projects" and a single project to filter the list and stats. Press `?` anywhere outside a text field to open the in-app help.

[110] **Template.** a reusable board blueprint you start new boards from. System templates ship with the platform; your org can also create its own.
[111] **Integrity issue.** a flag on a board whose project link is broken (cross-org, missing, or auto-detached). You resolve it by detaching the board from the project or reassigning it to a valid one.
[112] **Roles.** org roles resolve as viewer, member, admin, owner. Admins and owners get full access to any board in the org.

Feature reference

Board list (All Boards)

The home view lists your boards as cards. It shows a heading "Boards" and the subtitle "Visual collaboration whiteboards for your team".

The page has four stat cards across the top: **Total Boards**, **Recent**, **Starred**, and **Archived**. Below them is a search box and a grid of board cards.

To browse and open a board:

1. Open Board at `/board/`. **All Boards** is the default view.
2. To narrow the list to one project, use the project scope selector at the top of the sidebar. A banner shows "Showing: All Projects" or "Showing: <Project>", with a **Show All Projects** reset.
3. Type in the **Search boards…** box to filter by board name or description.
4. Click a board card to open its canvas.

Each board card shows the board's thumbnail or icon, an element-count caption ("N elements" or "Empty board"), a project badge, and a collaborator-count chip ("N collaborator(s)"). A lock icon appears on locked boards. An amber warning icon appears when a board has an integrity issue, with the tooltip "N integrity issue(s). Open the board to fix."

If there are no boards, the empty state reads "No boards yet" (or "No boards match your search"), with a **Create Board** button that takes you to the Templates browser.

The **…** menu on each card holds **Version history**, **Duplicate**, **Archive**, and **Delete**. See the lifecycle and version sections below.

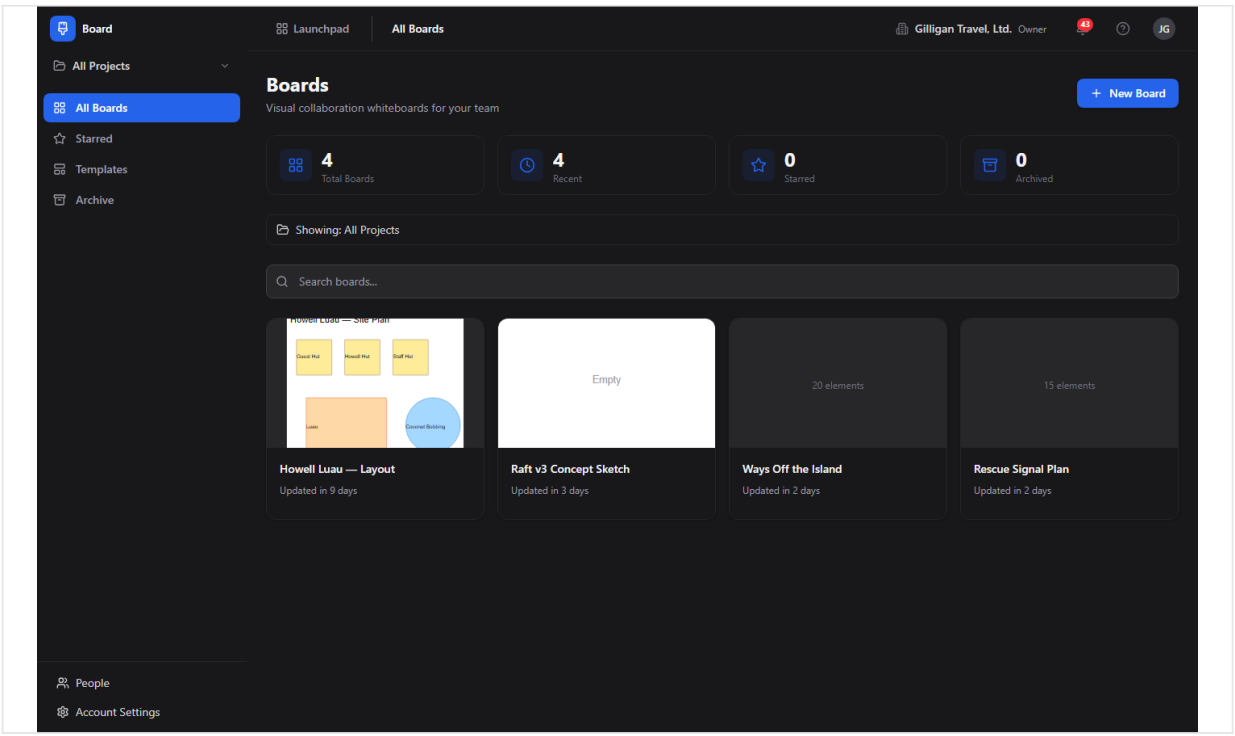


Figure 11.2 All boards

Creating a board from a template

Templates give you a pre-built canvas (a retro layout, a brainstorm board, an architecture template, and so on).

To create a board from a template:

1. From **All Boards**, click **New Board** (top-right), or click **Templates** in the sidebar. Both lead to the Templates browser.
2. The browser shows a heading "Templates" and the subtitle "Start from a pre-built template, or create a blank board".
3. Use the category tabs to filter: **General**, **Retro**, **Brainstorm**, **Planning**, **Architecture**, **Strategy**, or **All**.
4. On the template you want, click **Use**.
5. A new board is created from that template and its canvas opens.

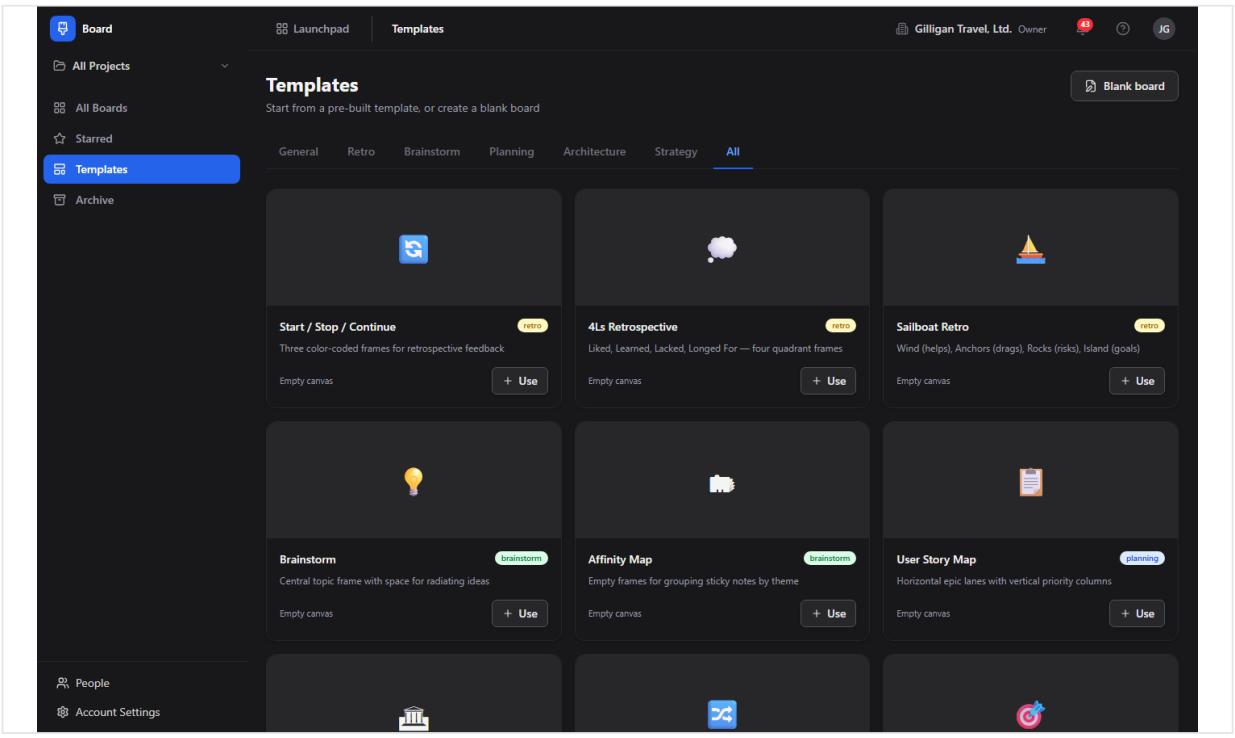


Figure 11.3 Board templates

Creating a blank board

To start from an empty canvas:

1. In the Templates browser, click **Blank board** (top-right), which opens the "Create New Board" page. (You can also pick the **Blank Board** card, captioned "Start with an empty canvas", on that page.)
2. The page shows a heading "Create New Board".
3. Pick an emoji with the **Icon** picker.
4. Type a name in the **Board name** field (placeholder "Untitled Board").
5. Click **Create Board**. To abandon, click **Cancel**.
6. The new board's canvas opens.

The infinite canvas

The canvas is a full-screen Excalidraw editor. It opens when you click a board card or finish creating a board.

The drawing tools are Excalidraw's own native toolbar, not Board's. From that toolbar you can add and edit shapes (rectangle, diamond, ellipse), arrows and lines, text, freehand drawings, frames, and images, plus zoom, pan, and multi-select. Use Excalidraw's toolbar for everything you draw.

Board overlays a few of its own controls on top of the canvas:

- A floating **board toolbar** at the top (covered below).

- A **connection status badge** at the top-right.
- An **integrity banner** at the top when the board's project link is broken.
- A **chat panel** that slides in from the right.

Your work saves automatically. The canvas persists locally as you draw and writes to the server a few seconds after you stop, and it saves once more when you close the tab. You do not need a Save button for canvas content.

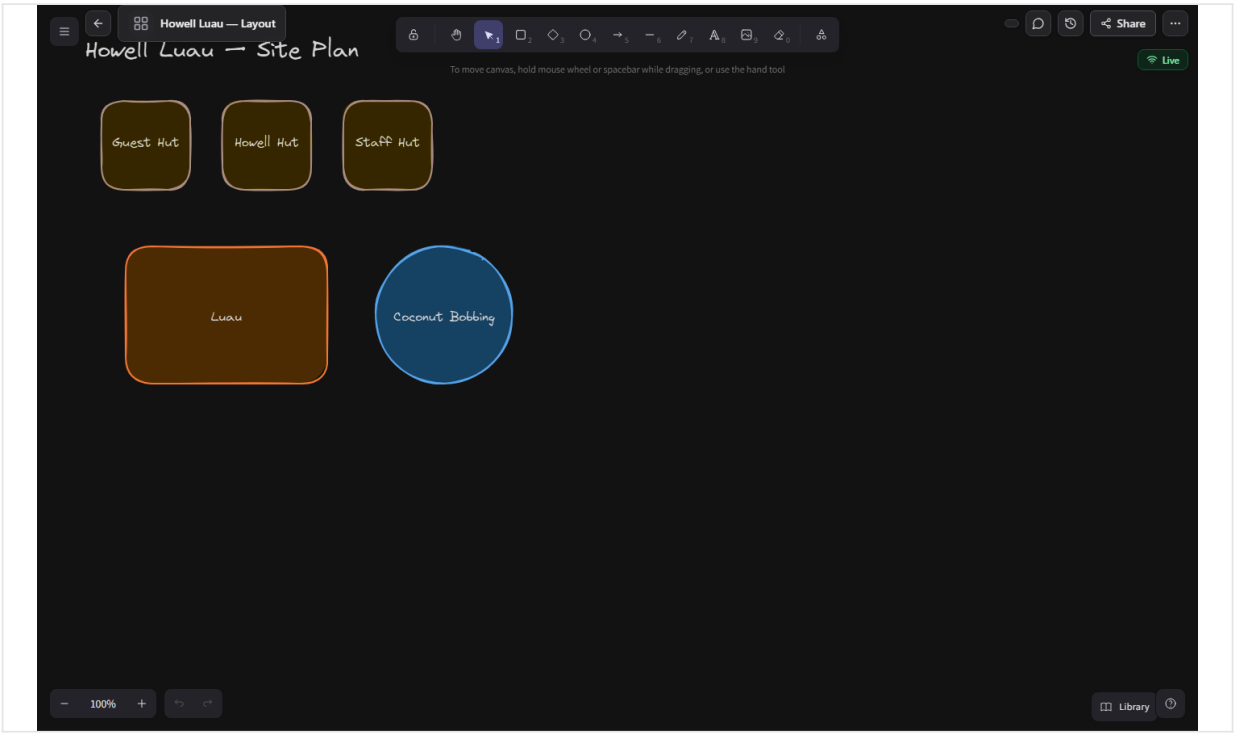


Figure 11.4 Board canvas

Adding sticky notes

Sticky notes are colored notes for capturing ideas. The default color is yellow.

Add stickies from Excalidraw's tools on the canvas (a rectangle with text). An AI agent can also add a sticky directly with one of six colors (yellow, green, blue, red, purple, orange) and a position; see Working with AI agents.

Adding text

Add free text from Excalidraw's text tool on the canvas. An agent can add a text element with a position; see Working with AI agents.

Shapes, arrows, and frames

Rectangles, diamonds, ellipses, arrows, lines, freehand strokes, images, and frames all come from Excalidraw's native toolbar. Group related elements inside a frame so the board can later be read or summarized by section.

The board toolbar

The floating toolbar at the top of the canvas holds board-level controls (it is separate from Excalidraw's drawing toolbar):

- A **back arrow** (tooltip "Back to boards") that returns to **All Boards**.
- An **icon picker** and the board name. Click the name to rename the board inline; press Enter to save or Escape to cancel.
- A lock icon, shown when the board is locked.
- The **presence strip**, which shows who else is present on the board and is the entry point to a voice huddle (see Real-time presence and audio).
- A **Toggle chat** button that opens the chat panel.
- A **Version history** button that opens the board's versions.
- A **Share** button that copies a shareable link to the board to your clipboard. The button briefly reads "Link copied" after a successful copy.
- A **…** menu containing **Lock board** / **Unlock board**, and **Export as PNG** and **Export as SVG**. Each export item renders the board on the server and downloads a real image file.

To rename a board from the toolbar:

1. Open the board's canvas.
2. Click the board name in the floating toolbar.
3. Type the new name.
4. Press Enter to save (or Escape to cancel).

To copy a shareable link:

1. Open the board's canvas.
2. Click the **Share** button in the floating toolbar.
3. The board's link is copied to your clipboard and the button shows "Link copied".
4. Paste the link to anyone who has access to the board. Access is still governed by the board's visibility and collaborators, so the link opens for people who are already allowed to see it.

Locking a board

Locking makes a board read-only for everyone except its creator and org admins and owners. Use it to freeze a finished board.

To lock or unlock a board:

1. Open the board's canvas.
2. In the floating toolbar, open the **…** menu.
3. Click **Lock board** (or **Unlock board** if it is already locked).
4. A locked board shows a lock icon in the toolbar and on its card in the list.

Chat

Each board has a side chat panel for discussion while you whiteboard.

To chat:

1. Open the board's canvas.
2. Click the **Toggle chat** button in the floating toolbar to open the chat panel (header "Chat").
3. Type in the **Type a message…** box and send.
4. Your messages appear right-aligned; others' messages appear on the left.

Note: chat is not pushed live over the realtime connection. The panel refreshes by polling, so new messages from others appear after the next refresh rather than the instant they are sent. The chat shows up to the last 100 messages.

> **NOTE**
> Board chat is the one part of a board that is not real time. The panel refreshes by polling, so a teammate's message shows up on the next refresh rather than the instant it is sent, and the panel keeps the last 100 messages.

Version history (snapshots)

A version is a named snapshot of the whole canvas. Save a version before a big change so you can roll back.

To save a version:

1. Open the board's canvas and click the **Version history** button in the floating toolbar (or open the **…** menu on the board card and choose **Version history**). This opens the "Version History" page.
2. Click **Save Version**.
3. In the dialog, type a **Version name**.
4. Confirm to create the snapshot.

The version list shows each version's name, a "Latest" chip on the newest one, the creator, and a relative time. The newest snapshot is at the top.

To restore a version:

1. Open the "Version History" page for the board.
2. Find the version you want and click **Restore**.
3. Confirm at the "Restore this version?" prompt.
4. The board's canvas is replaced with that snapshot.

Note: the Save Version dialog has a **Description (optional)** field, and the list is laid out to show a description, an element count, and the creator's name. The backend does not store the description and does not return an element count or creator name on versions today, so those fields render blank. Only the version name, the creator reference, and the timestamp are saved and shown reliably.

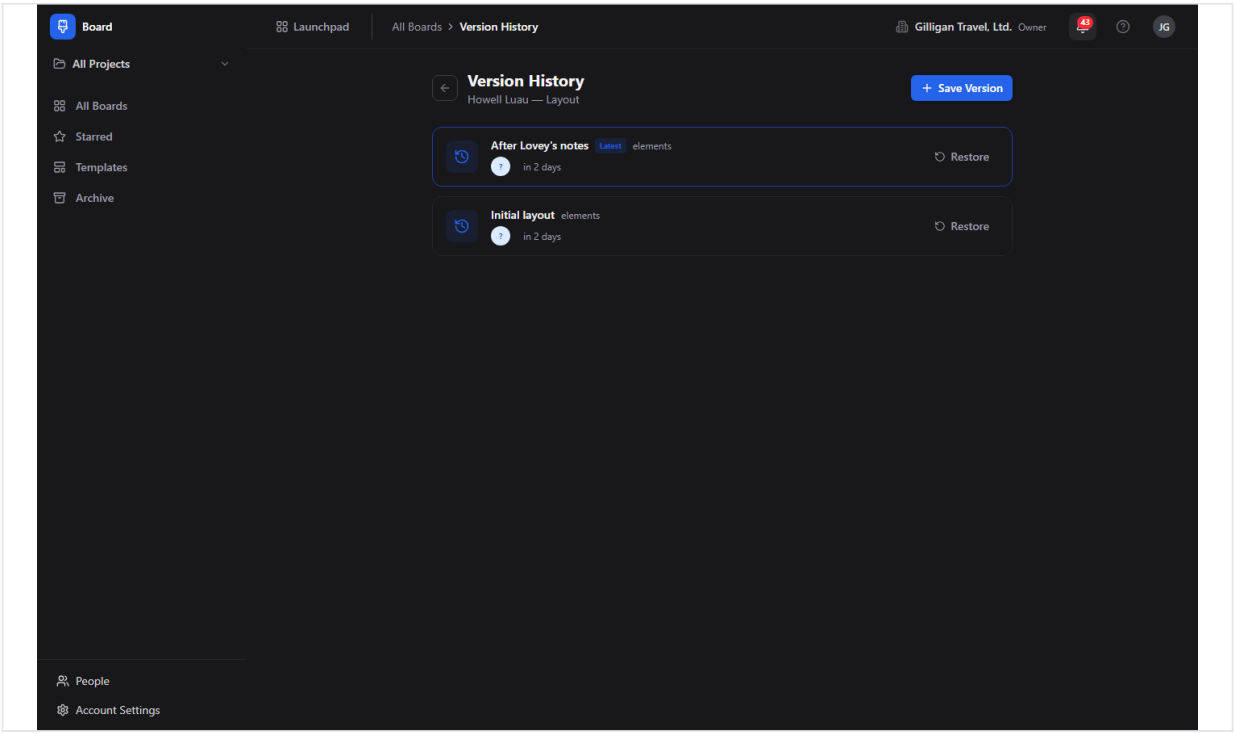


Figure 11.5 Version history

Archive, restore, and delete

Archiving hides a board without destroying it. Deleting is permanent.

To archive a board:

1. On **All Boards**, open the **...** menu on the board's card.
2. Click **Archive**. The board moves to the **Archive** view.

To view and restore archived boards:

1. Click **Archive** in the sidebar. The heading reads "Archive".
2. On an archived board's card, click **Restore** to bring it back, or **Delete permanently** to remove it for good.
3. If the archive is empty, you see "Archive is empty".

To delete a board permanently from the list:

1. On a board's **...** menu, click **Delete**.
2. Confirm in the "Delete board?" dialog. This permanently removes the board and its elements, collaborators, stars, versions, and links. This cannot be undone.

Duplicating a board

Duplicate copies a board and all its elements into a new board.

To duplicate:

1. On **All Boards**, open the **...** menu on the board's card.

2. Click **Duplicate**.
3. A copy is created, named "<original name> (copy)".

Starring a board

Stars are your personal favorites.

To star a board:

1. On a board card in **All Boards**, click the star overlay on the card to toggle it.
2. Open **Starred** in the sidebar to see all your starred boards (heading "Starred Boards"). If you have none, the view reads "No starred boards".

Templates browser

Beyond creating boards, the Templates browser is where org templates are surfaced. Each template card shows a thumbnail or icon, a name, a description, a category badge, an element count, and a **Use** button. Categories shown as tabs are **General**, **Retro**, **Brainstorm**, **Planning**, **Architecture**, **Strategy**, and **All**. If there are none, the view reads "No templates available". System templates are read-only; your org can create and manage its own (through the API or an agent today; see Working with AI agents).

Fixing a board with an integrity issue

A board can end up with a broken project link (the project is in another org, was deleted, or was auto-detached). When that happens, an amber banner appears at the top of the canvas.

To fix it:

1. Open the affected board (its card shows an amber warning icon in **All Boards**).
2. Read the message in the amber integrity banner at the top of the canvas.
3. To remove the broken link, click **Detach from project** (also labeled **Leave unattached**).
4. To point the board at a valid project, click **Reassign to a project**, pick a project from the radio list in the "Reassign to a project" dialog, and confirm.

Export

A board's canvas can be exported as JSON, SVG, or PNG.

To export an image from the canvas:

1. Open the board's canvas.
2. In the floating toolbar, open the **…** menu.
3. Click **Export as PNG** or **Export as SVG**. The menu item briefly reads "Exporting…" while the server renders the board.
4. A real image file downloads, named after the board.

Export is also available to AI agents: an agent renders SVG or PNG with the `board_export` tool, and the same server endpoint (`GET /boards/:id/export/:format`) can be called directly through the API.

Collaborators (no in-app management today)

A board's visibility (`private`, `project`, `organization`) controls who can reach it, and a board can also have explicit per-person collaborators with `view` or `edit` permission. There is a complete backend for adding, listing, changing, and removing collaborators, but there is no screen in the Board app to manage them today. The toolbar's **Share** button copies the board's link rather than opening a collaborator manager. Until a sharing screen ships, control who can see a board through its **visibility** setting, or have an agent or API client manage collaborators directly (the `board_add_collaborator`, `board_list_collaborators`, `board_update_collaborator`, and `board_remove_collaborator` tools, covered under Working with AI agents).

Real-time presence and audio

When more than one person is on a board, you see each other's live cursors on the canvas and presence indicators in the toolbar. The connection status badge at the top-right shows **Live** (green), **Connecting** (amber), or **Offline** (red), plus an "N editors" chip when other people are present. If your connection drops and reconnects, recent changes are replayed so the canvas catches up.

Audio for a board comes from the platform-wide presence system that the toolbar surfaces through its presence strip, not from a board-api capability. The strip reports the open canvas as a shared surface, and the platform call manager derives a per-board voice room from it, so a teammate can join a voice huddle on the same canvas. Audio conferencing is therefore a platform feature layered on top of Board, not something the Board service hosts itself.

Element limits

A board can hold a large number of elements, with a soft limit at 500 and a hard limit at 2000 live elements. Past the soft limit you get a warning; at the hard limit further additions are rejected. Deleted elements do not count.

Working with AI agents

Agents reach Board through 40 MCP tools backed by board-api. Every tool forwards your bearer token, so an agent acts with your permissions and a locked board stays locked. Most tools accept a board **name or UUID**; templates, projects, and phases also resolve by name. For the full catalog and arguments, see the Board MCP-tools reference in `docs/apps/board/`.

> **TIP**
> Tools resolve a board by name or UUID, and templates, projects, and phases resolve by name too. So an agent can act on "Q3 Retro" without you ever copying an id, and every tool forwards your bearer token so a locked board stays locked.

What agents commonly do:

- **Read and observe a canvas.** `board_read_elements` reads every element with its position, text, and type. `board_read_stickies` returns only sticky notes; `board_read_frames` returns frames with their child elements; `board_summarize` produces a frame-grouped summary. `board_search` searches element text across boards. `board_list`, `board_get`, `board_list_recent`, `board_list_starred`, `board_stats`, and `board_org_stats` cover discovery and metadata.
- **Build a board.** `board_create` makes a board (optionally from a template by name). `board_add_sticky` drops a sticky in one of six colors at a position; `board_add_text` adds a text element. `board_update` patches name, description, background, visibility, and icon.
- **Promote to Bam tasks (the marquee cross-app flow).** `board_promote_to_tasks` turns selected brainstorm stickies into Bam tasks in a named project and phase, and records a link from each sticky back to its task. There is no human button for this today; it is an agent-and-API flow. `board_list_links` shows the resulting links and `board_delete_link` removes one without touching the task or sticky.
- **Manage what humans cannot reach in the UI yet.** Collaborators (`board_add_collaborator`, `board_list_collaborators`, `board_update_collaborator`, `board_remove_collaborator`), templates (`board_list_templates`, `board_create_template`, `board_update_template`, `board_delete_template`, `board_instantiate_template`), versions (`board_list_versions`, `board_create_version`, `board_restore_version`), chat (`board_read_chat`, `board_post_chat`), stars (`board_star_toggle`), and integrity (`board_check_integrity`, `board_remediate_integrity`) all have tools even where Board has no screen.
- **Run lifecycle and export.** `board_duplicate`, `board_archive` (soft delete), `board_restore`, and `board_delete_permanent` (hard, irreversible delete) cover lifecycle. `board_export` exports SVG or PNG, the same render the canvas export menu downloads.

> ***Promote-to-tasks is the marquee handoff: a sticky from a brainstorm becomes a tracked Bam task, linked back to where it started.***

Things for a human reviewer to know:

- Promote-to-tasks creates real Bam tasks. Review the resulting tasks in the target project after an agent runs it.
- `board_delete_permanent` cascades through elements, collaborators, stars, versions, and links and cannot be undone. Prefer `board_archive` unless removal is final.
- Two MCP behaviors are known to drift from the backend and may not do what their description implies: `board_update` cannot toggle the lock state (its PATCH does not accept a `locked` field, so locking stays a separate action with no MCP tool of its own),

and a few tool descriptions and return shapes do not exactly match the live API (for example `board_create` lists different background and visibility defaults than the server applies, which are `dots` and `project`). Prefer the dedicated paths and verify results.

- Right after a fresh board is drawn, the per-element copy that powers search and promote may lag the canvas by a few seconds. If `board_search` or a promote finds nothing on a brand-new board, give the snapshot a moment.

> **WARNING**
> Deleting a board permanently removes its elements, collaborators, stars, versions, and links, and it cannot be undone. Archive instead unless the removal is truly final, since an archived board can always be restored.

Agent actions emit Bolt events you can automate on: `board.created`, `board.updated`, `board.locked`, and `board.elements_promoted`.

Cross-cutting agentic platform

Board's tools sit on the same platform surface every app shares, so agent work is identifiable, reviewable, and governed:

- **Identity and heartbeat.** Agent and service accounts are distinct from human users, and their actions are tagged as such in the activity log. Long-running agents report liveness with `agent_heartbeat`.
- **Approvals.** An agent can route a board action it should not take unilaterally (for example a permanent delete or a large promote-to-tasks run) into an approval queue with `proposal_create`; a human reviews and resolves it with `proposal_list` and `proposal_decide`.
- **Unified activity.** Board activity is queryable alongside every other app through the unified activity view and `activity_query` / `activity_by_actor`.
- **Visibility preflight.** Before an agent posts a board's contents into another shared surface, it calls `can_access` for each cited entity and drops anything the asking user is not allowed to see.
- **Policies and webhooks.** Per-agent kill switches and tool allowlists (a `board.*` glob covers Board's tools) gate every service-account call, and subscribed Bolt events can be pushed to an agent runner over a signed outbound webhook.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. The canvas shows who else is on the board, edits and cursors sync live, and the board carries audio conferencing so you can talk while you sketch together. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Create your first board from a template

Who: A team member new to Board. **Goal:** Get a working whiteboard started without setting up a layout by hand. **Before you start:** Be signed in to BigBlueBam.

Steps

1. Go to `/board/`. You land on **All Boards**.
2. Click **New Board** at the top-right. The Templates browser opens.
3. Use the category tabs (**General**, **Retro**, **Brainstorm**, **Planning**, **Architecture**, **Strategy**, **All**) to find a fitting layout.
4. On the template you want, click **Use**.
5. The new board opens on the canvas.

Result: A new board exists, pre-filled from the template, and you are looking at its canvas. It appears in **All Boards**.

Related: To start empty instead, see "Create a blank board". An agent can do the same with `board_create` (template by name) or `board_instantiate_template`.

Story: Create a blank board

Who: Anyone who wants an empty canvas. **Goal:** Start a whiteboard from scratch with a name and icon. **Before you start:** Be signed in.

Steps

1. Go to `/board/` and click **Templates** in the sidebar (or **New Board** from the list).
2. Click **Blank board** at the top-right of the Templates browser.
3. On the "Create New Board" page, pick an emoji with the **Icon** picker.
4. Type a name in **Board name** (placeholder "Untitled Board").
5. Click **Create Board**.

Result: An empty board opens on the canvas, ready to draw on.

Related: `board_create` does this for an agent.

Story: Brainstorm on the canvas

Who: A facilitator or contributor running an ideation session. **Goal:** Capture ideas as sticky notes, text, shapes, and arrows on one canvas. **Before you start:** Have a board open and edit access to it.

Steps

1. Open the board's canvas.

2. Use Excalidraw's native toolbar to add sticky notes, text, shapes, arrows, freehand drawings, and frames.
3. Drag elements to arrange them; group related ideas inside a frame.
4. Keep working. The canvas auto-saves as you go and again when you close the tab.

Result: Your ideas are captured on the board and saved automatically.

Related: An agent can seed a board with `board_add_sticky` and `board_add_text`, then group thinking with frames for `board_summarize`.

Story: Collaborate live with teammates

Who: Two or more people working a board at the same time. **Goal:** Edit together and see each other's changes and cursors. **Before you start:** Each person needs access to the same board.

Steps

1. Each person opens the same board's canvas.
2. Watch the connection status badge at the top-right for **Live**, and the "N editors" chip for who is present.
3. Add and move elements. Changes from others appear on your canvas in real time, and you see their cursors.
4. If your connection drops, wait for it to reconnect; recent changes are replayed automatically.

Result: Everyone is editing one shared canvas and stays in sync.

Related: To bring people in, copy the board link with the **Share** button (see "Share a board link"). To talk while you work, use the presence strip in the toolbar to start a voice huddle on the canvas. To type instead, see "Chat while you whiteboard".

Story: Share a board link

Who: Anyone who wants a teammate on a board. **Goal:** Hand someone a direct link to the board. **Before you start:** Have the board open. The recipient must already be allowed to see the board (through its visibility or as a collaborator).

Steps

1. Open the board's canvas.
2. Click the **Share** button in the floating toolbar.
3. The board's link is copied to your clipboard and the button shows "Link copied".
4. Paste the link into chat, email, or wherever your teammate will find it.

Result: Your teammate has a direct link that opens the board's canvas, subject to the board's visibility and collaborator rules.

Related: There is no in-app collaborator manager yet; widen who can open the link through the board's **visibility** setting, or have an agent add explicit collaborators with `board_add_collaborator`.

Story: Chat while you whiteboard

Who: Collaborators on a board. **Goal:** Discuss the board without leaving it. **Before you start:** Have the board open and edit access.

Steps

1. Open the board's canvas.
2. Click **Toggle chat** in the floating toolbar to open the chat panel.
3. Type in **Type a message…** and send.
4. Read the thread in the panel; your messages are right-aligned.

Result: The team has a running discussion attached to the board.

Related: Chat refreshes by polling, so messages from others appear on the next refresh rather than instantly. An agent can read or post with `board_read_chat` and `board_post_chat`.

Story: Star and find your boards

Who: A regular Board user with many boards. **Goal:** Keep important boards one click away and filter the list. **Before you start:** Have at least one board.

Steps

1. On **All Boards**, click the star overlay on a board card to favorite it.
2. Click **Starred** in the sidebar to see only starred boards.
3. Back on **All Boards**, use the project scope selector to filter by project.
4. Use the **Search boards…** box to find a board by name or description.

Result: Your favorites are grouped in **Starred**, and you can filter and search the full list.

Related: An agent can list boards with `board_list`, list starred with `board_list_starred`, toggle a star with `board_star_toggle`, and search element text with `board_search`.

Story: Snapshot and restore a board

Who: Anyone about to make a big change. **Goal:** Save a restore point and roll back if needed. **Before you start:** Have edit access to the board.

Steps

1. Open the board and click the **Version history** button in the floating toolbar.
2. On the "Version History" page, click **Save Version**.
3. Type a **Version name** and confirm.
4. Later, to roll back, return to "Version History", click **Restore** on the version you want, and confirm at "Restore this version?".

Result: A named snapshot is saved, and you can restore the canvas to it at any time.

Related: The dialog's Description field and the list's element-count and creator-name fields are not stored or returned by the backend today and render blank. An agent can snapshot and restore with `board_create_version`, `board_list_versions`, and `board_restore_version`.

Story: Promote brainstorm stickies into Bam tasks

Who: A facilitator (with an agent's help) turning ideas into tracked work. **Goal:** Convert selected sticky notes into Bam tasks in a project and phase. **Before you start:** Stickies exist on the board, and you know the target Bam project (and optionally phase). There is no human button for this; it runs through an agent or the API.

Steps

1. Identify the sticky notes you want to turn into tasks.
2. Ask an agent to promote them, naming the target project and phase. The agent uses `board_promote_to_tasks`.
3. The agent creates one Bam task per sticky and records a link from each sticky to its task.
4. Open the target project in Bam to review the new tasks.

Result: Each promoted sticky has a matching Bam task, linked back to the source element.

Related: This emits a `board.elements_promoted` Bolt event you can automate on. An agent can review the links with `board_list_links`. See the Bam help doc for task management.

Story: Lock a finished board

Who: The board creator or an org admin or owner. **Goal:** Freeze a board so others cannot change it. **Before you start:** Be the creator or an org admin or owner.

Steps

1. Open the board's canvas.
2. Open the **…** menu in the floating toolbar.
3. Click **Lock board**.
4. To allow edits again, repeat and click **Unlock board**.

Result: The board is read-only for everyone except its creator and org admins and owners, and shows a lock icon in the toolbar and on its card.

Related: An agent cannot toggle the lock through `board_update`; locking is a separate action with no MCP tool of its own today.

Story: Fix a board flagged with an integrity issue

Who: Anyone who opens a board with a broken project link. **Goal:** Clear the integrity warning by detaching or reassigning the project. **Before you start:** Have edit access to the board.

Steps

1. Open the board; its card in **All Boards** shows an amber warning icon.
2. Read the amber integrity banner at the top of the canvas.
3. To drop the broken link, click **Detach from project** (or **Leave unattached**).
4. To repoint it, click **Reassign to a project**, choose a project in the "Reassign to a project" dialog, and confirm.

Result: The integrity warning clears and the board is either unattached or attached to a valid project.

Related: An agent can do the same with `board_check_integrity` and `board_remediate_integrity` (action `detach` or `reassign`).

Story: Archive, restore, or delete a board

Who: A board owner cleaning up. **Goal:** Remove a board from the active list, recover it, or delete it for good. **Before you start:** Have edit access to the board.

Steps

1. On **All Boards**, open the board card's **...** menu and click **Archive**.
2. To recover it, click **Archive** in the sidebar, find the board, and click **Restore**.
3. To remove it permanently, click **Delete permanently** on the archived card, or use **Delete** on the **...** menu in the list and confirm "Delete board?".

Result: The board is archived (recoverable) or permanently deleted, depending on your choice.

Related: Use **Duplicate** on the **...** menu first if you want a copy before removing the original. An agent uses `board_archive`, `board_restore`, and `board_delete_permanent`.

Story: Export a board image

Who: Anyone who needs a picture of the board for a doc or deck. **Goal:** Get an SVG or PNG of the board. **Before you start:** Have access to the board.

Steps

1. Open the board's canvas.
2. Open the **...** menu in the floating toolbar.
3. Click **Export as PNG** or **Export as SVG**. The item shows "Exporting..." while the server renders the board.
4. The image file downloads, named after the board.

Result: You have an SVG or PNG rendering of the board saved to your downloads.

Related: An agent can export the same render with `board_export`, and the export endpoint is also reachable directly through the board-api.

Story: Have an agent set up and seed a board

Who: Someone who wants a board built and pre-populated without doing it by hand. **Goal:** Get a named board, scoped to a project, with starter stickies on it. **Before you start:** Be signed in. Know the project name (if you want it scoped) and the rough content you want seeded.

Steps

1. Ask an agent to create the board, naming it and the project. The agent uses `board_create` (optionally from a template by name).
2. Ask the agent to drop your starter ideas as stickies. It uses `board_add_sticky` once per note, choosing colors.
3. Open `/board/` and the new board appears in **All Boards**; open it to see the seeded canvas.

Result: A ready-to-use board exists with your starting content, and you can keep editing it live.

Related: The agent can also group the stickies into frames so a later `board_summarize` reads cleanly, and can promote them to Bam tasks with `board_promote_to_tasks`.

Related

- **Bam** - the core project and task app. Board scopes boards to Bam projects, and promote-to-tasks creates Bam tasks in a project and phase. See `docs/apps/api/` or the Bam help doc.
- **Bolt** - workflow automation. Board emits `board.created`, `board.updated`, `board.locked`, and `board.elements_promoted` events you can build rules on.
- **Platform presence and audio** - provides the toolbar presence strip and any voice huddle on a board. Audio is a platform feature, not part of the Board service.
- **Board MCP-tools reference** - the full list and arguments for the 40 Board MCP tools, in `docs/apps/board/`.
- **Board guide** - the product guide in `docs/apps/board/`, for narrative and background.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What is a board's default visibility setting?

a) private
b) project
c) organization
d) public

2. What is the default background pattern for a new board's canvas?

a) grid
b) lines
c) dots
d) plain

3. What are the soft and hard element limits on a single board, and what happens at each?

4. What is the default color of a sticky note?

a) yellow
b) blue
c) green
d) white

5. Board has no login of its own. How does it know who you are and what you can do?

6. When a board is locked, who can still edit it?

a) Anyone with the link
b) Only the board creator
c) The board creator or an org admin or owner
d) No one until it is unlocked

7. Which native toolbar do you draw with on the canvas, and which controls does Board overlay on top of it?

8. What does the Share button in the floating toolbar actually do?

a) Opens a collaborator manager
b) Copies a shareable link to the board to your clipboard
c) Emails the board to a teammate
d) Publishes the board publicly

9. Why is there no in-app way to manage collaborators today, and what controls who can reach a board instead?

10. Which three formats can a board's canvas be exported as?

a) PDF, SVG, and PNG
b) JSON, SVG, and PNG
c) JSON, PDF, and PNG
d) DOCX, SVG, and PNG

11. On the Save Version dialog, which fields render blank, and why?

12. What is a frame used for on a board?

a) Locking a region of the canvas

b) A labeled region that groups the elements inside it
c) Setting the background pattern
d) Marking a board as a favorite

13. What does the connection status badge at the top-right report, and what are its three states?

CHAPTER 12

Bolt

When this happens, do that.

Bolt is the suite's wiring closet. Other apps emit events as people work, and Bolt watches for them and reacts, running the actions you defined without anyone clicking a button. You build a rule as WHEN this happens, IF these conditions hold, THEN run these steps, and every firing is recorded as an execution you can open and read line by line. By the end of this chapter you will be able to build a rule from a template or by hand, watch it run, and see where an AI agent can author and audit the same automations you do.

At a glance

- The suite's event hub: every app reports its activity to one place
- Trigger-condition-action rules with 13 condition operators and AND or OR logic
- Templates: 16 pre-built blueprints you instantiate and customize
- Schedule triggers that fire on a cron clock, not just on app events
- Full execution log with per-step detail, retries, and single-event tracing

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Automations home . Enabling and disabling an automation . Creating and editing an automation . Simple mode . Visual mode . Settings sidebar

> *Bolt watches for events across the BigBlueBam apps and reacts to them automatically. When something happens in one app (a task is created, a deal goes stale, a ticket breaches its SLA), Bolt runs a rule you defined and carries out one or more actions. Reach for it when you want work to happen without a person clicking a button.*

Overview

Bolt is the suite's event-driven automation engine, and its event hub. You build an **automation** (also called a rule) out of three parts: a **trigger**[113] (the event to watch for), optional **conditions** (extra tests the event must pass), and an ordered list of **actions** (what to do when it fires). The shape is "WHEN this happens, IF these conditions hold, THEN run these steps."

Other BigBlueBam apps publish events as people work: Bam emits task and sprint events, Bond emits deal events, Helpdesk emits ticket events, and so on. Every one of those apps reports its events to Bolt through a shared helper, so Bolt is the single place where the suite's activity converges. Bolt matches each event against your enabled automations and runs the matching rules. Each action[114] is an MCP tool call (for example "create a task in Bam" or "send a webhook"), so Bolt can act inside any app in the suite without you wiring up integrations by hand.

> ***Bam tasks, Bond deals, Helpdesk tickets: Bolt is the one place the whole suite's activity converges.***

Every run is recorded as an **execution**[115] with per-action steps, so you can see exactly what fired, what passed, what failed, and why. You can browse runs for a single automation or across the whole organization in the Execution Log, and you can trace one ingested event through every rule it touched.

> ***Every run is recorded as an execution with per-action steps, so nothing fires in the dark.***

This document describes what the Bolt web app actually does today. Where a capability exists only through the API or an MCP tool and is not wired into the in-app UI, that is called out so you do not go looking for a button that is not there.

Key concepts

- **Automation (rule)** - the top-level object: a trigger, optional conditions, an ordered list of actions, and limit settings. The UI calls these "Automations".
- **Trigger** - what starts an automation. It has a **Trigger Source** (the app that emits the event, such as Bam or Bond) and an **Event Type** (the bare event name, such as `task.created`). A trigger can also carry an optional filter.

[113] **Trigger.** what starts an automation. It has a **Trigger Source** (the app that emits the event, such as Bam or Bond) and an **Event Type** (the bare event name, such as `task.created`). A trigger can also carry an optional filter.
[114] **Action.** one step Bolt runs when the automation fires. Each action calls an allowlisted MCP tool with parameters you supply. Actions run in order; this is a linear list, not a branching flow (see the note under Actions).
[115] **Execution.** one run of one automation against one event. Its status is one of: **Running**, **Success**, **Partial** (some steps failed but all ran), **Failed** (halted early), or **Skipped** (a guard or condition prevented it). A skipped run records a reason such as cooldown active, rate limited, or conditions not met.

- **Event source**[116] - the emitting app, stored alongside the bare event name. Bolt uses bare-name-plus-explicit-source naming, so the event is `deal.rotting` with `bond` as a separate source field, not `bond.deal.rotting`. The trigger dropdowns offer 14 sources: Bam, Banter, Beacon, Brief, Helpdesk, Schedule, Bond, Blast, Board, Bench, Bearing, Bill, Book, and Blank.
- **Trigger filter**[117] - an optional set of equality rules (key matches value) checked against the raw event payload before the automation fires.
- **Condition**[118] - an extra test on the event: a field, an operator, and a value, grouped with AND or OR logic. Conditions gate the whole automation; if they are not met, the run is skipped. There are 13 operators (equals, not_equals, contains, not_contains, starts_with, ends_with, greater_than, less_than, is_empty, is_not_empty, in, not_in, matches_regex). Fields that read the event body must be prefixed `event.` (for example `event.task.priority`).
- **Action** - one step Bolt runs when the automation fires. Each action calls an allowlisted MCP tool with parameters you supply. Actions run in order; this is a linear list, not a branching flow (see the note under Actions).
- **On Error**[119] - per-action behavior when a step fails: **Stop** (halt the run), **Continue** (run the rest anyway), or **Retry** (retry up to the Retry Count).
- **Templating**[120] - action parameters can interpolate values from the event with `{{ event.* }}`, the actor with `{{ actor.id }}` / `{{ actor.type }}`, automation metadata with `{{ automation.* }}`, the current time with `{{ now }}`, and a prior step's result with `{{ step[N].result.* }}`. There is no `{{ actor.name }}` and no top-level `{{ org.* }}`.
- **Execution** - one run of one automation against one event. Its status is one of: **Running**, **Success**, **Partial** (some steps failed but all ran), **Failed** (halted early), or **Skipped** (a guard or condition prevented it). A skipped run records a reason such as cooldown active, rate limited, or conditions not met.

---

[116] **Event source.** the emitting app, stored alongside the bare event name. Bolt uses bare-name-plus-explicit-source naming, so the event is `deal.rotting` with `bond` as a separate source field, not `bond.deal.rotting`. The trigger dropdowns offer 14 sources: Bam, Banter, Beacon, Brief, Helpdesk, Schedule, Bond, Blast, Board, Bench, Bearing, Bill, Book, and Blank.

[117] **Trigger filter.** an optional set of equality rules (key matches value) checked against the raw event payload before the automation fires.

[118] **Condition.** an extra test on the event: a field, an operator, and a value, grouped with AND or OR logic. Conditions gate the whole automation; if they are not met, the run is skipped. There are 13 operators (equals, not_equals, contains, not_contains, starts_with, ends_with, greater_than, less_than, is_empty, is_not_empty, in, not_in, matches_regex). Fields that read the event body must be prefixed `event.` (for example `event.task.priority`).

[119] **On Error.** per-action behavior when a step fails: **Stop** (halt the run), **Continue** (run the rest anyway), or **Retry** (retry up to the Retry Count).

[120] **Templating.** action parameters can interpolate values from the event with `{{ event.* }}`, the actor with `{{ actor.id }}` / `{{ actor.type }}`, automation metadata with `{{ automation.* }}`, the current time with `{{ now }}`, and a prior step's result with `{{ step[N].result.* }}`. There is no `{{ actor.name }}` and no top-level `{{ org.* }}`.

- **Execution step**[121] - one action attempt inside an execution, with its own status: success, failed, or skipped.
- **Template**[122] - a pre-built automation blueprint you can instantiate and customize. Bolt ships 16 templates.
- **Schedule trigger**[123] - the `Schedule` source fires automations on a cron schedule (`cron.fired`) rather than off another app's event.
- **Limits**[124] - per-automation guards: **Max Executions / Hour** (1 to 1000 in the UI, default 60) and **Cooldown (seconds)** (0 to 3600, default 0). There is also an internal max chain depth that prevents automations from triggering each other in an endless loop.

Where to find it

Bolt lives at `/bolt/`. Open it from the suite app switcher, or go straight to the URL.

Prerequisites:

- You must be signed in to BigBlueBam. Bolt has no login of its own. If you are not signed in, it shows "Please log in to BigBlueBam first to access Bolt." with a "Go to BigBlueBam Login" link that sends you to `/b3/`.
- Bolt is org-scoped. The automations, templates, and executions you see belong to your active organization.
- For an automation to actually fire, the originating app must publish its events, and the background worker and MCP server must be running. Building and saving rules works on its own, but live execution depends on those services.

The sidebar nav has three items: **Automations** (the home list), **Executions** (the org-wide Execution Log), and **Templates** (the template gallery). A project scope selector sits in the chrome and defaults to **All Projects**; it scopes the automation list and the stat cards. Press the `?` key anywhere in Bolt to open the in-app Help viewer.

[121] **Execution step.** one action attempt inside an execution, with its own status: success, failed, or skipped.
[122] **Template.** a pre-built automation blueprint you can instantiate and customize. Bolt ships 16 templates.
[123] **Schedule trigger.** the `Schedule` source fires automations on a cron schedule (`cron.fired`) rather than off another app's event.
[124] **Limits.** per-automation guards: **Max Executions / Hour** (1 to 1000 in the UI, default 60) and **Cooldown (seconds)** (0 to 3600, default 0). There is also an internal max chain depth that prevents automations from triggering each other in an endless loop.

Feature reference

Automations home

The home page (/) lists your automations and summarizes them. The heading reads "Automations" with the subtitle "Create trigger-condition-action workflows to automate your work."

To browse and filter automations:

1. Open **Automations** from the sidebar (the lightning / Zap icon).
2. Read the three stat cards at the top: **Total Automations**, **Enabled**, and **Disabled**.
3. Use the "Search automations..." box to filter by name.
4. Click a source chip ("All", then Bam, Banter, Beacon, Brief, Helpdesk, Schedule, Bond, Blast, Board, Bench, Bearing, Bill, Book, Blank) to show only automations on that trigger source.
5. Use the **Enabled** and **Disabled** toggles to filter by state.
6. Each automation card shows its name, a source badge, the trigger event, the description, "Last run", and an "Actions" count. If you have none yet, the list reads "No automations found".

The home list, stats, and filters are shown in the screenshot below.

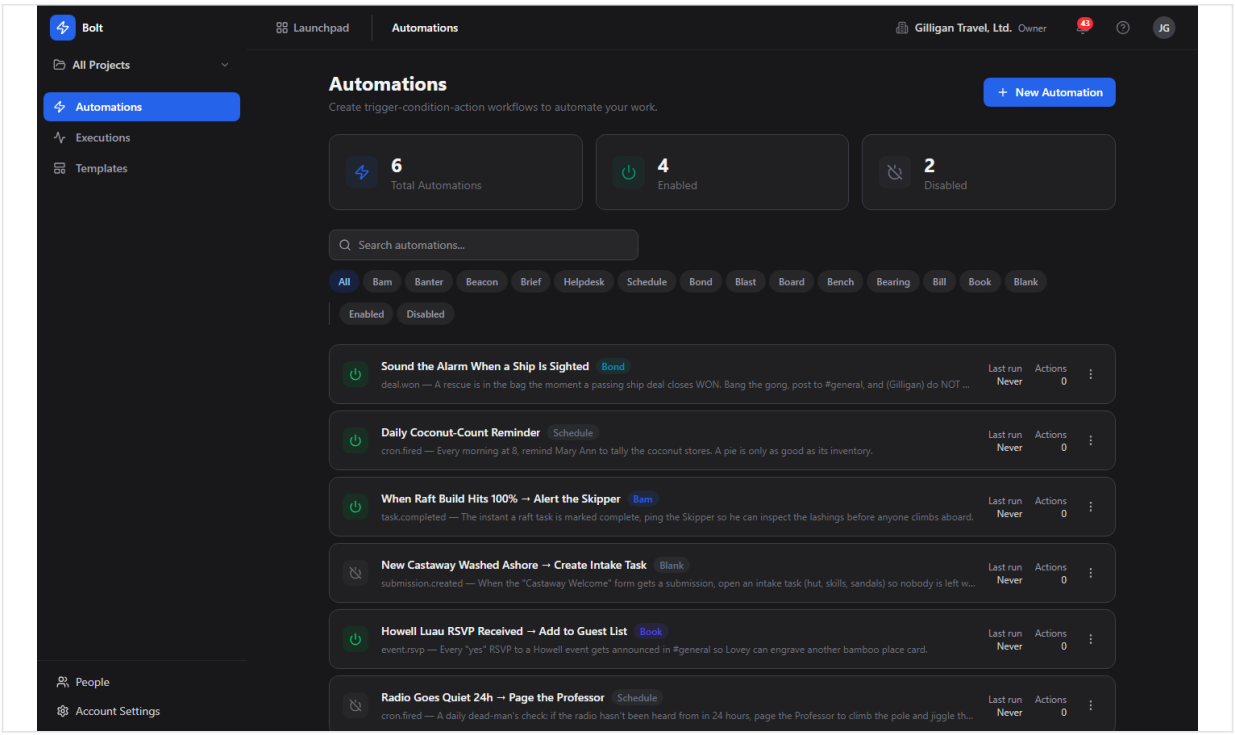


Figure 12.1 *Automations home - stat cards, source filter chips, and the automation list*

Enabling and disabling an automation

Each card carries a power toggle so you can turn a rule on or off without opening it. Disabled automations never fire.

To toggle an automation from the home list:

1. On the automation's card, click the power toggle (its tooltip reads "Enable" when off and "Disable" when on).
2. The card updates in place and the **Enabled** / **Disabled** stat cards adjust.

You can also set the state from inside the editor with the **Enabled** toggle in the Settings sidebar, or save directly to enabled with **Save & Enable** (see the editor below).

Creating and editing an automation

The editor (`/new` for a new rule, `/automations/:id` for an existing one) is where you define the trigger, conditions, actions, and limits. It has two modes, **Simple** and **Visual**, switched with the toggle near the top. There is no separate automation-detail page; opening a card from the home list takes you into the editor, which doubles as the detail view.

To start a new automation:

1. On the **Automations** home page, click **New Automation** (the Plus button). You land in the editor in Simple mode.
2. Type a name into the "Automation name..." field at the top, and optionally a description in "Add a description...".

The editor with its WHEN / IF / THEN sections and the Settings sidebar is shown below.

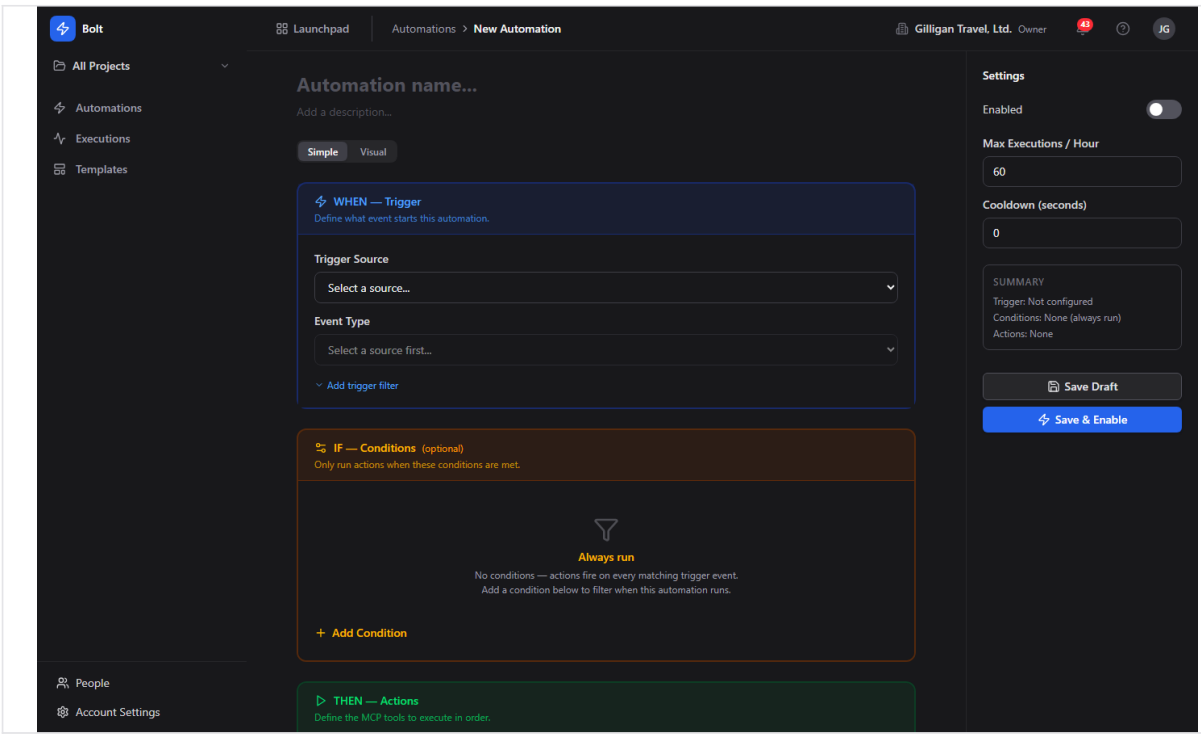


Figure 12.2 *Automation editor - the WHEN, IF, and THEN sections and the Settings sidebar*

Opening an existing automation from a home-list card lands you in the same editor, pre-filled with that rule's trigger, conditions, and actions.

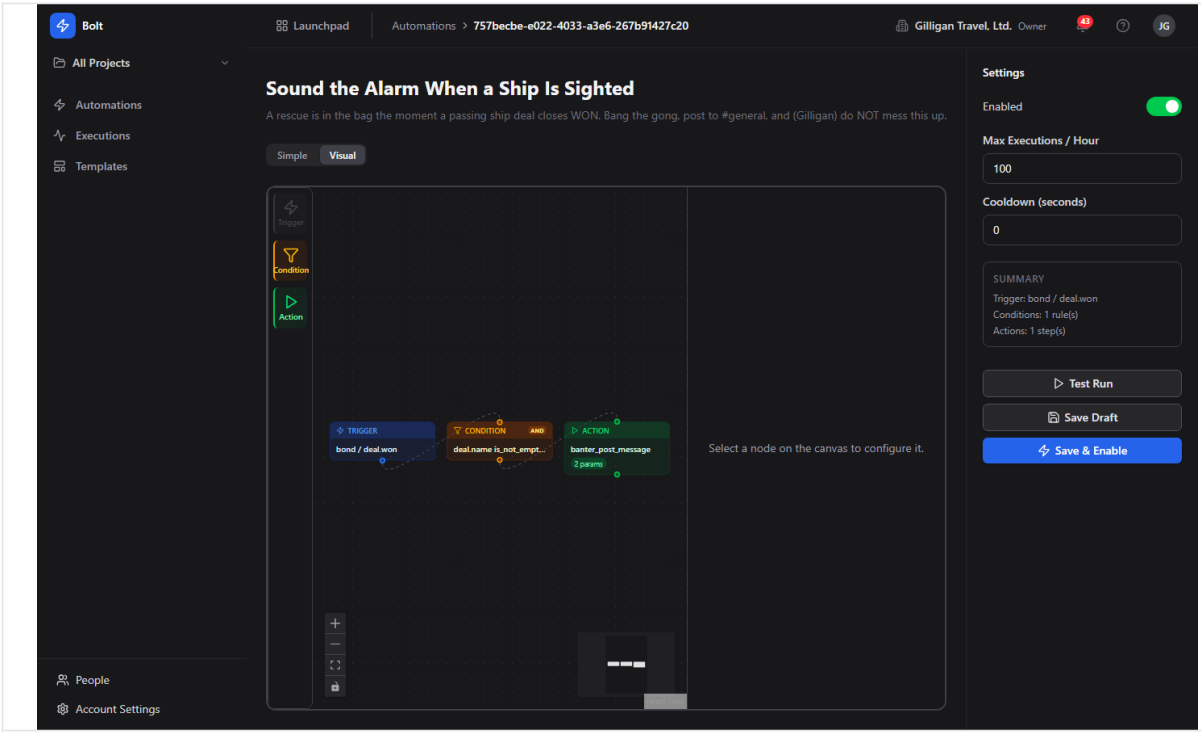


Figure 12.3 *Automation detail - an existing rule opened in the editor from a home-list card*

Simple mode

Simple mode lays the rule out as three stacked sections: **WHEN - Trigger** (blue), **IF - Conditions** (amber, optional), and **THEN - Actions** (green).

To set the trigger:

1. In the **WHEN - Trigger** section, choose a **Trigger Source** from the dropdown.
2. Choose an **Event Type**. Each option shows the event name and a short description.
3. If you picked the **Schedule** source, a **Schedule** editor appears. Use its Presets, Custom, or Advanced tabs (11 presets, a day-of-week picker, and a timezone) to build the cron schedule; the editor shows a plain-English description of when it will run.
4. To narrow which events match, click "Add trigger filter" and add equality rules. The link then reads "Filter (N rules)".

To add conditions (optional):

1. In the **IF - Conditions** section, click **Add Condition**. With no conditions the section reads "Always run".
2. For each row, set the logic toggle (the first reads "Where", later rows toggle AND / OR), pick a **field**, choose an **operator** (the operator list is filtered to the field's type and uses friendly labels), and enter a **value**.
3. Remove a condition with its remove control. Conditions gate the whole automation: if they fail, the run is skipped.

To add actions:

1. In the **THEN - Actions** section, click **Add Action**. You must add at least one ("Add at least one action to define what happens.").
2. For each action (numbered #N), pick a tool from the action select. Tools are grouped by source and each option shows the tool's description.
3. Fill in the **Parameters**. Required parameters are marked with "*". Some are enum or boolean selects; text parameters accept `{{ }}` templates. Click "Add optional parameter (N available)" to expose optional ones.
4. To control failure handling, click "Show advanced" and set **On Error** (Stop, Continue, or Retry) and a **Retry Count**.
5. Actions run top to bottom in the order listed.

Note on flow: Bolt runs actions as a single linear ordered list. Conditions decide whether the whole automation runs; they do not branch to different actions. Despite any "branching" language in older marketing, there are no per-edge branches. Variable interpolation between steps is real: a later action can reference an earlier one with `{{ step[N].result.* }}`.

Visual mode

Visual mode shows the same automation as a node graph on a canvas. Switching from Simple to Visual builds a graph from your current form state (or loads the saved graph if one exists).

To use Visual mode:

1. Click **Visual** on the mode toggle.
2. Add nodes from the palette: **Trigger**, **Condition**, and **Action**. Only one trigger is allowed.
3. Select a node to configure it in the Node inspector on the side. With nothing selected it reads "Select a node on the canvas to configure it."
4. Press `?` on the canvas to see the shortcuts: Delete or Backspace to remove, Escape to deselect, Ctrl+A to select all, scroll to zoom, and drag to pan.
5. Switch back to **Simple** at any time. If you have unsaved graph changes, Bolt warns "Unsaved graph changes may be lost. Switch to Simple mode?" before discarding them.

Note: Bolt compiles the graph down to the same linear ordered action list as Simple mode when it saves, so non-linear arrangements are flattened.

Settings sidebar

The right sidebar holds the rule's limits and the save controls.

1. **Enabled** - toggle whether the automation is active.
2. **Max Executions / Hour** - cap how many times this rule may run per hour (default 60; 1 to 1000).
3. **Cooldown (seconds)** - minimum gap between runs (default 0; 0 to 3600).
4. The **Summary** card restates the configured trigger, condition count, and action count.
5. **Save Draft** saves without enabling. **Save & Enable** saves and turns the rule on.

If the form has problems, a banner reads "Please fix the following errors before saving:" and lists each issue; fix them and save again.

Test Run

The editor shows a **Test Run** button in the Settings sidebar for an already-saved automation (it does not appear while creating a new one). It evaluates the automation's conditions against a sample event without running any actions.

To use it:

1. Open a saved automation in the editor.
2. Click **Test Run** in the Settings sidebar. Bolt sends a simulated event, seeded from the trigger filter when you have one, to the test endpoint.
3. Read the result box that appears under the button. A green box means the conditions passed (the actions would run on a real event); an amber box means they did not. The box shows a human-readable summary message.

The test only evaluates conditions; it never executes actions. To confirm an automation's actions actually run, enable it and inspect the resulting runs in the Execution Log. Agents can run the same conditions-only dry run with the `bolt_test` MCP tool, passing an explicit simulated event (see Working with AI agents).

> **NOTE**

> Test Run is a conditions-only dry check: a green box means the actions would run on a real event, an amber box means they would not. To confirm the actions themselves fire, enable the rule and read the resulting runs in the Execution Log.

Duplicating an automation

Duplicating clones a rule so you can adapt it without touching the original.

To duplicate from the home list:

1. On the automation's card, open the more-menu (the three-dot menu).
2. Click **Duplicate**. Bolt creates a copy named "<name> (copy)" that starts **disabled**, so it cannot fire until you review and enable it.

> **TIP**
> Duplicate a working rule before you experiment on it. The copy starts disabled and carries no execution history, so you can adapt it freely and only flip the toggle once it looks right.

Deleting an automation

To delete a rule:

1. On its card, open the more-menu and click **Delete**.
2. Confirm in the browser prompt. Deleting removes the automation and its conditions, actions, executions, and version history.

> **WARNING**
> Delete is irreversible and takes everything with it: the rule plus all of its conditions, actions, executions, and version history. Disable a rule instead when you only need to pause it.

Per-automation executions

Each automation has its own run history.

To view one automation's runs:

1. On its card, open the more-menu and click **View Executions** (or open it from the editor).
2. The page heading reads "<name> - Executions" with a "Back to <name>" link and the subtitle "Execution history for this automation."
3. Filter with the status chips: All, Success, Failed, Partial, Running, Skipped.
4. The table lists Status, Duration, Conditions Met, Error, and Started. Click a row to open the execution detail.

Execution Log (organization-wide)

The Execution Log (`/executions`) shows every automation run across your organization.

To monitor all runs:

1. Open **Executions** from the sidebar (the Activity icon). The heading reads "Execution Log" with the subtitle "History of all automation runs across your organization."
2. Filter with the status chips. When more than one automation has run, an "All automations" dropdown lets you narrow to a single rule.
3. The table lists Automation, Status, Duration, Conditions Met, and Started. Click a row to open the execution detail.

The org-wide log is shown below.

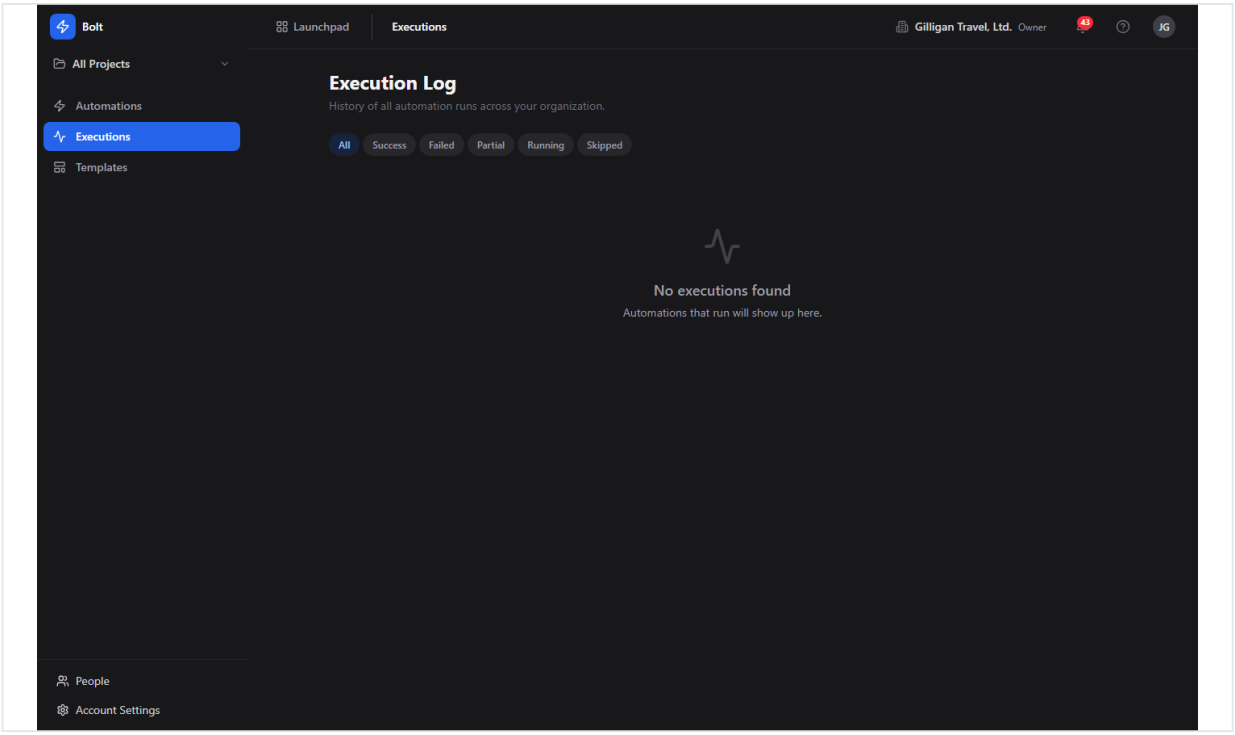


Figure 12.4 *Execution log - the organization-wide table of automation runs*

Execution detail

The execution detail page (`/executions/:id`) shows everything about a single run: what triggered it, whether its conditions passed, and what each step did.

To inspect a run:

1. Click any row in the Execution Log or a per-automation execution list.
2. The page heading is the automation's name, with a "Back to Executions" link and a status badge.
3. The summary cards show Duration, Conditions Met, Steps, and Completed.
4. If the run failed, an Error block explains why.
5. The "Trigger Event" panel shows the raw event JSON, and "Condition Evaluation" shows how each condition was scored.
6. The "Execution Steps" timeline lists each action attempt in order with its result.

Retrying a failed run

A failed or partial run can be re-queued from its detail page.

To retry:

1. Open the run's execution detail.
2. Click **Retry** (it appears only for runs whose status is Failed or Partial).
3. Bolt queues a fresh run. Retries count against the automation's hourly limit, so if you are over the cap the retry is rejected as rate limited.

Templates

Templates are pre-built automations you can instantiate and then customize. Bolt ships 16 of them.

To start from a template:

1. Open **Templates** from the sidebar (the LayoutTemplate icon). The heading reads "Templates" with the subtitle "Start with a pre-built automation template and customize it."
2. Each card shows a source badge, the template name, a description, and the trigger event.
3. Click **Use Template**. Bolt instantiates the template and drops you into the editor to adjust and save it.

The template gallery is shown below.

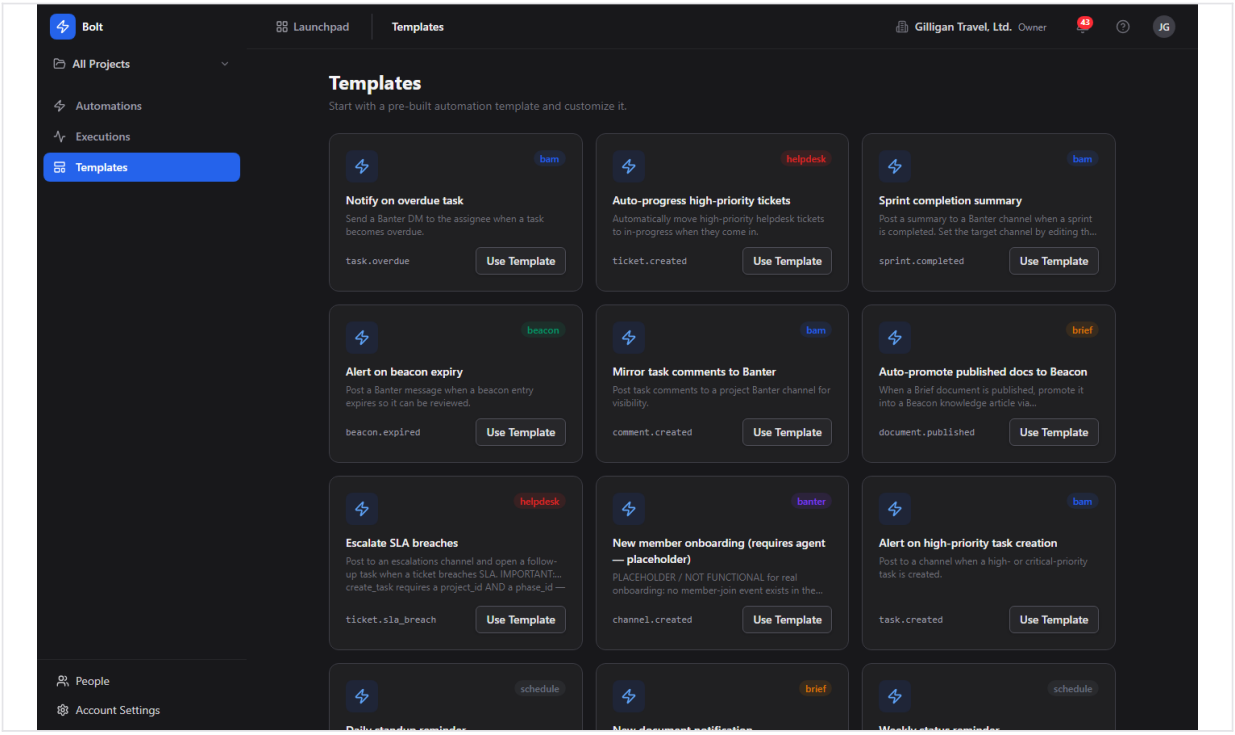


Figure 12.5 *Template gallery - the grid of pre-built templates, each with a source badge and Use Template button*

Note that some templates are starting points rather than turnkey rules. A few are labeled "(requires agent)" and wire a working notify step but leave the cross-app part (for example creating an invoice from a closed deal) for an AI agent to complete. Others need a value only you can supply (a cron schedule, a project and phase for a created task). Read each template's actions after instantiating and fill in anything marked required before you enable it.

> **TRY IT**
> Open Templates, click Use Template on "Alert on high-priority task creation", fill in any required parameter, then click Save & Enable and watch it appear on the Automations home as Enabled.

Working with AI agents

Bolt is built so an AI agent can author and operate automations the same way a person does, and so agents can serve as the execution substrate for the rules themselves. Bolt is also the suite's event hub: every app reports its activity here, which makes Bolt the natural place for an agent to observe what is happening across BigBlueBam.

Bolt exposes 26 MCP tools: 24 in the core Bolt tool module plus 2 observability tools.

- **Authoring and operating:** `bolt_create`, `bolt_update`, `bolt_patch` (metadata-only: name, description, enabled), `bolt_get`, `bolt_get_automation_by_name`, `bolt_list`, `bolt_stats`, `bolt_enable`, `bolt_disable`, `bolt_duplicate`, and `bolt_delete`. Mutating tools accept a name as well as a UUID and resolve it through `bolt_get_automation_by_name`, returning a clean "Automation not found" on a miss. This lets an agent run meta-automation tasks like "disable the Nightly Deploys rule" by name.
- **Templates:** `bolt_list_templates` and `bolt_instantiate_template` (with optional name, description, project, and cron overrides) let an agent stand up a rule from a built-in blueprint.
- **Versions:** `bolt_list_versions` and `bolt_restore_version` read and roll back the snapshots that every save records. This history has no in-app UI; agents and the REST API are the only way to use it (see below).
- **Inspecting and testing:** `bolt_executions` (runs for an automation), `bolt_list_executions` (org-wide runs), `bolt_execution_detail` (one run with its steps), `bolt_retry_execution` (re-queue a failed or partial run), and `bolt_test` (evaluate an automation's conditions against a simulated `event`). As in the UI, `bolt_test` only evaluates conditions; it does not execute actions. It returns `passed` (whether the actions would run), a per-condition `log`, and a human-readable `message`. The tool requires an explicit simulated `event` payload, so an agent can dry-check a rule against any payload it wants to imagine.
- **Catalogs:** `bolt_events` (the trigger event catalog, optionally filtered by source) and `bolt_actions` (the action allowlist with parameter schemas). An agent discovers valid triggers and actions here before building a rule.

- **AI drafting:** `bolt_generate` turns a natural-language prompt into a draft automation (it does not save; pass the result to `bolt_create`), and `bolt_explain` renders an automation definition in plain English. Both require an LLM provider configured in Bam Settings and return `AI_NOT_CONFIGURED` otherwise. This drafting has no in-app UI; it is agent and API only.
- **Observability:** `bolt_event_trace` returns the full evaluation trail for one ingested event id (every automation it matched, each condition outcome, and each action outcome), and `bolt_recent_events` lists recent ingested events that matched at least one automation, filtered by source, event, since, and limit.

Agents are also the execution substrate. Every action Bolt runs is an MCP tool call the background worker makes on the automation's behalf, so the same authority that lets an agent build a rule also powers the rule's steps.

Two capabilities exist in the backend and through MCP tools but have no in-app UI today, so do not look for them in the web app or present them to users as in-app buttons:

- **AI drafting** - the API and the `bolt_generate` / `bolt_explain` tools can draft an automation from a prompt and explain one in plain English, but no screen in the Bolt web app calls them.
- **Versioning** - every save snapshots a restorable version, and the API and the `bolt_list_versions` / `bolt_restore_version` tools can list and restore them, but the web app does not surface version history or a restore button.

Agent runs ride the suite's shared agentic platform, which sits alongside the Bolt-specific tools:

- **Identity and heartbeat.** Agents are first-class actors (`users.kind` of `agent` or `service`), so their writes are attributed in the activity log, and they report liveness with `agent_heartbeat`. Service-account tool calls fail closed against per-agent kill switches and glob allowlists (`bolt.*`), so an operator can revoke an agent's Bolt access centrally without editing any rule.
- **Approvals.** When a flow needs a human in the loop, an agent files a proposal with `proposal_create`, which a reviewer resolves with `proposal_list` and `proposal_decide`. Those decisions themselves emit `proposal.created` and `proposal.decided` platform events, which Bolt can trigger on.
- **Visibility.** Before an agent surfaces a cited entity into a shared place, it calls `can_access` to confirm the asker is allowed to see it, and drops anything that is not. Bolt automations that post cross-app results should respect the same gate.
- **Cross-app read plane.** `search_everything` and `activity_query` let an agent pull context from across the suite, including the unified activity view, before deciding how to build or change a Bolt rule.
- **Outbound webhooks.** Bolt fans subscribed events to agent runners over HMAC-signed webhooks with retry-and-backoff and auto-disable on repeated failure, and it emits a platform `catalog.drift_detected` event when an unknown event name is ingested, so an agent can notice and react to drift.

For the full tool catalog and parameter schemas, see the MCP-tools reference and guide in `docs/apps/bolt/`.

Working together (live presence)

Like every BigBlueBam app, Bolt carries the persistent Bureau presence dock, so collaboration is ambient rather than scheduled. From anywhere in Bolt you can see who is around and ring, knock, or invite a teammate into a live voice or video huddle that follows you in a floating window as you both move through the suite. Voice and video here are the digital equivalent of bumping into someone in the hallway or stopping by their desk, not a booked meeting. The deeper per-record collaboration (a presence strip showing who is on a specific item, and real-time co-editing of the same record) lives on the document, board, diagram, and task surfaces, described in the Introduction; in Bolt, the shared layer is the always-on Bureau dock.

User Stories

Story: Create your first automation from a template

Who: A new Bolt user who wants automation running quickly without building from scratch. **Goal:** Stand up a working rule and turn it on. **Before you start:** Be signed in to BigBlueBam with your target organization active. Pick a template whose action does not need values you cannot supply yet.

Steps

1. Open Bolt at `/bolt/` and click **Templates** in the sidebar.
2. Browse the gallery. Each card shows the source, name, description, and trigger event. Click **Use Template** on one you want, for example "Alert on high-priority task creation".
3. You land in the editor with the trigger, conditions, and actions pre-filled. Review each action's Parameters and fill in anything marked required (some templates need a project and phase, a cron schedule, or a notification target).
4. Open the **Settings** sidebar and check **Max Executions / Hour** and **Cooldown (seconds)**; adjust if needed.
5. Click **Save & Enable**.

Result: The rule appears on the Automations home page as Enabled and will run the next time a matching event is published.

Related: Build a rule by hand with "Create an automation by hand". An agent can do the same with `bolt_instantiate_template` then `bolt_enable`, or `bolt_create` then `bolt_enable`.

Story: Create an automation by hand

Who: A team lead who wants a specific rule the templates do not cover. **Goal:** Define a trigger, optional conditions, and actions, then enable the rule. **Before you start:** Be signed in with the right organization active. Know which event should start the rule and what you want it to do.

Steps

1. On the Automations home page, click **New Automation**.
2. Type a name into "Automation name...", and optionally a description.
3. In **WHEN - Trigger**, pick a **Trigger Source** and an **Event Type**. For a scheduled rule, pick **Schedule** and build the cron schedule in the Schedule editor.
4. In **IF - Conditions** (optional), click **Add Condition** and build each row: a field, an operator, and a value. Remember that fields reading the event body start with `event.` (for example `event.task.priority`).

5. In **THEN - Actions**, click **Add Action**, pick a tool, and fill in its Parameters. Use `{{ event.* }}` and `{{ step[N].result.* }}` templates where you need values from the event or a previous step. Use "Show advanced" to set **On Error** and **Retry Count**.
6. In the Settings sidebar, set **Max Executions / Hour** and **Cooldown (seconds)**.
7. Save the rule, then click **Test Run** to dry-check its conditions against a sample event. When you are satisfied, click **Save & Enable** to turn it on (or **Save Draft** to keep it off for now).

Result: The automation is saved. If you used Save & Enable, it will fire on the next matching event; if you saved a draft, enable it later from the home list power toggle.

Related: Confirm runs with "Watch a run and read its detail". An agent can author the same rule with `bolt_create` and ask `bolt_explain` to describe it in plain English.

Story: Auto-create a Bam task when a Bond deal goes stale

Who: A sales operations owner who wants neglected deals to become follow-up tasks automatically. **Goal:** When Bond flags a deal as rotting, Bolt creates a Bam task so someone follows up. **Before you start:** Be signed in with the organization that owns the Bond pipeline active. Bond's stale-deal detection runs as a daily background job that emits a `deal.rotting` event for deals past their stage's rotting threshold. Know the Bam project and phase the new task should land in.

Steps

1. On the Automations home page, click **New Automation** and name it, for example, "Follow up on rotting deals".
2. In **WHEN - Trigger**, set **Trigger Source** to **Bond** and **Event Type** to `deal.rotting`.
3. Leave **IF - Conditions** empty to act on every rotting deal, or add a condition such as `event.deal.value` greater_than a threshold to only chase larger deals.
4. In **THEN - Actions**, click **Add Action** and choose the Bam "create task" action. Fill in the required project and phase, and template the task title and body from the event, for example a title using `{{ event.deal.name }}`.
5. In the Settings sidebar, leave a sensible **Cooldown (seconds)** so the same deal does not generate repeated tasks within a short window.
6. Click **Save & Enable**.

Result: The next time the daily stale-deal job emits `deal.rotting`, Bolt creates a Bam task for that deal. Each run shows up in the Execution Log; open one to see the trigger event and the created-task step.

Related: This is the canonical cross-app chain (Bond to Bolt to Bam). To verify it, see "Watch a run and read its detail". An agent can build the same rule with `bolt_create` and audit firings with `bolt_event_trace` and `bolt_recent_events`.

Story: Turn an automation off and back on

Who: Anyone who needs to pause a rule temporarily (during a migration, a noisy period, or maintenance). **Goal:** Stop a rule from firing without deleting it, then resume it later. **Before you start:** The automation already exists.

Steps

1. Open the Automations home page.
2. Find the automation's card and click its power toggle. The tooltip reads "Disable" when the rule is on.
3. The card switches to Disabled and the **Disabled** stat card increments. The rule no longer fires.
4. When you are ready, click the toggle again (now reading "Enable") to resume it.

Result: The automation is paused or resumed immediately. While disabled, no events match it.

Related: An agent can do the same with `bolt_disable` and `bolt_enable`, addressing the rule by name.

Story: Watch a run and read its detail

Who: A user who wants to confirm an automation behaved correctly, or diagnose why it did not. **Goal:** Find a specific run and understand what triggered it, whether conditions passed, and what each step did. **Before you start:** The automation has fired at least once (or you have just enabled it and triggered the event).

Steps

1. Open **Executions** from the sidebar to see the org-wide Execution Log.
2. Narrow the list with the status chips (All, Success, Failed, Partial, Running, Skipped) and, if more than one rule has run, the "All automations" dropdown.
3. Click the run you want. The detail page opens with the automation name, a status badge, and summary cards for Duration, Conditions Met, Steps, and Completed.
4. Read the "Trigger Event" panel for the raw event, "Condition Evaluation" for how conditions scored, and the "Execution Steps" timeline for each action's result. If it failed, read the Error block.

Result: You can see exactly why a run succeeded, ran partially, failed, or was skipped.

Related: For one automation only, use **View Executions** from its card. An agent can pull the same data with `bolt_executions`, `bolt_execution_detail`, and `bolt_event_trace`.

Story: Retry a failed run

Who: A user whose automation failed because of a transient problem (a downstream app was briefly unavailable). **Goal:** Re-run the failed execution without rebuilding anything. **Before you start:** Open the run's execution detail; its status must be Failed or Partial.

Steps

1. On the execution detail page, confirm the status badge reads Failed or Partial.
2. Click **Retry**.
3. Bolt queues a fresh run. If the automation is already at its hourly cap, the retry is rejected as rate limited; wait for the window to reset or raise **Max Executions / Hour** on the rule.

Result: A new run is queued and appears in the Execution Log. Open it to confirm it succeeded this time.

Related: Adjust the rule's limits in the editor's Settings sidebar if retries are being throttled. An agent can re-queue the same run with `bolt_retry_execution`.

Story: Schedule a recurring automation

Who: Someone who wants a rule to run on a clock (a daily standup reminder, a weekly status nudge) rather than off another app's event. **Goal:** Build an automation that fires on a cron schedule. **Before you start:** Be signed in with the right organization active. Decide the cadence and timezone.

Steps

1. Click **New Automation** and name it.
2. In **WHEN - Trigger**, set **Trigger Source** to **Schedule**. The **Event Type** is the scheduled fire (`cron.fired`), and a **Schedule** editor appears.
3. In the Schedule editor, pick a preset (there are 11), or use the Custom or Advanced tabs to set the exact cadence and day-of-week. Set the timezone. The editor shows a plain-English description of when it will run.
4. In **THEN - Actions**, add the action to run on each tick (for example, post a reminder).
5. Click **Save & Enable**.

Result: The automation runs on the schedule you defined. Confirm fires in the Execution Log.

Related: Several schedule-based templates ("Daily standup reminder", "Weekly status reminder") give you a head start; instantiate one from Templates and just supply the cron.

Story: Operate Bolt from an AI agent

Who: An operator running an AI agent that needs to build, toggle, or audit automations programmatically. **Goal:** Manage rules and inspect firings through MCP tools instead of the web UI. **Before you start:** The agent runs with a service account that passes Bolt's policy allowlist (`bolt.*`) and reports liveness with `agent_heartbeat`. Know the automation by name or id.

Steps

1. List or find the rule with `bolt_list` or `bolt_get_automation_by_name`.

2. Discover valid triggers and actions with `bolt_events` and `bolt_actions`, then create or modify rules with `bolt_create`, `bolt_update`, or the metadata-only `bolt_patch`. To draft from a prompt, use `bolt_generate` and pass the result to `bolt_create`.
3. Turn rules on or off with `bolt_enable` and `bolt_disable`, addressing them by name. Clone with `bolt_duplicate` (the copy starts disabled).
4. Dry-check a rule's conditions against a simulated event with `bolt_test` (no actions run); read the returned `passed`, `log`, and `message`.
5. Audit behavior with `bolt_executions`, `bolt_list_executions`, `bolt_execution_detail`, `bolt_event_trace` (one event's full trail), and `bolt_recent_events` (recent matched events). Re-queue a failure with `bolt_retry_execution`.
6. When a change needs sign-off, file a `proposal_create` and let a human resolve it with `proposal_decide`.

Result: The agent manages and observes automations end to end without touching the UI, with its writes attributed to its agent identity and gated by the org's agent policy.

Related: See the MCP-tools reference in `docs/apps/bolt/` for parameter schemas.

Related

- **Bam** - the most common action target. Bolt creates and moves Bam tasks (for example the `deal.rotting` to task chain) and reacts to Bam events like `task.created`, `task.overdue`, `task.moved`, and `sprint.completed`.
- **Bond** - emits deal events (`deal.rotting`, `deal.won`) that drive sales automations.
- **Helpdesk** - emits ticket events (`ticket.created`, `ticket.sla_breach`) for support escalations.
- **Banter** - a frequent notification target for "post a message" actions, and the source of channel events.
- **Beacon** and **Brief** - knowledge and document events (publish, expire) that can promote or mirror content.
- **Bench**, **Bearing**, **Bill**, **Book**, **Blast**, **Blank**, **Board** - additional trigger sources and action targets across the suite.
- The in-app Help viewer (press `?`) mirrors this content inside Bolt.
- For the full MCP tool catalog and schemas, see the MCP-tools reference and guide in `docs/apps/bolt/`.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What three parts make up a Bolt automation?

a) A source, a target, and a schedule
b) A trigger, optional conditions, and an ordered list of actions
c) An event, a webhook, and a retry policy
d) A WHEN, an ELSE, and a THEN

2. How does Bolt name events and their source?

a) A single combined name like bond.deal.rotting
b) The bare event name plus an explicit separate source field, such as deal.rotting with bond as the source
c) A numeric event code with a source prefix
d) The source name only, with no event name

3. What are the five possible statuses of an execution?

4. What is the default value of Max Executions / Hour for an automation?

a) 10
b) 60
c) 100
d) 1000

5. How many condition operators does Bolt offer?

a) 7
b) 11
c) 13
d) 16

6. When an action fails, what are the three On Error behaviors Bolt can take?

7. How many templates does Bolt ship with?

a) 8
b) 14
c) 16
d) 26

8. What does the Schedule trigger source do, and what event type does it fire?

9. When you duplicate an automation, what state does the copy start in?

a) Enabled
b) Disabled
c) Running
d) Draft pending review

10. When does the Retry button appear on an execution detail page, and what happens if the automation is already at its hourly cap?

11. How does Bolt handle the actions in an automation?

a) As a branching flow where conditions route to different actions
b) As a single linear ordered list that runs top to bottom
c) As parallel steps that all run at once

d) As a random order chosen at runtime

12. When reading the event body in a condition, what prefix must the field name carry?

CHAPTER 13

Bond

Track the relationship, close the deal.

Bond is the CRM where your sales work lives. It tracks the people and companies you sell to, moves deals across a pipeline you shape yourself, and watches every card age so nothing rots unattended. This chapter takes you from an empty board to a working pipeline, a contact with a history, and a deal closed Won or Lost, then shows where an AI agent can drive the same records. By the end you will be able to set up stages, qualify a lead, work the board with swimlanes, and read the forecast.

At a glance

- A drag-and-drop pipeline board with rotting alerts that flag aging deals
- Contacts and companies with lifecycle stages, lead scores, and soft delete
- Configurable pipelines and stages, custom fields, and org-wide scoring rules
- Analytics for weighted forecast, win rate, velocity, and loss reasons
- A large MCP catalog, so an agent can work the CRM with a salesperson's reach

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Choosing the active pipeline . Pipeline Board . Creating a deal . Deal detail and outcomes . Stage History and Related items . Contacts list

> *Bond is the BigBlueBam CRM. It tracks the people and companies you sell to, moves deals through a configurable pipeline, and reports on forecast, win rate, and stale deals. Reach for it when you need to manage relationships and close revenue.*

> ***Shape your own stages, then drag deals across them. The pipeline bends to how your team actually sells.***

Overview

Bond organizes your sales work around four core objects: contacts (people), companies (organizations), deals (revenue opportunities), and pipelines (the ordered stages a deal[125] passes through). You log activities against any of these, score leads with rules you define, and watch deals age so nothing rots unattended.

The pipeline[126] board is the daily working surface. Each deal is a card in a stage[127] column; you drag cards between columns to move a deal forward, mark it Won or Lost, and group cards into swimlanes by owner or close month. The Contacts and Companies lists are your address book, and the Analytics page rolls everything up into forecast, velocity, and win/loss numbers.

Bond is org-scoped and connects to the rest of the suite. A deal's Related panel surfaces Bill invoices, Book events, and Bam tasks tied to that deal. Bond also emits events (deal won, deal lost, deal rotting, contact[128] created) that Bolt automation rules and other apps can react to, and its contacts feed Blast email segments and Bench reports.

This document describes what the Bond web app actually does today. Where an action exists only through the API or an MCP tool (not the in-app UI), that is called out explicitly so you do not go looking for a button that is not wired up yet.

Key concepts

- **Contact** - a person you track. Holds name, email, phone, job title, avatar, lifecycle stage[129], lead source, lead score[130], address, custom fields, and an owner.
- **Company**[131] - an organization. Holds name, domain, industry, company size bucket, annual revenue, phone, website, logo, address, custom fields, and an owner. Contacts and deals can be linked to a company.
- **Deal** - a revenue opportunity in a pipeline. Holds a name, value, currency, expected close date, probability, and a computed weighted value (value times probability). A deal is Open until it is closed Won or Lost.

[125] **Deal.** a revenue opportunity in a pipeline. Holds a name, value, currency, expected close date, probability, and a computed weighted value (value times probability). A deal is Open until it is closed Won or Lost.
[126] **Pipeline.** an ordered set of stages plus a currency. One pipeline can be the default. The default pipeline is selected automatically when you open the board; you can switch to another from the sidebar scope selector.
[127] **Stage.** a column on the board. Has a name, a type (Active, Won, or Lost), a probability percentage, an optional rotting threshold in days, and a color.
[128] **Contact.** a person you track. Holds name, email, phone, job title, avatar, lifecycle stage, lead source, lead score, address, custom fields, and an owner.
[129] **Lifecycle stage.** where a contact sits in your funnel: Lead, Subscriber, MQL (marketing qualified), SQL (sales qualified), Opportunity, Customer, Evangelist, or Other. New contacts default to Lead.
[130] **Lead score.** a number from 0 to 100 cached on each contact. Scoring rules add or subtract points based on contact attributes.
[131] **Company.** an organization. Holds name, domain, industry, company size bucket, annual revenue, phone, website, logo, address, custom fields, and an owner. Contacts and deals can be linked to a company.

- **Lifecycle stage** - where a contact sits in your funnel: Lead, Subscriber, MQL (marketing qualified), SQL (sales qualified), Opportunity, Customer, Evangelist, or Other. New contacts default to Lead.
- **Lead score** - a number from 0 to 100 cached on each contact. Scoring rules add or subtract points based on contact attributes.
- **Pipeline** - an ordered set of stages plus a currency. One pipeline can be the default. The default pipeline is selected automatically when you open the board; you can switch to another from the sidebar scope selector.
- **Stage** - a column on the board. Has a name, a type (Active, Won, or Lost), a probability percentage, an optional rotting threshold in days, and a color.
- **Activity**[132] - a timeline entry logged against a contact, company, and/or deal. Types include Note, Email Sent, Email Received, Call, Meeting, and Task, plus system-generated entries like stage changes and deal created/won/lost.
- **Rotting (stale) deal**[133] - an open deal whose days in its current stage exceed that stage's rotting threshold. The card turns orange, and red when it is more than 1.5 times over.
- **Custom field**[134] - an extra field you define per entity type (Contact, Company, or Deal). These are org-wide for that entity type, not per pipeline.
- **Lead scoring rule**[135] - a condition (field, operator, value) plus a point delta. Rules are org-wide and evaluated together to compute each contact's score.
- **Soft delete**[136] - contacts, companies, and deals are not erased when deleted. They are hidden and can be restored from their list with Include deleted.

TIP

You will only ever see the empty 'No pipeline selected' screen when the org truly has no pipelines. Once one exists, Bond auto-selects the default (or the first) so the board opens straight onto deals. Mark your main pipeline as default and skip the picker.

TRY IT

Open the Pipeline Board and scan the cards: orange means a deal has passed its stage's rotting threshold, and red means it is more than 1.5 times over. Click a red card, then Log Activity to record a follow-up and re-engage it.

[132] **Activity.** a timeline entry logged against a contact, company, and/or deal. Types include Note, Email Sent, Email Received, Call, Meeting, and Task, plus system-generated entries like stage changes and deal created/won/lost.
[133] **Rotting (stale) deal.** an open deal whose days in its current stage exceed that stage's rotting threshold. The card turns orange, and red when it is more than 1.5 times over.
[134] **Custom field.** an extra field you define per entity type (Contact, Company, or Deal). These are org-wide for that entity type, not per pipeline.
[135] **Lead scoring rule.** a condition (field, operator, value) plus a point delta. Rules are org-wide and evaluated together to compute each contact's score.
[136] **Soft delete.** contacts, companies, and deals are not erased when deleted. They are hidden and can be restored from their list with Include deleted.

Where to find it

Bond lives at `/bond/`. Open it from the Launchpad app switcher in the top-left chrome, or go straight to the URL.

Prerequisites:

- You must be signed in to BigBlueBam. If you are not, Bond shows "Please log in to BigBlueBam first to access Bond." with a "Go to BigBlueBam Login" button that sends you to `/b3/`.
- At least one pipeline must exist before you can create deals. If your org has no pipeline at all, the board shows "No pipeline selected" and a Create Pipeline button. Once a pipeline exists, the board selects it for you automatically (see Choosing the active pipeline below).
- Bond is org-scoped. Use the OrgSwitcher in the header to change which organization's data you see.

Roles and visibility:

- Members and viewers see only the deals and contacts they own. Admins and owners see everything in the org. This "own only" filtering applies to deal and contact lists, deal detail, stage history, duplicates, and related lookups.
- Most create and edit actions require read/write access. Administering pipelines, stages, scoring rules, custom fields, and import mappings requires admin access. Bulk contact import requires admin access.

The sidebar nav has five items: **Pipeline Board**, **Contacts**, **Companies**, **Analytics**, and **Bond Settings**. A pipeline scope selector sits at the top of the sidebar (showing the active pipeline name, or "Default Pipeline"); it sets the active pipeline used across the board and Analytics. Press the `?` key anywhere in Bond to open the in-app Help viewer.

Feature reference

Choosing the active pipeline

The pipeline scope selector at the top of the sidebar sets which pipeline the board and Analytics use. When pipelines exist, Bond auto-selects one for you on first load (the one marked default, or the first pipeline if none is marked default), so the board opens straight onto deals instead of an empty "No pipeline selected" screen. You only see that empty state when the org genuinely has no pipelines yet.

To switch pipelines:

1. Click the scope selector at the top of the sidebar (it shows the current pipeline name, or "Default Pipeline").
2. Pick a pipeline from the dropdown. The active one is marked with a check.
3. The Pipeline Board and Analytics now reflect that pipeline. If no pipelines exist yet, the dropdown reads "No pipelines found".

Pipeline Board

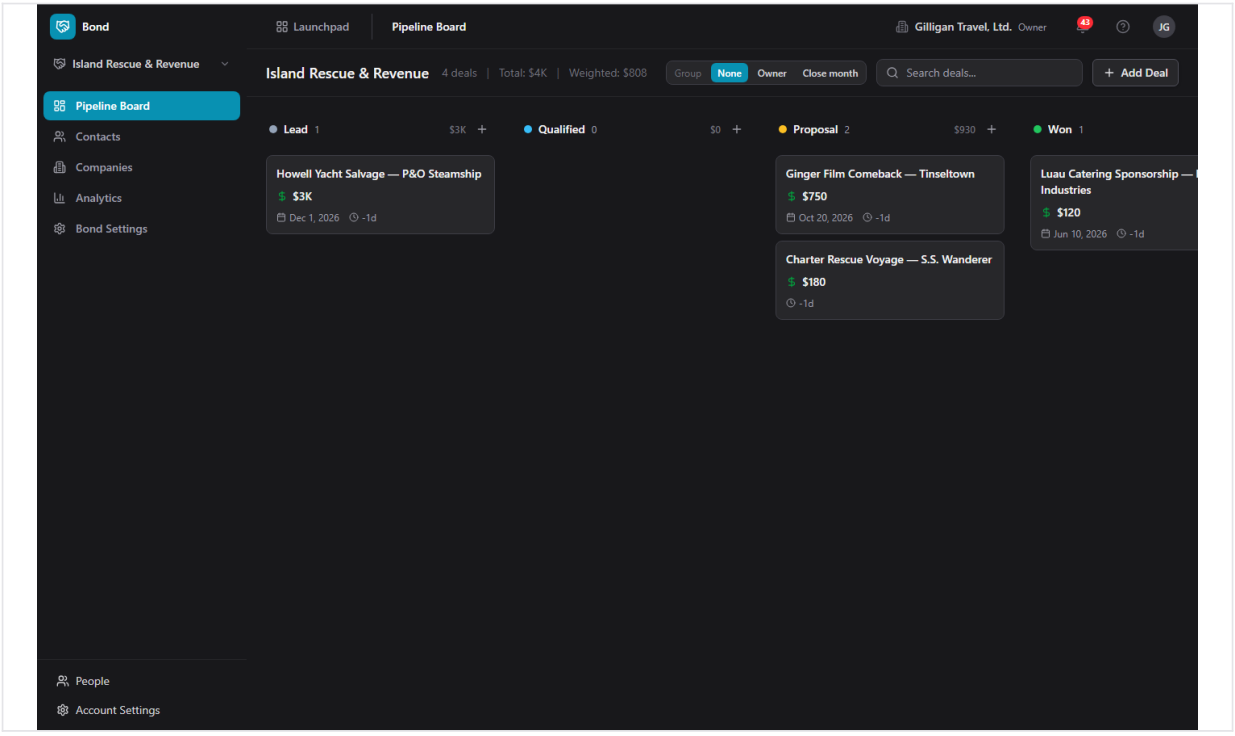


Figure 13.1 Pipeline board

The board (/) shows every deal in the active pipeline as a card in a stage column. The header shows the pipeline name and a summary: "N deals", "Total: $X", and "Weighted: $Y".

To read the board:

1. Open **Pipeline Board** from the sidebar. The default pipeline loads automatically.

2. Each column is a stage, with a color dot, the stage name, a deal count, and the stage's total value.
3. Each card shows the deal name, company, value, close date, days in stage, and the owner's avatar. Orange cards are rotting; red cards are severely overdue (more than 1.5 times the rotting threshold).

To move a deal to another stage:

1. Drag the deal card from its current column and drop it on the target stage column.
2. The deal's stage updates, the move is recorded in stage history, and a stage-changed event is emitted.

To search the board:

1. Type into the "Search deals..." box in the board header.
2. The board filters to deals whose name or company name matches.

To group deals into swimlanes:

1. In the board header, find the **Group** control.
2. Click **None**, **Owner**, or **Close month**.
3. Owner groups deals into a lane per owner name, with "Unassigned" last. Close month groups by close date (YYYY-MM), with "No close date" for deals lacking one.

Creating a deal

Deals are created from the board. The Create Deal dialog collects a name, an optional value, and an optional expected close date. It does not collect company, owner, probability, currency, or linked contacts; set those by editing the deal afterward (see Deal detail and outcomes), or through an MCP tool or the REST API (see Working with AI agents). You can also create a deal already linked to a contact from the contact's page (see Contact detail).

> **NOTE**
> The board's Create Deal dialog is deliberately minimal: name, optional value, optional close date. Set company, owner, probability, currency, and linked contacts afterward by editing the deal, or supply them up front through the bond_create_deal MCP tool, which an agent can use to collect the fields the dialog omits.

To add a deal at the first stage:

1. On the Pipeline Board, click **Add Deal** in the top-right.
2. In the "Create Deal" dialog, type a **Deal Name** (required), for example "Acme Corp - Enterprise Plan".
3. Optionally enter a **Value ($)** and an **Expected Close Date**.
4. Click **Create Deal**. The deal appears in the first stage of the pipeline.

To add a deal directly to a specific stage:

1. Hover the stage column header and click the + button ("Add deal to <stage>").
2. Fill in the same fields and click **Create Deal**. The deal lands in that stage.

Deal detail and outcomes

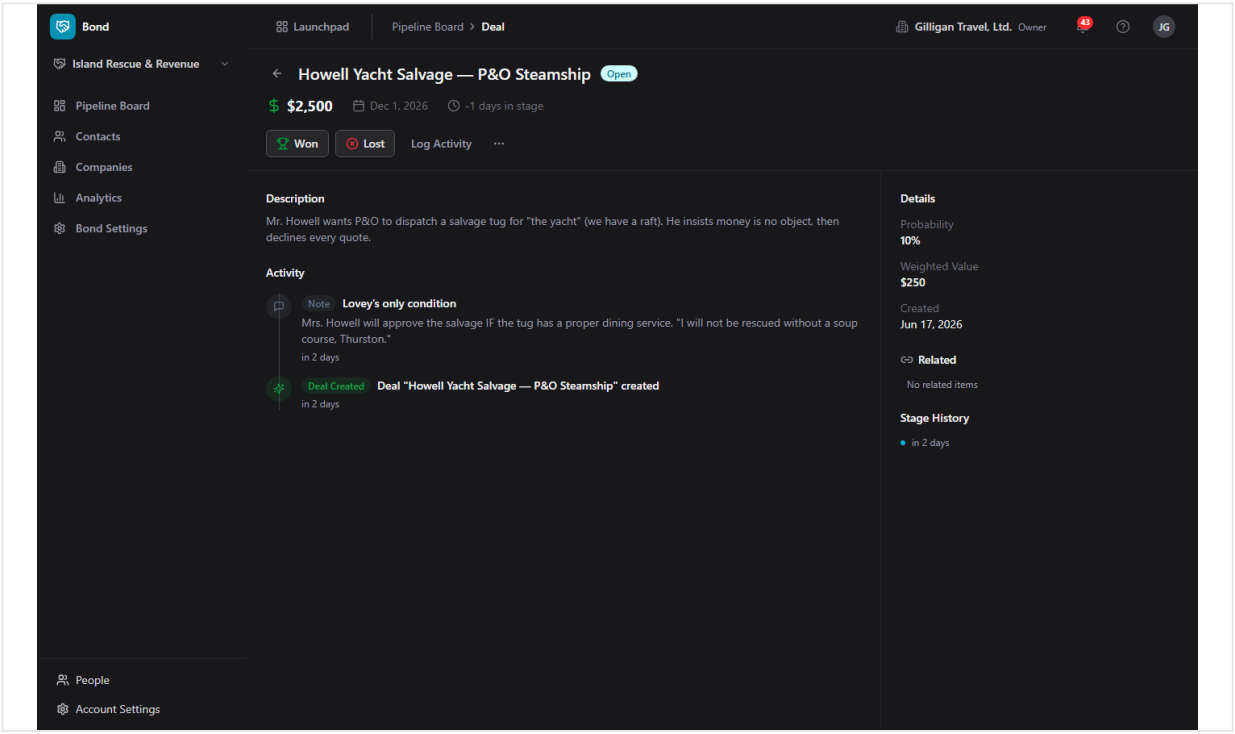


Figure 13.2 Deal detail

Open a deal card to see its full record at `/deals/:id`: the header with the deal name and a status badge (**Open**, **Won**, or **Lost**), value, company link, close date, days in stage, and owner. The left side shows the description, an inline Log Activity form, and the Activity timeline. The right side shows Details (Probability, Weighted Value, Created, Closed, Close Reason, Lost To), the Related panel, and Stage History.

To edit a deal:

1. Open the deal and click the overflow menu (the "..." button).
2. Choose **Edit Deal**. The "Edit Deal" dialog opens, pre-filled with the deal's current values.
3. Update the **Deal Name**, **Description**, **Value ($)**, **Expected Close Date**, and other fields, then click **Save Changes**. The change is saved through the API and the deal updates in place.

To close a deal as won:

1. Open the deal.
2. Click **Won** (the trophy button). The deal moves to a Won-type stage, its probability is set to 100, and a deal-won event is emitted.

To close a deal as lost:

1. Open the deal.
2. Click **Lost** (the X-circle button). The "Mark Deal as Lost" dialog opens.
3. Enter a **Reason** for the loss and, optionally, the **Lost to Competitor** you lost to.

4. Click **Mark as Lost**. The deal moves to a Lost-type stage, its probability is set to 0, and the reason and competitor are recorded (they show in the deal's Details under Close Reason and Lost To, and feed the Analytics "Top Loss Reasons" and "Top Competitors" sections).

To delete a deal:

1. Open the deal and click the overflow menu (the "..." button).
2. Choose **Delete Deal**. The deal is soft-deleted and you return to the board. It can be restored later through the API or MCP.

Stage History and Related items

On a deal's detail page:

- **Stage History** lists each transition (from stage to stage), when it happened, who made it, and how long the deal spent in the prior stage.
- **Related** surfaces cross-app links: Bill invoices (by number and status), Book events, and Bam tasks (by their human-readable ID). Each source is best-effort and simply shows nothing if it is unavailable. If there are no links, the panel reads "No related items".

Contacts list

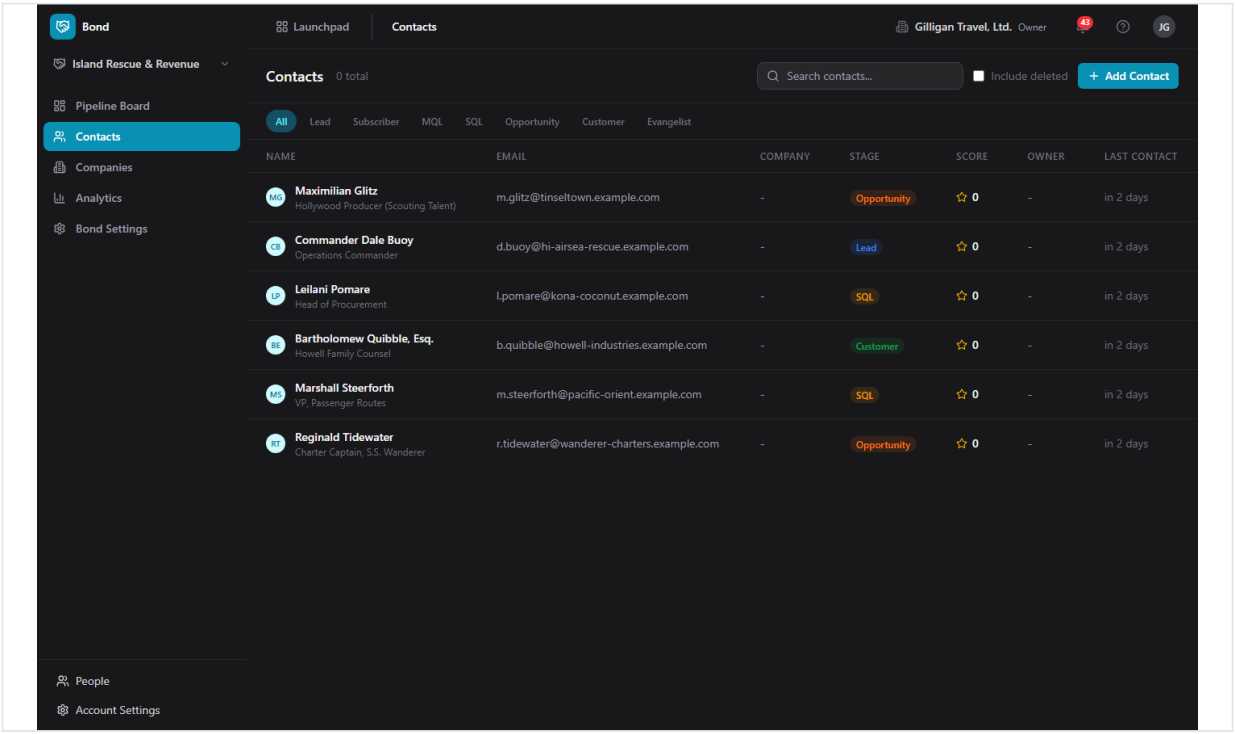


Figure 13.3 Contacts list

The Contacts list (`/contacts`) is your people directory. The header shows "Contacts", an "N total" count, a "Search contacts..." box, an **Include deleted** checkbox, and an **Add Contact** button.

To browse and filter contacts:

1. Open **Contacts** from the sidebar.
2. Click a lifecycle pill to filter: **All**, **Lead**, **Subscriber**, **MQL**, **SQL**, **Opportunity**, **Customer**, or **Evangelist**.
3. Type into "Search contacts..." to match by name, email, or phone.
4. Each row shows Name (with avatar and title), Email, Company, the lifecycle Stage badge, Score (star and number), Owner, and Last Contact.

To see deleted contacts and restore one:

1. Check **Include deleted**. Deleted rows appear struck through.
2. Click **Restore** on a deleted row to bring it back.

Creating a contact

To add a contact:

1. On the Contacts list, click **Add Contact** (also available from the empty-state "No contacts found").
2. In the "Create Contact" dialog, fill in any of **First Name**, **Last Name**, **Email**, **Phone**, and **Job Title**. At least one of first name, last name, or email is required.
3. Choose a **Lifecycle Stage** (Lead, Subscriber, MQL, SQL, Opportunity, Customer, Evangelist, or Other). It defaults to Lead.
4. Click **Create Contact**. The contact is added and you land on its detail page.

Bulk-importing contacts

Bond can import many contacts at once. Bulk import takes a JSON body with a `contacts` array (1 to 5000 records), not a spreadsheet upload, so there is no in-app CSV file picker; the import is driven through the REST API (`POST /contacts/import`) or an agent and requires admin access. Unmatched companies referenced by the records are resolved by the importer. For a single idempotent ingest by email, use the `bond_upsert_contact` tool instead (see Working with AI agents).

Contact detail

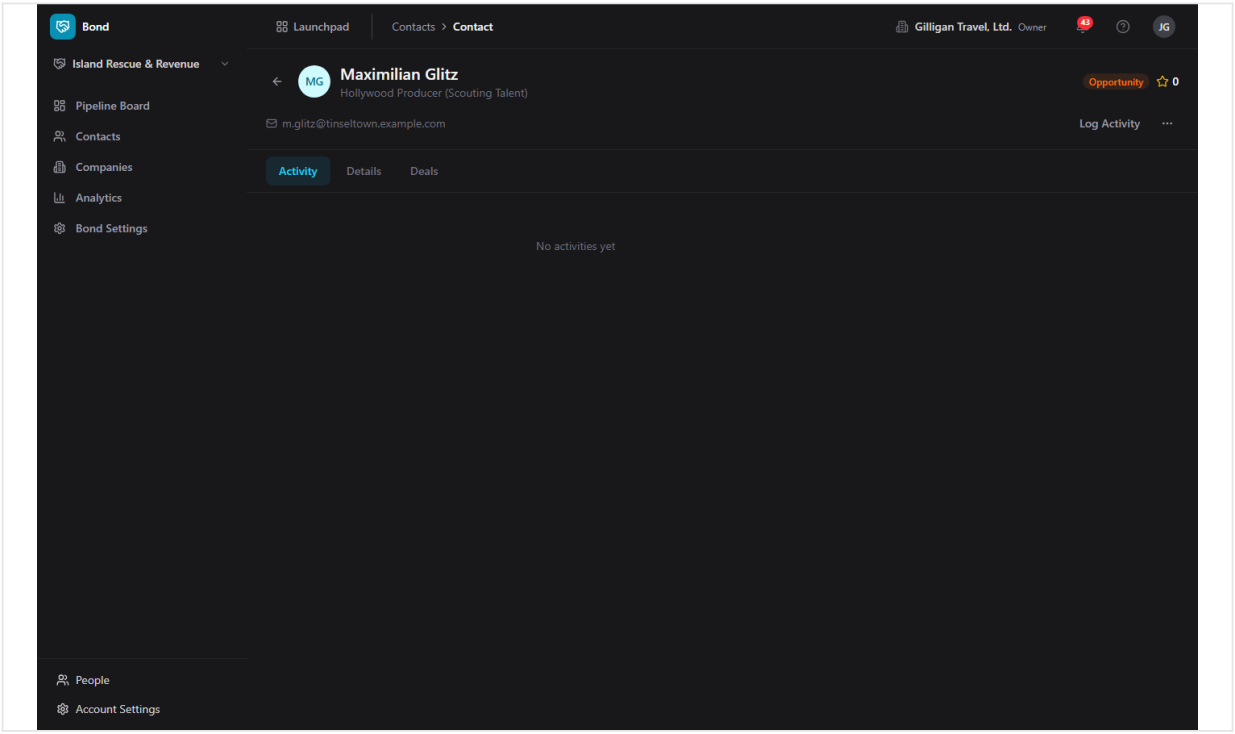


Figure 13.4 Contact detail

A contact's page (`/contacts/:id`) shows the avatar, name, title, a link to the company, the lifecycle badge, and the lead score. Below are the email (mailto), phone (tel), and deal count. Three tabs organize the body: **activity**, **details**, and **deals**.

To work a contact:

1. Open the contact.
2. Click **Log Activity** to record a note, call, meeting, or email (see Logging activity).
3. Use the **details** tab to read Lead Source, Owner, City, State/Region, Country, Created, and Last Contacted.
4. The **deals** tab shows a text summary of the contact's deals.

To edit a contact:

1. Click the overflow menu ("...") and choose **Edit Contact**. The "Edit Contact" dialog opens, pre-filled.
2. Update **First Name**, **Last Name**, **Email**, **Phone**, **Job Title**, or the **Lifecycle Stage**, then click **Save Changes**. The change is saved through the API.

To create a deal linked to this contact:

1. Click the overflow menu ("...") and choose **Create Deal**. The "Create Deal" dialog opens.
2. Type a **Deal Name**, an optional **Value ($)**, and pick a **Pipeline** (it defaults to your org's default pipeline).

3. Click **Create Deal**. The deal is created at the pipeline's first active stage and is linked to this contact.

To delete a contact:

1. Click the overflow menu ("...") and choose **Delete Contact**. The contact is soft-deleted; restore it from the Contacts list with Include deleted.

Companies list

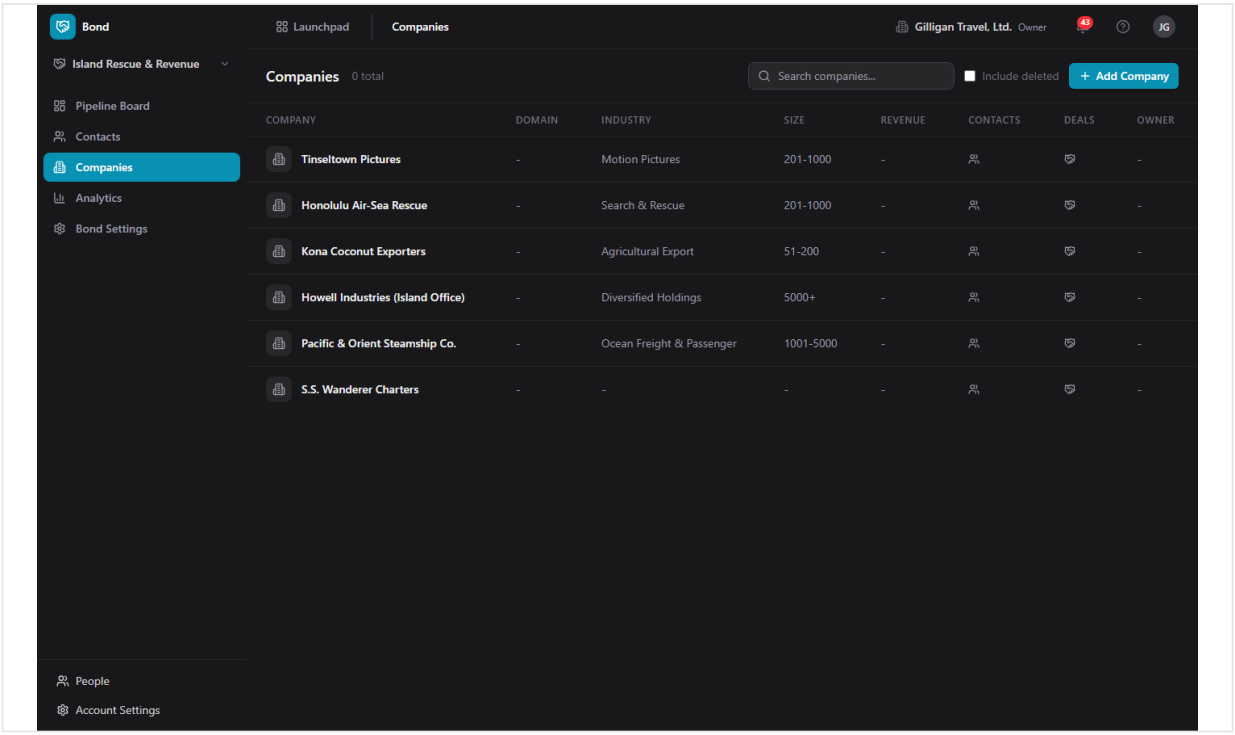


Figure 13.5 Companies list

The Companies list (`/companies`) holds the organizations you work with. The header shows "Companies", an "N total" count, a "Search companies..." box, an **Include deleted** checkbox, and an **Add Company** button.

To browse companies:

1. Open **Companies** from the sidebar.
2. Type into "Search companies..." to match by name or domain.
3. Each row shows the Company (logo and name), Domain, Industry, Size, Revenue, Contacts count, Deals count, and Owner.
4. Check **Include deleted** to see soft-deleted companies, each with a Restore action.

Creating a company

To add a company:

1. On the Companies list, click **Add Company** (also on the empty-state "No companies found").

2. In the "Create Company" dialog, type a **Company Name** (required).
3. Optionally fill in **Domain**, **Industry**, **Company Size** (a bucket: 1-10, 11-50, 51-200, 201-1000, 1001-5000, 5000+), and **Website**.
4. Click **Create Company**. The company is added and you land on its detail page.

Company detail

A company's page (`/companies/:id`) shows the logo, name, industry badge, and employee count, plus the domain link, phone, location, contact count, deal count, and revenue. Four tabs organize the body: **activity**, **details**, **contacts**, and **deals**.

To work a company:

1. Open the company.
2. Click **Log Activity** to record an activity against it.
3. Use the **details** tab to read Website, Owner, Address, and Created.
4. The **contacts** and **deals** tabs show text summaries.

To edit a company:

1. Click the overflow menu ("...") and choose **Edit Company**. The "Edit Company" dialog opens, pre-filled.
2. Update the company's fields and click **Save Changes**. The change is saved through the API.

To delete a company:

1. Click the overflow menu ("...") and choose **Delete Company**. The company is soft-deleted; restore it from the Companies list with Include deleted.

Logging activity

The Log Activity form appears on contact, company, and deal detail pages. Use it to keep a running timeline of your touches.

To log an activity:

1. On a contact, company, or deal detail page, click **Log Activity**.
2. Choose a type: **Note**, **Email Sent**, **Email Received**, **Call**, **Meeting**, or **Task**.
3. Enter a Subject and optional details in the "Add details..." box.
4. Click **Log Activity**. The entry appears in the Activity timeline with an icon and label for its type.

Analytics

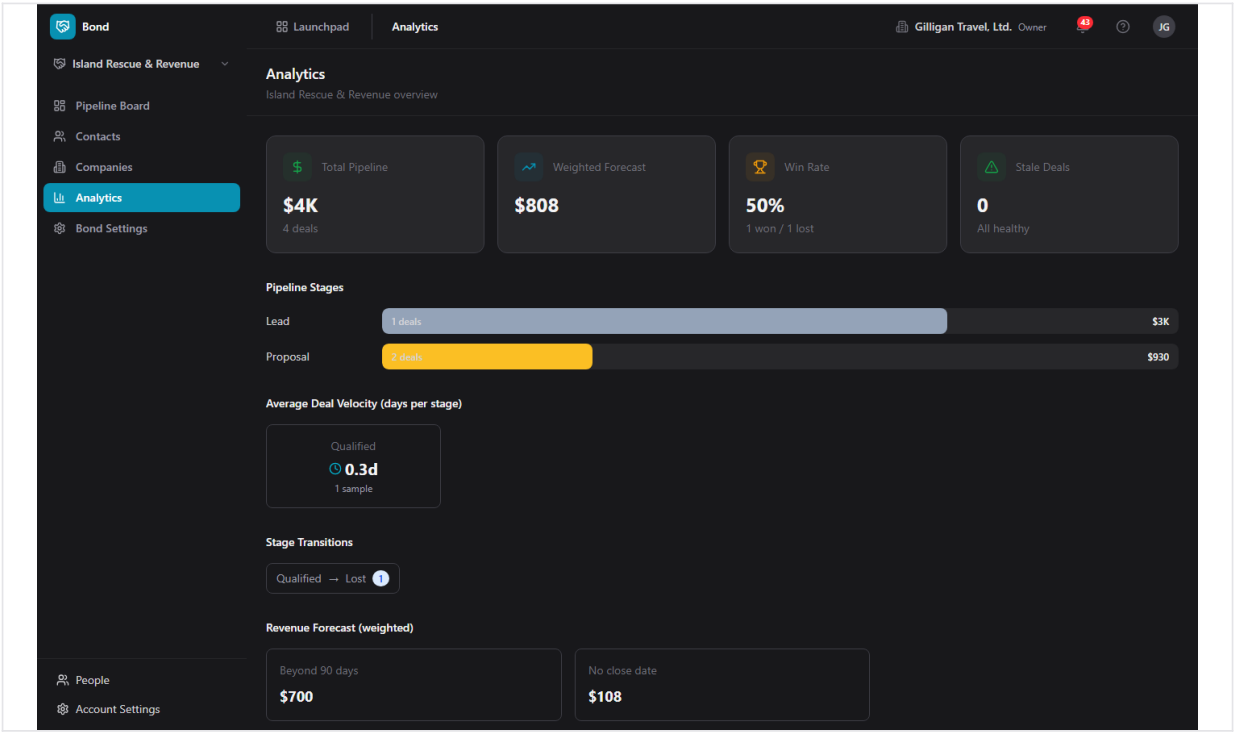


Figure 13.6 Analytics dashboard

The Analytics page (`/analytics`) reports on the effective pipeline (the active one, falling back to the default, then the first). The header reads "Analytics" with "<pipeline> overview".

To read your numbers:

1. Open **Analytics** from the sidebar.
2. The top stat cards show **Total Pipeline**, **Weighted Forecast**, **Win Rate** (a percent with won and lost counts), and **Stale Deals** ("Needs attention" or "All healthy").
3. **Pipeline Stages** shows a bar per active stage with deal count and value.
4. **Average Deal Velocity (days per stage)** shows cards with average days in each stage and the sample size.
5. **Stage Transitions** shows from-stage to to-stage with a count badge.
6. **Revenue Forecast (weighted)** breaks the weighted total into Next 30 days, Next 60 days, Next 90 days, Beyond 90 days, and No close date (only non-zero buckets show).
7. **Stale Deals (N)** lists clickable rows (name, stage, an X-of-Y days badge, value). Click a row to open that deal.
8. **Top Loss Reasons** and **Top Competitors** summarize why deals were lost and to whom (populated from the reason and competitor you enter when you mark a deal Lost).

Bond Settings: Pipelines

Settings opens at `/settings/pipelines`. The Pipelines tab lists your pipelines; expand one to see and edit its stages.

To create a pipeline:

1. Open **Bond Settings**, then the **Pipelines** tab.
2. Click **New Pipeline**.
3. In the "Create Pipeline" dialog, type a **Pipeline Name** and click **Create**.
4. The new pipeline is seeded with six default stages: Prospect (10%), Qualified (25%), Proposal (50%), Negotiation (75%), Closed Won (100%, Won), and Closed Lost (0%, Lost).

To inspect and manage stages:

1. Click a pipeline row to expand it.
2. Each stage row shows a color dot, the stage name, a type badge (Active, Won, or Lost), the probability percent, and "Nd rot" if a rotting threshold is set.
3. To add a stage, use the inline **Add stage** form: type a name, pick a type (Active, Won, or Lost), and click **Add**.
4. To delete a stage, hover the row and click the trash icon.

Note: each stage row shows a drag handle, but reordering stages is not wired in the Settings UI. Stage reordering, and editing a stage's probability, rotting days, or color, are available through the REST API or the `bond_reorder_stages` and `bond_update_stage` MCP tools. The Add stage form sets only the name and type.

Bond Settings: Custom Fields

Custom fields extend contacts, companies, or deals with extra data. They are defined per entity type and apply org-wide for that type (not per pipeline).

To add a custom field:

1. Open **Bond Settings**, then the **Custom Fields** tab.
2. Optionally narrow the list with the entity filter (All Types, Contact, Company, or Deal).
3. Click **New Field**.
4. In the "Create Custom Field" dialog, choose an **Entity Type** (Contact, Company, or Deal) and a **Field Type** (Text, Number, Date, Select, Multi Select, URL, Email, Phone, or Boolean).
5. Enter a **Label**; the **Field Key** is generated from it automatically (you can adjust it).
6. Check **Required field** if the field is mandatory.
7. For Select or Multi Select, build the **Options** list by entering a Value and Label and clicking **Add** for each option.
8. Click **Create Field**. Fields are grouped by entity type, each showing its label, key, type badge, and option count.

To delete a custom field:

1. Hover its row and click the trash icon.

Bond Settings: Lead Scoring

Lead scoring rules adjust each contact's lead score. Rules are org-wide and evaluated together; each matching enabled rule adds or subtracts its delta, and the final score is clamped to 0-100.

To create a scoring rule:

1. Open **Bond Settings**, then the **Lead Scoring** tab.
2. Click **New Rule**.
3. In the "Create Scoring Rule" dialog, enter a **Rule Name** and an optional **Description**.
4. Build the **Condition**: pick a Field (for example Lifecycle Stage, Lead Source, Job Title, City, Country, Current Score, or a custom field), an Operator (Equals, Not Equals, Contains, Greater Than, Less Than, Greater or Equal, Less or Equal, Exists, Not Exists), and a Value (the Value box hides for Exists and Not Exists).
5. Set a **Score Delta** (from -100 to 100) and leave **Enabled** checked to make the rule active.
6. Click **Create Rule**. Each rule row reads as a plain sentence, shows its +/- point delta, and offers edit and delete.

To edit a scoring rule:

1. Hover a rule row and click the edit (pencil) icon.
2. Adjust fields in the "Edit Scoring Rule" dialog and click **Save Changes**.

Note: there is no in-app button to recalculate scores. Recalculation runs through the REST endpoint or the `bond_score_lead` MCP tool (see Working with AI agents).

Restoring deleted records

Contacts, companies, and deals are soft-deleted, so a delete is recoverable.

To restore a deleted contact or company:

1. Open the **Contacts** or **Companies** list.
2. Check **Include deleted**.
3. Find the struck-through row and click **Restore**.

Deleted deals are restored through the REST API or the `bond_restore_deal` MCP tool, not from the board UI.

Working with AI agents

Bond exposes a large MCP catalog (over 70 tools across `bond-tools.ts`, plus the cross-cutting `bond_find_duplicates`), so an AI agent can do everything a salesperson does in the app, plus the admin configuration the current UI does not expose. Most write tools accept a name or a UUID (a pipeline or stage name, a contact email or name, a company name or domain, a deal title fragment, an owner email); an ambiguous or missing match returns a clean error instead of mutating data.

> ***Every contact, deal, and pipeline action is an MCP tool, so an agent can work the board with a salesperson's reach plus the admin config the UI hides.***

What agents commonly drive:

- **Contacts:** `bond_create_contact`, `bond_update_contact`, `bond_list_contacts`, `bond_get_contact`, `bond_search_contacts`, `bond_merge_contacts`, `bond_delete_contact`, and `bond_restore_contact`. `bond_update_contact` does the same edit the in-app Edit Contact dialog does, including lifecycle stage.
- **Idempotent contact ingestion:** `bond_upsert_contact` upserts by email. It is part of the platform's idempotent write plane and returns a `created` flag and an idempotency key, so repeated ingestion does not create duplicates. It also resurrects a soft-deleted contact with the same email.
- **Companies:** `bond_create_company`, `bond_update_company`, `bond_list_companies`, `bond_get_company`, `bond_search_companies`, `bond_delete_company`, `bond_restore_company`, `bond_list_company_contacts`, and `bond_list_company_deals`. `bond_update_company` matches the in-app Edit Company dialog.
- **Deals:** `bond_create_deal` (collects fields the in-app board dialog omits at creation, such as company, owner, and probability), `bond_update_deal` (the same edit the in-app Edit Deal dialog does), `bond_list_deals`, `bond_get_deal`, `bond_move_deal_stage`, `bond_close_deal_won`, and `bond_close_deal_lost` (records a close reason and the competitor you lost to, just like the in-app Mark Deal as Lost dialog). `bond_duplicate_deal`, `bond_delete_deal`, and `bond_restore_deal` round out the lifecycle, and `bond_list_deal_contacts`, `bond_add_deal_contact`, `bond_remove_deal_contact`, `bond_get_deal_stage_history`, `bond_list_deal_activities`, and `bond_get_deal_related` cover the deal detail surface.
- **Activities:** `bond_log_activity` mirrors the Log Activity form; `bond_list_activities`, `bond_get_activity`, `bond_update_activity`, and `bond_delete_activity` manage the timeline.
- **Pipelines and stages (admin):** `bond_list_pipelines`, `bond_get_pipeline`, `bond_create_pipeline`, `bond_update_pipeline`, `bond_delete_pipeline`, and the stage tools `bond_list_stages`, `bond_create_stage`, `bond_update_stage`, `bond_delete_stage`, and `bond_reorder_stages`. `bond_reorder_stages` and `bond_update_stage` do the stage reordering and the probability/rotting/color edits the Settings UI cannot.
- **Custom fields (admin):** `bond_list_custom_fields`, `bond_get_custom_field`, `bond_create_custom_field`, `bond_update_custom_field`, and `bond_delete_custom_field`.
- **Lead scoring:** `bond_list_scoring_rules`, `bond_create_scoring_rule`, `bond_update_scoring_rule`, and `bond_delete_scoring_rule` manage the rules; `bond_score_lead` recalculates a contact's score, the action the Settings UI has no button for.

- **Reporting:** `bond_get_pipeline_summary`, `bond_get_stale_deals`, `bond_get_forecast`, `bond_get_conversion_rates`, `bond_get_deal_velocity`, and `bond_get_win_loss` return the data the Analytics page renders.
- **Deduplication:** `bond_find_duplicates` returns ranked likely-duplicate contacts with confidence and signals (full-name trigram similarity, exact email match, exact normalized-phone match), and pairs with the platform `dedupe_record_decision` and `dedupe_list_pending` tools (entity type `bond.contact`) plus `bond_merge_contacts` to resolve them. There is no in-app dedupe screen; this loop is API and tool only.
- **Import mappings and settings:** `bond_create_import_mapping` and `bond_list_import_mappings` record external-system-to-Bond lookups for dedup; `bond_get_user_settings` and `bond_update_user_settings` manage per-user Bond preferences such as reply-to email.

Cross-cutting agent platform: Bond data is reachable through the platform read plane too. `search_everything` fans out across apps (including Bond) with optional asker-mode `can_access` filtering, and composite views like `account_view` assemble a contact or company picture across apps. Agents run with an identity (`users.kind` of `agent` or `service`), send `agent_heartbeat`, and are gated by `agent_policies` (per-agent kill switch plus glob tool allowlists such as `bond.*`); risky changes can be routed through the proposal queue (`proposal_create` / `proposal_decide`) for human approval, and outbound webhooks push subscribed Bolt events to agent runners. Before an agent cites a Bond entity in a shared surface, it should preflight with `can_access(asker_user_id, 'bond.contact', id)` (or the relevant entity type) and drop anything not allowed.

Event-driven automation: Bond emits Bolt events on the `bond` source for `deal.created`, `deal.updated`, `deal.stage_changed`, `deal.won`, `deal.lost`, `contact.created`, `contact.upserted`, `activity.logged`, and `deal.rotting`. A daily worker job emits `deal.rotting` for open deals that have aged past their stage's rotting threshold, so downstream Bolt rules can nudge an owner or hand off to another app.

Reviewing agent work: own-only visibility is enforced server-side, so a member or viewer service account only sees and acts on records it owns.

For the full tool catalog and schemas, see the Bond MCP-tools reference and guide in `docs/apps/bond/`.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. When you open a deal, a presence strip shows who else is on it, and you can ring a teammate into a huddle without leaving the page. Your location in Bond shows in the Bureau office. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Set up your first sales pipeline

Who: An org admin or owner setting up Bond. **Goal:** Have a working pipeline with the stages your team sells through. **Before you start:** You need admin access to Bond. A fresh org has no pipeline, so the board shows "No pipeline selected".

Steps

1. Open **Bond Settings** from the sidebar (or click **Create Pipeline** on the empty board, which takes you there).
2. On the **Pipelines** tab, click **New Pipeline**.
3. In the "Create Pipeline" dialog, type a **Pipeline Name** such as "Enterprise Sales" and click **Create**.
4. The pipeline is seeded with six stages: Prospect, Qualified, Proposal, Negotiation, Closed Won, and Closed Lost. Click the pipeline row to expand it.
5. Add any stages your team needs with the inline **Add stage** form (name plus type), and delete any you do not want with the trash icon on a stage row.

Result: A pipeline exists with its stages. Because at least one pipeline now exists, the Pipeline Board auto-selects it and renders columns instead of the "No pipeline selected" message, and you can start adding deals.

Related: Tune stage probability, rotting days, color, or order through the `bond_update_stage` and `bond_reorder_stages` MCP tools or the REST API; the Settings UI sets only stage name and type. See Pipeline board.

Story: Add and qualify a contact

Who: A salesperson or SDR capturing a new lead. **Goal:** Get a person into Bond, classify them, and start a history of touches. **Before you start:** You need read/write access. You are on the Contacts list.

Steps

1. Click **Add Contact**.
2. In the "Create Contact" dialog, fill in **First Name**, **Last Name**, **Email**, **Phone**, and **Job Title** (at least one of first name, last name, or email is required).
3. Choose a **Lifecycle Stage**, for example **Lead** or **MQL**, then click **Create Contact**.
4. On the contact's detail page, click **Log Activity**, pick **Call** or **Note**, write a subject and details, and click **Log Activity**.
5. To reclassify the contact later, open the overflow menu ("..."), choose **Edit Contact**, change the **Lifecycle Stage**, and click **Save Changes**.

Result: The contact exists with a lifecycle stage and a first activity in its timeline, and you can update its details and stage at any time.

Related: Agents can do the same with `bond_create_contact` and `bond_update_contact`. See Contacts list.

Story: Create a deal and move it to close

Who: A salesperson working an opportunity. **Goal:** Track a deal from creation through Won or Lost. **Before you start:** You need read/write access and at least one pipeline. You are on the Pipeline Board, which has the default pipeline already selected.

Steps

1. Click **Add Deal** (or the + on a specific stage column).
2. In the "Create Deal" dialog, type a **Deal Name**, an optional **Value ($)**, and an optional **Expected Close Date**, then click **Create Deal**.
3. To set a company, owner, or other fields the create dialog omits, open the deal, use the overflow menu's **Edit Deal**, and **Save Changes**.
4. As the deal progresses, drag its card from one stage column to the next. Each move is recorded in Stage History.
5. Open the deal and use **Log Activity** to record calls and meetings along the way.
6. When the deal closes, open it and click **Won**, or click **Lost** and enter a **Reason** (and optional **Lost to Competitor**) in the "Mark Deal as Lost" dialog.

Result: The deal shows an **Open**, **Won**, or **Lost** badge, its probability is set (100 for won, 0 for lost), and a lost deal records its reason and competitor for reporting.

Related: Agents can create deals with more fields up front via `bond_create_deal`, edit them with `bond_update_deal`, and close them with `bond_close_deal_won` / `bond_close_deal_lost`. See Deal detail.

Story: Work the board with swimlanes

Who: A sales manager scanning the pipeline. **Goal:** See deals grouped by who owns them or when they close, and spot the ones going stale. **Before you start:** You are on the Pipeline Board with deals in the active pipeline.

Steps

1. In the board header, use the **Group** control and click **Owner** to lane deals by owner, or **Close month** to lane them by close date.
2. Type into "Search deals..." to narrow to a name or company.
3. Scan for orange cards (rotting) and red cards (severely overdue, more than 1.5 times past the rotting threshold).
4. Click **None** to return to the flat board.

Result: The board is reorganized into swimlanes and you can see, at a glance, which owner or month is loaded and which deals are aging.

Related: Triage stale deals (next story) for a focused list.

Story: Triage stale deals

Who: A sales manager or rep keeping deals from rotting. **Goal:** Find deals stuck too long in a stage and re-engage them. **Before you start:** You are signed in with access to the pipeline's deals.

Steps

1. Open **Analytics** from the sidebar.
2. Read the **Stale Deals** stat card ("Needs attention" or "All healthy").
3. Scroll to the **Stale Deals (N)** section and click a row to open that deal.
4. On the deal, click **Log Activity** and record a follow-up call or note to re-engage it.

Result: Each stale deal is reviewed and gets a fresh touch, which resets its activity history.

Related: Agents can pull the same list with `bond_get_stale_deals`. A daily worker also emits `deal.rotting` to Bolt so automation rules can nudge owners. See Analytics dashboard.

Story: Forecast revenue

Who: A sales leader planning the quarter. **Goal:** Understand expected revenue and where deals are being lost. **Before you start:** You have a pipeline with open and closed deals.

Steps

1. Open **Analytics** from the sidebar.
2. Read **Weighted Forecast** and **Win Rate** at the top.
3. Review the **Revenue Forecast (weighted)** buckets: Next 30 days, Next 60 days, Next 90 days, Beyond 90 days, and No close date.
4. Read **Average Deal Velocity (days per stage)** to see where deals slow down, and **Top Loss Reasons** and **Top Competitors** to see why and to whom you lose.

Result: You have a weighted forecast, a win rate, velocity per stage, and a breakdown of losses.

Related: Agents can fetch the same figures with `bond_get_forecast`, `bond_get_pipeline_summary`, `bond_get_deal_velocity`, and `bond_get_win_loss`.

Story: Manage companies

Who: An account manager organizing accounts. **Goal:** Create a company, edit its details, and tie contacts and deals to it. **Before you start:** You need read/write access. You are on the Companies list.

Steps

1. Click **Add Company**.
2. In the "Create Company" dialog, type a **Company Name**, and optionally a **Domain**, **Industry**, **Company Size**, and **Website**, then click **Create Company**.
3. On the company's detail page, open the overflow menu ("...") and choose **Edit Company** to update its fields; click **Save Changes**.
4. Use the **contacts** and **deals** tabs to see linked records (shown as summaries), and **Log Activity** to record touches against the account.

Result: The company exists with its linked contact and deal counts, its details are editable in place, and its activity timeline is started.

Related: Agents can manage the same record with `bond_create_company` and `bond_update_company`. See Companies list.

Story: Configure lead scoring

Who: A marketing or sales ops admin. **Goal:** Automatically prioritize contacts by scoring their attributes. **Before you start:** You need admin access to Bond.

Steps

1. Open **Bond Settings**, then the **Lead Scoring** tab.
2. Click **New Rule**.
3. Enter a **Rule Name**, build the **Condition** (Field, Operator, Value), set a **Score Delta** between -100 and 100, and keep **Enabled** checked.
4. Click **Create Rule**. Repeat for each scoring signal you want.
5. To change a rule later, hover its row, click the edit icon, adjust it, and click **Save Changes**.

Result: Enabled rules combine to produce each contact's lead score, clamped to 0-100. The score shows as a star and number on contact rows and detail pages.

Related: There is no recalculate button in the app; trigger recalculation with the `bond_score_lead` MCP tool or the `/scoring/recalculate` REST endpoint. Agents can also manage the rules themselves with `bond_create_scoring_rule` and `bond_update_scoring_rule`.

Story: Extend the schema with custom fields

Who: An admin who needs to capture data Bond does not have a built-in field for. **Goal:** Add a custom field to contacts, companies, or deals. **Before you start:** You need admin access to Bond.

Steps

1. Open **Bond Settings**, then the **Custom Fields** tab.
2. Click **New Field**.
3. Choose the **Entity Type** and **Field Type**, enter a **Label** (the **Field Key** auto-fills), and check **Required field** if needed.

4. For Select or Multi Select fields, add **Options** by entering a Value and Label and clicking **Add** for each.
5. Click **Create Field**.

Result: The new field appears grouped under its entity type. Custom fields are org-wide for that entity type, not tied to a single pipeline.

Related: Agents can manage the same definitions with `bond_create_custom_field`, `bond_update_custom_field`, and `bond_delete_custom_field`.

Story: Restore a deleted record

Who: Anyone who deleted a contact or company by mistake. **Goal:** Bring back a soft-deleted record. **Before you start:** You are on the Contacts or Companies list.

Steps

1. Check **Include deleted** in the list header.
2. Find the struck-through row for the record you deleted.
3. Click **Restore** on that row.

Result: The record returns to the active list. (Deleted deals are restored with the `bond_restore_deal` MCP tool or the REST API rather than the board.)

Story: Deduplicate contacts (agent-assisted)

Who: An ops admin or an AI agent cleaning the contact database. **Goal:** Find and merge duplicate contacts. **Before you start:** This flow runs through MCP tools and the REST API; there is no in-app dedupe screen.

Steps

1. Run `bond_find_duplicates` (or call `GET /contacts/:id/duplicates`) to get ranked likely duplicates with confidence and signals.
2. Review the candidates and record a decision with the platform `dedupe_record_decision` tool.
3. When two records are truly the same person, merge them with `bond_merge_contacts` (or `POST /contacts/:id/merge`). The target absorbs the source's deals, activities, and company links, and the source is soft-deleted.

> **WARNING**
> Merging contacts is one-directional and consolidating: the target keeps the source's deals, activities, and company links, and the source is then soft-deleted. Confirm you have the right target before you merge, and remember the merge loop runs only through MCP tools and the REST API, not an in-app screen.

Result: Duplicate contacts are consolidated into a single record, and your decision is recorded so the pair is not re-flagged.

Related: This is the platform dedup loop for entity type `bond.contact`. Pending pairs are listed with `dedupe_list_pending`.

Story: Hand a deal off across the suite

Who: A rep coordinating delivery after a win. **Goal:** See and follow the invoices, events, and tasks tied to a deal. **Before you start:** You are on a deal's detail page, and related records exist in Bill, Book, or Bam.

Steps

1. Open the deal.
2. In the right-side **Related** panel, review linked Bill invoices (with number and status), Book events, and Bam tasks (with their human-readable IDs).
3. Click any related item to jump to it in the other app.

Result: You can move from the deal straight to the invoice, meeting, or task that continues the work, without searching for it.

Related: Bond contacts also feed Blast email segments, and Bond data is queryable in Bench reports.

Related

- **Bill** - invoices linked to a deal appear in its Related panel.
- **Book** - scheduled events linked to a deal appear in its Related panel.
- **Bam** - project tasks linked to a deal appear in its Related panel, by human-readable ID.
- **Blast** - pulls Bond contacts into email segments.
- **Bench** - reports and dashboards can query Bond data.
- **Bolt** - reacts to Bond events (`deal.created`, `deal.updated`, `deal.stage_changed`, `deal.won`, `deal.lost`, `contact.created`, `contact.upserted`, `activity.logged`, `deal.rotting`) on the `bond` source.
- Bond MCP-tools reference and guide in `docs/apps/bond/`.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What are the four core objects Bond organizes your sales work around?

a) Contacts, companies, deals, and pipelines
b) Leads, accounts, opportunities, and tasks
c) Contacts, deals, invoices, and events
d) People, companies, activities, and reports

2. What lifecycle stage do new contacts default to?

a) Subscriber
b) Lead
c) MQL
d) Opportunity

3. How is a deal's weighted value computed?

4. What is the range a contact's lead score is clamped to?

a) 0 to 10
b) 0 to 100
c) 1 to 5
d) -100 to 100

5. When does a deal card turn red rather than just orange?

a) When it has no owner assigned
b) When it has been open for more than 90 days
c) When it is more than 1.5 times over its stage's rotting threshold
d) When its probability drops below 25 percent

6. What are the three stage types a pipeline stage can have?

7. Which six stages does a new pipeline get seeded with?

a) Prospect, Qualified, Proposal, Negotiation, Closed Won, and Closed Lost
b) Lead, MQL, SQL, Opportunity, Customer, and Evangelist
c) New, Open, Working, Pending, Won, and Lost
d) Discovery, Demo, Quote, Contract, Won, and Lost

8. When you close a deal as Won, what does Bond set its probability to, and what happens when you close it as Lost?

9. Can members and viewers see every deal and contact in the org?

a) Yes, everyone sees everything
b) No, members and viewers see only the deals and contacts they own
c) Only if an admin grants them access per record
d) They see all contacts but only their own deals

10. When you delete a contact, company, or deal in Bond, is the record erased? How do you bring a contact or company back?

11. How is a bulk contact import driven, since there is no in-app CSV file picker?

a) By uploading a spreadsheet on the Contacts list
b) Through the REST API with a JSON contacts array (1 to 5000 records) or an agent, with admin access

c) By pasting rows into the Create Contact dialog
d) Through the Bond Settings import wizard

12. Custom fields are defined per entity type. Are they scoped per pipeline or org-wide?

13. Which Bond action has no in-app button and runs only through the REST endpoint or an MCP tool?

a) Creating a deal
b) Editing a contact's lifecycle stage
c) Recalculating a contact's lead score
d) Logging an activity

CHAPTER 14

Book

Set your hours, share a link, fill your calendar.

Book is scheduling without setup. The first time you open it, a personal calendar appears and you can start adding events, while behind the scenes Book turns your working hours into free time slots that power public booking pages and agent-driven meeting-finding. This chapter walks you from your weekly hours to a shared booking link, takes you through the cross-app Timeline that pulls Bam tasks and Bond deals alongside your events, and shows where an AI agent can do the scheduling math that has no human screen yet. By the end you will be able to publish a link an outside visitor can book, and know exactly which controls are live and which are still being finished.

At a glance

- A personal calendar provisioned automatically, with Week, Day, and Month views
- Working hours that compute free slots for availability and public booking pages
- Public booking links at /book/meet/<slug> that outsiders can use without logging in
- A cross-app Timeline that gathers Book events, Bam task due dates, and Bond deal close dates
- An agent can find a common free time across several people, the math Book has no screen for

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Week view . Day view . Month view . Timeline view . New Event and Edit Event . Event detail and RSVP

> *Book is the BigBlueBam scheduling app. Use it to keep personal and team calendars, view what is happening across the suite on a timeline, set the working hours that drive availability, and publish public links that let outside people book time with you.*

Overview

Book gives every BigBlueBam user a calendar[137] without any extra setup. The first time you open it, a personal calendar named **My Calendar** is created for you automatically, so you can start adding events right away. You can add more calendars later to separate work streams, projects, or teams.

Beyond plain events, Book computes availability. It takes the working hours[138] you define, subtracts the events that mark you as busy, and produces free time slots. Those slots power two things: public booking[139] pages, where someone outside your org can grab a slot on your calendar, and the availability tools that AI agents use to find a meeting time across several people.

> ***Working hours minus busy events equals availability, and that availability powers every booking page you publish.***

Book also pulls other apps into one place. The Timeline[140] view aggregates Book events with Bam task due dates and Bond deal close dates, so you can scan a week across the whole suite. Events can link to a Bam task, a Bond deal, or a Helpdesk ticket, and every Book-native event[141] gets a LiveKit huddle room so attendees have a place to meet by video.

You can also pull an outside calendar into Book. The Connections page lets you subscribe to any public iCalendar (`.ics`) feed; Book fetches it, mirrors its events onto a Book calendar, and keeps them in step on a recurring schedule. Google Calendar and Microsoft Outlook two-way sync are built on the same engine and ready to drop in, but they stay disabled until an operator supplies the OAuth credentials; the `.ics` path needs nothing.

A few areas of Book are still being finished. Where a screen exists but does not yet do what its label implies, this document says so plainly under the relevant feature so you do not lose time on a path that cannot complete today. The remaining one is reminders and recurrence. Read that note before relying on reminder and repeat behavior.

---

[137] **Calendar.** A container for events. Each calendar has a type (`personal`, `team`, `project`, `booking`, or `bureau`), a color, and a timezone. One personal calendar, **My Calendar**, is created for you on first use and cannot be deleted. The `bureau` type is reserved for a system-provisioned, per-org calendar that other apps write room bookings into; you do not create it by hand.

[138] **Working hours.** Your weekly availability windows, one row per day of the week. By default Monday to Friday, 09:00 to 17:00, are enabled. These windows are the raw material for availability and booking-page slots.

[139] **Booking.** The event created when a visitor books a slot on a booking page. It lands as a confirmed event on the page owner's default calendar.

[140] **Timeline.** A cross-app, date-ranged view that gathers Book events, Bam task due dates, and Bond deal close dates into one per-day list.

[141] **Event.** A time block on a calendar with a title, start and end time, optional location, description, and meeting URL. Events have a status (`tentative`, `confirmed`, or `cancelled`; new events default to `confirmed`) and a visibility (`free`, `busy`, `tentative`, or `out_of_office`; default `busy`). Cancelling an event is a soft delete: it flips status to `cancelled` rather than erasing the row.

Key concepts

- **Calendar** - A container for events. Each calendar has a type (`personal`, `team`, `project`, `booking`, or `bureau`), a color, and a timezone. One personal calendar, **My Calendar**, is created for you on first use and cannot be deleted. The `bureau` type is reserved for a system-provisioned, per-org calendar that other apps write room bookings into; you do not create it by hand.
- **Event** - A time block on a calendar with a title, start and end time, optional location, description, and meeting URL. Events have a status (`tentative`, `confirmed`, or `cancelled`; new events default to `confirmed`) and a visibility (`free`, `busy`, `tentative`, or `out_of_office`; default `busy`). Cancelling an event is a soft delete: it flips status to `cancelled` rather than erasing the row.
- **Show as (visibility)** - Controls whether an event blocks your availability. An event marked **Free** does not consume a slot; **Busy**, **Tentative**, and **Out of Office** do.
- **Attendee**[142] - A person invited to an event, identified by email. Attendees carry a response status (`needs_action` until they answer, then `accepted`, `declined`, or `tentative`) and a flag for the organizer.
- **RSVP**[143] - An attendee's response to an event. Only someone who is already an attendee can RSVP.
- **Working hours** - Your weekly availability windows, one row per day of the week. By default Monday to Friday, 09:00 to 17:00, are enabled. These windows are the raw material for availability and booking-page slots.
- **Availability slot**[144] - A computed free interval, equal to your working hours minus the events that mark you busy.
- **Booking page**[145] - A public scheduling link at `/book/meet/<slug>` that lets an outside visitor pick one of your free slots. It carries a duration, before and after buffers, advance and notice limits, and brand styling.
- **Booking** - The event created when a visitor books a slot on a booking page. It lands as a confirmed event on the page owner's default calendar.
- **Timeline** - A cross-app, date-ranged view that gathers Book events, Bam task due dates, and Bond deal close dates into one per-day list.
- **External connection**[146] - A subscription to an outside calendar that Book pulls events from. The `.ics` feed provider works today with no credentials: paste a public iCalendar URL and Book mirrors its events onto a Book calendar, updating in place on each sync.

[142] **Attendee.** A person invited to an event, identified by email. Attendees carry a response status (`needs_action` until they answer, then `accepted`, `declined`, or `tentative`) and a flag for the organizer.

[143] **RSVP.** An attendee's response to an event. Only someone who is already an attendee can RSVP.

[144] **Availability slot.** A computed free interval, equal to your working hours minus the events that mark you busy.

[145] **Booking page.** A public scheduling link at `/book/meet/<slug>` that lets an outside visitor pick one of your free slots. It carries a duration, before and after buffers, advance and notice limits, and brand styling.

[146] **External connection.** A subscription to an outside calendar that Book pulls events from. The `.ics` feed provider works today with no credentials: paste a public iCalendar URL and Book mirrors its events onto a Book calendar, updating in place on each sync. Google Calendar and Microsoft Outlook connections use the same sync engine but require operator-supplied OAuth credentials before they can be created (see Connections below).

Google Calendar and Microsoft Outlook connections use the same sync engine but require operator-supplied OAuth credentials before they can be created (see Connections below).

- **Synced (mirrored) event**[147] - A Book event that Book created from an external feed. It is kept in step with the feed automatically: a re-sync updates it in place, an event removed upstream is removed from Book, and disconnecting the feed removes all of its mirrored events.
- **iCal feed**[148] - A token-authenticated `.ics` URL that exposes one calendar to an outside calendar app. Available through the API and agents only; there is no button for it in the app yet.

> **TIP**
> Google and Outlook two-way sync stay disabled until an operator supplies OAuth credentials, but the .ics path needs nothing. Most calendar apps, including Google and Outlook, can publish a public .ics URL for any calendar you own, so paste that and Book mirrors it in with no setup.

Where to find it

Book is served at `/book/`. Reach it from the BigBlueBam Launchpad, or go to the path directly. Book does not have its own login screen. If you are not signed in, Book shows a "Please log in to BigBlueBam first to access Book." panel with a **Go to BigBlueBam Login** link to the main app at `/b3/`. Sign in there, then return to Book.

Prerequisites:

- A BigBlueBam account and an active session (sign in at `/b3/`).
- No calendar setup is required. Your personal **My Calendar** is provisioned the first time Book reads your calendars.
- Write actions (creating or editing events, calendars, and booking pages) require the `read_write` scope and the matching per-action permission (for example `book.event.create`). If a write is blocked, you lack the scope or permission for that resource.

Press **?** anywhere in Book (when you are not typing in a field) to open the in-app Help.

[147] **Synced (mirrored) event.** A Book event that Book created from an external feed. It is kept in step with the feed automatically: a re-sync updates it in place, an event removed upstream is removed from Book, and disconnecting the feed removes all of its mirrored events.

[148] **iCal feed.** A token-authenticated `.ics` URL that exposes one calendar to an outside calendar app. Available through the API and agents only; there is no button for it in the app yet.

Feature reference

The left sidebar has two groups. The top group holds the calendar and scheduling views: **Week**, **Day**, **Month**, **Timeline**, and **Booking Pages**. Below the **Book Settings** header sit **Calendars**, **Working Hours**, and **Connections**.

Week view

The default landing screen. It shows a 7-day grid with a time gutter and 24 hour rows, with event blocks placed by their start and end times.

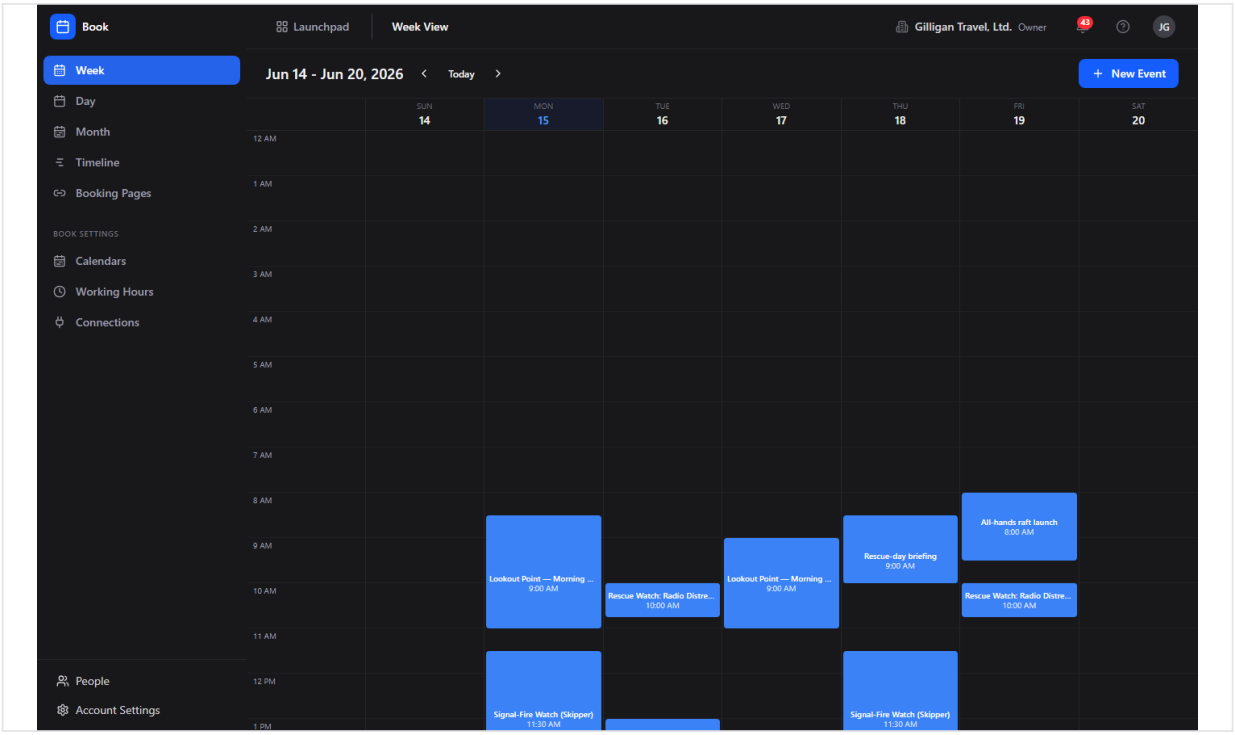


Figure 14.1 Week view

To use the Week view:

1. Open Book. You land on **Week** by default, or click **Week** in the sidebar.
2. Read the date range heading at the top (for example "Jun 9 - Jun 15, 2026").
3. Move between weeks with the left and right chevron arrows, or click **Today** to jump back to the current week.
4. Click an event block to open it. Note that clicking a block opens the event's **edit** form, not its read-only detail view.
5. Click **New Event** (top right, with a plus icon) to create an event.

Day view

A single day shown as one hourly column. Reached by **Day** in the sidebar.

To use the Day view:

1. Click **Day** in the sidebar.
2. Read the heading for the day shown.
3. Use the left and right chevron arrows to change day, or click **Today** to return to today.
4. Click **New Event** to add an event.

Month view

A full-month grid of day cells with event chips.

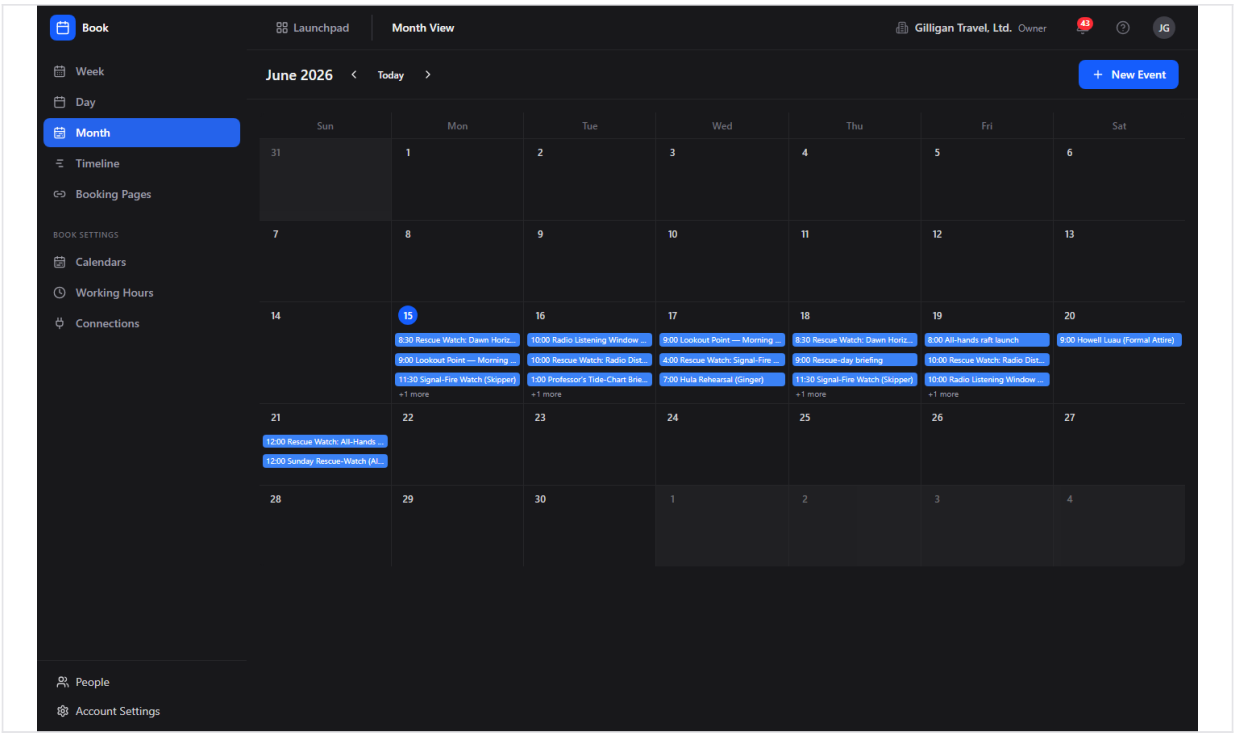


Figure 14.2 Month view

To use the Month view:

1. Click **Month** in the sidebar.
2. Read the month heading (for example "June 2026").
3. Use the left and right chevron arrows to change month.
4. Click an event chip to open that event (this opens the event edit form).
5. Click **New Event** to add an event.

Note: Book's calendar views do not support drag-to-create or drag-to-resize. Create and change events only through the **New Event** form and the edit form.

Timeline view

A cross-app view that groups items by day across a week. It gathers Book events (blue), Bam task due dates (orange), and Bond deal close dates (teal), with a legend reading "Book Events / Bam Tasks / Bond Deals".

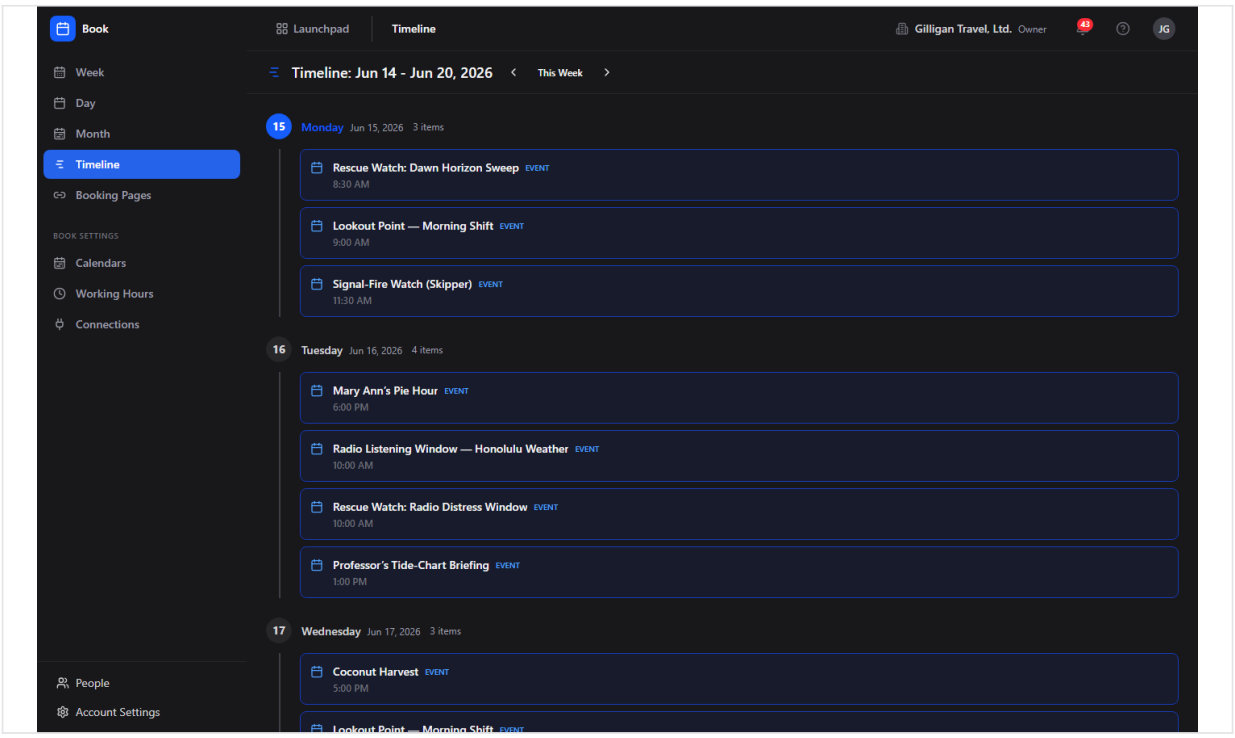


Figure 14.3 Cross-app timeline

To use the Timeline:

1. Click **Timeline** in the sidebar.
2. Read the heading (for example "Timeline: Jun 9 - Jun 15, 2026").
3. Move between weeks with the left and right chevron arrows, or click **This Week** to return to the current week.
4. Scan the per-day grouping; each item carries a source label (Event, Task, or Deal) and, for timed items, its time.
5. Click a Book item to open that event (this opens the event edit form).

The Timeline reads Book events, Bam task due dates, and Bond deal close dates from the API and renders each under the right source style, with timed items showing their time. Bam and Bond items appear with their own Task and Deal styling. To read the same data with an agent, use the `book_get_timeline` MCP tool.

New Event and Edit Event

The form for creating a new event or changing an existing one. Reached by **New Event** from any calendar view (which opens "New Event"), or by opening an event and choosing **Edit** (which opens "Edit Event").

To create an event:

1. From any calendar view, click **New Event**. The form opens with the heading **New Event**.
2. Enter a **Title** (required; placeholder "Team standup").

3. Choose a **Calendar** (required) from the dropdown. The form preselects your default calendar.
4. Toggle **All day** if the event has no specific time. When off, set **Start** and **End**. When on, set **Start Date** and **End Date**.
5. Optionally add a **Description** (placeholder "Add details, agenda, or notes...") and a **Location** (placeholder "Conference room, address, or video URL").
6. Optionally set **Repeat** (No repeat, Daily, Weekly, Every 2 weeks, Monthly) and **Show as** (Busy, Free, Tentative, Out of Office).
7. Click **Create Event**. If anything is missing you will see "Title is required", "Select a calendar", or "End must be after start".

To edit an event:

1. Open an event and click **Edit**, or click the event in a calendar view (which goes straight to this form). The heading reads **Edit Event**.
2. Change any field.
3. Click **Update Event**. Use **Cancel** or **Back to Event** to leave without saving.

Notes and current limitations:

- The form shows a **Reminder** dropdown and a **Color** picker, but neither is sent when you save. Book has no reminder feature and no reminder job, and event color is not stored on the event (color lives on the calendar). Treat both controls as inactive.
- You cannot add or edit attendees from this form. Attendees are set when an event is created through the API or an agent tool (`book_create_event`).
- **Repeat** is stored but not expanded. Choosing a repeat option records the rule on the single event; Book does not yet generate the future occurrences, so a "weekly" event remains one event.

Event detail and RSVP

The read-only detail view of an event, reached by visiting an event's direct URL (`/book/events/<id>`). It shows the event's status pill and visibility label, time and timezone, location, meeting URL link, and description.

To view an event and respond to it:

1. Open the event detail page. A **Back to Calendar** link sits at the top.
2. Review the details card: time and timezone, the **Location** line, the meeting URL link, and the description.
3. If you are an attendee, find the **Your response** panel and click **Accept**, **Maybe**, or **Decline**. "Maybe" records a tentative response.
4. See the **Attendees (N)** list for each person's response status; the organizer's row is marked "(organizer)".
5. If the event came from a booking page, an amber **Booked via scheduling link** panel appears with the booker's name and email.

6. Click **Edit** to change the event, or **Cancel** (trash icon) and confirm "Cancel this event?" to soft-cancel it.

Notes:

- RSVP only works if you are already an attendee. If you are not on the attendee list, the response is rejected.
- The calendar and Timeline views link straight to the edit form, so the detail view is reached mainly by opening an event's direct URL.
- Cancelling an event sets its status to `cancelled`; it then drops out of availability and the timeline rather than being deleted.

Booking Pages list

Your public scheduling links. The heading reads "Booking Pages" with the subtitle "Public scheduling links for clients and prospects". You only ever see your own pages here.

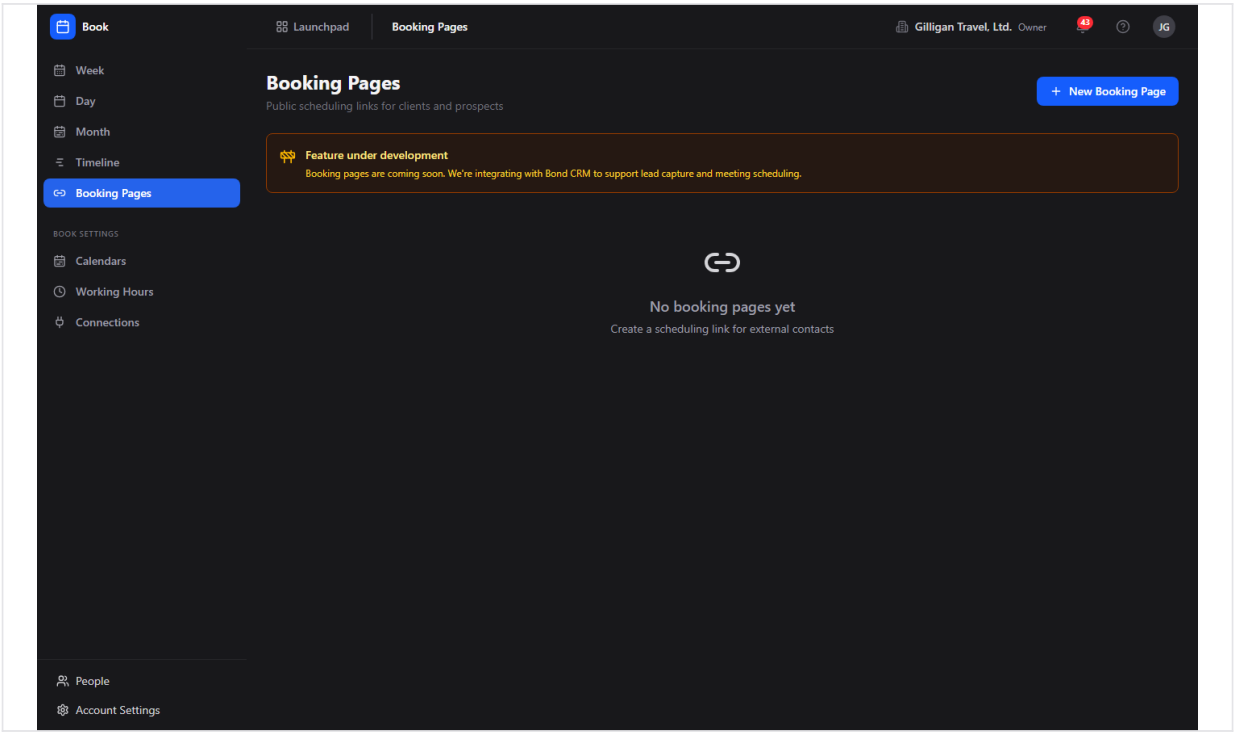


Figure 14.4 Booking pages

To manage booking pages:

1. Click **Booking Pages** in the sidebar.
2. Each page row shows its title, an **Active** or **Disabled** pill, its `/meet/<slug>` link, its duration ("{n} min"), its creation date, and its description.
3. Click **New Booking Page** to create one.
4. Click **Edit** on a page to change it, or the trash icon to delete it (confirm "Delete this booking page?").

Note: the list still renders an amber **Feature under development** banner that reads "Booking pages are coming soon. We're integrating with Bond CRM to support lead capture and meeting scheduling." That banner is stale. The page list, creation, editing, deletion, the public booking flow, and the on-booking Bond contact creation are all shipped and working. The screenshot above includes this banner.

Booking page editor

The form for creating or editing a booking page. Reached by **New Booking Page** ("New Booking Page" heading) or **Edit** ("Edit Booking Page" heading).

To create a booking page:

1. Click **New Booking Page**.
2. Enter a **Title** (placeholder "30-Minute Intro Call").
3. Enter a **URL Slug** after the "/meet/" prefix (placeholder "intro-call"). Slugs use lowercase letters, numbers, and hyphens only, and must be unique within your org. A duplicate slug is rejected with "Slug already in use".
4. Optionally add a **Description** and adjust **Duration (min)**, **Buffer Before (min)**, **Buffer After (min)**, and **Brand Color**.
5. Click **Create**. Use **Cancel** or **Back to Booking Pages** to leave.

To edit an existing page:

1. From the Booking Pages list, click **Edit** on the page you want.
2. The form opens with the heading **Edit Booking Page** and loads the page's current title, slug, description, duration, buffers, and brand color.
3. Change any field and click **Update**.

Current limitations:

- The editor exposes only title, slug, description, duration, buffers, and brand color. The booking page model also supports advance limit, minimum notice, confirmation message, redirect URL, logo, an enabled flag, and the cross-app flags `auto_create_bond_contact`, `auto_create_bam_task`, and `bam_project_id`. None of these are surfaced in the editor today, so to change them use the `book_update_booking_page` MCP tool. Because `auto_create_bond_contact` defaults to on at the data layer, new pages create a Bond contact on each booking by default even though there is no toggle for it in the editor.

> **NOTE**
> The booking-page editor exposes only title, slug, description, duration, buffers, and brand color. Because auto_create_bond_contact defaults to on at the data layer and the editor has no toggle for it, every new page creates or updates a Bond contact by email on each booking. To change that, set the flag with the book_update_booking_page MCP tool.

Public Meet page

The public scheduling page a visitor sees at `/book/meet/<slug>`. It does not require login. It shows the page title, "with {owner}", the duration, and the description, then a **Pick a time** section that groups available slots by day, then a **Your details** form with **Full name** (required), **Email** (required), and **Notes (optional)**, and a submit button labelled "Book {time}" (or "Pick a time above" until a slot is chosen). On success it shows a **Booking confirmed** screen with the page's confirmation message.

To book time as a visitor:

1. Open the page's `/book/meet/<slug>` link. The available times for the next two weeks load under **Pick a time**, grouped by day.
2. Click a time slot to select it.
3. Fill in **Full name** and **Email** (both required), and an optional note in **Notes (optional)**.
4. Click **Book {time}**.
5. The page shows the **Booking confirmed** screen with the owner's confirmation message. If the page has a redirect URL configured, you are sent there instead.

The slots offered come from the owner's working hours minus the events that mark them busy, cut into meeting-sized chunks that respect the page's duration and before and after buffers. Booking creates a confirmed event on the owner's default calendar. Two visitors cannot grab the same slot: the booking is guarded by a row lock, and a collision returns a 409 with "This time slot is no longer available."

> **TRY IT**
> Set your Working Hours, create a booking page with a unique lowercase slug, then open its /book/meet/<slug> link in a private window and book yourself a slot. Watch the confirmed event land on your default calendar.

When a booking comes in and the page has `auto_create_bond_contact` on (the default), Book creates or updates a matching Bond contact by email, attributed to the booking-page owner and tagged with a `booking_page` lead source. Re-booking with the same email does not create a duplicate contact. The Bam-task hook on booking only runs when a page explicitly enables `auto_create_bam_task` and sets a `bam_project_id`, which the editor does not expose; set those with `book_update_booking_page` if you want each booking to spawn a follow-up task.

Calendars

Manage the calendars you organize events into. Found under **Book Settings > Calendars**. The heading reads "Calendars" with the subtitle "Manage the calendars you use to organize events".

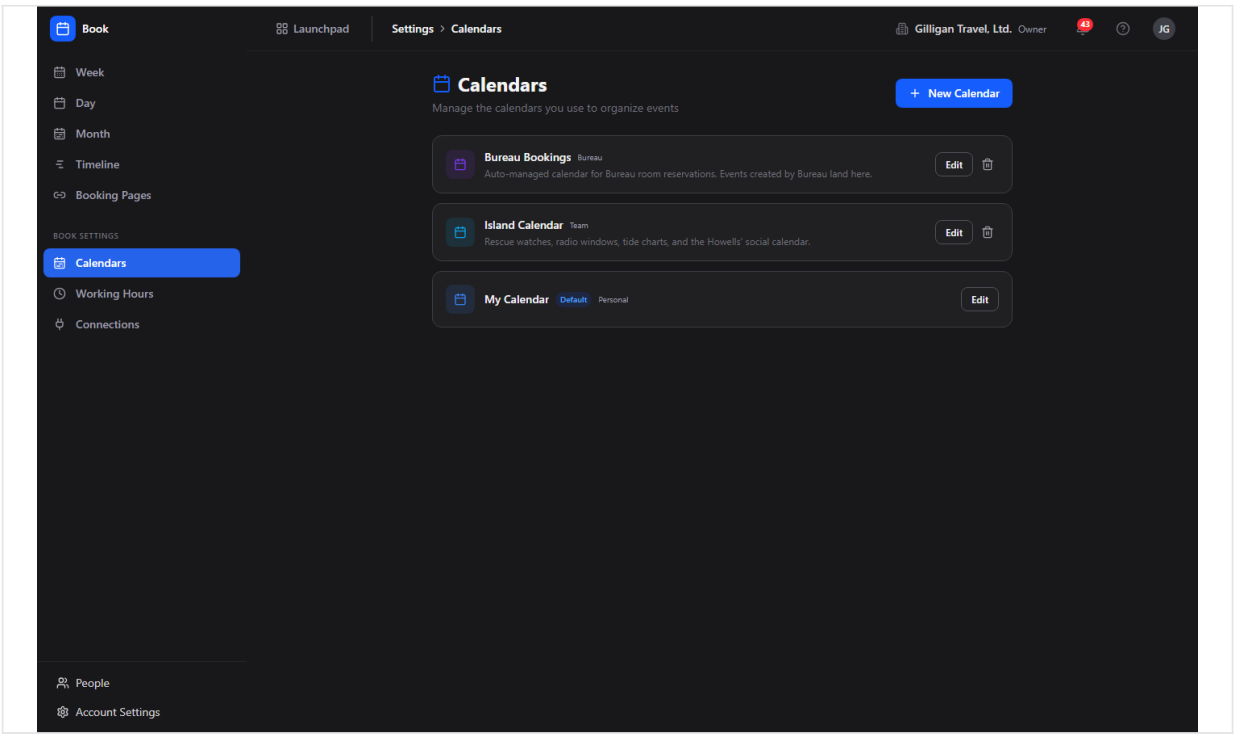


Figure 14.5 Calendars

To manage calendars:

1. Click **Calendars** under **Book Settings**.
2. Each calendar row shows its name, a **Default** pill where applicable, its type (lowercased), and its description.
3. Click **New Calendar** to open the inline **New calendar** form. Enter a name, pick a color from the preset swatches, and click **Create** (or **Cancel**).
4. To change a calendar, click **Edit** on its row, adjust the name and color, and click the check (save) button.
5. To remove a calendar, click the trash icon and confirm "Delete calendar [name]?". The default calendar cannot be deleted and has no trash icon; an attempt at the API returns "Cannot delete default calendar".

Note: your personal **My Calendar** is created automatically the first time this screen loads, so the list is never empty.

Working Hours

Set the weekly hours that drive your availability and booking-page slots. Found under **Book Settings > Working Hours**. The heading reads "Working Hours" with the subtitle "Set your available hours for booking pages and availability calculations".

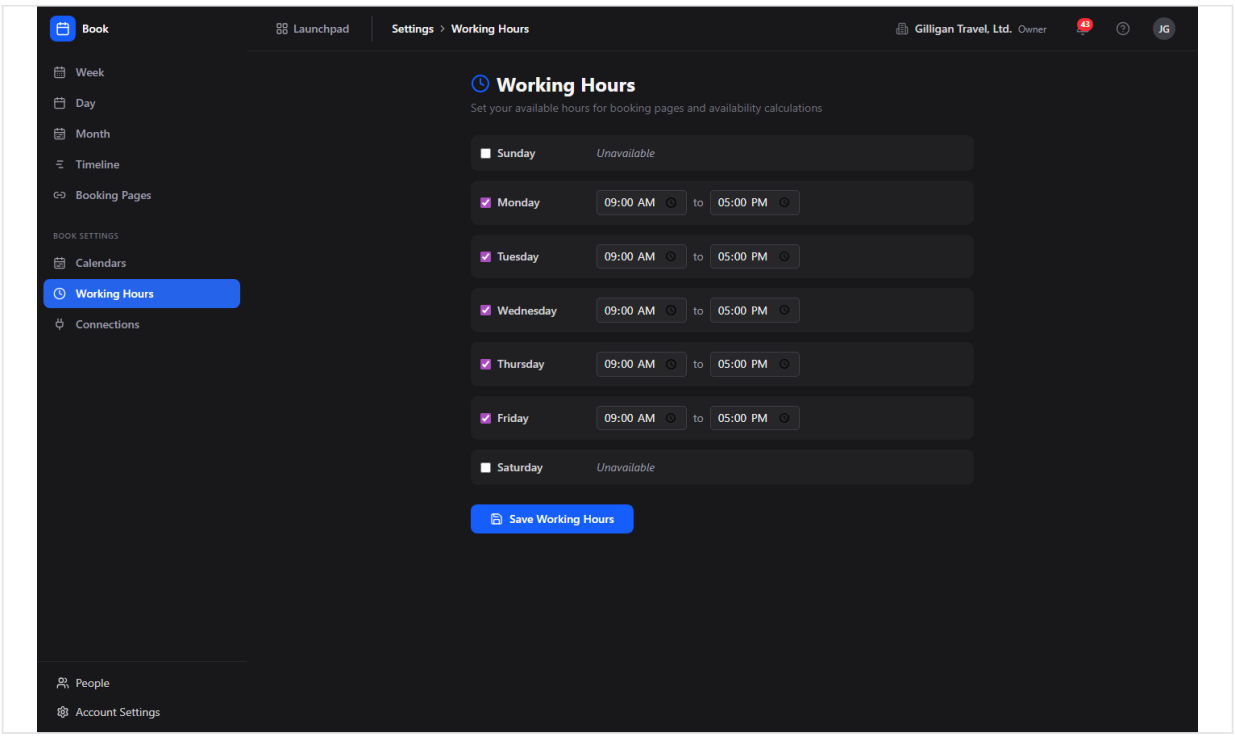


Figure 14.6 Working hours

To set your working hours:

1. Click **Working Hours** under **Book Settings**.
2. You see seven rows, Sunday through Saturday. Monday to Friday, 09:00 to 17:00, are enabled by default.
3. Check or uncheck a day to mark it available or "Unavailable".
4. For each enabled day, set its start and end times.
5. Click **Save Working Hours**. The save replaces your full weekly schedule with the enabled days shown, so make sure every day you want available is enabled before saving.

> **WARNING**
> Saving Working Hours is a full replace, not a merge. The save overwrites your entire weekly schedule with whatever days are currently enabled, so any day you forgot to re-enable is dropped. Confirm every day you want available is checked before you click Save Working Hours.

Connections

Subscribe to an outside calendar and pull its events into Book. Found under **Book Settings > Connections**. The heading reads "External Calendar Connections" with the subtitle "Subscribe to an external calendar feed (.ics URL) to pull its events into Book. Google and Outlook two-way sync require operator-supplied credentials."

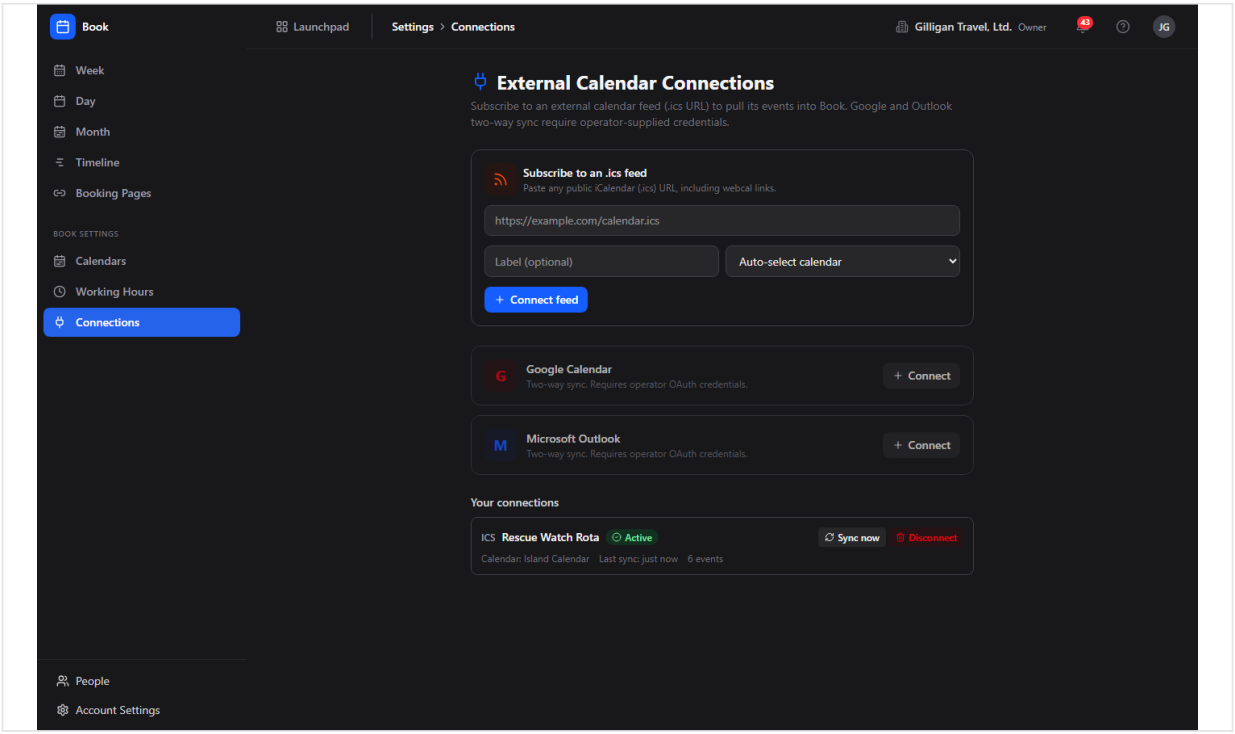


Figure 14.7 Calendar connections

The page has three parts: the **.ics feed** subscribe form (the working, no-credentials path), the **Google Calendar** and **Microsoft Outlook** provider cards (gated on operator OAuth credentials), and **Your connections**, the list of feeds you have already connected with their sync status.

To subscribe to an `.ics` feed:

1. Click **Connections** under **Book Settings**.
2. In the **Subscribe to an .ics feed** card, paste a public iCalendar URL (any `https://...ics` or `webcal://...` link).
3. Optionally give it a **Label** and pick the Book calendar to mirror its events into (leave **Auto-select calendar** to use your default).
4. Click **Connect feed**. Book validates the URL, stores it, and the connection appears under **Your connections**.
5. Click **Sync now** on the connection to pull the feed immediately. The card shows a green **Active** pill, the time of the last sync, and the count of events synced, along with a "Synced: N pulled, N added, N updated, N removed" line.

How sync works:

- Book fetches the feed, parses its events, and creates a matching Book event for each on the target calendar. A re-sync updates those events in place (it does not create duplicates), removes any that were cancelled or vanished upstream, and re-adds any you deleted by hand.

- Syncs also run on their own: a background sweep refreshes every `.ics` connection roughly every fifteen minutes, so a subscribed calendar stays current without you pressing **Sync now**.
- Click **Disconnect** to remove a connection. This also removes every event Book mirrored from that feed, so the calendar returns to just your own events.
- `.ics` feeds are read-only sources: Book pulls events in but never pushes your Book events back out to the feed.

Google and Outlook: the **Google Calendar** and **Microsoft Outlook** cards offer the same sync engine with two-way sync, but their **Connect** buttons stay disabled until an operator sets the provider credentials on the server (`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` for Google, `MICROSOFT_CLIENT_ID` / `MICROSOFT_CLIENT_SECRET` for Microsoft). Until then, use an `.ics` feed URL, which most calendar apps (including Google and Outlook) can publish for any calendar you own.

iCal feed

Book can expose a single calendar as a token-authenticated `.ics` feed that an outside calendar app (Apple Calendar, Google Calendar) can subscribe to.

There is no button for this in the app. The feed is available through the API only: mint a feed token for a calendar with `POST /calendars/:id/ical`, then hand the resulting `GET /calendars/:id/ical?token=...` URL to your external calendar app. If you need this, ask an admin or an agent to mint the token for the calendar you want to share.

In-app Help

Press **?** anywhere in Book (when you are not typing in a field), or navigate to `/book/help`, to open the shared Help viewer for Book.

Working with AI agents

Agents reach Book through 25 MCP tools. They use the same Book API and permissions you do, so an agent can only do what its account is allowed to do. Agents are especially useful for the scheduling math that Book has no human screen for: finding a common free time across several people.

> ***An agent can intersect everyone's calendars and hand back a meeting time, the one scheduling job Book has no human screen for.***

- **Reading events and the timeline:** `book_list_events` (the data behind Week, Day, Month reads), `book_get_event` (one event with attendees and linked-entity context), and `book_get_timeline` (the cross-app Timeline data).
- **Creating and changing events:** `book_create_event` (resolves a calendar by name and an attendee by email, and can set attendees, which the human form cannot),

`book_update_event` and `book_cancel_event` (resolve an event by UUID or title), and `book_rsvp_event` (RSVP by event UUID or title).

- **Availability and meeting time:** `book_get_availability` (one person's free slots), `book_get_team_availability` (free slots for two or more people), `book_find_meeting_time` (intersects team availability and returns up to three suggestions), and `book_find_meeting_time_for_users` (a mixed-roster finder that treats agents and service accounts as always available while respecting human working hours). These four have no human UI in Book today, so they are the main reason to involve an agent.
- **Calendars:** `book_list_calendars`, `book_create_calendar`, `book_update_calendar`, `book_delete_calendar` (the last three resolve a calendar by UUID or name).
- **Booking pages:** `book_list_booking_pages`, `book_create_booking_page`, `book_update_booking_page` (set `enabled: false` to take a page offline without deleting it, or set the cross-app flags the editor does not expose), and `book_delete_booking_page`.
- **Working hours:** `book_get_working_hours` and `book_set_working_hours` (a full replace; include every day you want available).
- **External connections:** `book_create_connection` (subscribe to an `.ics` feed URL, the no-OAuth path), `book_list_connections` (status of every connection, with feed URLs and tokens redacted), `book_sync_connection` (force a sync and get the imported/created/updated/removed counts back), and `book_delete_connection` (remove a connection and its mirrored events). Google and Microsoft connections cannot be created until an operator supplies the OAuth credentials.

Things to know when reviewing agent work:

- An agent can add attendees at creation time even though the human event form cannot. Check the attendee list on the event detail page to confirm who was invited.
- `book_update_booking_page` is the way to set the booking-page options the human editor does not surface (advance limit, minimum notice, confirmation message, redirect URL, logo, the enabled flag, and the cross-app auto-create flags).
- Some caveats that apply to people apply to agents too: recurrence is not expanded and reminders do not exist. An agent cannot work around these because the backend does not implement them. External calendar sync, by contrast, does run: an agent can subscribe to an `.ics` feed and sync it with `book_create_connection` and `book_sync_connection`.
- Book publishes five automation events to Bolt, all from source `book`: `event.created`, `event.updated`, `event.cancelled`, `event.rsvp`, and `booking.created`. Use these to drive cross-app automations (for example, post to a Banter channel when a booking comes in). A new public booking creates or updates a Bond contact by email when the page has `auto_create_bond_contact` on, so a `booking.created` automation can safely assume the matching contact exists.

Book agents also sit on the cross-cutting agentic platform that every BigBlueBam app shares:

- **Identity and heartbeat.** Agent and service accounts are first-class users (`users.kind` is `human`, `agent`, or `service`), and the actor type is recorded on every action they take. Agent runners report in with `agent_heartbeat`, `agent_self_report`, and `agent_audit`.
- **Approval queues.** An agent can route a scheduling change for human sign-off with `proposal_create`; reviewers act on it with `proposal_list` and `proposal_decide`. This is the right pattern when an agent wants to move or cancel an event on someone else's behalf.
- **Visibility preflight.** Before an agent surfaces another person's calendar items in a shared place, it calls `can_access` so it only shows entities the asking user is allowed to see.
- **Policies and webhooks.** Per-agent kill switches and tool allowlists (the `book.*` prefix governs these Book tools) are enforced on every service-account call, and subscribed Bolt events can be pushed to agent runners over signed outbound webhooks.

For the full tool catalog and schemas, see the Book MCP-tools reference and guide in `docs/apps/book/`.

Working together (live presence)

Like every BigBlueBam app, Book carries the persistent Bureau presence dock, so collaboration is ambient rather than scheduled. From anywhere in Book you can see who is around and ring, knock, or invite a teammate into a live voice or video huddle that follows you in a floating window as you both move through the suite. Voice and video here are the digital equivalent of bumping into someone in the hallway or stopping by their desk, not a booked meeting. The deeper per-record collaboration (a presence strip showing who is on a specific item, and real-time co-editing of the same record) lives on the document, board, diagram, and task surfaces, described in the Introduction; in Book, the shared layer is the always-on Bureau dock.

User Stories

Story: Set your working hours

Who: Any Book user getting started. **Goal:** Define the weekly windows that determine when you are available. **Before you start:** Be signed in to BigBlueBam and have Book open.

Steps

1. In the sidebar, under **Book Settings**, click **Working Hours**.
2. Review the seven day rows. Monday to Friday, 09:00 to 17:00, are enabled by default.
3. Check the days you want to be available and uncheck the rest (unchecked days show "Unavailable").
4. For each enabled day, set the start and end times.
5. Click **Save Working Hours**.

Result: Your weekly availability is saved. Book now uses these windows when it computes free slots for availability and for any booking pages you publish.

Related: Working Hours feature; availability tools `book_get_availability` and `book_get_team_availability`. Agents can set the same schedule with `book_set_working_hours`.

Story: Schedule a one-off meeting

Who: Anyone planning a meeting or a block of focused time. **Goal:** Put a single event on a calendar. **Before you start:** Be signed in. At least one calendar exists (your **My Calendar** is created automatically).

Steps

1. Open any calendar view (**Week**, **Day**, or **Month**) and click **New Event**.
2. Enter a **Title** (for example "Team standup").
3. Choose a **Calendar** (your default is preselected).
4. Leave **All day** off and set **Start** and **End**, or turn **All day** on and set **Start Date** and **End Date**.
5. Optionally add a **Description**, a **Location**, a **Repeat** value, and a **Show as** value.
6. Click **Create Event**.

Result: The event appears on the chosen calendar and shows in the Week, Day, and Month views. If you marked it Busy, Tentative, or Out of Office, it now reduces your availability.

Related: New Event feature. Agents do the same with `book_create_event`, which can also add attendees.

Story: View and adjust an event

Who: Anyone updating a meeting's details. **Goal:** Change the time, location, or description of an existing event. **Before you start:** The event exists and you have permission to update it.

Steps

1. In a calendar view, click the event. This opens the **Edit Event** form directly.
2. Change any field, for example **Start**, **End**, **Location**, or **Description**.
3. Click **Update Event**.

Result: The event is updated everywhere it appears.

Related: New Event and Edit Event feature. To see an event's read-only detail and attendee list instead, open its direct URL (`/book/events/<id>`). Agents update events with `book_update_event`.

Story: Cancel an event

Who: Anyone calling off a meeting. **Goal:** Remove an event from calendars and availability without losing its record. **Before you start:** The event exists and you can delete it.

Steps

1. Open the event's detail page at its direct URL (`/book/events/<id>`).
2. Click **Cancel** (the trash icon).
3. Confirm "Cancel this event?".

Result: The event's status becomes `cancelled`. It disappears from availability and the timeline but its record is retained (soft delete).

Related: Event detail feature. Agents cancel with `book_cancel_event`.

Story: RSVP to an invitation

Who: An attendee invited to an event. **Goal:** Tell the organizer whether you will attend. **Before you start:** You are listed as an attendee on the event.

Steps

1. Open the event's detail page (`/book/events/<id>`).
2. Find the **Your response** panel.
3. Click **Accept**, **Maybe**, or **Decline**. "Maybe" records a tentative response.

Result: Your response status updates in the **Attendees** list. If you are not an attendee, the response is rejected.

Related: Event detail and RSVP feature. Agents RSVP with `book_rsvp_event`.

Story: Manage multiple calendars

Who: Anyone separating work into distinct calendars. **Goal:** Add, rename, recolor, or remove calendars. **Before you start:** Be signed in with calendar write permission.

Steps

1. Under **Book Settings**, click **Calendars**.
2. Click **New Calendar**, enter a name, pick a color swatch, and click **Create**.

3. To change a calendar, click **Edit** on its row, adjust the name and color, and click the check (save) button.
4. To remove a non-default calendar, click its trash icon and confirm "Delete calendar [name]?".

Result: Your calendar list reflects the changes. The default calendar stays put; it has no trash icon and the API rejects an attempt to delete it with "Cannot delete default calendar".

Related: Calendars feature. Agents manage calendars with `book_list_calendars`, `book_create_calendar`, `book_update_calendar`, and `book_delete_calendar`.

Story: Create and share a public booking page

Who: Someone who wants outside people to book time with them. **Goal:** Publish a scheduling link at `/book/meet/<slug>` and share it. **Before you start:** Set your Working Hours first, since they define the offered slots. Have booking-page write permission.

Steps

1. Click **Booking Pages** in the sidebar, then **New Booking Page**.
2. Enter a **Title** (for example "30-Minute Intro Call").
3. Enter a **URL Slug** after "/meet/" using lowercase letters, numbers, and hyphens (for example "intro-call").
4. Optionally set **Description**, **Duration (min)**, **Buffer Before (min)**, **Buffer After (min)**, and **Brand Color**.
5. Click **Create**.
6. Copy the page's `/meet/<slug>` link from the Booking Pages list and share it. To change the page later, click **Edit** on its row.

Result: The page appears in your Booking Pages list with an **Active** pill and its `/meet/<slug>` link. A visitor who opens the link sees your available times, picks a slot, and books; the booking lands as a confirmed event on your default calendar, and a matching Bond contact is created or updated by email (unless the page's `auto_create_bond_contact` flag has been turned off).

Related: Booking Pages list, Booking page editor, and Public Meet page features. Agents create pages with `book_create_booking_page` and edit them (including the options the editor does not surface) with `book_update_booking_page`.

Story: Book time as an outside visitor

Who: A prospect or client with a booking-page link. **Goal:** Reserve a time on the page owner's calendar. **Before you start:** You have a `/book/meet/<slug>` link. No login is required.

Steps

1. Open the link. Under **Pick a time**, the next two weeks of available slots load, grouped by day.
2. Click a time slot to select it.

3. Fill in **Full name** and **Email**, and an optional **Notes (optional)**.
4. Click **Book {time}**.

Result: You see the **Booking confirmed** screen with the owner's confirmation message (or you are sent to the page's redirect URL, if set). The owner's default calendar gains a confirmed event, and a Bond contact is created or updated from your email when the page has that on.

Related: Public Meet page feature. The booking emits a `booking.created` Bolt event from source `book`, which Bolt rules can react to.

Story: See everything happening this week across apps

Who: Anyone who wants a single week-at-a-glance across the suite. **Goal:** Scan Book events, Bam task due dates, and Bond deal close dates together. **Before you start:** Be signed in.

Steps

1. Click **Timeline** in the sidebar.
2. Read the date-range heading (for example "Timeline: Jun 9 - Jun 15, 2026").
3. Move between weeks with the left and right chevron arrows, or click **This Week**.
4. Scan the per-day items, using the source labels and the "Book Events / Bam Tasks / Bond Deals" legend.

Result: You see the week's items grouped by day. Book events show in blue with their time, Bam tasks in orange on their due date, and Bond deals in teal on their close date. Click a Book item to open its event.

Related: Timeline feature. Agents read the same data with `book_get_timeline`.

Story: Find a common meeting time across several people (agent-driven)

Who: A user working with an AI agent, or the agent itself. **Goal:** Identify a time when everyone on a list is free. **Before you start:** Each person has set their Working Hours, and you have an agent with Book access. There is no human screen for this in Book today, so it runs through an agent.

Steps

1. Ask the agent to find a meeting time for the people you name (by email is fine; the tools resolve identifiers).
2. The agent calls `book_get_team_availability` to gather each person's free slots, or `book_find_meeting_time` to get up to three suggested times directly.
3. For a roster that mixes people with agents or service accounts, the agent uses `book_find_meeting_time_for_users`, which treats agents and service accounts as always available while honoring human working hours.
4. Review the suggested times and ask the agent to create the event with `book_create_event`.

Result: You get one or more candidate times that fit everyone's availability, and an event can be created on the spot.

Related: Working with AI agents. Tools: `book_get_availability`, `book_get_team_availability`, `book_find_meeting_time`, `book_find_meeting_time_for_users`, `book_create_event`.

Story: Subscribe to a Book calendar from an outside app (iCal, agent or API)

Who: Someone who wants a Book calendar to appear in Apple Calendar or Google Calendar. **Goal:** Get a subscribable `.ics` feed URL for one calendar. **Before you start:** There is no button for this in Book. You need an admin or an agent to mint the feed token through the API.

Steps

1. Ask an admin or agent to mint an iCal feed token for the specific calendar you want to share (`POST /calendars/:id/ical`).
2. Receive the public feed URL with its `?token=...` query (`GET /calendars/:id/ical?token=...`).
3. In your external calendar app, add a subscribed calendar by URL and paste the feed URL.

Result: Your outside calendar app shows that Book calendar's events. There is no in-app control to generate or revoke the feed today.

Related: iCal feed feature.

Related

- **Bam** (`/b3/`) - Sign in here first; Book has no login of its own. Bam tasks with due dates appear on the Book Timeline, and events can link to a Bam task.
- **Bond** (`/bond/`) - Bond deal close dates appear on the Book Timeline, and events can link to a Bond deal. A public booking creates or updates a Bond contact by email when the page has `auto_create_bond_contact` on (the default).
- **Helpdesk** (`/helpdesk/`) - Events can link to a Helpdesk ticket.
- **Bolt** (`/bolt/`) - Subscribe to Book's automation events (`event.created`, `event.updated`, `event.cancelled`, `event.rsvp`, `booking.created`, all from source `book`) to drive cross-app workflows.
- **Banter** (`/banter/`) - A common automation target, for example posting to a channel when a booking arrives.
- Book MCP-tools reference and guide in `docs/apps/book/` for the full catalog of the 25 agent tools and their schemas.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What is created for you automatically the first time you open Book, and can it be deleted?

a) A team calendar; yes, it can be deleted
b) A personal calendar named My Calendar; no, it cannot be deleted
c) A booking page; yes, it can be deleted
d) Nothing is created until you add a calendar

2. What status and visibility does a brand-new event get by default?

a) status tentative, visibility free
b) status confirmed, visibility busy
c) status confirmed, visibility free
d) status cancelled, visibility out_of_office

3. When you cancel an event, what actually happens to it?

4. Which event visibility setting does NOT consume an availability slot?

a) Busy
b) Free
c) Tentative
d) Out of Office

5. What are the default working hours when you first set up Book?

6. Where does the event created by a public booking land?

a) On a new calendar created for the booking
b) As a confirmed event on the page owner's default calendar
c) On the visitor's calendar
d) Nowhere until the owner accepts it

7. Which two controls appear on the New Event form but do nothing when you save, and why?

8. What happens if you choose a Repeat option like Weekly on an event?

a) Book generates all future occurrences immediately
b) The rule is stored on the single event but not expanded, so it remains one event
c) The event is rejected as unsupported
d) It creates a recurring booking page

9. Which calendar feed provider works today with no credentials, and which two require operator-supplied OAuth before they can be connected?

10. Roughly how often does the background sweep refresh every .ics connection on its own?

a) Every minute
b) Every fifteen minutes
c) Once an hour
d) Once a day

11. What does disconnecting an .ics feed do to the events Book pulled from it?

12. When two visitors try to grab the same booking slot at once, what stops a double-

booking?

a) The slot is simply offered to both
b) A row lock guards the booking, and a collision returns a 409 'This time slot is no longer available'
c) The second visitor is silently dropped
d) Both bookings are created and merged later

13. Why can't you add attendees from the human New Event form, and how can attendees be added instead?

CHAPTER 15

Brief

Write together, ship the words.

Brief is where your team writes together. Every document is a rich-text page you can co-edit live, comment on in the margins, version, and organize into folders scoped to a project. This chapter takes you from a blank page to a published document, shows how two people type at once without stepping on each other, and points out where an AI agent can do the things the buttons cannot. By the end you will be able to write, format, publish, and promote a document, and find any of them fast.

At a glance

- Rich-text editing on Tiptap, with a formatting toolbar and slash commands
- Real-time co-editing with live cursors and presence, merged conflict-free
- Four-status lifecycle: Draft, In Review, Approved, and Archived
- Folders, project scope, versions, comments, and stars to keep work tidy
- Promote a finished document one-way into a Beacon knowledge-base article

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Home . Browse the document list . Create a document . Write and format . The "Brief summary (optional)" field . Set visibility

> *Brief is your team's collaborative document editor: a place to write, format, co-edit, comment on, and organize documents, then promote the ones that matter into your knowledge base. Reach for it when you need a shared, structured document that more than one person works on at once.*

Overview

Brief stores your team's writing as **Documents**. Each document[149] has a title, a URL slug, an optional icon, a rich-text body, a status[150], a visibility[151] level, an optional project, and an optional folder[152]. Documents are edited in a rich-text editor[153] (built on Tiptap) with a formatting toolbar and slash commands, and two or more people can type in the same document at the same time with live cursors.

Beyond writing, Brief organizes documents into **Folders** and scopes them to **Projects**, keeps numbered **Versions** as you publish, supports threaded and text-anchored **Comments** for review, lets you **Star**[154] documents for quick access, and can **Promote** a finished document into a Beacon knowledge-base article. Documents can also be **Linked** to Bam tasks and Beacon articles so related work stays connected.

Brief is part of the BigBlueBam suite. It shares your platform login with Bam and the other apps, pulls its project list from Bam, and publishes events to Bolt so automations can react when a document is created, updated, published, or promoted. Search runs as keyword search over titles and body text, with optional semantic search when a vector store is configured.

Key concepts

- **Document** - A single piece of writing. It has a title, a globally unique slug, an optional icon (up to 2 characters), a rich-text body, a word count, a status, a visibility, an optional project, and an optional folder.
- **Editor** - The Tiptap rich-text surface where you write. It has a formatting toolbar, slash commands (type `/`), a live word count, a Table of Contents built from your headings, and a Settings sidebar for icon, visibility, project, and starting template[155].

[149] **Document.** A single piece of writing. It has a title, a globally unique slug, an optional icon (up to 2 characters), a rich-text body, a word count, a status, a visibility, an optional project, and an optional folder.

[150] **Status.** The document's place in its lifecycle. The four statuses are **Draft**, **In Review**, **Approved**, and **Archived**. New documents start as Draft. Saving a draft keeps it Draft; publishing sets it to Approved. There is no button in the editor that sets In Review today; that status is set only through the API or an agent (see Working with AI agents).

[151] **Visibility.** Who can see the document. There are three valid levels: **Organization** (all org members), **Project** (the creator, collaborators, or members of that project), and **Private** (the creator and collaborators only). The editor's Visibility menu also lists a "Public" option, but Public is not a valid setting and selecting it produces a validation error; choose Organization, Project, or Private instead (see Set visibility).

[152] **Folder.** A named container for documents. Folders can be nested and can be scoped to a project. You create folders from the sidebar; renaming and deleting folders is not available in the UI today (an agent can do both, see Working with AI agents).

[153] **Editor.** The Tiptap rich-text surface where you write. It has a formatting toolbar, slash commands (type `/`), a live word count, a Table of Contents built from your headings, and a Settings sidebar for icon, visibility, project, and starting template.

[154] **Star.** A per-user bookmark on a document. Starred documents show on the Home and Starred screens.

[155] **Template.** A pre-built starting point for a new document. You pick a template from the Templates screen and the editor pre-fills with its content.

- **Status** - The document's place in its lifecycle. The four statuses are **Draft**, **In Review**, **Approved**, and **Archived**. New documents start as Draft. Saving a draft keeps it Draft; publishing sets it to Approved. There is no button in the editor that sets In Review today; that status is set only through the API or an agent (see Working with AI agents).
- **Visibility** - Who can see the document. There are three valid levels: **Organization** (all org members), **Project** (the creator, collaborators, or members of that project), and **Private** (the creator and collaborators only). The editor's Visibility menu also lists a "Public" option, but Public is not a valid setting and selecting it produces a validation error; choose Organization, Project, or Private instead (see Set visibility).
- **Folder** - A named container for documents. Folders can be nested and can be scoped to a project. You create folders from the sidebar; renaming and deleting folders is not available in the UI today (an agent can do both, see Working with AI agents).
- **Project scope**[156] - A filter in the sidebar that limits the documents and folders you see to one project, or shows everything with "All Projects".
- **Version**[157] - A numbered snapshot of a document. Versions are listed read-only in the document detail sidebar; creating and restoring versions is done through the API or an agent (see Working with AI agents).
- **Comment**[158] - A note on a document. Comments can be threaded (a reply to another comment) and can be anchored to a span of text. Comments can be marked resolved and appear in the detail sidebar.
- **Star** - A per-user bookmark on a document. Starred documents show on the Home and Starred screens.
- **Template** - A pre-built starting point for a new document. You pick a template from the Templates screen and the editor pre-fills with its content.
- **Link**[159] - A typed connection from a document to a Bam task or a Beacon article. Links show read-only under "Linked Items" in the detail sidebar; creating links is done through an agent or the API (see Linked Items).
- **Promote to Beacon**[160] - Graduating a finished document into a Beacon knowledge-base article. This is one-way: once promoted, the document records the Beacon it became.

[156] **Project scope.** A filter in the sidebar that limits the documents and folders you see to one project, or shows everything with "All Projects".
[157] **Version.** A numbered snapshot of a document. Versions are listed read-only in the document detail sidebar; creating and restoring versions is done through the API or an agent (see Working with AI agents).
[158] **Comment.** A note on a document. Comments can be threaded (a reply to another comment) and can be anchored to a span of text. Comments can be marked resolved and appear in the detail sidebar.
[159] **Link.** A typed connection from a document to a Bam task or a Beacon article. Links show read-only under "Linked Items" in the detail sidebar; creating links is done through an agent or the API (see Linked Items).
[160] **Promote to Beacon.** Graduating a finished document into a Beacon knowledge-base article. This is one-way: once promoted, the document records the Beacon it became.

- **Real-time collaboration**[161] - Two or more people editing the same document at once. You see other editors' cursors and presence while you type, and readers on the detail page see edits appear live.

> **NOTE**
> Brief has no editor button for the In Review status. The two save buttons only produce Draft or Approved; In Review is set through the API or by an agent with brief_update.

Where to find it

Brief lives at `/brief/`. You must be logged in to BigBlueBam first; Brief has no login of its own and uses the shared platform session. If you are not signed in, Brief shows a gate page with a link to the BigBlueBam login at `/b3/`.

The left sidebar has five navigation items: **Home** (`/`), **Documents** (`/documents`), **Templates** (`/templates`), **Search** (`/search`), and **Starred** (`/starred`). Below the nav is a **Folders** section with a button to add a folder, and a project-scope selector at the top of the sidebar that shows the active project name or **"All Projects"**.

What you can do depends on your role and on each document's visibility. Editing a document requires edit access: you are the document's creator, or a SuperUser, or an org Admin or Owner (admins and owners can edit any document in the org). Creating documents, folders, and links, and promoting to Beacon, each require the matching permission and the `read_write` scope on your session or API key.

Press the `?` key (when your cursor is not in a text field) to open the in-app help from any Brief screen.

[161] **Real-time collaboration.** Two or more people editing the same document at once. You see other editors' cursors and presence while you type, and readers on the detail page see edits appear live.

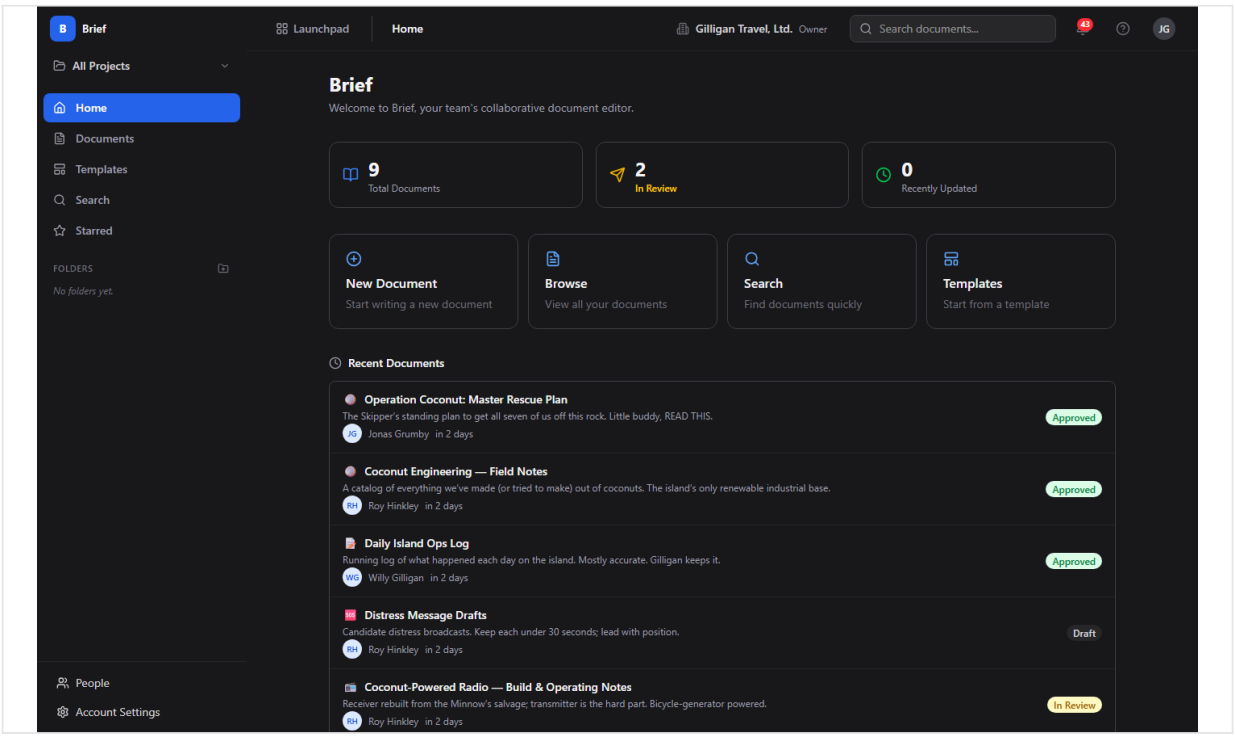


Figure 15.1 *Brief home*

Feature reference

Home

The Home screen welcomes you and gives quick entry points into Brief. It lives at `/brief/` and shows three stat cards, four quick-action cards, and lists of your recent and starred documents.

What you see:

1. A header reading "Brief" and "Welcome to Brief, your team's collaborative document editor."
2. Three stat cards: **Total Documents**, **In Review**, and **Recently Updated**. Each card links to the Documents list when clicked. Note: the "Recently Updated" card always reads 0 today; use the Documents list sorted by update time to find recent work.
3. Four quick-action cards: **New Document** (opens the editor at `/new`), **Browse** (opens the Documents list), **Search** (opens Search), and **Templates** (opens the Templates browser).
4. A **Recent Documents** list of up to 8 documents and a **Starred Documents** list of up to 5, each row showing the icon, title, author, and status.

Browse the document list

The Documents list shows every document you can see, with filters. It lives at `/documents`.

To browse and filter documents:

1. Open **Documents** from the sidebar.
2. Use the **"Search documents..."** box in the toolbar to filter the visible cards by title text.
3. Use the status chips - **All**, **Draft**, **In Review**, **Approved**, **Archived** - to filter by status.
4. The toolbar shows the current scope as "Showing docs for: <project>" or "Showing all org documents", set by the sidebar project-scope selector. If you arrived through a folder, a folder filter chip appears with an X to clear it.
5. Click any document card to open its detail page. Each card shows the icon, title, a star if pinned, the author, a relative time, the word count, and a status badge.
6. If there are more results, click **Load more** to page through them.

If no documents match, the list shows "No documents yet. Create your first one."

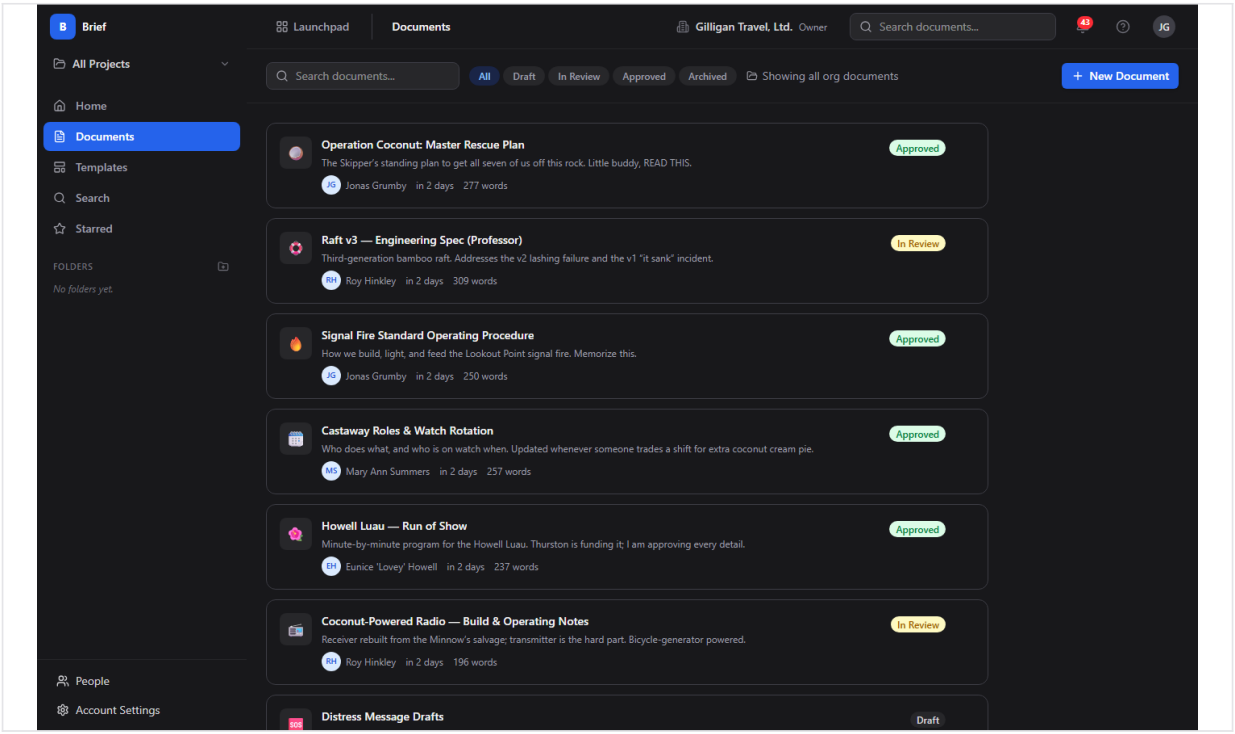


Figure 15.2 Document list

Create a document

You create documents in the editor, either blank or from a template.

To create a new document:

1. Click **New Document** on Home, or **New Document** in the Documents toolbar. This opens the editor at `/new`.
2. Type a title in the **"Document title..."** field at the top.
3. Write your content in the body. Use the toolbar, slash commands, or both (see Write and format).
4. Optionally set the icon, visibility, project, and starting template in the right sidebar (see Set visibility, Choose a project, and Start from a template).
5. Click **Save Draft** to save it as a Draft, or **Publish** to save it as Approved.

When you save, Brief takes you to the new document's detail page.

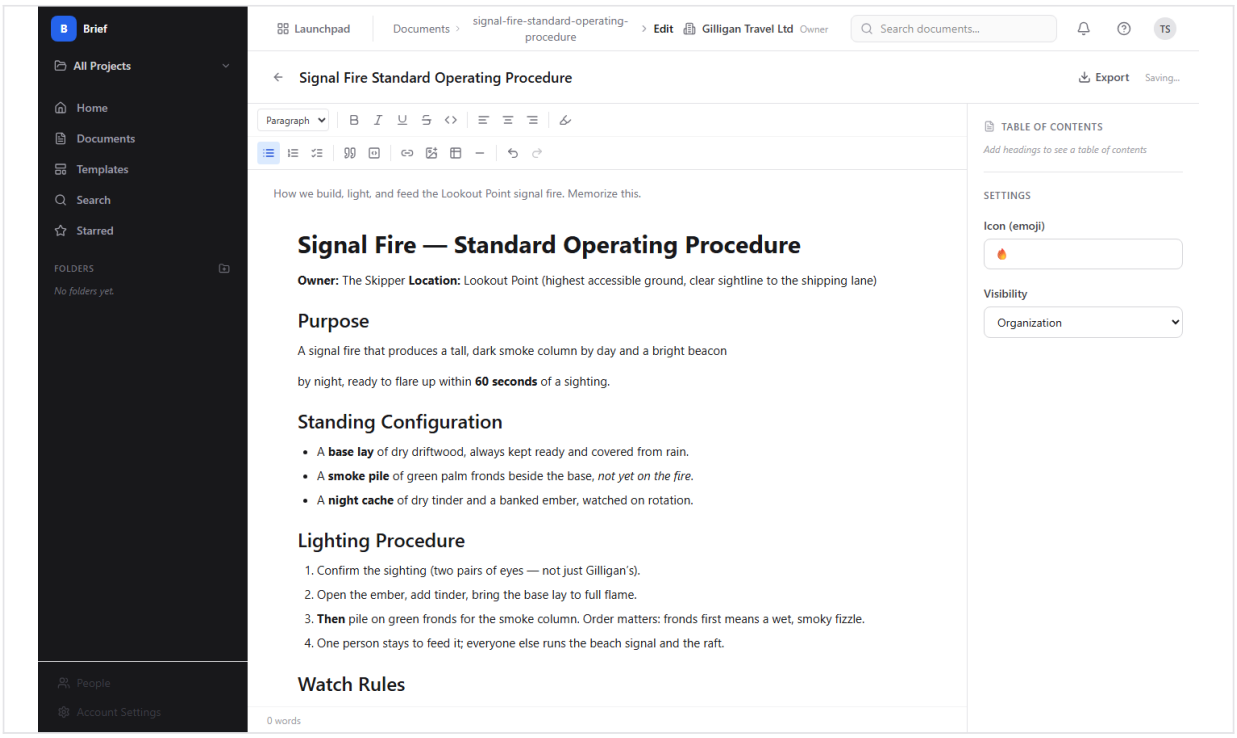


Figure 15.3 Document editor

Write and format

The editor gives you a formatting toolbar, slash commands, and inline embeds.

To format text with the toolbar:

1. In the editor, select the text you want to format.
2. Use the toolbar to apply a block style (**Paragraph**, **Heading 1** through **Heading 4**), inline styles (**Bold**, **Italic**, **Underline**, **Strikethrough**, **Inline code**), alignment (**left**, **center**, **right**), **Highlight**, lists (**Bullet**, **Ordered**, **Task** list), **Blockquote**, or **Code block**.
3. To add a link, click the link button, type a URL, and click **Add**. To add an image, click the image button and provide a URL when prompted. Use **Insert table (3x3)** to drop in a table and the horizontal-rule button for a divider.
4. Use **Undo** and **Redo** to step backward or forward.

To use slash commands:

1. In the body, type `/` on a new line.
2. Pick from the menu: **Heading 1**, **Heading 2**, **Heading 3**, **Bullet List**, **Numbered List**, **Task List**, **Code Block**, **Blockquote**, **Horizontal Rule**, **Table**, or **Image**.

The footer shows a live word count as you type. The right sidebar shows a **Table of Contents** built from your headings.

Note: the editor renders task-embed, mention, callout, beacon-embed, and channel-link nodes if a document already contains them, but the toolbar and slash menu do not include a control to insert a Beacon embed or a mention.

The "Brief summary (optional)" field

Below the toolbar the editor shows a **"Brief summary (optional)..."** input. This field is not stored by Brief today; anything you type in it is dropped on save and will not appear on the document later. Treat it as non-functional and put any summary text in the document body instead.

> **TIP**
> Skip the "Brief summary (optional)" input. Anything you type there is dropped on save and never appears on the document. Put summary text in the body instead.

Set visibility

Visibility controls who can open a document. You set it in the editor's right sidebar under **Settings**.

To set visibility:

1. In the editor, find the **Visibility** menu in the right sidebar.
2. Choose **Organization** (all org members), **Project** (creator, collaborators, or project members), or **Private** (creator and collaborators only).
3. Save or publish the document.

Do not choose **Public**. The menu lists it, but Public is not a valid value and selecting it makes the save fail with a validation error. The three valid choices are Organization, Project, and Private. New documents default to Organization in the editor.

> **WARNING**
> The Visibility menu lists Public, but it is not a valid value. Selecting it makes the save fail with a validation error. Choose Organization, Project, or Private instead.

Set an icon

To give a document an icon:

1. In the editor's right sidebar under **Settings**, find **Icon (emoji)**.
2. Type a short emoji or up to 2 characters.
3. Save or publish. The icon shows on cards, lists, and the detail header.

Choose a project

You can scope a new document to a project so it appears under that project's filter.

To choose a project for a new document:

1. In the editor's right sidebar, find **Project (optional)** (shown only when creating, not when editing an existing document).

2. Choose a project, or choose **Organization-wide (no project)** to leave it unscoped.
3. Save or publish.

Start from a template

Templates give you a pre-built starting point.

To start a document from a template:

1. Open **Templates** from the sidebar. Each card shows an icon, name, category, and description.
2. Click a template card. Brief opens the editor pre-filled with that template's content.
3. Finish writing and click **Save Draft** or **Publish**.

Alternatively, when creating a blank document you can pick a template from the **Start from template** menu in the editor sidebar (choose **Blank document** for none).

If there are no templates, the Templates screen shows "No templates available yet." with "Templates can be created by administrators." There is no template-authoring screen in Brief today; templates come from the API, an agent, or seed data. An agent can create, update, and delete org-level templates (see Working with AI agents).

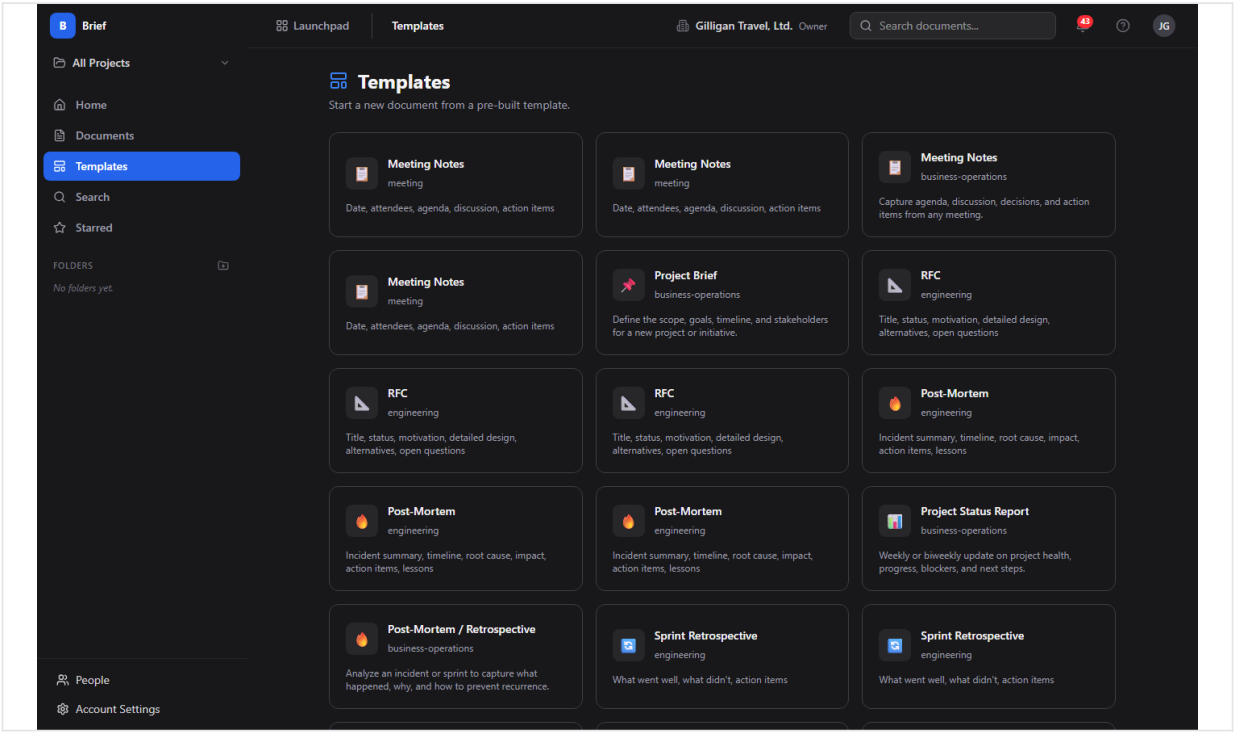


Figure 15.4 Template browser

Save a draft and publish

The editor has two save buttons in the header.

- **Save Draft** keeps the document's status as Draft.
- **Publish** sets the document's status to Approved.

To publish a document:

1. In the editor, finish your title and content.
2. Click **Publish**. The status becomes Approved and Brief returns you to the document detail page.

A title is required before either button is enabled.

Read a document

The document detail page shows the document and its metadata. It lives at `/documents/:idOrSlug`.

What you see:

1. A header with the icon, title, status badge, a star toggle, and an **Edit** button that opens the editor.
2. The document body. While someone is editing, readers see edits appear live. If a document has no content, the body shows "No content yet."
3. An action bar with **Duplicate**, **Export**, **Promote to Beacon**, and **Archive** (or **Restore** when the document is archived).
4. A right sidebar with **Status**, **Author**, **Project** (or "Organization-wide" if none), **Created**, **Last Updated**, **Published** (when present), **Word Count**, **Version**, **Linked Items**, and **Comments**.

Note: the **Published** date and the **Version** number are not stored by Brief today, so the Published row may be absent and the Version button can read "vundefined". The version list that expands underneath it is populated only if versions were created through the API or an agent.

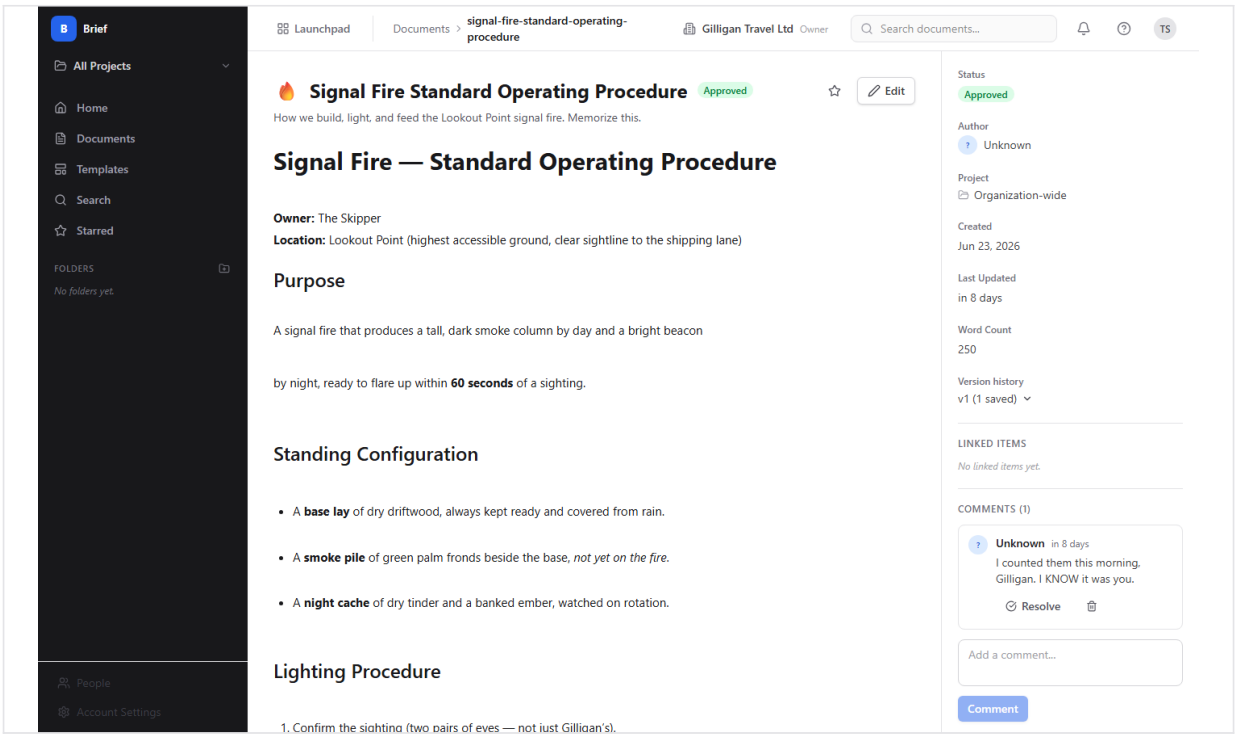


Figure 15.5 Document detail

Edit an existing document and co-edit in real time

To edit a document:

1. Open the document and click **Edit**, or open it directly at `/documents/:idOrSlug/edit`.
2. Make your changes. Your edits sync live to anyone else in the same document.
3. The header shows presence chips for everyone currently editing.
4. Click **Save Draft** or **Publish** when done.

When two or more people open the same document's editor, each sees the others' cursors and presence, and readers on the detail page see the changes as they happen. Brief uses a conflict-free shared editing model (Yjs) over the `/brief/ws` WebSocket, so concurrent edits merge without overwriting each other.

> ***Two people, one document, live cursors. Edits merge conflict-free, so nobody overwrites anybody.***

Comments

Comments let your team review a document in the detail sidebar. A comment can be a top-level note, a threaded reply, or anchored to a span of text.

To add a comment:

1. Open the document.
2. In the right sidebar under **Comments (N)**, type into the **"Add a comment..."** box.
3. Click **Comment**.

To resolve a comment, use the **Resolve** control on the comment. The comment's author, or an org Admin or Owner or SuperUser, can delete a comment.

Star a document

Starring bookmarks a document for quick access from Home and Starred.

To star or unstar a document:

1. Open the document.
2. Click the star button in the header. Its tooltip reads **Star document** when not starred and **Remove star** when it is.
3. Find your starred documents on the **Starred** screen and in the Starred list on Home.

Duplicate a document

To make a copy:

1. Open the document.
2. Click **Duplicate** in the action bar. Brief creates a Draft copy named "<title> (copy)" and opens it.

Export a document

To export a document to a file:

1. Open the document (or open it in the editor).
2. Click **Export** in the action bar or editor header.
3. Choose **Markdown (.md)** or **HTML (.html)**. The file downloads.

Archive and restore

Archiving hides a document from normal lists; you can restore it later.

To archive a document:

1. Open the document.
2. Click **Archive** in the action bar.

To restore an archived document:

1. Filter the Documents list by the **Archived** status chip and open the document, or open it by URL.
2. Click **Restore** in the action bar. The document returns to Draft status.

Promote to Beacon

When a document is ready to become lasting knowledge, promote it into a Beacon article.

> ***Promote graduates a finished document straight into a Beacon article, one way and on purpose.***

To promote a document:

1. Open the document.
2. Click **Promote to Beacon** in the action bar.
3. Brief creates a Beacon article from the document and takes you to it at `/beacon/<id>`.

Promotion is one-way; a document that has already been promoted cannot be promoted again.

Linked Items

The detail sidebar shows a read-only **Linked Items** section listing the Bam tasks and Beacon articles connected to the document. Task links open in Bam at `/b3/tasks/:id` and Beacon links open in Beacon.

There is no button in the Brief UI to create or remove a link. Links are created and removed through an agent or the API (see Working with AI agents).

Folders and project scope

Folders and the project-scope selector live in the sidebar and filter what you see.

To create a folder:

1. In the sidebar **Folders** section, click the add (+) button.
2. Type a name in the inline **"Folder name"** input.
3. Press Enter to save, or Escape to cancel.

To filter by project scope:

1. Click the project-scope selector at the top of the sidebar.
2. Choose **All Projects** or a specific project. Documents and folders filter to that scope.

Renaming, moving, and deleting folders are not available in the Brief UI today; an agent or the API can do all three (see Working with AI agents).

Search

Search finds documents by title, body content, or author.

To search:

1. Open **Search** from the sidebar, or click the **"Search documents..."** box in the header (which opens the Search screen).
2. Type at least 2 characters in the **"Type to search..."** box.
3. Results list each document's title, creator, time, and status. Click a result to open it.

> **TRY IT**
> Open Search from the sidebar, type two or more characters in the "Type to search..." box, and click a result to jump straight into that document.

If you type fewer than 2 characters, Search shows "Type at least 2 characters to search." If nothing matches, it shows "No documents found for "<query>"."

Note: search results do not currently show a body excerpt under each title. Keyword search matches against the title and body text.

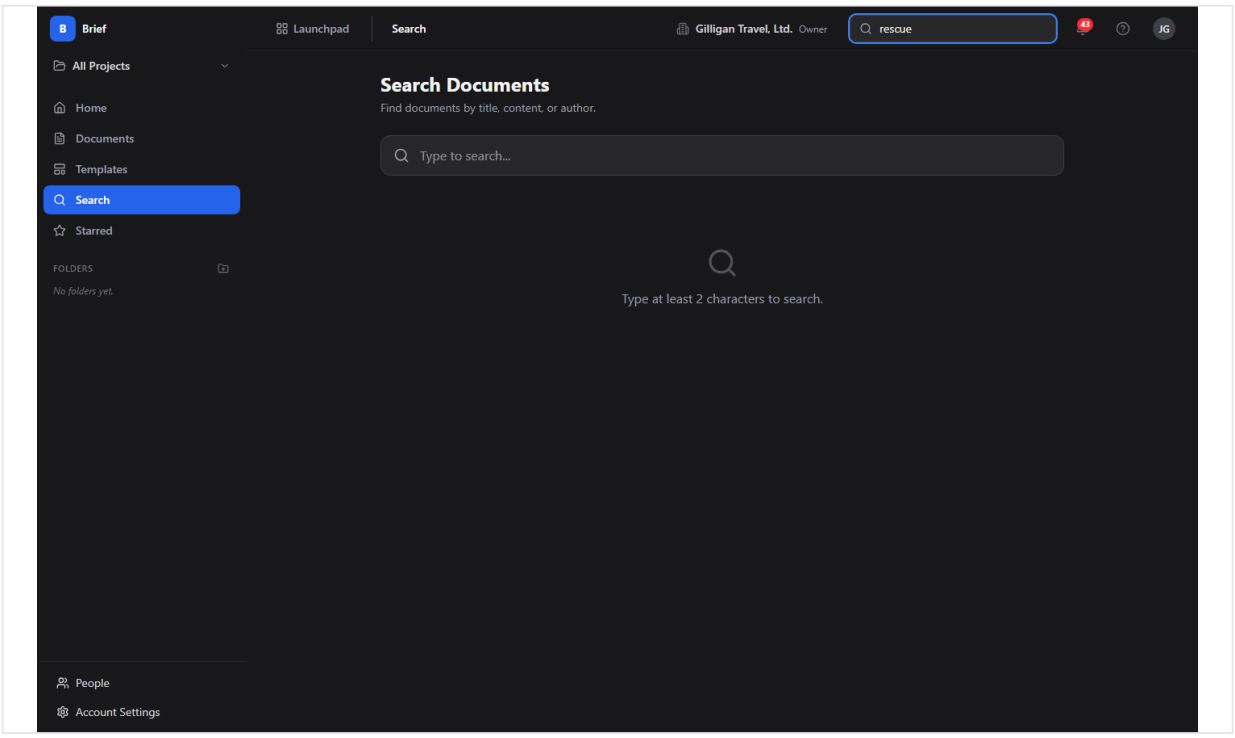


Figure 15.6 Search results

Working with AI agents

Agents work with Brief through the Model Context Protocol (MCP). Brief exposes 48 MCP tools that span the entire app: documents, search, comments, versions, links, collaborators, embeds, templates, and folders. Each tool forwards your bearer token, so an agent can only do what your account is allowed to do, and write tools require the `read_write` scope. Brief enforces Organization, Project, and Private visibility on the server regardless of how a tool is called.

Many of these tools have no human button in Brief. They are the only way to do things like append content, set the **In Review** status, create or remove links, manage collaborators, author templates, or rename and delete folders.

Authoring and maintenance:

- `brief_create` creates a document (title, project, folder, template, markdown content, visibility).
- `brief_update_content` replaces the document body; `brief_append_content` adds to the end of it. `brief_append_content` has no human button, so appending is agent or API only.
- `brief_update` changes metadata such as title, status, visibility, folder, icon, and pinned. This is the only way to set the **In Review** status, which has no control in the editor.
- `brief_archive`, `brief_restore`, `brief_duplicate`, and `brief_star` cover the lifecycle and bookmarking actions.

Search and reading:

- `brief_list`, `brief_get`, `brief_recent`, `brief_starred`, and `brief_stats` cover listing, opening, and counting documents. `brief_search` runs the keyword search; `brief_semantic_search` runs vector search and falls back to full-text search when the vector index is unavailable. Brief is also a registered source for the platform-wide `search_everything` tool.

Review:

- `brief_comment_list`, `brief_comment_add`, `brief_comment_resolve`, `brief_comment_edit`, and `brief_comment_delete` cover reading, adding, resolving, editing, and deleting comments. `brief_comment_react` and `brief_comment_unreact` add and remove emoji reactions, which have no UI.

Versions:

- `brief_versions` lists versions, `brief_version_get` fetches one, `brief_version_create` records a named snapshot, `brief_version_restore` restores a version, and `brief_version_diff` computes a line-by-line diff between two versions. There is no human button to create, restore, or diff versions, so version management is agent or API only.

Export:

- `brief_export_markdown` and `brief_export_html` return the document as Markdown or as a standalone styled HTML page, the same content the **Export** button downloads.

Cross-app links and graduation:

- `brief_links_list` lists every task and Beacon link on a document. `brief_link_task` creates a link to a Bam task (by UUID or by a human reference like FRND-42); `brief_link_beacon` creates a link to a Beacon article; `brief_link_remove` removes either kind. These tools are the only way to create or remove links, since "Linked Items" is read-only in the UI.
- `brief_promote_to_beacon` graduates a document into a Beacon article, the same action as the **Promote to Beacon** button.

Sharing, organizing, and templates:

- `brief_collaborators_list`, `brief_collaborator_add`, `brief_collaborator_update`, and `brief_collaborator_remove` manage per-document collaborators with **view**, **comment**, or **edit** permission. Brief has no share dialog, so collaborator management is agent or API only; to share a single document with specific people, use these tools or set the document's visibility to Project or Organization.
- `brief_folders_list`, `brief_folder_create`, `brief_folder_update`, and `brief_folder_delete` manage the folder tree. The UI can only create folders, so renaming, moving, and deleting are agent or API only.
- `brief_templates_list`, `brief_template_create`, `brief_template_update`, and `brief_template_delete` manage org-level templates. There is no template-authoring screen, so template CRUD is agent or API only.

- `brief_embeds_list` and `brief_embed_delete` read and remove embedded file records.

Cross-cutting agent platform. Brief plugs into the suite-wide agentic surfaces that every app shares:

- **Identity and heartbeat.** Agent and service accounts are first-class: `agent_heartbeat`, `agent_self_report`, and `agent_audit` let a runner announce itself and report its activity, and every write an agent makes is stamped with its actor type in the unified activity log.
- **Approvals.** When a change should pause for a human, agents file it with `proposal_create`; a reviewer lists and decides with `proposal_list` and `proposal_decide`. This is the safe path for an agent to draft a sensitive document, comment, or promotion and wait for sign-off before it lands.
- **Cross-app read plane.** `search_everything` fans out across Brief and the other apps, and `resolve_references` turns mentions into resolved entities. Brief contributes documents to both.
- **Visibility preflight.** Before posting a Brief document link into a shared surface in another app, an agent should call `can_access` for the reader and drop anything the reader is not allowed to see. Brief still enforces visibility on the server, so this is a courtesy check that avoids dangling links.
- **Policies and webhooks.** Per-agent kill switches and tool allowlists (`agent_policies`) gate which `brief.*` tools a given service account may call, and outbound webhooks push subscribed Bolt events (such as `document.published` and `document.promoted`) to agent runners.

For the full tool catalog see `docs/apps/brief/mcp-tools.md`.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. Documents are co-edited live, with each other's cursors and changes appearing as they happen, a presence strip showing who is in the document, and a live call you can start without leaving it. Your location in Brief shows in the Bureau office. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Write your first document and publish it

Who: A team member new to Brief. **Goal:** Create a document, write some content, and publish it. **Before you start:** You are signed in to BigBlueBam and have the `read_write` scope.

Steps

1. Open Brief at `/brief/`.
2. Click **New Document** on the Home screen.
3. Type a title in the **"Document title..."** field.
4. Write your content in the body, using the toolbar or typing `/` for slash commands.
5. In the right sidebar, set **Visibility** to **Organization**, **Project**, or **Private**. Do not choose **Public**.
6. Click **Publish**.

Result: The document is saved with status Approved and Brief opens its detail page.

Related: Save a draft and publish; Set visibility. An agent can do the same with `brief_create` followed by `brief_update_content`.

Story: Start from a template

Who: Anyone who wants a structured starting point. **Goal:** Create a document pre-filled from a template. **Before you start:** At least one template exists. You have `read_write` scope.

Steps

1. Open **Templates** from the sidebar.
2. Click the template card you want.
3. The editor opens pre-filled with the template's content. Add a title and your own content.
4. Click **Save Draft** or **Publish**.

Result: A new document exists based on the template.

Related: Start from a template; Create a document. An agent can author the template itself with `brief_template_create`.

Story: Organize documents with folders and project scope

Who: Someone keeping a growing set of documents tidy. **Goal:** Create a folder and scope work to one project. **Before you start:** You have folder-create permission and `read_write` scope.

Steps

1. In the sidebar **Folders** section, click the add (+) button.

2. Type a name in the **"Folder name"** input and press Enter.
3. Click the project-scope selector at the top of the sidebar and choose a project so the list and folders filter to it.
4. Create documents while that scope is active; choose the project in the editor's **Project (optional)** menu when creating.

Result: Your documents are grouped by folder and project and easy to find through the sidebar filters.

Related: Folders and project scope; Browse the document list. An agent can rename, move, or delete a folder with `brief_folder_update` and `brief_folder_delete`, which the UI cannot do.

Story: Co-edit a document in real time

Who: Two teammates drafting together. **Goal:** Edit the same document at the same time and see each other's changes. **Before you start:** Both have edit access to the document and `read_write` scope.

Steps

1. The first person opens the document and clicks **Edit**.
2. The second person opens the same document and clicks **Edit**.
3. Both type. Each sees the other's cursor and a presence chip in the header.
4. Anyone reading the document on its detail page sees the edits appear live.
5. When finished, click **Save Draft** or **Publish**.

Result: Both edits merge into one document with no overwriting, and readers saw the changes as they happened.

Related: Edit an existing document and co-edit in real time.

Story: Find a document

Who: Anyone who knows roughly what they are looking for. **Goal:** Locate a document by title, content, or author. **Before you start:** You are signed in.

Steps

1. Open **Search** from the sidebar, or click the **"Search documents..."** box in the header.
2. Type at least 2 characters in the **"Type to search..."** box.
3. Read the results and click the one you want.

Result: Brief opens the matching document.

Related: Search; Browse the document list. An agent can search with `brief_search`, or `brief_semantic_search` for a meaning-based match.

Story: Review a document with comments

Who: A reviewer giving feedback. **Goal:** Leave a comment and have it resolved. **Before you start:** You can see the document. The author or an admin can resolve.

Steps

1. Open the document.
2. In the right sidebar under **Comments**, type into the **"Add a comment…"** box.
3. Click **Comment**.
4. When the feedback is addressed, the author or an admin clicks **Resolve** on the comment.

Result: The comment is recorded on the document and marked resolved when done.

Related: Comments. An agent can use `brief_comment_add` and `brief_comment_resolve`, and can anchor a comment to a span of text with the `anchor_text` option.

Story: Star a document for quick access

Who: Anyone with documents they return to often. **Goal:** Bookmark a document so it is one click away. **Before you start:** You can see the document.

Steps

1. Open the document.
2. Click the star button in the header (tooltip **Star document**).
3. Open the **Starred** screen, or the Starred list on Home, to find it again.

Result: The document appears in your Starred views.

Related: Star a document. An agent can toggle the star with `brief_star`.

Story: Duplicate a document to reuse its structure

Who: Someone who wants to start from an existing document. **Goal:** Make an editable copy. **Before you start:** You can see the source document.

Steps

1. Open the document.
2. Click **Duplicate** in the action bar.
3. Brief opens the new copy, named "<title> (copy)" and set to Draft.

Result: A draft copy exists for you to edit independently.

Related: Duplicate a document. An agent can use `brief_duplicate`, optionally copying into a different project.

Story: Export a document to a file

Who: Anyone who needs the document outside Brief. **Goal:** Download the document as Markdown or HTML. **Before you start:** You can see the document.

Steps

1. Open the document, or open it in the editor.
2. Click **Export**.
3. Choose **Markdown (.md)** or **HTML (.html)**.

Result: The file downloads to your machine.

Related: Export a document. An agent can fetch the same content with `brief_export_markdown` or `brief_export_html`.

Story: Archive a document and restore it later

Who: Someone clearing out finished or obsolete documents. **Goal:** Remove a document from normal views without deleting it, then bring it back. **Before you start:** You have edit access to the document.

Steps

1. Open the document and click **Archive** in the action bar.
2. To find it later, filter the Documents list by the **Archived** status chip.
3. Open the archived document and click **Restore**.

Result: The document is hidden while archived and returns to Draft status when restored.

Related: Archive and restore. An agent can use `brief_archive` and `brief_restore`.

Story: Promote a document into Beacon knowledge

Who: Someone graduating a finished document into the knowledge base. **Goal:** Turn a Brief document into a Beacon article. **Before you start:** The document is not already promoted, and you have the promote permission and edit access.

Steps

1. Open the document.
2. Click **Promote to Beacon** in the action bar.
3. Brief creates the Beacon article and opens it at `/beacon/<id>`.

Result: The document is recorded as promoted and a matching Beacon article exists.

Related: Promote to Beacon. An agent can use `brief_promote_to_beacon`.

Story: Link a spec document to a task (agent driven)

Who: An agent connecting a specification document to the work that implements it. **Goal:** Attach a Brief document to a Bam task so it shows under Linked Items. **Before you start:** The agent has `read_write` scope and access to both the document and the task. Linking has no human UI.

Steps

1. The agent calls `brief_link_task` with the document (by UUID, slug, or exact title) and the task (by UUID or a human reference like FRND-42), choosing a link type such as `spec`.
2. The link is created.
3. A human opening the document sees it under **Linked Items** in the detail sidebar; clicking it opens the task in Bam.

Result: The document and task are connected and the relationship is visible to readers.

Related: Linked Items; Working with AI agents. An agent can also link a Beacon article with `brief_link_beacon` and remove either link with `brief_link_remove`.

Story: Snapshot and restore a version (agent driven)

Who: An agent or integration maintaining version history. **Goal:** Record a numbered snapshot of a document and roll back to it later. **Before you start:** The agent has edit access and `read_write` scope. There is no human button for this.

Steps

1. The agent calls `brief_version_create` with an optional label and change summary to record a snapshot.
2. A human opening the document sees the snapshot listed read-only under **Version** in the detail sidebar.
3. To roll back, the agent calls `brief_version_restore` with the target version, which restores the content and records a "Restored from version N" snapshot.

Result: The document has a versioned history and can be returned to an earlier state.

Related: Read a document; Working with AI agents. An agent can compare two snapshots with `brief_version_diff`.

Story: Agent authors a document end to end

Who: An automation that produces and files a document. **Goal:** Create, fill, link, and graduate a document without a human in the editor. **Before you start:** The agent has `read_write` scope and the relevant permissions.

Steps

1. The agent calls `brief_create` with a title and visibility.
2. It fills the body with `brief_update_content`, or builds it incrementally with `brief_append_content`.
3. It connects the document to related work with `brief_link_task`.
4. When the document is ready as knowledge, it calls `brief_promote_to_beacon`.

Result: A complete, linked document exists, and a Beacon article was created from it, all without manual editing.

Related: Working with AI agents; Promote a document into Beacon knowledge. If the promotion should pause for review, the agent can file a `proposal_create` first and wait for a human `proposal_decide`.

Related

- **Bam** - Brief pulls its project list from Bam and links documents to Bam tasks. Linked tasks open in Bam at `/b3/tasks/:id`.
- **Beacon** - Documents promote into Beacon knowledge-base articles, and documents can be linked to Beacon articles. See `docs/apps/beacon/help.md`.
- **Bolt** - Brief publishes `document.created`, `document.updated`, `document.published`, and `document.promoted` events (source `brief`) so Bolt automations can react.
- Brief's MCP tool catalog: `docs/apps/brief/mcp-tools.md`.
- Brief's product guide: `docs/apps/brief/guide.md`.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What are Brief's four document statuses?

a) Draft, In Review, Approved, and Archived
b) Draft, Published, Locked, and Deleted
c) Open, In Progress, Closed, and Archived
d) New, Editing, Final, and Retired

2. Which visibility level does the menu list but reject as invalid?

a) Private
b) Public
c) Project
d) Organization

3. What status does a document have after you click Publish, and what status after Save Draft?

4. How long can a document icon be?

a) Up to 1 character
b) Up to 2 characters
c) Up to 5 characters
d) Any length

5. What are the three valid visibility levels and who can see a document at each?

6. How many MCP tools does Brief expose to agents?

a) 12
b) 30
c) 48
d) 60

7. What technology does Brief use to merge concurrent edits without overwriting, and over what connection?

8. What happens when you promote a document to Beacon?

a) Brief creates a Beacon article from it, and the promotion is one-way
b) The document is deleted after the article is created
c) It can be promoted again to refresh the article
d) A draft copy is made and nothing is sent to Beacon

9. Which folder actions can the Brief UI do, and which require an agent or the API?

10. How many characters must you type before Search returns results?

a) At least 1
b) At least 2
c) At least 3
d) At least 5

11. Why should you avoid the "Brief summary (optional)" field, and what should you do instead?

12. What does the Duplicate action create?

a) A Draft copy named "<title> (copy)"

b) An Approved copy with the same title
c) A new version on the same document
d) A Beacon article from the document

13. When you restore an archived document, what status does it return to?

CHAPTER 16

Bureau

The office is wherever you all are.

Bureau is the virtual office that rides along with everything else in the suite, so "who is around right now" and "let's all go look at this together" are one click away. You browse floors, drop into rooms with live occupant dots, and a floating docked box shows where you are, who else is on your page, and the actions to knock, summon, hunt, and book. This chapter walks you from dropping into your first room to running a summon, booking a conference room, and reading door states. By the end you will know how presence works, how to interrupt someone politely, and where an agent can be a first-class occupant beside you.

At a glance

- A spatial office with floors, rooms, and live occupant dots for everyone around
- Each room is a real-time audio huddle you join just by entering
- A docked box that follows you across every app with one-click invite, hunt, and summon
- Polite interruptions: knock on closed offices, with a 30-second auto-timeout
- Server-side presence so your room and call survive moving between pages

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Browse floors . The live floor view (presence canvas) . Recent chats . Move yourself between rooms . See who is in a room . Knock and respond to a knock

> *Bureau is a virtual office layer that stays open around everything else you do in the suite, so that "who is around right now" and "let's all go look at this together" are one click away no matter which app you are in. Reach for it when you want a sense of presence, a quick huddle, or a way to pull your teammates onto the same screen.*

Overview

Bureau gives your org a spatial office[162]. You browse **floors**, drop into **rooms**, and see live occupant dots for everyone who is around. Each room[163] maps to a real-time audio room, so entering a room can start or join a voice huddle. Around all of that, a floating docked box[164] rides along inside every app in the suite, showing where you are, who else is on the same page, and giving you one-click actions to invite, hunt[165], or pull everyone to where you are looking.

Bureau solves the "remote office" feel for small-to-medium teams. It answers three questions that are otherwise hard to answer over chat alone: who is available right now, can I interrupt them, and can we all jump to the same place. Knocking on a colleague's office, summoning a room to a Board canvas, and booking[166] a conference room are all first-class actions.

> ***Bureau answers the three questions chat cannot: who is around, can I break in, and can we all land on the same screen.***

Bureau is woven into the rest of the suite rather than standing alone. The docked box is surfaced by Board and Banter (and every other SPA) so audio huddles follow you across products. Summons and rings jump people to a URL in Board, Brief, Bond, and others. Bookings mirror to Book events. Door states, knocks, and status changes emit events that Bolt can automate against, and daily floor[167] utilization rolls up into Bench.

The core objects you work with are floors, rooms (including personal **offices**), your **presence**[168] (a status plus where you currently are), **knocks** on closed doors, **summons** that pull a room to a destination, and **bookings** that reserve a room for a window.

Key concepts

- **Floor** - One navigable office map. A floor owns a layout, an optional background image, a slug used in its URL, and a position in the list. One floor can be marked the default.

[162] **Office.** A room of type Office that has an owner. The owner can be knocked on, sets the door, and admits or declines visitors. Think of it as a personal office.
[163] **Room.** An addressable space on a floor. Each room maps one-to-one to an audio room. Rooms come in 8 types: **Office (single owner)**, **Huddle**, **Conference**, **Meeting**, **Open space**, **Lounge**, **Focus**, and **Lobby**.
[164] **Docked box.** The floating Bureau overlay rendered inside every SPA. It shows your room, your co-occupants, the page you are viewing, the call controls, and the Bring everyone here / Invite / Hunt actions.
[165] **Hunt.** Locate one org member and jump to wherever they are. It respects DND and filters out destinations you cannot access.
[166] **Booking.** A reservation of a bookable room for a window, mirrored to a Book event. Access is **open** (non-attendees may wander in) or **locked** (private for the meeting).
[167] **Floor.** One navigable office map. A floor owns a layout, an optional background image, a slug used in its URL, and a position in the list. One floor can be marked the default.
[168] **Presence.** Your live state: which floor and room you are in, your status, and the page you are currently viewing. Your presence lives on the server, so moving between pages never drops your session.

- **Building**[169] - An optional grouping of floors for multi-site orgs. It exists in the data model and a new floor can carry a `building_id`, but there is no building management screen in the app today.
- **Room** - An addressable space on a floor. Each room maps one-to-one to an audio room. Rooms come in 8 types: **Office (single owner)**, **Huddle**, **Conference**, **Meeting**, **Open space**, **Lounge**, **Focus**, and **Lobby**.
- **Office** - A room of type Office that has an owner. The owner can be knocked on, sets the door, and admits or declines visitors. Think of it as a personal office.
- **Door state (privacy)** - How a room treats visitors. Three values: **Open** (anyone can enter), **Knock**[170] (visitors must knock to be let in), and **Private** (only people on the room's access list). The room stores a durable default; live overrides such as a short do-not-disturb window apply on top of it.
- **Room access list (ACL)** - Per-user grants for private rooms, with role **member** or **manager**. Managers can edit the room and manage its access list.
- **Presence** - Your live state: which floor and room you are in, your status, and the page you are currently viewing. Your presence lives on the server, so moving between pages never drops your session.
- **Presence status**[171] - One of 6 values: **available**, **busy**, **dnd**, **focus**, **away**, **in_meeting**. The docked box exposes a head-down **DND** toggle that flips between available and DND.
- **Knock** - An async request to enter a closed office. A knock auto-times-out after 30 seconds if the owner does not respond.
- **Summon (Bring everyone here)** - Pulls every eligible co-occupant of your current room to a URL in another app. Each recipient is access-checked first, so people who cannot see the destination are not pinged with a dead link.
- **Ring (Invite)** - Pushes an incoming-call overlay to one specific person on a content surface. It respects DND. An admin can Force Invite, which bypasses the accept prompt with a 3-second cancellable auto-navigate.
- **Hunt** - Locate one org member and jump to wherever they are. It respects DND and filters out destinations you cannot access.
- **Booking** - A reservation of a bookable room for a window, mirrored to a Book event. Access is **open** (non-attendees may wander in) or **locked** (private for the meeting).

[169] **Building.** An optional grouping of floors for multi-site orgs. It exists in the data model and a new floor can carry a `building_id`, but there is no building management screen in the app today.
[170] **Knock.** An async request to enter a closed office. A knock auto-times-out after 30 seconds if the owner does not respond.
[171] **Presence status.** One of 6 values: **available**, **busy**, **dnd**, **focus**, **away**, **in_meeting**. The docked box exposes a head-down **DND** toggle that flips between available and DND.

- **Surface huddle**[172] - The audio room attached to any content surface (a Board canvas, a Brief doc, a Bond deal). The docked box auto-joins it when you navigate to that surface.
- **Docked box** - The floating Bureau overlay rendered inside every SPA. It shows your room, your co-occupants, the page you are viewing, the call controls, and the Bring everyone here / Invite / Hunt actions.
- **Room chat**[173] - Ephemeral text chat scoped to a room or surface, kept 24 hours by default and recoverable later under Recent chats.

> **TIP**
> Room chat is ephemeral on purpose, but if a thread mattered, an admin can open its transcript and set Retention to 2 days, 3 days, 1 week, or Retain permanently before the 24-hour clock runs out.

> **NOTE**
> Only office rooms with an assigned owner can be knocked on. You cannot knock on your own office, and you cannot knock on an office that has no owner.

> ***A summon teleports your whole room to a URL, but only the people who can actually open it get pulled.***

Where to find it

Bureau is served at `/bureau/`. You must be logged in to BigBlueBam first; Bureau shares the Bam session. If you are not logged in you will see "Please log in to BigBlueBam first to access Bureau." with a **"Go to BigBlueBam Login"** link.

The sidebar (brand label **"Bureau"**) groups navigation under:

- **Floors**: **"All floors"** (the Floors landing), then one row per floor with a live occupancy badge.
- **Rooms**: **"Book a room"** (the Room booking screen at `/bureau/booking`).
- **History**: **"Recent chats"**.
- **Admin** (org admin, owner, or SuperUser only): **"Edit floors"**, **"Offices"**, and **"Settings"**.

Floor and office administration require org admin, owner, or SuperUser. Audio huddles require LiveKit to be configured and the platform calling switch to be on; if calling is disabled, joining a room's audio returns a "calling disabled" error.

[172] **Surface huddle.** The audio room attached to any content surface (a Board canvas, a Brief doc, a Bond deal). The docked box auto-joins it when you navigate to that surface.
[173] **Room chat.** Ephemeral text chat scoped to a room or surface, kept 24 hours by default and recoverable later under Recent chats.

Feature reference

Browse floors

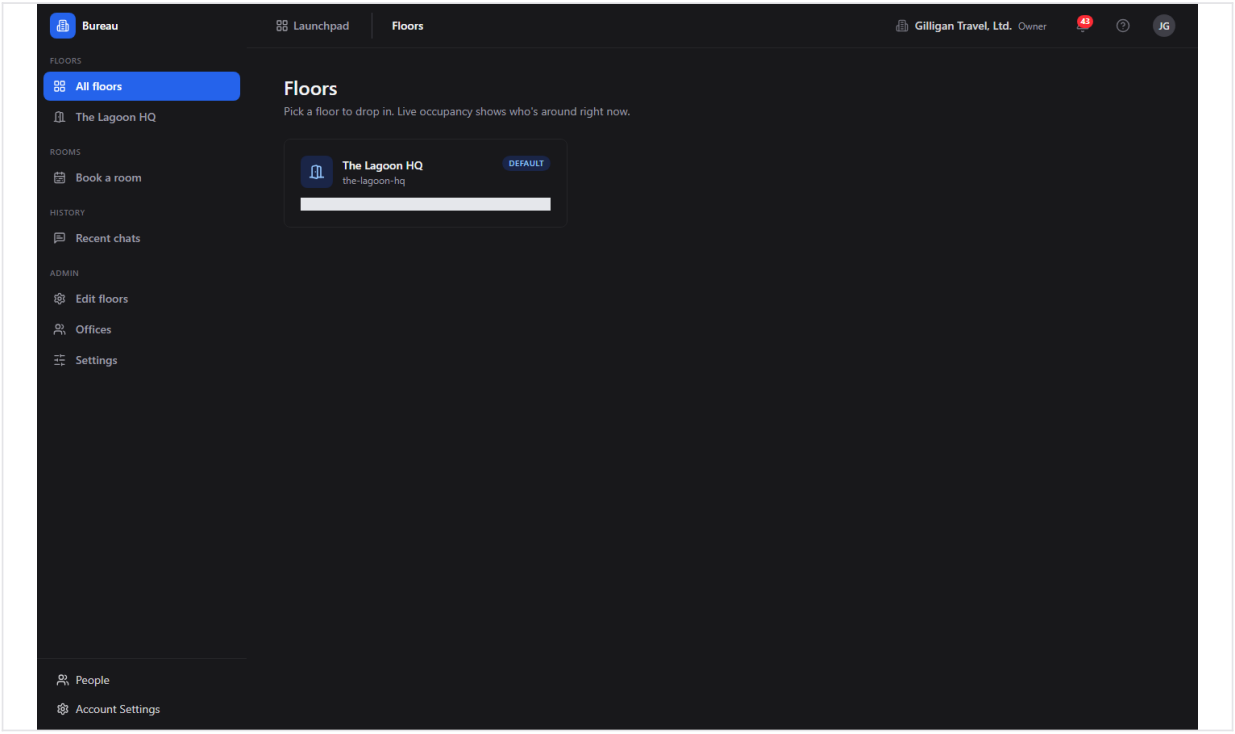


Figure 16.1 Floor directory

The Floors landing is where you pick a place to drop in.

To browse floors:

1. Open `/bureau/`. The page heading is **"Floors"** with the subtitle "Pick a floor to drop in. Live occupancy shows who's around right now."
2. Each floor card shows the floor name, its slug, a **"Default"** pill on the default floor, and a live count such as "3 people here now".
3. Click a floor card to open its live view.

If no floors exist yet you will see **"No floors yet"** and "Ask an org admin to create the first floor under Admin to Floors."

The live floor view (presence canvas)

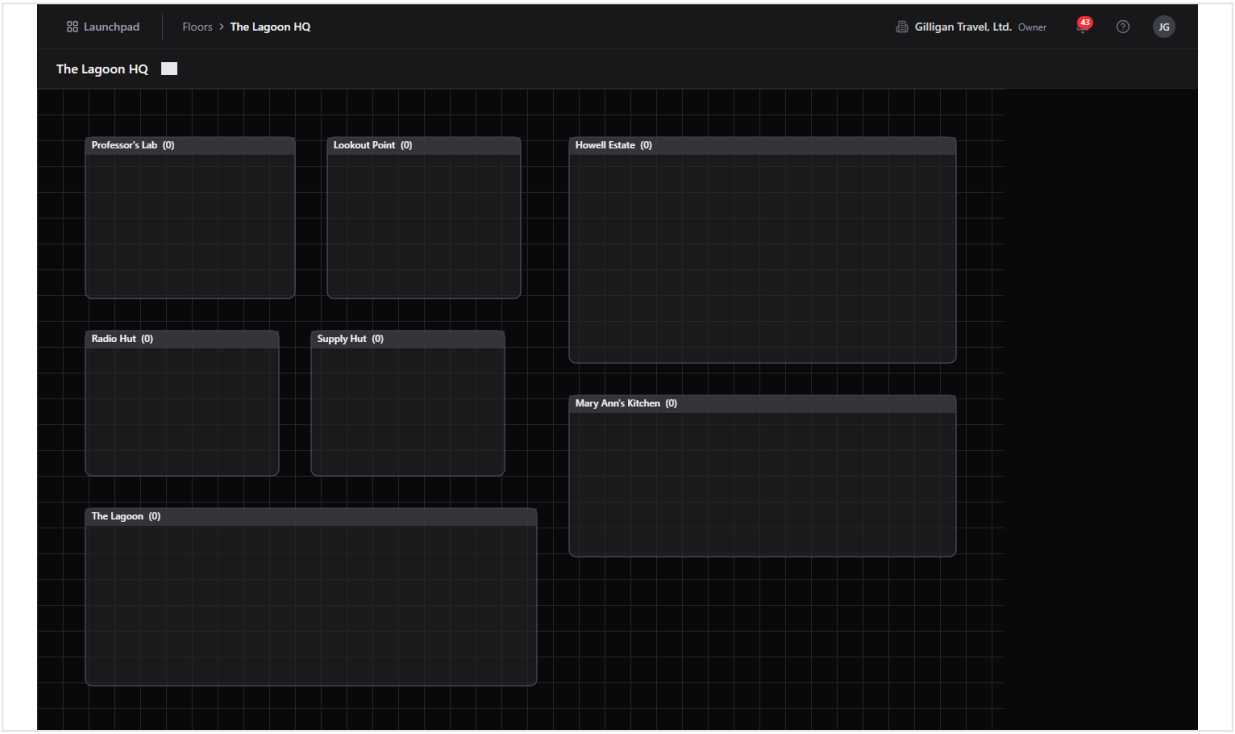


Figure 16.2 Live floor

The floor view is the spatial map. It draws one rectangle per room over a grid background, with a dot for each occupant. It is a Canvas2D scene (the canvas carries the class `floor-canvas`), so occupant dots are painted, not DOM elements.

To use the floor view:

1. From the Floors landing, click a floor. The URL becomes `/floors/:id`.
2. Read the status bar at the top: it shows the floor name and a connection chip that reads **"Live"** when the real-time link is healthy, or the raw connection status otherwise.
3. Click a room rectangle to enter it. Entering a room mints your audio token and places your dot inside that room. Rooms without an assigned layout zone are auto-arranged in a 4-column grid so the floor still navigates.
4. When you are inside a room, a **"Leave room"** button appears in the status bar. Click it to step back out.

The live map updates as people enter, leave, and change status, over the Bureau real-time connection.

Recent chats

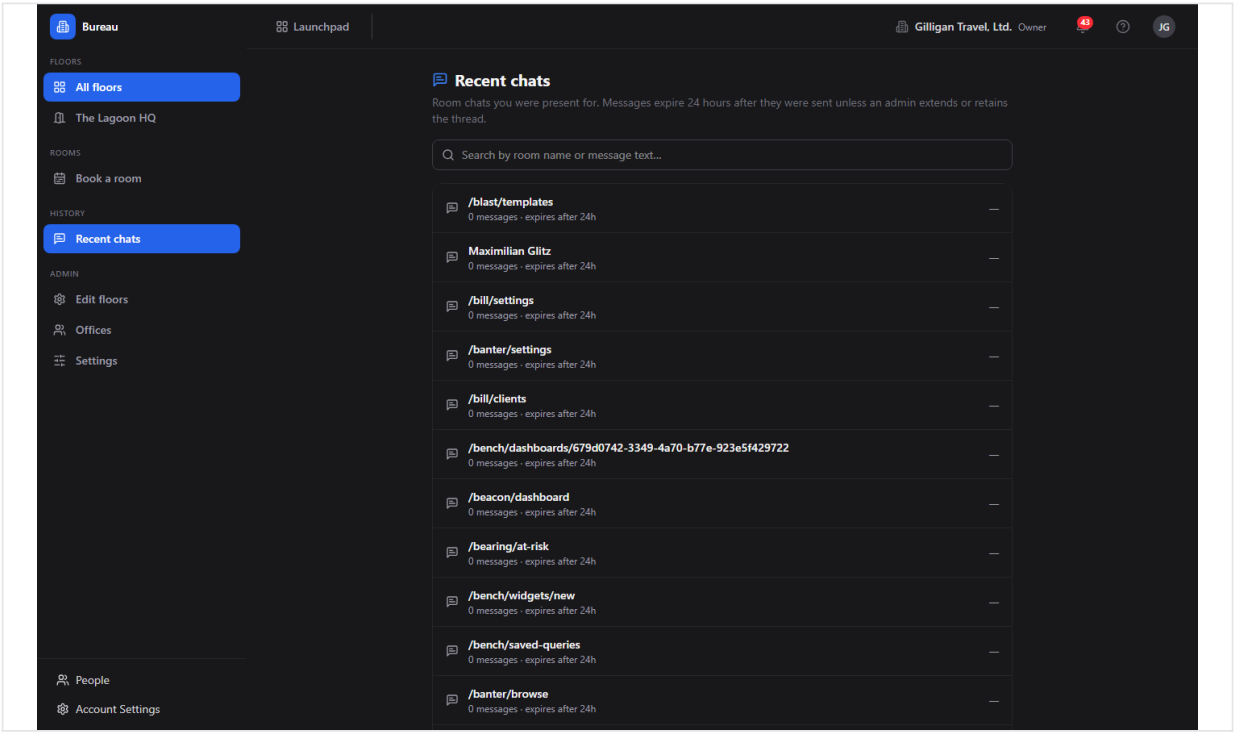


Figure 16.3 Recent chats

Room chats are ephemeral, but you can recover what was said for rooms you were present in.

To review past room chat:

1. In the sidebar, under History, click **"Recent chats"**. The heading is **"Recent chats"** with the subtitle "Room chats you were present for. Messages expire 24 hours after they were sent unless an admin extends or retains the thread."
2. Use the search box ("Search by room name or message text...") to filter the thread list.
3. Each thread shows its room label, message count, retention text, and last-message time. A shield icon marks a thread that has been retained.
4. Click a thread to open its transcript dialog. Expired messages show "All messages in this chat have expired."

If you are an admin, the transcript dialog offers a **"Retention"** dropdown with the options "24 hours (default)", "2 days", "3 days", "1 week (max)", and "Retain permanently". Choose one to extend or pin the thread.

If you have no threads yet, you will see "No room chats yet - open the chat panel on the Bureau widget anywhere in the suite."

Move yourself between rooms

Moving rooms is how you change where you are standing and which audio huddle you are in.

To move:

1. Open a floor at `/floors/:id`.
2. Click the destination room rectangle. You enter that room and your audio reconnects to it.
3. To leave without entering another room, click **"Leave room"**.

Door state is enforced on entry. You can always enter an Open room. A Knock room requires you to knock first (see below). A Private room requires you to be on its access list; otherwise entry is refused.

> **TRY IT**
> Open a floor at /floors/:id, click an Open room rectangle to drop your dot inside and connect audio, then click "Leave room" in the status bar to step back out.

AI agent note: an agent moves itself with `bureau_move_self`, which mints the room token and registers the agent's session so it appears in `bureau_who_is_in_room` and on the floor. Private rooms the agent is not on the access list for return a permission error.

See who is in a room

Before knocking, summoning, or joining, you often want to know who is already there.

To see occupants:

1. On the floor view, look at the dots inside each room rectangle.
2. Enter a room, then read the docked box: the **"In:"** row names the room and the people-here count, and the occupants list names everyone present (or **"Just you"** when you are alone). Agent occupants are prefixed "(A) ".

AI agent note: `bureau_who_is_in_room` returns a room's detail and its live occupant ids. It is access-filtered, so private rooms you cannot see return not-found. Use it before `bureau_summon` or `bureau_knock` to decide who would actually be pulled or disturbed. For co-presence by URL rather than by room, use `bureau_who_is_here`, which lists the other users on a given content surface.

Knock and respond to a knock

A knock is the polite way into a closed office.

To knock on an office:

1. On the floor, click the office whose door is set to Knock (or whose live override blocks direct entry).
2. The owner receives an incoming-knock toast in their docked box.
3. Wait for the owner's decision. If no one answers within 30 seconds the knock auto-times-out.

Only office rooms with an owner can be knocked on. You cannot knock on your own office, and you cannot knock on an office that has no owner.

To respond when someone knocks on your office, use the incoming-knock toast in your docked box:

1. **"Let in"** admits the visitor; they receive a fresh token and join your room.
2. **"Not now"** defers the knock; the visitor is told you are busy.
3. **"Decline"** politely rejects the visitor.

If you are in DND when someone tries to knock, the knock is blocked (the server returns 423) and the visitor is offered a leave-a-note path that delivers a Banter direct message prefixed "[Bureau knock note] ".

AI agent note: `bureau_knock` creates the knock; if the owner is in DND it returns a 423 with a leave-a-note pointer, which an agent should surface rather than retry. `bureau_respond_knock` resolves a knock as the owner with `admit`, `defer`, or `decline`. `bureau_knock_inbox` lists the pending knocks waiting at the agent's own door, and `bureau_leave_note` sends the DND fallback DM.

Summon (Bring everyone here)

A summon pulls everyone in your current room to whatever you are looking at in another app.

To bring everyone here:

1. Be in a Bureau room with at least one other person.
2. Navigate to the resource you want to share (for example a Board canvas, a Brief doc, or a Bond deal). The docked box **"Viewing:"** row shows your current location.
3. Click **"Bring everyone here"** in the docked box. Bureau access-checks each co-occupant against that destination, then sends a summon to everyone who can reach it.
4. Each recipient sees a **"Summon"** toast that names you and the destination, with **Join** and **Stay here** buttons. If auto-follow is enabled, the toast shows a "Going in Ns..." countdown with a "cancel" link, then navigates automatically.
5. Anyone who lacks access lands on the denied list, so you can follow up by granting access and re-summoning.

AI agent note: `bureau_summon` is high-impact and uses the `confirm_action` two-step. Call it once to preview the planned recipients, then again with `confirm_action: true` to send. The caller must be in a Bureau room or the call fails with 400. After sending, `bureau_get_summon` inspects who was eligible versus denied, and `bureau_summon_grant_access` grants access to denied users and re-summons them in one step (also a two-step confirm).

Ring (Invite)

Ring calls one specific person to a surface, like ringing their phone.

To invite one person:

1. On any located page, click **"Invite..."** in the docked box. The Invite popover opens.
2. Pick the member to ring. They receive an incoming-call overlay.
3. When they accept, they are navigated to your surface and auto-join the huddle.

DND is respected: if the recipient is head-down, a normal ring does not get through. Admins can right-click **"Invite..."** to Force Invite, which bypasses the accept prompt with a 3-second cancellable auto-navigate.

Hunt

Hunt finds one teammate and takes you to them.

To hunt a teammate:

1. Click **"Hunt..."** in the docked box. The Hunt popover opens.
2. Pick the member you want to find.
3. You jump to wherever they currently are.

Hunt is org-scoped: the teammate must share your active org. It respects DND (a head-down member cannot be located) and runs a destination-access preflight, so if you cannot see the surface they are on, the result is the same as "not located" - their location is never leaked through a URL you could not open.

AI agent note: `bureau_where_is_user` is the real, implemented Hunt tool. It returns `{ located: true, url, app, label, status }` or `{ located: false }` when the target is offline, in DND, in a different org, or on a surface the caller cannot see. Prefer it over the legacy `bureau_locate_user` stub.

Book and cancel a room

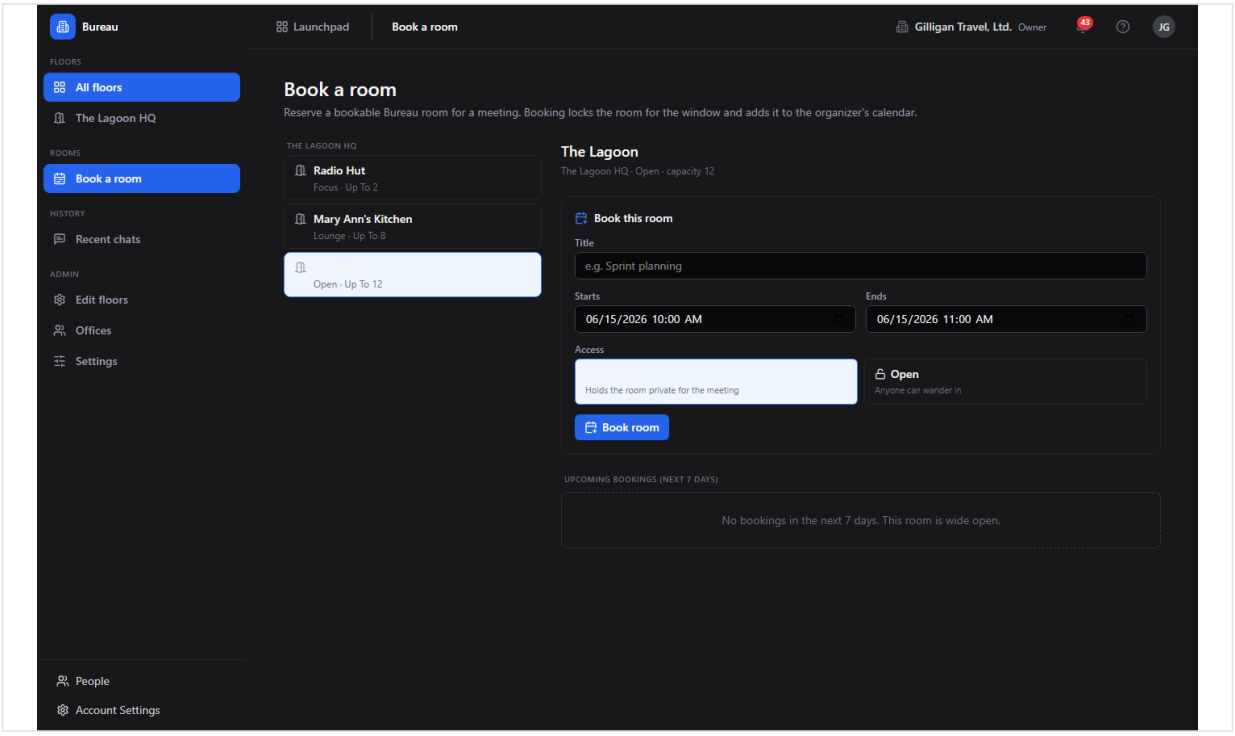


Figure 16.4 Room booking

Bookable rooms can be reserved for a window; the reservation mirrors to a Book event. There is a dedicated **Room booking** screen in the Bureau SPA for this.

To book a room:

1. Open `/bureau/booking`. The page heading is **"Book a room"** with the subtitle "Reserve a bookable Bureau room for a meeting. Booking locks the room for the window and adds it to the organizer's calendar."
2. The left column lists the org's bookable rooms grouped by floor. Each room shows its type and capacity. Click one to open its schedule on the right. If the org has no bookable rooms you see **"No bookable rooms"** and a hint that an admin can mark a room bookable in the floor editor.
3. In the **"Book this room"** form, enter a **Title** (placeholder "e.g. Sprint planning"), pick **Starts** and **Ends** (datetime-local inputs default to the next hour and the hour after), and choose **Access**: **Locked** ("Holds the room private for the meeting") or **Open** ("Anyone can wander in").
4. Click **"Book room"**. Bureau writes the booking, anchors it to a Book event, and schedules lifecycle jobs that flip the room's privacy at the start time and clear it at the end time. Validation errors (missing title, end-before-start) and the server's "member bookings are disabled" 403 surface inline.
5. The **"Upcoming bookings (next 7 days)"** list below the form shows each reservation with its title, window, and the lock/open access badge.

To cancel a booking, click **"Cancel"** on its row in the upcoming-bookings list. Only the organizer or an org admin sees the Cancel button. Cancellation is a soft delete that drops the scheduled jobs and makes a best-effort attempt to cancel the linked Book event.

> **WARNING**
> Cancelling a booking is a soft delete that also makes a best-effort attempt to cancel the linked Book event. Only the organizer or an org admin sees the Cancel button, so confirm you are the one who should be tearing the reservation down.

Booking is gated by the org's `members_can_book` setting (see Admin - Settings below): when it is off, only admins and owners can reserve rooms, and a member's booking attempt 403s with the server message shown inline.

AI agent note: `bureau_book_room` and `bureau_cancel_booking` both use the `confirm_action` two-step: preview first, then confirm. `bureau_list_bookings` lists active bookings for a room over a window (default next 7 days), including the Book event back-link, and `bureau_update_booking` edits a booking's title, window, or access (organizer or admin only).

Door states (set a room's privacy)

A room's door state controls who can walk in.

The three states are **Open**, **Knock**, and **Private**. The durable default is set on the room, and live overrides (a lock or a short DND window) apply on top of it.

To change the durable default in the app, edit the room in the Floor editor and set its **Door default** (see Admin - Floor editor below). To change it directly, the office owner, a room manager, or an org admin can patch the room's door.

The Bureau SPA's room inspector only sets the durable default. Live lock and DND overrides are driven through the docked box and the real-time connection. The door button in the docked box (tooltip **"Toggle door privacy"**) flips the live override while you are in a room.

AI agent note: `bureau_set_door_state` sets a room's durable default privacy. It is restricted server-side to the office owner (for offices), a room manager, or org admins and owners.

Set your status and DND

Your status tells teammates whether you can be interrupted.

The canonical status set is **available**, **busy**, **dnd**, **focus**, **away**, and **in_meeting**. The surface for changing it is the docked box and the live connection, not a settings page.

To go head-down:

1. In the docked box header, click the **"DND"** toggle (its tooltip reads "Go head-down: block invites and hunts (DND)").
2. While DND is on, invites ring as blocked (callers get the leave-a-note path to your Banter direct messages) and hunts cannot locate you. Only an admin or SuperUser Force Invite gets through.
3. Click **"DND"** again to return to available.

The in-app surface for changing status interactively is the docked box DND toggle; the rest of the status set rides the live connection. Your chosen status is now persisted on the server, so it survives a reconnect and applies even when you have no live web session - which is what lets an agent set it for you over MCP.

AI agent note: `bureau_set_status` is now wired (it PATCHes `/me/status` in the Bureau API and persists durably), so an agent can change your status on your behalf. The tool exposes the four-value subset **available**, **busy**, **away**, **dnd**; the underlying endpoint also accepts **focus** and **in_meeting** for status set through the docked box. Your live status, room, and floor are read back org-wide by `bureau_get_presence` and per-user by `bureau_locate_user`.

The cross-app docked box (audio huddles, mic, cam, screen)

The docked box is the floating Bureau overlay that rides inside every app, including Board and Banter. It is your portable presence and call console.

The box header shows a status dot and the **"Bureau"** label, plus:

- **"CHAT"** toggle - opens the ephemeral room chat panel (its tooltip reads "Open room chat (ephemeral - 24h)"); a badge shows unread messages.
- **"DND"** toggle - the head-down switch described above.
- A popout (picture-in-picture) button and a collapse chevron.

Below the header:

- The **"In:"** row names your current room (or **"No room"**) and the people-here count.
- The occupants list names who is with you, or shows **"Just you"**. Agents are prefixed "(A) ".
- The **"Viewing:"** row shows a human-readable name of the page you are on.
- **"Bring everyone here"** appears when you are in a room with others on a located page.
- **"Invite..."** and **"Hunt..."** are available on located pages.

When you are in an audio huddle, the call strip shows your controls:

- The mic button reads **"mic"** when muted and **"mic on"** when live. Click to toggle.
- The cam button reads **"cam"** when off and **"cam on"** when on. Click to toggle.
- The screen button reads **"screen"** when you are not sharing and **"share"** when you are. Click to start or stop a screen share.
- The door button (tooltip **"Toggle door privacy"**), shown only while you are in a room, flips that room's live privacy override between Private and Open.
- Video tiles render for participants with camera or screen on.

The box is draggable by its header and collapsible. Because your presence lives on the server, navigating between pages does not drop your room or your call; the box re-attaches to the surface huddle of wherever you land.

Admin - Floors list

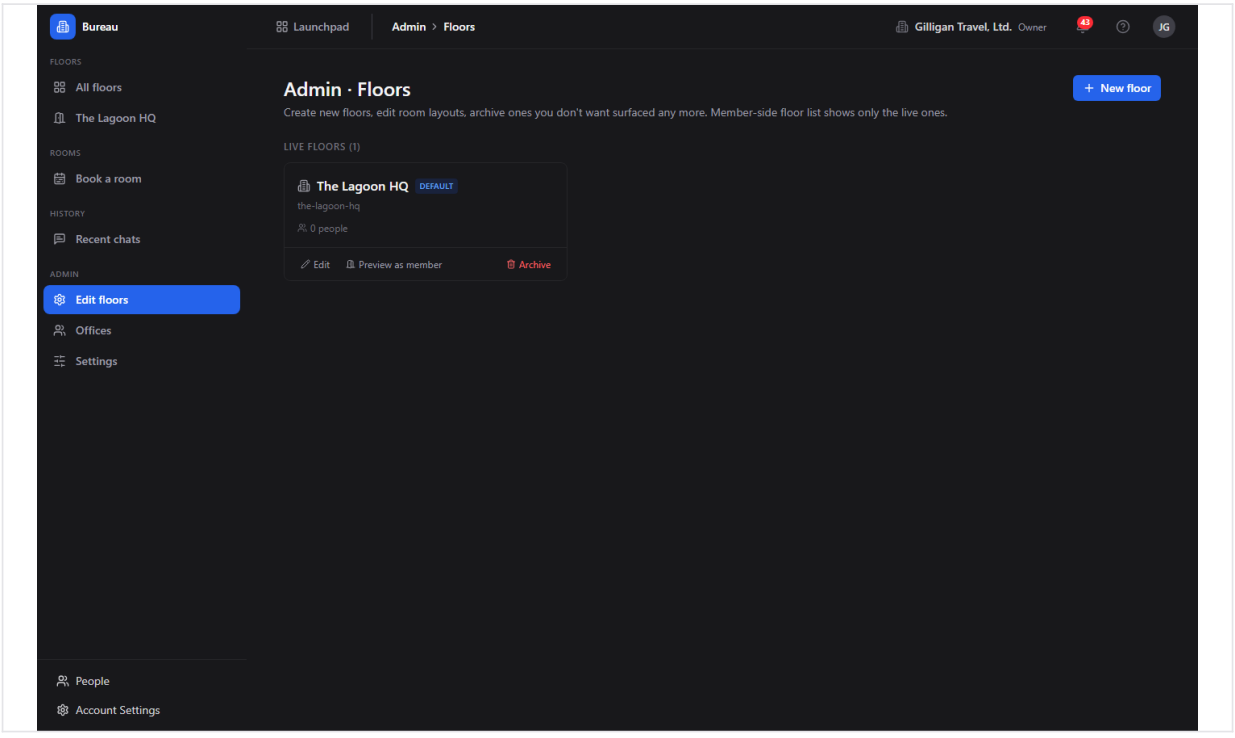


Figure 16.5 Manage floors

Admins manage the org's floors here.

To manage floors:

1. In the sidebar Admin group, click **"Edit floors"**. The heading is **"Admin · Floors"**.
2. The page lists "Live floors (n)" and "Archived (n)".
3. Live floor cards offer **"Edit"**, **"Preview as member"**, and **"Archive"**. Archived rows offer **"Restore"**.
4. To add a floor, click **"New floor"**. In the dialog, enter a **Name** (placeholder "e.g. Engineering 3F") and a **Slug** (auto-generated, lowercase letters, numbers, and dashes; helper "Used in the floor URL: /floors/{slug}"). Click **"Create + open editor"**.

Archiving a floor prompts a confirmation and hides it from the member-side floor list; you can restore it later from this page. Non-admins see **"Floor administration requires admin or owner"**.

Admin - Floor editor

The Floor editor is the canvas where you place rooms on a floor.

To lay out a floor:

1. Open a floor in the editor (via "New floor" or "Edit"). The top toolbar shows a Back arrow, an editable floor-name input, a "{n} zones · WxH" readout, the tool selector **"Select"** / **"New room"** / **"New office"**, the image-underlay control, and **"Save"**.
2. To add a space, pick **"New room"** or **"New office"** and click-drag on the canvas. The empty-inspector hint reads "Click a zone to edit it, or pick "New room" / "New office" and click-drag on the floor."
3. Select a zone to open the room inspector on the right. Set:
4. To delete a room, use **"Remove zone"** in the inspector footer. On save it soft-deletes the room and "Live occupants are evicted to the lobby."
5. To set a background underlay, use the image-underlay control to upload an image; you can later Replace or Remove it.
6. Click **"Save"**. Bureau writes the layout and floor name, then creates, updates, or deletes rooms to match the canvas.

Admin - Offices

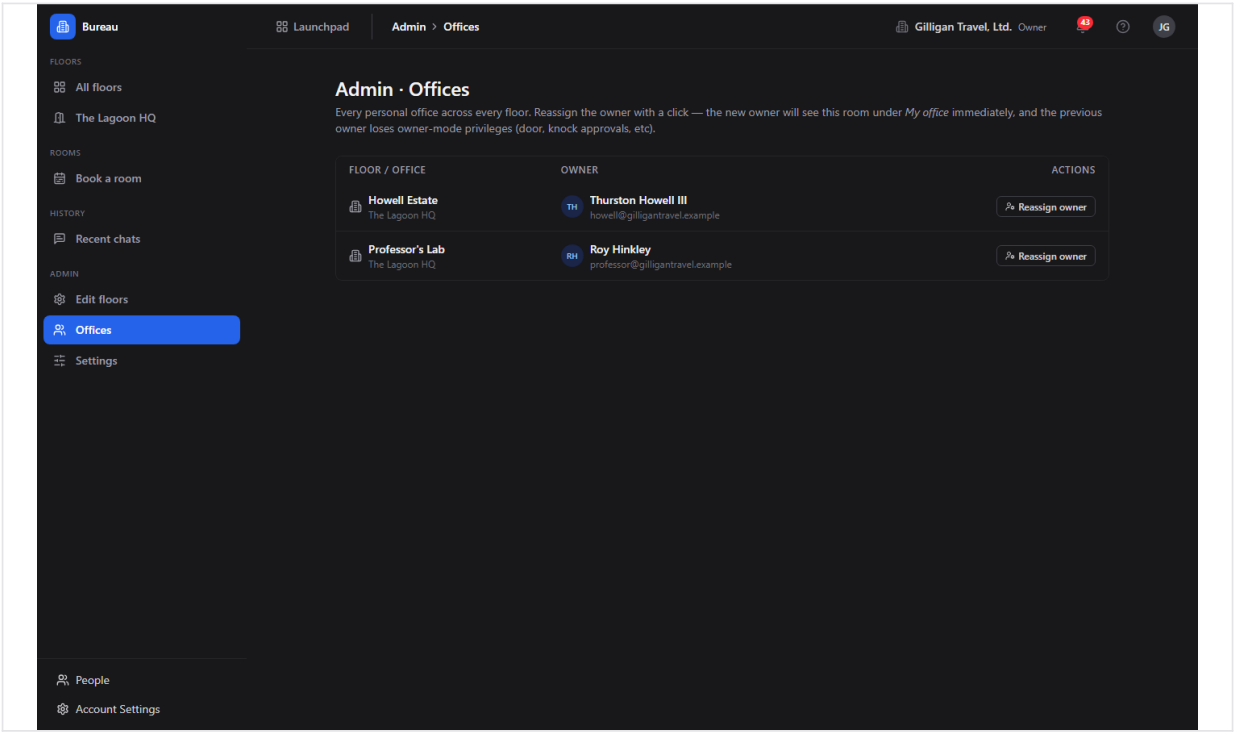


Figure 16.6 Office assignments

The Offices screen is where you assign or reassign who owns each personal office.

To assign an office owner:

1. In the sidebar Admin group, click **"Offices"**. The heading is **"Admin · Offices"**.
2. The table has columns **"Floor / Office"**, **"Owner"** (avatar, name, and email, or **"Unassigned"**), and **"Actions"**.
3. Click **"Reassign owner"** on a row. In the "Reassign {office}" dialog, search by name or email (placeholder "Search name or email..."); the current owner carries a "Current" tag.
4. Pick the new owner to assign the office to them. The new owner sees this room under "My office" immediately, and the previous owner loses owner-mode privileges (door, knock approvals).

Non-admins see **"Office administration requires admin or owner"**.

Admin - Settings

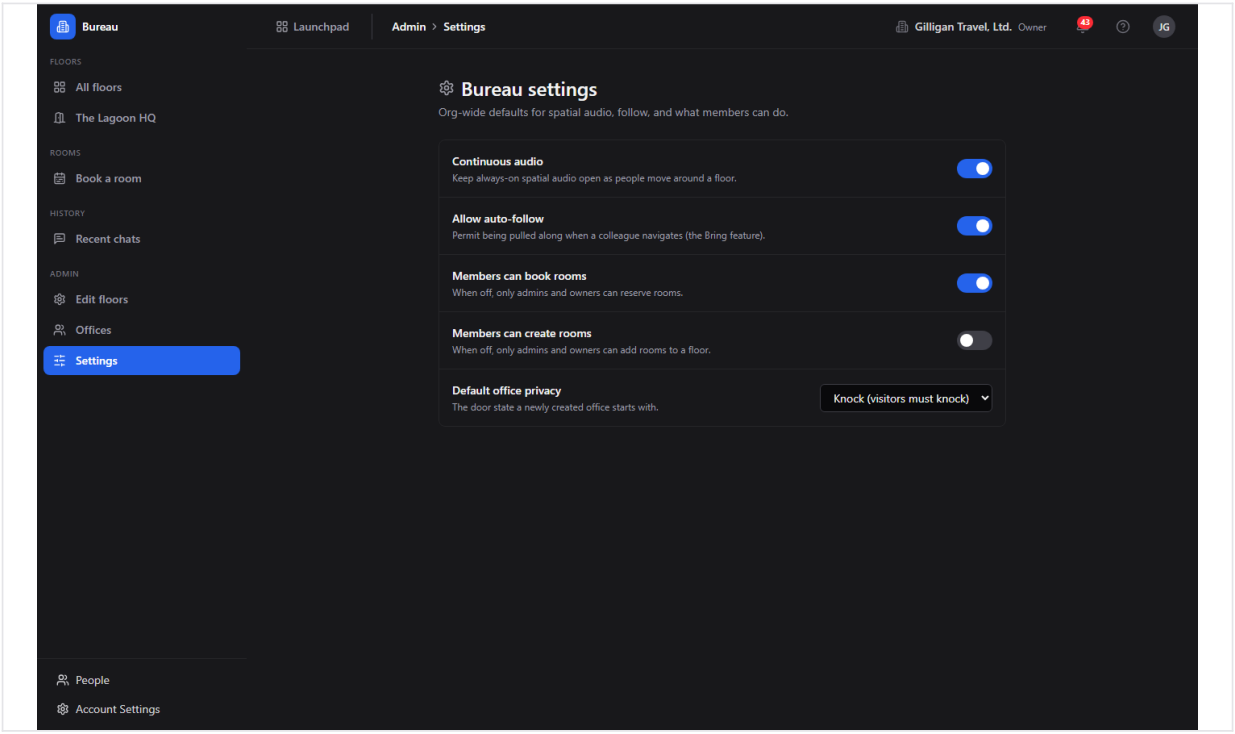


Figure 16.7 Admin settings

Bureau has org-level settings that affect who can book and create rooms, how auto-follow behaves, and how spatial audio and new offices default. There is a dedicated **Bureau settings** screen for them in the SPA.

To review or change org settings:

1. In the sidebar Admin group, open **"Settings"** (or go to `/bureau/admin/settings`). The heading is **"Bureau settings"** with the subtitle "Org-wide defaults for spatial audio, follow, and what members can do."
2. The toggles and select are:
3. Any member can read the settings; only org admins, owners, or SuperUsers can change them. For a non-admin the controls render read-only with the banner "These settings are read-only for your role. Only org admins or owners can change them." Each change saves immediately and shows a brief **"Saved"** badge next to the row.

AI agent note: `bureau_get_settings` reads the org's settings and `bureau_update_settings` changes them (admin/owner gated server-side), matching the toggles on this screen.

Working with AI agents

Agents are first-class spatial occupants in Bureau. An agent that enters a room shows up on the floor and in the occupants list with an "(A) " prefix. Agents drive Bureau through over 30 `bureau_*` MCP tools, all forwarding the calling user's token and gated by the same server-side permission checks as the UI.

What agents commonly do:

- **Perceive the office** with `bureau_list_floors`, `bureau_get_floor`, `bureau_who_is_in_room`, and `bureau_who_is_here` (co-presence by URL).
- **Move into a room** with `bureau_move_self` (the canonical "appear in the room" action).
- **Find a person** with `bureau_where_is_user` (the real Hunt; returns the user's current surface or `{ located: false }`).
- **Socialize** with `bureau_knock`, `bureau_respond_knock`, `bureau_knock_inbox`, and `bureau_leave_note`. On a DND block, surface the leave-a-note hint rather than retrying.
- **Teleport a room** with `bureau_summon` (high-impact, two-step confirm; caller must be in a room), then inspect or recover with `bureau_get_summon` and `bureau_summon_grant_access`.
- **Book and manage rooms** with `bureau_book_room` (two-step confirm), `bureau_list_bookings`, `bureau_update_booking`, and `bureau_cancel_booking` (two-step confirm).
- **Set a door** with `bureau_set_door_state`.
- **Administer floors and rooms** with `bureau_create_floor`, `bureau_update_floor`, `bureau_set_floor_background`, `bureau_delete_floor` (two-step confirm), `bureau_create_room`, `bureau_update_room`, and `bureau_delete_room` (two-step confirm), all admin/owner gated server-side.
- **Manage offices** with `bureau_list_offices`, `bureau_assign_office`, and `bureau_my_office`.
- **Recover chats** with `bureau_list_chats`, `bureau_get_chat_messages`, and `bureau_set_chat_retention`.
- **Read or change org settings** with `bureau_get_settings` and `bureau_update_settings`.

Six high-impact tools use the `confirm_action` two-step (call once to preview, then again with `confirm_action: true` to commit): `bureau_summon`, `bureau_summon_grant_access`, `bureau_book_room`, `bureau_cancel_booking`, `bureau_delete_floor`, and `bureau_delete_room`.

The three presence tools that were previously stubs are now backed by the real Redis presence store (`GET /v1/presence`, `GET /v1/presence/locate`, `PATCH /v1/me/status`):

- **`bureau_get_presence`** snapshots the org-wide presence map: who is on Bureau right now, their status, and the room and floor they are in.
- **`bureau_locate_user`** returns one user's current room, floor, status, and location, or a null "not located" envelope when they have no live session. For the access-checked Hunt (which respects DND and a destination-access preflight), prefer **`bureau_where_is_user`**.
- **`bureau_set_status`** sets the caller's status and persists it durably so it survives a reconnect and applies with no live web session. The tool exposes the four-value subset

available, **busy**, **away**, **dnd**; the endpoint also accepts **focus** and **in_meeting** for status set through the docked box.

Bureau participates in the suite-wide agentic platform alongside its own tools. Service-account agents are bound by the §15 `agent_policies` kill switch and tool allowlists (the `bureau.*` prefix), and every action is written to the unified activity view with the actor's kind (human, agent, or service). Cross-app result posting must pass the platform `can_access` visibility preflight, which is exactly the check Bureau already runs internally before a summon, hunt, or co-presence read reveals a destination. High-impact spatial actions can be routed through the platform approval queue (`proposal_create` / `proposal_decide`), and agents can subscribe to Bureau's Bolt events through outbound webhooks. Agents should also call `agent_heartbeat` so their session is treated as live.

A human reviewing agent work in Bureau should know that a summon DMs every eligible co-occupant in real time, that a hunt reveals where a teammate is, and that a booking can create or anchor a Book calendar event and can flip a room private for its window. The confirm step exists so these are previewed before they fire. For the full catalog and signatures, see the MCP-tools reference in `docs/apps/bureau/`.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. Bureau is the office itself: live floors and rooms where the team gathers, and the presence layer the rest of the suite plugs into, so your location and your live room follow you everywhere you work. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: Drop into the office and join a room

Who: Any logged-in team member, first time in Bureau. **Goal:** Get into a room and be present with whoever is around. **Before you start:** You are logged in to BigBlueBam. At least one floor exists.

Steps

1. Open `/bureau/`. The **"Floors"** landing lists the available floors with live occupancy counts.
2. Click a floor card to open its live view at `/floors/:id`.
3. On the canvas, click an Open room rectangle to enter it. Your dot appears inside, and your audio connects.
4. Read the docked box: the **"In:"** row names your room and the occupants list shows who is with you (or **"Just you"**).
5. When you are done, click **"Leave room"** in the floor view status bar.

Result: You are present in a room, joined to its audio huddle, and visible to teammates on the floor.

Related: See who is in a room; The cross-app docked box. An agent does the same move with `bureau_move_self`.

Story: Knock on a busy colleague's office

Who: A team member who needs a quick word. **Goal:** Get into a colleague's closed office without barging in. **Before you start:** The colleague has a personal office with an owner; its door is Knock or otherwise closed.

Steps

1. Open the floor and click the colleague's office.
2. Bureau sends a knock to the owner; their docked box shows an incoming-knock toast.
3. Wait. If they click **"Let in"** you are admitted and join their room. If they click **"Not now"** the knock is deferred; if they click **"Decline"** it is rejected.
4. If no one answers within 30 seconds, the knock times out.

Result: You are either admitted to the office or told to come back later, without interrupting work uninvited.

Related: Triage knocks at your own office. If the owner is in DND, you are offered a leave-a-note path that lands in their Banter DMs. Agents use `bureau_knock` and, on a DND block, `bureau_leave_note`.

Story: Triage knocks at your own office

Who: An office owner who wants to manage interruptions. **Goal:** Decide who gets in and when, and block interruptions during focus time. **Before you start:** You own an office (a room of type Office assigned to you).

Steps

1. When someone knocks, an incoming-knock toast appears in your docked box.
2. Click **"Let in"** to admit, **"Not now"** to defer, or **"Decline"** to reject.
3. To stop interruptions entirely, click the **"DND"** toggle in the docked box header to go head-down. Knocks are then blocked and visitors are offered the leave-a-note path.
4. Click **"DND"** again when you are ready to be reachable.

Result: You control who enters your office, and head-down mode shields you while still letting people leave a note.

Related: Set your status and DND. Agents resolve knocks with `bureau_respond_knock` and review waiting visitors with `bureau_knock_inbox`.

Story: Bring everyone here (teleport the room)

Who: A facilitator standing in a room with teammates. **Goal:** Pull everyone to a specific resource in another app so you are all looking at the same thing. **Before you start:** You are in a Bureau room with at least one other person, and you can open the destination resource.

Steps

1. Stay in your room and navigate to the resource (for example a Board canvas or a Brief doc). The docked box **"Viewing:"** row reflects your location.
2. Click **"Bring everyone here"** in the docked box.
3. Bureau access-checks each co-occupant and sends a **"Summon"** toast to everyone who can reach the destination, with **Join** and **Stay here** buttons.
4. If a teammate appears on the denied list because they lack access, grant them access to the destination and bring everyone here again.

Result: Eligible teammates land on the same URL and join the same huddle; people without access are not sent a dead link.

Related: Ring (Invite) for one person. Agents use `bureau_summon` with the `confirm_action` two-step, then `bureau_summon_grant_access` to grant and re-summon the denied users.

Story: Invite one person to your screen

Who: Anyone who needs one specific colleague, now. **Goal:** Ring a single person and bring them to your surface. **Before you start:** You are on a located page (a content surface that supports ring).

Steps

1. Click **"Invite..."** in the docked box.

2. In the Invite popover, pick the member to ring. They get an incoming-call overlay.
3. When they accept, they are navigated to your surface and auto-join the huddle.
4. If you are an admin and the call cannot wait, right-click **"Invite..."** to Force Invite, which auto-navigates them after a 3-second cancellable countdown.

Result: The colleague is ringing-and-joined to exactly where you are.

Related: Hunt to go to them instead of pulling them to you. Ring respects DND unless an admin forces it.

Story: Hunt a teammate

Who: Someone trying to find where a colleague is working. **Goal:** Jump to wherever a specific teammate currently is. **Before you start:** The teammate is online and not head-down, shares your active org, and you can access where they are.

Steps

1. Click **"Hunt..."** in the docked box.
2. In the Hunt popover, pick the member.
3. You are navigated to their current location.

Result: You arrive where the teammate is, ready to join them.

Related: A head-down (DND) teammate cannot be located, and a surface you cannot access reads as "not located". Invite pulls them to you instead. Agents use `bureau_where_is_user`.

Story: Book the conference room

Who: An organizer planning a meeting. **Goal:** Reserve a bookable room for a window and have it appear on the calendar. **Before you start:** The target room is marked bookable, and your org allows you to book (`members_can_book`).

Steps

1. Open `/bureau/booking` (sidebar Rooms to **"Book a room"**) and pick the room from the left column.
2. In the **"Book this room"** form, enter a title, set the start and end times, and choose access: **Locked** to hold the room private for the meeting, or **Open** to let others wander in.
3. Click **"Book room"**. Bureau writes the booking, anchors it to a Book event, and schedules the privacy flip at the start and the clear at the end. The reservation appears under **"Upcoming bookings"**.
4. To cancel, click **"Cancel"** on the booking's row as the organizer or an admin; the lifecycle jobs drop and the linked Book event is cancelled best-effort.

Result: The room is reserved for your window and mirrored to Book; a locked booking holds it private when it starts.

Related: The Room booking screen at `/bureau/booking` is where people do this in the app. Agents use `bureau_book_room`, `bureau_list_bookings`, `bureau_update_booking`, and `bureau_cancel_booking`; the book and cancel tools use the `confirm_action` two-step.

Story: Recover what was said in a room

Who: Anyone who needs to revisit a room conversation. **Goal:** Read back ephemeral room chat before it expires, or pin it. **Before you start:** You were present in the room when the messages were sent. Room chats are kept 24 hours by default.

Steps

1. In the sidebar, click **"Recent chats"**.
2. Use the search box to find the room by name or message text.
3. Click a thread to open its transcript dialog.
4. If you are an admin, set the **"Retention"** dropdown to "2 days", "3 days", "1 week (max)", or "Retain permanently" to extend or pin the thread.

Result: You read the past conversation, and an admin can keep it from expiring.

Related: Room chat is opened live from the docked box **"CHAT"** toggle. Agents use `bureau_list_chats`, `bureau_get_chat_messages`, and `bureau_set_chat_retention`.

Story: Build a floor (admin)

Who: An org admin or owner setting up the office. **Goal:** Create a floor and place its rooms and offices. **Before you start:** You are an org admin, owner, or SuperUser.

Steps

1. In the sidebar Admin group, click **"Edit floors"**, then **"New floor"**.
2. Enter a **Name** and confirm the auto-generated **Slug**, then click **"Create + open editor"**.
3. In the Floor editor, pick **"New room"** or **"New office"** and click-drag on the canvas to place spaces.
4. Select each zone and set its **Name**, **Type**, **Capacity (people)**, **Door default**, **Bookable**, and (for offices) **Owner** in the inspector.
5. Optionally upload an image underlay for the floor background.
6. Click **"Save"**.

Result: A new floor exists with its rooms and offices, ready for people to drop in. Removed zones evict any live occupants to the lobby.

Related: Assign an office owner; Door states. Agents use `bureau_create_floor`, `bureau_create_room`, and `bureau_update_room`.

Story: Assign an office owner (admin)

Who: An org admin or owner. **Goal:** Give a personal office an owner so it can be knocked on and managed. **Before you start:** You are an org admin, owner, or SuperUser, and at least one office room exists.

Steps

1. In the sidebar Admin group, click **"Offices"**.
2. Find the office in the **"Floor / Office"** column.
3. Click **"Reassign owner"** in the **"Actions"** column.
4. Search by name or email in the dialog and pick the new owner.

Result: The office shows the new owner in the **"Owner"** column, and that owner now receives knocks and controls the door.

Related: Build a floor; Knock and respond to a knock. Agents use `bureau_list_offices` and `bureau_assign_office`.

Story: Agent staffs a room and pulls humans in

Who: An AI agent acting for a user. **Goal:** Appear in a room, see who is there, and summon the right people to a resource. **Before you start:** The agent has the user's token and the target room is reachable.

Steps

1. Call `bureau_get_floor` to enumerate rooms on the floor.
2. Call `bureau_move_self` on the target room so the agent appears as an occupant.
3. Call `bureau_who_is_in_room` to confirm who is present.
4. Call `bureau_summon` with `confirm_action: false` to preview the eligible recipients, then again with `confirm_action: true` to send everyone to the destination URL.
5. Optionally call `bureau_get_summon`, then `bureau_summon_grant_access` to grant access to anyone on the denied list and re-summon them.

Result: The agent is present in the room and the eligible co-occupants are pulled to the shared resource. Recipients who lack access are reported in the denied count.

Related: Bring everyone here; See who is in a room. The agent must be in a room before `bureau_summon` will succeed, and cross-app result posting should pass the platform `can_access` preflight.

Related

- **Board** - Summons and rings commonly land people on a Board canvas, and Board surfaces the same docked box so the canvas huddle follows you in.
- **Brief** - A Brief doc is a frequent summon and ring destination; each doc has a surface huddle the docked box auto-joins.
- **Bond** - Bond deals are valid summon and ring destinations for pulling the room onto a record.
- **Banter** - Surfaces the same docked box, and is the fallback delivery channel for the leave-a-note path when someone is in DND ("[Bureau knock note] " direct messages).
- **Book** - Bureau bookings mirror to Book events; cancelling a booking cancels the linked Book event best-effort.
- **Bolt** - Bureau emits events (entering and leaving rooms, status changes, knocks, room booked, room locked, summon issued) on the `bureau` source for automation.
- **Bench** - Daily floor utilization rolls up for reporting in Bench.
- MCP tools: this app exposes over 30 `bureau_*` tools. `bureau_summon`, `bureau_summon_grant_access`, `bureau_book_room`, `bureau_cancel_booking`, `bureau_delete_floor`, and `bureau_delete_room` use the `confirm_action` two-step. The presence tools `bureau_get_presence`, `bureau_locate_user`, and `bureau_set_status` are now backed by the real Redis presence store; prefer `bureau_where_is_user` for the access-checked, DND-respecting Hunt. See the MCP-tools reference in `docs/apps/bureau/`.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. How many room types does Bureau offer?

a) 4
b) 6
c) 8
d) 12

2. What are the three door states a room can have?

a) Open, Knock, and Private
b) Public, Locked, and Hidden
c) Available, Busy, and DND
d) Member, Manager, and Owner

3. How long does a knock wait before it auto-times-out if the owner does not respond?

4. How long are room chat messages kept by default?

a) 1 hour
b) 24 hours
c) 7 days
d) Forever

5. Name the six presence status values Bureau supports.

6. What does a summon do when a co-occupant lacks access to the destination?

a) Sends them the link anyway
b) Puts them on a denied list rather than pinging them with a dead link
c) Grants them access automatically
d) Cancels the entire summon

7. When someone knocks and you respond from the incoming-knock toast, what are the three responses you can give?

8. If you are in DND when someone tries to knock, what happens?

a) They are admitted automatically
b) The knock is blocked (the server returns 423) and they are offered a leave-a-note path
c) The knock queues until you exit DND
d) Nothing; DND does not affect knocks

9. What two access modes can a booking have, and what does each mean?

10. Which setting gates whether ordinary members can reserve rooms?

a) continuous_audio
b) members_can_book
c) allow_auto_follow
d) members_can_create_rooms

11. How does Hunt treat a teammate who is in DND or on a surface you cannot access?

a) It reveals their location anyway
b) It returns the same result as "not located"
c) It overrides their DND
d) It notifies them you tried to find them

12. Why does navigating between pages not drop your room or your call?

13. How are agent occupants shown in the occupants list and on the floor?

a) With a robot emoji
b) With an "(A) " prefix
c) In a separate panel
d) They are hidden from humans

CHAPTER 17

Helpdesk

File it, track it, close it.

Helpdesk is the public face of your support. Customers register on your org's branded portal, file tickets, follow the conversation as agents reply, and close issues out, all without ever needing a Bam account. Every ticket quietly spawns a linked Bam task, so your staff work support from the same project board they use for everything else, and the trail survives even if the ticket is later deleted. By the end of this chapter you will be able to stand up a portal, file and track a ticket, and see where an agent can triage the queue through MCP tools.

At a glance

- Branded, multi-tenant portals at /helpdesk/<org-slug>/, one per org with a settings row
- Customer sign-in that is fully separate from the Bam suite single sign-on
- Five-status ticket lifecycle with a conversation timeline, attachments, and reopen
- Every ticket mirrored to a linked Bam task so support work lives on the board
- 13 MCP tools so an agent can triage, answer, search, and configure tickets

In this chapter

Overview . Key concepts . Where to find it . Feature reference . Choose your support portal (org picker) . Register (create an account) . Login . Verify email . My Tickets list . New Ticket

> *Helpdesk is the public-facing support portal where your customers register, file tickets, track replies, and close out issues. Behind the scenes every ticket spawns a Bam task so your support staff can work it from the project board.*

> ***Every support ticket mirrors to a linked Bam task, so your staff work it from the project board instead of a separate inbox.***

Overview

Helpdesk gives each organization its own branded support portal[174]. Customers reach it at a public URL, create an account scoped to that org, and submit support requests as tickets. They follow each ticket[175] as a conversation: agents reply, the status[176] moves through its lifecycle, and the customer[177] can attach files, change priority[178], mark duplicates, reopen, or close.

Customers and agents are two different identities. Customers are end users who live only in Helpdesk; they sign in with an org-scoped email and password that is completely separate from the BigBlueBam (Bam) suite single sign-on. Support agents are your Bam staff users. Agents do not use the customer portal SPA. They work tickets through the agent[179] REST surface and through MCP tools, and they see the matching Bam task[180] inside the Bam app. Every ticket is mirrored to a Bam task on creation, so customer replies, status changes, and closures leave a durable trail on the project board even if the ticket is later deleted.

Helpdesk is multi-tenant. Any organization that has a Helpdesk settings row gets a portal at `/helpdesk/<org-slug>/`, and optionally per-project portals at `/helpdesk/<org-slug>/<project-slug>/`. The portal that a customer lands on determines which org their account and tickets belong to.

The core objects you work with are the **ticket** (the support request), its **messages** (the conversation), its **status** and **priority**, optional **attachments**, and the **settings** that an admin[181] uses to configure each portal.

Key concepts

- **Portal** - an org's public support site, served at `/helpdesk/<org-slug>/`. Each org with a Helpdesk settings row gets one. Per-project portals are available at `/helpdesk/<org-slug>/<project-slug>/`. The portal a customer lands on decides which org owns their account and tickets.

[174] **Portal.** an org's public support site, served at `/helpdesk/<org-slug>/`. Each org with a Helpdesk settings row gets one. Per-project portals are available at `/helpdesk/<org-slug>/<project-slug>/`. The portal a customer lands on decides which org owns their account and tickets.

[175] **Ticket.** a single customer support request. It has a human-readable number shown as `#123`, a subject, a description, a status, a priority, and an optional category. Each ticket is owned by one customer and back-linked to a Bam task.

[176] **Status.** where the ticket is in its lifecycle. The five canonical statuses are **Open**, **In Progress**, **Waiting on Customer**, **Resolved**, and **Closed**. The stored values are `open`, `in_progress`, `waiting_on_customer`, `resolved`, and `closed`.

[177] **Customer.** an end user with a Helpdesk account scoped to one org. Customers have their own email and password sign-in, separate from the suite single sign-on. They use the portal SPA.

[178] **Priority.** how urgent the ticket is. Customers can set **Low**, **Medium**, or **High**. Agents can additionally set **Critical**. The badge displays all four.

[179] **Agent.** a support staff member. Agents authenticate with a per-agent agent API key (prefixed `hdag_`) on the agent routes, not with the customer portal. They do not use the customer portal SPA.

[180] **Bam task.** the project-board task created automatically for every ticket. It lives in the portal's default project so your staff can triage, assign, and track support work alongside other project work.

[181] **Admin.** an org owner or admin who configures the portal settings.

- **Customer** - an end user with a Helpdesk account scoped to one org. Customers have their own email and password sign-in, separate from the suite single sign-on. They use the portal SPA.
- **Ticket** - a single customer support request. It has a human-readable number shown as `#123`, a subject, a description, a status, a priority, and an optional category[182]. Each ticket is owned by one customer and back-linked to a Bam task.
- **Status** - where the ticket is in its lifecycle. The five canonical statuses are **Open**, **In Progress**, **Waiting on Customer**, **Resolved**, and **Closed**. The stored values are `open`, `in_progress`, `waiting_on_customer`, `resolved`, and `closed`.
- **Priority** - how urgent the ticket is. Customers can set **Low**, **Medium**, or **High**. Agents can additionally set **Critical**. The badge displays all four.
- **Message**[183] - a post on a ticket. Authored by the customer, an agent, or the system. Agent messages flagged as internal notes are hidden from customers.
- **Category** - an optional label chosen from the list an admin configured for the portal. The Category field only appears when categories are configured.
- **Agent** - a support staff member. Agents authenticate with a per-agent agent API key (prefixed `hdag_`) on the agent routes, not with the customer portal. They do not use the customer portal SPA.
- **Admin** - an org owner or admin who configures the portal settings.
- **Duplicate**[184] - a ticket flagged as a copy of another. A customer "mark as duplicate" is annotative: it posts updates on the primary instead. An agent "merge" actually moves the messages onto the primary and closes the source.
- **Attachment**[185] - a file uploaded to a ticket. It appears in the Attachments block once it has been scanned, with a Download link.
- **Bam task** - the project-board task created automatically for every ticket. It lives in the portal's default project so your staff can triage, assign, and track support work alongside other project work.

> **WARNING**
> An agent merge is not customer-reversible. It physically moves the source ticket's messages onto the primary and closes the source, so confirm the true primary before merging. A customer's plain mark-as-duplicate is only annotative and can be unmarked.

[182] **Category.** an optional label chosen from the list an admin configured for the portal. The Category field only appears when categories are configured.
[183] **Message.** a post on a ticket. Authored by the customer, an agent, or the system. Agent messages flagged as internal notes are hidden from customers.
[184] **Duplicate.** a ticket flagged as a copy of another. A customer "mark as duplicate" is annotative: it posts updates on the primary instead. An agent "merge" actually moves the messages onto the primary and closes the source.
[185] **Attachment.** a file uploaded to a ticket. It appears in the Attachments block once it has been scanned, with a Download link.

Where to find it

The portal is served at `/helpdesk/`. Visiting the bare site root redirects to the Helpdesk portal, so customers normally arrive there by default.

- `/helpdesk/` with no org shows the **Choose your support portal** picker.
- `/helpdesk/<org-slug>/` is a specific org's portal.
- `/helpdesk/<org-slug>/<project-slug>/` is a per-project portal, when one is configured.

A customer needs an account on the org's portal before they can file tickets. This account uses Helpdesk's own customer sign-in, which is separate from the suite single sign-on that staff use for Bam and the other apps; a customer never needs a Bam account. Registration is built in unless the admin has disabled signups. There is no separate install step: any org with a Helpdesk settings row is live.

There is no Knowledge Base in Helpdesk. If you are looking for a searchable knowledge base, that is the separate Beacon app.

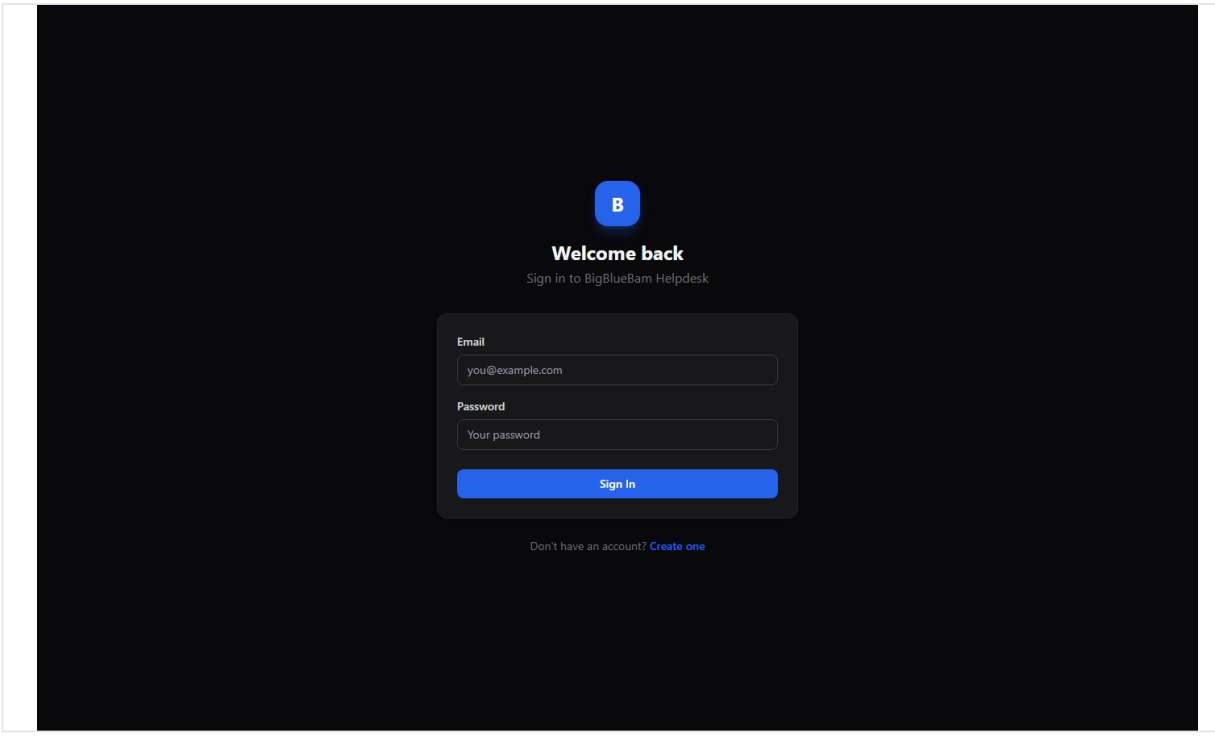


Figure 17.1 Support portal

Feature reference

Choose your support portal (org picker)

This is the landing page when you reach `/helpdesk/` without a specific org. It lists every organization that has a configured portal so you can pick which one to contact.

To choose a portal:

1. Open `/helpdesk/`.
2. Under the heading **Choose your support portal**, read the subtext: "Each organization has its own helpdesk. Pick the one you want to contact."
3. Click the row for the org you want. Each row shows the org name and its `/helpdesk/<slug>/` path.
4. You land on that org's portal.

If no portals are configured, you see the empty state: "No helpdesk portals are configured yet...".

Register (create an account)

Customers self-register on the org's portal. Each account is scoped to that one org and uses Helpdesk's own customer sign-in, not the suite single sign-on.

To create an account:

1. From the portal, go to the **Create your account** page (the subtext reads "Get support through BigBlueBam Helpdesk").
2. Fill in **Email**, **Display Name**, and **Password**. The password field shows the hint "Min. 12 characters" and must be at least 12 characters.
3. Click **Create Account**.
4. If email verification is not required, you are signed in and taken to your tickets. If it is required, you see a "Check your email" notice.

> **TRY IT**
> Open /helpdesk/, pick an org's portal, click Create your account, and register with a password of at least 12 characters. Then file a New Ticket and watch the detail page show you a number like #123.

If signups are disabled for the org, registration returns an error. If the org restricts allowed email domains, an address outside those domains is rejected. An email already in use on that org is rejected as well.

Login

To sign in:

1. Open the **Welcome back** page (subtext "Sign in to BigBlueBam Helpdesk").
2. Enter your **Email** and **Password**.

3. Click **Sign In**.
4. You land on **My Tickets**.

The footer link "Dont have an account? Create one" sends you to registration. If signups are disabled for the org, that link routes to the Bam beta gate instead. Repeated failed logins lock the account temporarily.

Verify email

When an org requires email verification, registration sends a link with a token. The verification page reads the token from the URL.

To verify:

1. Open the verification link from your email. The page is titled **Email Verification**.
2. Watch for the status: "Verifying your email...", then either "Your email has been verified!" or "Verification Failed".
3. Click **Go to Login** and sign in.

Verification tokens expire after 24 hours. Note that in a stack without outbound email configured, the verification email is not actually delivered; the token is created but the send is not yet wired up.

My Tickets list

Your tickets list is the home screen after you sign in. It shows every ticket you own, newest activity first.

To use the list:

1. Open **My Tickets** (the logo and the "My Tickets" nav button both bring you here).
2. Use the search box (placeholder "Search subject, number, or category...") to filter by subject, ticket number, or category.
3. Use the filter chips to narrow by status: **all**, **open**, **in progress**, **waiting on customer**, **resolved**, and **closed**. Each chip maps to a real ticket status, so picking one shows exactly the tickets in that state.
4. Read the columns: **#**, **Subject**, **Status**, **Priority**, **Category**, **Updated**. A **New** dot marks a ticket with activity you have not seen.
5. Click a row to open the ticket.

The list refreshes in real time when a new message is posted or a status changes. If you have no tickets, the empty state reads "No tickets yet" with "Create your first one!". If a search matches nothing, it reads "No tickets match your search."

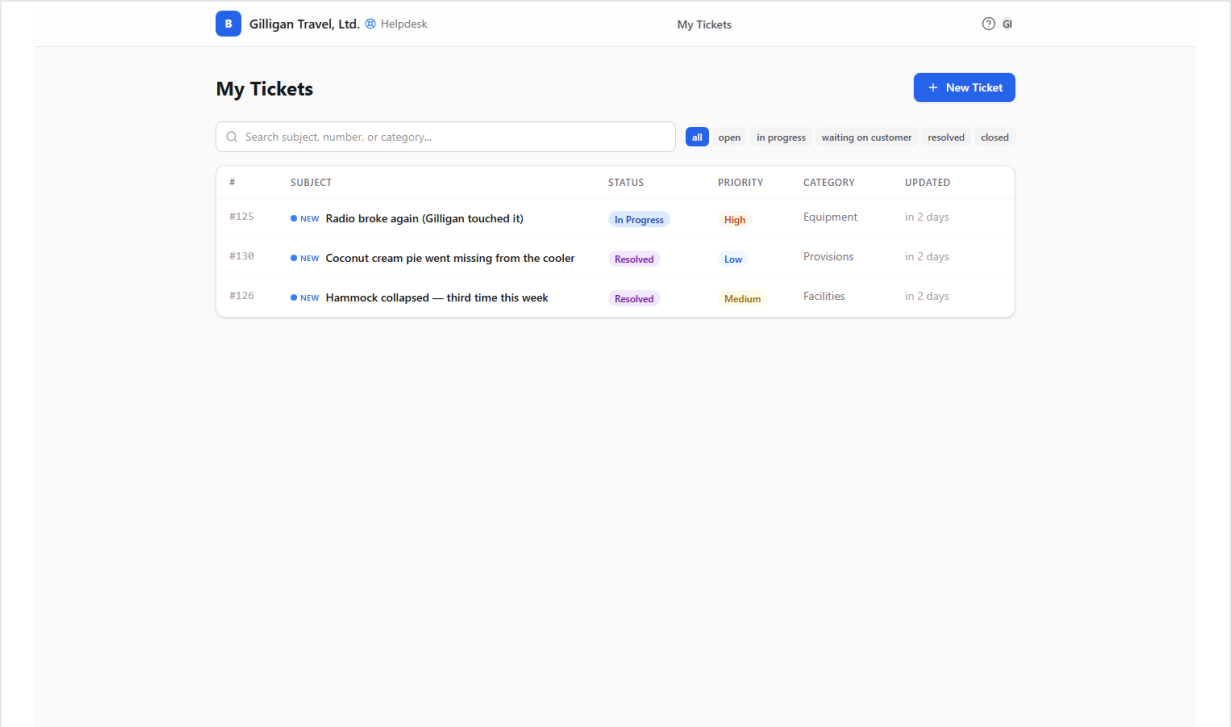


Figure 17.2 *My tickets*

New Ticket

This is where a customer files a support request.

To create a ticket:

1. From **My Tickets**, click **New Ticket** (the button with the plus icon). The page is headed **New Ticket** and has a "Back to tickets" link.
2. Fill in **Subject** ("Brief summary of your issue"). This is required.
3. If the portal has categories, choose one from the **Category** select ("Select a category..."). The field is hidden when no categories are configured.
4. Set **Priority** to Low, Medium, or High. Medium is the default. Customers cannot set Critical.
5. Fill in **Description** in the rich-text editor ("Describe your issue in detail..."). This is required. You can paste or upload inline images.
6. Click **Submit Ticket** (or **Cancel** to back out).
7. You land on the new ticket's detail page.

When you submit, the portal creates the ticket and spawns a Bam task in the portal's project so your support staff can work it. If you submit the same subject and description again within an hour, the portal returns your existing ticket instead of creating a duplicate.

> **TIP**
> Submitting the same subject and description again within an hour does not double-file. The portal hands back your existing ticket, so an accidental second click never clutters the

queue.

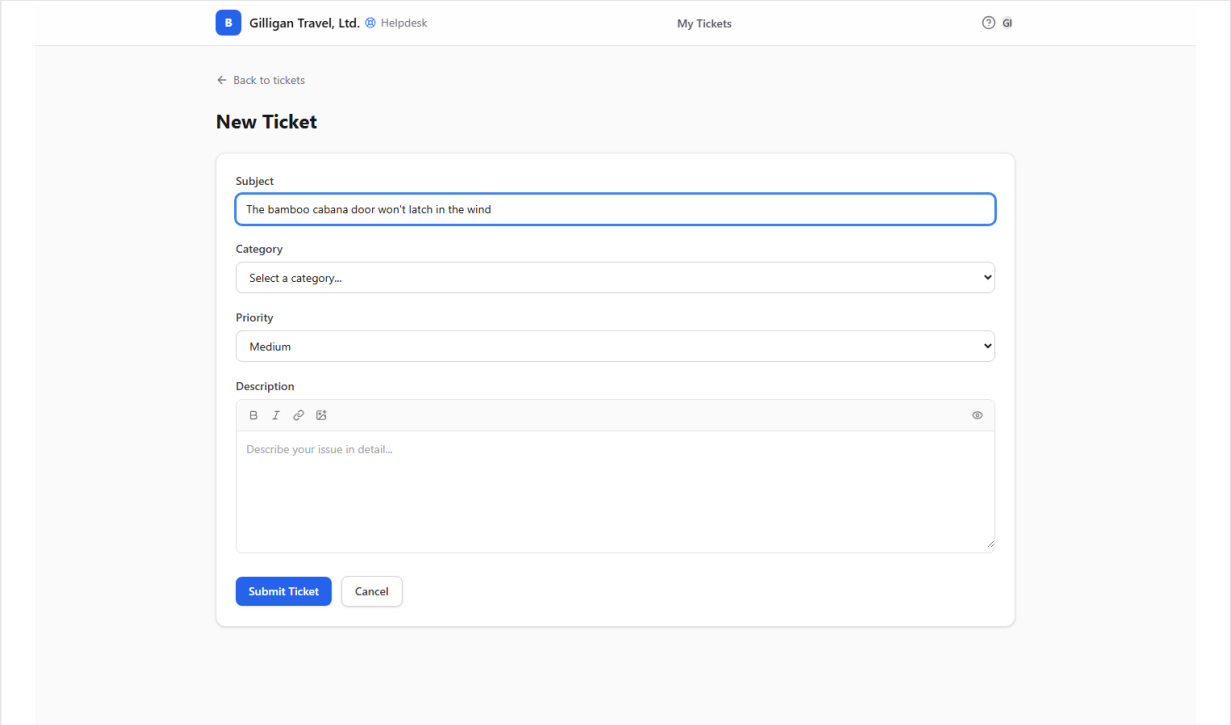


Figure 17.3 New ticket

Ticket detail (the conversation)

The ticket detail page is the full conversation and the place to take action on a ticket.

The header shows "#{number} - {subject}", a presence strip of who else is viewing, the **Status** badge, the **Priority** badge with a dropdown, the category chip when set, and "Created on {date}".

The body shows the **Description**, an **Attachments** block, and the timeline of messages and status changes. Your own messages appear on the right labeled "You"; agent and system messages appear on the left. Status changes appear as italic lines, for example "Status changed from X to Y". If the conversation is long, click "Load older messages". When there are no messages yet, the timeline reads "No messages yet."

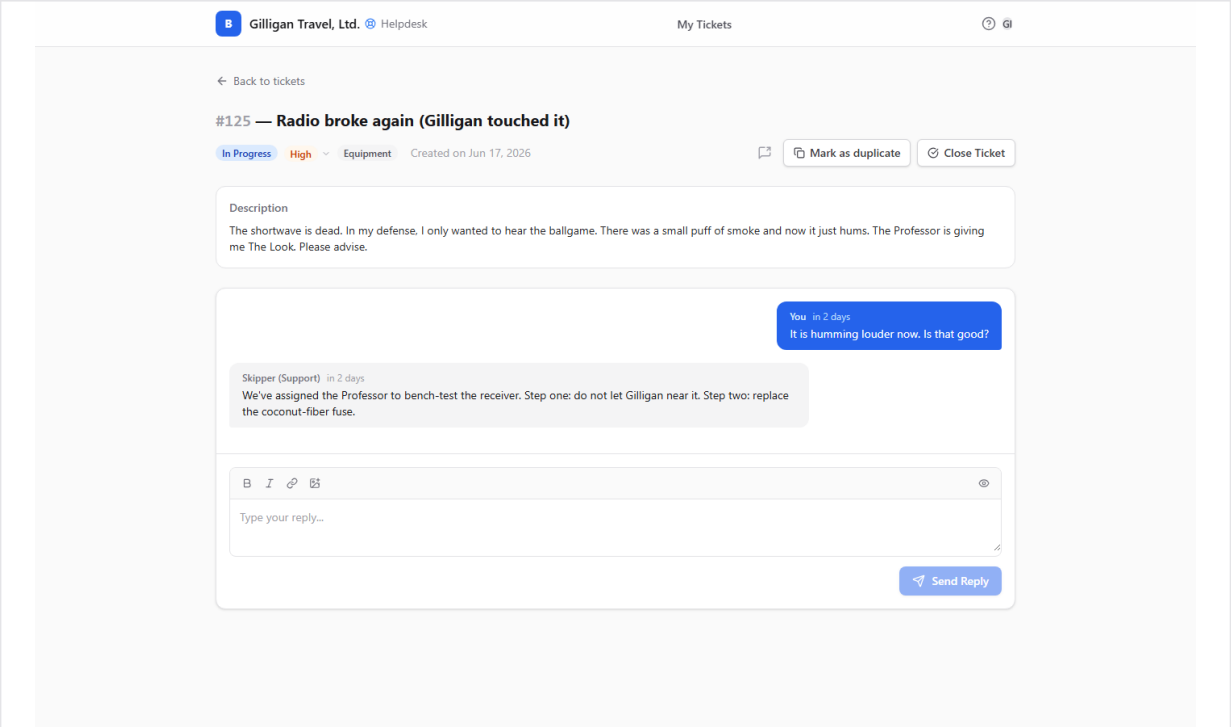


Figure 17.4 Ticket detail

Reply to a ticket

1. Scroll to the reply box at the bottom (it is hidden once a ticket is Resolved or Closed).
2. Type into the rich-text field (placeholder "Type your reply..."). You can add inline images.
3. Click **Send Reply**.

If the ticket was **Waiting on Customer**, sending a reply automatically flips it back to **Open**.

Change priority

1. In the header, click the **Priority** badge to open its dropdown.
2. Pick **Low**, **Medium**, or **High**.

Close a ticket

1. Click **Close Ticket** in the header.
2. Confirm in the "Close this ticket?" dialog.

Closing the ticket also moves its linked Bam task to a terminal state.

Reopen a ticket

1. On a Resolved or Closed ticket, find the banner "This ticket has been resolved." or "This ticket has been closed."
2. Click **Reopen**.

Mark as duplicate

Use this when your ticket is a copy of one you already filed.

1. Click **Mark as duplicate** (the copy icon). It is available only when the ticket is not already closed, resolved, or a duplicate.
2. In the **Mark as duplicate** dialog, type the primary ticket's number in the input ("e.g. #123").
3. Click **Confirm** (or **Cancel**).
4. A banner appears: "This ticket was marked as a duplicate of #N… Updates will be posted on that ticket." Updates now go on the primary. The primary ticket shows a "Duplicates of this ticket" callout.

To undo, click **Unmark** in the banner. You cannot unmark if an agent has already merged the ticket.

Share to Banter

Use this to hand a ticket off to a Banter channel for internal discussion.

1. Click the **Share to Banter** button (the share icon) in the header.
2. In the popover, headed **Share to Banter**, pick a channel from the **Channel** select ("Select a channel…").
3. Optionally type into the **Message (optional)** field ("Add a note…").
4. Click **Share** (or **Cancel**).

Attachments

1. In the **Attachments** block (the paperclip with a count), each file shows its name, size, type, and scan status.
2. Click **Download** to retrieve a file once it has been scanned. Until then it shows **Unavailable**.
3. To add a file, upload it from the ticket. Files are owner-scoped: only the ticket owner can delete an attachment.

Notifications prompt

On the tickets list, if your browser has not yet been asked, a banner offers browser push notifications: "Get notified when agents respond to your tickets."

To enable:

1. Click **Enable notifications** on the banner.
2. Allow notifications in your browser prompt.

Dismiss the banner with the X if you do not want them. Note that whether a notification actually arrives also depends on the org's notify settings and on outbound email being configured.

In-app Help

The Help viewer is built into the portal. Press the ? key from anywhere in the portal to open it; closing it returns you to your tickets. It has no separate page of its own.

The agent queue (REST and MCP only)

Support agents work tickets through the agent REST surface under `/helpdesk/api/agents/`, not through the customer portal. There is no agent SPA in Helpdesk. Agents authenticate with a per-agent agent API key (prefixed `hdag_`) sent in the `X-Agent-Key` header. A plain Bam session cookie alone is not accepted on the agent routes.

What agents can do on the agent surface:

- List org tickets, filtered by status, assignee, and SLA state, via the agent queue.
- Resolve a ticket by its `#number`, enriched with the requester and the task assignee.
- Search tickets by subject and description.
- Open full ticket detail, including internal notes that customers never see.
- Post a public reply or an internal note. The first agent reply stamps the first-response time.
- Update status, priority, or category.
- Close a ticket.
- Merge duplicates: move the messages onto a primary ticket and close the source.
- Pull a ranked list of similar tickets to find duplicates or related issues.

The agent queue shows an SLA badge for each ticket (breached, imminent, or ok). The badge's first-response target is currently a fixed 4 hours with a 0.75 imminent threshold. This is independent of the per-org SLA setting that the background breach monitor actually enforces (default 8 hours for first response, 48 hours for resolution), so the queue badge and the recorded breach events can disagree. See Related.

> **NOTE**
> The agent queue's SLA badge is hardwired to a 4-hour first-response target, but the background breach monitor enforces the per-org SLA setting (default 8 hours for first response, 48 for resolution). The badge and the recorded breaches can therefore disagree.

Most agents drive these actions through the MCP tools in Working with AI agents below, or through Bam's own ticket views.

Settings and admin

An admin configures each org's portal. Admins authenticate with a Bam org owner or admin session, or with an `hdag_` agent API key. Settings are org-scoped; the row is created on first save if it does not exist.

Configurable settings:

- **Default project** - the project where incoming tickets create their Bam tasks. Set this from the org's non-archived projects.
- **Default phase** - the phase for newly created tasks.
- **Default priority** - the priority applied by default (low, medium, high, or critical).
- **Categories** - the list of categories customers can choose from on New Ticket.

- **Welcome message** - text shown on the portal.
- **Allowed email domains** - restricts which email domains may register.
- **Require email verification** - whether new customers must verify their email before signing in.
- **Auto-close days** - configured number of days before idle tickets close.
- **Notify on status change** and **Notify on agent reply** - whether to send the customer an email on those events.

To set the default project for incoming tickets:

1. As an admin, open the Helpdesk settings for the org.
2. Choose a project from the org's projects (the picker lists id, slug, and name for each non-archived project).
3. Save. New tickets now create their Bam task in that project. If no default is set, the portal falls back to the project named in the tenant context, then the oldest project.

To disable customer signups:

1. As an admin, set the signup-disabled flag for the org.
2. After that, the registration route returns a "signup disabled" error, and the portal's "Create one" link routes to the Bam beta gate instead of registration.

Note that the SLA minute settings live on the settings row but are not part of the settings update schema; they are editable only directly in the database or through a future UI. Auto-close days and the notify toggles are stored as configuration; confirm with your operator whether the backing automation is enabled in your deployment before relying on them.

Working with AI agents

Helpdesk exposes 13 MCP tools so agents and automations can triage, answer, search, and configure tickets. They proxy to the Helpdesk API and forward the caller's token, so an agent acts with its own authority. Be aware that the agent surface routes (`/helpdesk/api/agents/`) require an `hdag_` agent API key, so a tool calling those routes acts as a configured support agent, not as a customer.

> ***Thirteen MCP tools mean an agent can triage, answer, search, and configure tickets with its own authority.***

Reading and finding tickets:

- **list_tickets** - list tickets with optional status, assignee, and client filters.
- **get_ticket** - open a ticket with its messages.
- **helpdesk_get_ticket_by_number** - resolve a `#number` to the full record, enriched with requester and task assignee.
- **helpdesk_search_tickets** - fuzzy search across tickets by subject and body, returning a compact projection ordered by most recently updated.

- **helpdesk_find_similar_tickets** - rank tickets similar to a given one, for finding duplicates or related issues. This is the dedupe surface a triage agent uses before merging.

Acting on tickets:

- **reply_to_ticket** - post a public reply or an internal note. It resolves the ticket number to the record first.
- **update_ticket_status** - change a ticket's status. Its status values are the canonical set: `open`, `in_progress`, `waiting_on_customer`, `resolved`, `closed`.

Configuration and intake:

- **helpdesk_get_public_settings** - read the public settings subset (email-verification requirement, categories, welcome message). No auth required.
- **helpdesk_get_settings** - read the full org settings (admin).
- **helpdesk_update_settings** - edit settings (admin).
- **helpdesk_set_default_project** - set the project that incoming tickets create their Bam task in, by org slug and project slug (admin).
- **helpdesk_upsert_user** - idempotently create or update a customer by org and email, for example from an external system reconciling its user list. The update path never overwrites the password.

Reporting:

- **helpdesk_ticket_count_by_phrase** - count tickets matching a phrase across time buckets, for trend analysis.

Two agent flows are worth calling out for human reviewers:

- **Ticket to Bam task.** Every ticket the portal creates already spawns a Bam task in the default project. An agent working tickets sees the same trail there. When you review an agent's work, check both the ticket conversation and the linked task.
- **Similar-ticket search before merging.** A triage agent should call **helpdesk_find_similar_tickets** to find candidates before merging duplicates. A human should confirm the proposed primary before a merge closes the source ticket, since merge moves messages and is not a customer-reversible action.

Helpdesk also participates in the cross-cutting agentic platform that spans the whole suite:

- **Identity, audit, and heartbeat.** Agent and service callers carry an explicit kind, and their actions are stamped onto the activity trail. Long-running runners report liveness with `agent_heartbeat`. Helpdesk ticket actions show up in this trail like any other app's.
- **Unified activity view.** Helpdesk's per-ticket activity log is one of the sources UNIONed into the platform's unified activity view, so a cross-app agent can see ticket events alongside Bam, Bond, and the rest. Helpdesk uses `actor_type=agent` to mean a human support agent; the platform remaps that to the human kind so it does not collide with the platform's agent identity.

- **Approval queues.** Sensitive proposals can be routed to a durable approval queue (`proposal_create`, `proposal_list`, `proposal_decide`) so a human signs off before an agent acts. Pair this with merges and bulk status changes.
- **Visibility preflight.** Before an agent posts ticket content into a shared cross-app surface, it must call `can_access` for every cited entity and drop anything the asker is not allowed to see.
- **Agent policies and outbound webhooks.** A per-agent kill switch and tool allowlist (the `helpdesk.*` prefix among them) gate which Helpdesk tools an agent may call, and outbound webhooks push subscribed Helpdesk Bolt events (ticket created, message posted, status changed, closed, reopened, user upserted, SLA breached) to agent runners.

For the complete tool catalog, see the MCP tools reference in `docs/apps/helpdesk/`.

Working together (live presence)

BigBlueBam treats collaboration as ambient, not as a scheduled meeting. When you open a ticket, a presence strip shows who else is on it, so support and engineering can drop into a huddle on the same ticket instead of trading messages. Voice and video here are the digital version of bumping into a colleague in the hallway or stopping by their desk: a quick question, a shared look at the same thing, then back to work. Your presence travels with you across the suite through the Bureau virtual office. The Introduction covers the full pervasive-presence model.

User Stories

Story: File your first support ticket

Who: A customer reaching support for the first time. **Goal:** Get a support request in front of an agent. **Before you start:** Know which organization you need support from. Signups must be enabled on that org's portal. You do not need a Bam suite account; Helpdesk has its own customer sign-in.

Steps

1. Go to `/helpdesk/`. On the **Choose your support portal** page, click your org's row.
2. On the portal, open **Create your account**. Enter **Email**, **Display Name**, and a **Password** of at least 12 characters, then click **Create Account**.
3. If the org requires verification, open the link in your email, wait for "Your email has been verified!", click **Go to Login**, and sign in.
4. On **My Tickets**, click **New Ticket**.
5. Fill in **Subject** and **Description**. Pick a **Category** if the portal offers one, and set **Priority** (Low, Medium, or High).
6. Click **Submit Ticket**.

Result: You land on the ticket's detail page with a ticket number like `#123`. Behind the scenes a Bam task is created so support staff can pick it up.

Related: Track and reply to a ticket; an agent can intake a customer with **helpdesk_upsert_user**.

Story: Track a ticket and reply to an agent

Who: A customer with an open ticket. **Goal:** Read the agent's reply and respond. **Before you start:** You have filed at least one ticket and are signed in.

Steps

1. Open **My Tickets**. Rows with a **New** dot have activity you have not seen.
2. Click the ticket to open it.
3. Read the timeline. Agent and system messages are on the left; your own are on the right.
4. In the reply box, type into "Type your reply...".
5. Click **Send Reply**.

Result: Your reply appears in the timeline. If the ticket was **Waiting on Customer**, it flips back to **Open**.

Related: Attach a file or inline image.

Story: Filter your tickets by status

Who: A customer with several tickets. **Goal:** Narrow the list to just the tickets in one state. **Before you start:** You are on **My Tickets** with more than one ticket.

Steps

1. Above the list, find the filter chips: **all**, **open**, **in progress**, **waiting on customer**, **resolved**, **closed**.
2. Click the chip for the status you want, for example **waiting on customer** to see the tickets where the agent is waiting on you.
3. Click **all** to clear the filter.

Result: The list shows only the tickets in the chosen state. You can combine a chip with the search box to narrow further.

Related: My Tickets list feature.

Story: Attach a file or inline image

Who: A customer who needs to send a screenshot, log, or document. **Goal:** Get a file onto the ticket. **Before you start:** The ticket is open (the reply box is hidden once a ticket is Resolved or Closed).

Steps

1. Open the ticket.
2. To send an inline image, paste or upload it into the rich-text reply or description editor.
3. To send a file, add it as an attachment from the ticket.
4. Find it in the **Attachments** block.

Result: The file shows its name, size, type, and scan status. A **Download** link appears once it has been scanned; until then it shows **Unavailable**.

Story: Change a ticket's priority

Who: A customer whose issue became more or less urgent. **Goal:** Reset the priority. **Before you start:** You are on the ticket detail page.

Steps

1. In the header, click the **Priority** badge to open its dropdown.
2. Choose **Low**, **Medium**, or **High**.

Result: The badge updates. Customers cannot set Critical; only agents can.

Story: Mark a ticket as a duplicate

Who: A customer who filed the same issue twice. **Goal:** Point this ticket at the original so updates land in one place. **Before you start:** You know the primary ticket's number. The ticket you are marking is not already closed, resolved, or a duplicate.

Steps

1. Open the duplicate ticket and click **Mark as duplicate**.
2. In the dialog, type the primary ticket's number ("e.g. #123").
3. Click **Confirm**.

Result: A banner says the ticket is a duplicate of #N and updates will be posted on that ticket. The primary shows a "Duplicates of this ticket" callout. Click **Unmark** to undo, as long as an agent has not merged it.

Story: Close a ticket, then reopen it

Who: A customer whose issue is resolved, or who needs to reopen one. **Goal:** Close a settled ticket, or reopen one that turned out not to be done. **Before you start:** You are on the ticket detail page.

Steps

1. To close, click **Close Ticket** and confirm "Close this ticket?".
2. To reopen later, find the "This ticket has been resolved." or "This ticket has been closed." banner and click **Reopen**.

Result: Closing also moves the linked Bam task to a terminal state. Reopening returns the ticket to an active state and brings back the reply box.

Story: Share a ticket to a Banter channel

Who: A customer or agent who wants to discuss a ticket in chat. **Goal:** Post a link to this ticket into a Banter channel. **Before you start:** You can see the ticket and have at least one Banter channel available.

Steps

1. On the ticket, click the **Share to Banter** button.
2. In the popover, pick a channel from the **Channel** select.
3. Optionally add a note in **Message (optional)**.
4. Click **Share**.

Result: The ticket is posted to the chosen Banter channel.

Story: Turn on browser notifications

Who: A customer who wants to know the moment an agent replies. **Goal:** Get a browser push when there is ticket activity. **Before you start:** You are on the **My Tickets** list and your browser has not been asked yet.

Steps

1. On the banner "Get notified when agents respond to your tickets.", click **Enable notifications**.
2. Allow notifications in the browser prompt.

Result: Your browser is permitted to show ticket notifications. Whether one arrives also depends on the org's notify settings and on outbound email being configured.

Story: Work the agent queue

Who: A support agent. **Goal:** Triage and answer tickets. **Before you start:** You have a per-agent agent API key (prefixed `hdag_`). A Bam session alone is not accepted on the agent routes.

Steps

1. Pull the agent queue, filtered by assignee, status, or SLA state, with **list_tickets** or by querying the agent queue route directly. Each ticket carries an SLA badge.
2. Open a ticket with **get_ticket** (or resolve a `#number` first with **helpdesk_get_ticket_by_number**). As an agent you see internal notes that customers do not.
3. Reply with **reply_to_ticket**, choosing a public reply or an internal note.
4. Move the ticket forward with **update_ticket_status**, using a canonical status (open, in_progress, waiting_on_customer, resolved, closed).
5. Close the ticket when it is done.

Result: The ticket reflects your reply and new status. The linked Bam task is updated in step with the ticket.

Related: Merge duplicates; note that the queue's SLA badge target is a fixed 4 hours and may not match the org's configured SLA.

Story: Merge duplicate tickets

Who: A support agent cleaning up duplicate reports. **Goal:** Consolidate two tickets into one. **Before you start:** You have agent access and a candidate ticket that may be a duplicate.

Steps

1. Call **helpdesk_find_similar_tickets** on the ticket to get a ranked list of candidates.
2. Confirm which ticket is the true primary.
3. Merge the source into the primary through the agent merge action.

Result: The source ticket's messages move onto the primary and the source is closed. Unlike a customer's "mark as duplicate", an agent merge is not customer-reversible, so confirm the primary before merging.

Story: Set up a new support portal

Who: An org admin or owner. **Goal:** Configure the org's portal so tickets route correctly. **Before you start:** You have a Bam admin or owner session, or an `hdag_` agent key. The org has at least one project.

Steps

1. Open the org's Helpdesk settings (read them with **helpdesk_get_settings**).

2. Set the **Default project** so incoming tickets create their Bam task in the right place. Use **helpdesk_set_default_project** or set `default_project_id` directly.
3. Set the **Categories** customers can choose from, a **Welcome message**, and a **Default priority**.
4. Restrict **Allowed email domains** and decide whether to **Require email verification**.
5. Save with **helpdesk_update_settings**. The settings row is created if it did not exist.

Result: The portal is configured. New tickets land in the right project, and customers see your categories and welcome message.

Related: Disable signups by setting the signup-disabled flag; after that the "Create one" link routes to the Bam beta gate.

Story: Reconcile a customer from an external system

Who: An admin or an integration agent. **Goal:** Make sure a customer exists in Helpdesk without disturbing an existing account. **Before you start:** You have admin authority and the org slug. The intake request must carry the org context.

Steps

1. Call **helpdesk_upsert_user** with the org slug and the customer's email.
2. Provide the display name and any other fields.

Result: The customer is created if new, or updated if they already exist, matched on org and email. The password is never overwritten on update. A `user.upserted` event is emitted for downstream automations.

Story: Report on ticket trends

Who: An admin or agent watching support load. **Goal:** See how often a phrase appears in tickets over time. **Before you start:** You have admin authority.

Steps

1. Call **helpdesk_ticket_count_by_phrase** with the phrase you want to track.
2. Choose the time buckets (for example by hour, day, or week) and the time window.

Result: A time-bucketed count of tickets matching the phrase, which you can use to spot spikes or recurring issues.

Flagged behavior (known mismatches)

These are real behaviors in the current code that can surprise you. Document them so users do not chase them as bugs.

- **No Knowledge Base in Helpdesk.** Some older guide material describes a Helpdesk knowledge base. There is no such feature in the Helpdesk code. The knowledge base product is the separate **Beacon** app. Do not look for a KB in Helpdesk.
- **Agent-queue SLA badge target is fixed.** The agent queue's SLA badge uses a fixed 4-hour first-response target with a 0.75 imminent threshold. The actual breach monitor uses the per-org SLA settings (default 8 hours for first response, 48 hours for resolution). The badge and the recorded breaches can therefore disagree.

Related

- **Bam** - every ticket spawns and stays linked to a Bam task in the portal's default project. Agents work tickets from Bam's ticket views and project board. Closing a ticket moves its task to a terminal state.
- **Banter** - share a ticket into a Banter channel with the **Share to Banter** action for internal discussion.
- **Beacon** - the actual knowledge base product. Helpdesk has no knowledge base of its own.
- **Bolt** - Helpdesk emits ticket lifecycle events (created, message posted, status changed, closed, reopened, user upserted, and SLA breached) that Bolt automations can react to.
- MCP tools reference and the app guide live in `docs/apps/helpdesk/`. The 13 Helpdesk MCP tools are listed under Working with AI agents above.

Check yourself

Every answer is findable in this chapter. Check your work against the Answer Key appendix at the back of the book.

1. What are the five canonical ticket statuses?

a) Open, In Progress, Waiting on Customer, Resolved, Closed
b) New, Open, Pending, Done, Archived
c) Open, Assigned, Escalated, Resolved, Closed
d) Draft, Open, Blocked, Resolved, Closed

2. Which priority can an agent set that a customer cannot?

a) Low
b) Medium
c) High
d) Critical

3. What is the minimum length for a customer's portal password?

a) 8 characters
b) 10 characters
c) 12 characters
d) There is no minimum

4. If you submit the same subject and description again within an hour, what happens?

a) A second ticket is created
b) The portal returns your existing ticket instead of creating a duplicate
c) Submission is blocked with an error
d) The original ticket is overwritten

5. What credential do support agents use on the agent routes?

a) A Bam session cookie alone
b) An hdag_ agent API key in the X-Agent-Key header
c) The customer portal sign-in
d) An OAuth token from the suite single sign-on

6. If you are looking for a knowledge base, which app should you use?

a) Helpdesk's built-in KB
b) Beacon
c) Brief
d) Bench

7. What happens to a Waiting on Customer ticket when the customer sends a reply?

8. What happens to a ticket's linked Bam task when the ticket is closed?

9. How long is an email verification token valid before it expires?

10. How is a customer's mark-as-duplicate different from an agent's merge?

11. When no default project is set on the portal, where do new tickets create their Bam task?

12. Which MCP tool ranks tickets similar to a given one to find duplicates before merging?

a) helpdesk_search_tickets
b) helpdesk_find_similar_tickets

c) helpdesk_get_ticket_by_number
d) list_tickets

Answer Key

Answers to every "Check yourself" review, grouped by chapter. Each answer cites the section where you can confirm it.

Chapter 2. Bam

1. b) 6 letters *(see: Key concepts: Project)*
2. b) One *(see: Key concepts: Sprint)*
3. It can be carried forward to a target sprint, moved to the backlog, or cancelled. *(see: Key concepts: Carry-forward)*
4. b) Open subtasks *(see: Key concepts: Done-gate)*
5. New tasks default to Medium, and Bam supports seven custom field types: text, number, date, select, multi-select, checkbox, and url. *(see: Key concepts: Priority and Custom field)*
6. d) Mind map *(see: Key concepts: Saved view)*
7. a) owner, admin, and member, plus a guest role *(see: Key concepts: Organization)*
8. todo, active, blocked, review, done, and cancelled. *(see: Key concepts: Task state)*
9. a) admin, member, and viewer *(see: Key concepts: Project)*
10. No Swimlanes, By Assignee, By Priority, and By Epic. *(see: Key concepts: Swimlane)*
11. c) You cannot; watchers are populated by the system as people get involved with the task *(see: Key concepts: Watcher)*
12. An epic is open, in progress, or closed, and it rolls up task counts and story points. *(see: Key concepts: Epic)*

Chapter 3. Banter

1. c) 40,000 characters *(see: Key concepts: Message)*
2. a) public, private, and a DM type *(see: Key concepts: Channel)*
3. The roles are owner, admin, member, and viewer; a viewer is read-only and cannot post. *(see: Key concepts: Channel role)*
4. b) 3 to 8 people *(see: Key concepts: Direct message (DM))*
5. Only channel admins can pin or unpin, and a pin is channel-wide and visible to every member in the channel's pinned list. *(see: Key concepts: Pin)*
6. c) Only you *(see: Key concepts: Bookmark)*
7. b) do not disturb *(see: Key concepts: Presence)*
8. Banter creates #general, marks it the default, and auto-joins every active member, with the creator becoming owner. *(see: Key concepts: #general)*
9. a) Everyone, Admins only, Organization owners only *(see: Admin: organization policy)*
10. Channel creation is rate-limited to 5 per hour. *(see: Create a channel)*
11. b) It stays in the thread only *(see: Key concepts: Thread)*
12. Banter no longer starts or joins calls; every call write endpoint returns HTTP 410 Gone, Banter keeps only a read-only view of past calls and transcripts, and live audio happens in the suite-wide Bureau docked box. *(see: Past calls and audio)*
13. Quiet hours is a per-channel policy that holds non-urgent posts until the channel's allowed hours, used mostly by automated and agent posts. *(see: Key concepts: Quiet hours)*

14. a) Ctrl/Cmd+K *(see: Keyboard shortcuts)*

Chapter 4. Beacon

1. a) Draft, Active, Pending Review, Archived, Retired *(see: Key concepts: Status)*
2. c) 500 characters *(see: Key concepts: Beacon)*
3. Verified recently, Content is stale, Expiring soon, and Needs verification. *(see: Key concepts: Freshness)*
4. d) Private *(see: Key concepts: Visibility)*
5. It moves the article to Pending Review so an owner can review it. *(see: Key concepts: Challenge)*
6. c) Blocks *(see: Key concepts: Link)*
7. It resets the expiry date and raises the verification count. *(see: Lifecycle actions)*
8. b) Organization *(see: Create and edit an article)*
9. c) 10 MB *(see: Attachments)*
10. Project, then Organization, then System. *(see: Expiry Policy Settings)*
11. c) 1 to 20 *(see: Tags)*
12. It moves expired Active articles to Pending Review, and moves Pending Review articles to Archived after the grace period (it also deletes very old Drafts). *(see: Expiry Policy Settings)*
13. b) A created flag *(see: Working with AI agents)*
14. Agents cannot post comments or upload files; those actions stay human-only. *(see: Working with AI agents)*

Chapter 5. Bearing

1. b) At least one Period *(see: Where to find it)*
2. a) Number, Percentage, Currency, Yes/No *(see: Key concepts: Key Result)*
3. A goal's overall progress is the average of its key results' progress, and each key result's progress is clamped between 0 and 100 percent. *(see: Key concepts: Key Result)*
4. b) Organization, Team, Project, Individual *(see: Key concepts: Scope)*
5. It derives the status automatically by comparing actual progress against expected progress, which is where the goal should be based on how much of the period has elapsed, unless someone overrides it. *(see: Key concepts: Expected progress)*
6. c) Deletion is blocked with a conflict error *(see: Periods)*
7. b) planning *(see: Periods)*
8. The Create Goal dialog does not offer the Individual scope or an owner field; new goals are owned by you, and reassigning a goal to another owner is done through the agent tool bearing_goal_update or REST, not the human UI. *(see: Create a Goal)*
9. b) Updates the KR, rolls up the goal's progress, re-derives the goal's status, and writes a snapshot *(see: Record progress on a Key Result)*
10. It binds the key result to a Bam epic, project, sprint, or task so a background recompute job keeps the key result and its goal current as linked tasks reach a done state. Linking is available

only through the MCP tools and REST, not a human screen in this build. *(see: Working with AI agents)*

11. b) By user ID first, then by email *(see: Watch a goal)*
12. My Goals splits goals into Active and Completed sections, where Completed means a goal that is achieved or missed. *(see: My Goals)*

Chapter 6. Bench

1. c) Nothing; it queries other products' tables through a registry *(see: Overview)*
2. c) Eleven *(see: Widget Wizard)*
3. Private, Project, or Organization. *(see: Key concepts: Dashboard)*
4. b) At least one *(see: Key concepts: Measure)*
5. count, sum, avg, min, or max. *(see: Key concepts: Measure)*
6. d) It is forced to Private *(see: Dashboards list)*
7. b) The first measure with its first aggregation, plus the first two dimensions, limited to 50 rows *(see: Ad-Hoc Explorer)*
8. Email, Banter Channel, and Brief Document. *(see: Key concepts: Scheduled report)*
9. a) PDF, PNG, and CSV *(see: Key concepts: Scheduled report)*
10. Their backing tables have no organization column, so the query builder's org filter fails and the query returns nothing. *(see: Sources that do not return data)*
11. b) Over 30%, with high, medium, or low severity *(see: Working with AI agents)*
12. The Default Cache TTL is 60 seconds and the Statement Timeout is 10,000 ms. *(see: Bench Settings)*

Chapter 7. Bill

1. b) In integer cents *(see: Overview)*
2. c) Finalizing an invoice *(see: Overview)*
3. Its number is assigned permanently and its line items can no longer be edited; invoices are only editable while in draft. *(see: Overview)*
4. b) No, it is computed from the due date *(see: Key concepts: Invoice status)*
5. draft, sent, viewed, partially_paid, paid, and void. *(see: Key concepts: Invoice status)*
6. An expense starts pending, a reviewer moves it to approved (or rejected), and an approved expense can be moved on to reimbursed. *(see: Key concepts: Expense)*
7. b) The most specific rate that applies *(see: Key concepts: Rate)*
8. b) Bill rejects the payment *(see: Invoice detail)*
9. With Auto-finalize on, every generated invoice is assigned a number and marked sent automatically; with it off (Draft), each run produces a draft for you to review, finalize, and send by hand. *(see: Recurring billing)*
10. b) No, it is view-and-PDF only and you record payments yourself *(see: Public invoice view)*

11. It produces {prefix}-{number:05d}, for example INV-00001; year-based forms like INV-2026-0042 from some seed data are not what finalize generates. *(see: Working with AI agents)*
12. b) Only approved expenses *(see: Reports)*
13. It is a UI and REST action via POST /bill/api/v1/expenses/:id/reimburse, with no dedicated MCP tool. *(see: Working with AI agents)*

Chapter 8. Blank

1. b) draft *(see: Key concepts: Form status)*
2. b) 10 *(see: Public form)*
3. Public (anyone with the link), Organization (members of your org), and Project members (members of a chosen Bam project). *(see: Key concepts: Visibility)*
4. c) SVG *(see: Responses)*
5. It must have at least one field. *(see: Publish dialog)*
6. b) Its fields and submissions *(see: Forms list)*
7. b) Section Header, Paragraph, Hidden Field, Page Break *(see: Key concepts: Field type)*
8. https://YOUR-DOMAIN/forms/<slug>, the server-rendered public form page. *(see: Key concepts: Public form URL)*
9. d) Nowhere; there is no Close button in the SPA, so closing is done through the API or an agent *(see: Story: Close a form to stop accepting responses)*
10. From their signed-in account, then the form's email field, then an explicit email in the response, in that order. *(see: Access control and notifications)*
11. b) Every file was validated and stored *(see: Responses)*
12. A BigBlueBam session or a bbam_ / bbam_svc_ API key, plus membership in an organization (all forms and submissions are scoped to your org). *(see: Where to find it: Prerequisites)*

Chapter 9. Blast

1. c) Six *(see: Key concepts: Campaign status)*
2. c) sent *(see: Key concepts: Campaign status)*
3. An unsubscribe mechanism (such as an {{unsubscribe_url}} merge field or text containing unsubscribe or opt-out) and a physical mailing address. *(see: Key concepts: CAN-SPAM gate, and Why a send was rejected)*
4. a) draft and scheduled *(see: Campaign detail and Send Now)*
5. It pulls its audience from Bond (contacts and segments over Bond contact fields) and pushes activity into Bolt so automation rules can react. *(see: Overview)*
6. b) In the Bam app under Account Settings then Integrations *(see: SMTP (platform relay))*
7. It does not send immediately; it schedules the campaign about an hour out and returns a note that a human must confirm before it goes. *(see: Working with AI agents)*
8. b) 25 *(see: Campaign detail and Send Now)*

9. ALL conditions (AND), which requires every condition, and ANY condition (OR), which matches any one. *(see: Key concepts: Segment, and Segment builder and preview)*
10. c) admin *(see: Sender Domains)*
11. It marks the recipient complained, increments the complaints counter, and automatically unsubscribes that address org-wide. *(see: Provider webhooks (automatic, no UI))*
12. b) Hard-coded templated strings *(see: Working with AI agents)*

Chapter 10. Blueprint

1. c) As a typed relational graph of node and edge rows *(see: Overview)*
2. a) Rounded, Rectangle, Diamond, Ellipse, and Hexagon *(see: Key concepts: Shape)*
3. c) bam.task *(see: Key concepts: Linked entity)*
4. a) Default, Dependency, Flow, Reference, and Inherits *(see: Key concepts: Edge)*
5. d) org_chart *(see: Generate a diagram from a Bam project)*
6. b) Mermaid (.mmd) and JSON *(see: Export a diagram)*
7. A pinned node is excluded from auto-layout. It keeps the position you set by hand when you run a layout pass, and it cannot be dragged until you unpin it. *(see: Key concepts: Pinned and Move and pin nodes)*
8. Private (you, plus collaborators), Project (project members, or the org if there is no project), and Organization (everyone in your org). *(see: Key concepts: Visibility)*
9. An edge from A to B means A is the parent of B. It is the default, with target-parent reversing it and none skipping parent links entirely. *(see: Promote a whole graph to Bam tasks)*
10. Owner, Editor, Commenter, and Viewer, granted as explicit per-diagram access on top of project or org visibility. *(see: Key concepts: Collaborator and Comments, People, and version History panels)*
11. Each snapshot is numbered automatically, one higher than the latest, and it saves a complete immutable copy of the whole graph that you can restore later. *(see: Snapshots (versions))*
12. blueprint_generate accepts 1 to 500 node specs and up to 2000 edge specs, with replace true to wipe the existing graph first or auto_layout false to skip the post-generate pass. *(see: Working with AI agents)*

Chapter 11. Board

1. b) project *(see: Key concepts: Visibility)*
2. c) dots *(see: Key concepts: Background)*
3. The soft limit is 500 elements, where you get a warning, and the hard limit is 2000 live elements, at which further additions are rejected. Deleted elements do not count. *(see: Element limits)*
4. a) yellow *(see: Key concepts: Sticky note)*
5. It reads your shared BigBlueBam session cookie, and your permissions come from Bam. If you are not signed in, Board shows a 'Please log in to BigBlueBam first' screen. *(see: Overview)*
6. c) The board creator or an org admin or owner *(see: Key concepts: Lock)*

7. You draw with Excalidraw's own native toolbar. Board overlays a floating board toolbar, a connection status badge, an integrity banner when the project link is broken, and a chat panel. *(see: The infinite canvas)*
8. b) Copies a shareable link to the board to your clipboard *(see: The board toolbar)*
9. There is a complete backend for collaborators but no screen in the Board app yet, and the Share button only copies the link. Until a sharing screen ships, control access through the board's visibility setting, or have an agent or API client manage collaborators directly. *(see: Collaborators (no in-app management today))*
10. b) JSON, SVG, and PNG *(see: Export)*
11. The Description field and the list's element-count and creator-name fields render blank because the backend does not store the description and does not return an element count or creator name on versions today. Only the version name, creator reference, and timestamp are saved and shown reliably. *(see: Version history (snapshots))*
12. b) A labeled region that groups the elements inside it *(see: Key concepts: Frame)*
13. It reports the realtime connection state for the canvas, shown as Live (green), Connecting (amber), or Offline (red), plus an 'N editors' chip when other people are present. *(see: Real-time presence and audio)*

Chapter 12. Bolt

1. b) A trigger, optional conditions, and an ordered list of actions *(see: Overview)*
2. b) The bare event name plus an explicit separate source field, such as deal.rotting with bond as the source *(see: Key concepts: Event source)*
3. Running, Success, Partial (some steps failed but all ran), Failed (halted early), and Skipped (a guard or condition prevented it). *(see: Key concepts: Execution)*
4. b) 60 *(see: Key concepts: Limits)*
5. c) 13 *(see: Key concepts: Condition)*
6. Stop (halt the run), Continue (run the rest anyway), or Retry (retry up to the Retry Count). *(see: Key concepts: On Error)*
7. c) 16 *(see: Templates)*
8. The Schedule source fires automations on a cron schedule rather than off another app's event, and the event type is cron.fired. *(see: Key concepts: Schedule trigger)*
9. b) Disabled *(see: Duplicating an automation)*
10. Retry appears only for runs whose status is Failed or Partial, and if the rule is over its hourly cap the retry is rejected as rate limited. *(see: Retrying a failed run)*
11. b) As a single linear ordered list that runs top to bottom *(see: Simple mode)*
12. Fields that read the event body must be prefixed with event., for example event.task.priority. *(see: Key concepts: Condition)*

Chapter 13. Bond

1. a) Contacts, companies, deals, and pipelines *(see: Overview)*
2. b) Lead *(see: Key concepts: Lifecycle stage)*

3. It is the deal's value multiplied by its probability. *(see: Key concepts: Deal)*
4. b) 0 to 100 *(see: Key concepts: Lead score)*
5. c) When it is more than 1.5 times over its stage's rotting threshold *(see: Key concepts: Rotting (stale) deal)*
6. Active, Won, or Lost. *(see: Key concepts: Stage)*
7. a) Prospect, Qualified, Proposal, Negotiation, Closed Won, and Closed Lost *(see: Bond Settings: Pipelines)*
8. Closing Won sets probability to 100 and moves it to a Won-type stage; closing Lost sets probability to 0, moves it to a Lost-type stage, and records the reason and competitor. *(see: Deal detail and outcomes)*
9. b) No, members and viewers see only the deals and contacts they own *(see: Where to find it: Roles and visibility)*
10. No, deletes are soft: the record is hidden, not erased. Restore a contact or company by checking Include deleted in its list and clicking Restore on the struck-through row. *(see: Key concepts: Soft delete / Restoring deleted records)*
11. b) Through the REST API with a JSON contacts array (1 to 5000 records) or an agent, with admin access *(see: Bulk-importing contacts)*
12. They are org-wide for that entity type (Contact, Company, or Deal), not per pipeline. *(see: Key concepts: Custom field / Bond Settings: Custom Fields)*
13. c) Recalculating a contact's lead score *(see: Bond Settings: Lead Scoring)*

Chapter 14. Book

1. b) A personal calendar named My Calendar; no, it cannot be deleted *(see: Overview and Key concepts: Calendar)*
2. b) status confirmed, visibility busy *(see: Key concepts: Event)*
3. Cancelling is a soft delete: the event's status flips to cancelled rather than the row being erased, so it drops out of availability and the timeline but its record is retained. *(see: Key concepts: Event, and Story: Cancel an event)*
4. b) Free *(see: Key concepts: Show as (visibility))*
5. Monday to Friday, 09:00 to 17:00, are enabled by default. *(see: Key concepts: Working hours, and the Working Hours feature)*
6. b) As a confirmed event on the page owner's default calendar *(see: Key concepts: Booking, and Public Meet page)*
7. The Reminder dropdown and the Color picker. Book has no reminder feature or job, and event color is not stored on the event because color lives on the calendar, so both controls are inactive. *(see: New Event and Edit Event: Notes and current limitations)*
8. b) The rule is stored on the single event but not expanded, so it remains one event *(see: New Event and Edit Event: Notes and current limitations)*
9. The .ics feed provider works with no credentials; Google Calendar and Microsoft Outlook require operator-supplied OAuth credentials before a connection can be created. *(see: Connections, and Key concepts: External connection)*
10. b) Every fifteen minutes *(see: Connections: How sync works)*

11. Disconnecting removes the connection and every event Book mirrored from that feed, so the calendar returns to just your own events. *(see: Connections: How sync works, and Key concepts: Synced (mirrored) event)*
12. b) A row lock guards the booking, and a collision returns a 409 'This time slot is no longer available' *(see: Public Meet page)*
13. The human form does not support adding or editing attendees; attendees are set when an event is created through the API or an agent tool, book_create_event. *(see: New Event and Edit Event: Notes, and Working with AI agents)*

Chapter 15. Brief

1. a) Draft, In Review, Approved, and Archived *(see: Key concepts: Status)*
2. b) Public *(see: Key concepts: Visibility)*
3. Publish sets the document to Approved; Save Draft keeps it as Draft. *(see: Save a draft and publish)*
4. b) Up to 2 characters *(see: Key concepts: Document)*
5. Organization (all org members), Project (the creator, collaborators, or members of that project), and Private (the creator and collaborators only). *(see: Key concepts: Visibility)*
6. c) 48 *(see: Working with AI agents)*
7. A conflict-free shared editing model (Yjs) over the /brief/ws WebSocket. *(see: Edit an existing document and co-edit in real time)*
8. a) Brief creates a Beacon article from it, and the promotion is one-way *(see: Promote to Beacon)*
9. The UI can only create folders; renaming, moving, and deleting folders are done through an agent or the API. *(see: Folders and project scope)*
10. b) At least 2 *(see: Search)*
11. It is not stored by Brief today, so anything typed there is dropped on save; put summary text in the document body instead. *(see: The "Brief summary (optional)" field)*
12. a) A Draft copy named "<title> (copy)" *(see: Duplicate a document)*
13. It returns to Draft status. *(see: Archive and restore)*

Chapter 16. Bureau

1. c) 8 *(see: Key concepts: Room)*
2. a) Open, Knock, and Private *(see: Key concepts: Door state (privacy))*
3. 30 seconds. *(see: Key concepts: Knock)*
4. b) 24 hours *(see: Key concepts: Room chat)*
5. available, busy, dnd, focus, away, and in_meeting. *(see: Key concepts: Presence status)*
6. b) Puts them on a denied list rather than pinging them with a dead link *(see: Key concepts: Summon (Bring everyone here))*
7. "Let in" admits them, "Not now" defers the knock, and "Decline" rejects the visitor. *(see: Knock and respond to a knock)*
8. b) The knock is blocked (the server returns 423) and they are offered a leave-a-note path *(see: Knock and respond to a knock)*

9. Open means non-attendees may wander in, and Locked means the room is held private for the meeting. *(see: Key concepts: Booking)*
10. b) members_can_book *(see: Admin - Settings)*
11. b) It returns the same result as "not located" *(see: Hunt)*
12. Because your presence lives on the server, so the docked box re-attaches to the surface huddle wherever you land. *(see: The cross-app docked box)*
13. b) With an "(A) " prefix *(see: Working with AI agents)*

Chapter 17. Helpdesk

1. a) Open, In Progress, Waiting on Customer, Resolved, Closed *(see: Key concepts: Status)*
2. d) Critical *(see: Key concepts: Priority)*
3. c) 12 characters *(see: Register (create an account))*
4. b) The portal returns your existing ticket instead of creating a duplicate *(see: New Ticket)*
5. b) An hdag_ agent API key in the X-Agent-Key header *(see: The agent queue (REST and MCP only))*
6. b) Beacon *(see: Where to find it)*
7. Sending a reply automatically flips the ticket back to Open. *(see: Reply to a ticket)*
8. Closing the ticket also moves its linked Bam task to a terminal state. *(see: Close a ticket)*
9. Verification tokens expire after 24 hours. *(see: Verify email)*
10. A customer mark-as-duplicate is annotative: it posts updates on the primary and can be unmarked. An agent merge actually moves the messages onto the primary and closes the source, and it is not customer-reversible. *(see: Key concepts: Duplicate)*
11. The portal falls back to the project named in the tenant context, then the oldest project. *(see: Settings and admin)*
12. b) helpdesk_find_similar_tickets *(see: Working with AI agents)*

Glossary

Key terms used throughout this book. Each is footnoted on first use in a chapter.

#general. the default channel. The first time anyone opens Banter in an org that has no channels, Banter creates `#general`, marks it the default, and auto-joins every active member (the creator becomes owner). You cannot delete or leave the default channel.

Action. one step Bolt runs when the automation fires. Each action calls an allowlisted MCP tool with parameters you supply. Actions run in order; this is a linear list, not a branching flow (see the note under Actions).

Activity. a timeline entry logged against a contact, company, and/or deal. Types include Note, Email Sent, Email Received, Call, Meeting, and Task, plus system-generated entries like stage changes and deal created/won/lost.

Admin. an org owner or admin who configures the portal settings.

Agent. a support staff member. Agents authenticate with a per-agent agent API key (prefixed `hdag_`) on the agent routes, not with the customer portal. They do not use the customer portal SPA.

Agent pattern subscription. a rule that lets an agent or service account listen to a channel and react when messages match a pattern.

At Risk. The view that lists goals whose status is at_risk or behind, sorted by how far they have fallen behind expected progress.

Attachment. a file uploaded to a ticket. It appears in the Attachments block once it has been scanned, with a Download link.

Attendee. A person invited to an event, identified by email. Attendees carry a response status (`needs_action` until they answer, then `accepted`, `declined`, or `tentative`) and a flag for the organizer.

Automation (rule). the top-level object: a trigger, optional conditions, an ordered list of actions, and limit settings. The UI calls these "Automations".

Availability slot. A computed free interval, equal to your working hours minus the events that mark you busy.

Background. the canvas backdrop pattern. One of `dots` (default), `grid`, `lines`, or `plain`.

Bam task. the project-board task created automatically for every ticket. It lives in the portal's default project so your staff can triage, assign, and track support work alongside other project work.

Beacon. A single knowledge article. Has a globally unique slug, a title, a summary (up to 500 characters), a Markdown body, a version number, a status, a visibility, an owner, an optional project, and an expiry date.

Board. one infinite-canvas whiteboard. It has a name, a description, an emoji icon, a background pattern, a visibility setting, an optional project association, and a locked flag.

Booking. The event created when a visitor books a slot on a booking page. It lands as a confirmed event on the page owner's default calendar.

Booking page. A public scheduling link at `/book/meet/<slug>` that lets an outside visitor pick one of your free slots. It carries a duration, before and after buffers, advance and notice limits, and brand styling.

Bookmark. your own private save of a message, with an optional note. Bookmarks are visible only to you and live on the Bookmarks page.

Building. An optional grouping of floors for multi-site orgs. It exists in the data model and a new floor can carry a `building_id`, but there is no building management screen in the app today.

Calendar. A container for events. Each calendar has a type (`personal`, `team`, `project`, `booking`, or `bureau`), a color, and a timezone. One personal calendar, **My Calendar**, is created for you on first use and cannot be deleted. The `bureau` type is reserved for a system-provisioned, per-org calendar that other apps write room bookings into; you do not create it by hand.

Campaign. A single bulk email send. Has a name, subject, body (HTML), optional plain-text body, an optional template, an optional segment, From name/email, reply-to, and rollup counters.

Campaign status. One of six states: `draft`, `scheduled`, `sending`, `paused`, `sent`, `cancelled`. A draft is editable, deletable, and sendable. A scheduled campaign can be sent or cancelled. A sending campaign can be paused or cancelled. `sent` is the terminal success state and the only one analytics counts. There is no "archived" state.

CAN-SPAM gate. A compliance check that runs when you send or schedule. The HTML body must contain both an unsubscribe mechanism and a physical mailing address, or the send is rejected.

Carry-forward. When you complete an active sprint, each incomplete task can be carried forward to a target sprint, moved to the backlog, or cancelled. Carried tasks show a carry-forward badge and remember their original sprint for reporting.

Category. an optional label chosen from the list an admin configured for the portal. The Category field only appears when categories are configured.

Challenge. A flag that an Active article may be wrong. Challenging moves it to Pending Review.

Channel. a named conversation space. Channels are `public` (anyone in the org can find and join), `private` (invite only), or a DM type (`dm` / `group_dm`). Each channel has a name, an optional topic and description, members with roles, and a slug that is unique within your org.

Channel role. your standing inside one channel: `owner`, `admin`, `member`, or `viewer`. A **viewer** is read-only and cannot post. Channel admins manage pins, members, and settings.

Check-in. Recording a new current value for a key result. Each check-in updates the KR's progress, rolls up to the goal's progress, and writes a point-in-time snapshot for the sparkline and history chart.

Client. The recipient of an invoice. Carries name, email, phone, address, tax ID, default payment terms (in days), default payment instructions, and notes. A Bill client is separate from a Bond contact or company, though it can be linked to a Bond company.

Collaborator. a user granted explicit access to one diagram at a role of owner, editor, commenter, or viewer, on top of any project- or org-level visibility. Managed in the editor's **People** panel, or by an agent through the API.

Comment. a note attached to the whole diagram or anchored to a single node, with a resolved flag for review threads. Read, written, and resolved in the editor's **Comments** panel, or by an agent through the API.

Company. an organization. Holds name, domain, industry, company size bucket, annual revenue, phone, website, logo, address, custom fields, and an owner. Contacts and deals can be linked to a company.

Condition. an extra test on the event: a field, an operator, and a value, grouped with AND or OR logic. Conditions gate the whole automation; if they are not met, the run is skipped. There are 13 operators (equals, not_equals, contains, not_contains, starts_with, ends_with, greater_than, less_than, is_empty, is_not_empty, in, not_in, matches_regex). Fields that read the event body must be prefixed `event.` (for example `event.task.priority`).

Confirmation. what a respondent sees after submitting: a message, a redirect, or a custom page.

Contact. a person you track. Holds name, email, phone, job title, avatar, lifecycle stage, lead source, lead score, address, custom fields, and an owner.

Custom field. A project-scoped field stored on each task. The seven field types are text, number, date, select, multi-select, checkbox, and url. A field can be required and can be shown on the card.

Customer. an end user with a Helpdesk account scoped to one org. Customers have their own email and password sign-in, separate from the suite single sign-on. They use the portal SPA.

Dashboard. A named container of widgets with a saved layout. Each dashboard has a **visibility** of Private, Project, or Organization, an optional linked project, and an optional auto-refresh interval. One dashboard per org can be the default.

Data source. A registered `(product, entity)` pair that maps to one underlying table with declared measures, dimensions, and filters. Bench can only query registered sources and columns, and every query is automatically scoped to your organization.

Deal. a revenue opportunity in a pipeline. Holds a name, value, currency, expected close date, probability, and a computed weighted value (value times probability). A deal is Open until it is closed Won or Lost.

Diagram. the top-level artifact. Has a name, a type, a layout algorithm, a visibility level, an optional project, and an archived flag.

Diagram type. a tag that picks the type icon and the sidebar grouping. The New diagram dialog offers Flowchart, Graph, Sequence, Class, and Mind map. Generate from Bam writes the type org_chart.

Dimension. A field to group by (for example, deal stage or campaign).

Direct message (DM). a private conversation with one other person. A **group DM** is a private conversation with 3 to 8 people total.

Docked box. The floating Bureau overlay rendered inside every SPA. It shows your room, your co-occupants, the page you are viewing, the call controls, and the Bring everyone here / Invite / Hunt actions.

Document. A single piece of writing. It has a title, a globally unique slug, an optional icon (up to 2 characters), a rich-text body, a word count, a status, a visibility, an optional project, and an optional folder.

Done-gate. A task cannot move into a closed state while it still has open subtasks. The board surfaces this as an alert and the API returns an INCOMPLETE_SUBTASKS error.

Door state (privacy). How a room treats visitors. Three values: **Open** (anyone can enter), **Knock** (visitors must knock to be let in), and **Private** (only people on the room's access list). The room stores a durable default; live overrides such as a short do-not-disturb window apply on top of it.

Duplicate. a ticket flagged as a copy of another. A customer "mark as duplicate" is annotative: it posts updates on the primary instead. An agent "merge" actually moves the messages onto the primary and closes the source.

Edge. a connection between two nodes. Carries a kind (Default, Dependency, Flow, Reference, Inherits) and an end marker (Filled arrow, Open arrow, No arrow / None).

Editor. The Tiptap rich-text surface where you write. It has a formatting toolbar, slash commands (type `/`), a live word count, a Table of Contents built from your headings, and a Settings sidebar for icon, visibility, project, and starting template.

Element. anything on the canvas: a sticky note, text, a shape, a connector (arrow or line), an image, or a frame. The canvas scene is stored whole, and a per-element copy is kept so the board can be searched, read, and promoted.

Engagement event. An append-only record of an open, click, unsubscribe, bounce, or complaint. These feed the analytics counters and are published to Bolt.

Epic. A grouping of tasks across sprints, with a status of open, in progress, or closed. Epics roll up task counts and story points.

Event. A time block on a calendar with a title, start and end time, optional location, description, and meeting URL. Events have a status (`tentative`, `confirmed`, or `cancelled`; new events default to `confirmed`) and a visibility (`free`, `busy`, `tentative`, or `out_of_office`;

default `busy`). Cancelling an event is a soft delete: it flips status to `cancelled` rather than erasing the row.

Event source. the emitting app, stored alongside the bare event name. Bolt uses bare-name-plus-explicit-source naming, so the event is `deal.rotting` with `bond` as a separate source field, not `bond.deal.rotting`. The trigger dropdowns offer 14 sources: Bam, Banter, Beacon, Brief, Helpdesk, Schedule, Bond, Blast, Board, Bench, Bearing, Bill, Book, and Blank.

Execution. one run of one automation against one event. Its status is one of: **Running**, **Success**, **Partial** (some steps failed but all ran), **Failed** (halted early), or **Skipped** (a guard or condition prevented it). A skipped run records a reason such as cooldown active, rate limited, or conditions not met.

Execution step. one action attempt inside an execution, with its own status: success, failed, or skipped.

Expected progress. Where a goal should be based on how much of the period has elapsed. Bearing compares actual against expected to decide whether a goal is on track, at risk, or behind.

Expense. A project cost: description, amount in cents, category, vendor, date, and a billable flag. Expenses start as `pending`, a reviewer moves them to `approved` or `rejected`, and an approved expense can be moved on to `reimbursed`.

Expiry policy. Per-scope rules (System, Organization, Project) that set the minimum, maximum, and default number of days before an article expires, plus a grace period.

External connection. A subscription to an outside calendar that Book pulls events from. The `.ics` feed provider works today with no credentials: paste a public iCalendar URL and Book mirrors its events onto a Book calendar, updating in place on each sync. Google Calendar and Microsoft Outlook connections use the same sync engine but require operator-supplied OAuth credentials before they can be created (see Connections below).

Field. a single input or element on the form. Fields are ordered, can be marked required, and each has a stable **field key** (letters, digits, and underscores; must start with a letter or underscore) used as the storage key for answers.

Field type. what kind of input a field is, for example Short Text, Email, Rating, NPS, or Dropdown. You pick the type from the builder palette when you add a field. A few layout-only types (Section Header, Paragraph, Hidden Field, and Page Break) carry no submitted answer.

Filter. A condition on a field. Operators include equals, not equals, greater/less than (and -or-equal), in, is null, is not null, between, and like.

Floor. One navigable office map. A floor owns a layout, an optional background image, a slug used in its URL, and a position in the list. One floor can be marked the default.

Folder. A named container for documents. Folders can be nested and can be scoped to a project. You create folders from the sidebar; renaming and deleting folders is not available in the UI today (an agent can do both, see Working with AI agents).

Form. the top-level object you build and share. It has a name, a slug (used in its public URL), a description, fields, and behavior settings.

Form status. the lifecycle state of a form: **draft** (still being built), **published** (live and accepting responses), or **closed** (no longer accepting responses). New forms start as draft.

Form type. a label on the form (Public, Internal, or Embedded) found on the per-form Settings page.

Frame. a labeled region on the canvas that groups the elements inside it. Frames are used when reading or summarizing a board by section.

Freshness. A computed signal based on when the article was last verified and when it expires. The detail page and cards show one of four states: **Verified recently**, **Content is stale**, **Expiring soon**, or **Needs verification**.

Goal. An objective owned by one user inside a period. A goal has a title, an optional description, a scope, a status, and a cached progress percentage that is the average of its key results.

Goal status. Where a goal stands: **draft**, **on_track**, **at_risk**, **behind**, **achieved**, **missed**, or **cancelled**. Bearing derives status automatically from how actual progress compares to expected progress, unless someone overrides it.

Hunt. Locate one org member and jump to wherever they are. It respects DND and filters out destinations you cannot access.

iCal feed. A token-authenticated `.ics` URL that exposes one calendar to an outside calendar app. Available through the API and agents only; there is no button for it in the app yet.

Integrity issue. a flag on a board whose project link is broken (cross-org, missing, or auto-detached). You resolve it by detaching the board from the project or reassigning it to a valid one.

Invoice. A billable document addressed to one client. It snapshots your company details (the "from" side) and the client details (the "to" side) when it is created, and carries an invoice date, due date, line items, tax, totals, and payment history. Its number is literally `DRAFT` until you finalize it.

Invoice status. The lifecycle of an invoice. The values that the app actually sets are `draft` (editable, no number yet), `sent` (finalized, number assigned, edits locked), `viewed` (a client opened the public link), `partially_paid` (some payment recorded, balance remains), `paid` (paid in full), and `void` (cancelled). **Overdue** is not a stored status: it is computed from the due date as any unpaid, finalized invoice that is past due. The Invoices list **Overdue** filter pill and the Dashboard **Overdue** tile both draw from that computed set, so they populate correctly.

Key Result (KR). A measurable outcome under a goal. Each KR has a metric type (**Number**, **Percentage**, **Currency**, or **Yes/No**), a start value, a target value, a current value, and an optional unit. KR progress is clamped between 0 and 100 percent.

Knock. An async request to enter a closed office. A knock auto-times-out after 30 seconds if the owner does not respond.

Knowledge Graph. The network of Beacons. Nodes are articles; edges are either explicit typed links or implicit "Tag Affinity" edges between articles that share enough tags.

Label. A project-scoped tag with a name and color.

Layout algorithm. the auto-arrange engine, run server-side by ELK. The layout dropdown offers Layered, Force-directed, Tree, and Grid, and all four apply a layout. Tree maps to ELK's mrtree and Grid maps to ELK's rectpacking. The radial algorithm is reachable only through the API or an agent.

Lead score. a number from 0 to 100 cached on each contact. Scoring rules add or subtract points based on contact attributes.

Lead scoring rule. a condition (field, operator, value) plus a point delta. Rules are org-wide and evaluated together to compute each contact's score.

Lifecycle stage. where a contact sits in your funnel: Lead, Subscriber, MQL (marketing qualified), SQL (sales qualified), Opportunity, Customer, Evangelist, or Other. New contacts default to Lead.

Limits. per-automation guards: **Max Executions / Hour** (1 to 1000 in the UI, default 60) and **Cooldown (seconds)** (0 to 3600, default 0). There is also an internal max chain depth that prevents automations from triggering each other in an endless loop.

Line item. One row on an invoice: a description, quantity, unit (default `hours`), and unit price in cents. The amount is quantity times unit price. Line items can only be added, edited, or removed while the invoice is a draft.

Link. A typed connection between two articles: **Related To**, **Supersedes**, **Depends On**, **Conflicts With**, or **See Also**. Links are read-only in the UI today (created through MCP tools or the API; see Links between articles).

Linked entity. a cross-product reference from a node to an object in another app, such as a Bam task, a Beacon entry, a Bond deal, or a Bearing goal. Only Bam tasks have live two-way sync.

Lock. a board-level read-only switch. When a board is locked, only the board creator or an org admin or owner can edit it.

Logic (conditional display). a field can be configured to show only when another field meets a condition (for example, equals or contains a value). This is stored on the field and is available to agents through the MCP field tools; the visual builder does not yet expose a conditional editor.

Materialized view. A pre-computed rollup that the worker refreshes automatically. Three are exposed to you as queryable `bench:` data sources: Daily Task Throughput, Pipeline Snapshot, and Campaign Engagement.

Measure. A numeric aggregation in a query: count, sum, avg, min, or max of a field. A query needs at least one measure.

Mention. typing `@` plus a name notifies that person. Org admins can also define @mention user groups (for example `@engineering`) that notify everyone in the group.

Merge fields. Placeholders the sender fills in per recipient: `{{first_name}}`, `{{last_name}}`, `{{email}}`, `{{company}}`, and `{{unsubscribe_url}}`.

Message. a single post. Messages support rich text (bold, italic, code, links), @mentions, and emoji, up to 40,000 characters. You can edit or delete your own messages.

Node. a vertex on the canvas. Carries a shape, label, markdown description, position, width and height, a pinned flag, fill color, an optional parent (for nesting), and an optional cross-product link.

Office. A room of type Office that has an owner. The owner can be knocked on, sets the door, and admits or declines visitors. Think of it as a personal office.

On Error. per-action behavior when a step fails: **Stop** (halt the run), **Continue** (run the rest anyway), or **Retry** (retry up to the Retry Count).

Organization (org). The top-level tenant. Org roles are owner, admin, and member, plus a guest role. Priorities are configured at the org level.

Payment. A recorded receipt against an invoice: an amount in cents, a method (Bank Transfer, Credit Card, Check, Cash, Stripe, PayPal, or Other), an optional reference, and a date.

Period. A named time box that goals live inside, for example "Q2 2026". A period has a type, a start date, an end date, and a status. The whole Bearing UI is scoped to one selected period at a time.

Period status. The lifecycle of a period. The Bearing UI shows these as **draft**, **active**, **completed**, and **archived**. You activate a period to make it the working scope and complete it when the period is over.

Phase. A board column. A phase can carry a WIP limit, a color, start and terminal flags, and an auto-state that is applied when a task enters the phase.

Pin. a channel-wide marker that any member can see in the channel's pinned list. Only channel admins can pin or unpin.

Pinned. a node excluded from auto-layout. A pinned node keeps the position you set by hand when you run a layout pass.

Pipeline. an ordered set of stages plus a currency. One pipeline can be the default. The default pipeline is selected automatically when you open the board; you can switch to another from the sidebar scope selector.

Portal. an org's public support site, served at `/helpdesk/<org-slug>/`. Each org with a Helpdesk settings row gets one. Per-project portals are available at

`/helpdesk/<org-slug>/<project-slug>/`. The portal a customer lands on decides which org owns their account and tickets.

Presence. your live status: online, idle, in a call, do not disturb, or offline.

Presence status. One of 6 values: **available**, **busy**, **dnd**, **focus**, **away**, **in_meeting**. The docked box exposes a head-down **DND** toggle that flips between available and DND.

Priority. A per-org configurable value. The default set is Critical, High, Medium, Low, and None. New tasks default to Medium.

Project. A board with its own phases, states, labels, epics, sprints, custom fields, and templates. Every project has a task id prefix (2 to 6 uppercase letters, for example MAGE or FRND) that prefixes every task id. Project roles are admin, member, and viewer.

Project scope. A filter in the sidebar that limits the documents and folders you see to one project, or shows everything with "All Projects".

Promote to Beacon. Graduating a finished document into a Beacon knowledge-base article. This is one-way: once promoted, the document records the Beacon it became.

Public form URL. the shareable link to your published form, in the form `https://YOUR-DOMAIN/forms/<slug>`. This is the server-rendered page the Publish dialog hands you.

Public view token. A long opaque token on each invoice that grants read-only access (and a PDF) without logging in. It is how a sent invoice can be viewed by someone outside your org.

Quiet hours. a per-channel policy that holds non-urgent posts until the channel's allowed hours. Used mostly by automated and agent posts.

Rate. A billing rate scoped to your org, a project, a user, or a user-on-a-project, with an amount in cents and a type (hourly, daily, or fixed). When Bill bills tracked time, it resolves the most specific rate that applies.

Reaction. an emoji you toggle on a message. Click once to add, click again to remove.

Real-time collaboration. Two or more people editing the same document at once. You see other editors' cursors and presence while you type, and readers on the detail page see edits appear live.

Recipient / send log. One row per person per campaign. It tracks the status of that one email (queued, sent, delivered, bounced, complained, or failed) and drives the **Recipients** table on the campaign detail page.

Recurring schedule. A subscription that bills one client on a cadence (weekly, monthly, quarterly, or annually) from a saved line-item template. It carries a status (`active`, `paused`, or `cancelled`), a next-run date, a count of invoices generated so far, and a mode: auto-finalize (each generated invoice gets a number and is marked sent) or draft (each generated invoice is left as a draft to review). A daily worker sweep materializes invoices from schedules whose next-run date has arrived, and you can also generate one on demand.

Ring (Invite). Pushes an incoming-call overlay to one specific person on a content surface. It respects DND. An admin can Force Invite, which bypasses the accept prompt with a 3-second cancellable auto-navigate.

Roles. org roles resolve as viewer, member, admin, owner. Admins and owners get full access to any board in the org.

Room. An addressable space on a floor. Each room maps one-to-one to an audio room. Rooms come in 8 types: **Office (single owner)**, **Huddle**, **Conference**, **Meeting**, **Open space**, **Lounge**, **Focus**, and **Lobby**.

Room access list (ACL). Per-user grants for private rooms, with role **member** or **manager**. Managers can edit the room and manage its access list.

Room chat. Ephemeral text chat scoped to a room or surface, kept 24 hours by default and recoverable later under Recent chats.

Rotting (stale) deal. an open deal whose days in its current stage exceed that stage's rotting threshold. The card turns orange, and red when it is more than 1.5 times over.

RSVP. An attendee's response to an event. Only someone who is already an attendee can RSVP.

Saved query. A named, reusable search configuration you can reopen later. Scope can be Private, Project, or Organization.

Saved view. A saved filter, sort, swimlane, and view-type preset. It can be private to you or shared with the project. Saved views persist the Board, List, Timeline, and Calendar view types.

Schedule trigger. the `Schedule` source fires automations on a cron schedule (`cron.fired`) rather than off another app's event.

Scheduled message. a message queued to post later at a set time, or deferred automatically when a channel is inside its quiet hours.

Scheduled report. A cron-driven dashboard snapshot delivered to a target on a schedule. Delivery methods are Email, Banter Channel, and Brief Document; export formats are PDF, PNG, and CSV.

Scope. Who a goal is for: **Organization**, **Team**, **Project**, or **Individual**. The dashboard can filter and group goals by scope.

Segment. A saved filter over Bond contacts, built from one or more conditions joined by **ALL conditions (AND)** or **ANY condition (OR)**. Each segment caches a contact count so you can see roughly how many people match.

Sender domain. A domain you add and verify (SPF, DKIM, DMARC) so mailbox providers accept your mail.

Settings. Your company identity and the defaults new invoices inherit: invoice prefix, default tax rate, payment terms, payment instructions, footer text, and terms text.

Shape. the visual form of a node. The palette offers Rounded, Rectangle, Diamond, Ellipse, and Hexagon.

Show as (visibility). Controls whether an event blocks your availability. An event marked **Free** does not consume a slot; **Busy**, **Tentative**, and **Out of Office** do.

Snapshot (version). a saved, immutable copy of the whole graph that you can restore later. Browsed and restored from the editor's **History** panel.

Soft delete. contacts, companies, and deals are not erased when deleted. They are hidden and can be restored from their list with Include deleted.

Sprint. A time-boxed iteration with a status of planned, active, completed, or cancelled. Only one sprint can be active per project at a time. The default sprint length is 14 days. A sprint carries a goal, start and end dates, velocity, and retrospective notes.

Stage. a column on the board. Has a name, a type (Active, Won, or Lost), a probability percentage, an optional rotting threshold in days, and a color.

Star. your personal favorite flag on a diagram, used to filter the list.

Start and due dates. Each task can carry a start date and a due date. These drive the Timeline (Gantt) and Calendar views and feed the overdue and My Work groupings.

Status. The article's place in its lifecycle. The five statuses you will see in the UI are **Draft**, **Active**, **Pending Review**, **Archived**, and **Retired**. New articles start as Draft; publishing makes them Active.

Status update. A status post against a goal. It records a status tag, an optional body, and a snapshot of the goal's status and progress at the time it was posted.

Sticky note. a colored note. The default color is yellow. (Excalidraw has no native sticky; Board builds one from a rectangle with bound text.)

Submission. one response to a form. Submissions are listed in the Responses view and aggregated in Analytics.

Summon (Bring everyone here). Pulls every eligible co-occupant of your current room to a URL in another app. Each recipient is access-checked first, so people who cannot see the destination are not pinged with a dead link.

Surface huddle. The audio room attached to any content surface (a Board canvas, a Brief doc, a Bond deal). The docked box auto-joins it when you navigate to that surface.

Swimlane. A horizontal grouping of the board. The options are No Swimlanes, By Assignee, By Priority, and By Epic.

Synced (mirrored) event. A Book event that Book created from an external feed. It is kept in step with the feed automatically: a re-sync updates it in place, an event removed upstream is removed from Book, and disconnecting the feed removes all of its mirrored events.

Tags. Free-form labels on an article. Tags drive filtering, search tag expansion, and implicit graph edges. They are a search and filter facet: you add and remove tags as chips on the Search

screen to widen or narrow results. Note: the editor shows a "Tags (comma-separated)" field, but tags typed there are not saved today (a known limitation; see Create and edit an article). Tags that appear on articles come from agent or API writes, not from the editor field.

Task. The unit of work. Each task has a human id in the form PREFIX-N (for example MAGE-38). A task holds a title, rich-text description, phase, state, sprint, epic, assignee, reporter, priority, story points, time estimate and logged time, start and due dates, labels, custom fields, links, parents and subtasks, comments, and attachments.

Task state. The status of a task, orthogonal to its phase. Each state belongs to a category (todo, active, blocked, review, done, or cancelled) and may be a closed state. Tasks in a closed state count toward velocity.

Template. A reusable email design: name, subject template, HTML body, an optional visual block layout, and a description. Templates carry a version number that bumps automatically each time you save an edit.

Templating. action parameters can interpolate values from the event with `{{ event.* }}`, the actor with `{{ actor.id }}` / `{{ actor.type }}`, automation metadata with `{{ automation.* }}`, the current time with `{{ now }}`, and a prior step's result with `{{ step[N].result.* }}`. There is no `{{ actor.name }}` and no top-level `{{ org.* }}`.

Thread. replies attached to a parent message. A reply stays in the thread unless you tick **Also send to channel**, which also mirrors it into the main timeline.

Ticket. a single customer support request. It has a human-readable number shown as `#123`, a subject, a description, a status, a priority, and an optional category. Each ticket is owned by one customer and back-linked to a Bam task.

Timeline. A cross-app, date-ranged view that gathers Book events, Bam task due dates, and Bond deal close dates into one per-day list.

Trigger. what starts an automation. It has a **Trigger Source** (the app that emits the event, such as Bam or Bond) and an **Event Type** (the bare event name, such as `task.created`). A trigger can also carry an optional filter.

Trigger filter. an optional set of equality rules (key matches value) checked against the raw event payload before the automation fires.

Unsubscribe. An org-wide suppression keyed on email. Once an address unsubscribes (or files a spam complaint), it is excluded from future sends across the whole org.

Verification. A recorded review confirming an article is still accurate. Verifying resets the expiry date and raises the verification count.

Version. a named snapshot of a board's canvas that you can restore later.

Visibility. Who can see the article: **Public**, **Organization**, **Project**, or **Private**. Public and Organization are visible to all org members; Private is visible only to the owner or creator; Project is visible to the owner, creator, or members of that project.

Watcher. A user subscribed to a task's notifications. Watchers are populated by the system as people get involved with a task. There is no manual add-watcher or remove-watcher control in the Bam UI.

Widget. One visualization on a dashboard. A widget has a name, a chart type, a data source and entity, and a query configuration. The renderer draws bar, line, area, pie, donut, KPI card, counter, and table widgets distinctly; any other type falls back to a table.

Working hours. Your weekly availability windows, one row per day of the week. By default Monday to Friday, 09:00 to 17:00, are enabled. These windows are the raw material for availability and booking-page slots.

Index

Key terms and the pages where they appear.

#

#general *57, 58, 60, 74, 447, 457*

A

Action *18, 19, 25, 30, 34, 35, 36, 47, 64, 66, 71, 78, 89, 92, 141, 150, 156, 164, 172, 176, 177, 292, 293, 294, 345*
Activity *22, 25, 27, 35, 40, 41, 47, 70, 71, 89, 90, 97, 99, 100, 105, 106, 124, 135, 150, 166, 178, 204, 216, 226, 317*
Admin *18, 30, 31, 32, 34, 41, 44, 46, 49, 51, 55, 57, 59, 60, 63, 65, 69, 70, 71, 75, 76, 267, 268, 424, 425*
Agent *18, 19, 20, 22, 24, 25, 27, 29, 30, 46, 47, 53, 57, 59, 69, 71, 72, 86, 243, 244, 267, 370, 371, 424, 425*
Agent pattern subscription *59, 457*
At Risk *89, 94, 111, 112, 113, 114, 115, 116, 117, 121, 127, 128, 130, 132, 457, 461*
Attachment *40, 57, 61, 75, 97, 107, 108, 200, 207, 210, 425, 432, 439, 457*
Attendee *345, 350, 351, 357, 358, 362, 457, 466*
Automation (rule) *292, 457*
Availability slot *345, 367, 457*

B

Background *112, 122, 123, 129, 130, 176, 189, 200, 202, 203, 210, 221, 247, 266, 267, 278, 279, 287, 288, 289, 294, 304, 308, 357, 396*
Bam task *22, 27, 47, 53, 71, 79, 236, 242, 243, 248, 249, 251, 252, 253, 259, 278, 284, 308, 343, 344, 345, 348, 349, 371, 424*
Beacon *20, 22, 23, 27, 85, 86, 87, 88, 89, 91, 92, 93, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 243, 293, 371*
Board *18, 20, 22, 23, 26, 27, 29, 30, 31, 32, 33, 34, 36, 37, 38, 39, 46, 49, 266, 267, 268, 293, 316, 398, 424*
Booking *19, 20, 22, 27, 125, 151, 343, 344, 345, 346, 347, 350, 351, 352, 353, 354, 358, 361, 363, 364, 366, 367, 368, 395, 396*
Booking page *344, 345, 350, 351, 352, 353, 363, 367, 458*
Bookmark *57, 59, 63, 64, 76, 370, 371, 389, 447, 458*
Building *135, 153, 154, 294, 303, 307, 397, 458*

C

Calendar *20, 23, 29, 30, 32, 34, 38, 40, 51, 52, 343, 344, 345, 346, 347, 348, 349, 350, 351, 353, 354, 355, 356, 357, 358*
Campaign *25, 137, 138, 148, 215, 216, 217, 219, 220, 221, 222, 223, 226, 227, 228, 229, 230, 232, 233, 234, 235, 236, 237, 238, 450*
Campaign status *217, 238, 450, 458*
CAN-SPAM gate *215, 217, 229, 238, 450, 458*
Carry-forward *20, 29, 30, 31, 37, 50, 447, 458*
Category *31, 38, 141, 153, 162, 163, 171, 172, 182, 270, 276, 281, 378, 424, 425, 428, 429, 430, 433, 438, 458, 461, 468*
Challenge *87, 92, 97, 99, 104, 105, 448, 458*

Channel *22, 46, 53, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 74, 75, 76, 77, 78, 138*
Channel role *59, 447, 458*
Check-in *113, 121, 123, 124, 127, 132, 459*
Client *20, 27, 98, 161, 162, 163, 166, 168, 169, 171, 172, 174, 175, 177, 178, 181, 182, 183, 184, 185, 186, 187, 256, 267, 277*
Collaborator *243, 255, 256, 257, 263, 267, 269, 277, 278, 282, 283, 288, 384, 451, 459*
Comment *40, 46, 49, 54, 96, 105, 107, 204, 244, 254, 256, 261, 369, 371, 380, 381, 384, 385, 388, 389, 459*
Company *112, 132, 162, 163, 164, 174, 175, 181, 186, 217, 227, 235, 316, 317, 320, 321, 323, 324, 325, 326, 328, 329, 330, 331, 334*
Condition *138, 191, 216, 217, 224, 225, 234, 291, 292, 293, 295, 298, 299, 301, 303, 304, 307, 308, 309, 313, 314, 317, 329, 336, 451*
Confirmation *25, 66, 77, 92, 104, 164, 176, 181, 190, 191, 198, 202, 203, 207, 209, 228, 230, 235, 249, 256, 261, 352, 353, 358, 364*
Contact *19, 22, 26, 27, 162, 163, 171, 190, 212, 215, 216, 217, 218, 226, 228, 234, 236, 237, 252, 262, 315, 316, 317, 318, 320*
Custom field *31, 55, 317, 328, 329, 336, 447, 453, 459*
Customer *20, 27, 120, 126, 234, 316, 317, 323, 340, 423, 424, 425, 426, 427, 428, 429, 431, 433, 434, 435, 438, 439, 440, 441, 442*

D

Dashboard *29, 32, 34, 40, 49, 52, 85, 86, 87, 97, 98, 99, 104, 111, 112, 113, 114, 115, 116, 119, 120, 136, 137, 138, 163*
Data source *135, 136, 137, 138, 142, 143, 144, 146, 147, 148, 153, 154, 459, 469*
Deal *18, 19, 20, 22, 23, 25, 27, 137, 162, 177, 178, 183, 186, 243, 252, 262, 263, 291, 292, 293, 303, 316, 317, 344, 398*
Diagram *18, 23, 180, 206, 231, 241, 242, 243, 244, 245, 246, 247, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 259, 260, 261*
Diagram type *243, 460*
Dimension *137, 141, 142, 144, 153, 158, 460*
Direct message (DM) *447, 460*
Docked box *58, 68, 80, 81, 395, 396, 397, 398, 402, 403, 404, 406, 412, 414, 415, 416, 417, 419, 447, 455, 460, 465, 467*
Document *18, 19, 22, 23, 27, 101, 130, 138, 145, 146, 155, 157, 162, 180, 200, 206, 209, 210, 231, 242, 260, 265, 370, 371, 372*
Done-gate *32, 49, 447, 460*
Door state (privacy) *397, 454, 460*
Duplicate *34, 35, 36, 39, 46, 52, 53, 64, 65, 96, 139, 155, 158, 169, 170, 171, 177, 193, 203, 204, 208, 222, 229, 233, 425*

E

Edge *20, 94, 96, 103, 241, 242, 243, 247, 249, 251, 253, 254, 255, 256, 258, 259, 260, 261, 263, 298, 451, 460*
Editor *18, 21, 40, 86, 87, 91, 92, 96, 98, 99, 101, 102, 105, 108, 143, 191, 195, 215, 216, 222, 223, 224, 243, 244, 370*
Element *190, 267, 269, 272, 274, 276, 277, 278, 279, 283, 284, 288, 451, 452, 460, 461*
Engagement event *216, 217, 460*
Epic *30, 31, 32, 34, 35, 37, 46, 50, 51, 54, 56, 112, 123, 130, 447, 448, 460, 467, 468*

Event *19, 21, 36, 47, 53, 71, 78, 106, 129, 157, 176, 177, 183, 185, 211, 216, 217, 284, 292, 293, 294, 344, 345, 346, 396*
Event source *293, 452, 461*
Execution *139, 147, 291, 292, 293, 294, 299, 300, 301, 302, 303, 304, 308, 309, 310, 311, 313, 452, 461*
Execution step *294, 461*
Expected progress *112, 113, 117, 128, 448, 457, 461, 462*
Expense *162, 163, 172, 173, 177, 178, 182, 183, 187, 188, 449, 461*
Expiry policy *85, 87, 88, 98, 99, 105, 448, 461*
External connection *345, 453, 461*

F

Field *31, 35, 38, 39, 43, 51, 52, 55, 64, 66, 75, 77, 86, 87, 88, 91, 92, 96, 99, 137, 138, 190, 293, 317, 425*
Field type *126, 190, 195, 328, 336, 450, 461*
Filter *32, 34, 38, 45, 50, 51, 65, 75, 86, 87, 89, 92, 95, 96, 112, 113, 138, 141, 144, 163, 216, 244, 292, 293, 371*
Floor *148, 395, 396, 397, 398, 399, 400, 402, 405, 406, 408, 409, 410, 411, 414, 417, 418, 419, 421, 458, 461, 466*
Folder *370, 371, 372, 374, 382, 383, 384, 387, 388, 393, 460, 461*
Form *19, 22, 30, 31, 86, 87, 118, 121, 129, 132, 168, 169, 171, 173, 175, 177, 178, 187, 189, 190, 191, 192, 193, 194, 242*
Form status *191, 201, 208, 450, 462*
Form type *191, 202, 462*
Frame *150, 267, 272, 278, 282, 288, 452, 460, 462*
Freshness *20, 85, 86, 87, 90, 93, 95, 97, 98, 102, 104, 105, 108, 448, 462*

G

Goal *30, 31, 36, 49, 50, 51, 52, 53, 74, 75, 76, 77, 78, 79, 80, 102, 103, 104, 105, 106, 111, 112, 113, 115, 243*
Goal status *113, 123, 462*

H

Hunt *23, 395, 396, 397, 398, 404, 407, 411, 412, 416, 419, 420, 455, 460, 462*

I

iCal feed *40, 52, 346, 357, 365, 462*
Integrity issue *268, 269, 276, 284, 462*
Invoice *18, 22, 27, 161, 162, 163, 164, 166, 167, 168, 169, 170, 171, 174, 175, 176, 177, 178, 179, 181, 182, 183, 184, 185, 186*
Invoice status *163, 187, 449, 462*

K

Key Result (KR) *113, 462*

Knock *180, 206, 231, 306, 360, 395, 397, 398, 402, 403, 405, 409, 411, 414, 415, 418, 419, 420, 454, 460, 463*
Knowledge Graph *20, 85, 86, 87, 89, 94, 95, 103, 463*

L

Label *31, 38, 46, 91, 95, 103, 123, 129, 140, 142, 144, 147, 148, 191, 195, 198, 199, 207, 210, 242, 243, 244, 245, 248, 425*
Layout algorithm *242, 243, 460, 463*
Lead score *316, 317, 324, 329, 336, 340, 341, 453, 459, 463*
Lead scoring rule *317, 463*
Lifecycle stage *234, 316, 317, 323, 324, 329, 330, 333, 334, 340, 341, 452, 459, 463*
Limits *51, 70, 79, 190, 277, 288, 294, 296, 299, 310, 345, 371, 451, 452, 458, 463, 465*
Line item *163, 168, 169, 177, 181, 183, 184, 187, 463*
Link *19, 21, 22, 27, 30, 39, 40, 46, 52, 54, 58, 61, 62, 69, 74, 85, 86, 163, 190, 191, 242, 268, 345, 371, 425*
Linked entity *243, 248, 252, 262, 451, 463*
Lock *64, 139, 161, 177, 267, 269, 273, 278, 284, 353, 368, 405, 406, 428, 451, 454, 463*
Logic (conditional display) *191, 463*

M

Materialized view *138, 148, 463*
Measure *137, 142, 144, 146, 153, 154, 158, 449, 464*
Mention *57, 59, 60, 61, 70, 71, 74, 78, 79, 80, 377, 464*
Merge fields *217, 222, 464*
Message *22, 52, 57, 58, 59, 60, 61, 62, 63, 64, 65, 67, 68, 69, 70, 71, 72, 74, 75, 76, 77, 78, 79, 190, 425*

N

Node *20, 22, 94, 95, 96, 104, 241, 242, 243, 244, 247, 248, 249, 251, 252, 253, 254, 255, 256, 258, 259, 260, 261, 262, 263*

O

Office *19, 20, 23, 27, 48, 73, 97, 101, 105, 125, 151, 172, 257, 280, 332, 344, 345, 350, 361, 367, 386, 395, 396, 397, 398*
On Error *293, 298, 308, 313, 452, 464*
Organization (org) *31, 464*

P

Payment *161, 162, 163, 169, 170, 171, 174, 177, 178, 181, 182, 185, 186, 187, 449, 459, 462, 464, 466*
Period *20, 87, 98, 99, 105, 109, 111, 112, 113, 114, 115, 116, 118, 119, 120, 122, 123, 126, 128, 129, 130, 131, 132, 136, 148*
Period status *112, 464*
Phase *30, 31, 34, 35, 36, 38, 39, 46, 49, 55, 265, 266, 278, 284, 287, 303, 307, 308, 433, 464, 468*

Pin *57, 59, 63, 71, 76, 82, 241, 248, 254, 260, 263, 401, 417, 447, 451, 464*
Pinned *59, 63, 76, 242, 243, 248, 249, 260, 374, 383, 447, 451, 464*
Pipeline *27, 138, 148, 308, 315, 316, 317, 318, 319, 320, 324, 325, 327, 328, 329, 330, 331, 333, 334, 335, 337, 340, 341, 453, 459*
Portal *20, 27, 423, 424, 425, 426, 427, 429, 432, 433, 434, 435, 438, 441, 442, 444, 445, 455, 457, 458, 459, 464, 465*
Presence *17, 18, 19, 20, 23, 24, 27, 48, 59, 73, 82, 90, 101, 125, 151, 180, 206, 231, 254, 257, 273, 277, 372, 396, 397*
Presence status *397, 420, 454, 465*
Priority *30, 31, 32, 34, 35, 36, 39, 42, 49, 51, 55, 141, 148, 293, 303, 307, 424, 425, 428, 429, 430, 431, 433, 438, 439*
Project *18, 19, 20, 22, 27, 29, 30, 31, 32, 34, 86, 87, 112, 137, 162, 190, 242, 243, 266, 267, 268, 344, 370, 371, 424*
Project scope *87, 268, 269, 283, 294, 369, 371, 382, 387, 388, 454, 465*
Promote to Beacon *371, 379, 381, 382, 384, 390, 454, 465*
Public form URL *191, 213, 450, 465*
Public view token *164, 177, 465*

Q

Quiet hours *59, 71, 78, 83, 447, 465, 466*

R

Rate *39, 64, 82, 161, 162, 163, 164, 168, 169, 171, 173, 174, 175, 177, 178, 181, 184, 186, 187, 202, 203, 219, 221, 226, 292*
Reaction *40, 59, 63, 72, 76, 81, 465*
Real-time collaboration *265, 372, 465*
Recipient / send log *217, 465*
Recurring schedule *163, 175, 184, 187, 344, 465*
Ring (Invite) *397, 403, 415, 466*
Roles *18, 25, 30, 31, 55, 58, 82, 162, 228, 263, 268, 318, 447, 453, 458, 464, 465, 466*
Room *20, 23, 24, 58, 61, 68, 73, 75, 277, 344, 345, 350, 395, 396, 397, 398, 400, 401, 402, 403, 404, 405, 406, 407, 408*
Room access list (ACL) *466*
Room chat *398, 401, 406, 417, 420, 454, 466*
Rotting (stale) deal *317, 453, 466*
RSVP *343, 345, 350, 351, 358, 362, 366, 466*

S

Saved query *87, 94, 103, 138, 144, 146, 466*
Saved view *32, 38, 55, 447, 466*
Schedule trigger *294, 313, 452, 466*
Scheduled message *59, 466*
Scheduled report *138, 145, 158, 449, 466*
Scope *39, 52, 85, 87, 94, 98, 100, 103, 105, 111, 112, 113, 115, 116, 118, 119, 120, 122, 126, 132, 138, 140, 145, 316, 371*
Segment *216, 217, 220, 221, 224, 225, 226, 229, 230, 232, 233, 234, 236, 238, 451, 458, 466*
Sender domain *217, 235, 238, 466*

Settings *42, 43, 45, 59, 60, 66, 67, 69, 77, 79, 87, 98, 105, 138, 146, 147, 159, 162, 163, 164, 174, 190, 191, 370, 424*
Shape *29, 49, 51, 144, 154, 158, 198, 213, 242, 243, 247, 248, 258, 267, 292, 315, 451, 460, 464, 467*
Show as (visibility) *345, 453, 467*
Snapshot (version) *243, 467*
Soft delete *251, 278, 315, 317, 344, 345, 362, 405, 453, 461, 467*
Sprint *20, 25, 29, 30, 31, 32, 34, 35, 36, 37, 39, 40, 46, 49, 50, 51, 52, 54, 55, 71, 79, 112, 123, 130, 132*
Stage *129, 137, 178, 183, 230, 234, 236, 308, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 327, 328, 329, 330, 331, 333*
Star *244, 245, 251, 256, 261, 263, 267, 276, 278, 283, 323, 336, 370, 371, 374, 379, 381, 383, 389, 467*
Start and due dates *29, 30, 31, 32, 38, 51, 467, 468*
Status *30, 31, 36, 38, 40, 46, 52, 59, 70, 79, 86, 89, 112, 113, 163, 191, 217, 292, 294, 344, 345, 370, 396, 397, 424*
Status update *113, 121, 127, 128, 467*
Sticky note *18, 267, 288, 451, 460, 467*
Submission *187, 189, 190, 191, 192, 193, 194, 196, 197, 199, 200, 201, 202, 203, 204, 208, 209, 210, 212, 213, 445, 467*
Summon (Bring everyone here) *397, 403, 454, 467*
Surface huddle *398, 407, 419, 455, 467*
Swimlane *32, 34, 38, 55, 447, 466, 467*
Synced (mirrored) event *346, 454, 467*

T

Tags *38, 85, 86, 87, 90, 92, 95, 96, 99, 100, 102, 108, 226, 448, 463, 467, 468*
Task *18, 19, 22, 23, 27, 29, 30, 31, 32, 34, 35, 36, 37, 38, 39, 40, 41, 138, 243, 292, 293, 317, 344, 371, 424*
Task state *31, 55, 447, 468*
Template *34, 38, 39, 46, 49, 155, 163, 175, 176, 178, 184, 208, 215, 216, 217, 220, 222, 223, 224, 229, 232, 233, 268, 294, 370*
Templating *293, 468*
Thread *18, 22, 27, 57, 59, 62, 64, 68, 71, 72, 74, 76, 78, 79, 82, 105, 254, 256, 261, 283, 398, 401, 417, 447, 468*
Ticket *19, 22, 23, 25, 27, 35, 46, 54, 57, 71, 79, 190, 212, 252, 262, 291, 292, 312, 344, 366, 423, 424, 425, 427, 428*
Timeline *20, 29, 30, 32, 34, 38, 51, 55, 57, 58, 59, 62, 74, 77, 119, 120, 124, 301, 309, 317, 321, 326, 330, 334, 344*
Trigger *35, 78, 149, 176, 186, 218, 291, 292, 293, 294, 295, 296, 297, 298, 299, 301, 302, 303, 304, 307, 308, 309, 310, 312, 313*
Trigger filter *293, 298, 299, 468*

U

Unsubscribe *71, 78, 122, 215, 216, 217, 220, 221, 223, 228, 229, 232, 233, 234, 235, 236, 450, 458, 460, 464, 468*

V

Verification *86, 87, 93, 95, 97, 99, 105, 235, 427, 428, 434, 435, 438, 442, 445, 448, 455, 462, 468*
Version *20, 40, 48, 73, 86, 90, 92, 99, 101, 109, 125, 151, 216, 217, 222, 223, 233, 241, 243, 250, 251, 254, 255, 266, 371*
Visibility *47, 72, 86, 87, 91, 92, 100, 101, 102, 106, 107, 108, 124, 137, 139, 141, 149, 150, 190, 242, 243, 266, 267, 344, 370*

W

Watcher *32, 35, 55, 113, 121, 123, 129, 447, 469*
Widget *20, 23, 135, 136, 137, 139, 140, 141, 142, 143, 149, 152, 153, 154, 156, 158, 401, 449, 469*
Working hours *343, 344, 345, 347, 353, 354, 355, 358, 361, 363, 364, 367, 453, 457, 469*